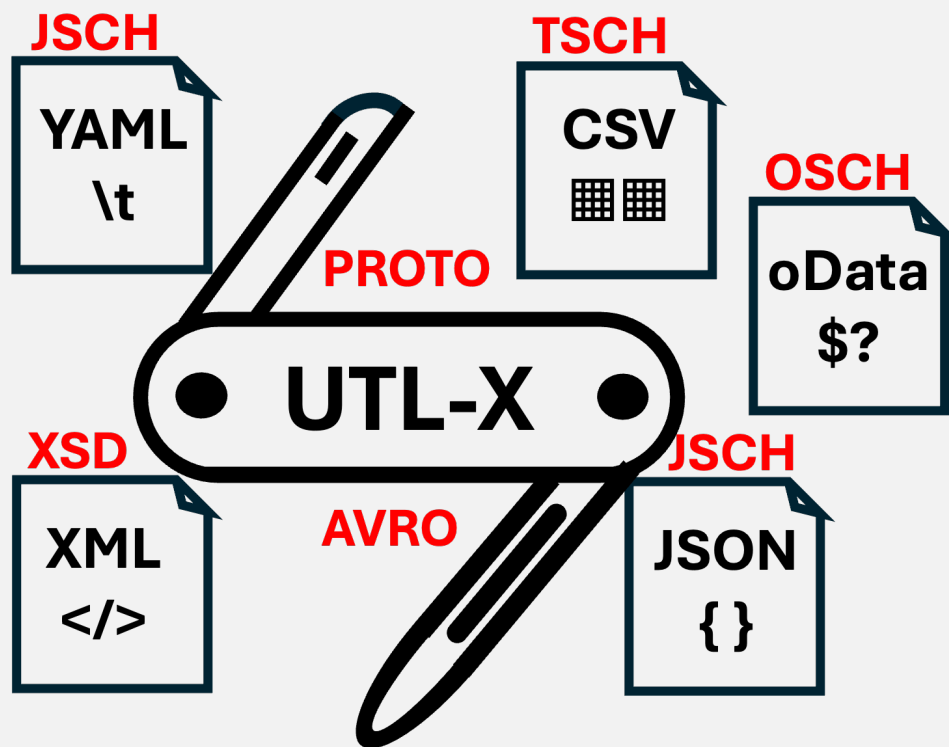


UTLX

Universal Transformation Language eXtended



One Language, All Formats

Data Mapping and Transformation Across XML, JSON, CSV, YAML, OData, and Beyond

Ir. Marcel A. Grauwen

UTL-X: One Language, All Formats — Data Mapping and Transformation Across XML, JSON, CSV, YAML, OData, and Beyond

Copyright © 2026 Ir. Marcel A. Grauven. All rights reserved.

Published by GLOMIDCO B.V., The Netherlands

ISBN 978-90-819728-1-9

First edition, 2026.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews and critical articles.

UTL-X is open source software. The language specification, CLI tool, and standard library are freely available at <https://github.com/grauven/utl-x>.

Typeset with Typst in New Computer Modern.

About the Author

Ir. Marcel A. Grauwen is a Dutch software engineer and architect with over twenty-five years of experience in enterprise integration, data transformation, and middleware platforms.

He has designed and built integration solutions on Tibco BusinessWorks, MuleSoft, SAP CPI, Azure Logic Apps, and IBM Integration Bus — and grew increasingly frustrated with the fundamental limitation they all shared: format-specific transformation logic tied to vendor platforms.

UTL-X was born from that frustration. The idea was simple: one transformation language that works on all data formats, runs anywhere, and belongs to no vendor. Marcel designed the language, built the runtime (CLI, IDE daemon, and production engine), wrote the 652-function standard library, and created the USDL schema classification system.

Before UTL-X, Marcel held senior engineering and architecture roles in financial services, energy and utilities, logistics, and the semiconductor equipment industry — including many years of integration work for ASML, the world's leading lithography machine manufacturer. These are domains where data formats are many, schemas are strict, and getting the mapping wrong has real consequences.

Marcel holds an Ir. degree (Master of Science in Engineering) and is based in the Netherlands.

Table of Contents

Preface	14
Why This Book	14
Who This Book Is For	14
How to Read This Book	15
A Note on Versions	15
1 Introduction	17
1.1 The Data Transformation Problem	17
1.2 What Is UTL-X?	17
1.3 The Swiss Army Knife Analogy	17
1.4 UTL-X vs The Alternatives	18
1.5 Architecture Overview	18
1.6 The UTL-X Ecosystem	18
1.7 A First Example	19
1.8 What's Ahead	20
2 Licensing: Open Source and Commercial	21
2.1 UTL-X Is Open Source (AGPL-3.0)	21
2.2 What You Can Do (Without Paying Anything)	21
2.3 The Key AGPL Condition	21
2.4 Your Transformations Are Yours	21
2.5 Usage Scenarios	22
2.6 The Subprocess Exception	22
2.7 The Commercial License	22
2.8 Dual Licensing: How It Works	23
2.9 Frequently Asked Questions	23
3 Transformation: The Cornerstone of Integration	24
3.1 The Three Pillars of Integration	24
3.2 Why Transformation Is the Hardest Pillar	24
3.3 Transformation in the Enterprise Service Bus	25
3.4 The JSON-to-XML-to-XSLT Anti-Pattern	25
3.5 The UTL-X Approach: No Intermediate Conversion	25
3.6 Why UTLXe Needs Messaging Capabilities	26
3.7 The Canonical Data Model	26
3.8 What This Means for You	26
4 Getting Started	28
4.1 Installation	28
4.2 Your First Transformation	29
4.3 The .utlx File Structure	30
4.4 Trying It Out: Five Quick Examples	32
4.5 Your Own Functions	32
4.6 What's Next	33
5 UTL-X on the Command Line	34
5.1 Three Ways to Use the CLI	34
5.2 Expression Mode: The jq Experience	34
5.3 Identity Mode: The Flip	35
5.4 Shell Scripting Patterns	36
5.5 The UTL-X jq Cheat Sheet	37
5.6 When to Use Which Mode	37
5.7 Performance Notes	37
6 The Three Executables: utlx, utlxd, and utlxe	39

6.1	One Language, Three Runtimes	39
6.2	utlx — The CLI	39
6.3	utlxd — The Daemon	40
6.4	utlxe — The Engine	40
6.5	Comparison at a Glance	42
6.6	The Wrapper Pattern	42
6.7	When to Use Which	42
7	The UTL-X IDE	44
7.1	Installing the VS Code Extension	44
7.2	Two Modes: IDE Runtime and IDE Design Time	44
7.3	The Three-Panel Layout	49
7.4	The IDE vs UTLXe	50
7.5	The UDM Tree Browser	50
7.6	The Function Library	50
7.7	Real-Time Diagnostics	52
7.8	Autocompletion	52
7.9	The Header Editor	52
7.10	Keyboard Shortcuts	53
7.11	Tips for Productive IDE Usage	53
8	Language Fundamentals	54
8.1	Scalar Types	54
8.2	Objects	54
8.3	Arrays	59
8.4	The \$input Variable	59
8.5	Variable Binding (let)	60
8.6	Conditional Expressions	62
8.7	Comments	63
8.8	Type Annotations (Optional)	63
8.9	Putting It Together	63
9	Operators	65
9.1	Arithmetic Operators	65
9.2	Comparison Operators	65
9.3	Logical Operators	66
9.4	Null Handling Operators	66
9.5	Access and Navigation Operators	67
9.6	Data Construction Operators	68
9.7	Flow Operators	68
9.8	String Concatenation	69
9.9	Operator Precedence	69
9.10	Reserved Operator	70
10	The Universal Data Model (UDM)	71
10.1	Why UDM Exists	71
10.2	UDM Types	71
10.3	How Formats Map to UDM	72
10.4	UDM Navigation	73
10.5	UDM and Type Coercion	74
10.6	UDM is Format-Agnostic, Transformations are Format-Agnostic	75
10.7	UDM and Flat/Relational Data	75
10.8	UDM and Schema Formats (Tier 2)	76
10.9	When UDM Matters	78

11	Schema-to-Schema Mapping	79
11.1	Why Schema Mapping Matters	79
11.2	How Traditional Tools Do It	79
11.3	The UTL-X Approach	81
11.4	Tier 1: Instance Data Mapping	82
11.5	Tier 2: Metadata Mapping (Schema-to-Schema)	83
11.6	Nested Schemas: Include and Import	84
11.7	JSON Schema to JSON Schema Mapping	85
11.8	Schema-Driven Validation in UTL-X	85
11.9	The Bridge: USDL	86
12	USDL: Universal Schema Definition Language	87
12.1	What USDL Is	87
12.2	How USDL Works — Automatically	87
12.3	The USDL Directive Classification	88
12.4	Core Directives (Universal)	88
12.5	Shared Directives (Most Formats) (80%+ Format Support)	89
12.6	Format-Native Directives	90
12.7	How Directive Levels Interact During Conversion	94
12.8	Converting Between Formats via USDL	95
12.9	USDL for Validation	95
12.10	USDL as Documentation	95
12.11	USDL vs Standalone Schema Languages	96
13	Format Support Overview	97
13.1	Tier 1: Data Formats	97
13.2	Tier 2: Schema Formats	98
13.3	Format Detection	98
13.4	Format Conversion Matrix	99
13.5	Identity Mode Defaults	99
13.6	Format Options Summary	99
13.7	BOM (Byte Order Mark)	100
13.8	What's Next	100
14	Expressions and Operators	102
14.1	Arithmetic Operators	102
14.2	Comparison Operators	102
14.3	Logical Operators	102
14.4	String Concatenation	103
14.5	Object Construction	103
14.6	Array Construction	104
14.7	The Pipe Operator (<code> ></code>)	104
14.8	Nullish Coalescing (<code>??</code>)	105
14.9	Safe Navigation (<code>?.</code>)	105
14.10	Operator Precedence	105
14.11	Practical Expression Patterns	106
14.12	Complete Operator Reference	107
15	Functions and Lambdas	109
15.1	Lambda Expressions	109
15.2	User-Defined Functions	110
15.3	Higher-Order Functions	111
15.4	Function Composition	117
15.5	Closures	118

15.6	Practical Patterns	119
15.7	The 652 Standard Library Functions	120
16	Standard Library Overview	121
16.1	String Functions (83 functions)	121
16.2	Array Functions (67 functions)	121
16.3	Math Functions (37 functions)	122
16.4	Date/Time Functions (68 functions)	123
16.5	Type Functions (27 functions)	123
16.6	Encoding Functions (30 functions)	123
16.7	Security Functions (16 functions)	124
16.8	XML Functions (60 functions)	124
16.9	Other Categories	125
16.10	Finding the Right Function	126
16.11	A Note on Naming	126
17	Security Library and Message Security	127
17.1	The Security Library (stdlib/crypto)	127
17.2	Transport Security: mTLS	128
17.3	Message-Level Encryption	128
17.4	Digital Signatures and Integrity	129
17.5	Large Files and Sensitive Data	130
17.6	What NOT to Log	130
18	Pattern Matching and Control Flow	131
18.1	Match Expressions	131
18.2	Try/Catch — Error Recovery	132
18.3	Practical Control Flow Patterns	133
18.4	Control Flow Summary	135
19	Schema Validation	136
19.1	Where Validation Lives — and Why	136
19.2	Why Validate?	137
19.3	Supported Schema Formats	137
19.4	The Validation Orchestrator	138
19.5	Validation Policies	139
19.6	JSON Schema Validation	139
19.7	XSD Validation	140
19.8	Syntactical vs Semantic Validation	140
19.9	Validation in Practice	141
19.10	Validation and the Conformance Suite	141
20	Pipeline Chaining and Multi-Hop Transformations	142
20.1	Single-Hop vs Multi-Hop	142
20.2	Why Pipelines?	142
20.3	Multi-Input Transformations (Single Step)	143
20.4	Defining Pipelines	143
20.5	Pipeline Patterns	144
20.6	Multi-Hop with Additional Inputs per Stage	145
20.7	Performance Characteristics	145
21	Data Restructuring: Flat, Hierarchical, and Back	147
21.1	The Six Restructuring Patterns	147
21.2	Pattern 1: Flat → Hierarchical (join)	147
21.3	Pattern 2: Record Enrichment (lookupBy)	148
21.4	Pattern 3: Sequential Grouping (chunkBy)	149

21.5	Pattern 4: Denormalization (unnest)	149
21.6	Pattern 5: Key-Based Grouping (groupBy — exists)	150
21.7	Pattern 6: Array Flattening (flatten — exists)	150
21.8	Combining Patterns	151
21.9	Choosing the Right Pattern	152
21.10	Naming Convention: Why Some Functions End in “By”	152
22	XML Transformations	154
22.1	Reading XML	154
22.2	Writing XML	156
22.3	XML Encoding Detection and Conversion	157
22.4	XML Canonicalization (C14N)	157
22.5	QName Functions (Qualified Names)	158
22.6	XML Attributes in JSON/YAML Output	160
22.7	Common XML Patterns	160
22.8	CDATA Sections	163
22.9	XML Comments and Processing Instructions	165
22.10	XML Documentation (xs:annotation)	166
22.11	XML Round-Trip	166
23	XML Attribute Handling: Design Decisions	167
23.1	How UTL-X Handles Attributes Internally	167
23.2	The <code>_text</code> Convention	167
23.3	The <code>writeAttributes</code> Option	167
23.4	What Gets Kept vs Dropped	168
23.5	Why Drop by Default?	168
23.6	JSON and YAML Consistency	169
23.7	Industry Conventions	169
23.8	FHIR XML: The value Attribute Convention	169
24	JSON Transformations	171
24.1	JSON and UDM: A Natural Fit	171
24.2	Reading JSON	172
24.3	Writing JSON	172
24.4	JSON-Specific Functions	174
24.5	Common JSON Patterns	175
24.6	JSON to Other Formats	178
24.7	JSON Schema	179
24.8	When JSON Isn’t Simple	180
25	CSV Transformations	182
25.1	CSV and UDM	182
25.2	CSV Options	182
25.3	Regional Number Formats	183
25.4	Type Inference	184
25.5	Reading CSV	185
25.6	Writing CSV	186
25.7	Common CSV Patterns	188
25.8	CSV-Specific Functions	189
25.9	Edge Cases	190
26	YAML Transformations	191
26.1	YAML and UDM	191
26.2	Reading YAML	192
26.3	Writing YAML	192

26.4	YAML Output Options	193
26.5	YAML-Specific Functions	194
26.6	Dynamic Keys	194
26.7	Common YAML Patterns	198
26.8	YAML vs JSON: When to Use Which	199
27	OData Transformations	201
27.1	OData v4 JSON and UDM	201
27.2	Reading OData	202
27.3	Writing OData	203
27.4	OData Schema (EDMX/CSDL)	204
27.5	Common OData Patterns	205
27.6	OData vs Standard JSON: When to Use Which	206
28	Schema Formats	208
28.1	The USDL Bridge	208
28.2	XSD (XML Schema Definition)	208
28.3	JSON Schema	209
28.4	Avro Schema	210
28.5	Protobuf (Protocol Buffers)	210
28.6	OData Schema (EDMX/CSDL)	211
28.7	Table Schema (TSCH)	211
28.8	Schema-to-Schema Conversion Examples	212
28.9	Schema Functions Summary	213
29	XSD Design Patterns Deep Dive	214
29.1	Why Patterns Matter	214
29.2	The Five Core Patterns	214
29.3	Advanced Patterns	217
29.4	The Pattern Matrix	218
29.5	Pattern Detection	218
29.6	Pattern Conversion	219
29.7	Patterns in Cross-Format Conversion	220
29.8	Real-World XSD Architectures	220
29.9	XSD Patterns in Integration Tools	220
30	Cross-Format Patterns	222
30.1	The Power of Format Agnosticism	222
30.2	XML to JSON	222
30.3	JSON to XML	223
30.4	CSV to JSON	224
30.5	JSON to CSV	225
30.6	XML to CSV	225
30.7	YAML to JSON and JSON to YAML	226
30.8	XML to YAML and YAML to XML	226
30.9	Multi-Format Integration Flows	226
30.10	The Canonical Data Model Pattern	228
30.11	Format Conversion Quick Reference	228
31	Enterprise Integration	230
31.1	Where UTL-X Fits	230
31.2	Use Case: European E-Invoicing (Peppol/UBL)	230
31.3	Use Case: Healthcare (HL7 FHIR)	232
31.4	Use Case: Financial Services (ISO 20022)	233
31.5	Use Case: Retail and E-Commerce	234

31.6	Workflow and Low-Code Integration	234
31.7	RPA (Robotic Process Automation)	236
31.8	Message Broker Integration	237
31.9	Error Handling in Production	241
32	Engine Lifecycle: Design-Time, Init-Time, Runtime	243
32.1	Phase 1: Design-Time	243
32.2	Phase 2: Init-Time (Load and Compile)	243
32.3	Phase 3: Runtime (Execute)	244
32.4	How Strategies Compare	245
32.5	The Three Phases Visualized	245
32.6	Worker Threading Model	246
33	Cloud Deployment	247
33.1	The Docker Image	247
33.2	Azure Deployment	247
33.3	GCP Deployment	248
33.4	AWS Deployment	249
33.5	Kubernetes (Any Cloud)	249
33.6	Health and Monitoring	251
33.7	Hot Reload	252
34	SDKs and Wrapper Libraries	253
34.1	The Wrapper Pattern	253
34.2	Available SDKs	253
34.3	The Uniform SDK Interface	256
34.4	Alternative: HTTP API (No SDK Needed)	256
34.5	Alternative: Dapr Sidecar (Cloud-Native)	257
34.6	SDK Comparison	257
35	Migration Guides	258
35.1	Migrating from XSLT	258
35.2	Migrating from MuleSoft DataWeave	259
35.3	Migrating from jq	260
35.4	Migrating from TIBCO BusinessWorks	261
35.5	Migrating from Custom Code	261
36	Performance and Optimization	263
36.1	Execution Strategies	263
36.2	Memory Model	263
36.3	GraalVM Native Image	264
36.4	Benchmarking	265
36.5	Production Tuning	265
36.6	Performance Anti-Patterns	266
37	Message Parsing, Memory, and Size Boundaries	268
37.1	How UTL-X Parses Messages	268
37.2	Memory Expansion Factor	268
37.3	Message Size Boundaries	270
37.4	The Poison Message Problem	272
37.5	Swap: Never Use It	273
37.6	Memory Sizing Formula	274
37.7	The Split-Before-Transform Pattern	275
37.8	UTL-X vs Streaming and Big Data Tools	275
38	Logging, Observability, and Compliance	276
38.1	What UTL-X Logs	276

38.2	When NOT to Log: Compliance Requirements	277
38.3	Designing for Compliance	278
38.4	Prometheus Metrics: Safe by Design	279
38.5	The Security Library Connection	279
39	Quality Assurance and Testing	280
39.1	The Five Testing Layers	280
39.2	Layer 1: Unit Tests	280
39.3	Layer 2: Format Tests	280
39.4	Layer 3: Module Tests	280
39.5	Layer 4: Integration Tests	281
39.6	Layer 5: Conformance Suite	281
39.7	Auto-Capture: Recording Tests from Usage	282
39.8	UTLXe Engine Tests	282
39.9	Test-Driven Transformation Development	283
39.10	Continuous Integration	283
40	Semantic Validation: Beyond Syntax	286
40.1	The Gap Between Syntax and Meaning	286
40.2	What Schemas Cannot Express	286
40.3	Semantic Validation Languages in Industry	286
40.4	How Semantic Validation Could Work in UTL-X	287
40.5	Semantic Validation in Practice Today	289
40.6	Common Semantic Validation Rules	289
40.7	The Schematron Bridge	290
40.8	Status	290
41	API Contracts and Data Contracts	291
41.1	Two Dimensions of Contracts	291
41.2	Hierarchical API Contract Standards	292
41.3	Abstract vs Concrete Contracts	292
41.4	Relational Data Contract Standards	293
41.5	How UTL-X Fits in the Contract Landscape	294
41.6	The Contract-Driven Future	295
42	Formats Not Yet Covered	297
42.1	Medium Priority: EDI / EDIFACT	297
42.2	Medium Priority: HL7 v2	297
42.3	Medium Priority: Parquet / Arrow	298
42.4	Low Priority: TOML	298
42.5	Low Priority: JSON-LD	298
42.6	Low Priority: MessagePack / CBOR	298
42.7	Low Priority: GraphQL SDL	299
42.8	Low Priority: ORC (Apache Optimized Row Columnar)	299
42.9	Low Priority: RAML	299
42.10	Potential: JSON Sample Generation (Mock Data)	299
42.11	Declining: Apache Thrift / FlatBuffers	300
42.12	Format Priority Matrix	300
43	Competitive Landscape	301
43.1	Command-Line Transformation Tools	301
43.2	Enterprise Integration Platforms (iPaaS)	302
43.3	Hyperscaler Native Tools	302
43.4	ERP Vendors	303
43.5	Open-Source Alternatives	303

43.6	Positioning Matrix	304
43.7	UTL-X's Unique Position	304
44	Open-M: The Integration Controller	305
44.1	What Is Open-M?	305
44.2	Why Open-M + UTL-X?	305
44.3	MPPM: Message Processing Pipeline Model	305
44.4	Open-M Architecture	307
44.5	What This Means for UTL-X Users	308
45	AI and UTL-X	309
45.1	The Natural Language Round-Trip	309
45.2	Why UTL-X Is Uniquely Suited for AI	309
45.3	What Works Today	310
45.4	The Vision: AI-Native Transformation Development	311
45.5	Capabilities of an AI-Assisted UTL-X Workflow	311
45.6	AI at Design-Time Only	312
45.7	The Virtuous Circle	312
46	Why Kotlin and GraalVM	314
46.1	Why Kotlin	314
46.2	Why Not Other Languages?	315
46.3	Why GraalVM	315
46.4	Why NOT JavaScript/TypeScript	317
46.5	Why NOT WebAssembly (WASM)	317
46.6	One Implementation, Zero Deviation	317
47	Case Studies and Recipes	320
47.1	Case Study 1: European E-Invoicing (Peppol / UBL)	320
47.2	Case Study 2: Healthcare — FHIR Integration	322
47.3	Case Study 3: Retail — Multi-Channel Order Normalization	325
47.4	Recipes: Common Transformation Patterns	326
48	Grammar Reference	331
48.1	File Structure	331
48.2	Header Syntax	331
48.3	Expression Grammar (Precedence)	332
48.4	Literals	333
48.5	Object Construction	333
48.6	Array Construction	333
48.7	Let Bindings	333
48.8	Function Definitions	336
48.9	Lambda Expressions	336
48.10	Conditional Expressions	337
48.11	Match Expression	337
48.12	Try/Catch	337
48.13	Safe Navigation	337
48.14	Property Access	337
48.15	Pipe Operator	337
48.16	Spread Operator	338
48.17	Reserved Words	338
48.18	Operators Reference	338
48.19	Identifier Rules	339
48.20	USDL Grammar (Universal Schema Definition Language)	339
49	Appendices	343

49.1 Appendix A: Format Conversion Matrix	343
49.2 Appendix B: Common Error Messages	343
49.3 Appendix C: UTL-X vs Competitors (Feature Comparison)	344
49.4 Appendix D: Industry Standards Quick Reference	345
49.5 Appendix E: Glossary	346
49.6 Appendix F: Version History	346
49.7 Appendix G: License	347
INDEX	496
LIBRARY REFERENCE	349

Preface

Why This Book

Integration mapping hasn't meaningfully advanced in a decade. This book exists because that needs to change.

For twenty-five years, the industry answer to “how do I transform data between systems?” has been XSLT. It works — for XML. The moment your source or target is JSON, CSV, or YAML, you are on your own. You reach for jq, or a Python script, or a Node.js function, and you start maintaining a second tool for what is conceptually the same job.

Then DataWeave arrived and did something genuinely right: it abstracted away the format. Write your transformation logic once, and it worked on XML, JSON, or CSV without rewriting anything. The catch was that DataWeave was owned by MuleSoft, which is owned by Salesforce. You could not use it outside the Anypoint Platform without paying for it. There was no fork, no escape hatch, no community that could take it somewhere the vendor chose not to go.

So most teams fell back to the status quo: XSLT for XML, jq for JSON, custom scripts for everything else. A growing pile of tools, each doing the same thing differently, each with its own syntax, its own edge cases, its own maintenance burden. A Dutch wholesale distributor receiving purchase orders from twelve European partners — XML over SOAP from one, JSON to a REST API from another, CSV on SFTP from a third, YAML webhooks from a fourth — needs not one transformation layer but four. When SAP shows up with OData, it becomes five.

This is the quiet tax every integration project pays. It compounds. It is paid in onboarding time when a new developer has to learn which tool handles which format. It is paid in bugs that appear at format boundaries. It is paid every time a business requirement changes and the same logic has to be updated in four different scripts instead of one.

UTL-X was built to eliminate that tax. One transformation language, all formats, open source.

This book teaches you UTL-X from the ground up: the language, the runtime, the IDE tooling, the production engine, and the Azure deployment model. It is not a reference manual — the reference documentation is online. It is a book about how to think in UTL-X: how the Universal Data Model works, why format-agnostic transformation changes what is possible, and how to build real integration pipelines that are maintainable, testable, and scale.

By the end, you will understand not just how to write UTL-X transformations, but why the approach works — and why it was worth building in the first place.

Who This Book Is For

This book is written for developers and architects who work with data integration, ETL pipelines, API transformation, or enterprise messaging — and who are tired of solving the same problem in five different languages.

You do not need prior experience with UTL-X. You do need to be comfortable reading code. The examples use XML, JSON, CSV, and YAML as input and output formats; a working knowledge of at least two of these will help you follow along without friction.

This book is for you if:

You are a **backend developer** who has written custom parsing and transformation code more times than you can count, and you sense there should be a better abstraction than a new script for every new format.

You are an **integration architect** who has evaluated MuleSoft, Boomi, or Talend, found the licensing costs prohibitive for what you actually need, and wants a genuinely open alternative that does not lock you into a platform.

You are a **data engineer** building ETL pipelines who currently uses a mix of Python, SQL, and format-specific tools, and wants a single declarative layer that handles the transformation step across all formats.

You are a **DevOps or platform engineer** deploying transformation workloads on Azure or Kubernetes who needs a lightweight, containerised tool with Prometheus metrics, health probes, and Kafka integration out of the box.

You are a **technical lead** evaluating UTL-X for your team and want to understand the full stack — language, runtime, IDE support, and production deployment — before committing.

This book is probably not for you if you are building something that does not involve transforming data between formats. UTL-X is a specialised tool; it does one thing and does it well. If your problem is something else — workflow orchestration, data storage, API gateway routing — other tools will serve you better.

How to Read This Book

This book is organised in eight parts. You do not need to read it linearly — but knowing the structure will help you find what you need.

Part I: Foundation — Start here. Covers the core concepts: the UTL-X header, the Universal Data Model, basic expressions, and your first transformations. Everyone reads this.

Part II: Language Mastery — Read sequentially. Deepens your understanding of the language: functions, operators, control flow, error handling, data restructuring, private functions, and advanced patterns. By the end you will be fluent.

Part III: Formats Deep Dive — Pick what you need. Each chapter covers one format family in depth: XML, JSON, CSV, YAML, OData, schema formats. Read the chapters relevant to your integration landscape.

Part IV: Real-World Applications — When you are ready for production. Enterprise integration patterns, the UTLXe engine lifecycle, cloud deployment on Azure, SDKs, migration guides, performance tuning, logging, and quality assurance.

Part V: Future Outlook — Where UTL-X is heading. Semantic validation, API contracts, formats not yet covered, the competitive landscape, AI integration, and architectural decisions.

Part VI: Case Studies — Real-world walkthroughs. End-to-end examples showing UTL-X solving actual integration problems.

Part VII: Reference — Grammar specification, appendices, and cross-format patterns.

Part VIII: Standard Library Encyclopedia — All 652 functions documented individually with examples. Use as a lookup reference.

A Note on Versions

This book covers UTL-X v1.3.0. The CLI tool (`utlx`) and its 650+ standard library functions are stable. The production engine (`utlxe`) and IDE daemon (`utlxd`) are in active development; chapters covering those components reflect their state at time of writing and may evolve. The language itself — the transformation syntax, the UDM, the type system — is stable and the core focus of this book.

All examples in this book can be run with the free, open source UTL-X binaries available at:

<https://github.com/grauwen/utl-x/releases>

Part I

Foundation

Understanding the language, the data model, and the philosophy

1 Introduction

Every enterprise integration project faces the same fundamental problem: systems don't speak the same language. Your ERP sends XML. Your REST API expects JSON. Your legacy batch process reads CSV. Your DevOps pipeline is configured in YAML. And somewhere, an OData service from Microsoft Dynamics 365 is waiting for its own peculiar JSON dialect.

Traditionally, you'd learn a different tool for each format. XSLT for XML. Liquid templates for JSON. Custom Python scripts for CSV. And for YAML? Good luck — most tools pretend it doesn't exist.

UTL-X changes this. One language. All formats. Write your transformation logic once, and it works whether your input is XML, JSON, CSV, YAML, or OData.

1.1 The Data Transformation Problem

Consider a typical integration scenario. A Dutch wholesale distributor receives purchase orders from 12 European partners. Partner A sends XML via SOAP. Partner B posts JSON to a REST API. Partner C drops CSV files on an SFTP server. Partner D sends YAML webhooks from their Kubernetes-based platform.

Without a format-agnostic tool, this distributor needs:

- An XSLT stylesheet for Partner A's XML
- Custom C# code for Partner B's JSON
- A Python script for Partner C's CSV
- Yet another script for Partner D's YAML

Four languages. Four maintenance burdens. Four sets of bugs. And when a new partner appears with OData from SAP? A fifth.

With UTL-X, all four partners are handled by the same transformation language. The `.utlx` file doesn't care whether the input is XML or CSV — it operates on the *Universal Data Model* (UDM), an internal representation that normalizes all formats into one tree structure.

1.2 What Is UTL-X?

UTL-X — *Universal Transformation Language Extended* — is a format-agnostic, functional transformation language for data integration. It is:

- **Format-agnostic:** one transformation works with XML, JSON, CSV, YAML, and OData
- **Functional:** declarative, composable, testable — no mutable state, no side effects
- **Open source:** licensed under AGPL-3.0, free for any use (see Chapter 2)
- **Production-ready:** 86,000+ messages per second, schema validation, pipeline chaining
- **Batteries included:** 652 built-in functions across 18 categories

At its core, UTL-X answers a simple question: “*Given this input data, produce that output data.*” Everything else — format parsing, type coercion, attribute handling, namespace resolution — is handled automatically.

1.3 The Swiss Army Knife Analogy

UTL-X is often compared to a Swiss Army knife for data. The analogy is apt:

- Each **blade** is a format: XML, JSON, CSV, YAML, OData
- The **handle** is the UDM — the common core that connects all blades
- The **tools** are the 652 standard library functions: string manipulation, date arithmetic, array operations, cryptographic hashing, and more
- You carry **one tool** instead of five — and it handles any situation

1.4 UTL-X vs The Alternatives

How does UTL-X compare to existing transformation tools?

Capability	XSLT	DataWeave	jq	JSONata	UTL-X
JSON native	No	Yes	Yes	Yes	Yes
XML native	Yes	Yes	No	No	Yes
CSV native	No	Yes	No	No	Yes
YAML native	No	Yes	No	No	Yes
OData native	No	No	No	No	Yes
Schema validation	XSD	Limited	No	No	7 formats
Open source	Standard	No	Yes	Yes	Yes (AGPL)
Production engine	Yes	Yes	No	No	Yes
Functions	50	200	50	30	652
Cloud Marketplace	No	MuleSoft	No	No	Azure/GCP/AWS

XSLT is the 25-year veteran — powerful for XML, but XML-only and verbose. DataWeave (MuleSoft) is the closest commercial competitor — format-agnostic and functional, but proprietary and expensive (\$1,250+/month). jq is the beloved command-line JSON processor — fast and elegant, but JSON-only. JSONata is a compact JSON query language — clean syntax, but limited scope.

UTL-X combines the breadth of DataWeave with the openness of jq, adds production deployment capabilities, and extends to formats that no other tool covers natively.

1.5 Architecture Overview

UTL-X’s architecture is built around one central idea: *all data is the same, regardless of format*.

The pipeline works in three stages:

1. **Parse:** The input — whether XML, JSON, CSV, YAML, or OData — is parsed into the Universal Data Model (UDM). The UDM is a tree of typed nodes: scalars (strings, numbers, booleans), objects (with properties and optionally XML attributes), and arrays.
2. **Transform:** The `.utlx` transformation operates on the UDM tree. It can access any property, iterate arrays, apply functions, and construct new structures. The transformation is format-agnostic — it doesn’t know or care whether the data came from XML or JSON.
3. **Serialize:** The resulting UDM tree is serialized to the target format. XML gets elements and attributes. JSON gets objects and arrays. CSV gets rows and columns. Each serializer handles format-specific concerns (XML namespaces, JSON types, CSV delimiters) transparently.

This three-stage architecture is what makes format-agnosticism possible. The transformation logic never touches raw XML tags or JSON braces — it works with the *meaning* of the data, not its *syntax*.

1.6 The UTL-X Ecosystem

UTL-X is not a single tool — it’s an ecosystem of three executables, each designed for a different stage of the development lifecycle:

Executable	Purpose	When to use
<code>utlx</code>	CLI — command-line tool	Development, scripting, CI/CD
<code>utlxd</code>	Daemon — IDE integration	VS Code extension, live preview
<code>utlxe</code>	Engine — production runtime	Cloud deployment, Azure Marketplace

All three share the same parser, interpreter, UDM, and standard library. A transformation written in VS Code (via `utlxd`) runs identically on the command line (via `utlx`) and in production (via `utlxe`). Chapter 5 explains each executable in detail.

1.7 A First Example

Let's see UTL-X in action. Given this XML order:

```
<Order id="ORD-001">
  <Customer>Alice Johnson</Customer>
  <Items>
    <Item sku="WIDGET-01" price="50.00" qty="2"/>
    <Item sku="GADGET-02" price="75.00" qty="1"/>
  </Items>
</Order>
```

We want to produce this JSON:

```
{
  "orderId": "ORD-001",
  "customer": "Alice Johnson",
  "total": 175.00
}
```

Here is the UTL-X transformation:

```
%utlx 1.0
input xml
output json
---
{
  orderId: $input.Order.@id,
  customer: $input.Order.Customer,
  total: sum(map($input.Order.Items.Item,
    (item) -> toNumber(item.@price) * toNumber(item.@qty)
  ))
}
```

Let's break it down:

- `%utlx 1.0` — declares this is a UTL-X version 1.0 transformation
- `input xml` — the input data is XML
- `output json` — the output should be JSON
- `---` — separates the header from the body
- `\$input.Order.\@id` — accesses the `id` attribute on the `Order` element
- `\$input.Order.Customer` — accesses the text content of `Customer`
- `map(...)` — iterates over each `Item`, computing price times quantity
- `sum(...)` — adds up all the line totals

Run it:

```
cat order.xml | utlx transform order-to-json.utlx
```

Output:

```
{
  "orderId": "ORD-001",
  "customer": "Alice Johnson",
  "total": 175.0
}
```

That's UTL-X. One transformation file. Input XML, output JSON. All the format details — XML attributes, element text, JSON types — handled automatically.

Now change output `json` to output `yaml`, and the same transformation produces:

```
orderId: ORD-001
customer: Alice Johnson
total: 175.0
```

Or output `csv`:

```
orderId,customer,total
ORD-001,Alice Johnson,175.0
```

The transformation logic didn't change. Only the output declaration. That is the power of format-agnosticism.

1.8 What's Ahead

This book is organized in eight parts:

- **Part I** (you are here) lays the foundation: installation, the three executables, the UDM, schema mapping, and format support
- **Part II** teaches the language: expressions, functions, pattern matching, validation, and pipelines
- **Part III** dives deep into each format: XML, JSON, CSV, YAML, OData, and schema formats
- **Part IV** covers real-world applications: enterprise integration, cloud deployment, SDKs, performance, and compliance
- **Part V** looks ahead: semantic validation, API contracts, AI-assisted generation, and why Kotlin was the right choice
- **Part VI** presents case studies from e-invoicing, healthcare, financial services, and more
- **Part VII** provides reference material: grammar, appendices, and glossary
- **Part VIII** is the complete standard library encyclopedia: all 652 functions with signatures and examples

Whether you're an integration developer replacing XSLT, a data engineer building multi-format pipelines, or an architect evaluating transformation technologies — this book will take you from first transformation to production deployment.

Let's get started.

2 Licensing: Open Source and Commercial

Before diving into the technical details of UTL-X, it's important to understand what you can and cannot do with it. Licensing matters — especially in enterprise environments where legal and compliance teams review every dependency.

The short version: UTL-X is free and open source for almost every use case. The only exception is embedding it inside a proprietary product you distribute — and even then, a commercial license is available.

2.1 UTL-X Is Open Source (AGPL-3.0)

UTL-X is licensed under the **GNU Affero General Public License v3.0** (AGPL-3.0). This is one of the strongest open-source licenses, designed to keep software free and open — even when deployed as a cloud service.

If you've worked with GPL-licensed software before, AGPL is similar with one important addition: the *network use* clause. Under regular GPL, sharing obligations only trigger when you *distribute* the software. Under AGPL, they also trigger when you make the software available over a *network* — for example, as a SaaS offering.

This matters for UTL-X because transformation engines are typically deployed as services, not distributed as installable software.

2.2 What You Can Do (Without Paying Anything)

Under the AGPL-3.0 license, you are free to:

- **Use UTL-X for any purpose** — personal, educational, or commercial
- **Run UTL-X internally** within your organization, on any number of servers
- **Modify the source code** to fit your needs
- **Deploy UTL-X as a service** within your organization
- **Write any number of .utlx transformations** — they are yours (more on this below)

There is no usage limit, no trial period, no feature restriction. The full UTL-X engine with all 652 standard library functions, all 11 format parsers, and all execution strategies is available under AGPL-3.0.

2.3 The Key AGPL Condition

If you **modify** UTL-X and make your modified version available to users over a network (as a web service, API, or cloud service), you must make your modified source code available to those users under the same AGPL-3.0 license.

This is the *quid pro quo* of AGPL: you benefit from the open-source community's work, and in return, your improvements flow back to the community.

Note the emphasis on *modified*. If you use UTL-X as-is (without changing the engine source code), there is nothing to share — the original source is already public on GitHub.

2.4 Your Transformations Are Yours

This is the most important clarification in this chapter:

The AGPL license covers the UTL-X software — the parser, interpreter, engine, and standard library. It does NOT cover the transformations you write or the data you process.

Your .utlx files are *input* to the program, not part of the program. They are your intellectual property. You can keep them proprietary, sell them, or do anything you want with them.

The analogy is precise: GCC (the GNU C compiler) is licensed under GPL. But the C programs you compile with GCC are *yours* — GCC's license does not “infect” your code. Similarly, UTL-X's AGPL license does not affect your .utlx transformation files.

This means:

- Your transformation logic is private
- Your schemas are private
- Your pipeline configurations are private
- Your test data is private
- Only the UTL-X engine itself is AGPL

2.5 Usage Scenarios

The following table covers every common scenario:

Scenario	License	Cost	Why
Personal use / learning	AGPL (free)	Free	Any use is permitted
Internal company use (unmodified)	AGPL (free)	Free	No distribution, no network clause
Internal company use (modified)	AGPL (free)	Free	Internal use is not distribution
Open-source project using UTL-X	AGPL (free)	Free	Your project must be AGPL-compatible
Consulting (writing .utlx for clients)	AGPL (free)	Free	.utlx files are data, not software
Azure Marketplace deployment	Commercial	\$35–105/mo	Commercial license included
GCP / AWS Marketplace	Commercial	\$44–131/mo	Commercial license included
Embedding in proprietary SaaS	Commercial	Contact	AGPL requires source sharing
OEM / white-label distribution	Commercial	Contact	Distributing modified AGPL code

The vast majority of users fall into the first five rows — all free, no obligations.

2.6 The Subprocess Exception

A technical but important detail: if your application communicates with UTL-X via **HTTP API**, **subprocess** (stdio-proto), or **Dapr sidecar**, there is no *linking* between your code and UTL-X. Your application’s license is not affected by AGPL.

The C# and Go wrappers use subprocess communication (protobuf over stdin/stdout). They spawn UTL-X as a separate process. This is not linking — it’s inter-process communication. Your C#, Go, or Python application remains under whatever license you choose.

This is the same principle that allows proprietary applications to call GPL-licensed command-line tools (like `grep` or `git`) without becoming GPL themselves.

2.7 The Commercial License

2.7.1 Why It Exists

Some organizations have policies that prohibit AGPL dependencies — regardless of how they’re used. Others want to embed UTL-X inside a proprietary product without source-sharing obligations. The commercial license serves these cases.

2.7.2 Cloud Marketplace (Simplest Path)

When you deploy UTL-X from the Azure Marketplace (or GCP/AWS when available), a commercial license is included in the subscription. No AGPL obligations apply. The licensing is handled by Microsoft/Google/Amazon through the Marketplace billing.

- **Azure:** Starter \$35/month, Professional \$105/month
- **GCP:** Starter \$44/month, Professional \$131/month

- **AWS:** Starter \$44/month, Professional \$131/month

2.7.3 Direct Commercial License

For companies embedding UTL-X in proprietary products, OEM distribution, or organizations with AGPL-incompatible compliance policies, a direct commercial license is available. Pricing is per-deployment or per-organization, negotiated based on use case.

Contact: licensing@utlx-lang.org

2.8 Dual Licensing: How It Works

UTL-X uses a *dual licensing* model — the same source code is available under two licenses. You choose which one applies to your use:

License A: AGPL-3.0 (free)	License B: Commercial (paid)
Use, modify, deploy freely	No source-sharing obligation
Share modifications if network service	Embed in proprietary products
Your .utlx files are NOT affected	Azure/GCP/AWS Marketplace includes this
Community-driven improvements	Direct support available

Most users never need the commercial license. The AGPL is free, permissive for internal use, and doesn't touch your transformation files. The commercial license exists as a safety net for the minority of use cases where AGPL is a blocker.

2.9 Frequently Asked Questions

“Can I use UTL-X in my company for free?”

Yes. Internal use is always free under AGPL. You can run UTL-X on your servers, write any transformations, process any data. No payment needed. No source-sharing needed — internal use is not distribution.

“Do I need to open-source my .utlx files?”

No. Your transformation files are data, not part of the UTL-X software. AGPL does not apply to them. They are your intellectual property.

“Can I use UTL-X in my SaaS product?”

If you use UTL-X *unmodified*: yes, under AGPL — but you must make clear to users that UTL-X (AGPL) is used, and provide access to its source code. If you *modify* UTL-X: you must share your modifications under AGPL. If you want no obligations: buy a commercial license.

“What if I deploy from Azure Marketplace?”

Commercial license is included. No AGPL obligations. You pay \$35–105/month through your Azure bill.

“Does AGPL affect my entire application?”

Only if you *link* or *embed* UTL-X directly in your application code (e.g., importing UTL-X as a Java library). If you communicate via HTTP, subprocess, or Dapr — your application is not affected. The Go and C# wrappers use subprocess communication, specifically to avoid this.

With licensing clarified, let's move on to what matters most: using UTL-X to transform data.

3 Transformation: The Cornerstone of Integration

Before we install UTL-X and write our first transformation, it's worth understanding *why* data transformation exists — and why it's the single most important capability in enterprise integration.

If you're already convinced and eager to write code, skip to Chapter 4. But if you're an architect evaluating UTL-X, or a developer wondering why a dedicated transformation language matters when you could just write Python, this chapter is for you.

3.1 The Three Pillars of Integration

Every integration platform — whether it's a \$500K enterprise service bus or a shell script connecting two APIs — rests on three pillars:

1. **Messaging:** getting data from point A to point B. This is the transport layer — message queues (Service Bus, SQS, Kafka), HTTP APIs, file transfers (SFTP), event streams (Event Hub, Pub/Sub). Messaging answers: *“How does data travel?”*
2. **Transformation:** converting data from the format and structure that system A produces into the format and structure that system B expects. Transformation answers: *“How does data change shape?”*
3. **Routing and Orchestration:** deciding *where* to send data and *when*, handling sequences, parallel flows, error recovery, and retry logic. Routing answers: *“Where does data go, and in what order?”*

Traditional Enterprise Service Buses (ESBs) like Tibco BusinessWorks, MuleSoft, and IBM Integration Bus bundle all three pillars into one platform. This is convenient but expensive — you pay for the entire platform even if you only need better transformation.

UTL-X takes a different approach: it focuses exclusively on pillar two — transformation — and does it better than any integrated platform. Messaging is handled by dedicated brokers (Azure Service Bus, Kafka, GCP Pub/Sub). Routing is handled by orchestrators (Logic Apps, Open-M, Step Functions). Each component does one thing well.

3.2 Why Transformation Is the Hardest Pillar

Messaging is largely a solved problem. Whether you choose Kafka, Service Bus, or RabbitMQ, they all reliably move bytes from A to B. The differences are in guarantees (at-least-once vs exactly-once), throughput, and pricing — but the fundamental problem is solved.

Routing is complex but well-understood. BPMN, state machines, and workflow engines have decades of theory and practice behind them.

Transformation, however, is where most integration projects spend 60-80% of their effort. Why?

- **Every system has its own data model.** SAP uses IDoc segments. Dynamics 365 uses OData entities. Your partner's API uses a custom JSON schema. No two systems agree on field names, types, or structure.
- **Formats multiply the problem.** It's not just different fields — it's different *serialization formats*. XML with namespaces and attributes. JSON with nested arrays. CSV with regional number formatting. YAML with anchors and aliases. Each format has its own parsing rules, edge cases, and gotchas.
- **Business logic hides in the mapping.** A transformation isn't just moving field A to field B. It's: “If the customer is in the EU, apply reverse-charge VAT. If the quantity exceeds 100, use the bulk price. If the currency is not EUR, convert using today's exchange rate.” This logic is specific to your business — no generic tool can auto-generate it.
- **Transformations change constantly.** Every API update, every new partner, every regulatory change (e-invoicing deadlines, FHIR mandates, SWIFT MX migration) requires transformation updates. The mapping is the most frequently modified component in any integration.

3.3 Transformation in the Enterprise Service Bus

In a traditional ESB architecture, transformation is embedded in the middleware:

Platform	Transformation tool	Underlying technology
Tibco BusinessWorks	Mapper activity	XSLT (generated)
SAP CPI	Message Mapping	XSLT / Groovy scripts
Azure Logic Apps	Data Mapper	XSLT (JSON→XML internally)
MuleSoft	DataWeave	Format-agnostic (like UTL-X)
IBM Integration Bus	Graphical mapper	ESQL / Java / XSLT
Boomi	Map component	Visual mapping (proprietary)

Notice a pattern? Four out of six platforms use XSLT under the hood — even for JSON data. This forces every message through an XML conversion step, regardless of the original format. MuleSoft’s DataWeave is the exception — and it’s the most direct competitor to UTL-X’s approach.

3.4 The JSON-to-XML-to-XSLT Anti-Pattern

When JSON became the dominant API format in the 2010s, most integration platforms had a problem: their transformation engines only understood XML. Rather than rebuilding their engines, they added a conversion layer:

The message takes four conversion steps:

1. JSON input is converted to an XML representation
2. The XSLT transformation processes the XML
3. The XSLT output is XML
4. The XML is converted back to JSON

This is wrong for several reasons:

Performance: four serialization/deserialization steps instead of one. Each step allocates memory, parses text, and builds a tree. For a 100 KB JSON message, this adds 10-20ms of pure overhead — before any transformation logic runs.

Data loss: JSON and XML are not equivalent. JSON has typed values (numbers, booleans, null). XML has only strings. A JSON number 42 becomes XML text, and when converted back, the type information may be lost — is “42” a number or a string?

Attribute mismatch: JSON has no concept of XML attributes. Conventions like using @id as a property name are workarounds, not standards. Different tools handle it differently. Round-trip fidelity is not guaranteed.

Developer experience: The developer must think in XML even when the data is JSON. They write XSLT templates referencing “Order/Customer” for data that’s actually a JSON object. The mental model and the actual data diverge.

Azure Logic Apps’ Data Mapper does exactly this — internally converting JSON to an XML “infoset” before applying XSLT transformations. SAP CPI’s Message Mapping operates on an XML DOM. Tibco BW uses XML throughout, with JSON adapters at the boundaries.

3.5 The UTL-X Approach: No Intermediate Conversion

UTL-X doesn’t convert between formats before transforming. It parses the input *once* into the Universal Data Model, transforms it, and serializes the output *once* to the target format:

One parse. One transform. One serialize. No intermediate XML. No lost types. No attribute confusion.

And because the UDM is format-agnostic, the *same transformation* works regardless of input format. Change the input declaration from “input json” to “input xml”, and the transformation expression stays the same — `$input.Order.Customer` accesses the customer whether the data came from JSON or XML.

3.6 Why UTLXe Needs Messaging Capabilities

If UTL-X only does transformation (pillar two), why does the production engine (UTLXe) integrate with message brokers?

Because in modern event-driven architectures, transformation doesn’t happen in isolation. Messages arrive from brokers, need to be transformed, and the results need to go back to brokers:

Without messaging integration, UTLXe would be a passive HTTP endpoint — the calling application would need to:

1. Pull the message from the broker
2. HTTP POST it to UTLXe
3. Receive the response
4. Push the result to the output broker

That’s four steps of orchestration just to transform a message. With Dapr integration, UTLXe listens directly on the broker and publishes results back — zero orchestration code needed. The transformation is a *first-class participant* in the event-driven flow, not an afterthought called via HTTP.

This is the same reason MuleSoft’s Anypoint, Tibco’s BWCE, and SAP CPI bundle messaging with transformation — even though they’re conceptually separate pillars.

3.7 The Canonical Data Model

One of the most powerful integration patterns is the *Canonical Data Model* — and UTL-X is ideally suited for it.

The idea: instead of building point-to-point transformations between every pair of systems ($N \times (N-1)$ transformations for N systems), define a single internal format — the “canonical model” — and build only N inbound + N outbound transformations:

Without a canonical model (point-to-point):

- 6 systems = 30 transformations (each pair needs both directions)
- Adding system 7 = 12 new transformations

With a canonical model:

- 6 systems = 12 transformations (6 inbound + 6 outbound)
- Adding system 7 = 2 new transformations

UTL-X makes canonical models practical because:

- The canonical model can be in *any format* — JSON is most common, but XML or YAML work too
- Schema validation ensures all inbound transformations produce valid canonical output
- The same `.utlx` syntax works for every inbound and outbound transformation
- New systems are added by writing one new transformation file, not by modifying existing ones

3.8 What This Means for You

If you’re evaluating UTL-X, here’s the bottom line:

- **If you’re replacing XSLT:** UTL-X handles XML natively, plus JSON, CSV, YAML, and OData — all in one language, with modern functional syntax instead of XML-based XSLT
- **If you’re replacing custom code:** UTL-X is declarative, testable, and hot-reloadable — no redeployment for mapping changes
- **If you’re building a new integration:** UTL-X is the transformation layer — pair it with your choice of message broker and orchestrator

- **If you're an architect:** UTL-X enables canonical data models with format-agnostic transformations and schema-driven development

With this context in mind, let's install UTL-X and start transforming data.

4 Getting Started

This chapter gets UTL-X running on your machine in under five minutes. By the end, you'll have installed the CLI, run your first transformation, and understood the structure of a `.utlx` file.

4.1 Installation

UTL-X is distributed as a native binary — a single file, no JVM required, instant startup. Choose the method that fits your platform.

4.1.1 macOS (Homebrew) — Recommended

```
brew tap grauwen/utlx
brew install utlx
```

Verify:

```
utlx --version
# UTL-X CLI v1.3.0
# Universal Transformation Language Extended
```

Homebrew installs the GraalVM native binary. Startup time is under 10 milliseconds — as fast as `grep` or `jq`.

To update later:

```
brew update && brew upgrade utlx
```

4.1.2 Windows (Chocolatey)

```
choco install utlx
```

Verify:

```
utlx --version
```

If Chocolatey is not available, download the binary directly (see below).

4.1.3 Linux / macOS / Windows — Direct Download

Native binaries are available from GitHub Releases for every platform:

```
# macOS (Apple Silicon)
curl -L https://github.com/grauwen/utl-x/releases/download/v1.3.0/utlx-macos-arm64 -o utlx
chmod +x utlx
sudo mv utlx /usr/local/bin/

# Linux (x64)
curl -L https://github.com/grauwen/utl-x/releases/download/v1.3.0/utlx-linux-x64 -o utlx
chmod +x utlx
sudo mv utlx /usr/local/bin/

# Windows: download utlx-windows-x64.exe from
# https://github.com/grauwen/utl-x/releases/tag/v1.3.0
# and place in a directory on your PATH
```


4.1.4 Building from Source

If you prefer to build UTL-X yourself, or need the JVM version for development:

Prerequisites:

- JDK 17 or later (`java -version` to check)
- Git

Steps:

```
# Clone the repository
git clone https://github.com/grauwen/utl-x.git
cd utl-x

# Build the CLI JAR
./gradlew :modules:cli:jar

# Run using the wrapper script
./utlx --version
```

The wrapper script (`utlx`) automatically finds and runs the compiled JAR. On Windows, use `utlx.bat` (Command Prompt) or `utlx.ps1` (PowerShell).

Building the native binary (requires GraalVM):

```
# Install GraalVM (macOS)
brew install --cask graalvm/tap/graalvm-community-jdk22

# Build native image
./gradlew :modules:cli:nativeCompile

# Binary at: modules/cli/build/native/nativeCompile/utlx
```

The native binary starts in under 10ms and uses 40MB of memory, compared to 250ms startup and 150MB for the JVM JAR. For development, the JAR is fine. For scripting and CI/CD, use the native binary.

4.1.5 Native Binary vs JVM JAR

Aspect	Native binary (GraalVM)	JVM JAR
Startup time	< 10ms	250ms
Memory	40 MB	150 MB
File size	80 MB (single file)	38 MB JAR + JVM
Requires JVM?	No	Yes (JDK 17+)
Install method	brew, choco, download	Build from source
Best for	CLI use, scripting, CI/CD	Development, debugging

When in doubt, use the native binary. It's what `brew install utlx` gives you.

4.2 Your First Transformation

Create a file called `hello.utlx`:

```
%utlx 1.0
input json
```

```
output json
---
{
  greeting: concat("Hello, ", $input.name, "!"),
  reversed: reverse($input.name),
  length: length($input.name)
}
```

Run it:

```
echo '{"name": "World"}' | utlx transform hello.utlx
```

Output:

```
{
  "greeting": "Hello, World!",
  "reversed": "dlroW",
  "length": 5
}
```

Congratulations — you’ve just run your first UTL-X transformation.

4.3 The .utlx File Structure

Every UTL-X transformation file has the same structure:

<pre>%utlx 1.0 input json output xml --- { name: \$input.customer.name, total: sum(\$input.items.price) }</pre>	<pre>← Version declaration ← Input format ← Output format (+ options) ← Separator ← Transformation body</pre>
---	---

The **header** (above ---) declares:

- `%utlx 1.0` — the UTL-X language version (currently always 1.0)
- `input <format>` — what format the input data is in (xml, json, csv, yaml, odata)
- `output <format>` — what format to produce, with optional settings

The **body** (below ---) is the transformation expression. It’s a single expression that produces the output. No `return` keyword — the entire body IS the result.

4.3.1 Input Format Options

<pre>input json input xml input csv input csv {delimiter: ";"} input yaml input odata</pre>	<pre>// standard JSON // XML (auto-detects encoding) // CSV (comma-delimited, with headers) // semicolon-delimited // YAML // OData JSON</pre>
---	--

4.3.2 Output Format Options

```
output json                // pretty-printed JSON
output json {writeAttributes: true} // preserve XML attributes
output xml {encoding: "UTF-8"}    // XML with encoding declaration
output csv {delimiter: ";", headers: true, regionalFormat: "european"}
output yaml                  // YAML block style
```

4.3.3 Multi-Input

When your transformation needs data from multiple sources, use `input:` (with a colon) followed by comma-separated name-format pairs:

```
%utlx 1.0
input: orders xml, customers json
output json
---
{
  orderCount: count($orders.Order),
  customerName: $customers.name
}
```

Each input gets its own variable: the name you declare becomes a `$`-prefixed variable in the transformation body. `orders` becomes `$orders`, `customers` becomes `$customers`.

4.3.4 Multi-Input Syntax Rules

The header declaration syntax follows a consistent pattern:

```
// Single input (no name needed – accessed as $input):
input json

// Single named input (name BEFORE format):
input orders json

// Multiple inputs (colon required, comma-separated):
input: orders xml, customers json

// Multiple inputs with options:
input: orders xml {encoding: "ISO-8859-1"}, customers json

// Multiple inputs on separate lines (comma is the separator, not the line break):
input:
  orders xml,
  customers json,
  products csv {delimiter: ";;"}
```

Note the order: it is always **name then format** — `orders json`, not `json orders`. This reads naturally: “input called orders, in JSON format.” The colon after `input:` signals that named inputs follow. Without the colon, `input json` means a single unnamed input in JSON format.

A few things to watch for:

- **Multiple input lines are NOT supported.** You cannot write `input orders json` on one line and `input customers json` on the next. Use `input:` with commas instead.

- **The name must be a plain identifier.** Names like `input1` are valid, though `input: input1 json` reads a bit awkwardly — consider more descriptive names like `orderData` or `customerList`.
- **Each format can be different.** `input: orders xml`, `prices csv`, `config yaml` mixes three formats in one transformation.

When the transformation runs via `stdin`, the engine expects a JSON envelope where each key matches an input name:

```
{"orders": [{"id": "ORD-001"}], "customers": [{"name": "Alice"}]}
```

The engine splits this envelope by key and routes `orders` to the XML parser and `customers` to the JSON parser. Chapter 20 covers multi-input in the context of pipeline chaining.

4.4 Trying It Out: Five Quick Examples

Here are five transformations you can run right now to see UTL-X in action:

1. **XML to JSON** — the most common integration task:

```
echo '<User><Name>Alice</Name><Age>30</Age></User>' | utlx --from xml --to json
```

2. **JSON to YAML** — configuration format conversion:

```
echo '{"server": "prod-01", "port": 8080, "debug": false}' | utlx --from json --to yaml
```

3. **Extract a field** — like `jq` but for any format:

```
echo '{"users": [{"name": "Alice"}, {"name": "Bob"}]}' | utlx -e 'map($input.users, (u) -> u.name)'
```

4. **CSV to JSON** — data import:

```
echo 'name,age\nAlice,30\nBob,25' | utlx --from csv --to json
```

5. **XML attribute access** — the `@` syntax:

```
echo '<Product id="P-001" price="29.99">Widget</Product>' | utlx --from xml -e '{id: $input.Product.@id, name: $input.Product, price: toNumber($input.Product.@price)}'
```

Each of these runs in under 10 milliseconds on a native binary. That's the UTL-X developer experience: instant feedback, any format.

4.5 Your Own Functions

When a transformation has reusable logic, define your own functions with the `function` keyword:

```
%utlx 1.0
input json
output json
---
function ShippingClass(weight) {
  if (weight > 30) "freight"
  else if (weight > 5) "parcel"
```

```
    else "letter"
  }

  function LineTotal(item) {
    item.qty * item.price
  }

  map($input.items, (item) -> {
    product: item.name,
    total: LineTotal(item),
    shipping: ShippingClass(item.weight)
  })
```

Notice that user-defined functions start with an **uppercase** letter (`ShippingClass`, `LineTotal`) while built-in functions use lowercase (`map`, `filter`, `concat`). This is enforced by the parser — it prevents your functions from accidentally colliding with any of the 652 built-in `stdlib` functions. When you see an uppercase name, you know it's yours. When you see lowercase, you know it's built-in.

Chapter 15 covers functions and lambdas in full detail.

4.6 What's Next

You now have UTL-X installed and running. The next chapters cover:

- **Chapter 5:** UTL-X on the command line — expression mode, identity flip, shell scripting
- **Chapter 6:** The three executables — CLI, daemon, and production engine
- **Chapter 7:** The IDE — VS Code extension, live preview, UDM tree browser

5 UTL-X on the Command Line

UTL-X was designed to feel at home in a terminal. If you’ve used `jq` for JSON processing, UTL-X will feel familiar — but it works with XML, CSV, YAML, and OData too. This chapter covers the three CLI modes, shell scripting patterns, and when to use each.

5.1 Three Ways to Use the CLI

Mode	Syntax	Best for
Transform	<code>utlx transform script.utlx input.xml</code>	Complex transformations with a <code>.utlx</code> file
Expression (-e)	<code>echo data utlx -e '.name'</code>	Quick one-liners, field extraction, piping
Identity (flip)	<code>cat data.xml utlx</code>	Instant format conversion, no script needed

5.2 Expression Mode: The jq Experience

Expression mode (-e) is the most powerful way to use UTL-X interactively. Instead of writing a `.utlx` file, you pass the transformation as a command-line argument:

```
echo '{"name": "Alice", "age": 30}' | utlx -e '.name'
# "Alice"
```

The dot (.) is shorthand for `$input`. So `.name` means `$input.name`, and `.` alone means the entire input — the identity expression. This mirrors `jq`’s syntax deliberately: `utlx -e '.'` is the UTL-X equivalent of `jq '.'`.

5.2.1 Raw Output (-r)

By default, string output includes quotes. Use `-r` (raw) to strip them — essential for piping to other commands:

```
# With quotes (default)
echo '{"name": "Alice"}' | utlx -e '.name'
# "Alice"

# Raw (no quotes) – pipe-friendly
echo '{"name": "Alice"}' | utlx -e '.name' -r
# Alice

# Use in shell variables
NAME=$(echo '{"name": "Alice"}' | utlx -e '.name' -r)
echo "Hello, $NAME"
# Hello, Alice
```

5.2.2 Expressions with Any Format

Unlike `jq`, expression mode works with XML, CSV, and YAML too:

```
# Extract from XML
echo '<User><Name>Alice</Name></User>' | utlx --from xml -e '.User.Name' -r
# Alice

# Extract from XML attribute
echo '<Product id="P-001"/>' | utlx --from xml -e '.Product.@id' -r
# P-001
```

```
# Extract from YAML
echo 'server: prod-01\nport: 8080' | utlx --from yaml -e '.port' -r
# 8080

# Extract from CSV (first row)
echo 'name,age\nAlice,30\nBob,25' | utlx --from csv -e '.[0].name' -r
# Alice
```

5.2.3 Complex Expressions

Expressions can be as simple or complex as needed:

```
# Object construction
echo '{"first": "Alice", "last": "Johnson"}' | utlx -e '{fullName: concat(.first, "
", .last)}'
# {"fullName": "Alice Johnson"}

# Array operations
echo '[3, 1, 4, 1, 5, 9]' | utlx -e 'sort($input)'
# [1, 1, 3, 4, 5, 9]

# Filter and map
echo '{"users": [{"name": "Alice", "active": true}, {"name": "Bob", "active": false}]}' \
| utlx -e 'map(filter(.users, (u) -> u.active), (u) -> u.name)'
# ["Alice"]

# Aggregation
echo ' [{"amount": 10}, {"amount": 20}, {"amount": 30}]' | utlx -e 'sum(map($input, (x) ->
x.amount))'
# 60
```

5.3 Identity Mode: The Flip

The simplest way to use UTL-X — pipe data in, get a different format out. No script, no expression, no arguments:

```
# XML → JSON (automatic)
cat order.xml | utlx
# {"Order": {"Customer": "Alice", "Total": 299.99}}

# JSON → XML (automatic)
echo '{"name": "Alice"}' | utlx
# <?xml version="1.0" encoding="UTF-8"?><name>Alice</name>
```

UTL-X auto-detects the input format and “flips” to the most useful output format:

Input detected as	Output format (default flip)
XML	JSON
JSON	XML
CSV	JSON
YAML	JSON

Override the output format with `--to`:

```
# XML → YAML (instead of default JSON)
cat order.xml | utlx --to yaml

# JSON → CSV
echo '[{"name": "Alice", "age": 30}, {"name": "Bob", "age": 25}]' | utlx --to csv

# CSV → YAML
cat report.csv | utlx --to yaml
```

Override the input format with `--from` (when auto-detection fails):

```
# Force input as CSV (e.g., when piped from another command)
some-command | utlx --from csv --to json
```

5.4 Shell Scripting Patterns

5.4.1 Batch File Conversion

Convert all XML files in a directory to JSON:

```
for f in *.xml; do
    utlx --from xml --to json < "$f" > "${f%.xml}.json"
done
```

5.4.2 Pipeline Integration

UTL-X is pipe-friendly — combine it with standard Unix tools:

```
# Extract customer names from XML orders, sort, deduplicate
cat orders.xml | utlx --from xml -e 'map(.Orders.Order, (o) -> o.Customer)' \
| utlx -e 'sort($input)' \
| utlx -e 'unique($input)'

# Count items in a JSON array
curl -s https://api.example.com/users | utlx -e 'count($input)' -r

# Filter CSV rows and convert to JSON
cat report.csv | utlx --from csv -e 'filter($input, (row) -> toNumber(row.amount) > 100)' \
--to json
```

5.4.3 Using UTL-X in CI/CD

Validate a configuration file in a GitHub Actions workflow:

```
- name: Validate config
  run: |
    cat config.yaml | utlx --from yaml -e 'if (.version != null && .database != null)
    "valid" else "invalid"' -r
```

Transform API responses in deployment scripts:

```
# Extract image tag from Kubernetes deployment
kubectl get deployment myapp -o json | utlx -e '.spec.template.spec.containers[0].image' -r
```


5.4.4 Combining with jq

UTL-X and jq can coexist. Use jq for JSON-only tasks where its syntax is more concise, and UTL-X when you need XML, CSV, or YAML:

```
# jq for simple JSON field extraction (shorter syntax)
echo '{"name": "Alice"}' | jq -r '.name'

# UTL-X when input is XML (jq can't do this)
echo '<User><Name>Alice</Name></User>' | utlx --from xml -e '.User.Name' -r

# Combine: UTL-X converts XML to JSON, jq processes further
cat complex.xml | utlx --from xml --to json | jq '.Orders[] | select(.total > 100)'
```

5.5 The UTL-X jq Cheat Sheet

For developers coming from jq, here's how common patterns translate:

Task	jq	UTL-X
Extract field	jq '.name'	utlx -e '.name'
Raw output	jq -r '.name'	utlx -e '.name' -r
Array length	jq '. length'	utlx -e 'count(.)'
Map array	jq '[][. .name]'	utlx -e 'map(., (x) -> x.name)'
Filter array	jq '[][. select(.active)]'	utlx -e 'filter(., (x) -> x.active)'
Sort	jq 'sort_by(.name)'	utlx -e 'sortBy(., (x) -> x.name)'
Keys	jq 'keys'	utlx -e 'keys(.)'
Sum	jq '[][.price] add'	utlx -e 'sum(map(., (x) -> x.price))'
Compact	jq -c '.'	utlx -e '.' --no-pretty

The key difference: every UTL-X command above also works with `--from xml`, `--from csv`, or `--from yaml`. jq is JSON-only.

5.6 When to Use Which Mode

Situation	Use
Quick field extraction	utlx -e '.field' -r
Format conversion (no logic)	cat data.xml utlx (identity)
Format conversion (explicit target)	cat data.xml utlx --to yaml
Complex transformation with reuse	utlx transform script.utlx input.xml
Shell pipeline	some-cmd utlx -e '...' -r next-cmd
CI/CD validation	utlx -e 'if (...) "ok" else "fail"' -r
Production service	Not CLI — use utlxe (see Chapter 6)

The CLI (`utlx`) is for humans and scripts. The engine (`utlxe`) is for production services. They share the same language — a transformation that works in `-e` mode works identically in `utlxe`.

5.7 Performance Notes

The native binary (GraalVM) is critical for CLI usage:

- **Startup:** < 10ms (native) vs 250ms (JVM JAR)
- **For single invocations:** startup time dominates — native is 25x faster end-to-end

- **For shell loops:** `for f in *.xml; do utlx ...; done` — native processes 100 files/second, JVM 4 files/second (startup penalty per invocation)
- **For pipes:** `cat data | utlx -e '...'` — native feels instant, JVM has noticeable delay

If you're using UTL-X in scripts or CI/CD, always use the native binary. The JVM JAR is for development only (when you need debugging or hot-reload).

6 The Three Executables: *utlx*, *utlxd*, and *utlxe*

UTL-X is one language, but it ships as three separate executables. Each is built for a different stage of the development and deployment lifecycle. Understanding which to use — and when — is essential before diving into the language itself.

6.1 One Language, Three Runtimes

All three executables share the same core:

- The same ANTLR-based parser (identical grammar, identical AST)
- The same interpreter (identical expression evaluation)
- The same Universal Data Model (identical type system)
- The same 650+ standard library functions
- The same 11 format parsers and serializers
- The same 7 schema validators

A transformation that works in one executable works identically in all three. A bug fix in the shared core fixes all three simultaneously. This is by design — the conformance suite (500+ tests) validates behavior across executables.

The difference is in what surrounds the core: how input arrives, how output is delivered, and how the process is managed.

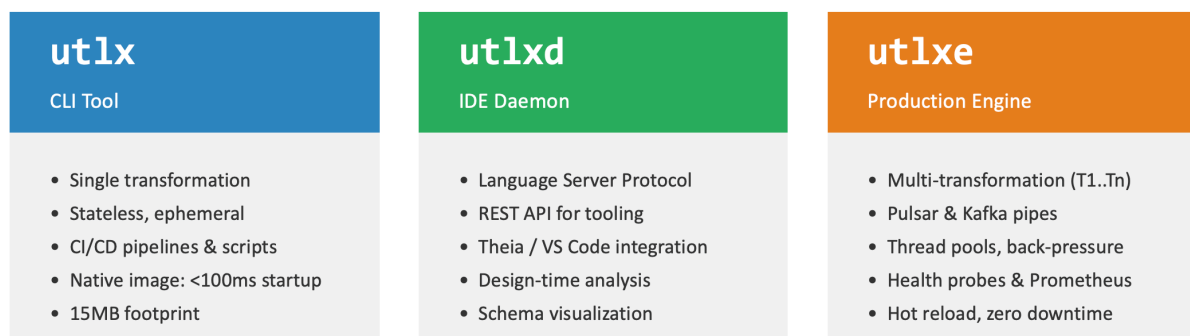


Figure 1: The three UTL-X executables — *utlx* (CLI tool), *utlxd* (IDE daemon), and *utlxe* (production engine). All share the same core; each targets a different use case.

6.2 *utlx* — The CLI

The command-line interface is for developers. It's the tool you install on your laptop, use in shell scripts, and run in CI/CD pipelines.

Lifecycle: single invocation. You run a command, it processes data, it exits. No persistent process, no state between runs.

Distribution: native binary (GraalVM) via Homebrew, Chocolatey, or direct download. Also available as a JVM JAR for development.

Key characteristics:

Startup time	< 10ms (native), 250ms (JVM)
Memory	40 MB (native), 150 MB (JVM)
Workers	1 (single-threaded)
Protocol	stdin / stdout / files
Strategies	TEMPLATE only
Hot reload	No — each run is independent

Metrics	None
Best for	Development, scripting, CI/CD, one-off conversions

Modes:

- **Transform:** `utlx transform script.utlx input.xml` — run a `.utlx` file against input data
- **Expression:** `echo data | utlx -e '.name' -r` — inline one-liner, jq-style
- **Identity (flip):** `cat data.xml | utlx` — auto-detect format, convert to the complement
- **REPL:** `utlx repl` — interactive mode for experimentation
- **Validate:** `utlx validate script.utlx` — check syntax without running
- **Functions:** `utlx functions --search date` — browse the standard library

The CLI is covered in depth in Chapter 5.

6.3 utlxd — The Daemon

The daemon is the bridge between UTL-X and your IDE. It runs as a background process on your development machine, communicating with the VS Code extension via the Language Server Protocol (LSP).

Lifecycle: long-running. Starts when VS Code opens a `.utlx` file, stops when VS Code closes. Keeps the parsed state of your transformations in memory for instant feedback.

Distribution: JVM JAR only. The daemon needs full Kotlin reflection for language features like autocompletion and type inference — GraalVM native image doesn't support this yet.

Key characteristics:

Startup time	1 second
Memory	200 MB
Workers	1 (serves one IDE session)
Protocol	LSP over stdio
Strategies	TEMPLATE only
Hot reload	Yes — re-parses on every keystroke
Metrics	None
Best for	VS Code extension, IDE integration

Features provided to the IDE:

- **Real-time diagnostics:** syntax errors highlighted as you type — no need to save and run
- **Autocompletion:** function names (all 650+), property paths (from parsed input), keywords
- **Hover information:** function signatures, parameter types, return types
- **Live preview:** transformation result updates as you type (with sample data)
- **Go-to-definition:** navigate to function definitions and variable bindings
- **Format document:** auto-indent and style your `.utlx` file

The daemon is not used in production — it's a developer tool. You never deploy `utlxd` to a server. The IDE chapter (Chapter 7) covers it in detail.

6.4 utlxe — The Engine

The engine is for production. It's a long-running transformation service that accepts messages via HTTP, gRPC, or message brokers, transforms them at high throughput, and returns results.

Lifecycle: long-running. Starts with the container, runs indefinitely. Transformations are loaded at startup or hot-reloaded via API.

Distribution: Docker image (ghcr.io/utlx-lang/utlxe). Available on Azure Marketplace as a managed application.

Key characteristics:

Startup time	3 seconds (JVM)
Memory	512 MB – 8 GB (configurable)
Workers	8–128 concurrent threads
Protocol	HTTP REST (8085), gRPC, stdio-proto, stdio-json
Strategies	TEMPLATE, COPY, COMPILED, AUTO
Hot reload	Yes — load/update/unload via HTTP API
Metrics	Prometheus (port 8081)
Best for	Cloud deployment, Azure/GCP/AWS Marketplace

Transport modes:

- `--mode http` — HTTP REST API on port 8085. The primary mode for cloud deployment. Endpoints: `/api/transform`, `/api/load`, `/api/execute/{id}`, `/api/health`
- `--mode grpc` — gRPC service for low-latency, binary protocol communication
- `--mode stdio-proto` — protobuf over stdin/stdout, used by the C# and Go wrappers
- `--mode stdio-json` — line-delimited JSON over stdin/stdout, for backward compatibility

Execution strategies:

The engine offers four strategies that trade initialization time for runtime performance:

Strategy	Init time	Runtime	Best for
TEMPLATE	Instant	Interprets AST	Development, simple transforms
COPY	Fast (build skeleton)	Clone + fill	Schema-driven, predictable structure
COMPILED	Seconds at startup (one-time, before ready)	JVM bytecode	Maximum throughput, complex logic
COPY + COMPILED	Seconds at startup (one-time, before ready)	Clone skeleton + JVM bytecode	Ultimate throughput: pre-built structure + compiled logic
AUTO	Depends	Depends	Production default (schema → COPY, else → TEMPLATE)

COPY + COMPILED is not a separate **strategy**: value — it is what COPY does automatically: when you select COPY (or AUTO chooses it), the engine also compiles the fill logic to bytecode where it can, combining the pre-built skeleton with compiled expressions for the fastest path.

COMPILED strategy compiles UTL-X expressions to JVM bytecode using the ASM library — the same technology that Java itself uses. This achieves throughput of 86,000+ messages per second on a single instance with 8 workers. Chapter 32 covers the engine lifecycle (design-time, init-time, runtime) and Chapter 36 covers performance tuning.

Cloud deployment:

UTLXe is designed for containerized environments:

- Azure Container Apps — available on Azure Marketplace (Starter \$35/month, Professional \$105/month)
- GCP Cloud Run — Terraform module included, \$44/month
- AWS ECS/Fargate — CloudFormation template, \$44/month
- Any Kubernetes cluster — Docker image from ghcr.io

Chapter 33 covers cloud deployment in detail.

6.5 Comparison at a Glance

Aspect	utlx (CLI)	utlxd (Daemon)	utlxe (Engine)
Purpose	Development	IDE integration	Production
Lifecycle	Single run	Long-running	Long-running
Runtime	Native or JVM	JVM only	JVM (Docker)
Startup	< 10ms	1s	3s
Workers	1	1	8–128
Protocol	stdin/stdout	LSP	HTTP / gRPC / proto
Strategies	TEMPLATE	TEMPLATE	All four
Hot reload	No	Yes (LSP)	Yes (API)
Metrics	None	None	Prometheus
Distribution	brew / choco	VS Code ext	Docker / Marketplace
Branch	main	development	development

6.6 The Wrapper Pattern

What if your application is written in C#, Go, or Python — not Kotlin/JVM? You don't need to rewrite UTL-X. Instead, you use a *wrapper library* that spawns UTLXe as a subprocess and communicates via protobuf:

The wrapper is 200–300 lines of thin client code. The engine — 100,000+ lines of transformation logic — exists once. All languages get identical behavior, verified by the same 500+ conformance tests.

Available wrappers:

Language	Status	Use case
C# / .NET	Built	Azure Functions, ASP.NET, console apps
Go	Built	Open-M controller, Go microservices
Python	Planned	Data science, Django/Flask, AWS Lambda
Java / Kotlin	Native (no wrapper)	Direct API — UTLXe IS JVM

Chapter 34 covers SDKs and wrappers in detail.

6.7 When to Use Which

I want to...	Use
Write a transformation	utlxd (VS Code) + utlx (to test)
Convert a file quickly	utlx (identity mode or -e)
Process files in a shell script	utlx (native binary)
Validate syntax in CI/CD	utlx validate

Deploy a transformation API	<code>utlxe -mode http</code>
Connect to Azure Service Bus	<code>utlxe + Dapr sidecar</code>
Embed in a C# application	<code>utlxe</code> via C# wrapper
Embed in a Go service	<code>utlxe</code> via Go wrapper
Run performance benchmarks	<code>utlxe -mode http</code> with load testing

The typical developer workflow: write in VS Code (*utlxd*), test on the command line (*utlx*), deploy to production (*utlxe*). Same *.utlx* file at every stage.

7 The UTL-X IDE

While the CLI is powerful for quick transformations and scripting, professional integration development happens in an IDE. UTL-X provides a VS Code extension backed by the `utlxd` daemon, giving you real-time feedback, autocompletion, and live preview as you write transformations.

This chapter describes the IDE experience and its two modes: IDE Runtime and IDE Design Time. If you're using UTL-X purely from the command line, you can skip ahead to Chapter 8 — but the IDE will make you significantly more productive.

7.1 Installing the VS Code Extension

The UTL-X extension is available from the VS Code marketplace:

1. Open VS Code
2. Go to Extensions (Ctrl+Shift+X / Cmd+Shift+X)
3. Search for “UTL-X”
4. Click Install

The extension automatically starts the `utlxd` daemon when you open a `.utlx` file. No manual configuration needed — the daemon manages its own lifecycle.

7.2 Two Modes: IDE Runtime and IDE Design Time

The IDE operates in two distinct modes, each serving a different workflow:

7.2.1 IDE Runtime

IDE Runtime is the IDE's default mode — a graphical overlay on top of the UTL-X CLI. It provides:

- A three-panel layout: Input, Transformation, Output
- Syntax highlighting, autocompletion, and real-time error diagnostics
- Live preview: as you type, the transformation runs against sample input and shows the result
- The function library browser for all 650+ `stdlib` functions
- The UDM tree browser showing parsed input structure

In IDE Runtime, you *experiment*. You paste sample data, write a `.utlx` transformation, and see the output instantly. There is no schema awareness — you work with the data as-is, guided by the live preview. This is how you prototype transformations, explore unfamiliar data structures, and iterate quickly.

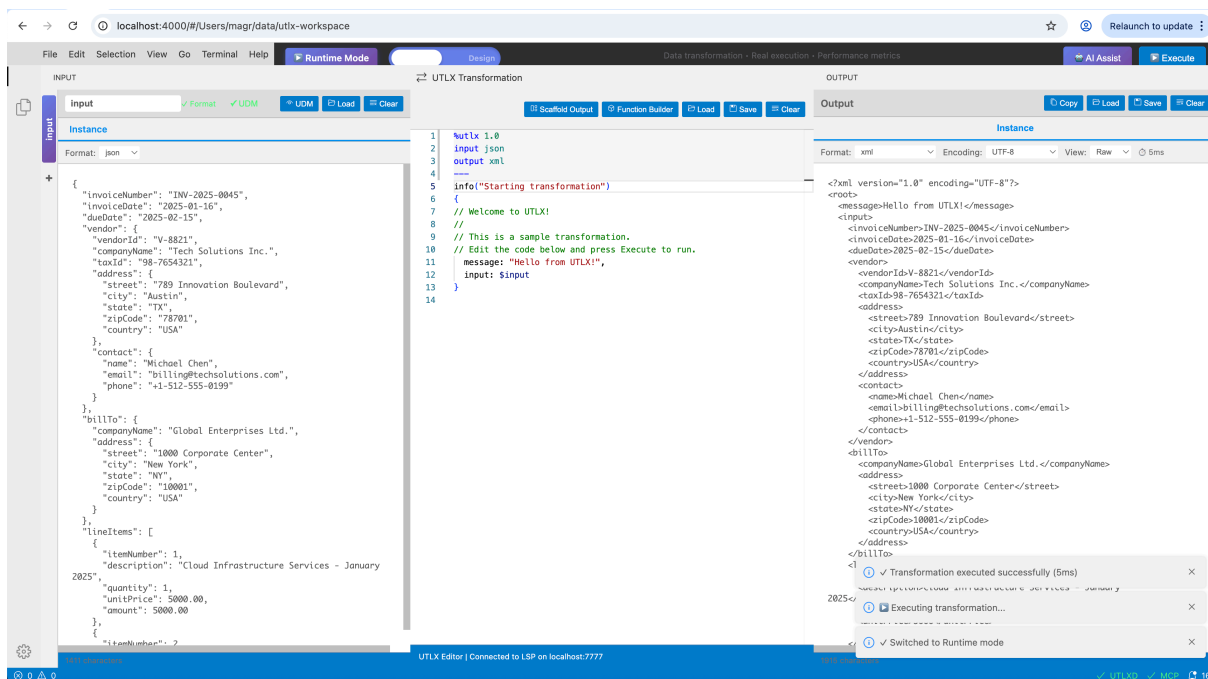


Figure 2: IDE Runtime — JSON input (left), UTL-X transformation (center), XML output (right). The blue “Runtime Mode” badge indicates IDE Runtime is active.

IDE Runtime is essentially the CLI with a GUI on top. The `utlxd` daemon runs the same parser and interpreter as `utlx`, but does so on every keystroke and pipes the result to the output panel.

7.2.2 IDE Design Time

IDE Design Time is for professional schema-driven development. The key difference: *both the input schema and output schema are known in advance*. Your job is to write the `.utlx` that bridges the gap between them.

This distinction is fundamental and worth understanding clearly, because it is easy to confuse what the IDE shows at design-time with what UTL-X processes at runtime.

7.2.2.1 What happens at runtime

At runtime — whether in the CLI, IDE Runtime, or the UTLXe production engine — UTL-X processes **instance documents**. An instance document is a concrete message: a specific JSON object, a specific XML file, a specific CSV with actual rows of data. These are always Tier 1 formats (JSON, XML, CSV, YAML). The `.utlx` transformation receives an instance document as `$input` and produces an instance document as output.

```
%utlx 1.0
input json          // ← receives a JSON instance document
output xml          // ← produces an XML instance document
...
// $input is a concrete JSON object like {"orderId": "ORD-42", "total": 299.99}
```

This is true even when you are transforming data that *conforms* to a schema. The schema is not the input — the instance is.

7.2.2.2 What happens at design-time

At design-time, the IDE adds a layer *above* the instance level. You provide the **schemas** that describe the input and output contracts — an XSD, a JSON Schema, an Avro schema, a Protobuf definition. The

IDE uses these schemas to show you the *structure* of the data: what fields exist, what types they have, which are required, what constraints apply.

This is the schema-to-schema perspective. The IDE displays the input schema structure on the left and the output schema structure on the right. Your job is to write the .utlx transformation that maps from one to the other.

Input Schema	.utlx Transformation	Output Schema
JSON Schema:	%utlx 1.0	XSD:
{	input json	<xs:element
name="Order">		
"orderId": string (required)	output xml	<xs:element
name="id"		
"customer": {	---	type="xs:string"/
>		
"name": string	<Order>	<xs:element
name="customer"		
"email": string	<id>{\$input.orderId}</id>	
type="xs:string"/>		
}	<customer>	<xs:element
name="total"		
"total": number	{\$input.customer.name}	
type="xs:decimal"/>		
}	</customer>	</xs:element>
	<total>{\$input.total}</total>	
	</Order>	

But here is the critical point: **the schema is not the runtime input**. When this transformation runs in production, \$input will not contain a JSON Schema — it will contain a concrete JSON object like {"orderId": "ORD-42", "customer": {"name": "Alice"}, "total": 299.99}. The schemas are a design-time aid that the IDE uses to guide you; they are never passed to the transformation engine.

7.2.2.3 Why this distinction matters

Consider a common enterprise scenario: you need to transform purchase orders from a supplier's JSON API into UBL XML invoices for your ERP system.

Without IDE Design Time (IDE Runtime only):

- You paste a sample JSON purchase order in the input panel
- You write the transformation by trial and error
- You have no way of knowing if your XML output is valid UBL — you are guessing at the target structure
- If the supplier adds a new field next month, you won't know about it until a message fails

With IDE Design Time:

- You load the supplier's JSON Schema (from their API documentation) as the input schema
- You load the UBL XSD as the output schema
- The IDE shows you *every* field on both sides: what's required, what's optional, what types are expected
- As you map fields, the IDE validates that your output conforms to UBL — missing required elements are flagged immediately
- Completeness tracking shows which target fields you haven't mapped yet

The schemas make the transformation *provably correct* before a single production message flows through it.

7.2.2.4 Tier 1 vs Tier 2: which mode applies

IDE Design Time is exclusively for **Tier 1 to Tier 1** mapping — transforming instance documents (JSON, XML, CSV, YAML) while using schemas as structural guides. The schemas help you build the transformation but are not the runtime input or output.

The moment either the input or output is a **Tier 2 format** (a schema itself — XSD, JSON Schema, Avro, Protobuf, OData Schema, Table Schema), only IDE Runtime applies. You are no longer mapping instances guided by schemas — you are processing schemas *as data*. This is a fundamentally different operation.

UTL-X supports all four combinations:

Input	Output	Mode	Example
Tier 1	Tier 1	IDE Design Time or IDE Runtime	JSON purchase order → XML UBL invoice
Tier 1	Tier 2	IDE Runtime only	CSV spreadsheet with field definitions → XSD schema generation
Tier 2	Tier 1	IDE Runtime only	Extract type names and field counts from an XSD → JSON report
Tier 2	Tier 2	IDE Runtime only	Convert Avro schema → Protobuf definition (Chapter 28)

Tier 1 → Tier 1 (IDE Design Time or IDE Runtime): The most common case. A JSON message arrives, you transform it to XML. Both sides are instance documents. In IDE Design Time, schemas guide you. In IDE Runtime, you work directly with sample data. Either mode works.

Tier 1 → Tier 2 (IDE Runtime only): The input is an instance document, but the output is a schema. For example: a CSV spreadsheet describing database columns (field name, type, nullable) is transformed into an XSD or JSON Schema. The CSV is Tier 1 data; the output is a Tier 2 schema. There is no “output schema” for the IDE to validate against — the output *IS* a schema. Only IDE Runtime makes sense.

```
%utlx 1.0
input csv           // Tier 1: spreadsheet with column definitions
output xsd          // Tier 2: generate an XSD from the metadata
---
// $input is CSV rows like: name,type,required
//                          orderId,string,true
//                          total,decimal,true
$input
```

Tier 2 → Tier 1 (IDE Runtime only): The input is a schema, and you extract information from it into a regular data format. For example: read an XSD schema and produce a JSON report listing all type names, field counts, and constraints. The XSD is parsed as data (Tier 2 input); the output is a plain JSON document (Tier 1). No IDE Design Time schema-to-schema perspective applies — the input *IS* a schema.

```
%utlx 1.0
input xsd           // Tier 2: an XSD schema as input
output json         // Tier 1: extract schema metadata as JSON
---
{
  typeCount: count(keys($input["%types"])),
```

```
types: keys($input["%types"])
}
```

Tier 2 → Tier 2 (IDE Runtime only): Schema-to-schema conversion — the input is a schema in one format and the output is a schema in another. See Chapter 28 for full coverage.

```
%utlx 1.0
input xsd                // Tier 2: XSD schema
output jsch              // Tier 2: convert to JSON Schema
---
$input
```

7.2.2.5 Summary: IDE Design Time vs IDE Runtime

Concept	IDE Design Time	IDE Runtime
Input	Schema as structural guide	Instance document or schema as data
Output	Schema as validation target	Instance document or schema as data
Applicable formats	Tier 1 → Tier 1 only	Any combination (T1→T1, T1→T2, T2→T1, T2→T2)
Purpose	Guide the developer, validate correctness	Execute transformation against sample data
\$input	Sample data (for live preview)	Sample data or schema as data
Schemas	Displayed as structural guides	Not used

7.2.2.6 What IDE Design Time provides

- **Schema-to-schema perspective:** the input and output structures are displayed side-by-side, derived from their schemas (XSD, JSON Schema, Avro, Protobuf, Table Schema, OData Schema)
- **Output validation:** the IDE checks your transformation output against the declared output schema in real-time
- **Completeness tracking:** unmapped target fields are flagged — you can see at a glance which parts of the output contract are not yet covered
- **Scaffolding:** the IDE can generate a transformation skeleton with all target fields, placeholder expressions, and TODO comments for fields requiring business logic
- **Type checking:** source-to-target type mismatches are highlighted (e.g., mapping a string where an integer is expected)

This is how Tibco BW, SAP CPI, and MuleSoft approach mapping — define the contracts first, then build the transformation. UTL-X does this for *all* schema formats, not just XML/XSD.

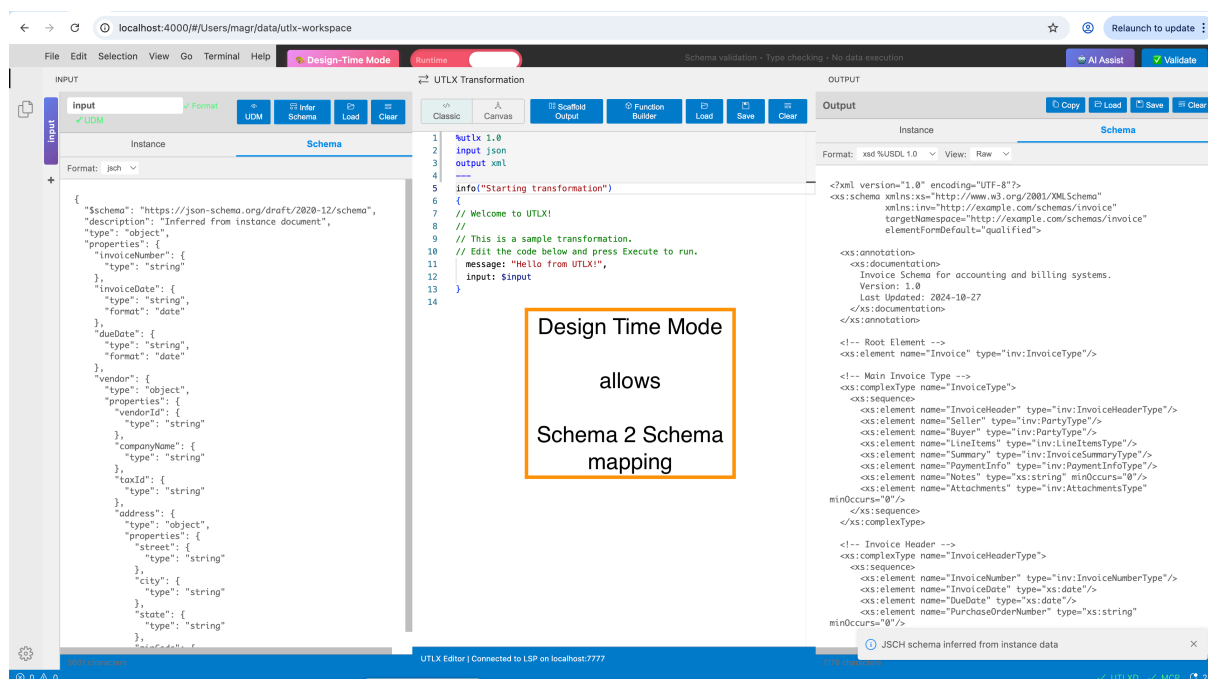


Figure 3: IDE Design Time — JSON Schema on the input side (left), inferred output schema (right). The green “Design Time Mode” badge indicates IDE Design Time is active. The schemas guide the developer; at runtime, `$input` receives instance documents, not schemas.

7.2.3 When to Use Which Mode

Scenario	Mode	Why
Exploring unfamiliar data	IDE Runtime	No schema yet — discover the structure first
Quick format conversion	IDE Runtime	Paste input, write mapping, done
Enterprise integration project	IDE Design Time	Contracts are known — validate against them
Building a mapping between two APIs	IDE Design Time	Both schemas available from OpenAPI/XSD
Debugging a failing transformation	IDE Runtime	Paste the failing message, see what happens
Handoff to production (utlxe)	IDE Design Time	Ensures output matches the production contract

7.3 The Three-Panel Layout

Both modes share the same visual layout:

Left panel — Input: paste or load sample input data (IDE Runtime) or display the input schema structure (IDE Design Time). Select the format (XML, JSON, CSV, YAML, OData).

Center panel — Transformation: the .utlx editor with syntax highlighting, autocompletion, and real-time error diagnostics. This is where you write your transformation logic.

Right panel — Output: the live preview of the transformation result (IDE Runtime) or the expected output schema with validation status (IDE Design Time).

7.4 The IDE vs UTLXe

The IDE and the UTLXe production engine serve different purposes — they are not the same runtime:

Aspect	IDE (utlxd)	Production Engine (utlxe)
Purpose	Develop and test .utlx files	Execute .utlx files at scale
Input	Sample data pasted by developer	Live messages from Kafka, HTTP, file system
Concurrency	Single transformation at a time	8–128 workers processing in parallel
Validation	Optional schema check in IDE Design Time	Enforced schema validation per pipeline config
Output	Displayed in output panel	Routed to target system (API, queue, file)
Metrics	None	Prometheus counters, histograms, health probes
Lifecycle	Starts/stops with VS Code	Long-running daemon or container

The .utlx file is the common artifact. You create it in the IDE, test it with sample data, validate it against schemas — then deploy it *unchanged* to utlxe. No compilation, no packaging, no build step. The source IS the production artifact.

See Chapter 32 for the UTLXe engine lifecycle, Chapter 33 for cloud deployment, and Chapter 19 for the production engine header and configuration.

7.5 The UDM Tree Browser

The UDM tree browser is a panel that shows the parsed Universal Data Model as an interactive tree. Load sample data in the input panel, and the tree browser displays the complete structure:

Each node in the tree shows:

- **Property name:** the field name in the data
- **UDM type:** Scalar, Object, Array, DateTime, etc.
- **Value:** for scalar nodes, the actual value
- **Format-specific information:**
 - XML: element name, attributes (@attr), namespace declarations
 - JSON: type (string, number, boolean, null, object, array)
 - CSV: column index, header name, detected type
 - YAML: anchor/alias references
 - OData: @odata.type, navigation properties

Click a node to insert the accessor path into the transformation editor. For example, clicking the “Name” node under “Customer” inserts `$input.Customer.Name` at the cursor position. This eliminates typos and speeds up field mapping.

7.6 The Function Library

The function library panel lets you browse all 650+ standard library functions without leaving the editor:

- **Browse by category:** 18 categories (String, Array, Math, Date, Type, Encoding, XML, JSON, CSV, YAML, Security, Binary, Financial, Geospatial, Utility, Object, Core, Other)
- **Search:** type a keyword to find functions by name or description
- **Function details:** name, signature (parameter types and return type), description, example
- **Click to insert:** click a function name to insert a call template at the cursor

For example, searching “date” shows: `now()`, `today()`, `parseDate()`, `formatDate()`, `addDays()`, `diffDays()`, and more — with signatures and examples for each.

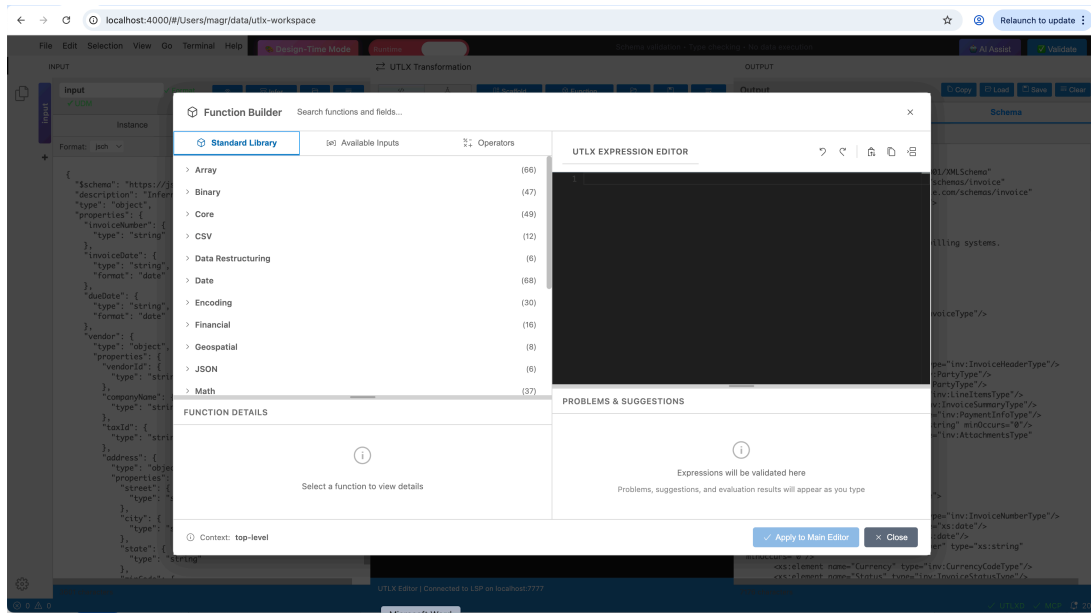


Figure 4: The Function Builder — Standard Library tab showing all 18 function categories with counts. Select a category to browse functions, view signatures, and insert them into the editor.

The Function Builder has three tabs:

- **Standard Library:** browse all 650+ functions by category
- **Available Inputs:** shows the fields available from `$input` with their detected types — useful for exploring unfamiliar data structures without switching to the tree browser
- **Operators:** browse all UTL-X operators (logical, pipe, safe navigation, nullish coalescing, lambda arrow, spread) with descriptions

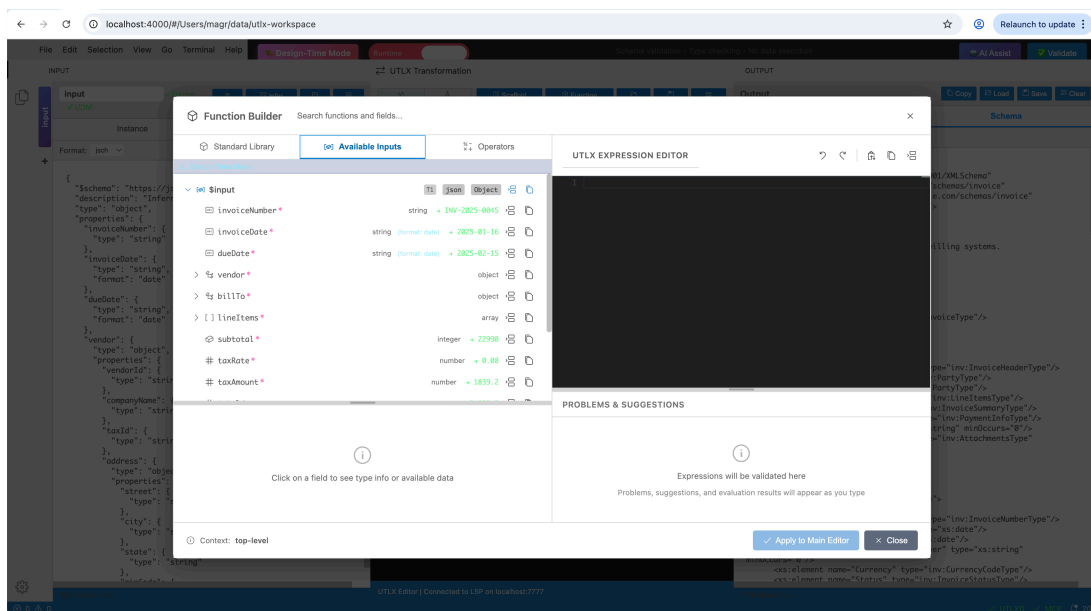


Figure 5: Available Inputs tab — shows input fields with their detected types (array, string, number, integer). Click a field to insert its accessor path.

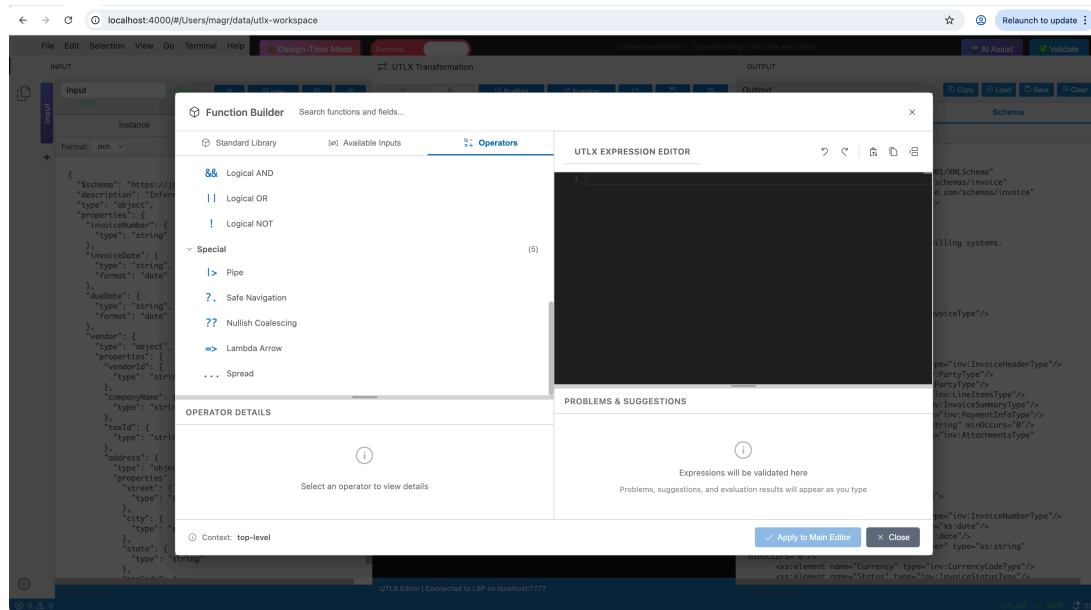


Figure 6: Operators tab — browse logical operators, pipe, safe navigation, nullish coalescing, lambda arrow, and spread with descriptions.

7.7 Real-Time Diagnostics

The IDE shows errors as you type, without saving or running:

- **Syntax errors:** missing brackets, invalid keywords, unterminated strings — highlighted with red underlines and error messages
- **Unknown functions:** calling a function that doesn't exist in the stdlib — immediately flagged
- **Type warnings:** passing a string where a number is expected (when detectable from context)
- **Schema violations:** (IDE Design Time) when output doesn't match the declared output schema

Errors appear in the Problems panel (Ctrl+Shift+M) with line numbers and descriptions. Click an error to jump to the offending line.

7.8 Autocompletion

The IDE provides context-aware autocompletion:

- **After \$input.:** property names from the parsed input data (IDE Runtime) or from the input schema (IDE Design Time)
- **After a function name and parenthesis:** parameter hints showing expected types
- **At the start of an expression:** function names, keywords (if, let, match, try)
- **Inside an object literal:** available property names from the output schema (IDE Design Time)

Autocompletion uses the utlxd daemon's real-time understanding of your transformation — it's not a static word list but a live analysis of the AST and available data.

7.9 The Header Editor

The .utlx header (the part above the --- separator) can be edited visually:

- **Input format:** dropdown to select XML, JSON, CSV, YAML, OData
- **Output format:** dropdown with format-specific options
- **Output options:** checkboxes and fields for options like writeAttributes, encoding, delimiter
- **Multi-input:** add or remove named inputs with their formats
- **Schema references:** attach input and output validation schemas (IDE Design Time)

The header editor generates the header block — you can always switch to text editing if you prefer manual control.

7.10 Keyboard Shortcuts

Shortcut	Action
Ctrl+Space	Trigger autocompletion
F12	Go to definition
Shift+F12	Find all references
Ctrl+Shift+F	Format document
Ctrl+Shift+M	Show Problems panel
Ctrl+K Ctrl+I	Show hover information
Ctrl+Shift+P then “UTL-X”	All UTL-X commands

7.11 Tips for Productive IDE Usage

Start with sample data. Before writing a single line of transformation code, paste representative sample data in the input panel. The UDM tree browser and autocompletion become dramatically more useful when they have real data to analyze.

Use the live preview. Don’t run transformations from the terminal during development. The live preview updates on every keystroke — you see the output change as you type. This is faster than any save-compile-run cycle.

Work schema-first when schemas exist. If you have schemas for both input and output, switch to IDE Design Time and load them. The IDE validates your output in real-time, catching contract violations before you even save the file.

Browse functions, don’t memorize. With 650+ functions, nobody remembers them all. Use the function library search. Type what you want to do — “convert date”, “filter array”, “encrypt” — and the library shows relevant functions.

Trust the red underlines. If the IDE shows an error, fix it before moving on. Real-time diagnostics catch 90% of issues that would otherwise only surface at runtime.

8 Language Fundamentals

This chapter teaches the building blocks of UTL-X: the types you work with, how you access data, how you bind variables, and how you make decisions. Everything that follows — functions, patterns, pipelines — builds on these fundamentals.

If you’ve written JavaScript, Python, or any functional language, most of this will feel familiar. UTL-X deliberately borrows syntax from widely-known languages so you can be productive immediately.

8.1 Scalar Types

Scalars are the atomic values — the leaves of any data tree.

Strings: enclosed in double quotes.

```
"Hello, World"
"Line one\nLine two"
"She said \"hello\""
```

Numbers: integers and decimals. UTL-X uses 64-bit floating point internally (like JavaScript), but preserves integer representation when possible.

```
42
3.14
-17
1.5e10
```

Booleans:

```
true
false
```

Null: represents the absence of a value.

```
null
```

Type coercion happens automatically in some contexts — for example, concatenating a number with a string converts the number to a string. For explicit conversion, use the `stdlib` functions: `toString()`, `toNumber()`, `toBoolean()`.

8.2 Objects

Objects are collections of named properties — the workhorses of data transformation.

8.2.1 Object Literals

Create objects with curly braces and key-value pairs:

```
{
  name: "Alice",
  age: 30,
  active: true
}
```

Keys don’t need quotes unless they contain special characters. Values can be any expression — including other objects, arrays, or function calls.

8.2.2 Property Access

Access properties with dot notation:

```
$input.customer.name
$input.order.items[0].price
```

Or bracket notation for dynamic keys:

```
$input["customer"]["name"]
let field = "name"
$input.customer[field]
```

8.2.3 Nested Access

Dot notation chains naturally for deep structures:

```
$input.Order.Customer.Address.City
```

This works identically whether the data came from XML, JSON, CSV, or YAML — the UDM normalizes all formats into the same property access pattern.

8.2.4 XML Attribute Access

XML attributes use the @ prefix:

```
$input.Order.@id           // the "id" attribute on <Order>
$input.Product.@price      // the "price" attribute on <Product>
```

This is specific to XML input — JSON, CSV, and YAML don't have attributes. See Chapter 22 for XML transformation details and Chapter 23 for the attribute design decisions.

8.2.5 Wildcard Access

The * selector returns *all* children or attributes as an array:

```
$input.IDOC.*              // all child elements of IDOC as an array
$input.Order.*             // all attribute values of <Order> as an array
```

This is name-wildcard navigation — “give me everything at this level.” It does not filter by value; it collects all children (or all attributes) into an array.

```
// XML: <Order id="A" status="open"><Line/><Line/><Line/></Order>

$input.Order.*             // [Line, Line, Line] – all child elements
$input.Order.@*            // ["A", "open"]      – all attribute values
```

Wildcard access is particularly useful for flat data like SAP IDocs, where different segment types are siblings and you need to filter them by name:

```
let headers = filter($input.IDOC.*, (seg) -> seg._name == "E1EDK01")
let lines = filter($input.IDOC.*, (seg) -> seg._name == "E1EDP01")
```

8.2.6 Bracket Notation: Indexing Only

Square brackets are for **index access** — by position (number) or by key (string/variable):

```
$input.items[0]           // first element
$input.items[2]           // third element
$input.config["db-host"]  // key with special characters
let key = "name"
$input.user[key]          // dynamic key access
```

Square brackets are NOT for predicate filtering. This is a deliberate design decision:

```
// DOES NOT WORK – UTL-X has no predicate syntax in brackets:
$input.items[price > 10]

// CORRECT – use filter() with a lambda:
filter($input.items, (item) -> item.price > 10)
```

The full picture of access syntax:

Syntax	What it does	Type
obj.name	Access one property by name	Navigation
obj.*	Access ALL properties as array	Navigation (wildcard)
obj.\@id	Access one attribute by name	Navigation (XML)
obj.\@*	Access ALL attributes as array	Navigation (XML wildcard)
obj.^key	Access metadata by key	Metadata
arr\[0\]	Access element by position	Indexing
arr\[key\]	Access element by dynamic key	Indexing
filter(arr, ...)	Keep elements matching condition	Filtering (function)
arr\[x > 10\]	Not supported — use filter()	N/A

The distinction is clear: `.` navigates by name (with `*` as “all names”), `@` accesses attributes, `^` accesses metadata, `[]` indexes by position or key, and `filter()` selects by value.

8.2.7 Metadata Access (^)

The caret (^) prefix accesses **metadata** — internal information about a UDM node that is not part of the data itself. Metadata is set by format parsers during parsing and is never included in the output.

```
$input.^schemaType        // "xsd-element", "xsd-schema", "jsch-schema", ...
$input.^xsdPattern        // "russian-doll", "venetian-blind", "salami-slice", ...
$input.^xsdVersion        // "1.0" or "1.1"
$input.^xsdGlobalElements // "1" (count as string)
$input.^xsdGlobalTypes    // "5"
$input.^nsContext         // namespace context string
```

Metadata is read-only and transient — it exists during transformation but is not serialized to the output. It’s the parser’s way of communicating structural information to the transformation without polluting the data.

The most practical use case is XSD pattern detection:

```
%utlx 1.0
input xsd
output json
---
{
  pattern: $input.^xsdPattern,          // "venetian-blind"
  version: $input.^xsdVersion,          // "1.0"
  globalElements: toNumber($input.^xsdGlobalElements ?? "0"),
  globalTypes: toNumber($input.^xsdGlobalTypes ?? "0"),
  recommendation: if ($input.^xsdPattern == "russian-doll")
    "Consider converting to Venetian Blind for better reuse"
  else
    "Pattern is suitable for cross-format conversion"
}
```

This transformation reads an XSD and produces a pattern analysis report — using metadata that the XSD parser attached during parsing. See Chapter 29 for the full XSD pattern discussion.

8.2.8 Computed Property Names

When the key is dynamic, use square brackets:

```
let key = "fullName"
{ [key]: concat($input.first, " ", $input.last) }
// produces: {"fullName": "Alice Johnson"}
```

8.2.9 The Spread Operator

The spread operator (...) copies all properties from one object into another. It is one of the most-used features in UTL-X — essential whenever you want to keep most fields and change a few.

8.2.9.1 Basic Usage

```
let base = {name: "Alice", age: 30}
let override = {age: 31, active: true}
{...base, ...override}
// {name: "Alice", age: 31, active: true}
```

Properties from `override` are merged into `base`. When both have the same key (`age`), the **last** one wins — `age: 31` overrides `age: 30`.

8.2.9.2 The “Keep Everything, Change Some” Pattern

This is the dominant use case. You receive an object with 20 fields and need to change 2:

```
// Without spread: list all 20 fields manually
{
  id: $input.id,
  name: $input.name,
  email: $input.email,
  // ... 17 more fields ...
  status: "CONFIRMED" // the one field you actually want to change
}

// With spread: copy everything, override what you need
{
  ...$input,
```

```
    status: "CONFIRMED",
    confirmedAt: now()
  }
```

If the source adds a new field tomorrow, the spread version picks it up automatically. The manual version silently drops it.

8.2.9.3 Order Matters

Properties are applied left to right. Later values override earlier ones:

```
{
  ...$input,           // start with all original fields
  status: "DONE",      // override status
  ...$input.extras     // merge in extras (could override status again!)
}
```

To ensure your explicit fields always win, put them **after** the spread:

```
{...$input, status: "DONE"} // status is always "DONE"
{status: "DONE", ...$input} // status is whatever $input.status was!
```

8.2.9.4 Spread Is Shallow

Spread copies properties one level deep. Nested objects are not cloned — they’re referenced:

```
let order = {id: "A", customer: {name: "Alice", city: "Amsterdam"}}

// This does NOT deep-merge the customer:
{...order, customer: {city: "Rotterdam"}}
// Result: {id: "A", customer: {city: "Rotterdam"}}
// customer.name is GONE — the entire customer object was replaced

// To update a nested field, spread at each level:
{
  ...order,
  customer: {
    ...order.customer,
    city: "Rotterdam"
  }
}
// Result: {id: "A", customer: {name: "Alice", city: "Rotterdam"}}
```

This nested spread pattern is common when transforming YAML configurations — see the Kubernetes manifest example in Chapter 26.

8.2.9.5 Excluding Fields

Spread copies **all** properties — including ones you might not want. There is no built-in “spread except” syntax. To exclude a field, explicitly override it with a new value or restructure:

```
// Problem: order has a "lines" array you don't want in the flat output
let order = {id: "A", customer: "Alice", lines: [{...}, {...}]}

// Spread includes "lines" — unwanted:
```

```
{...order, total: 100}
// {id: "A", customer: "Alice", lines: [...], total: 100}

// Solution: list only the fields you need
{
  id: order.id,
  customer: order.customer,
  total: 100
}
```

This is a conscious trade-off: spread is convenient for “keep most, change some” but not ideal for “keep some, drop most.” Use explicit field listing when you need to exclude.

8.3 Arrays

Arrays are ordered collections — essential for processing lists of items, rows, or repeated elements.

8.3.1 Array Literals

```
[1, 2, 3]
["Alice", "Bob", "Charlie"]
[{name: "Alice"}, {name: "Bob"}]
```

8.3.2 Index Access

Zero-based indexing:

```
$input.items[0]    // first item
$input.items[2]    // third item
```

Negative indexing is not supported — use `last()` instead:

```
last($input.items) // last item
```

8.3.3 Array Operations

Arrays are processed with higher-order functions (Chapter 15 covers these in depth):

```
// Transform each element
map($input.items, (item) -> item.name)

// Keep only matching elements
filter($input.items, (item) -> item.price > 100)

// Accumulate a result
reduce($input.items, 0, (sum, item) -> sum + item.price)

// Sort
sortBy($input.items, (item) -> item.name)
```

8.4 The \$input Variable

Every UTL-X transformation has access to `$input` — the parsed input data as a UDM tree.

```
$input           // the entire input document
$input.Order     // a child property
$input.Order.Items[0] // array element
$input.Order.@id  // XML attribute
```

For multi-input transformations, each input has its own named variable:

```
// Header: input: orders xml, customers json
$orders.Order[0].Total // from the XML input
$customers[0].name     // from the JSON input
```

8.4.1 Safe Navigation

What if a property doesn't exist? Without safe navigation, you get a null reference. With `?.`, you get null gracefully:

```
$input.order?.discount // null if "order" or "discount" is missing
$input.customer?.address?.city // null if any step is missing
```

8.4.2 Recursive Descent

Search for a property name anywhere in the tree with `..`:

```
$input..ProductCode // finds "ProductCode" at any depth
```

This returns an array of all matching values — useful when the exact path varies or the structure is deeply nested.

8.5 Variable Binding (let)

Use `let` to bind a value to a name. Variables are immutable — once bound, they cannot be changed.

```
let name = $input.customer.name
let total = sum(map($input.items, (i) -> i.price * i.qty))
{
  customerName: name,
  orderTotal: total,
  averagePrice: total / count($input.items)
}
```

Variables make transformations readable by naming intermediate results. Without `let`, you'd nest everything:

```
// Without let – hard to read
{
  orderTotal: sum(map($input.items, (i) -> i.price * i.qty)),
  averagePrice: sum(map($input.items, (i) -> i.price * i.qty)) / count($input.items)
}

// With let – clear and no duplication
let total = sum(map($input.items, (i) -> i.price * i.qty))
{
  orderTotal: total,
```



```
    averagePrice: total / count($input.items)
  }
```

Multiple `let` bindings chain naturally — each sees the previous bindings. But the separator between `let` bindings depends on context. This is one of the few places where UTL-X syntax requires attention:

8.5.1 Top-Level: No Separator Needed

At the top level of a `.utlx` body (outside any braces), `let` bindings are separated by newlines:

```
let items = $input.Order.Items.Item
let prices = map(items, (i) -> toNumber(i.@price) * toNumber(i.@qty))
let total = sum(prices)
let tax = total * 0.21
{
  subtotal: total,
  tax: tax,
  grandTotal: total + tax
}
```

No commas, no semicolons — just newlines. The final expression (the object literal) is the return value.

8.5.2 Inside Object Literals: Commas Required

When `let` bindings appear *inside* an object literal (between `{` and `}`), they need commas — just like properties:

```
{
  let subtotal = 100,
  let tax = subtotal * 0.08,
  let total = subtotal + tax,

  subtotal: subtotal,
  tax: tax,
  total: total
}
```

The commas separate `let` bindings from each other and from the property definitions. Think of `let` inside braces as “object members” — they follow the same comma rules as properties.

8.5.3 Inside Lambdas Returning Arrays: Semicolons Required

When `let` bindings are followed by an array (not an object), the parser needs semicolons to distinguish “array return” from “array indexing”:

```
// Inside a map lambda that returns an array:
items |> map(item => {
  let price = toNumber(item.Price);
  let tax = price * 0.08;

  [item.Name, price + tax]
})
```

Without semicolons, the parser would read `let tax = price * 0.08[item.Name, ...]` — interpreting the array as an index operation on the `let` value.

8.5.4 Summary of Let Separators

Context	Separator	Example
Top-level (.utlx body)	Newline	let x = 1 (newline) let y = 2
Inside object { }	Comma	{let x = 1, let y = 2, result: x + y}
Lambda returning array	Semicolon	let x = 1; let y = 2; [x, y]

When in doubt, use commas inside braces and semicolons inside lambdas. At the top level, newlines are always sufficient.

Note: Newlines are sufficient as separators between `let` bindings, between properties, and between match cases. Commas and semicolons are also accepted (backward compatible) but not required when entries are on separate lines. See Chapter 48 (Grammar Reference) for the full separator rules.

8.6 Conditional Expressions

8.6.1 if / else

UTL-X's `if` is an *expression*, not a statement — it always produces a value:

```
if ($input.age >= 18) "adult" else "minor"
```

Use parentheses around the condition. Both branches are required — `if` without `else` is not allowed (every expression must produce a value).

Nest for complex logic:

```
if ($input.score >= 90) "A"
else if ($input.score >= 80) "B"
else if ($input.score >= 70) "C"
else "F"
```

8.6.2 Ternary Operator

For inline conditions, the ternary operator is more concise:

```
$input.active ? "yes" : "no"
```

8.6.3 Nullish Coalescing (??)

Provide a default value when something is null:

```
$input.nickname ?? $input.name ?? "Unknown"
```

This evaluates left to right — the first non-null value wins.

8.6.4 Combining Conditionals

Real-world transformations combine these freely:

```
{
  status: if ($input.paid) "PAID" else "PENDING",
  discount: $input.loyaltyLevel ?? "NONE",
}
```

```

    shipping: $input.express ? "OVERNIGHT" : "STANDARD",
    greeting: concat("Dear ", $input.title ?? "Customer")
  }

```

8.7 Comments

Single-line comments start with `//`:

```

// Calculate the order total including tax
let total = sum(prices) // sum all line items

```

There are no multi-line comments in UTL-X. This is deliberate — it keeps transformations scannable. Every line is either code or a comment, never ambiguous.

8.8 Type Annotations (Optional)

UTL-X supports optional type annotations for documentation:

```

let name: String = $input.customer.name
let total: Number = sum(prices)

```

Type annotations are *not enforced at runtime* in the current version. They serve as documentation — making your intent clear to other developers (and to the IDE, which uses them for better autocompletion).

8.9 Putting It Together

Here's a complete transformation that uses all the fundamentals covered in this chapter:

```

%utlx 1.0
input xml
output json
---
// Extract order data
let order = $input.Order
let items = order.Items.Item
let currency = order.@currency ?? "EUR"

// Calculate financial summary
let lineAmounts = map(items, (item) ->
  toNumber(item.@price) * toNumber(item.@qty)
)
let subtotal = sum(lineAmounts)
let taxRate = if (order.Customer.@country == "NL") 0.21 else 0
let tax = subtotal * taxRate

// Build output
{
  orderId: order.@id,
  customer: order.Customer,
  currency: currency,
  items: map(items, (item) -> {
    sku: item.@sku,
    description: item,
    quantity: toNumber(item.@qty),
    unitPrice: toNumber(item.@price),
    lineTotal: toNumber(item.@price) * toNumber(item.@qty)
  })
}

```

```
    }),  
    subtotal: subtotal,  
    tax: tax,  
    total: subtotal + tax,  
    status: order.@status ?? "NEW"  
  }  
}
```

This transformation demonstrates: property access, attribute access (@), safe navigation (??), let bindings, map with lambdas, conditional logic (if/else), arithmetic, and object construction. All in 25 lines of readable, declarative code.

The next chapter covers UTL-X's operators in depth; Chapter 10 then explores the Universal Data Model — the internal representation that makes all of this work across formats.

9 Operators

Operators are the building blocks of every UTL-X expression. This chapter covers all 32 operators — arithmetic, comparison, logical, null handling, access, construction, and flow — with examples and precedence rules. Chapter 14 (Expressions and Operators) builds on this reference, covering precedence in depth and how operators combine in practical expression patterns.

9.1 Arithmetic Operators

Standard arithmetic on numbers:

```
10 + 3      // 13      addition
10 - 3      // 7       subtraction
10 * 3      // 30      multiplication
10 / 3      // 3.333... division (always floating point)
10 % 3      // 1       modulo (remainder)
2 ** 10     // 1024    exponentiation
-42         // -42     unary minus (negation)
```

Division always produces a floating-point result — `10 / 3` is `3.333...`, not `3`. There is no integer division operator. Use `floor(10 / 3)` if you need integer results.

Division by zero throws a runtime error. Guard if the divisor might be zero:

```
if (count > 0) total / count else 0
```

Operator	Name	Example	Result
+	Addition	<code>10 + 3</code>	13
-	Subtraction	<code>10 - 3</code>	7
*	Multiplication	<code>10 * 3</code>	30
/	Division (always float)	<code>10 / 3</code>	3.333...
%	Modulo (remainder)	<code>10 % 3</code>	1
**	Exponentiation	<code>2 ** 10</code>	1024
- (prefix)	Unary negation	<code>-42</code>	-42

9.2 Comparison Operators

Compare values and produce booleans:

```
10 == 10    // true    equality
10 != 5     // true    inequality
10 > 5      // true    greater than
10 < 5      // false   less than
10 >= 10    // true    greater or equal
10 <= 5     // false   less or equal
```

String comparison is lexicographic (alphabetical) and case-sensitive:

```
"apple" < "banana" // true
"Alice" == "alice" // false (case-sensitive)
"10" < "9"         // true (string comparison: "1" < "9")
```

Null comparisons:

```

null == null    // true
null != "abc"   // true
null == 0       // false (null is not zero)
null == ""      // false (null is not empty string)

```

Operator	Name	Example	Result
==	Equality	10 == 10	true
!=	Inequality	10 != 5	true
<	Less than	3 < 5	true
<=	Less or equal	5 <= 5	true
>	Greater than	10 > 5	true
>=	Greater or equal	10 >= 10	true

9.3 Logical Operators

Combine boolean expressions:

```

true && false    // false    logical AND
true || false    // true     logical OR
!true            // false    logical NOT

```

Short-circuit evaluation: `&&` stops at the first `false`, `||` stops at the first `true`. This is useful for guarding expressions:

```

// Safe: count() only evaluates if items is not null
$input.items != null && count($input.items) > 0

```

9.3.1 Truthy and Falsy Values

In boolean contexts, UTL-X treats these as *false*:

- `false`
- `null`
- `0`
- `""` (empty string)

Everything else is *truthy*, including empty arrays `[]` and empty objects `{}`.

Operator	Name	Example	Result
&&	Logical AND (short-circuit)	true && false	false
	Logical OR (short-circuit)	true false	true
!	Logical NOT	!true	false

9.4 Null Handling Operators

Two operators for dealing with null values — the most common source of runtime errors in data transformation:

9.4.1 Nullish Coalescing (??)

Provide a default value when an expression is null:

```
$input.nickname ?? $input.name ?? "Anonymous"
// First non-null value wins
```

Evaluates left to right. Essential for handling optional fields:

```
{
  currency: $input.currency ?? "EUR",
  country: $input.address?.country ?? "NL",
  discount: $input.promoCode ?? "NONE"
}
```

9.4.2 Safe Navigation (?.)

Access properties on potentially null values without throwing errors:

```
$input.order?.customer?.address?.city
// Returns null if any step is null – no error
```

Without `?.`, a null intermediate value causes a runtime error. With `?.`, null propagates silently.

Combine `?.` with `??` for safe access with defaults:

```
$input.order?.customer?.name ?? "Unknown Customer"
```

Operator	Name	Example	Result
??	Nullish coalescing	<code>null ?? "default"</code>	"default"
?.	Safe navigation	<code>null?.name</code>	null (no error)

9.5 Access and Navigation Operators

These operators navigate the UDM tree — accessing properties, attributes, metadata, and children:

```
$input.name           // dot: property access
$input["content-type"] // bracket: dynamic key or special characters
$input[0]             // bracket: array index
$input.*              // wildcard: all children as array
$input.@id            // at: XML attribute
$input.*@             // at-wildcard: all attributes as array
$input.^schemaType    // caret: metadata
```

Each is a different kind of navigation:

Operator	What it accesses	Example
<code>.name</code>	Named property	<code>\$input.Order.Customer</code>
<code>["key"]</code>	Property by string (dynamic or special chars)	<code>\$input["cbc:ID"]</code>
<code>[0]</code>	Array element by index	<code>\$input.items[0]</code>
<code>.*</code>	All child properties as array	<code>\$input.IDOC.*</code>
<code>.@name</code>	XML attribute by name	<code>\$input.Order.@id</code>
<code>.*@</code>	All XML attributes as array	<code>\$input.Order.*@</code>
<code>.^key</code>	Internal metadata by key	<code>\$input.^xsdPattern</code>

The `.` navigates data (what the user sent). The `@` navigates XML attributes (format-specific metadata on elements). The `^` navigates internal metadata (parser-generated, never in output). See Chapter 8 for detailed examples of each.

9.6 Data Construction Operators

9.6.1 Spread (...)

Copy all properties from one object into another, or concatenate arrays:

```
{...baseObject, status: "CONFIRMED"}    // copy + override
{...obj1, ...obj2}                      // merge (later wins)
[...array1, ...array2]                  // concatenate arrays
```

Spread is shallow — nested objects are referenced, not deep-copied. See Chapter 8 for the full spread operator guide.

9.6.2 Computed Property Names ([expr]:)

Use an expression result as a property name:

```
let field = "fullName"
{[field]: concat($input.first, " ", $input.last)}
// {"fullName": "Alice Johnson"}
```

The brackets evaluate the expression and use the result as the key. See Chapter 26 (dynamic keys) for advanced patterns.

9.7 Flow Operators

9.7.1 Pipe (|>)

Pass the result of the left expression as the first argument to the right function. Reads left-to-right instead of nested inside-out:

```
// Without pipe – read from inside out:
sort(unique(map($input.items, (item) -> item.category)))

// With pipe – read left to right:
$input.items
  |> map((item) -> item.category)
  |> unique()
  |> sort()
```

Both produce the same result. The pipe version reads like a recipe: “take items, map to categories, remove duplicates, sort.”

9.7.2 Lambda Arrows (-> and =>)

Define anonymous functions (lambdas). Both arrows are equivalent:

```
// Thin arrow (standard):
(x) -> x * 2
(x, y) -> x + y

// Fat arrow (JavaScript-style):
```



```
(x) => x * 2
x => x * 2           // single param: parentheses optional with =>
```

The `->` is the convention used throughout this book. The `=>` is accepted for developers coming from JavaScript/TypeScript. The `=>` also appears in match expression arms:

```
match ($input.status) {
  "ACTIVE" => "green",
  "PENDING" => "yellow",
  _ => "gray"
}
```

See Chapter 15 for the full lambda and function guide.

9.7.3 Binding (=)

Used in `let` bindings to assign a value to a name:

```
let total = $input.quantity * $input.price
let taxRate = 0.21
let tax = total * taxRate
```

The `=` is NOT assignment (there are no mutable variables in UTL-X). It is a one-time binding — the name refers to the value for the rest of its scope. See Chapter 8 for `let` binding scoping rules.

9.8 String Concatenation

UTL-X does NOT use `+` for string concatenation — use the `concat()` function:

```
concat("Hello", " ", "World")    // "Hello World"
```

This is deliberate. `+` is arithmetic only. Mixing `+` for both numbers and strings leads to ambiguity ("`3`" + `4` — is it "`34`" or `7`?). UTL-X avoids this by keeping arithmetic and string operations separate.

For joining arrays of strings: `join(array, separator)`.

9.9 Operator Precedence

From highest to lowest precedence. Higher precedence binds tighter:

Precedence	Operators	Associativity
1 (highest)	Postfix: <code>.</code> <code>?</code> <code>[]</code> <code>()</code> <code>@</code> <code>^</code>	Left
2	Unary: <code>!</code> <code>-</code>	Right
3	Exponentiation: <code>**</code>	Right
4	Multiplicative: <code>*</code> <code>/</code> <code>%</code>	Left
5	Additive: <code>+</code> <code>-</code>	Left
6	Comparison: <code><</code> <code>></code> <code><=</code> <code>>=</code>	Left
7	Equality: <code>==</code> <code>!=</code>	Left
8	Logical AND: <code>&&</code>	Left
9	Logical OR: <code> </code>	Left
10	Nullish coalescing: <code>??</code>	Left

11	Ternary: ? :	Right
12	Pipe: >	Left
13 (lowest)	Conditional: if/else	Right

When in doubt, use parentheses. They make intent explicit and prevent precedence surprises:

```
// Ambiguous: does && bind tighter than ||?  
a || b && c      // means: a || (b && c) – && is higher  
  
// Clear: parentheses make intent obvious  
(a || b) && c    // different meaning, explicit
```

9.10 Reserved Operator

Operator	Name	Status
..	Recursive descent	Token defined, parser not implemented. Reserved for future use.

10 The Universal Data Model (UDM)

The Universal Data Model is the single most important concept in UTL-X. It's the abstraction that makes format-agnosticism possible — the internal representation that lets your transformation work identically whether the input is XML, JSON, CSV, YAML, or OData.

This chapter explains what UDM is, how each format maps to it, and why certain design decisions were made. If you want to understand *why* UTL-X works the way it does — not just *how* — this is the chapter.

10.1 Why UDM Exists

Consider the problem: XML has elements, attributes, namespaces, and mixed content. JSON has objects, arrays, strings, numbers, booleans, and null. CSV has rows and columns. YAML has all of JSON's types plus anchors, aliases, and tags.

Without an intermediate representation, a transformation engine would need separate logic for every format combination:

- XML → JSON: handle attributes, namespaces, text nodes
- JSON → XML: handle types (number → string), arrays (→ repeated elements)
- CSV → JSON: handle headers, type detection
- XML → CSV: handle hierarchy flattening
- ... and so on for every pair

With N formats, that's $N \times (N-1)$ conversion paths — each with its own edge cases and bugs.

UDM eliminates this by providing one common tree structure:

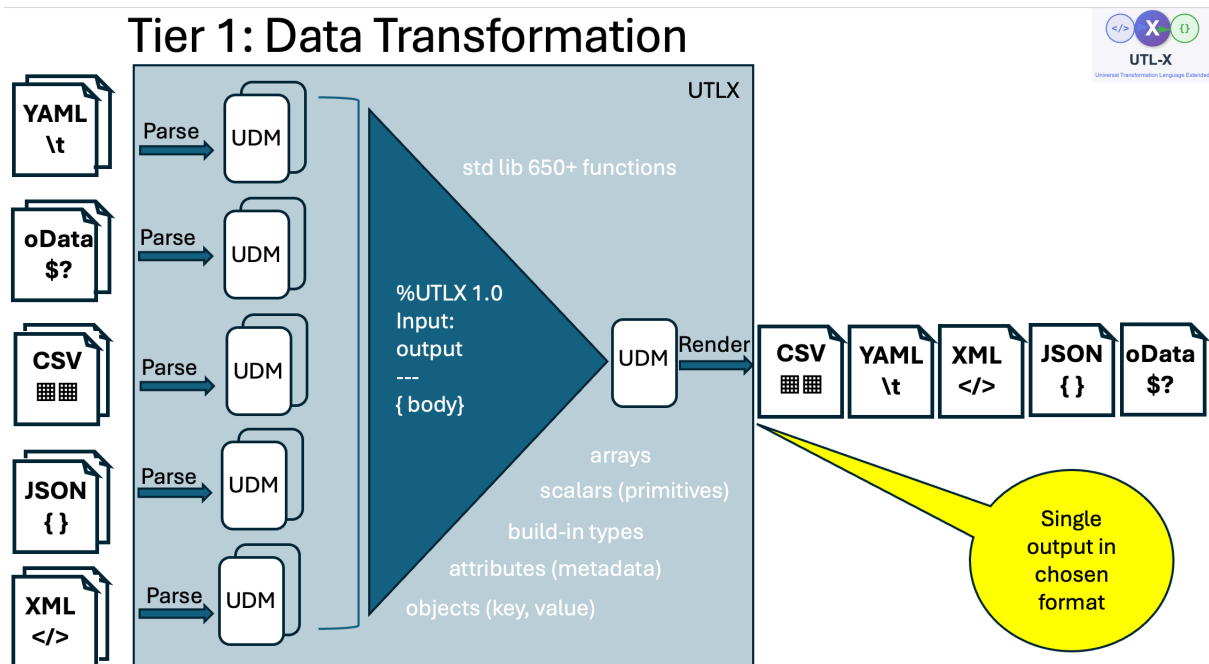


Figure 7: Tier 1 data transformation — all input formats (XML, JSON, CSV, YAML, OData) are parsed into UDM. The transformation operates on UDM. The result is rendered to the chosen output format.

Every input format is parsed into UDM. Every transformation operates on UDM. Every output format is serialized from UDM. The transformation never touches raw XML tags or JSON braces — it works with the *meaning* of the data.

10.2 UDM Types

UDM has eight types. Every piece of data from any format maps to one of these:

UDM Type	What it holds	Examples
Scalar	A single value: string, number, boolean, or null	“Alice”, 42, 3.14, true, null
Object	Named properties + optional XML attributes + meta-data	{name: “Alice”, age: 30}
Array	Ordered list of UDM values	[1, 2, 3], [{...}, {...}]
DateTime	Timestamp with timezone (ISO 8601)	2026-04-28T14:30:00Z
Date	Date only, no time	2026-04-28
LocalDateTime	Date and time without timezone	2026-04-28T14:30:00
Time	Time only, no date	14:30:00
Binary	Raw byte array	Binary file content

There’s also a Lambda type for function values, but you rarely encounter it directly — it’s used internally for `map`, `filter`, and other higher-order functions.

The first three types — Scalar, Object, and Array — account for 99% of real-world data. The temporal types (DateTime, Date, LocalDateTime, Time) provide first-class date handling without string parsing at every access.

10.3 How Formats Map to UDM

10.3.1 JSON → UDM

JSON maps to UDM almost directly — JSON was the inspiration for UDM’s structure:

JSON	UDM
String: “Alice”	Scalar(string: “Alice”)
Number: 42	Scalar(number: 42)
Boolean: true	Scalar(boolean: true)
Null: null	Scalar(null)
Object: {“name”: “Alice”}	Object(properties: {name → Scalar(“Alice”)})
Array: [1, 2, 3]	Array(elements: [Scalar(1), Scalar(2), Scalar(3)])

JSON-to-UDM is lossless. UDM-to-JSON is lossless. Round-trip fidelity is guaranteed.

10.3.2 XML → UDM

XML is more complex because it has features that JSON lacks: attributes, namespaces, mixed content, and the distinction between elements and text.

XML	UDM
Element with children: <Order><Customer>...</Customer></Order>	Object(properties: {Customer → ...})
Element with text: <Name>Alice</Name>	Object(properties: {_text → Scalar(“Alice”)})
Attribute: id=“123”	Stored in Object.attributes map
Repeated elements: <Item/><Item/>	Array of Objects
Namespace: xmlns=“...”	Stored in Object.metadata

The key design decisions for XML:

Text content uses `__text`. When an XML element contains only text — like `<Name>Alice</Name>` — the text is stored as a property called `__text` inside the UDM Object. This is an internal convention that you never see in output (the serializers unwrap it automatically). It exists because UDM Objects store properties as a map, and the text content needs a key.

When you write `$input.Order.Customer`, UTL-X automatically unwraps the `__text` and returns “Alice” — not the internal `{__text: "Alice"}` wrapper. This unwrapping is described in detail in Chapter 23.

Attributes are separate. XML attributes are stored in a separate `attributes` map on the UDM Object, not mixed with child element properties. This prevents name collisions — an element could theoretically have both a child element and an attribute with the same name. Access attributes with the `@` prefix: `$input.Order.@id`.

Repeated elements become arrays. When an XML element appears multiple times with the same name, they’re automatically grouped into a UDM Array. `<Item/><Item/><Item/>` becomes an Array of three Objects. Single elements stay as Objects (not wrapped in an array) unless array hints are provided.

10.3.3 CSV → UDM

CSV maps to an Array of Objects, where each row becomes an Object with column headers as property names:

```
name,age,city
Alice,30,Amsterdam
Bob,25,Rotterdam
```

Becomes:

```
Array([
  Object({name: "Alice", age: 30, city: "Amsterdam"}),
  Object({name: "Bob", age: 25, city: "Rotterdam"})
])
```

Numbers are auto-detected: “30” becomes `Scalar(number: 30)`, not `Scalar(string: “30”)`. Booleans too: “true” becomes `Scalar(boolean: true)`.

10.3.4 YAML → UDM

YAML is a superset of JSON, so the mapping is the same as JSON. YAML-specific features (anchors, aliases, tags) are resolved during parsing — the resulting UDM is identical to what JSON would produce for the same data.

10.3.5 OData → UDM

OData JSON is standard JSON with metadata conventions (`@odata.context`, `@odata.type`). These metadata properties are parsed as regular Object properties — accessible via `$input["@odata.context"]`.

10.4 UDM Navigation

10.4.1 Dot Notation

The primary way to access data:

```
$input.Order.Customer.Name      // nested property access
$input.Order.Items.Item[0].Price // array element access
$input.Order.@id                // XML attribute access
```

10.4.2 Safe Navigation

Handle missing properties gracefully with `?..`:

```
$input.Order?.Customer?.Name           // returns null if any step is missing
```

Without `?..`, a missing intermediate property causes an error. With `?..`, you get `null` — which you can then handle with `??` (nullish coalescing):

```
$input.Order?.Customer?.Name ?? "Unknown Customer"
```

10.4.3 Recursive Descent

Find a property name at any depth with `...`:

```
$input..ProductCode                     // finds ProductCode anywhere in the tree
```

Returns an array of all matches. Useful for deeply nested or variable-depth structures.

10.5 UDM and Type Coercion

UTL-X provides both automatic and explicit type coercion.

10.5.1 Automatic Coercion

In certain contexts, UTL-X converts types automatically:

- String concatenation: `concat("Total: ", 42)` — number 42 becomes string "42"
- XML text unwrapping: `$input.Customer` — the internal `__text` Object unwraps to the scalar value

10.5.2 Explicit Coercion

Use `stdlib` functions for explicit conversion:

```
toString(42)           // "42"
toNumber("42")         // 42
toBoolean("true")      // true
toNumber("not-a-num")  // 0 (returns 0 for unparseable input)
```

10.5.3 XML Text Node Unwrapping

This is the most important coercion in UTL-X. When XML is parsed:

```
<Name>Alice</Name>
```

The UDM contains: `Object(properties: {__text: Scalar("Alice")})`.

But when you write `$input.Name`, you get "Alice" — not the wrapper object. UTL-X automatically unwraps the `__text` node during property access. This is handled by the interpreter (for `TEMPLATE` strategy) and `RuntimeOps` (for `COMPILED` strategy).

The unwrapping rules:

- If an Object has only a `__text` property and no real attributes → unwrap to the scalar
- If an Object has `__text` AND attributes → keep as Object (attributes would be lost)
- If an Object has child elements → no unwrapping (it's a real object)

Chapter 23 covers the design decisions behind these rules in detail.

10.6 UDM is Format-Agnostic, Transformations are Format-Agnostic

The key insight: because UDM normalizes all formats into one tree, your transformation expression is *identical* regardless of input format.

The expression `$input.Order.Customer` works whether the input is:

- XML: `<Order><Customer>Alice</Customer></Order>`
- JSON: `\{"Order": \{"Customer": "Alice"\}\}`
- YAML: `Order:\n Customer: Alice`

Change `input.xml` to `input.json` in the header — the body stays the same. This is what “format-agnostic” means in practice: the transformation logic is decoupled from the serialization format.

10.7 UDM and Flat/Relational Data

UDM is inherently hierarchical — Objects contain Objects which contain Objects. This mirrors XML and JSON naturally. But not all data is hierarchical.

10.7.1 The Flat Data Challenge

Consider SAP IDoc segments exported as XML. Orders and order lines are *siblings*, not nested:

```
<IDOC>
  <E1EDK01><BELNR>ORD-001</BELNR><CURRENCY>EUR</CURRENCY></E1EDK01>
  <E1EDP01><BELNR>ORD-001</BELNR><MATNR>Widget</MATNR><MENGE>2</MENGE></E1EDP01>
  <E1EDP01><BELNR>ORD-001</BELNR><MATNR>Gadget</MATNR><MENGE>1</MENGE></E1EDP01>
  <E1EDK01><BELNR>ORD-002</BELNR><CURRENCY>USD</CURRENCY></E1EDK01>
  <E1EDP01><BELNR>ORD-002</BELNR><MATNR>Gizmo</MATNR><MENGE>5</MENGE></E1EDP01>
</IDOC>
```

The order lines belong to orders — but the relationship is in the `BELNR` field value, not in the XML structure. UDM parses this as siblings in the same array, with no awareness that `BELNR` is a join key.

This pattern appears in SAP IDocs, EDI/EDIFACT segments, database exports, and any flat file where parent-child relationships are expressed through reference keys rather than nesting.

10.7.2 How to Handle It in UTL-X

Use `groupBy` to build a lookup, then `map` to construct the hierarchy:

```
let headers = $input.IDOC.E1EDK01
let linesByOrder = groupBy($input.IDOC.E1EDP01, (line) -> line.BELNR)

map(headers, (header) -> {
  orderId: header.BELNR,
  currency: header.CURRENCY,
  lines: map(linesByOrder[header.BELNR] ?? [], (line) -> {
    product: line.MATNR,
    quantity: toNumber(line.MENGE)
  })
})
```

The `groupBy` creates an indexed map ($O(N)$ to build), and each header does an $O(1)$ key lookup — efficient even with thousands of records.

For multi-level nesting (`header` → `line` → `schedule`), chain multiple `groupBy` calls:

```
let linesByOrder = groupBy($input.lines, (l) -> l.orderId)
let schedulesByLine = groupBy($input.schedules, (s) -> s.lineId)

map($input.headers, (h) -> {
  ...h,
  lines: map(linesByOrder[h.orderId] ?? [], (l) -> {
    ...l,
    schedules: schedulesByLine[l.lineId] ?? []
  })
})
```

10.7.3 Hierarchical vs Relational: A Design Trade-Off

Tools like IBM’s Mercator (later WTX) solved this with an “intermediate card” — a declarative mapping that specifies join keys and cardinalities. The engine then automatically restructures flat data into hierarchies.

UTL-X takes a different approach: the `groupBy` + `map` pattern uses existing language constructs instead of a separate mapping artifact. This is more verbose but more flexible — you can add conditions, transformations, and error handling at each level.

The `nestBy()` stdlib function simplifies the common case:

```
let enrichedOrders = nestBy(
  $input.IDOC.E1EDK01,           // parent records
  $input.IDOC.E1EDP01,           // child records
  (header) -> header.BELNR,       // parent key extractor
  (line) -> line.BELNR,          // child key extractor
  "lines"                        // name for the new property on each parent
)
```

The name reads naturally: “nest E1EDP01 records by BELNR into a property called lines.” The 5th parameter (“lines”) is a string that becomes a *new property name* on each parent object. After `nestBy()`, every E1EDK01 gains a `.lines` property containing its matched E1EDP01 records — you access it with dot notation like any other property:

```
// After nestBy(), .lines is a regular property:
map(enrichedOrders, (order) -> {
  orderId: order.BELNR,
  lineCount: count(order.lines),
  total: sum(map(order.lines, (l) -> toNumber(l.MENGE) * toNumber(l.PRICE)))
})
```

See Chapter 21 (Data Restructuring) for the full analysis, performance characteristics, and multi-level nesting patterns.

10.8 UDM and Schema Formats (Tier 2)

The diagrams above show Tier 1 data transformations — instance documents (JSON, XML, CSV, YAML, OData) parsed into UDM. But UTL-X also handles Tier 2 schema formats: XSD, JSON Schema, Avro, Protobuf, OData Schema (EDMX), and Table Schema.

When a Tier 2 format is the input, the parser produces UDM enriched with USDL directives (`%types`, `%fields`, `%kind`) — see Chapter 12. The transformation can then work with both the raw schema structure and the normalized USDL properties.

Tier 2: Schema → Tier 2: Schema Meta Data Transformation

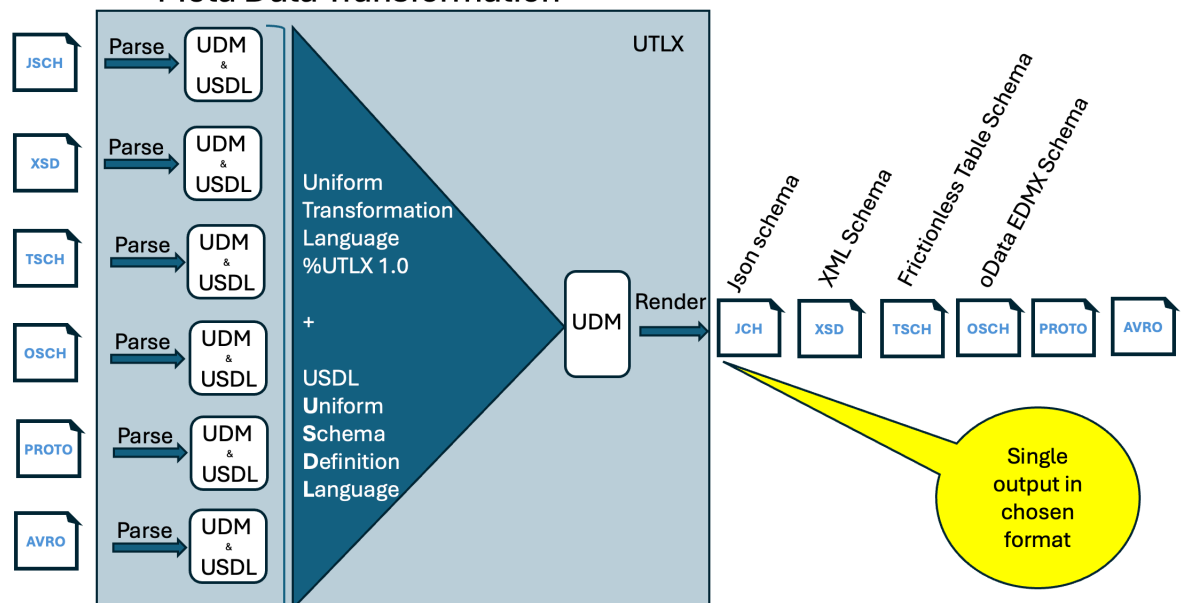


Figure 8: Tier 2 schema transformation — schema formats (JSON Schema, XSD, Table Schema, OData Schema, Protobuf, Avro) are parsed into UDM + USDL. The transformation can convert between any schema format pair.

Schema-to-schema conversion (e.g., input `xsd` / output `jsch`) works because USDL normalizes all schema concepts — types, fields, constraints — into a common vocabulary, just as UDM normalizes all data formats into a common tree.

10.8.1 Combined Transformations: Tier 1 + Tier 2

In some scenarios, a transformation receives both instance data (Tier 1) and schema metadata (Tier 2) as inputs. For example, a multi-input transformation might read a JSON message alongside its JSON Schema to produce validated, enriched output — or the output itself might be a schema format.

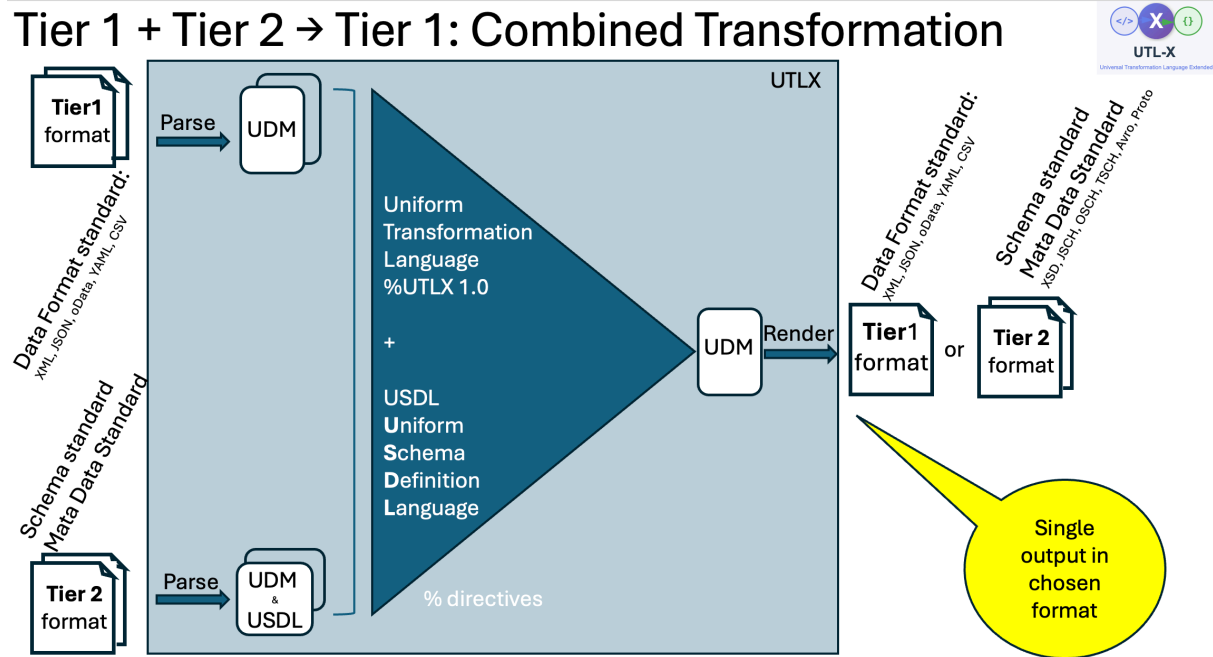


Figure 9: Combined transformation — Tier 1 instance data and Tier 2 schema metadata are parsed into UDM (with UDSL for Tier 2). The output can be either a Tier 1 data format or a Tier 2 schema format.

This flexibility means UTL-X handles not just data transformation but also schema generation (CSV metadata → XSD), schema inspection (XSD → JSON report), and data-plus-schema pipelines — all using the same language and the same UDM foundation.

10.9 When UDM Matters

Most of the time, you don't think about UDM — you just write `$input.name` and it works. UDM matters when:

- You're debugging unexpected behavior (especially XML attributes and `_text`)
- You're working with flat/relational data (IDoc segments, CSV with foreign keys)
- You're optimizing memory usage (UDM expansion factors — see Chapter 37)
- You're working with the COMPILED strategy (which generates bytecode against UDM types)
- You're writing a wrapper library (which serializes UDM to/from protobuf)
- You're contributing to UTL-X itself (all core logic operates on UDM)

For day-to-day transformation development, UDM is invisible — which is exactly the point. It does its job and gets out of the way.

11 Schema-to-Schema Mapping

This chapter is about *thinking in schemas* — defining your input and output contracts before writing transformation code. It covers how traditional tools approach schema mapping (and why their XSLT-centric approach is broken for JSON), how UTL-X handles it differently, and the full matrix of schema-to-schema conversions.

If you just want to transform data and don't care about schemas, you can skip this chapter. But if you're building production integrations — especially in regulated industries like finance, healthcare, or government — schema-driven development is non-negotiable.

11.1 Why Schema Mapping Matters

A schema defines the *contract* between two systems: what fields exist, what types they are, which are required, and what constraints apply. Without schemas, integration is ad-hoc — you discover missing fields, wrong types, and structural mismatches at runtime, often in production.

Professional integration starts with schemas:

1. Define the input schema: “What does the source system send?”
2. Define the output schema: “What does the target system expect?”
3. Write the transformation: “How do I map from one to the other?”
4. Validate: “Does my output actually match the contract?”

This is how every mature integration platform works — Tibco BW, SAP CPI, MuleSoft, and now UTL-X. The difference is *which schema formats* they support and *how* they do the mapping.

11.2 How Traditional Tools Do It

11.2.1 The XSLT Approach

In Tibco BW, SAP CPI, and Azure Logic Apps, the workflow is:

1. Load source XSD (XML Schema)
2. Load target XSD
3. Draw mapping lines between elements in a visual mapper
4. The tool generates an XSLT stylesheet
5. Runtime: XML input → XSLT processor → XML output

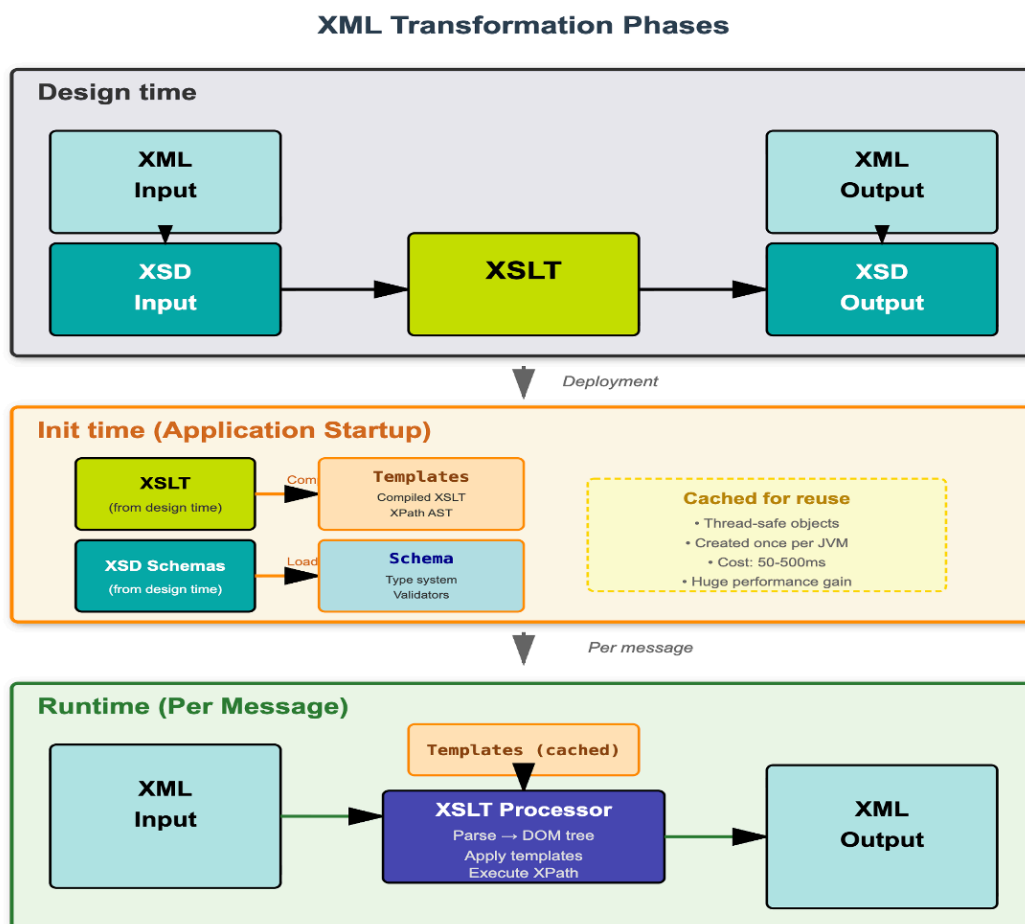


Figure 10: XML transformation phases — Design time: XSD schemas define input/output contracts, XSLT maps between them. Init time: XSLT compiled and cached. Runtime: XML parsed, XSLT applied, XML produced. This model works — but only for XML.

This works well for XML-to-XML transformations. The problem is: what about JSON?

11.2.2 The JSON Problem

Most modern APIs use JSON. But the mapping tools above only understand XSD (XML schemas). So they do this:

1. JSON arrives
2. Convert JSON to XML (using an internal XML “infoset” representation)
3. Load the source XSD (generated from JSON Schema or inferred)
4. Apply XSLT transformation
5. XML output
6. Convert XML back to JSON

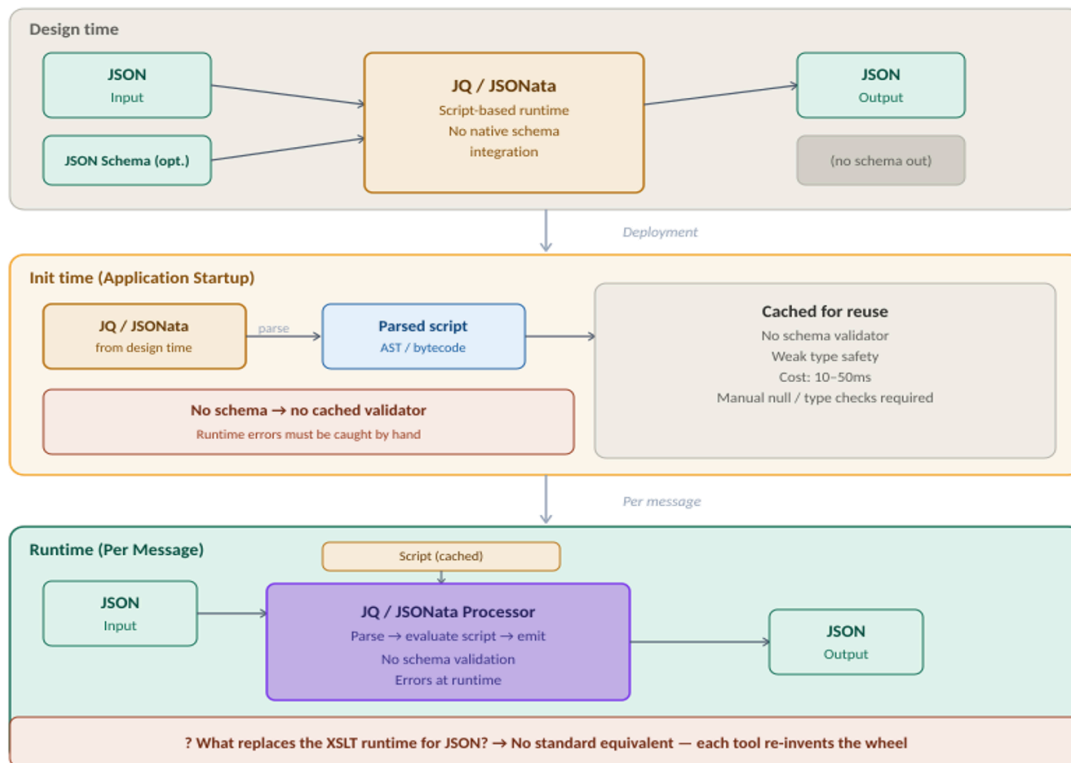


Figure 11: JSON transformation phases — no standard equivalent to XSLT exists. jq and JSONata fill parts of the gap but have no native schema binding, no cached validators, and no type system. Each tool re-invents the wheel.

Four conversion steps. Two of them lossy:

JSON → XML is lossy: JSON has typed values (numbers, booleans). XML has only text. JSON arrays have no element names. JSON has no attributes. These semantic gaps mean the conversion is an approximation, not an equivalence.

XML → JSON is lossy: XML attributes have no JSON equivalent. Single vs repeated elements (object vs array) is ambiguous. Namespace prefixes are stripped or mangled.

This anti-pattern is not theoretical — it's how Azure Logic Apps' Data Mapper, SAP CPI's Message Mapping, and Tibco BW work today. MuleSoft's DataWeave is the notable exception — it's format-agnostic, like UTL-X.

11.3 The UTL-X Approach

UTL-X doesn't convert between formats before mapping. The transformation operates on the UDM (Universal Data Model), which represents all formats natively:

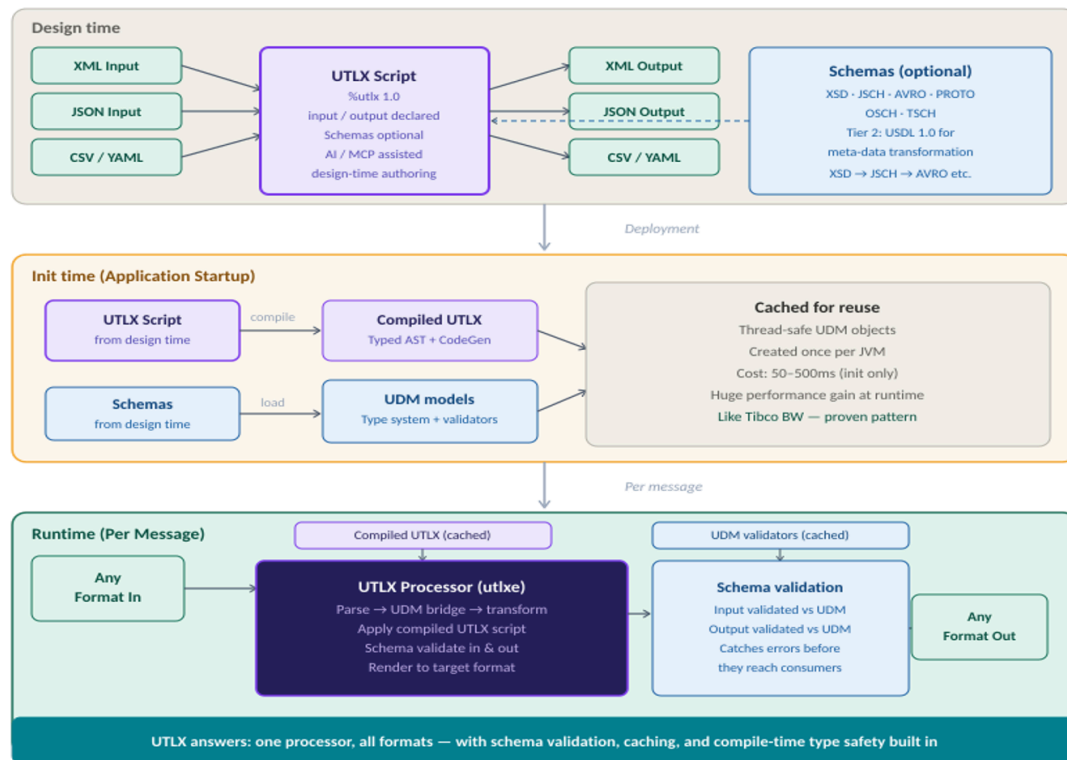


Figure 12: UTL-X transformation phases — one processor, all formats. Design time: any format declared, schemas optional. Init time: script compiled to typed AST, UDM models and validators cached. Runtime: parse any input, apply compiled UTL-X, validate and render to any output format.

- JSON input: parsed directly to UDM (lossless)
- XML input: parsed directly to UDM (attributes and namespaces preserved)
- CSV input: parsed directly to UDM (rows become objects)
- No intermediate XML. No XSLT. No lost types.

The schema mapping is format-agnostic too. You can map from a JSON Schema to an XSD-defined structure — UTL-X handles the format differences in parsing and serialization, not in the mapping logic.

11.4 Tier 1: Instance Data Mapping

Instance data mapping is what most people think of when they hear “transformation”: converting actual data from one structure to another.

11.4.1 Every Data Format Has an Accompanying Schema Format

Data Format (Tier 1)	Schema Format (Tier 2)	Standard
JSON	JSON Schema	IETF drafts (04 through 2020-12)
XML	XSD	W3C XML Schema Definition
CSV	Table Schema (TSCH)	Frictionless Data
YAML	JSON Schema	Shared with JSON
OData	EDMX / CSDL (OSCH)	OASIS OData

The schema describes the *structure* of the data. UTL-X can validate input and output against these schemas. The transformation maps between instances (actual data), guided by the schemas.

11.4.2 Common Instance Mapping Scenarios

- JSON → JSON: REST API v1 response → REST API v2 request

- XML → JSON: SOAP response → REST API request
- CSV → XML: bank statement → SEPA payment instruction (pain.001)
- JSON → CSV: API data → Excel-compatible report
- XML → XML: UBL invoice → country-specific tax format
- OData → JSON: Dynamics 365 data → internal canonical model

In every case, UTL-X handles the format conversion automatically. You focus on the field mapping — which field goes where, what calculations are needed, what conditions apply.

11.5 Tier 2: Metadata Mapping (Schema-to-Schema)

This is where UTL-X is unique: it can transform *schemas themselves*, not just data instances.

11.5.1 Why Metadata Mapping Is Powerful

Imagine you have an XSD for your XML API, but your new consumer needs a JSON Schema for their OpenAPI specification. Traditionally, you'd manually recreate the schema — tedious, error-prone, and impossible to keep synchronized.

With UTL-X, you transform the schema:

```
%utlx 1.0
input xsd
output jsch
---
$input
```

The XSD is parsed as structured data (it's XML, after all). The JSON Schema is produced by the JSCH serializer. Types, constraints, and structure are mapped automatically.

11.5.2 The Full Schema-to-Schema Matrix

UTL-X supports conversion between all Tier 2 schema formats:

From / To	XSD	JSCH	Avro	Proto	OSCH	TSCH
XSD	–	Yes	Yes	Yes	Yes	Yes
JSCH	Yes	–	Yes	Yes	Yes	Yes
Avro	Yes	Yes	–	Yes	Partial	Yes
Proto	Yes	Yes	Yes	–	Partial	Yes
OSCH	Yes	Yes	Partial	Partial	–	Yes
TSCH	Yes	Yes	Yes	Yes	Yes	–

“Partial” means some semantic concepts don't map perfectly between formats (e.g., OData navigation properties have no direct Avro equivalent). The conversion preserves what it can and documents what it can't.

11.5.3 Practical Use Cases for Schema Mapping

API gateway migration: Your organization is moving from SOAP (WSDL/XSD) to REST (OpenAPI/JSON Schema). Transform the XSD types to JSON Schema — automatically, not manually.

Kafka pipeline: Your producer uses JSON Schema but your schema registry requires Avro. Transform the schema format without changing the data format.

Data contract generation: You have an XSD for your XML feed. Generate a Table Schema (TSCH) for CSV data quality validation of the same data in flat-file form.

Documentation: Transform any schema into a human-readable table of field names, types, and constraints.

Code generation: Transform a schema into TypeScript interfaces, Java classes, or database DDL.

11.6 Nested Schemas: Include and Import

Real-world schemas are rarely single files. They reference other schemas — sometimes dozens of them. A UBL invoice schema imports common types, which import data types, which import core components. Understanding how these references work is essential for schema-to-schema mapping.

11.6.1 XSD: Include vs Import

XSD has two mechanisms for composing schemas, and the difference matters:

xs:include brings components from another schema file into the **same namespace**. The included file either has the same target namespace or no namespace at all (the “chameleon” pattern — it adopts the including schema’s namespace). After include, all types are in one namespace as if they were defined in a single file.

```
<!-- order.xsd - target namespace: urn:example:orders -->
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  targetNamespace="urn:example:orders">
  <xs:include schemaLocation="common-types.xsd"/> <!-- same namespace -->
  <xs:element name="Order" type="OrderType"/>
</xs:schema>
```

xs:import brings components from a **different namespace**. This is how cross-domain schemas are composed — an order schema imports customer types from the CRM namespace, address types from a shared namespace, and country codes from an ISO standard namespace. Each retains its own namespace prefix.

```
<!-- order.xsd - imports from a different namespace -->
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:cust="urn:example:customers"
  targetNamespace="urn:example:orders">
  <xs:import namespace="urn:example:customers"
    schemaLocation="customer-types.xsd"/> <!-- different namespace -->
  <xs:element name="Order">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="Customer" type="cust:CustomerType"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

The rule: **include** = same family (same namespace), **import** = different family (different namespace). Enterprise schemas like UBL and ISO 20022 use both extensively — a typical UBL invoice pulls in 10+ schema files via a chain of includes and imports.

11.6.2 Other Format Mechanisms

Each schema format has its own composition mechanism:

- **JSON Schema:** `$ref` — local (`#/definitions/Address`) and remote (`./address.json#/Address`) references. Draft 2020-12 added `$dynamicRef` for extensibility.

- **Avro:** Named types referenced by their full name across schemas. No explicit import — types are resolved by name from the registry.
- **Protobuf:** `import "other.proto"` — explicit file import. `import public` re-exports for transitive visibility.
- **OpenAPI:** `\$ref` to `#/components/schemas/...` (local) or external files. Supports splitting a large API spec across files.
- **RAML:** `!include` for fragments, `uses:` for libraries, overlays and extensions for modular specs.

11.6.3 How UTL-X Handles Nested Schemas

UTL-X resolves all references during parsing:

1. Follow includes and imports recursively (loading referenced files)
2. Build a complete type graph (all types from all files visible)
3. Make all types available for mapping (regardless of which file defined them)
4. Preserve reference structure in output (for round-trip fidelity)

This means you can transform a complex, multi-file XSD with imports into a single JSON Schema with `\$ref` definitions — or flatten everything into one file. The type graph is the same either way; only the serialization changes.

11.7 JSON Schema to JSON Schema Mapping

A special but common case: transforming between JSON Schema versions.

JSON Schema has evolved through multiple drafts — draft-04, draft-06, draft-07, 2019-09, and 2020-12. Each version introduced new keywords, changed semantics, or deprecated features.

UTL-X can migrate schemas between versions:

- draft-04 → 2020-12: update `\$schema` URI, replace `definitions` with `\$defs`, update `\$ref` syntax
- Add or remove fields while preserving constraints
- Rename properties while maintaining validation rules

This is valuable for API versioning — evolve the schema while maintaining backward compatibility.

11.8 Schema-Driven Validation in UTL-X

Schemas aren't just for documentation — UTL-X uses them for runtime validation:

11.8.1 Pre-Validation

Validate the input *before* transformation:

```
%utlx 1.0
input json {schema: "order-input.json"}
output xml
---
// transformation body
```

If the input doesn't match `order-input.json` (a JSON Schema), the transformation fails with a validation error — before any mapping code runs. This catches data quality issues at the source.

11.8.2 Post-Validation

Validate the output *after* transformation:

```
%utlx 1.0
input json
output xml {schema: "invoice-output.xsd"}
```

```
---  
// transformation body
```

If the output doesn't match `invoice-output.xsd`, the transformation fails — catching mapping bugs before they reach the target system.

11.8.3 The Validation Orchestrator

UTLXe's validation orchestrator runs validation in a sandwich pattern:

1. Pre-validate input against input schema
2. Execute transformation
3. Post-validate output against output schema

If any step fails, the message is rejected with a detailed error — field name, constraint violated, expected vs actual value. No partial results, no corrupted output.

11.9 The Bridge: USDL

The next chapter introduces USDL — UTL-X's own Universal Schema Definition Language. USDL is to schemas what UDM is to data: a format-agnostic representation that bridges all schema formats.

Where UDM asks “what does the data look like?”, USDL asks “what *should* the data look like?” — types, constraints, cardinality, and documentation in one portable format.

12 USDL: Universal Schema Definition Language

If UDM is the universal representation for *data*, USDL is the universal representation for *schemas*. Just as UDM lets you write format-agnostic transformations, USDL lets you write format-agnostic schema definitions — one schema that can be read from any schema format, edited, and written to any other schema format.

12.1 What USDL Is

USDL is NOT a separate language or file format. It is a **convention** — a set of property names starting with % that represent schema information in a format-agnostic way:

```
{
  "%namespace": "com.example.orders",
  "%types": {
    "Order": {
      "%kind": "object",
      "%fields": {
        "id": { "%type": "string", "%required": true },
        "total": { "%type": "number", "%minimum": 0 }
      }
    }
  }
}
```

These % properties can live alongside any other data in a UDM tree. The % prefix is reserved for USDL — it never collides with raw format properties (XML elements don't start with %, JSON keys don't start with %, etc.).

12.2 How USDL Works — Automatically

USDL works through three automatic mechanisms. No special tags or declarations are needed.

12.2.1 1. Enrichment (Input — Tier 2 Schema Formats)

When UTL-X reads a schema format (XSD, JSON Schema, Avro, Protobuf, EDMX, Table Schema), the parser automatically **enriches** the UDM with USDL % properties alongside the raw structure:

```
%utlx 1.0
input xsd
output json
---
// Both views available on the same $input:
{
  rawElementName: $input["xs:element"].@name,      // raw XSD access
  types: $input["%types"],                          // USDL access
  pattern: $input.^xsdPattern                       // metadata access
}
```

The raw schema structure is preserved AND the USDL abstraction is layered on top. No information loss.

12.2.2 2. Detection (Output — Tier 2 Schema Serializers)

All six Tier 2 serializers automatically detect %types in the UDM and generate the native schema format:

```
%utlx 1.0
input json
output xsd
```

```

---
// Write USDL in the body – the XSD serializer detects %types and generates XSD
{
  "%namespace": "com.example",
  "%types": {
    "Customer": {
      "%kind": "object",
      "%fields": {
        "name": { "%type": "string", "%required": true },
        "email": { "%type": "string", "%format": "email" }
      }
    }
  }
}

```

The serializer doesn't care HOW %types got there — enrichment, data, or literal. It just detects and generates.

12.2.3 3. Carrier (YAML/JSON as Human-Readable Schema)

YAML and JSON naturally carry USDL properties as regular data. This enables a powerful round-trip:

1. Read schema: input xsd → \$input has raw + USDL %types
2. Dump to YAML: output yaml → human-readable YAML with %types
3. Edit: developer modifies the YAML in a text editor
4. Read back: input yaml → %types flows through as data
5. Generate: output xsd → serializer sees %types → new XSD

YAML is the human-readable editing layer for schemas. Read any schema as YAML, edit in any text editor, write back as any schema format.

USDL enrichment is fully implemented. All 6 Tier 2 parsers add % properties alongside the raw structure on input. All 6 serializers detect %types on output. Both raw and USDL access work simultaneously: \$input["xs:element"] (raw) and \$input["%types"] (USDL). Enrichment failure never blocks — \$input["_diagnostics"]["%_status"] reports "complete", "partial", or "failed".

12.3 The USDL Directive Classification

USDL organizes its directives in three levels of portability — from universal concepts that every schema format shares, down to features unique to a single standard:

Level	Scope	What it covers	Applies to
Core	Portable	Types, names, descriptions — the fundamentals	All formats
Shared	Widespread	Constraints, validation — supported by 80%+	Most formats
Format-native	Exclusive	Features unique to one format family	One format
Reserved	Future	Future USDL versions	Planned

This classification is the key to understanding USDL. Core and Shared directives work everywhere. Format-native directives target a specific output format. When converting USDL to a format that doesn't support a Format-native directive, it's preserved as a comment or metadata.

12.4 Core Directives (Universal)

These directives are supported by every schema format:

Directive	What it defines	Example
%namespace	Schema namespace/package	%namespace: "com.example.orders"
%version	Schema version	%version: "1.0.0"
%types	Type definitions block	%types: { Customer: {...} }
%kind	Type kind (object, enum, union)	%kind: "object"
%name	Type or field name	%name: "Customer"
%type	Field data type	%type: "string"
%description	Human-readable description	%description: "Customer ID"
%fields	Field definitions block	%fields: { id: {...} }
%values	Enum values	%values: ["ACTIVE", "INACTIVE"]

12.4.1 Example: Customer schema using only Core directives

This works in ALL output formats because every format understands types, names, and descriptions:

```
%utlx 1.0
input auto
output json
---
{
  "%namespace": "com.example.crm",
  "%version": "1.0.0",
  "%types": {
    "Customer": {
      "%kind": "object",
      "%description": "A customer record",
      "%fields": {
        "id": { "%type": "string", "%description": "Unique identifier" },
        "name": { "%type": "string", "%description": "Full name" },
        "email": { "%type": "string", "%description": "Email address" },
        "active": { "%type": "boolean" }
      }
    }
  }
}
```

12.5 Shared Directives (Most Formats) (80%+ Format Support)

These express constraints and validation rules supported by most formats:

Directive	What it constrains	Supported by	Example value
%required	Field is mandatory	All formats	true
%nullable	Field can be null	JSON Schema, Avro, Proto	true
%array	Field is an array	All formats	true
%default	Default value	JSON Schema, Avro, XSD	"EUR"
%minLength	Min string length	JSON Schema, XSD	1
%maxLength	Max string length	JSON Schema, XSD	255
%pattern	Regex pattern	JSON Schema, XSD	see code examples
%minimum	Min number value	JSON Schema, XSD	0

%maximum	Max number value	JSON Schema, XSD	999999
%enum	Allowed values	All formats	see code examples
%format	Semantic format hint	JSON Schema	“email”
%example	Example value	JSON Schema, OpenAPI	see code examples

12.5.1 Example: Customer with Core + Shared constraints

```
%utlx 1.0
input auto
output json
---
{
  "%namespace": "com.example.crm",
  "%version": "1.0.0",
  "%types": {
    "Customer": {
      "%kind": "object",
      "%fields": {
        "id": { "%type": "string", "%required": true, "%pattern": "^C-[0-9]{5}$" },
        "name": { "%type": "string", "%required": true, "%minLength": 1, "%maxLength":
100 },
        "email": { "%type": "string", "%format": "email" },
        "age": { "%type": "integer", "%minimum": 0, "%maximum": 150 },
        "status": { "%type": "string", "%enum": ["ACTIVE", "INACTIVE"], "%default":
"ACTIVE" },
        "orders": { "%type": "Order", "%array": true, "%required": false }
      }
    },
    "Order": {
      "%kind": "object",
      "%fields": {
        "orderId": { "%type": "string", "%required": true },
        "total": { "%type": "decimal", "%minimum": 0 },
        "currency": { "%type": "string", "%enum": ["EUR", "USD", "GBP"] }
      }
    }
  }
}
```

Formats that don't support a constraint (e.g., Protobuf has no %pattern) preserve it as a documentation comment.

12.6 Format-Native Directives

These target a specific schema format. They express features that only make sense in one context.

12.6.1 Protobuf-Specific (proto)

Directive	What it sets	Example value
%fieldNumber	Protobuf field tag number	1
%packed	Packed encoding for repeated fields	true
%oneof	Protobuf oneof group name	“payment_method”
%map	Protobuf map type	true
%reserved	Reserved field numbers	see code examples

```
%utlx 1.0
input auto
output proto
---
{
  "%namespace": "com.example.orders",
  "%types": {
    "Order": {
      "%kind": "object",
      "%fields": {
        "id": { "%type": "string", "%required": true, "%fieldNumber": 1 },
        "customer": { "%type": "string", "%fieldNumber": 2 },
        "total": { "%type": "double", "%fieldNumber": 3 },
        "items": { "%type": "LineItem", "%array": true, "%fieldNumber": 4 }
      }
    }
  }
}
```

Generates:

```
syntax = "proto3";
package com.example.orders;

message Order {
  string id = 1;
  string customer = 2;
  double total = 3;
  repeated LineItem items = 4;
}
```

The `%fieldNumber` directive is Protobuf-specific — it has no meaning in JSON Schema or XSD.

12.6.2 Avro-Specific (`avro`)

Directive	What it sets	Example value
<code>%logicalType</code>	Avro logical type	“date”
<code>%aliases</code>	Alternative field names	see code example
<code>%precision</code>	Decimal precision (digits)	10
<code>%scale</code>	Decimal scale (decimal places)	2

```
%utlx 1.0
input auto
output avro
---
{
  "%namespace": "com.example.finance",
  "%types": {
    "Transaction": {
      "%kind": "object",
      "%fields": {
        "id": { "%type": "string", "%required": true },
        "amount": { "%type": "decimal", "%precision": 10, "%scale": 2 },
        "date": { "%type": "date", "%logicalType": "date" },

```

```

      "oldId": { "%type": "string", "%aliases": ["legacy_id"] }
    }
  }
}

```

Generates Avro schema with logical types, decimal precision, and field aliases.

12.6.3 XSD-Specific (xsd)

Directive	What it sets	Example value
%elementFormDefault	XSD element form	“qualified”
%attributeFormDefault	XSD attribute form	“unqualified”
%choice	XSD choice group (xs:choice)	true
%all	XSD all group (xs:all)	true
%xml	XML-specific options	wrapped: true

```

%utlx 1.0
input auto
output xsd
---
{
  "%namespace": "urn:example:invoicing:v1",
  "%elementFormDefault": "qualified",
  "%types": {
    "Invoice": {
      "%kind": "object",
      "%fields": {
        "id": { "%type": "string", "%required": true },
        "issueDate": { "%type": "date", "%required": true },
        "paymentMethod": {
          "%choice": true,
          "%fields": {
            "creditCard": { "%type": "CreditCardPayment" },
            "bankTransfer": { "%type": "BankTransferPayment" }
          }
        }
      }
    }
  }
}

```

The %choice directive generates an xs:choice group — unique to XSD.

12.6.4 Database/SQL-Specific (future — no DDL output format yet)

Directive	What it sets	Example value
%primaryKey	Primary key field	true
%autoIncrement	Auto-increment	true
%foreignKey	Foreign key reference	“customers.id”
%index	Create index on field	true
%unique	Unique constraint	true

%sqlType	Override SQL type	“VARCHAR(50)”
----------	-------------------	---------------

These directives describe database concerns. UTL-X has no output `sql` format — there is no DDL serializer. These directives are preserved as metadata when converting between schema formats, and could be consumed by external code generators or a future DDL output format. They can be included in any USDL schema for documentation purposes. Example:

```
%utlx 1.0
input auto
output json
---
{
  "%types": {
    "Customer": {
      "%kind": "object",
      "%fields": {
        "id": { "%type": "integer", "%primaryKey": true, "%autoIncrement": true },
        "name": { "%type": "string", "%maxLength": 100, "%required": true, "%index":
true },
        "email": { "%type": "string", "%maxLength": 255, "%unique": true },
        "country": { "%type": "string", "%sqlType": "CHAR(2)" }
      }
    },
    "Order": {
      "%kind": "object",
      "%fields": {
        "id": { "%type": "integer", "%primaryKey": true, "%autoIncrement": true },
        "customerId": { "%type": "integer", "%foreignKey": "Customer.id", "%index":
true },
        "total": { "%type": "decimal", "%sqlType": "DECIMAL(10,2)" }
      }
    }
  }
}
```

12.6.5 OData-Specific (osch)

Directive	What it sets	Example value
%entityType	OData entity type marker	true
%navigation	Navigation property	true
%target	Navigation target entity	“Orders”
%cardinality	Relationship cardinality	“many”

These generate OData EDMX/CSDL metadata documents. Example:

```
%utlx 1.0
input auto
output osch
---
{
  "%namespace": "com.example.crm",
  "%types": {
    "Customer": {
      "%kind": "object",
```

```

    "%entityType": true,
    "%fields": {
      "id": { "%type": "integer", "%required": true },
      "name": { "%type": "string" },
      "orders": { "%type": "Order", "%navigation": true, "%target": "Orders",
"%cardinality": "many" }
    }
  }
}
}

```

12.6.6 JSON Schema-Specific (jsch)

Directive	What it sets	Example value
%additionalProperties	Allow extra properties	false
%prefixItems	Tuple validation (array positional types)	see code examples
%contentEncoding	Binary content encoding	“base64”
%contentType	Content MIME type	“application/json”
%discriminator	Polymorphism discriminator field	“type”
%if / %then / %else	Conditional schema application	see code examples
%dependentSchemas	Schema dependencies between fields	see code examples
%dependentRequired	Required field dependencies	see code examples

These are JSON Schema 2020-12 keywords that have no equivalent in XSD, Avro, or Protobuf. When converting to other formats, they are preserved as documentation comments.

12.6.7 Table Schema-Specific (tsch — Frictionless Data)

Directive	What it sets	Example value
%missingValues	Values treated as null/missing	see code examples
%groupChar	Thousands separator for number parsing	“,”
%decimalChar	Decimal separator	“.”
%bareNumber	Allow non-numeric characters in number fields	false
%trueValues	Strings treated as boolean true	see code examples
%falseValues	Strings treated as boolean false	see code examples

Table Schema directives describe CSV/tabular data conventions — how numbers are formatted, what values mean “missing,” and how booleans are represented in text. These are unique to tabular data and have no equivalent in XML or JSON schema formats.

12.7 How Directive Levels Interact During Conversion

When converting USDL to a specific format, each level is handled differently:

Level	Behavior during conversion
Core	Always converted — every format supports these
Shared	Converted when supported, preserved as comment when not
Format-native	Converted only for the matching format, ignored/commented for others
Reserved	Passed through as metadata

Example: converting a schema with `%fieldNumber: 3` and `%pattern: "^[A-Z]+$"`:

- **To Protobuf:** `%fieldNumber` becomes `= 3` (native), `%pattern` becomes a comment
- **To JSON Schema:** `%fieldNumber` ignored, `%pattern` becomes `"pattern": "^[A-Z]+$"` (native)
- **To XSD:** `%fieldNumber` ignored, `%pattern` becomes `xs:pattern` (native)
- **To Avro:** both become metadata properties

This is the power of the classification system: write ONE schema with ALL the information, and each output format takes what it understands.

12.8 Converting Between Formats via USDL

12.8.1 XSD to JSON Schema

```
%utlx 1.0
input xsd
output jsch
---
$input
```

12.8.2 Avro to Protobuf

```
%utlx 1.0
input avro
output proto
---
$input
```

12.8.3 Any Schema to Human-Readable YAML

```
%utlx 1.0
input xsd
output yaml
---
$input
```

Outputs the schema as clean, readable YAML with USDL directives — useful for documentation and understanding unfamiliar schemas.

12.9 USDL for Validation

USDL schemas can be referenced for transformation validation:

```
%utlx 1.0
input json {schema: "customer.usdl.json"}
output xml {schema: "invoice.usdl.yaml"}
---
// transformation body – validated against both schemas
```

The validation orchestrator converts the USDL to the format-appropriate validator.

12.10 USDL as Documentation

USDL output in YAML is human-readable by design — it serves as documentation without additional tooling:

```
%utlx 1.0
input xsd
output yaml
---
$input
```

Reading the YAML output, you see field names, types, constraints, and descriptions in one place — no need to decode XSD’s verbose XML syntax or JSON Schema’s nested structure.

12.11 USDL vs Standalone Schema Languages

Feature	USDL	JSON Schema	XSD	Avro
Multi-format output	Yes (all 6)	JSON only	XML only	Avro only
Classification system	Yes (Core/Shared/Format-native)	No	No	No
Format-specific features	Format-native	JSON keywords	XML facets	Avro types
Human-readable	Very (YAML)	Moderate	Low (XML)	Moderate
Part of UTL-X	Yes (dialect)	Standalone	Standalone	Standalone

USDL is not a replacement for JSON Schema or XSD — those have their own ecosystems. USDL is the *bridge* between them: write schema information once using classified directives, generate whatever format you need. The Rosetta Stone for schemas — and it lives inside `%utlx 1.0`.

13 Format Support Overview

UTL-X supports 11 formats organized in two tiers. Tier 1 formats are *data formats* — you transform instances of these daily. Tier 2 formats are *schema formats* — you work with these when defining contracts, validating data, or converting between schema standards.

This chapter provides an overview of all supported formats, their key characteristics, and the conversion matrix. Detailed treatment of each format is in Part III (Chapters 21–28).

13.1 Tier 1: Data Formats

Data formats carry the actual business data — orders, invoices, patient records, sensor readings, configurations.

13.1.1 JSON

The lingua franca of modern APIs. Every REST service, every frontend framework, every NoSQL database speaks JSON.

- **Parser:** recursive descent, handles all JSON types including nested structures
- **Number handling:** preserves integer vs float distinction (42 vs 42.0)
- **Encoding:** UTF-8 (standard), auto-detects BOM
- **Options:** `{writeAttributes: true}` for XML attribute preservation
- **Round-trip:** JSON → UDM → JSON is lossless

13.1.2 XML

The enterprise workhorse. SOAP services, EDI, UBL invoices, HL7 healthcare, government data — XML is everywhere in enterprise integration.

- **Parser:** custom recursive descent with namespace support
- **Attributes:** stored separately in UDM, accessed via `\@` prefix
- **Namespaces:** full support including prefixed elements and inheritance
- **Text content:** internal `_text` convention (auto-unwrapped, invisible in output)
- **Repeated elements:** auto-detected as arrays, configurable via array hints
- **Encoding:** auto-detects from XML declaration (UTF-8, ISO-8859-1, Windows-1252)
- **Options:** `{encoding: "UTF-8"}` for output encoding

13.1.3 CSV

The universal import/export format. Every spreadsheet, every database, every legacy system can produce and consume CSV.

- **Parser:** handles quoted fields, escaped delimiters, multi-line values
- **Headers:** auto-detected from first row (configurable)
- **Type detection:** numbers, booleans, dates auto-detected from string content
- **Delimiters:** comma (default), semicolon, pipe, tab
- **Regional formats:** USA (1,234.56), European (1.234,56), French (1 234,56), Swiss (1'234.56)
- **BOM:** detects and handles UTF-8 BOM
- **Options:** `{delimiter: ";", headers: true, regionalFormat: "european", decimals: 2}`

13.1.4 YAML

The configuration standard. Kubernetes manifests, CI/CD pipelines, Docker Compose, OpenAPI specs — YAML dominates DevOps.

- **Parser:** SnakeYAML-based, handles all YAML 1.1 features
- **Aliases and anchors:** resolved during parsing (transparent to transformation)
- **Multi-document:** supported (separated by `---`)
- **Output styles:** block (default, human-readable) or flow (compact, JSON-like)
- **Options:** `{writeAttributes: true}`, same as JSON

- **Round-trip:** YAML is a superset of JSON — same UDM mapping

13.1.5 OData

The Microsoft ecosystem format. Dynamics 365, SharePoint, SAP Gateway, Power Platform — OData JSON is standard JSON with metadata conventions.

- **Parser:** standard JSON parser with OData metadata awareness
- **Metadata:** @odata.context, @odata.type, @odata.id accessible as properties
- **Entity sets:** standard value array pattern
- **Navigation properties:** nested entity access
- **Options:** {metadata: "...", context: "...", wrapCollection: true}

13.2 Tier 2: Schema Formats

Schema formats describe the *structure* of data — types, constraints, cardinality. They're used for validation, documentation, code generation, and contract enforcement.

13.2.1 XSD (XML Schema Definition)

The W3C standard for XML validation. Every enterprise XML standard (UBL, HL7, ISO 20022, EDIFACT) ships with XSD.

- **Versions:** 1.0 and 1.1
- **Patterns:** Russian Doll, Venetian Blind, Salami Slice, Garden of Eden, Swiss Army Knife (Chapter 29)
- **Features:** complex types, simple types, restrictions, extensions, imports

13.2.2 JSCH (JSON Schema)

The schema language for JSON. Used in OpenAPI, AsyncAPI, data contracts.

- **Drafts:** draft-04, draft-06, draft-07, 2019-09, 2020-12
- **Features:** type, properties, required, pattern, enum, \$ref, allOf/oneOf/anyOf

13.2.3 Avro

Apache Avro schema for data pipelines. The standard schema format for Kafka and Hadoop ecosystems.

- **Features:** records, enums, arrays, maps, unions, logical types (date, timestamp, decimal, UUID)

13.2.4 Protobuf (Protocol Buffers)

Google's schema format for gRPC and efficient serialization.

- **Version:** proto3
- **Features:** messages, enums, repeated fields, oneof, maps, packages

13.2.5 OSCH (OData Schema / EDMX)

The schema format for OData services. Describes entity types, navigation properties, and service endpoints.

- **Format:** XML-based CSDL (Common Schema Definition Language)

13.2.6 TSCH (Table Schema)

The Frictionless Data standard for CSV metadata. Describes column names, types, and constraints for tabular data.

- **Format:** JSON-based field definitions

13.3 Format Detection

UTL-X auto-detects the input format in identity mode and expression mode:

Detection rule	Format
----------------	--------

Starts with < or <?xml	XML
Starts with \{ or [	JSON
Contains commas/semicolons with consistent column count	CSV
Contains : patterns without braces	YAML
Explicit --from flag	Overrides auto-detection

Auto-detection works for 95% of cases. Use `--from` when the input is ambiguous (e.g., a YAML file that looks like JSON, or a CSV with only one column).

13.4 Format Conversion Matrix

UTL-X can convert between any pair of Tier 1 formats:

From / To	JSON	XML	CSV	YAML	OData
JSON	–	Yes	Yes	Yes	Yes
XML	Yes	–	Yes	Yes	Yes
CSV	Yes	Yes	–	Yes	Yes
YAML	Yes	Yes	Yes	–	Yes
OData	Yes	Yes	Yes	Yes	–

Every conversion is supported. Some are more natural than others:

- **JSON ↔ YAML:** lossless (YAML is a superset of JSON)
- **JSON ↔ XML:** mostly lossless (XML attributes need `writeAttributes` for full fidelity)
- **CSV → JSON/XML:** lossless (rows become objects/elements)
- **JSON/XML → CSV:** requires flattening (hierarchical → tabular, may lose nesting)
- **OData → JSON:** strip metadata for clean JSON

13.5 Identity Mode Defaults

When using identity mode (no script, just format conversion), UTL-X picks sensible defaults:

Input	Default output	Override with
XML	JSON	<code>--to yaml, --to csv</code>
JSON	XML	<code>--to yaml, --to csv</code>
CSV	JSON	<code>--to xml, --to yaml</code>
YAML	JSON	<code>--to xml, --to csv</code>

The default “flip” is always to JSON (except JSON input, which flips to XML). This reflects the most common use case: “I have XML/CSV/YAML and I need JSON.”

13.6 Format Options Summary

All format options use the `{key: value}` syntax in the output declaration:

Format	Option	Example
JSON	<code>writeAttributes</code>	<code>{writeAttributes: true}</code>
XML	<code>encoding</code>	<code>{encoding: "UTF-8"}</code>
CSV	<code>delimiter</code>	<code>{delimiter: ";"}</code>
CSV	<code>headers</code>	<code>{headers: true}</code>

CSV	regionalFormat	{regionalFormat: "european"}
CSV	decimals	{decimals: 2}
CSV	bom	{bom: true}
YAML	writeAttributes	{writeAttributes: true}
Any	schema (planned)	{schema: "contract.json"}

Options are optional — sensible defaults apply when not specified. Multiple options combine: `output csv {delimiter: ";", headers: true, regionalFormat: "european"}`.

13.7 BOM (Byte Order Mark)

A BOM is an invisible character (Unicode U+FEFF) that some tools prepend to text files to signal the encoding. It was designed for UTF-16 (where byte order matters — hence the name), but it's also commonly added to UTF-8 files by Windows tools, especially Excel and Notepad.

The problem: a BOM is invisible in text editors but very much visible to parsers. A JSON file that starts with a BOM contains `\uFEFF{"name": "Alice"}` — the `\uFEFF` before the opening brace causes a parse error in strict parsers. A CSV file with a BOM has an invisible character prepended to the first column header, so `\uFEFFName` doesn't match `Name` in your transformation.

UTL-X handles BOM automatically across all formats:

- **On input:** the JSON, CSV, and YAML parsers detect and strip a leading BOM before parsing. No configuration needed — it just works. You never see the BOM in `$input`.
- **On output:** CSV can optionally prepend a BOM with `{bom: true}`. This is useful when the output will be opened in Excel on Windows, which uses the BOM to detect UTF-8 encoding for non-ASCII characters (accented names, CJK characters). JSON, XML, and YAML output never includes a BOM.

In practice, BOM issues appear most often when:

- Receiving CSV files from Windows systems (Excel adds BOM by default)
- Processing JSON files saved with Windows Notepad
- Integrating with legacy systems that expect or produce BOM-prefixed files

UTL-X's approach: strip on input (always), add on output (only when explicitly requested for CSV). This matches the W3C recommendation: UTF-8 files should not have a BOM, but parsers should tolerate one.

13.8 What's Next

Part I is complete. You now understand:

- What UTL-X is and why it exists (Chapters 1–3)
- How to install and use it (Chapters 4–5)
- The three executables and the IDE (Chapters 6–7)
- The language fundamentals (Chapter 8)
- The Universal Data Model (Chapter 10)
- Schema mapping and USDL (Chapters 11–11)
- All supported formats (this chapter)

Part II dives into the language itself: expressions, functions, pattern matching, validation, and pipeline chaining. By the end of Part II, you'll be able to write any transformation UTL-X is capable of.

Part II

The Language

Mastering expressions, functions, patterns, and advanced features

14 Expressions and Operators

UTL-X is an expression-based language — everything produces a value. There are no statements, no void functions, no side effects. A transformation body is one expression (possibly with `let` bindings) that evaluates to the output.

This chapter covers all operators, their precedence, and practical patterns. If you've used JavaScript, Kotlin, or any C-family language, most operators will be immediately familiar. The individual operators are introduced in Chapter 9 (Operators); here the focus is on precedence, object/array construction, and how operators combine in real transformations — see Chapter 9 for the compact per-operator reference.

14.1 Arithmetic Operators

Standard arithmetic on numbers:

```
10 + 3      // 13      addition
10 - 3      // 7       subtraction
10 * 3      // 30      multiplication
10 / 3      // 3.333... division (always floating point)
10 % 3      // 1       modulo (remainder)
2 ** 10     // 1024    exponentiation
-42         // -42     unary minus
```

Division always produces a floating-point result — `10 / 3` is `3.333...`, not `3`. There is no integer division operator. Use `floor(10 / 3)` if you need integer results.

Division by zero throws a runtime error. Use a guard if the divisor might be zero:

```
if (count > 0) total / count else 0
```

14.2 Comparison Operators

Compare values and produce booleans:

```
10 == 10    // true    equality
10 != 5     // true    inequality
10 > 5      // true    greater than
10 < 5      // false   less than
10 >= 10    // true    greater or equal
10 <= 5     // false   less or equal
```

String comparison is lexicographic (alphabetical):

```
"apple" < "banana" // true
"Alice" == "alice" // false (case-sensitive)
```

Null comparisons:

```
null == null // true
null != "abc" // true
null == 0     // false (null is not zero)
```

14.3 Logical Operators

Combine boolean expressions:

```

true && false    // false    logical AND
true || false    // true     logical OR
!true           // false     logical NOT

```

Short-circuit evaluation: `&&` stops at the first `false`, `||` stops at the first `true`. This is useful for guarding expressions:

```

// Safe: second expression only evaluates if items is not null
$input.items != null && count($input.items) > 0

```

14.3.1 Truthy and Falsy Values

In boolean contexts, UTL-X treats these as *falsy*:

- `false`
- `null`
- `0`
- `""` (empty string)

Everything else is *truthy*, including empty arrays `[]` and empty objects `{}`.

14.4 String Concatenation

UTL-X does not use `+` for string concatenation — use the `concat()` function:

```

concat("Hello", " ", "World")    // "Hello World"
concat($input.first, " ", $input.last)

```

This is deliberate. `+` is arithmetic only. Mixing `+` for both numbers and strings leads to ambiguity ("`3 + 4` — is it "`34`" or `7`?). UTL-X avoids this by keeping arithmetic and string operations separate.

For complex string building, chain multiple `concat()` calls or use `join()`:

```

join([first, middle, last], " ") // "Alice M. Johnson"

```

14.5 Object Construction

Objects are created with curly braces. This is both the most common expression and the most powerful:

```

// Simple object
{name: "Alice", age: 30}

// Nested objects
{
  customer: {name: "Alice", city: "Amsterdam"},
  order: {total: 299.99, currency: "EUR"}
}

// Computed keys
let field = "fullName"
{[field]: concat($input.first, " ", $input.last)}
// produces: {"fullName": "Alice Johnson"}

// Spread: merge objects
let defaults = {currency: "EUR", country: "NL"}

```

```
{...defaults, name: $input.name, total: $input.total}
// produces: {currency: "EUR", country: "NL", name: "...", total: ...}
```

14.5.1 Conditional Properties

Include properties only when a condition is true by spreading a conditional object:

```
{
  name: $input.name,
  ...if ($input.discount != null) {discount: $input.discount} else {}
}
```

Or use the nullish coalescing pattern to provide defaults:

```
{
  name: $input.name,
  status: $input.status ?? "NEW",
  priority: $input.priority ?? "NORMAL"
}
```

14.6 Array Construction

Arrays are created with square brackets and processed with higher-order functions:

```
// Literal arrays
[1, 2, 3]
["Alice", "Bob", "Charlie"]

// Array from transformation
map($input.items, (item) -> item.name)
// produces: ["Widget", "Gadget", "Gizmo"]

// Combining arrays with spread
let a = [1, 2, 3]
let b = [4, 5, 6]
[...a, ...b]
// produces: [1, 2, 3, 4, 5, 6]

// Filtering
filter($input.items, (item) -> item.price > 100)
```

14.7 The Pipe Operator ($|>$)

The pipe operator passes the result of the left expression as the first argument to the right function. It makes data flow readable — left to right instead of nested inside-out:

```
// Without pipe – read from inside out
sort(unique(map($input.items, (item) -> item.category)))

// With pipe – read left to right
$input.items
  |> map((item) -> item.category)
  |> unique()
  |> sort()
```

Both produce the same result: a sorted list of unique categories. But the pipe version reads like a recipe: “take items, map to categories, remove duplicates, sort.”

Pipes chain naturally for multi-step processing:

```
$input.orders
  |> filter((o) -> o.status == "ACTIVE")
  |> map((o) -> {id: o.id, total: o.total})
  |> sortBy((o) -> o.total)
  |> map((o) -> concat(o.id, ": ", toString(o.total)))
```

14.8 Nullish Coalescing (??)

Provide a default value when an expression is null:

```
$input.nickname ?? $input.name ?? "Anonymous"
```

Evaluates left to right — the first non-null value wins. Essential for handling optional fields:

```
{
  currency: $input.currency ?? "EUR",
  country: $input.address?.country ?? "NL",
  discount: $input.promoCode ?? "NONE"
}
```

14.9 Safe Navigation (?.)

Access properties on potentially null values without throwing errors:

```
$input.order?.customer?.address?.city
// Returns null if any step is null – no error
```

Without `?.`, a null intermediate value causes a runtime error. With `?.`, null propagates silently.

Combine with `??` for defaults:

```
$input.order?.customer?.name ?? "Unknown Customer"
```

14.10 Operator Precedence

From highest to lowest precedence:

Precedence	Operators	Associativity
1 (highest)	Unary: <code>!</code> , <code>-</code>	Right
2	Exponentiation: <code>**</code>	Right
3	Multiplicative: <code>*</code> , <code>/</code> , <code>%</code>	Left
4	Additive: <code>+</code> , <code>-</code>	Left
5	Comparison: <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>	Left
6	Equality: <code>==</code> , <code>!=</code>	Left
7	Logical AND: <code>&&</code>	Left
8	Logical OR: <code> </code>	Left

9	Nullish coalescing: ??	Left
10	Ternary: ? :	Right
11	Pipe: >	Left
12 (lowest)	Conditional: if/else	Right

When in doubt, use parentheses. They make intent explicit and prevent precedence surprises:

```
// Ambiguous: does && bind tighter than ||?
a || b && c      // means: a || (b && c) – && is higher

// Clear: parentheses make intent obvious
(a || b) && c    // different meaning, explicit
```

14.11 Practical Expression Patterns

14.11.1 Price Calculation

```
let qty = toNumber($input.quantity)
let unit = toNumber($input.unitPrice)
let discount = toNumber($input.discount ?? "0") / 100
let net = qty * unit * (1 - discount)
let vat = net * 0.21
{
  net: round(net * 100) / 100,
  vat: round(vat * 100) / 100,
  gross: round((net + vat) * 100) / 100
}
```

14.11.2 Conditional Formatting

```
{
  status: if ($input.paid && $input.shipped) "COMPLETE"
        else if ($input.paid) "AWAITING_SHIPMENT"
        else "PENDING_PAYMENT",

  urgency: if ($input.daysOverdue > 30) "CRITICAL"
          else if ($input.daysOverdue > 7) "WARNING"
          else "NORMAL",

  label: concat(
    $input.firstName ?? "",
    if ($input.middleName != null) concat(" ", $input.middleName) else "",
    " ",
    $input.lastName ?? ""
  )
}
```

14.11.3 Data Normalization

```
{
  email: lowerCase(trim($input.email ?? "")),
  phone: replace(replace($input.phone ?? "", " ", ""), "- ", ""),
  country: upperCase($input.countryCode ?? "NL"),
}
```

```

    amount: abs(toNumber($input.amount ?? "0"))
  }

```

These patterns compose naturally — every sub-expression produces a value, and values flow through operators and functions without intermediate variables (unless you want readability via `let`).

14.12 Complete Operator Reference

All UTL-X operators grouped by category:

14.12.1 Arithmetic

Operator	Name	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division (always float)	10 / 3	3.333...
%	Modulo (remainder)	10 % 3	1
**	Exponentiation	2 ** 10	1024
- (prefix)	Unary negation	-42	-42

14.12.2 Comparison

Operator	Name	Example	Result
==	Equality	10 == 10	true
!=	Inequality	10 != 5	true
<	Less than	3 < 5	true
<=	Less or equal	5 <= 5	true
>	Greater than	10 > 5	true
>=	Greater or equal	10 >= 10	true

14.12.3 Logical

Operator	Name	Example	Result
&&	Logical AND (short-circuit)	true && false	false
	Logical OR (short-circuit)	true false	true
!	Logical NOT	!true	false

14.12.4 Null Handling

Operator	Name	Example	Result
??	Nullish coalescing	null ?? "default"	"default"
?.	Safe navigation	null?.name	null

14.12.5 Access and Navigation

Operator	Name	Example	Reference
.	Property access	obj.name	Chapter 8
[]	Index / dynamic key access	arr[0], obj[key]	Chapter 8

<code>.*</code>	Wildcard (all children)	<code>obj.*</code>	Chapter 8
<code>.@</code>	XML attribute access	<code>elem.@id</code>	Chapter 8
<code>.*@*</code>	All attributes	<code>elem.*@*</code>	Chapter 8
<code>.^</code>	Metadata access	<code>elem.^xsdPattern</code>	Chapter 8

14.12.6 Data Construction

Operator	Name	Example	Reference
<code>...</code>	Spread (objects and arrays)	<code>{...obj}, [...arr]</code>	Chapter 8
<code>[expr]:</code>	Computed property name	<code>{[key]: value}</code>	Chapter 8
<code>:</code>	Property definition	<code>{name: "Alice"}</code>	

14.12.7 Function and Flow

Operator	Name	Example	Reference
<code>-></code>	Lambda arrow (standard)	<code>(x) -> x * 2</code>	Chapter 15
<code>=></code>	Lambda arrow (fat arrow)	<code>x => x * 2</code>	Chapter 15
<code>\ ></code>	Pipe	<code>arr \ > filter(...)</code>	This chapter
<code>=</code>	Binding (in <code>let</code>)	<code>let x = 42</code>	Chapter 8

14.12.8 Reserved (Not Yet Implemented)

Operator	Name	Status
<code>..</code>	Recursive descent	Token defined, parser not implemented

15 Functions and Lambdas

Functions are the building blocks of UTL-X transformations. Lambdas — anonymous inline functions — are what make `map`, `filter`, `reduce`, and the entire functional programming model work. This chapter covers both, from basic syntax to advanced composition patterns.

15.1 Lambda Expressions

A lambda is an anonymous function — a piece of logic you pass to another function. If you've used arrow functions in JavaScript, closures in Kotlin, or lambdas in Python, you'll recognize the syntax immediately.

15.1.1 Basic Syntax

```
(parameter) -> expression
```

The parameter is on the left, the arrow `->` separates it from the body, and the body is a single expression that produces the result. No `return` keyword — the expression IS the return value.

```
// Double a number
(x) -> x * 2

// Extract a name
(user) -> user.name

// Format a greeting
(name) -> concat("Hello, ", name, "!")
```

UTL-X also accepts the fat arrow `=>` as an alternative to `->`. With `=>`, single-parameter lambdas can omit the parentheses:

```
// These are all equivalent:
(x) -> x * 2
(x) => x * 2
x => x * 2           // fat arrow allows omitting parentheses for single parameter
```

This book uses `->` as the standard convention. The `=>` syntax is provided for developers coming from JavaScript/TypeScript.

15.1.2 Multiple Parameters

```
(x, y) -> x + y
(item, index) -> {position: index, value: item}
(accumulator, current) -> accumulator + current.price
```

15.1.3 No Parameters

```
() -> "constant value"
() -> now()
```

15.1.4 Lambdas in Action

Lambdas are rarely written on their own — they're passed to higher-order functions:

```
// map: transform each element
map([1, 2, 3], (x) -> x * 2)
// [2, 4, 6]

// filter: keep elements that match
filter($input.users, (u) -> u.active == true)
// [{name: "Alice", active: true}, ...]

// reduce: accumulate a result
reduce([10, 20, 30], 0, (sum, x) -> sum + x)
// 60

// sortBy: order by a computed value
sortBy($input.products, (p) -> p.price)
// [{price: 9.99, ...}, {price: 29.99, ...}, ...]
```

15.2 User-Defined Functions

For logic you want to name and reuse within a transformation, use `function`:

```
function CalculateTax(amount, rate) {
  amount * rate / 100
}

function FormatCurrency(amount, currency) {
  concat(currency, " ", toString(round(amount * 100) / 100))
}

{
  subtotal: $input.total,
  tax: CalculateTax($input.total, 21),
  formatted: FormatCurrency($input.total * 1.21, "EUR")
}
```

15.2.1 The PascalCase Rule

User-defined function names **must** start with an uppercase letter (PascalCase). The parser enforces this:

```
function CalculateTax(amount, rate) { amount * rate / 100 } // ✓ valid
function calculateTax(amount, rate) { amount * rate / 100 } // ✗ parser error
```

If you try `function calculateTax(...)`, the parser rejects it with: *“User-defined functions must start with uppercase letter (PascalCase). Got: ‘calculateTax’. Try: ‘CalculateTax’.”*

Why? UTL-X has 652 built-in `stdlib` functions, all using `camelCase`: `map`, `filter`, `groupBy`, `parseDate`, `toNumber`. By requiring user-defined functions to start uppercase, there can never be a collision. When you see `CalculateTax(...)` in a transformation, you know immediately it’s user-defined. When you see `parseDate(...)`, you know it’s `stdlib`. No ambiguity, no shadowing, no surprises.

Name starts with	What it is	Example
lowercase	Built-in <code>stdlib</code> function	<code>map</code> , <code>filter</code> , <code>concat</code> , <code>parseDate</code>
Uppercase	User-defined function	<code>CalculateTax</code> , <code>FormatPhone</code> , <code>ValidateVAT</code>

The keyword `def` is accepted as a shorthand alias for `function`, but `function` is the standard keyword used throughout UTL-X.

15.2.2 Function Definitions Are Expressions

A function definition binds a name to a callable. These are equivalent:

```
// Named function
function Double(x) { x * 2 }

// Lambda bound to a variable (no naming restriction – lambdas are values)
let double = (x) -> x * 2
```

Both can be called as `Double(5)` or `double(5) → 10`. The function form is more readable for named functions and enforces PascalCase. The `let` form binds a lambda to a variable — variable names follow normal rules (no uppercase requirement).

15.2.3 Recursive Functions

Functions can call themselves:

```
function Factorial(n) {
  if (n <= 1) 1
  else n * Factorial(n - 1)
}

Factorial(5) // 120
```

Use recursion sparingly — deep recursion can exhaust the stack. For most data processing, `map`, `filter`, and `reduce` are better choices.

15.3 Higher-Order Functions

A higher-order function is a function that takes another function as an argument. Instead of telling the function *what value* to use, you tell it *what logic* to apply — and it applies that logic to each element, each key, or each step.

If you come from XSLT, this concept doesn't exist there — XSLT uses `xsl:for-each` and template matching instead. If you come from SQL, think of `WHERE` as a filter and `SELECT` as a map — but written as functions you can compose. If you come from JavaScript, Python, or Kotlin, you already know this pattern.

The core idea: **data stays in the array, logic is passed in as a lambda.**

```
// Traditional loop thinking: "go through each order, check if active, collect results"
// Higher-order thinking: "filter orders by active status"
filter($input.orders, (o) -> o.status == "ACTIVE")

// Traditional: "go through each item, calculate tax, build new list"
// Higher-order: "map items to items-with-tax"
map($input.items, (item) -> {
  name: item.name
  priceWithTax: item.price * 1.21
})
```

UTL-X's standard library includes these higher-order functions:

Function	Lambda returns	Result	Use case
<code>map</code>	new value	transformed array	Transform each element

<code>filter</code>	boolean	subset of array	Keep matching elements
<code>find</code>	boolean	first match or null	Find one element
<code>findIndex</code>	boolean	index or <code>-1</code>	Find position
<code>reduce</code>	new accumulator	single value	Sum, concatenate, aggregate
<code>sortBy</code>	comparable value	sorted array	Custom sort order
<code>groupBy</code>	key value	object of groups	Group by computed key
<code>mapGroups</code>	new value	transformed groups	Process each group
<code>any</code>	boolean	boolean	At least one matches?
<code>all</code>	boolean	boolean	Every element matches?
<code>none</code>	boolean	boolean	No element matches?
<code>flatMap</code>	array	flattened array	Map + flatten
<code>mapKeys</code>	new key	transformed object	Rename object keys
<code>mapValues</code>	new value	transformed object	Transform object values
<code>mapEntries</code>	new entry	transformed object	Transform both key and value
<code>filterEntries</code>	boolean	subset of object	Filter object properties
<code>lookupBy</code>	key value	enriched array	Enrich from reference data
<code>nestBy</code>	key value	nested array	Build parent-child hierarchy
<code>chunkBy</code>	group key	chunked array	Positional grouping

Any user-defined function can also be higher-order — if it accepts a function parameter and calls it:

```
// User-defined higher-order function
function ApplyToAll(items, transform) {
  map(items, (item) -> transform(item))
}

// Usage:
ApplyToAll($input.prices, (p) -> round(p * 1.21, 2))
```

This is standard functional programming — functions are values that can be passed around, stored in variables, and applied dynamically.

The essential higher-order functions are:

15.3.1 `map` — Transform Each Element

```
map(array, (element) -> newValue)
```

Produces a new array where each element is transformed:

```
map([1, 2, 3], (x) -> x * 2)
// [2, 4, 6]

map($input.employees, (emp) -> {
  fullName: concat(emp.firstName, " ", emp.lastName),
  salary: emp.salary
})
```

15.3.2 filter — Select Matching Elements

`filter` is one of the most-used functions — and one of the most misunderstood. It takes an array and a lambda that returns true or false for each element. Only elements where the lambda returns true are kept:

```
filter(array, (element) -> booleanCondition)
```

15.3.2.1 Basic Examples

```
// Keep orders over 100
filter($input.orders, (o) -> o.total > 100)

// Keep active admin users
filter($input.users, (u) -> u.role == "admin" && u.active)

// Keep non-null values
filter($input.items, (item) -> item != null)

// Keep strings containing "error"
filter($input.logLines, (line) -> contains(line, "error"))
```

15.3.2.2 Common Anti-Pattern: Array Predicate Syntax

If you've used XPath, jq, or JSONPath, you might try this:

```
// WRONG – this does NOT work in UTL-X:
$input.items[price > 10]
$input.orders[status == "ACTIVE"]
$input.users[age >= 18]
```

UTL-X does NOT support predicate filtering inside bracket notation. Square brackets are for **index access only** — `$input.items[0]` gets the first element, `$input.items[2]` gets the third.

The correct UTL-X way:

```
// CORRECT – use filter() with a lambda:
filter($input.items, (item) -> item.price > 10)
filter($input.orders, (o) -> o.status == "ACTIVE")
filter($input.users, (u) -> u.age >= 18)
```

Why doesn't UTL-X support `$input.items[price > 10]`? Because bracket notation serves one purpose (indexing), and adding predicate filtering would create ambiguity: is `items[x]` an index access (get element at position x) or a filter (keep elements where x is truthy)? UTL-X avoids this ambiguity by keeping indexing and filtering as separate, explicit operations.

15.3.2.3 Common Anti-Pattern: Filtering Then Counting

```
// INEFFICIENT – filtering twice:
let activeUsers = filter($input.users, (u) -> u.active)
let activeCount = count(filter($input.users, (u) -> u.active))

// BETTER – filter once, count the result:
let activeUsers = filter($input.users, (u) -> u.active)
let activeCount = count(activeUsers)
```

Always bind the filter result to a `let` variable if you use it more than once.

15.3.2.4 Common Anti-Pattern: Filter + Map When You Need Both

```
// COMMON MISTAKE – filtering and mapping separately, losing context:
let expensiveItems = filter($input.items, (i) -> i.price > 100)
let expensiveNames = map(expensiveItems, (i) -> i.name)
// Works, but you've lost the price information

// BETTER – map first (keeping what you need), then filter:
$input.items
  |> map((i) -> {name: i.name, price: i.price, category: i.category})
  |> filter((i) -> i.price > 100)

// OR – filter first, then map (more efficient if most items are filtered out):
$input.items
  |> filter((i) -> i.price > 100)
  |> map((i) -> {name: i.name, price: i.price, category: i.category})
```

The order (filter-then-map vs map-then-filter) depends on your goal:

- Filter first when you want to **reduce the dataset** before transforming (fewer items to map)
- Map first when the **filter condition depends on the transformed data**

15.3.2.5 Multi-Condition Filters

Combine conditions with `&&` (and) and `||` (or):

```
// AND: all conditions must be true
filter($input.products, (p) ->
  p.inStock && p.price < 50 && p.category == "Electronics"
)

// OR: any condition can be true
filter($input.events, (e) ->
  e.severity == "ERROR" || e.severity == "CRITICAL"
)

// Complex: combine AND and OR with parentheses
filter($input.orders, (o) ->
  o.status == "PENDING" && (o.total > 1000 || o.priority == "HIGH")
)
```

15.3.2.6 Negation: Exclude Instead of Include

```
// Keep everything EXCEPT cancelled orders
filter($input.orders, (o) -> o.status != "CANCELLED")

// Remove null/empty values
filter($input.tags, (tag) -> tag != null && tag != "")

// Exclude a list of values
let excluded = ["DRAFT", "CANCELLED", "ARCHIVED"]
filter($input.orders, (o) -> !contains(excluded, o.status))
```

15.3.2.7 Filter with Safe Navigation

When the filter property might not exist on every element:

```
// WRONG – crashes if an element doesn't have "address"
filter($input.customers, (c) -> c.address.country == "NL")

// CORRECT – safe navigation handles missing properties
filter($input.customers, (c) -> c.address?.country == "NL")

// ALSO CORRECT – explicit null check
filter($input.customers, (c) -> c.address != null && c.address.country == "NL")
```

15.3.2.8 Filter Returns an Array (Always)

filter always returns an array — even if zero or one elements match:

```
let matches = filter($input.items, (i) -> i.id == "X-001")
// matches is [] if nothing found
// matches is [{id: "X-001", ...}] if one found
// matches is [{...}, {...}] if multiple found (duplicates)

// If you want the FIRST match (not an array), use find():
let match = find($input.items, (i) -> i.id == "X-001")
// match is the object directly, or null if not found
```

This is a common confusion: filter returns [item] (array of one), find returns item (the object itself or null). Use filter when you expect multiple results, find when you expect one.

15.3.2.9 Lambda Return Types: Objects vs Bare Expressions

A subtle but important distinction: lambdas that return **objects** behave differently from lambdas that return **bare expressions**. This matters for filter, find, sortBy, and any/all — functions that expect a boolean or comparable value from the lambda.

```
// WRONG – returns an object, not a boolean:
filter($input.items, (x) -> {
  let threshold = 100
  result: x.price > threshold
})
// This does NOT filter! The lambda returns {"result": true} – an object.
// Any non-null object is truthy, so ALL items pass the filter.

// CORRECT – return a bare expression:
filter($input.items, (x) -> x.price > 100)

// CORRECT – with let, use let...in for bare expression return:
filter($input.items, (x) -> (let threshold = 100 in x.price > threshold))
```

The rule is simple:

- **map** — lambda returns an object → use { } with properties (this is the common case)
- **filter, find, any, all** — lambda returns a boolean → use a bare expression, NOT { }
- **sortBy** — lambda returns a comparable value → use a bare expression
- **reduce** — lambda returns the new accumulator → can be either, depending on what you accumulate

When you need let bindings in a filter or sortBy lambda, use the let...in form which returns a bare value:

```
// let...in returns the expression after "in" – not an object
filter($input.orders, (o) -> (
  let minDate = parseDate("2026-01-01", "yyyy-MM-dd")
  in isAfter(parseDate(o.date, "yyyy-MM-dd"), minDate)
))

sortBy($input.items, (item) -> (
  let score = item.rating * item.reviews
  in -score
))
```

15.3.3 reduce — Accumulate a Result

```
reduce(array, initialValue, (accumulator, element) -> newAccumulator)
```

Combines all elements into a single value:

```
// Sum
reduce([10, 20, 30], 0, (sum, x) -> sum + x)
// 60

// Build a comma-separated string
reduce(["Alice", "Bob", "Charlie"], "", (acc, name) ->
  if (acc == "") name else concat(acc, ", ", name)
)
// "Alice, Bob, Charlie"

// Find maximum
reduce($input.scores, 0, (max, s) -> if (s > max) s else max)
```

15.3.3.1 MapReduce in UTL-X

If you come from Hadoop, Spark, or streaming architectures, you know the MapReduce pattern: first **map** each element independently (parallelizable), then **reduce** the results into a single aggregate. UTL-X does not have a dedicated `mapReduce()` function — because `map()` piped into `reduce()` already IS mapReduce:

```
// MapReduce: compute total revenue from orders
$input.orders
  |> map((o) -> o.quantity * o.unitPrice) // map phase: extract line totals
  |> reduce(0, (acc, lineTotal) -> acc + lineTotal) // reduce phase: sum them
// Output: 4250.00
```

The pipe operator (`|>`) makes the two phases read naturally as a pipeline — map first, then reduce. This is the same data flow as a Hadoop job, a Spark RDD chain, or a Kafka Streams topology, but expressed in a single UTL-X expression.

For common aggregations, UTL-X provides pre-built map+reduce combinations that are shorter and more readable:

```
// These are all mapReduce under the hood:
sumBy($input.orders, (o) -> o.quantity * o.unitPrice) // sum
avgBy($input.orders, (o) -> o.total) // average
```



```
maxBy($input.orders, (o) -> o.total)           // max (returns the object)
countBy($input.orders, (o) -> o.status == "SHIPPED") // conditional count
```

When the built-in aggregations are not enough — when you need a custom accumulator shape — use the full `map |> reduce` pattern:

```
// Custom aggregation: group totals by currency
$input.orders |> reduce({}, (acc, order) -> {
  ...acc,
  [order.currency]: (acc[order.currency] ?? 0) + order.total
})
// Output: {"EUR": 3200, "USD": 1050, "GBP": 890}
```

15.3.4 find — First Match

```
find(array, (element) -> condition)
```

Returns the first element where the lambda returns true, or null if no match:

```
find($input.users, (u) -> u.email == "alice@example.com")
```

15.3.5 sortBy — Order by Computed Value

```
sortBy(array, (element) -> sortKey)
```

Sorts by the value returned by the lambda:

```
sortBy($input.employees, (e) -> e.lastName) // alphabetical
sortBy($input.products, (p) -> p.price)     // cheapest first
sortBy($input.orders, (p) -> -p.total)      // most expensive first (negate)
```

15.3.6 groupBy — Group by Key

```
groupBy(array, (element) -> groupKey)
```

Creates a map where keys are the lambda results and values are arrays of matching elements:

```
groupBy($input.employees, (e) -> e.department)
// {"Engineering": [...], "Sales": [...], "Marketing": [...]}
```

See Chapter 10 for how `groupBy` is used for flat-to-hierarchical transformation.

15.4 Function Composition

15.4.1 Pipe Chains

The pipe operator (`|>`) chains functions into readable data flows:

```
$input.transactions
|> filter((t) -> t.amount > 0)
|> map((t) -> {
```

```

    date: t.date,
    amount: t.amount,
    category: t.category ?? "Uncategorized"
  })
|> sortBy((t) -> t.date)
|> map((t) -> concat(t.date, ": ", toString(t.amount), " (", t.category, ")"))

```

Read top to bottom: filter positive transactions, map to a clean structure, sort by date, format as strings.

15.4.2 Nested Functions

When pipes aren't suitable, functions nest naturally:

```

join(
  sort(unique(map($input.items, (i) -> i.category))),
  ", "
)
// "Electronics, Furniture, Kitchen"

```

Read inside out: map to categories, remove duplicates, sort, join with commas.

Note: this is the *string* `join()` function (2 parameters: array + separator). For flat-to-hierarchical data restructuring (nesting children under parents by key), a separate function called `nestBy()` is available — see Chapter 10 and Chapter 21 (Data Restructuring). The names are deliberately different to avoid confusion: `join()` joins strings, `nestBy()` nests data.

15.4.3 Combining map and filter

A very common pattern — filter first, then transform:

```

$input.employees
|> filter((e) -> e.active && e.department == "Engineering")
|> map((e) -> {
  name: concat(e.firstName, " ", e.lastName),
  level: e.seniorityLevel,
  yearsOfService: dateDiff(now(), parseDate(e.startDate, "yyyy-MM-dd"), "years")
})

```

Or transform first, then filter (when the filter condition depends on the transformation):

```

$input.orders
|> map((o) -> {
  ...o,
  lineTotal: sum(map(o.lines, (l) -> l.qty * l.price))
})
|> filter((o) -> o.lineTotal > 1000)

```

15.5 Closures

Lambdas capture variables from their surrounding scope:

```

let threshold = 100
let currency = $input.settings.currency

map($input.items, (item) -> {

```

```

    name: item.name,
    price: item.price,
    currency: currency,                // captured from outer scope
    expensive: item.price > threshold // captured from outer scope
  })

```

The lambda “closes over” `threshold` and `currency` — it remembers their values even though they were defined outside the lambda. This is standard closure behavior, familiar from JavaScript, Kotlin, Python, and every functional language.

15.6 Practical Patterns

15.6.1 Flatten and Restructure

```

// Input: array of objects with nested arrays
// Output: flat array of all nested items with parent context

$input.departments
|> map((dept) ->
  map(dept.employees, (emp) -> {
    department: dept.name,
    employee: emp.name,
    salary: emp.salary
  })
)
|> flatten()

```

15.6.2 Aggregate with Multiple Results

```

let items = $input.order.items
{
  count: count(items),
  total: sum(map(items, (i) -> i.price * i.qty)),
  average: sum(map(items, (i) -> i.price)) / count(items),
  cheapest: sortBy(items, (i) -> i.price) |> first(),
  mostExpensive: sortBy(items, (i) -> -i.price) |> first(),
  categories: unique(map(items, (i) -> i.category))
}

```

15.6.3 Lookup Table Pattern

```

let countryNames = {
  "NL": "Netherlands",
  "DE": "Germany",
  "FR": "France",
  "BE": "Belgium"
}

map($input.addresses, (addr) -> {
  ...addr,
  countryName: countryNames[addr.countryCode] ?? addr.countryCode
})

```

15.6.4 Conditional Transformation

```
map($input.items, (item) ->
  if (item.type == "PRODUCT") {
    kind: "physical",
    name: item.name,
    weight: item.weight ?? 0,
    price: item.price
  } else if (item.type == "SERVICE") {
    kind: "digital",
    name: item.name,
    duration: item.hours ?? 1,
    rate: item.price
  } else {
    kind: "unknown",
    name: item.name,
    raw: item
  }
)
```

15.7 The 652 Standard Library Functions

The functions shown in this chapter — `map`, `filter`, `reduce`, `find`, `sortBy`, `groupBy`, `concat`, `sum`, `count`, `unique`, `flatten` — are just the beginning. UTL-X includes 652 functions across 18 categories.

The next chapter provides an overview organized by category. The complete reference with signatures and examples is in Part VIII (Chapter 50).

16 Standard Library Overview

UTL-X ships with 650+ built-in functions organized in 18 categories. This chapter provides a guided tour — what each category offers, when you’d use it, and the most important functions in each. The complete reference with signatures and examples for every function is in Part VIII (Chapter 50).

You don’t need to memorize 650+ functions. You need to know *what categories exist* so you can search for the right function when you need it. The IDE’s function library and `utlx functions --search` command make discovery easy.

16.1 String Functions (83 functions)

The largest category. Text manipulation is central to data transformation — field formatting, concatenation, parsing, validation.

16.1.1 Most-Used String Functions

Function	What it does	Example
<code>concat(...)</code>	Join strings	<code>concat("Hello", " ", name)</code>
<code>upperCase(s)</code>	UPPERCASE	<code>upperCase("hello") → "HELLO"</code>
<code>lowerCase(s)</code>	lowercase	<code>lowerCase("HELLO") → "hello"</code>
<code>trim(s)</code>	Remove whitespace	<code>trim(" hello ") → "hello"</code>
<code>contains(s, sub)</code>	Check substring	<code>contains("hello", "ell") → true</code>
<code>startsWith(s, prefix)</code>	Check prefix	<code>startsWith("NL123", "NL") → true</code>
<code>replace(s, old, new)</code>	Replace text	<code>replace("2026-04-30", "-", "/")</code>
<code>split(s, delim)</code>	Split to array	<code>split("a,b,c", ",") → ["a", "b", "c"]</code>
<code>substring(s, start, len)</code>	Extract part	<code>substring("hello", 0, 3) → "hel"</code>
<code>length(s)</code>	Character count	<code>length("hello") → 5</code>
<code>matches(s, regex)</code>	Regex test	<code>matches("abc123", "[a-z]+[0-9]+") → true</code>

16.1.2 Case Conversion

Beyond simple upper/lower, UTL-X supports naming convention conversions:

```
camelCase("order_line_item")    // "orderLineItem"
snakeCase("orderLineItem")      // "order_line_item"
kebabCase("orderLineItem")      // "order-line-item"
capitalize("hello world")       // "Hello world"
```

These are essential for API mapping — converting between Java-style camelCase, Python-style snake_case, and URL-style kebab-case.

16.1.3 Padding and Formatting

```
leftPad("42", 5, "0")           // "00042" (invoice numbers)
rightPad("EUR", 5, " ")         // "EUR  " (fixed-width output)
truncate("Long description...", 50) // first 50 chars
```

16.2 Array Functions (67 functions)

The second-largest category. Arrays are everywhere — order lines, user lists, CSV rows, XML repeated elements.

16.2.1 Transform and Filter

```
map(items, (i) -> i.name)           // transform each
filter(items, (i) -> i.active)       // keep matching
flatMap(items, (i) -> i.tags)        // map + flatten
```

16.2.2 Search and Access

```
find(items, (i) -> i.id == "X")      // first match or null
first(items)                          // first element
last(items)                          // last element
count(items)                         // length
contains(items, "value")              // is value in array?
```

16.2.3 Aggregate

```
sum(numbers)                         // add all
avg(numbers)                         // average
min(numbers)                         // smallest
max(numbers)                         // largest
reduce(items, init, (acc, x) -> ...) // custom aggregation
```

16.2.4 Reshape

```
sort(items)                          // natural order
sortBy(items, (i) -> i.name)          // sort by field
reverse(items)                       // reverse order
unique(items)                        // remove duplicates
flatten([[1,2],[3,4]])               // [1,2,3,4]
groupBy(items, (i) -> i.category)     // group into map
take(items, 5)                       // first 5
drop(items, 5)                       // skip first 5
slice(items, 2, 7)                   // elements 2-6
chunk(items, 3)                      // groups of 3
zip(array1, array2)                  // pair elements
```

16.3 Math Functions (37 functions)

Numeric operations for financial calculations, statistics, and data processing.

```
abs(-42)                            // 42
ceil(3.2)                           // 4
floor(3.8)                           // 3
round(3.456)                         // 3
round(3.456 * 100) / 100             // 3.46 (2 decimal places)
sqrt(144)                            // 12
pow(2, 10)                           // 1024
min(10, 20)                          // 10
max(10, 20)                          // 20
```

For financial precision, use `round()` explicitly — UTL-X uses floating-point arithmetic, which can produce rounding artifacts ($0.1 + 0.2 = 0.30000000000000004$). Always round currency amounts: `round(amount * 100) / 100`.

16.4 Date/Time Functions (68 functions)

The third-largest category. Date handling is critical in integration — every message has timestamps, due dates, and validity periods.

16.4.1 Current Time

```
now()           // 2026-04-30T10:30:00Z (with timezone)
today()         // 2026-04-30 (date only)
```

16.4.2 Parse and Format

```
parseDate("2026-04-30", "yyyy-MM-dd") // parse string to date
formatDate(date, "dd/MM/yyyy")         // format date to string
parseDate("30-04-2026", "dd-MM-yyyy")  // European format
formatDate(date, "MMMM d, yyyy")        // "April 30, 2026"
```

16.4.3 Arithmetic

```
addDays(date, 30)           // 30 days later
addMonths(date, 3)          // 3 months later
addYears(date, 1)           // next year
dateDiff(date1, date2, "days") // days between dates
```

16.4.4 Extract Components

```
year(date)      // 2026
month(date)     // 4
day(date)       // 30
hour(datetime)  // 10
minute(datetime) // 30
```

16.5 Type Functions (27 functions)

Check types and convert between them.

16.5.1 Type Checking

```
isString("hello") // true
isNumber(42)      // true
isBoolean(true)   // true
isArray([1,2])    // true
isObject({a: 1})  // true
isNull(null)      // true
getType("hello")  // "string"
```

16.5.2 Type Conversion

```
toString(42)      // "42"
toNumber("42")    // 42
toBoolean("true") // true
toNumber("not-a-num") // 0 (safe – returns 0 for unparseable input)
```

16.6 Encoding Functions (30 functions)

Encode and decode data for transport and storage.

```

base64Encode("hello")           // "aGVsbG8="
base64Decode("aGVsbG8=")       // "hello"
urlEncode("hello world")       // "hello%20world"
urlDecode("hello%20world")     // "hello world"
urlEncodeComponent("a=b&c=d")  // "a%3Db%26c%3Dd"
hexEncode("hello")             // "68656c6c6f"
hexDecode("68656c6c6f")       // "hello"

```

16.7 Security Functions (16 functions)

Hashing, HMAC, AES, RSA, and JWT/JWS functions. Part of the `stdlib/crypto` package.

```

md5("hello")                   // "5d41402abc4b2a76..."
sha256("hello")               // "2cf24dba5fb0a30e..."
hmacSha256("message", "secret-key") // HMAC signature

```

See Chapter 17 for security library details including encryption, digital signatures, and compliance considerations.

16.8 XML Functions (60 functions)

XML-specific operations for encoding, namespace handling, QName manipulation, and canonicalization.

```

detectXMLEncoding(xmlString)   // "UTF-8" or "ISO-8859-1"
convertXMLEncoding(xmlString, "UTF-8") // re-encode
stripBOM(data)                 // remove byte order mark

```

16.8.1 QName Functions (9)

For decomposing, constructing, and matching XML Qualified Names — essential for namespace-aware processing (UBL, XBRL, SOAP, ISO 20022). See Chapter 22 for usage examples.

Function	Returns	Example
<code>localName(element)</code>	Local name without prefix	"Envelope" from <code>soap:Envelope</code>
<code>namespacePrefix(element)</code>	Prefix only	"soap" from <code>soap:Envelope</code>
<code>qualifiedName(element)</code>	Full prefixed name	"soap:Envelope"
<code>namespaceUri(element)</code>	Namespace URI	"http://schemas.xmlsoap.org/..."
<code>hasNamespace(element)</code>	Boolean	true if element has a namespace
<code>getNamespaces(element)</code>	Map of prefix → URI	{"soap": "http://...", "wsa": "http://..."}
<code>createQname(local, uri, prefix?)</code>	QName object	Build QName from parts
<code>resolveQname(string, context)</code>	QName object	Resolve "cbc:ID" using namespace context
<code>matchesQname(element, pattern)</code>	Boolean	Match by string or structured QName

16.8.2 Canonicalization Functions (11)

W3C XML Canonicalization (C14N) for digital signatures, hashing, and round-trip verification. See Chapter 22 for details.


```

c14n(xml)                // W3C C14N 1.0 (no comments)
c14nWithComments(xml)    // C14N 1.0 with comments
excC14n(xml)             // Exclusive C14N (SOAP signatures)
c14nHash(xml)            // SHA-256 of canonical form
c14nEquals(xml1, xml2)    // semantic comparison
c14nFingerprint(xml)     // short fingerprint for logging

```

16.9 Other Categories

16.9.1 JSON Functions (6)

Parse and render JSON within transformations:

```

parseJson("{\"name\": \"Alice\"}")    // parse string to UDM
renderJson(object)                   // UDM to JSON string

```

16.9.2 CSV Functions (12)

Parse and render CSV:

```

parseCsv("name,age\nAlice,30")        // parse to array of objects
renderCsv(arrayOf0bjects)              // to CSV string

```

16.9.3 YAML Functions (22)

Parse and render YAML:

```

parseYaml("name: Alice\nage: 30")     // parse to UDM
renderYaml(object)                    // to YAML string

```

16.9.4 Financial Functions (16)

Currency formatting and financial calculations:

```

formatCurrency(1234.56, "EUR")         // "EUR 1,234.56"
parseAmount("1.234,56", "european")    // 1234.56

```

16.9.5 Geospatial Functions (8)

Distance calculations:

```

distance(52.37, 4.90, 48.86, 2.35)     // Amsterdam to Paris in km

```

16.9.6 Binary Functions (47)

Compression, byte manipulation:

```

compress(data, "gzip")                 // compress
decompress(data, "gzip")               // decompress

```

16.9.7 Utility Functions (27)

General-purpose helpers:

```
range(1, 10)           // [1, 2, 3, ..., 9]
repeat("ab", 3)        // "ababab"
coalesce(null, null, "found") // "found" (first non-null)
```

16.9.8 Object Functions (1)

```
keys(object)           // ["name", "age"]
values(object)          // ["Alice", 30]
```

16.10 Finding the Right Function

With 650+ functions, discovery matters more than memorization.

16.10.1 In the CLI

```
utlx functions --search "date"  # find date-related functions
utlx functions --module string  # list all string functions
utlx functions                  # list all categories
```

16.10.2 In the IDE

The function library panel in VS Code lets you browse by category and search by name or description. Click a function to see its signature and an example.

16.10.3 In This Book

Chapter 50 (Part VIII) contains the complete reference — every function with its signature, parameter types, return type, description, and a runnable example. Use it as a desk reference.

16.11 A Note on Naming

UTL-X uses camelCase for all function names: `upperCase`, `lowerCase`, `parseDate`, `formatDate`, `toNumber`, `groupBy`, `sortBy`. This is consistent and predictable — if you know the concept, you can guess the name.

Common mistake: `lowercase` (wrong) → `lowerCase` (correct). The IDE catches this immediately with a red underline.

17 Security Library and Message Security

Security in data transformation is often an afterthought — until an audit finds credit card numbers in log files or patient data crossing a network boundary unencrypted. This chapter covers UTL-X's security functions, the practical decisions around message encryption in integration flows, and what should (and absolutely should NOT) happen with sensitive data during transformation.

17.1 The Security Library (stdlib/crypto)

UTL-X's security functions live in the `crypto` package within the standard library (`stdlib/crypto`). These include hashing, HMAC, AES encryption, RSA signing, and JWT/JWS operations — all using JDK built-in cryptography with zero external dependencies.

17.1.1 Available Functions

Category	Functions	Use case
Hash	md5, sha1, sha256, sha512	Data integrity, checksums, fingerprinting
HMAC	hmacSha256, hmacSha512	Message authentication, API signing
Base64	base64Encode, base64Decode	Binary-to-text encoding
Hex	hexEncode, hexDecode	Byte representation
URL	urlEncode, urlDecode, urlEncodeComponent	URL-safe encoding

17.1.2 Hashing

Hash functions produce a fixed-size fingerprint from arbitrary input. They're one-way — you can't reverse a hash to get the original data.

```
sha256("sensitive-data")
// "6f2c7b22f1c4e3a8d9f..." (64 hex characters, always)

md5($input.document)
// "5d41402abc4b2a76..." (32 hex characters)
```

Common uses in integration:

- **Deduplication:** hash the message content, compare to previously seen hashes
- **Integrity verification:** hash before sending, hash after receiving, compare
- **Anonymization:** hash a customer ID to create a pseudonymous identifier
- **Cache keys:** hash the transformation source + input to cache compiled results

17.1.3 HMAC (Hash-based Message Authentication)

HMAC combines a hash with a secret key — proving both integrity AND authenticity:

```
hmacSha256("message-body", "shared-secret-key")
// "a1b2c3d4..."

// Verify: receiver computes the same HMAC with the same key
// If they match, the message hasn't been tampered with
```

Common uses:

- **API authentication:** sign requests for webhook verification (Stripe, GitHub, Shopify)
- **Message integrity:** ensure messages haven't been modified in transit
- **Token generation:** create short-lived authentication tokens

Note: UUID functions (`generateUuid`, `generateUuidV7`, etc.) are utility functions, not security functions. They live in the `core/util` packages — see Chapter 16.

17.2 Transport Security: mTLS

Before discussing message-level security, let's be clear about what's already handled:

Transport Layer Security (TLS) encrypts the connection between two endpoints. Every message UTL-X receives over HTTPS is encrypted in transit. The message content is decrypted at the TLS termination point (the container's ingress) and is in cleartext inside the container.

Mutual TLS (mTLS) goes further — both client AND server present certificates, proving identity in both directions. Azure Container Apps supports mTLS. Dapr sidecars use mTLS automatically between services in Kubernetes.

For most integration scenarios, TLS/mTLS is sufficient:

- Messages are encrypted between endpoints
- Identity is verified (who sent this message?)
- No additional encryption needed inside the trusted network

17.3 Message-Level Encryption

17.3.1 When It's Required

Some regulations require encryption of the message *payload* itself — not just the connection:

- **PCI DSS:** cardholder data must be encrypted at rest and in transit
- **HIPAA:** protected health information (PHI) must be encrypted
- **NEN 7510:** Dutch healthcare information security standard
- **Government classified data:** often requires end-to-end encryption

17.3.2 The Encrypt-Transform-Re-encrypt Pattern

When message-level encryption is required, the transformation flow becomes:

```
Encrypted message arrives
  ↓ decrypt (UTLX needs the decryption key)
  ↓ transform (cleartext in memory, briefly)
  ↓ encrypt (UTLX needs the encryption key for the target)
Encrypted message sent
```

This works but adds complexity:

- UTLX needs access to decryption AND encryption keys
- Key management: rotation, storage, access control
- Performance: decrypt + encrypt adds 1-5ms per message
- Debugging: encrypted messages cannot be inspected in logs or queues
- The message IS in cleartext in UTLX's memory during transformation — this is unavoidable

17.3.3 Why This Is a Maintenance Burden

Keys to manage:	per source × per target = $N \times M$ keys
Certificate lifecycle:	each expires, must be renewed
Key rotation:	planned rotations, emergency rotations
Access control:	who can access which keys?
Audit:	prove keys are handled correctly
Testing:	tests need test keys (not production keys)

For 5 source systems and 3 target systems: 15 key pairs to manage. Each with its own rotation schedule. This is why most integration platforms rely on transport encryption (TLS) and access controls — message-level encryption is reserved for regulated data that truly requires it.

17.3.4 Recommendation

Use **transport encryption (TLS/mTLS)** for all communication. Add **message-level encryption** only when regulation specifically requires it (PCI DSS, HIPAA, classified data). Don't encrypt "just in case" — the operational cost is significant and usually unnecessary.

17.4 Digital Signatures and Integrity

17.4.1 When You Need Signatures

Digital signatures prove two things: **integrity** (the message hasn't been modified) and **authenticity** (the message came from a known sender). They don't encrypt — they verify.

Common in:

- **Peppol e-invoicing:** UBL invoices are digitally signed with XAdES
- **Banking (ISO 20022):** payment messages signed for non-repudiation
- **API webhooks:** HMAC signature in HTTP header (simpler than PKI)
- **JWT tokens:** signed claims for API authentication

17.4.2 Impact on Transformation

Critical rule: if you transform a digitally signed message, the signature becomes invalid. The signature covers the exact bytes of the original — any change (even whitespace) breaks it.

The correct flow:

1. Verify the signature (before transformation)
2. Transform the content (signature is now invalid)
3. Re-sign the output (with the sender's key for the target)

This means UTLX must:

- Verify signatures on input (pre-validation step)
- Sign output (post-transformation step)
- Have access to the appropriate keys

17.4.3 What UTL-X Can Do Today

```
// HMAC verification (simple API webhooks)
let expectedSig = hmacSha256($input.body, "webhook-secret")
if (expectedSig != $input.headers.signature)
  error("Invalid webhook signature")

// Hash for integrity check
let contentHash = sha256(renderJson($input))
{...$input, integrityHash: contentHash}
```

17.4.4 What UTL-X Already Has

- **JWS/JWT:** decode, inspect, and extract claims from JWT and JWS tokens (decodeJWT, decodeJWS, getJWTClaim, getJWSHeader, getJWSPayload, isJWTExpired, and more)
- **AES encryption:** symmetric encrypt/decrypt with AES-128 and AES-256 (encryptAES, decryptAES, encryptAES256, decryptAES256, generateKey, generateIV)
- **Hashing:** SHA-256, SHA-512, SHA-1, SHA-384, SHA-224, SHA3, MD5, HMAC variants
- **Masking:** PII protection (mask, sha256 for irreversible hashing)

17.4.5 What UTL-X Cannot Do Yet (Future)

- **JWS signing and verification:** creating signed tokens and verifying signatures (decode/inspect is available, but signing requires private key management)
- **XMLDSig / XAdES:** XML digital signature verification — required for Peppol e-invoicing compliance

- **RSA sign/verify:** asymmetric cryptography for digital signatures
- **Key Vault integration:** retrieve keys and secrets from Azure Key Vault, GCP Secret Manager, AWS Secrets Manager

17.5 Large Files and Sensitive Data

17.5.1 The Claim Check Pattern

Sensitive large files (contracts, medical images, financial documents) should NOT travel through the message body. Use the **claim check pattern**:

1. Store the file in secure object storage (Azure Blob, S3, GCS) with encryption at rest
2. Put a **reference** in the message (URL, blob name, CMIS object ID)
3. UTL-X transforms the message metadata — not the file content
4. The consuming application retrieves the file from storage using the reference

```
// Transform the reference, not the file:
{
  invoiceId: $input.invoiceId,
  document: {
    storageType: "azure-blob",
    container: "invoices",
    blobName: concat($input.invoiceId, ".pdf"),
    url: concat("https://storage.blob.core.windows.net/invoices/", $input.invoiceId,
".pdf")
  }
}
```

17.5.2 Avoid Base64 Embedding

Embedding binary content as Base64 in messages is an anti-pattern for sensitive data:

- 33% size increase (Base64 overhead)
- The entire file is in UTL-X memory during transformation
- The file content may appear in logs, error messages, queue storage
- No access control — anyone with the message has the file

For files under 50 KB (small images, digital signatures), Base64 is acceptable. For anything larger or sensitive, use the claim check pattern.

17.6 What NOT to Log

This topic is covered in depth in Chapter 38 (Logging and Compliance). The short version:

Never log: credit card numbers (PCI DSS), patient data (HIPAA/GDPR), passwords, API keys, session tokens, private keys.

Always log: transformation ID, timestamp, duration, success/failure, error field names (not values).

UTL-X's default logging (INFO level) logs no message content — only operational metadata. DEBUG and TRACE levels can expose sensitive data and must NEVER be enabled in production with sensitive workloads.

18 Pattern Matching and Control Flow

Most transformation logic is straightforward: map fields, apply functions, construct objects. But real-world integration always has conditional paths — different output structures based on message type, different calculations based on country, different handling for errors.

This chapter covers UTL-X's control flow mechanisms: `match` expressions for multi-way branching, `try/catch` for error recovery, and practical patterns for routing and polymorphic data.

18.1 Match Expressions

`match` is UTL-X's multi-way branch — like `switch` in Java or `when` in Kotlin, but more powerful because it supports value matching, guard clauses, and variable binding.

18.1.1 Basic Value Matching

```
match ($input.status) {  
  "NEW" -> "Order received",  
  "PROCESSING" -> "In progress",  
  "SHIPPED" -> "On the way",  
  "DELIVERED" -> "Completed",  
  _ -> "Unknown status"  
}
```

The `_` is the wildcard — it matches anything. Always include it as the last case to handle unexpected values. Without a wildcard, an unmatched value produces null.

18.1.2 Matching with Guards

Add conditions after the pattern with `if`:

```
match ($input.amount) {  
  n if n > 10000 -> "high-value",  
  n if n > 1000 -> "medium-value",  
  n if n > 0 -> "standard",  
  0 -> "zero",  
  _ -> "negative"  
}
```

The variable `n` is bound to the matched value — you can use it in the guard condition and in the result expression.

18.1.3 Complex Matching

`Match` can branch on any expression, not just simple values:

```
match (getType($input.data)) {  
  "string" -> {type: "text", value: $input.data},  
  "number" -> {type: "numeric", value: $input.data},  
  "array" -> {type: "list", count: count($input.data)},  
  "object" -> {type: "record", keys: keys($input.data)},  
  _ -> {type: "unknown"}  
}
```

18.1.4 Match vs If/Else Chains

For two or three conditions, `if/else` is fine:

```
if ($input.country == "NL") "domestic"
else if ($input.country == "BE" || $input.country == "DE") "neighboring"
else "international"
```

For more than three conditions, `match` is clearer:

```
match ($input.country) {
  "NL" -> "domestic",
  "BE" -> "neighboring",
  "DE" -> "neighboring",
  "FR" -> "EU-west",
  "IT" -> "EU-south",
  "PL" -> "EU-east",
  _ -> "international"
}
```

The rule of thumb: **two branches** → **if/else**, **three or more** → **match**.

18.2 Try/Catch — Error Recovery

Not every input is clean. Dates might be unparseable. Numbers might be text. Fields might be missing. `try/catch` lets you handle errors gracefully instead of failing the entire transformation.

18.2.1 Basic Try/Catch

```
try {
  parseDate($input.dateString, "yyyy-MM-dd")
} catch (e) {
  today()    // fallback: use today's date if parsing fails
}
```

The `try` block contains the expression that might fail. If it succeeds, the result is used. If it throws an error, the `catch` block runs instead.

18.2.2 Using the Error Variable

The catch variable `e` contains the error message:

```
{
  result: try {
    toNumber($input.price) * toNumber($input.quantity)
  } catch (e) {
    0    // fallback to zero
  },
  error: try {
    toNumber($input.price) * toNumber($input.quantity)
    null    // no error
  } catch (e) {
    e    // capture the error message
  }
}
```

18.2.3 Try/Catch in Map

A common pattern — transform an array, handling errors per element instead of failing the entire batch:


```
map($input.records, (record) -> {
  id: record.id,
  amount: try { toNumber(record.amount) } catch (e) { 0 },
  date: try { parseDate(record.date, "yyyy-MM-dd") } catch (e) { null },
  valid: try { toNumber(record.amount); true } catch (e) { false }
})
```

Each record is processed independently. A bad date in record 5 doesn't prevent records 1-4 and 6+ from being transformed correctly.

18.2.4 When to Use Try/Catch vs Null Coalescing

```
// Use ?? when the value might be null/missing:
$input.discount ?? 0

// Use try/catch when the operation might FAIL:
try { parseDate($input.date, "yyyy-MM-dd") } catch (e) { null }
```

?? handles absence (null). try/catch handles failure (exceptions). Different problems, different tools.

18.3 Practical Control Flow Patterns

18.3.1 Message Type Routing

Integration messages often carry a type indicator. Route to different transformation logic based on type:

```
match ($input.messageType) {
  "ORDER" -> {
    type: "order",
    orderId: $input.data.orderId,
    customer: $input.data.customer,
    lines: $input.data.lines
  },
  "INVOICE" -> {
    type: "invoice",
    invoiceId: $input.data.invoiceId,
    total: $input.data.total,
    dueDate: $input.data.dueDate
  },
  "CREDIT_NOTE" -> {
    type: "credit",
    creditId: $input.data.creditId,
    amount: -abs($input.data.amount)
  },
  _ -> error(concat("Unknown message type: ", $input.messageType))
}
```

18.3.2 API Version Handling

Different API versions send different field names. Handle both:

```
{
  // v2 uses "fullName", v1 uses separate "firstName" + "lastName"
  name: if ($input.fullName != null) $input.fullName
        else concat($input.firstName ?? "", " ", $input.lastName ?? ""),
}
```

```
// v2 uses ISO date, v1 uses US format
date: try { parseDate($input.date, "yyyy-MM-dd") }
      catch (e) { try { parseDate($input.date, "MM/dd/yyyy") }
      catch (e2) { null } },

// v2 includes "status", v1 doesn't
status: $input.status ?? "ACTIVE"
}
```

18.3.3 Country-Specific Business Rules

```
let vatRate = match ($input.customer.country) {
  "NL" -> 21,
  "DE" -> 19,
  "FR" -> 20,
  "BE" -> 21,
  "LU" -> 17,
  "IE" -> 23,
  _ -> 0    // non-EU: reverse charge, 0% VAT
}

let isReverseCharge = vatRate == 0 && $input.customer.vatId != null

{
  subtotal: $input.total,
  vatRate: vatRate,
  vatAmount: if (isReverseCharge) 0 else $input.total * vatRate / 100,
  reverseCharge: isReverseCharge,
  total: if (isReverseCharge) $input.total else $input.total * (1 + vatRate / 100)
}
```

18.3.4 Data Cleansing with Fallbacks

```
{
  email: try {
    let e = lowerCase(trim($input.email))
    if (matches(e, "^[\\w.-]+@[\\w.-]+\\.([a-z]){2,}$")) e else null
  } catch (err) { null },

  phone: try {
    replace(replace(replace($input.phone ?? "", " ", ""), "-", ""), "+", "00")
  } catch (err) { null },

  amount: try {
    abs(toNumber(replace($input.amount ?? "0", ",", ".")))
  } catch (err) { 0 },

  country: upperCase(trim($input.countryCode ?? "")) ?? "XX"
}
```

18.3.5 Polymorphic Output (Different Structure Based on Input)

```
if (count($input.items) == 1) {
  // Single-item order: flat structure
  orderId: $input.orderId,
```

```

    product: $input.items[0].name,
    quantity: $input.items[0].qty,
    price: $input.items[0].price
  } else {
    // Multi-item order: nested structure with summary
    orderId: $input.orderId,
    itemCount: count($input.items),
    items: map($input.items, (i) -> {name: i.name, qty: i.qty, price: i.price}),
    total: sum(map($input.items, (i) -> i.qty * i.price))
  }

```

18.4 Control Flow Summary

Mechanism	Use when	Example
if/else	2-3 simple conditions	if (active) “yes” else “no”
ternary ? :	Inline condition	active ? “yes” : “no”
?? (nullish)	Default for missing value	name ?? “Unknown”
?. (safe nav)	Property might not exist	order?.customer?.name
match	3+ conditions or complex routing	match (type) { ... }
try/catch	Operation might fail	try { parseDate(...) } catch { null }
error()	Deliberately fail	error(“Invalid input”)

The key insight: in UTL-X, every control flow construct is an *expression* that produces a value. There are no statements. This means you can use `if/else`, `match`, and `try/catch` anywhere a value is expected — inside objects, as function arguments, in array elements, nested in other expressions.

19 Schema Validation

UTLXe engine feature. Runtime schema validation — validating input and output data against schemas during transformation — is a UTLXe production engine capability. The CLI (`utlx`) does not validate data payloads at runtime. This is a deliberate design decision — see “*Where Validation Lives — and Why*” below for the reasoning.

A transformation that produces wrong output is worse than one that fails — because wrong output is sent downstream, corrupts data in the target system, and the error is discovered hours or days later. Schema validation catches these errors at transformation time, before they leave UTL-X.

This chapter covers UTL-X’s validation capabilities: what schemas are supported, how the validation orchestrator works, and the difference between syntactical and semantic validation.

19.1 Where Validation Lives — and Why

The CLI does not validate data payloads at runtime. This is not a missing feature. It is a deliberate design decision.

19.1.1 The CLI Must Accept Anything

The real world is full of malformed messages — missing fields, wrong types, unexpected structures, legacy formats that don’t match their own documentation. The CLI’s job is to be a workhorse: read whatever comes in, transform it, produce output.

If the CLI rejected non-conforming input, it would be useless for the very situations where UTL-X is needed most — cleaning up, normalizing, and restructuring messy data. A transformation developer needs to *see* what the bad data looks like, not be told it’s invalid.

More fundamentally: **transformation IS the fix**. Often the whole point of writing the transformation is to handle the malformed data — add missing fields, fix types, normalize structures. If the CLI rejected the input, you couldn’t write the fix for it. The tool would prevent you from solving the problem it exists to solve.

And schemas lie. In practice, the “official” schema for an API or format is frequently outdated, incomplete, or aspirational. SAP IDoc documentation says a field is mandatory, but production IDocs arrive without it. An API’s JSON Schema says `required: ["address"]`, but 30% of real records have no address. The CLI must deal with reality, not the spec.

There is also a discovery workflow to consider. Before you can validate, you need to understand. A developer receiving an unknown IDoc or a partner’s XML for the first time needs the CLI to parse it, explore it with `$input.*`, and figure out the actual structure. Validation would block this discovery phase entirely.

19.1.2 UTLXe Operates in Middleware Where Validation Is Vital

The engine sits in production pipelines between systems — receiving orders, invoices, payments, patient records. Here, a malformed message that passes through silently can corrupt a downstream database, trigger a wrong payment, or violate regulatory compliance. The validation orchestrator (pre-validate → transform → post-validate) guarantees that only conforming data enters and leaves the pipeline.

Fail-fast saves money. A malformed invoice that passes through UTLXe unchecked might only be caught by the tax authority 30 days later — costing investigation time, penalties, and re-processing. Pre-validation catches it in milliseconds.

Auditability matters. In regulated industries (banking, healthcare, e-invoicing), you need proof that every message was validated. The validation orchestrator creates that audit trail. A CLI session leaves no trace.

And the blast radius is different. The CLI processes one message for one developer. UTLXe processes thousands per second for potentially millions of downstream consumers. The cost of letting a bad message through scales with the audience.

19.1.3 The Right Default in the Right Executable

This split mirrors how compilers work. A C compiler will compile code with warnings and let you run it. But a CI/CD pipeline enforces **-Werror** — same tool, different strictness depending on context. UTL-X makes this the default rather than a flag: you don’t need developers to remember to turn validation off, or operations teams to remember to turn it on. The right behavior is built into the right executable.

Aspect	CLI (utlx)	Engine (UTLXe)
Purpose	Development, exploration, testing	Production middleware
Input tolerance	Accept anything	Validate against schema
Output checking	Developer inspects visually	Post-validation enforces contract
Bad data	Let the developer see it and fix it	Reject it before it propagates
Schema role	Static script analysis (<code>utlx validate</code>)	Runtime data validation
Audience	One developer	Thousands of downstream consumers

19.2 Why Validate?

Consider a Peppol e-invoicing flow. Your transformation maps Dynamics 365 JSON to UBL 2.1 XML. Without validation:

- A missing `cbc:InvoiceTypeCode` element → Peppol Access Point rejects the invoice hours later
- A wrong VAT calculation → the tax authority flags the invoice weeks later
- An invalid IBAN → the payment fails days later

With validation:

- Pre-validation catches input data issues before the transformation runs
- Post-validation catches mapping bugs before the output leaves UTL-X
- The error message tells you exactly which field, which constraint, what’s wrong

Validation is the difference between “it compiled and ran” and “it produced correct output.”

19.3 Supported Schema Formats

UTL-X includes seven schema validators — one for each schema format:

Validator	Schema format	Validates	Standard
JsonSchemaValidator	JSON Schema	JSON data	Draft-04 through 2020-12
XsdValidator	XSD	XML data	W3C XSD 1.0 / 1.1
AvroSchemaValidator	Avro Schema	Avro-compatible data	Apache Avro 1.11
ProtobufValidator	Protobuf	Proto3-compatible data	Protocol Buffers 3
TableSchemaValidator	Table Schema (TSCH)	Tabular / CSV data	Frictionless Data
ODataSchemaValidator	OData Schema (OSCH)	OData entities	OASIS OData v4
YamlSchemaValidator	YAML via JSON Schema	YAML data	YAML validated as JSON

Each validator understands the schema format's native constraint language. JSON Schema's `required`, `pattern`, `minimum`. XSD's `minOccurs`, `xs:pattern`, `xs:restriction`. Avro's field defaults and unions. They're not wrappers — they validate against the real schema semantics.

19.4 The Validation Orchestrator

UTLXe's `ValidationOrchestrator` runs validation in a sandwich pattern around the transformation:

- | |
|---|
| 1. PRE-VALIDATION
Validate input data against input schema
→ Fail fast if input is invalid |
| 2. TRANSFORMATION
Execute the <code>.utlx</code> transformation |
| 3. POST-VALIDATION
Validate output data against output schema
→ Catch mapping bugs before output leaves |

If pre-validation fails, the transformation never runs — saving compute time and preventing cascading errors from bad input. If post-validation fails, the output is rejected — the mapping has a bug that needs fixing.

19.4.1 Configuring Validation

Validation is configured per transformation in the UTLXe engine. There are two ways to attach schemas:

Via the dynamic load API (protobuf/gRPC): The transport config map includes `validate_input`, `input_schema`, `input_schema_format` (and the same for output). This is the primary mechanism for runtime validation.

Via `transform.yaml` in a bundle:

```
# transform.yaml
strategy: TEMPLATE
validationPolicy: STRICT
inputs:
  - name: input
    schema: "order-input.json"
output:
  schema: "invoice-output.xsd"
```

Schema references can be declared inline in the `.utlx` header using the `{schema: "file"}` option:

```
%utlx 1.0
input json {schema: "order-input.json"}
output xml {schema: "invoice-output.xsd"}
---
// transformation body
```

This syntax is accepted by the parser and makes the `.utlx` file self-documenting — the schema contract is version-controlled alongside the transformation. The CLI ignores the schema option (it is a development tool — you see the output yourself). The UTLXe production engine uses it to validate input and output against the declared schemas at runtime.

19.4.2 CLI Static Validation

The CLI offers a separate validation capability — static analysis of a `.utlx` script against a schema:

```
utlx validate --schema order-output.json transformation.utlx
```

This checks whether the transformation’s *output structure* (inferred via type analysis) is compatible with the schema. It does NOT validate runtime data — it validates the script itself at compile time. Think of it as type checking: “will this transformation always produce valid output?” rather than “is this specific input valid?”

19.5 Validation Policies

Not every validation failure should stop the transformation. The policy controls what happens:

Policy	On failure	Use when
STRICT	Transformation fails with error	Production — contract enforcement
WARN	Warning logged, transformation continues	Migration — monitor before enforcing
SKIP	Schema loaded but not validated	Documentation only

Start with **WARN** during development (see what would fail). Switch to **STRICT** in production (enforce the contract).

19.6 JSON Schema Validation

JSON Schema is the most commonly used validator — it validates JSON, YAML, and OData data.

19.6.1 What JSON Schema Checks

```
{
  "type": "object",
  "required": ["orderId", "customer", "total"],
  "properties": {
    "orderId": {"type": "string", "pattern": "^ORD-[0-9]{3,}$"},
    "customer": {"type": "string", "minLength": 1},
    "total": {"type": "number", "minimum": 0},
    "currency": {"type": "string", "enum": ["EUR", "USD", "GBP"]},
    "lines": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["product", "qty"],
        "properties": {
          "product": {"type": "string"},
          "qty": {"type": "integer", "minimum": 1}
        }
      }
    }
  }
}
```

This schema enforces: `orderId` is required and matches a pattern, `customer` is a non-empty string, `total` is a non-negative number, `currency` is one of three values, `lines` is a non-empty array of objects with required `product` and `qty` fields.

19.6.2 Validation Errors

When validation fails, the error message pinpoints the problem:

```
Validation error at $.customer: required field missing
Validation error at $.total: -50 is less than minimum 0
Validation error at $.orderId: "INV-001" does not match pattern "^ORD-[0-9]{3,}$"
Validation error at $.lines[2].qty: "five" is not of type integer
```

Each error includes the JSON Path to the failing field, the constraint that was violated, and the actual value. This makes debugging fast — you know exactly what’s wrong and where.

19.7 XSD Validation

XSD validation is essential for enterprise XML — UBL invoices, HL7 messages, ISO 20022 payments all ship with XSD schemas.

```
%utlx 1.0
input json
output xml {schema: "UBL-Invoice-2.1.xsd"}
---
// If the output doesn't match the UBL XSD, transformation fails
```

XSD checks: element names, types, cardinality (minOccurs/maxOccurs), patterns, enumerations, namespace compliance. The JDK’s built-in `javax.xml.validation` handles XSD 1.0; XSD 1.1 support uses the same API.

19.8 Syntactical vs Semantic Validation

Everything described so far is **syntactical validation** — checking the structure, types, and constraints defined in a schema.

19.8.1 What Syntactical Validation Catches

- Is this field a string? A number? A date?
- Is this field present (required)?
- Does this string match a pattern (regex)?
- Is this number within range (min/max)?
- Is this array the right length?
- Does this element have the right children (XSD structure)?

19.8.2 What Syntactical Validation Cannot Catch

- Does the invoice total equal the sum of line items?
- If the country is NL, does the VAT ID start with “NL”?
- Is the end date after the start date?
- Is this IBAN valid (checksum)?
- Can an order move from SHIPPED back to PENDING?

These are **semantic** rules — they involve business logic, cross-field relationships, and domain knowledge. No structural schema can express them.

19.8.3 The Gap

UTL-X currently supports syntactical validation (7 schema validators). Semantic validation — business rules expressed in languages like Schematron, FEEL, or custom assertions — is a planned future capability. Chapter 40 covers the semantic validation vision in detail, including the proposed **assert** syntax:


```
// Future syntax (not yet implemented):
validate {
  assert sum($output.lines.total) == $output.grandTotal
  assert $output.issueDate < $output.dueDate
  assert if ($output.country == "NL") matches($output.vatId, "NL[0-9]{9}B[0-9]{2}")
}
```

For now, semantic rules must be expressed as part of the transformation logic — using `if/else`, `match`, or `try/catch` to check conditions and produce errors or warnings.

19.9 Validation in Practice

19.9.1 Development Workflow

1. Write your transformation
2. Add an output schema (JSON Schema or XSD)
3. Set policy to WARN
4. Run with sample data — check which fields fail validation
5. Fix the transformation until all warnings clear
6. Switch to STRICT for production

19.9.2 Production Workflow

1. Message arrives
2. Pre-validate against input schema → reject bad input immediately
3. Transform
4. Post-validate against output schema → catch mapping bugs
5. Send output — guaranteed to match the contract

19.9.3 When to Skip Validation

- **Performance-critical pipelines:** validation adds 1-5ms per message. At 86K msg/s, this matters. Skip validation when the input source is trusted and the transformation is well-tested.
- **Pass-through transformations:** when `\$input` is the entire body (no field mapping), validation of the output is the same as validation of the input — redundant.
- **Development/exploration:** when experimenting with data, validation errors slow you down. Use SKIP until the mapping stabilizes.

19.10 Validation and the Conformance Suite

The UTL-X conformance suite (500+ tests) validates transformation correctness — not schema compliance. These are different concerns:

- **Conformance tests:** “Does this transformation produce the expected output?” (functional correctness)
- **Schema validation:** “Does this output match the contract?” (structural compliance)

Both are important. Conformance tests catch regressions. Schema validation catches contract violations. Use both in production.

20 Pipeline Chaining and Multi-Hop Transformations

UTLX engine feature. Pipeline chaining is a production engine capability — it requires UTLX to orchestrate multi-step flows with in-memory UDM hand-off between stages. The CLI (`utlx`) executes single transformations. You can simulate pipelines on the CLI by chaining `utlx transform` calls with Unix pipes, but each step serializes and re-parses — there is no in-process UDM hand-off.

Most transformations are a single step: one input, one transformation, one output. But real-world integration flows are rarely that simple. An order arrives as XML, needs to be normalized, enriched with customer data from a CRM, validated against business rules, and finally rendered as a UBL invoice. Each of these concerns is different — and writing them as one monolithic transformation makes the code unreadable, untestable, and unmaintainable.

Pipeline chaining solves this by splitting complex flows into a sequence of small, focused transformations. Each step does one thing well. The output of step N becomes the input of step N+1. And critically: the UDM passes in-memory between steps — no serialization, no parsing, no overhead.

20.1 Single-Hop vs Multi-Hop

20.1.1 Single-Hop (Simple Transformation)

One input, one transformation, one output. The vast majority of transformations are single-hop:

```
cat order.xml | utlx transform order-to-json.utlx
```

Use single-hop when: the mapping is straightforward, no external enrichment is needed, and the entire logic fits comfortably in one file.

20.1.2 Multi-Hop (Pipeline)

Multiple transformation steps executed in sequence:

```
Input → Step 1 (normalize) → Step 2 (validate) → Step 3 (enrich) → Output
      UDM hand-off          UDM hand-off          UDM hand-off
      (no serialize)        (no serialize)        (no serialize)
```

The key advantage: NO serialization between steps. The UDM tree passes in-memory from one step to the next. A 5-step pipeline saves 10-25ms compared to HTTP calls between steps — because each serialization/deserialization cycle costs 2-5ms.

20.2 Why Pipelines?

20.2.1 Separation of Concerns

Each step does ONE thing:

- Step 1: Parse and normalize the incoming format
- Step 2: Validate against business rules
- Step 3: Enrich with lookup data
- Step 4: Transform to target format
- Step 5: Validate output against schema

Changing the enrichment logic doesn't touch the normalization. Swapping the output format doesn't affect validation. Each concern is isolated.

20.2.2 Reusability

A “validate Dutch VAT ID” step can be reused across every pipeline that processes Dutch invoices. Write it once, use it everywhere.

20.2.3 Testability

Each step can be tested independently with its own input and expected output. When a pipeline breaks, you can pinpoint exactly which step failed by inspecting the intermediate UDM between steps.

20.2.4 Maintainability

A 500-line monolithic transformation is hard to understand. Five 100-line steps with clear names (`normalize.utlx`, `validate-vat.utlx`, `enrich-customer.utlx`, `calculate-totals.utlx`, `render-ubl.utlx`) are self-documenting.

20.3 Multi-Input Transformations (Single Step)

Before discussing pipelines, it’s important to understand multi-input — a single transformation that receives data from multiple sources:

```
%utlx 1.0
input: orders json, customers json
output json
---
map($orders, (order) -> {
  ...order,
  customerName: find($customers, (c) -> c.id == order.customerId)?.name
})
```

Each input gets its own named variable (`$orders`, `$customers`). When using `stdin`, the engine parses the input as a JSON envelope and splits it by key name into separate named inputs:

```
{"orders": [...], "customers": [...]}
```

Multi-input is for a **single step** that needs multiple data sources. Pipelines are for **multiple steps** executed in sequence. They complement each other — a pipeline step can itself be a multi-input transformation.

20.4 Defining Pipelines

20.4.1 Pipeline via CLI (Unix Pipes)

The simplest way — chain transformations with the shell pipe:

```
cat order.xml \
| utlx transform normalize.utlx \
| utlx transform validate.utlx \
| utlx transform enrich.utlx --input customers=customers.json \
| utlx transform render-invoice.utlx
```

Each `utlx transform` reads from `stdin`, transforms, and writes to `stdout`. The shell connects them. This works but involves serialization between steps (JSON on the pipe).

20.4.2 Pipeline via UTLXe (In-Process)

UTLXe executes pipeline steps in-process with UDM hand-off — no serialization between steps:

```
{
  "steps": [
    {"transformationId": "normalize", "payload": "<Order>...</Order>"},
    {"transformationId": "validate"},
    {"transformationId": "enrich"},
    {"transformationId": "render-invoice"}
  ]
}
```

This is significantly faster for multi-step flows because the UDM tree passes directly from one step to the next in memory.

20.5 Pipeline Patterns

20.5.1 Enrichment Pipeline

The most common pattern. Each stage adds context:

```
Input → Normalize → Enrich (+ customer data) → Enrich (+ pricing) → Output
```

The order arrives with IDs. Each enrichment stage resolves an ID to its full record — customer ID to customer name, product code to product description, currency code to exchange rate.

20.5.2 Validation Sandwich

Catches errors at both ends:

```
Input → Pre-validate → Transform → Post-validate → Output
```

Pre-validation prevents wasted transformation work on bad input. Post-validation catches mapping bugs before the output leaves. This is the pattern that UTLXe's `ValidationOrchestrator` implements (Chapter 19).

20.5.3 Fan-Out

One input generates multiple outputs:

```
Input → Transform → Output A (invoice)
                      → Output B (delivery note)
                      → Output C (packing slip)
```

Same data, different output formats. Implemented as a pipeline that splits at the last stage into parallel transformations.

20.5.4 Fan-In (Aggregation)

Multiple inputs combined into one:

```
Input A (orders) ─┐
Input B (returns) → Merge → Transform → Output (report)
Input C (credits) ┘
```

Use multi-input transformation at the first stage to combine sources, then pipeline the merged data through subsequent steps.

20.5.5 Conditional Routing

Route messages to different transformation paths based on content:

```
Input → Classify → if (type == "ORDER") → Order pipeline
                  if (type == "INVOICE") → Invoice pipeline
                  if (type == "RETURN") → Return pipeline
```

The classify step inspects the message and routes it to the appropriate pipeline. One entry point, multiple processing paths.

20.6 Multi-Hop with Additional Inputs per Stage

20.6.1 The Problem

In real integration flows, later stages often need **ADDITIONAL** data that wasn't in the original input:

- Step 1: Parse incoming order (XML) — needs nothing extra
- Step 2: Validate — needs nothing extra
- Step 3: Enrich with customer data — needs a **customer lookup** (JSON from CRM API)
- Step 4: Calculate pricing — needs a **price list** (CSV from database)
- Step 5: Generate invoice — needs **template settings** (YAML config)

Each step needs the result of the previous step **PLUS** additional inputs from external sources. This is different from multi-input (where all data arrives at step 1) — here, the additional data is injected at specific pipeline stages.

20.6.2 Use Cases

Data enrichment: Order arrives with customer ID. Step 2 looks up the customer in the CRM. Step 3 looks up product details in the catalog. Step 4 applies tax rules from the tax table. Without per-stage inputs, you'd have to pre-join **ALL** reference data before step 1 — wasteful if validation fails in step 2.

Multi-system integration: A SAP IDoc arrives. Step 1 parses SAP format. Step 2 enriches with Dynamics 365 customer data (OData). Step 3 enriches with Salesforce opportunity data (JSON). Step 4 generates UBL XML for Peppol. Each step brings data from a different source system.

Configuration-driven output: The final step applies an output template — a YAML config defining layout, branding, and language. Same data, different templates produce different outputs (invoice, delivery note, packing slip). The template is an additional input to the final stage, not to the entire pipeline.

20.6.3 Status

- **Single-step multi-input:** built and working
- **Pipeline chaining (sequential steps):** built and working
- **Additional inputs per pipeline hop:** planned (EF01 — not yet implemented)
- **Current workaround:** pre-join all data before the pipeline, or use multi-input at the first step

20.7 Performance Characteristics

Characteristic	Value
Inter-step overhead	Zero — UDM hand-off in memory (no serialize/parse)
Savings vs HTTP calls	10-25ms for a 5-step pipeline (2-5ms per step saved)
Message concurrency	One worker handles all steps of one message sequentially
Pipeline concurrency	N workers = N messages flowing through the pipeline simultaneously
Back-pressure	Controlled throughput — workers block when downstream is slow

Pipeline stages are sequential for a single message — step 2 waits for step 1 to finish. But different messages are processed concurrently across workers. With 32 workers, 32 messages flow through the pipeline simultaneously, each at its own stage.

The trade-off is latency vs throughput: pipeline latency equals the sum of all step latencies (sequential), but pipeline throughput equals one message completion per average-step-time (pipelined across workers).

21 Data Restructuring: Flat, Hierarchical, and Back

Real-world integration is rarely a simple field-to-field mapping. The source data is structured one way, the target expects it another way. Orders arrive flat, the target wants them nested. Nested data needs to become CSV. Sequential segments need grouping by header. A customer ID needs to become a customer name.

This chapter covers the complete toolkit for restructuring data in UTL-X: from flat to hierarchical, from hierarchical to flat, and everything in between.

21.1 The Six Restructuring Patterns

Pattern	Direction	Function
Parent-child nesting	Flat → Hierarchical	<code>nestBy()</code>
Record enrichment	Flat → Enriched	<code>lookupBy()</code>
Sequential grouping	Flat → Grouped	<code>chunkBy()</code>
Denormalization	Hierarchical → Flat	<code>unnest()</code>
Key-based grouping	Flat → Map	<code>groupBy()</code>
Array flattening	Nested → Flat	<code>flatten()</code>

21.2 Pattern 1: Flat → Hierarchical (join)

The most common integration challenge: you receive flat records with key references, and the target expects nested parent-child structures.

21.2.1 The Problem

Orders and order lines arrive as separate flat arrays. Each line references its parent order by key:

```
{
  "orders":      [{"orderId": "A", "customer": "Acme"}, {"orderId": "B", "customer":
"Globex"}],
  "orderLines": [{"orderId": "A", "product": "Widget"}, {"orderId": "A", "product":
"Gadget"}, {"orderId": "B", "product": "Gizmo"}]
}
```

Target expects nested:

```
[{"orderId": "A", "customer": "Acme", "lines": [{"product": "Widget"}, {"product":
"Gadget"}]},
 {"orderId": "B", "customer": "Globex", "lines": [{"product": "Gizmo"}]}]
```

21.2.2 The Solution: `nestBy()`

```
nestBy(
  $input.orders,           // parent array
  $input.orderLines,       // child array
  (order) -> order.orderId, // parent key
  (line) -> line.orderId,   // child key
  "lines"                  // property name for nested children
)
```

The name reads naturally: “nest orderLines by orderId into a property called lines.” The 5th parameter “lines” is a string that tells `nestBy()` what to name the new property it creates on each parent. After this call, each order has a `.lines` property containing its matched order lines:

```
let enrichedOrders = nestBy(...)

// Now use order.lines like any property:
map(enrichedOrders, (order) -> {
  orderId: order.orderId,
  lineCount: count(order.lines),
  total: sum(map(order.lines, (l) -> l.qty * l.price))
})
```

21.2.3 Today’s Workaround (without join)

Until `nestBy()` is implemented, use `filter()` per parent:

```
map($input.orders, (order) -> {
  orderId: order.orderId,
  customer: order.customer,
  lines: filter($input.orderLines, (l) -> l.orderId == order.orderId)
})
```

This works but is $O(N \text{ times } M)$ — for 500 orders and 5,000 lines, that’s 2.5 million comparisons. `nestBy()` does it in 5,500 operations.

21.3 Pattern 2: Record Enrichment (lookupBy)

You have a record with a reference key and need to add fields from a lookup table — not nest the entire matching record, just extract specific fields.

21.3.1 The Problem

An order has `customerId` but needs `customerName`:

```
{"orderId": "ORD-001", "customerId": "C-42", "total": 299.99}
```

Customer table:

```
[{"id": "C-42", "name": "Acme Corp", "country": "NL"}]
```

You want: `{orderId: "ORD-001", customerName: "Acme Corp", country: "NL"}` — not a nested customer object.

21.3.2 The Solution: lookupBy()

```
let customer = lookupBy(order.customerId, $input.customers, (c) -> c.id)

{
  orderId: order.orderId,
  total: order.total,
  customerName: customer?.name ?? "Unknown",
  country: customer?.country
}
```


`lookupBy()` finds the first matching record and returns it. You then pick the fields you need with dot notation. Use `?.` for safety in case no match is found.

21.3.3 Today's Workaround (without `lookupBy`)

Use `find()`:

```
let customer = find($input.customers, (c) -> c.id == order.customerId)
```

Identical result. `lookupBy()` adds the potential for internal caching when used repeatedly inside `map()`.

21.4 Pattern 3: Sequential Grouping (`chunkBy`)

For data where parent-child is determined by position, not by key fields.

21.4.1 The Problem

SAP IDoc segments arrive in sequence — headers and lines interleaved:

```
E1EDK01 (header)    ← order 1 starts
E1EDP01 (line)      ← belongs to order 1
E1EDP01 (line)      ← belongs to order 1
E1EDK01 (header)    ← order 2 starts
E1EDP01 (line)      ← belongs to order 2
```

No key connects lines to headers. Position is the only relationship: “everything between one header and the next belongs together.”

21.4.2 The Solution: `chunkBy`()

```
let groups = chunkBy($input.segments, (seg) -> seg.type == "E1EDK01")

map(groups, (chunk) -> {
  header: chunk[0],
  lines: filter(drop(chunk, 1), (seg) -> seg.type == "E1EDP01")
})
```

`chunkBy()` splits the flat sequence into groups. A new group starts whenever the lambda returns true. Each group is an array where the first element is the header and the rest are its children.

21.4.3 Today's Workaround (without `chunkBy`)

Use `reduce()` with manual state tracking — possible but extremely verbose and error-prone. `chunkBy()` turns 15 lines of reduce logic into a single function call.

21.5 Pattern 4: Denormalization (`unnest`)

The reverse of `nestBy()` — take nested data and flatten it. Essential for producing CSV, database rows, or flat file output.

21.5.1 The Problem

Hierarchical order data:

```
[{"orderId": "A", "customer": "Acme", "lines": [
  {"product": "Widget", "qty": 10},
  {"product": "Gadget", "qty": 5}
]}
```

Target is flat CSV:

```
orderId,customer,product,qty
A,Acme,Widget,10
A,Acme,Gadget,5
```

Each parent field must be repeated for every child row.

21.5.2 The Solution: `unnest()`

```
%utlx 1.0
input json
output csv
---
unnest($input.orders, "lines")
```

One function call. The "lines" parameter tells `unnest()` which nested array to expand. Parent fields (orderId, customer) are automatically repeated for each child. The `lines` property itself is removed from the output.

For multi-level flattening, chain:

```
$input.orders |> unnest("lines") |> unnest("schedules")
```

21.5.3 Today's Workaround (without `unnest`)

```
flatten(map($input.orders, (order) ->
  map(order.lines, (line) -> {
    orderId: order.orderId,
    customer: order.customer,
    ...line
  })
))
```

Works but requires manually listing parent fields or using `spread` (which includes the `lines` array itself, causing unwanted nesting in the output).

21.6 Pattern 5: Key-Based Grouping (`groupBy` — exists)

Group flat records into a map by key. Already available in UTL-X.

```
groupBy($input.employees, (e) -> e.department)
// {"Engineering": [{...}, {...}], "Sales": [{...}], "Marketing": [{...}]}
```

Use `groupBy` when you want a map (key → array) rather than a nested hierarchy. Common for: aggregation per group, partitioning data by category, building lookup indexes.

`groupBy()` returns an Object — use key-based lookup for O(1) access: `groups[order.orderId]`.

21.7 Pattern 6: Array Flattening (`flatten` — exists)

Unwrap one level of nested arrays. Already available in UTL-X.

```
flatten([[1, 2], [3, 4], [5, 6]])
// [1, 2, 3, 4, 5, 6]
```

Use `flatten` when `map()` produces arrays and you want a single flat array instead of an array of arrays. Common pattern: `map + flatten = flatMap`.

21.8 Combining Patterns

Real integrations often combine multiple patterns:

21.8.1 IDoc → Enriched Hierarchical JSON

```
// 1. Group sequential segments (chunkBy)
let orderGroups = chunkBy($input.segments, (seg) -> seg.type == "E1EDK01")

// 2. Structure into parent-child (map)
let orders = map(orderGroups, (chunk) -> {
  ...chunk[0],
  lines: filter(drop(chunk, 1), (seg) -> seg.type == "E1EDP01")
})

// 3. Enrich with customer data (lookupBy)
map(orders, (order) -> {
  let customer = lookupBy(order.KUNNR, $input.customers, (c) -> c.id)

  orderId: order.BELNR,
  customer: customer?.name ?? "Unknown",
  country: customer?.country,
  lines: map(order.lines, (line) -> {
    product: line.MATNR,
    quantity: toNumber(line.MENGE)
  })
})
```

Three patterns combined: `chunkBy` (sequential → grouped), `map` (grouped → hierarchical), `lookupBy` (enrichment).

21.8.2 Hierarchical → Flat CSV Report

```
// 1. Unnest orders → lines
let flat = unnest($input.orders, "lines")

// 2. Enrich with product descriptions (lookupBy)
map(flat, (row) -> {
  let product = lookupBy(row.productCode, $input.catalog, (p) -> p.code)

  ...row,
  productDescription: product?.description ?? row.productCode,
  weight: product?.weight ?? 0
})

// Output as CSV
```

Two patterns: `unnest` (hierarchical → flat), `lookupBy` (enrichment).

21.9 Choosing the Right Pattern

I have / I need	Use
Flat records with keys → nested parent-child	nestBy()
Record with ID → add fields from reference table	lookupBy() or find()
Sequential segments → groups by header	chunkBy()
Nested data → flat rows for CSV/database	unnest()
Flat records → map of key → array	groupBy()
Array of arrays → single flat array	flatten()
Conditional children → nest only matching	filter() then nestBy()
Many-to-many via bridge table	Two nestBy() calls through bridge
Composite key matching	concat() key then nestBy() or lookupBy()

21.10 Naming Convention: Why Some Functions End in “By”

The restructuring functions follow a consistent naming convention. Functions that take a **lambda** for key extraction use the **By** suffix — “do X **by** this key function.” Functions that take a plain value (string, property name) do not.

Function	Takes lambda?	By suffix?	Rationale
groupBy()	Yes	Yes	Group <i>by</i> key function
sortBy()	Yes	Yes	Sort <i>by</i> key function
nestBy()	Yes	Yes	Nest children <i>by</i> key function
chunkBy()	Yes	Yes	Chunk <i>by</i> predicate function
lookupBy()	Yes	Yes	Look up <i>by</i> key function
unnest()	No (string)	No	Unnest a named property — no key function
flatten()	No	No	Flatten arrays — no key function

Part III

Formats Deep Dive

Working with XML, JSON, CSV, YAML, OData, and schema formats

22 XML Transformations

XML is the most feature-rich format UTL-X handles — and the most common in enterprise integration. SOAP services, UBL invoices, HL7 healthcare messages, SAP IDocs, SWIFT payments, government data — XML dominates the enterprise world.

This chapter covers everything specific to XML: how to read elements and attributes, how to write XML output with namespaces, how the parser handles edge cases, and real-world patterns from Peppol, FHIR, and SWIFT.

22.1 Reading XML

22.1.1 Element Access

XML elements map to UDM Object properties. Access them with dot notation:

```
<Order>
  <Customer>Alice Johnson</Customer>
  <Total>299.99</Total>
</Order>
```

```
$input.Order.Customer    // "Alice Johnson"
$input.Order.Total       // 299.99 (auto-detected as number)
```

UTL-X auto-unwraps text content — you get the value directly, not a wrapper object. This behavior is described in Chapter 10.

22.1.2 Attribute Access

XML attributes use the `@` prefix:

```
<Product id="P-001" price="29.99" currency="EUR">Widget</Product>
```

```
$input.Product.@id       // "P-001"
$input.Product.@price    // "29.99" (string – attributes are always strings)
$input.Product.@currency // "EUR"
$input.Product           // "Widget" (text content, auto-unwrapped)
```

Note: attribute values are always strings. Use `toNumber()` for numeric attributes:

```
toNumber($input.Product.@price) // 29.99 (number)
```

22.1.3 Repeated Elements

When an XML element appears multiple times, UTL-X automatically creates an array:

```
<Items>
  <Item>Widget</Item>
  <Item>Gadget</Item>
  <Item>Gizmo</Item>
</Items>
```

```
$input.Items.Item           // ["Widget", "Gadget", "Gizmo"] (array)
$input.Items.Item[0]        // "Widget"
count($input.Items.Item)    // 3
```

22.1.4 The Single-Element Problem

If there's only ONE Item, it's parsed as a single object — not an array of one:

```
<Items>
  <Item>Widget</Item>
</Items>
```

```
$input.Items.Item           // "Widget" (NOT ["Widget"])
```

This can break code that expects an array. Solution: use array hints or always wrap with `flatten(...)`:

```
// Safe: always treat as array
let items = if (isArray($input.Items.Item)) $input.Items.Item else [$input.Items.Item]
map(items, (i) -> ...)
```

The XML parser supports array hints — element names that should always be treated as arrays, even with a single occurrence. This is configured at the engine level.

22.1.5 Nested Access

Deep access chains naturally:

```
$input.Invoice.AccountingSupplierParty.Party.PartyName.Name
$input.Bundle.entry[0].resource.Patient.name[0].family
```

Use safe navigation for optional elements:

```
$input.Invoice?.AccountingCustomerParty?.Party?.Contact?.ElectronicMail
```

22.1.6 Namespace-Prefixed Elements

XML with namespace prefixes:

```
<cac:AccountingSupplierParty>
  <cac:Party>
    <cbc:Name>Windmill Trading B.V.</cbc:Name>
  </cac:Party>
</cac:AccountingSupplierParty>
```

Access with the full prefixed name:

```
$input["cac:AccountingSupplierParty"]["cac:Party"]["cbc:Name"]
```

Bracket notation is required because `:` is not valid in dot notation. This is common in UBL, FHIR, and XBRL documents.

22.2 Writing XML

22.2.1 Basic Element Output

When output `xml` is declared, objects become elements:

```
%utlx 1.0
input json
output xml
---
{
  Order: {
    Customer: $input.name,
    Total: $input.amount
  }
}
```

Produces:

```
<?xml version="1.0" encoding="UTF-8"?>
<Order>
  <Customer>Alice Johnson</Customer>
  <Total>299.99</Total>
</Order>
```

22.2.2 Creating Attributes

Use `\@` prefix in property names:

```
{
  Product: {
    @id: "P-001",
    @currency: "EUR",
    _text: $input.description
  }
}
```

Produces:

```
<Product id="P-001" currency="EUR">Widget</Product>
```

The `_text` property sets the text content when the element also has attributes. Without attributes, text content is set directly: `{Product: "Widget"}`.

22.2.3 Self-Closing Elements

Empty objects with attributes produce self-closing elements:

```
{Item: {@sku: "W-01", @price: "50.00", @qty: "2"}}
```

Produces:

```
<Item sku="W-01" price="50.00" qty="2"/>
```


22.2.4 Namespace Declarations

Add XML namespace declarations as attributes:

```
{
  Invoice: {
    "@xmlns": "urn:oasis:names:specification:ubl:schema:xsd:Invoice-2",
    "@xmlns:cbc": "urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2",
    "cbc:ID": $input.invoiceNumber,
    "cbc:IssueDate": $input.date
  }
}
```

Produces:

```
<Invoice xmlns="urn:oasis:names:specification:ubl:schema:xsd:Invoice-2"
  xmlns:cbc="urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2">
  <cbc:ID>INV-2026-001</cbc:ID>
  <cbc:IssueDate>2026-04-30</cbc:IssueDate>
</Invoice>
```

22.2.5 Encoding

Control output encoding:

```
output xml {encoding: "UTF-8"} // default
output xml {encoding: "ISO-8859-1"} // legacy systems
```

22.3 XML Encoding Detection and Conversion

UTL-X can detect and convert between XML encodings:

```
detectXMLEncoding($input) // "UTF-8" or "ISO-8859-1"
convertXMLEncoding($input, "UTF-8") // re-encode to UTF-8
```

This is essential when integrating with legacy systems that use ISO-8859-1 or Windows-1252. UTL-X reads any encoding and outputs in the declared encoding.

22.4 XML Canonicalization (C14N)

XML canonicalization produces a deterministic, normalized representation of an XML document. Two XML documents that are semantically identical but differ in whitespace, attribute order, or namespace declarations will produce the same canonical form. This is essential for XML digital signatures (XMLD-Sig), change detection, and round-trip verification.

UTL-X implements the W3C XML Canonicalization standards:

```
// Standard C14N (W3C 1.0) – sort attributes, normalize whitespace, remove comments
c14n($input)

// C14N with comments preserved
c14nWithComments($input)

// Exclusive C14N – only includes namespaces actually used (ideal for SOAP signatures)
excC14n($input)
excC14nWithComments($input)
```

```
// C14N 1.1 – handles XML 1.1 features
c14n11($input)
c14n11WithComments($input)
```

22.4.1 Hashing and Comparison

For verifying that a transformation preserves XML content — or detecting changes:

```
// Hash the canonical form (default SHA-256)
c14nHash($input)           // "a1b2c3d4..."
c14nHash($input, "SHA-512") // different algorithm

// Compare two XML documents semantically (ignoring attribute order, whitespace)
c14nEquals(xml1, xml2)      // true if same canonical form

// Generate a fingerprint (shorter than full hash, useful for logging)
c14nFingerprint($input)
```

22.4.2 Round-Trip Verification

When transforming XML → JSON → XML, you can verify that no data was lost:

```
// Original XML
let original = $input

// Transform to JSON and back
let asJson = renderJson($input)
let backToXml = parse(asJson, "json")

// Compare canonical forms
c14nEquals(original, backToXml) // true if round-trip is lossless
```

This is particularly valuable in compliance scenarios (e-invoicing, healthcare) where you must prove that your transformation pipeline does not alter the business content.

22.5 QName Functions (Qualified Names)

XML uses Qualified Names (QNames) to identify elements and attributes within namespaces. A QName like `cbc:InvoiceTypeCode` combines a prefix (`cbc`), a local name (`InvoiceTypeCode`), and an implied namespace URI (`urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2`).

UTL-X provides 9 functions for decomposing, constructing, and matching QNames:

```
// Decompose a QName
localName($input.Invoice)           // "Invoice" (without prefix)
namespacePrefix($input.Invoice)     // "cbc" (prefix only)
qualifiedName($input.Invoice)       // "cbc:Invoice" (full prefixed name)
namespaceUri($input.Invoice)        // "urn:oasis:names:..." (namespace URI)

// Check and list namespaces
hasNamespace($input.Invoice)        // true
getNamespaces($input.Invoice)       // {"cbc": "urn:oasis:...", "cac": "urn:oasis:..."}

// Construct a QName
createQName("Invoice", "urn:oasis:names:...", "cbc")
```

```
// → {localName: "Invoice", namespaceUri: "urn:oasis:...", prefix: "cbc", qualifiedName:
"cbc:Invoice"}

// Resolve a prefixed name using namespace context
resolveQName("cbc:Invoice", $input)
// → {localName: "Invoice", prefix: "cbc", namespaceUri: "urn:oasis:...", qualifiedName:
"cbc:Invoice"}

// Match by QName (string or structured)
matchesQName($input.Invoice, "cbc:Invoice") // true
```

These functions are essential for enterprise XML formats where namespace awareness determines correctness — you cannot just compare element names as strings when the same local name appears in different namespaces.

22.5.1 XBRL (Financial Reporting)

XBRL (eXtensible Business Reporting Language) is the most namespace-intensive XML format in production use. Every financial fact is identified by a QName — the namespace tells you which taxonomy (US-GAAP, IFRS, local GAAP) defines the concept, and the local name identifies the concept itself:

```
<xbrli:xbrl xmlns:us-gaap="http://fasb.org/us-gaap/2024"
            xmlns:ifrs="http://xbrl.ifrs.org/taxonomy/2024-03-27">
  <us-gaap:Revenue contextRef="FY2024" unitRef="USD" decimals="-6">42000000000</us-
gaap:Revenue>
  <us-gaap:NetIncome contextRef="FY2024" unitRef="USD" decimals="-6">15000000000</us-
gaap:NetIncome>
  <ifrs:Revenue contextRef="FY2024" unitRef="EUR" decimals="-6">38000000000</ifrs:Revenue>
</xbrli:xbrl>
```

Notice: `us-gaap:Revenue` and `ifrs:Revenue` have the same local name but different namespaces — they are different concepts with different definitions. String comparison (`"Revenue" == "Revenue"`) would wrongly treat them as the same. QName functions handle this correctly:

```
%utlx 1.0
input xml
output json
---
let facts = $input["xbrli:xbrl"].*

// Extract all US-GAAP facts (filter by namespace)
let usgaap = filter(facts, (fact) ->
  namespaceUri(fact) == "http://fasb.org/us-gaap/2024"
)

// Build a financial summary from XBRL facts
map(usgaap, (fact) -> {
  concept: localName(fact),
  namespace: namespacePrefix(fact),
  value: toNumber(fact),
  context: fact.@contextRef,
  unit: fact.@unitRef,
  decimals: toNumber(fact.@decimals)
})
```

QName functions are also needed for:

- **Inline XBRL (iXBRL):** HTML documents with embedded XBRL facts tagged by QName
- **Taxonomy switching:** mapping from one reporting standard to another (US-GAAP → IFRS)
- **XBRL validation:** verifying that facts reference valid taxonomy concepts
- **Regulatory submissions:** SEC (US), ESMA (EU), HMRC (UK) all require XBRL filings

22.6 XML Attributes in JSON/YAML Output

When XML is transformed to JSON or YAML, attributes need special handling because JSON and YAML have no concept of attributes. Chapter 23 covers this in detail. The short version:

- Non-leaf element attributes: preserved as `\@key` properties (always)
- Leaf element attributes: dropped by default, preserved with `{writeAttributes: true}`
- Self-closing element attributes: preserved (no text to unwrap)

22.7 Common XML Patterns

22.7.1 SOAP (Simple Object Access Protocol)

SOAP is still dominant in banking, government, insurance, and healthcare integration. A SOAP message has a fixed envelope structure:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:wsa="http://www.w3.org/2005/08/addressing"
  xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/...">
  <soap:Header>
    <wsa:To>https://api.example.com/payments</wsa:To>
    <wsa:Action>ProcessPayment</wsa:Action>
    <wsa:MessageID>urn:uuid:550e8400-e29b-41d4-a716-446655440000</wsa:MessageID>
    <wsse:Security>
      <wsse:UsernameToken>
        <wsse:Username>serviceAccount</wsse:Username>
      </wsse:UsernameToken>
    </wsse:Security>
  </soap:Header>
  <soap:Body>
    <pay:ProcessPayment xmlns:pay="urn:example:payments">
      <pay:Amount currency="EUR">1500.00</pay:Amount>
      <pay:Creditor>Acme Corp</pay:Creditor>
    </pay:ProcessPayment>
  </soap:Body>
</soap:Envelope>
```

22.7.1.1 Extracting the Business Payload

The most common pattern — strip the envelope and extract the body content:

```
let body = $input["soap:Envelope"]["soap:Body"]
let payload = body[keys(body)[0]] // first child of Body = the operation
{
  ...payload
}
```

22.7.1.2 Accessing SOAP Headers

SOAP headers carry routing, security, and transaction metadata. Access them alongside the body:

```
let envelope = $input["soap:Envelope"]
let header = envelope["soap:Header"]
```

```

let body = envelope["soap:Body"]
let payload = body[keys(body)[0]]

{
  // Metadata from headers
  destination: header["wsa:To"],
  action: header["wsa:Action"],
  messageId: header["wsa:MessageID"],
  authenticatedUser: header["wsse:Security"]?.["wsse:UsernameToken"]?.["wsse:Username"],

  // Business data from body
  amount: toNumber(payload["pay:Amount"]),
  currency: payload["pay:Amount"].@currency,
  creditor: payload["pay:Creditor"]
}

```

Common SOAP headers you'll encounter:

- **WS-Addressing** (wsa:) — To, Action, MessageID, ReplyTo, FaultTo — routing and correlation
- **WS-Security** (wsse:) — UsernameToken, BinarySecurityToken, Timestamp — authentication and signatures
- **WS-ReliableMessaging** (wsrm:) — Sequence, AckRequested — guaranteed delivery
- **Custom headers** — application-specific metadata (transaction IDs, tenant identifiers, trace context)

22.7.1.3 SOAP 1.1 vs 1.2

The envelope namespace differs between versions:

```

// SOAP 1.1
let body = $input["soap:Envelope"]["soap:Body"]
// namespace: http://schemas.xmlsoap.org/soap/envelope/

// SOAP 1.2
let body = $input["soap12:Envelope"]["soap12:Body"]
// namespace: http://www.w3.org/2003/05/soap-envelope

// Version-agnostic (check which prefix exists):
let envelope = $input["soap:Envelope"] ?? $input["soap12:Envelope"]
let body = envelope["soap:Body"] ?? envelope["soap12:Body"]

```

In practice, the namespace prefix may vary (soap, soapenv, SOAP-ENV, soap12). Use `getNamespaces()` to discover what prefix the sender used, or access by position using `.*`:

```

// Truly prefix-agnostic: get the first (and only) child of root
let envelope = $input.*[0]
let header = envelope.*[0] // first child = Header (if present)
let body = envelope.*[1] // second child = Body
// Note: if no Header, Body is the first child – check with localName()

```

22.7.1.4 Handling SOAP Faults

SOAP faults are error responses with a structured format:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>

```

```

<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Invalid payment amount</faultstring>
  <detail>
    <errorCode>PAY-001</errorCode>
    <message>Amount must be positive</message>
  </detail>
</soap:Fault>
</soap:Body>
</soap:Envelope>

```

```

let body = $input["soap:Envelope"]["soap:Body"]
let fault = body["soap:Fault"]

if (fault != null) {
  error: true,
  code: fault.faultcode,
  message: fault.faultstring,
  detail: fault.detail?.errorCode,
  detailMessage: fault.detail?.message
} else {
  let payload = body[keys(body)[0]]
  error: false,
  ...payload
}

```

22.7.1.5 Constructing SOAP Requests

Build a SOAP envelope for calling a web service:

```

%utlx 1.0
input json
output xml
---
{
  "soap:Envelope": {
    "@xmlns:soap": "http://schemas.xmlsoap.org/soap/envelope/",
    "@xmlns:pay": "urn:example:payments",
    "soap:Header": {
      "wsa:To": { "@xmlns:wsa": "http://www.w3.org/2005/08/addressing", "_text":
$input.endpoint },
      "wsa:Action": { "@xmlns:wsa": "http://www.w3.org/2005/08/addressing", "_text":
"ProcessPayment" }
    },
    "soap:Body": {
      "pay:ProcessPayment": {
        "pay:Amount": { "@currency": $input.currency, "_text": toString($input.amount) },
        "pay:Creditor": $input.creditorName,
        "pay:IBAN": $input.creditorIban
      }
    }
  }
}

```

This transforms a simple JSON payment request into a fully namespaced SOAP envelope with WS-Addressing headers — ready to POST to a SOAP endpoint.

22.7.2 UBL Invoice (Peppol BIS 3.0)

```
let inv = $input.Invoice
{
  invoiceId: inv["cbc:ID"],
  issueDate: inv["cbc:IssueDate"],
  currency: inv["cbc:DocumentCurrencyCode"],
  supplier: inv["cac:AccountingSupplierParty"]["cac:Party"]["cac:PartyName"]["cbc:Name"],
  customer: inv["cac:AccountingCustomerParty"]["cac:Party"]["cac:PartyName"]["cbc:Name"],
  total: toNumber(inv["cac:LegalMonetaryTotal"]["cbc:PayableAmount"]),
  lines: map(inv["cac:InvoiceLine"], (line) -> {
    lineId: line["cbc:ID"],
    description: line["cac:Item"]["cbc:Name"],
    quantity: toNumber(line["cbc:InvoicedQuantity"]),
    unitPrice: toNumber(line["cac:Price"]["cbc:PriceAmount"])
  })
}
```

The bracket notation `inv["cbc:ID"]` is a string-based property lookup — it accesses the child element named `cbc:ID` on the UDM Object. This works exactly like dot notation (`inv.name`) but allows characters that dot notation cannot handle, such as the colon in namespace prefixes. Since UBL elements all have prefixes like `cbc:` and `cac:`, bracket notation with string keys is the only way to navigate them.

22.7.3 HL7 FHIR XML

FHIR uses `value` attributes instead of text content (see FHIR analysis in the architecture docs):

```
let patient = $input.Patient
{
  resourceType: "Patient",
  id: patient.id.@value,
  name: patient.name.family.@value,
  birthDate: patient.birthDate.@value,
  active: toBoolean(patient.active.@value)
}
```

The `\@value` accessor handles FHIR's convention naturally — no special FHIR mode needed.

22.7.4 SAP IDoc XML

SAP exports IDocs as flat XML segments (see Chapter 21 on data restructuring for the grouping pattern):

```
// Simple field extraction from IDoc:
{
  orderNumber: $input.IDOC.E1EDK01.BELNR,
  customer: $input.IDOC.E1EDK01.KUNNR,
  currency: $input.IDOC.E1EDK01.WAERK,
  lines: map($input.IDOC.E1EDP01, (line) -> {
    material: line.MATNR,
    quantity: toNumber(line.MENGE),
    price: toNumber(line.NETPR)
  })
}
```

22.8 CDATA Sections

CDATA (`<![CDATA[...]]>`) is an XML mechanism for including text that would otherwise need escaping. Inside a CDATA section, characters like `<`, `>`, and `&` are treated as literal text, not XML markup:

```
<Script><![CDATA[
  if (price < 100 && quantity > 0) {
    discount = price * 0.1;
  }
]]></Script>
```

Without CDATA, the < and & would need escaping as < and &. CDATA is common in:

- Embedded scripts or code snippets in XML configurations
- HTML content inside XML elements (CMS systems, email templates)
- SQL queries stored in XML configuration files
- Log messages with special characters

22.8.1 How UTL-X Handles CDATA

On input, CDATA sections are transparent — the parser extracts the content and treats it as plain text. The CDATA markers are stripped:

```
// Input: <Script><![CDATA[if (x < 10) alert("low");]]></Script>
$input.Script    // 'if (x < 10) alert("low");'
```

You work with the text content directly — no CDATA markers, no escaping. The content is a regular string.

22.8.2 XML Inside CDATA

A common real-world pattern — not part of the XML standard, but widely used in integration — is embedding XML inside CDATA. The outer XML parser sees the inner XML as plain text, avoiding namespace conflicts and double-parsing:

```
<Message>
  <Header>
    <MessageId>MSG-001</MessageId>
  </Header>
  <Payload><![CDATA[
    <Order xmlns="urn:example:orders">
      <Customer>Acme Corp</Customer>
      <Total>299.99</Total>
    </Order>
  ]]></Payload>
</Message>
```

This appears in SOAP envelopes (inner payload wrapped to avoid namespace collisions), message queues (JMS/MQ wrapping XML bodies), SAP PI/PO adapters, API gateway logs, and configuration files that embed XML snippets.

UTL-X handles this naturally — the CDATA content arrives as a string, and you parse it with `parse()`:

```
// $input.Message.Payload is a string containing XML
let innerXml = parse($input.Message.Payload, "xml")

// Now navigate the inner XML:
{
  messageId: $input.Message.Header.MessageId,
  customer: innerXml.Order.Customer,
```



```
total: toNumber(innerXml.Order.Total)
}
```

The `parse()` function takes the CDATA string and parses it as XML into a navigable UDM tree. This two-level parsing — outer XML automatically, inner XML explicitly — is the correct approach because UTL-X cannot know whether a CDATA string contains XML, JSON, SQL, or plain text.

The reverse — embedding XML output inside CDATA in another XML document — requires `render()`:

```
{
  Message: {
    Header: { MessageId: "MSG-002" },
    Payload: render({
      Order: {
        Customer: $input.customer,
        Total: $input.total
      }
    }, "xml")    // renders the inner XML as a string
  }
}
```

The inner XML is serialized to a string by `render()`, and the outer XML serializer will entity-escape it (producing `<Order>`... rather than CDATA). The result is semantically identical — any XML parser reads both forms the same way.

If a downstream system specifically requires CDATA markers (some legacy systems check for them), this is a known limitation — UTL-X uses entity escaping on output, not CDATA.

22.9 XML Comments and Processing Instructions

22.9.1 Comments

XML comments (`<!-- ... -->`) are **not preserved** during parsing. They are skipped and do not appear in the UDM:

```
<!-- This is an order from Acme Corp -->
<Order>
  <Customer>Alice</Customer> <!-- primary contact -->
</Order>
```

After parsing, only the `Order` and `Customer` elements exist. The comments are gone. This is standard behavior for data transformation tools — comments are metadata about the XML document, not part of the data.

If you need to **generate** XML comments in output, this is not currently supported. Comments are a formatting concern, not a data concern.

22.9.2 Processing Instructions

XML processing instructions (`<?target data?>`) — like `<?xml-stylesheet type="text/xsl" href="style.xsl"?>` — are also not preserved. They are parsed and discarded. The XML declaration (`<?xml version="1.0" encoding="UTF-8"?>`) is handled separately: it's read for encoding detection and regenerated on output with the declared encoding.

22.10 XML Documentation (xs:annotation)

XSD schemas can include documentation for types and elements using `xs:annotation` and `xs:documentation`:

```
<xs:element name="OrderId">
  <xs:annotation>
    <xs:documentation>
      Unique identifier for the order. Format: ORD-NNNNN.
      Assigned by the order management system at creation time.
    </xs:documentation>
  </xs:annotation>
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:pattern value="ORD-[0-9]{5}"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

This is not data inside an XML instance — it's metadata inside an XSD schema. UTL-X handles it in two ways:

Reading: When parsing an XSD (input `xsd`), documentation is extracted and mapped to the USDL `%description` directive. It becomes part of the schema model and is available for transformation.

Writing: When generating XSD (output `xsd {addDocumentation: true}`), USDL `%description` values are emitted as `xs:documentation` elements. Without the option, documentation is omitted for a cleaner schema.

Converting: When converting XSD to JSON Schema, `xs:documentation` becomes the `description` keyword. When converting to Avro, it becomes the `doc` field. The USDL classification system maps documentation across all schema formats.

22.11 XML Round-Trip

XML → UDM → XML preserves structure, attributes, namespaces, and text content. The round-trip is not byte-for-byte identical (whitespace and formatting may differ) but is semantically equivalent.

What IS preserved: element names, element order, attributes, attribute values, text content, namespaces, CDATA content (as text).

What is NOT preserved: insignificant whitespace, comment nodes, processing instructions, DTD declarations, entity references (resolved during parsing), CDATA markers (content preserved, markers not).

For most integration use cases, this is correct behavior — you care about the data, not the formatting. Use `c14nEquals()` (see XML Canonicalization section above) to verify semantic equivalence.

23 XML Attribute Handling: Design Decisions

JSON and YAML have no concept of “attributes.” XML has two ways to attach data to an element: child elements (`<Customer><Name>Alice</Name></Customer>`) and attributes (`<Customer name="Alice"/>`). When converting XML to JSON or YAML, what happens to the attributes? There is no standard answer — every tool makes different choices.

This chapter documents UTL-X’s design decisions, the industry conventions that informed them, and the `writeAttributes` option that gives you control.

23.1 How UTL-X Handles Attributes Internally

XML attributes are stored separately from child elements in the UDM. When UTL-X parses `<Total currency="EUR">299.99</Total>`, it creates:

```
UDM.Object(
  name = "Total",
  properties = {"_text" → Scalar("299.99")},    ← text content
  attributes = {"currency" → "EUR"}             ← XML attributes
)
```

Properties and attributes live in separate maps — they never collide. An element could theoretically have a child element and an attribute with the same name, and UTL-X handles both correctly.

Access is clean and unambiguous:

```
$input.Total          // 299.99 (text content, auto-unwrapped)
$input.Total.@currency // "EUR" (attribute value)
```

The `@` prefix distinguishes attribute access from property access. No guessing, no ambiguity.

23.2 The `_text` Convention

When an XML element has text content, UDM stores it as a property named `_text`. This is an **internal** convention — you never see it in output.

Why does `_text` exist? UDM Objects store properties as a `Map<String, UDM>`. There is no dedicated “value” field for text content. The `_text` key gives the text a place in the map — necessary for elements that have both text AND attributes or child elements.

When you write `$input.Total`, UTL-X automatically unwraps `_text` and returns the value `299.99` directly. You never need to write `$input.Total._text` — the unwrapping is invisible.

This was refined through two bug fixes:

- `_text` was not auto-unwrapped during property access — fixed so `$input.Total` returns `299.99`, not `{_text: 299.99}`
- `_text` leaked into JSON/YAML output — fixed so serializers unwrap it before writing

After these fixes, `_text` is a pure internal implementation detail.

23.3 The `writeAttributes` Option

When XML is serialized to JSON or YAML, leaf elements with both text AND attributes present a dilemma: should the output be a simple value (losing the attribute) or an object (preserving it but changing the structure)?

UTL-X follows the MuleSoft DataWeave convention with a `writeAttributes` option:

```
output json // default: writeAttributes=false
output json {writeAttributes: true} // opt-in: preserve leaf attributes
```

23.3.1 Default Behavior (writeAttributes: false)

```
<Total currency="EUR">299.99</Total>
```

JSON output:

```
"Total": 299.99
```

The currency attribute is dropped. The output is clean — most JSON consumers don't expect XML attributes and would not know what to do with them.

23.3.2 With writeAttributes: true

```
<Total currency="EUR">299.99</Total>
```

JSON output:

```
"Total": {"@currency": "EUR", "#text": 299.99}
```

The attribute is preserved with an @ prefix. The text content moves to a #text key. The element becomes an object instead of a plain value.

23.4 What Gets Kept vs Dropped

Not all attributes are affected by the writeAttributes setting. The rule depends on the element type:

XML Element	Output (default)	Attributes	Why
<Order id="ORD-001">...</Order>	"@id": "ORD-001"	Kept	Non-leaf: has children, always an object
<Customer>Alice</Customer>	"Alice"	N/A	No attributes to lose
<Total currency="EUR">299.99</Total>	299.99	Dropped	Leaf + text: unwrapped to plain value
<Sales type="direct"/>	"@type": "direct"	Kept	Self-closing: no text to unwrap

The rule: only **leaf elements with both text AND attributes** lose their attributes in default mode. Non-leaf elements (which have children) are already objects, so their attributes fit naturally as @-prefixed properties. Self-closing elements have no text to unwrap, so they stay as objects with their attributes.

23.5 Why Drop by Default?

This is a deliberate design choice, not an oversight:

- **Most JSON/YAML consumers don't expect attributes.** A downstream REST API expects "total": 299.99, not "total": {"@currency": "EUR", "#text": 299.99}. Clean output is more important than completeness for the common case.
- **DataWeave compatibility.** MuleSoft's DataWeave uses the same default (writeAttributes: false). Developers moving between tools find familiar behavior.

- **Explicit access is always available.** Even with `writeAttributes: false`, you can still read attributes in your transformation: `$input.Total.@currency` works regardless of the output setting. The setting only affects **serialization**, not **access**.
- **Power users opt in.** When you need full fidelity — XML digital signatures, compliance round-trips, schema-driven output — set `writeAttributes: true`. One option, full control.

23.6 JSON and YAML Consistency

Both serializers use the same convention for attributes:

```
// JSON
{"@currency": "EUR", "#text": 299.99}

// YAML
'@currency': EUR
'#text': 299.99
```

The `@` prefix for attributes and `#text` for text content are identical in both formats. Previously, the YAML serializer used a non-standard `_attributes` block — a UTL-X invention that no other tool recognized. This was replaced with inline `@` prefix to match JSON and industry conventions.

23.7 Industry Conventions

UTL-X's choice is informed by established XML-to-JSON conventions:

Convention	Attribute prefix	Text key	Used by
Badgerfish	@	\$	Java XML libs, .NET Json.NET
Parker	(dropped)	plain value	Simple converters
GData	(none — collision risk)	\$t	Google APIs
DataWeave	@	(value or <code>__text</code>)	MuleSoft
yq	+@ (configurable)	+content	CLI tool
UTL-X	@	#text	Follows DataWeave convention

The `@` prefix for attributes is the de facto industry standard. The text key varies (`$`, `$t`, `__text`, `#text`) — UTL-X uses `#text` which is distinct from any reserved UTL-X symbol and reads naturally.

23.8 FHIR XML: The value Attribute Convention

HL7 FHIR uses `value` attributes instead of text content — a deliberate design choice for extensibility (every FHIR element can have child extensions):

```
<!-- FHIR style (value attribute): -->
<birthDate value="1955-03-12"/>

<!-- Standard XML style (text content): -->
<birthDate>1955-03-12</birthDate>
```

UTL-X handles FHIR naturally via the `@` accessor:

```
$input.Patient.birthDate.@value // "1955-03-12"
$input.Patient.name[0].family.@value // "Simpson"
$input.Patient.active.@value // "true"
```

No special FHIR mode needed — the `@` accessor works for all XML attributes, including FHIR's `value` convention. Since FHIR elements are self-closing (no text content), `writeAttributes` does not affect them — their attributes are always preserved.

24 JSON Transformations

JSON is the native language of modern APIs, microservices, and web applications. It maps almost perfectly to UTL-X's Universal Data Model: JSON objects become UDM Objects, JSON arrays become UDM Arrays, and JSON scalars (strings, numbers, booleans, null) become UDM Scalars. This near-perfect alignment makes JSON the simplest format to work with in UTL-X — and the one most transformations involve.

24.1 JSON and UDM: A Natural Fit

Unlike XML (which has attributes, mixed content, namespaces, and text nodes) or CSV (which is flat and untyped), JSON maps to UDM without any structural translation:

JSON	UDM	Notes
{}	Object	Properties map directly to UDM Object properties
[]	Array	Elements map directly to UDM Array elements
"text"	Scalar (String)	No wrapping, no <code>_text</code> — just the value
42	Scalar (Number)	Integers preserved as Long, decimals as Double
true/false	Scalar (Boolean)	Direct mapping
null	Null	Direct mapping

This means JSON input has none of the quirks described in the XML chapters — no `_text` unwrapping, no `@attribute` prefixes, no namespace handling. What you see in the JSON is exactly what you get in `$input`.

24.1.1 What You See Is What You Get

```
{
  "order": {
    "id": "ORD-001",
    "customer": "Acme Corp",
    "items": [
      {"product": "Widget", "qty": 10, "price": 25.00},
      {"product": "Gadget", "qty": 5, "price": 49.99}
    ],
    "priority": true,
    "notes": null
  }
}
```

Every path works exactly as you'd expect:

```
$input.order.id          // "ORD-001"
$input.order.customer    // "Acme Corp"
$input.order.items[0]    // {product: "Widget", qty: 10, price: 25.00}
$input.order.items[1].product // "Gadget"
$input.order.priority     // true
$input.order.notes       // null
count($input.order.items) // 2
```

No surprises. No unwrapping. No format-specific accessors. This is why JSON is the recommended format for learning UTL-X — you can focus on the language without worrying about format quirks.

24.2 Reading JSON

24.2.1 Property Access

Dot notation is the primary way to navigate JSON structures:

```
$input.user.name           // simple property
$input.user.address.city   // nested property
$input.user.roles[0]       // first array element
$input.user.metadata.tags[2] // nested array element
```

Bracket notation works for dynamic keys and keys with special characters:

```
let field = "name"
$input.user[field]           // dynamic property access

$input["content-type"]       // key with hyphen
$input["2024-results"]       // key starting with number
```

24.2.2 Safe Navigation

When a property might not exist, use `?.` to avoid null reference errors:

```
$input.user.address?.city    // null if address is missing (not an error)
$input.user.phone?.mobile?.number // null if any level is missing
```

Without `?.`, accessing a property on null throws an error. With `?.`, the expression short-circuits to null. Combine with `??` for defaults:

```
$input.user.address?.city ?? "Unknown"
$input.user.phone?.mobile?.number ?? $input.user.phone?.landline ?? "N/A"
```

24.2.3 Number Precision

UTL-X's JSON parser preserves integer precision. Numbers without a decimal point or exponent are parsed as 64-bit integers (Long), not floating-point:

```
// Input: {"id": 9007199254740993, "price": 29.99}

$input.id      // 9007199254740993 (Long – no precision loss)
$input.price   // 29.99 (Double)
```

This matters for large IDs, timestamps, and financial identifiers that exceed JavaScript's `Number.MAX_SAFE_INTEGER`. UTL-X gets this right where many JSON tools silently lose precision.

24.3 Writing JSON

24.3.1 Object Construction

UTL-X object literals map directly to JSON objects:

```
%utlx 1.0
input json
output json
---
```



```
{
  orderId: $input.order.id,
  customerName: $input.order.customer,
  lineCount: count($input.order.items),
  total: sum(map($input.order.items, (i) -> i.qty * i.price))
}
```

Output:

```
{
  "orderId": "ORD-001",
  "customerName": "Acme Corp",
  "lineCount": 2,
  "total": 499.95
}
```

Property names are automatically quoted in JSON output. You write `orderId`: (no quotes needed in UTL-X), and the serializer produces `"orderId":`.

24.3.2 Array Construction

Use `map()` and `filter()` to build arrays:

```
// Transform each item
map($input.order.items, (item) -> {
  sku: item.product,
  quantity: item.qty,
  lineTotal: item.qty * item.price
})
```

Or construct arrays directly with bracket syntax:

```
[
  $input.order.id,
  $input.order.customer,
  toString($input.order.total)
]
```

24.3.3 Spread Operator

The spread operator `...` copies all properties from one object into another — essential for JSON-to-JSON transformations where you want to keep most fields and change a few:

```
// Keep everything, override one field, add one field
{
  ...$input.order,
  status: "CONFIRMED",
  confirmedAt: now()
}
```

This is more maintainable than listing every field — if the source adds new fields, they flow through automatically.

24.3.4 Conditional Fields

Include or exclude fields based on conditions:

```
{
  id: $input.id,
  name: $input.name,
  // Only include email if it exists
  ...if ($input.email != null) { email: $input.email } else {},
  // Only include address if country is provided
  ...if ($input.country != null) {
    address: {
      country: $input.country,
      city: $input.city ?? "Unknown"
    }
  } else {}
}
```

24.3.5 Pretty Printing vs Compact Output

By default, UTL-X outputs pretty-printed JSON with 2-space indentation. Use `--no-pretty` on the command line for compact output:

```
# Pretty (default)
echo '{"x":1}' | utlx transform.utlx
# {"x": 1}    ← with whitespace

# Compact
echo '{"x":1}' | utlx transform.utlx --no-pretty
# {"x":1}    ← no whitespace
```

Pretty printing is for human readability. Compact output saves bandwidth in production pipelines.

24.4 JSON-Specific Functions

24.4.1 `parseJson` and `renderJson`

These functions convert between JSON strings and UDM values *within* a transformation. Use them when JSON is embedded inside another format — a JSON string inside a CSV column, or a JSON payload inside an XML element.

```
// Parse a JSON string into a navigable UDM value
let config = parseJson($input.configJson)
config.database.host    // "localhost"
config.database.port    // 5432

// Render a UDM value back to a JSON string
let payload = renderJson({
  event: "order.created",
  data: $input.order
})
// payload is a string: '{"event":"order.created","data":{...}}'

// renderJson with pretty printing
let readable = renderJson($input.order, true)
```

These are NOT needed for normal JSON-to-JSON transformations — UTL-X handles the parsing and serialization automatically via the `input json` / `output json` declarations. They're for the special case where JSON is a *value* inside your data, not the format of your data.

24.4.2 JSON Canonicalization (RFC 8785)

UTL-X implements RFC 8785 (JSON Canonicalization Scheme / JCS) for deterministic JSON serialization. This is essential for digital signatures, content hashing, and change detection — anywhere two representations of the same data must produce identical bytes.

```
// Canonicalize: sorted keys, no whitespace, deterministic number format
let canonical = canonicalizeJSON($input)
// or equivalently:
let canonical = jcs($input)

// Hash the canonical form (default SHA-256)
let hash = canonicalJSONHash($input)
// "a1b2c3d4..."

// Hash with a different algorithm
let md5 = canonicalJSONHash($input, "MD5")

// Compare two JSON values semantically (ignoring key order, whitespace)
jsonEquals(json1, json2)    // true if same content

// Check if a string is already in canonical form
isCanonicalJSON(someString) // true/false

// Get byte size of canonical form (useful for Content-Length)
canonicalJSONSize($input)   // number of UTF-8 bytes
```

JCS rules:

- Object keys sorted by Unicode code point (lexicographic)
- No whitespace between tokens
- Numbers in ECMAScript format (no unnecessary decimals, no leading zeros)
- Strings with minimal escaping (only required characters)

24.4.3 parse and render (Generic)

The generic `parse()` and `render()` functions accept a format parameter:

```
let data = parse(jsonString, "json")    // same as parseJson(jsonString)
let str = render(data, "json")          // same as renderJson(data)
let str = render(data, "json", true)    // pretty-printed
```

These are useful in generic transformations that handle multiple formats dynamically.

24.5 Common JSON Patterns

24.5.1 REST API Response Transformation

Normalize different API response shapes into a consistent internal format:

```
%utlx 1.0
input json
output json
---
// API returns {data: [...], meta: {total: N, page: P}}
// Normalize to {items: [...], pagination: {total: N, page: P, hasMore: bool}}

{
```

```

items: map($input.data, (item) -> {
  id: item.id,
  name: item.attributes?.name ?? item.name,
  createdAt: item.created_at ?? item.createdAt
}),
pagination: {
  total: $input.meta.total,
  page: $input.meta.page,
  pageSize: count($input.data),
  hasMore: $input.meta.page * count($input.data) < $input.meta.total
}
}

```

24.5.2 Flattening Nested API Responses

Many APIs nest data deeply. Flatten it for downstream consumers:

```

%utlx 1.0
input json
output json
---
map($input.data, (user) -> {
  id: user.id,
  name: user.attributes.name,
  email: user.attributes.email,
  city: user.relationships.address?.data?.attributes?.city,
  companyName: user.relationships.company?.data?.attributes?.name
})

```

24.5.3 Key Renaming (snake_case to camelCase)

A common integration task — the source API uses `snake_case`, the target expects `camelCase`:

```

%utlx 1.0
input json
output json
---
{
  userId: $input.user_id,
  firstName: $input.first_name,
  lastName: $input.last_name,
  emailAddress: $input.email_address,
  phoneNumber: $input.phone_number,
  createdAt: $input.created_at,
  isActive: $input.is_active
}

```

For large payloads with many fields, the spread operator plus individual overrides is more maintainable — but UTL-X does not have a built-in `camelCase()` function for dynamic key renaming. Each field must be mapped explicitly.

24.5.4 Merging Multiple JSON Objects

Combine data from different sources:

```
%utlx 1.0
input json
output json
---
// $input has "user" and "preferences" from different API calls
{
  ...$input.user,
  ...$input.preferences,
  fullName: concat($input.user.firstName, " ", $input.user.lastName),
  locale: $input.preferences.language ?? "en"
}
```

24.5.5 Filtering and Reshaping Arrays

```
%utlx 1.0
input json
output json
---
// Keep only active products over $10, reshape for the catalog
$input.products
|> filter((p) -> p.active && p.price > 10)
|> sortBy((p) -> p.price)
|> map((p) -> {
  sku: p.id,
  title: p.name,
  priceFormatted: concat("$", toString(p.price)),
  inStock: p.inventory > 0,
  categories: p.tags ?? []
})
```

24.5.6 Building Lookup Maps

Transform an array into a keyed object for fast access:

```
%utlx 1.0
input json
output json
---
// Convert [{id: "A", name: "Alice"}, {id: "B", name: "Bob"}]
// into {"A": {name: "Alice"}, "B": {name: "Bob"}}
reduce($input.users, {}, (acc, user) ->
  {...acc, [user.id]: {name: user.name, email: user.email}}
)
```

24.5.7 Aggregation

Compute summary statistics from JSON data:

```
%utlx 1.0
input json
output json
---
let items = $input.order.items
{
  orderId: $input.order.id,
  lineCount: count(items),
}
```

```

    subtotal: sum(map(items, (i) -> i.qty * i.price)),
    avgPrice: sum(map(items, (i) -> i.price)) / count(items),
    cheapest: sortBy(items, (i) -> i.price) |> first(),
    categories: unique(map(items, (i) -> i.category))
  }

```

24.6 JSON to Other Formats

24.6.1 JSON to XML

JSON has no concept of attributes, so the conversion is straightforward — every property becomes a child element:

```

%utlx 1.0
input json
output xml {root: "Order"}
---
{
  OrderId: $input.id,
  Customer: $input.customer,
  Items: map($input.items, (item) -> {
    Product: item.product,
    Quantity: item.qty,
    Price: item.price
  })
}

```

Output:

```

<Order>
  <OrderId>ORD-001</OrderId>
  <Customer>Acme Corp</Customer>
  <Items>
    <Product>Widget</Product>
    <Quantity>10</Quantity>
    <Price>25.0</Price>
  </Items>
  <Items>
    <Product>Gadget</Product>
    <Quantity>5</Quantity>
    <Price>49.99</Price>
  </Items>
</Order>

```

24.6.2 JSON to CSV

Flatten nested JSON to tabular CSV:

```

%utlx 1.0
input json
output csv
---
map($input.order.items, (item) -> {
  orderId: $input.order.id,
  customer: $input.order.customer,

```

```

    product: item.product,
    qty: item.qty,
    price: item.price,
    lineTotal: item.qty * item.price
  })

```

Output:

```

orderId,customer,product,qty,price,lineTotal
ORD-001,Acme Corp,Widget,10,25.0,250.0
ORD-001,Acme Corp,Gadget,5,49.99,249.95

```

The key insight: CSV is flat, so you must denormalize — repeat parent fields (orderId, customer) for each child row. For complex hierarchies, the `unnest()` function (Chapter 21) automates this.

24.6.3 JSON to YAML

JSON to YAML is nearly trivial — the structures are equivalent:

```

%utlx 1.0
input json
output yaml
---
$input

```

A pass-through transformation produces valid YAML from any JSON input. Of course, you can reshape while converting:

```

%utlx 1.0
input json
output yaml
---
{
  application: $input.app.name,
  version: $input.app.version,
  database: {
    host: $input.config.db_host,
    port: $input.config.db_port,
    name: $input.config.db_name
  }
}

```

24.7 JSON Schema

UTL-X can read and write JSON Schema as a data format — treating schema definitions as transformable data. This is not runtime validation (checking that data conforms to a schema) but schema-as-data manipulation.

24.7.1 Reading JSON Schema

```

%utlx 1.0
input jsch
output yaml
---
$input

```

This reads a JSON Schema file, parses it into USDL (Universal Schema Definition Language), and outputs it as human-readable YAML. Useful for understanding complex schemas or converting between schema formats.

Supported drafts: draft-04 (best effort), draft-07, and 2020-12.

24.7.2 Writing JSON Schema

```
%utlx 1.0
input xsd
output jsch
---
$input
```

Converts an XSD schema to JSON Schema 2020-12 via the USDL intermediate representation. The USDL classification system (Chapter 12) determines which constraints survive the conversion.

24.7.3 Schema Functions

```
// Parse a JSON Schema string into a navigable USDL structure
let schema = parseJSONSchema(jsonSchemaString)

// Render a USDL structure as a JSON Schema string
let schemaStr = renderJSONSchema(usdlSchema)
let schemaStr = renderJSONSchema(usdlSchema, true) // pretty-printed
```

24.8 When JSON Isn't Simple

Despite JSON's clean mapping to UDM, there are edge cases:

24.8.1 Dates Are Strings

JSON has no date type. Dates are always strings — "2026-04-30" or "2026-04-30T14:30:00Z". Parse them explicitly:

```
let orderDate = parseDate($input.createdAt, "yyyy-MM-dd'T'HH:mm:ss'Z'")
let formattedDate = formatDate(orderDate, "dd-MM-yyyy")
```

24.8.2 Numbers Can Be Ambiguous

Is 42 an integer or a float? In JSON, both 42 and 42.0 are “numbers.” UTL-X preserves the distinction (Long vs Double), but downstream systems may not. Be explicit when it matters:

```
{
  quantity: toInteger($input.qty),    // ensure integer
  price: toDecimal($input.price)      // ensure decimal
}
```

24.8.3 Null vs Missing vs Empty

JSON distinguishes between null, missing, and empty:

```
{"a": null, "b": "", "c": []}
// "d" is missing entirely
```



```
$input.a          // null
$input.b          // "" (empty string, NOT null)
$input.c          // [] (empty array, NOT null)
$input.d          // null (missing property returns null)

// To treat all "empty-ish" values the same:
let value = $input.a
let isEmpty = value == null || value == "" || (isArray(value) && count(value) == 0)
```

24.8.4 Deeply Nested Optional Structures

Real-world JSON APIs often have deeply nested optional fields. Chain `?.` for safety:

```
// GraphQL-style deeply nested response
$input.data?.viewer?.repositories?.edges[0]?.node?.name ?? "unknown"
```

Without safe navigation, any `null` in the chain would crash the transformation. With `?.`, it gracefully returns `null`, and `??` provides the fallback.

25 CSV Transformations

CSV is the oldest and most universal data exchange format. Every spreadsheet, every database export, every log file can produce CSV. It's also the most deceptive format — seemingly simple, but full of edge cases: regional number formats, quoted fields with embedded delimiters, missing headers, BOM markers, and the eternal question of whether the separator is a comma, semicolon, or tab.

UTL-X handles all of this with a comprehensive CSV parser and serializer, including regional number formatting for European, French, and Swiss conventions.

25.1 CSV and UDM

CSV maps to UDM differently depending on whether headers are present:

With headers (default): The first row becomes property names. Each subsequent row becomes a UDM Object keyed by those headers. The full CSV becomes a UDM Array of Objects:

```
Name, Age, Active      ← headers become property names
Alice, 30, true        ← {Name: "Alice", Age: 30, Active: true}
Bob, 25, false         ← {Name: "Bob", Age: 25, Active: false}
```

```
$input[0].Name        // "Alice"
$input[1].Age          // 25
count($input)          // 2
```

Without headers: Each row becomes a UDM Array of scalars. The full CSV becomes an Array of Arrays:

```
Alice, 30, true        ← ["Alice", 30, true]
Bob, 25, false         ← ["Bob", 25, false]
```

```
$input[0][0]           // "Alice"
$input[1][1]           // 25
```

25.2 CSV Options

CSV comes in many dialects. UTL-X supports all common variations through header options.

25.2.1 Input Options

```
%utlx 1.0
input csv {delimiter: ";", headers: true}
output json
---
$input
```

Option	Type	Default	What it does
delimiter	String	", "	Field separator: ", ", ";", " ", "\t"
headers	Boolean	true	First row contains column names

25.2.2 Output Options

```
%utlx 1.0
input json
output csv {delimiter: ";", headers: true, regionalFormat: "european", decimals: 2}
---
$input
```

Option	Type	Default	What it does
delimiter	String	", "	Field separator
headers	Boolean	true	Emit header row
bom	Boolean	false	Prepend UTF-8 BOM (for Excel compatibility)
regionalFormat	String	"none"	Number format: "usa", "european", "french", "swiss"
decimals	Int	2	Decimal places for numbers
useThousands	Boolean	true	Include thousands separator

25.3 Regional Number Formats

This is where CSV gets tricky. A number written as 1,234.56 in the US is 1.234,56 in Germany, 1 234,56 in France, and 1'234.56 in Switzerland. The comma that separates *fields* in US CSV is the *decimal separator* in European CSV. This is why European CSV files use semicolons as delimiters.

25.3.1 Output Formatting

UTL-X supports four regional formats on output:

Format	Option value	Thousands	Decimal	Example
USA	"usa" or "us"	,	.	1,234.56
European	"european" or "eu"	.	,	1.234,56
French	"french" or "fr"	thin space	,	1 234,56
Swiss	"swiss" or "ch"	'	.	1'234.56
None	"none" (default)	none	.	1234.56

```
%utlx 1.0
input json
output csv {delimiter: ";", regionalFormat: "european", decimals: 2}
---
map($input.invoices, (inv) -> {
  number: inv.id,
  customer: inv.customer,
  amount: inv.total,
  vat: inv.total * 0.21
})
```

Output (European CSV — semicolons, comma decimals):

```
number;customer;amount;vat
INV-001;Acme Corp;1.234,56;259,26
INV-002;Globex Inc;567,89;119,26
```

25.3.2 Input Parsing

The CSV parser does NOT auto-detect regional number formats on input. A European number like 1.234,56 arrives as a string because `toDouble()` fails on the dot-comma format. This is by design — the parser cannot guess whether 1,234 means “one thousand two hundred thirty-four” (US) or “one point two three four” (European).

Use the regional parsing functions in your transformation:

```
%utlx 1.0
input csv {delimiter: ";"}
output json
---
map($input, (row) -> {
  product: row.Product,
  price: parseEUNumber(row.Price),          // "1.234,56" → 1234.56
  weight: parseEUNumber(row.Weight)         // "0,75" → 0.75
})
```

Available parsing functions:

Function	Input	Output
<code>parseUSNumber("1,234.56")</code>	US format	1234.56
<code>parseEUNumber("1.234,56")</code>	European format	1234.56
<code>parseFrenchNumber("1 234,56")</code>	French format	1234.56
<code>parseSwissNumber("1'234.56")</code>	Swiss format	1234.56

And the corresponding rendering functions for formatting numbers as strings within a transformation body:

```
renderEUNumber(1234.56)    // "1.234,56"
renderUSNumber(1234.56)   // "1,234.56"
renderFrenchNumber(1234.56) // "1 234,56"
renderSwissNumber(1234.56) // "1'234.56"
```

25.4 Type Inference

The CSV parser automatically detects types for unquoted values:

CSV value	UDM type	Notes
true / false	Boolean	Case-insensitive
null / nil / n/a	Null	Case-insensitive
Empty field	Null	Treated as null
42 / 3.14	Number	Standard dot-decimal format only
Anything else	String	Including regional numbers like 1.234,56

Dates are NOT auto-detected. A value like 2026-04-30 arrives as a string. Parse it explicitly:

```
map($input, (row) -> {
  ...row,
  orderDate: parseDate(row.OrderDate, "yyyy-MM-dd"),
})
```

```
deliveryDate: parseDate(row.DeliveryDate, "dd/MM/yyyy")
})
```

This is intentional — date formats are ambiguous (01/02/2026 could be January 2nd or February 1st), so UTL-X requires you to declare the format.

25.5 Reading CSV

25.5.1 Basic CSV to JSON

The most common pattern — turn a flat CSV into structured JSON:

```
%utlx 1.0
input csv
output json
---
map($input, (row) -> {
  orderId: row.OrderId,
  customer: row.Customer,
  total: row.Amount,
  date: parseDate(row.OrderDate, "yyyy-MM-dd")
})
```

25.5.2 BOM Handling

On input: some tools (especially Excel on Windows) prepend a UTF-8 BOM (Byte Order Mark, U+FEFF) to CSV files. UTL-X strips this automatically — no configuration needed. The BOM is detected and removed before parsing begins.

On output: use {bom: true} to prepend a BOM to the CSV output. This is useful when the target is Excel on Windows, which uses the BOM to detect UTF-8 encoding for non-ASCII characters (accented names, CJK characters, currency symbols):

```
output csv {bom: true}
```

Without the BOM, Excel may misinterpret UTF-8 as Windows-1252, turning ë into Å« and € into garbage. See Chapter 13 for more on BOM across all formats.

25.5.3 Quoting and Special Characters

The parser follows RFC 4180 quoting rules:

- Fields containing the delimiter, quotes, or newlines must be quoted
- Quotes inside quoted fields are doubled: "She said ""hello"" → She said "hello"
- Multi-line values inside quotes are supported

```
Name,Description,Price
Widget,"A small, useful device",29.99
"Gadget Pro","Includes ""turbo"" mode",49.99
Manual,"Line 1
Line 2
Line 3",9.99
```

All three rows parse correctly — the comma inside “A small, useful device” doesn’t split the field, the doubled quotes produce a literal quote, and the multi-line description is preserved as a single string value.

25.5.4 CSV Without Headers

When the CSV has no header row:

```
%utlx 1.0
input csv {headers: false}
output json
---
map($input, (row) -> {
  name: row[0],
  age: row[1],
  active: row[2]
})
```

Without headers, each row is an array of values accessed by index. You assign meaningful names in the transformation.

25.5.5 Tab-Separated Values (TSV)

```
%utlx 1.0
input csv {delimiter: "\t"}
output json
---
$input
```

25.5.6 Semicolon-Separated (European)

```
%utlx 1.0
input csv {delimiter: ";"}
output json
---
map($input, (row) -> {
  product: row.Product,
  price: parseEUNumber(row.Price),
  quantity: toInteger(row.Qty)
})
```

25.6 Writing CSV

25.6.1 JSON to CSV

The transformation result must be an array of objects for proper CSV output. The keys of the first object become the header row:

```
%utlx 1.0
input json
output csv
---
map($input.orders, (order) -> {
  orderId: order.id,
  customer: order.customerName,
  total: order.total,
  status: order.status
})
```

Output:

```
orderId,customer,total,status
ORD-001,Acme Corp,299.99,CONFIRMED
ORD-002,Globex Inc,150.00,PENDING
```

25.6.2 Column Ordering

The order of properties in your output object determines the column order in the CSV. This gives you full control:

```
// Columns appear in this exact order:
{
  date: row.orderDate,      // column 1
  customer: row.customerName, // column 2
  product: row.productName,  // column 3
  amount: row.total          // column 4
}
```

25.6.3 Flattening Nested JSON for CSV

CSV is flat — no nesting. To export nested JSON as CSV, you must denormalize. The `unnest()` function (Chapter 21) automates this, but you can also do it manually:

```
%utlx 1.0
input json
output csv
---
flatten(map($input.orders, (order) ->
  map(order.lines, (line) -> {
    orderId: order.id,
    customer: order.customer,
    product: line.product,
    qty: line.qty,
    price: line.price,
    lineTotal: line.qty * line.price
  })
))
```

Each parent field (`orderId`, `customer`) is repeated for every child row — that’s denormalization.

25.6.4 CSV with BOM for Excel

See BOM Handling above — use `output csv {bom: true}`.

25.6.5 Suppressing Headers

```
%utlx 1.0
input json
output csv {headers: false}
---
map($input.rows, (row) -> {
  a: row.name,
  b: row.value
})
```

Output has no header row — just data rows. Useful for appending to existing CSV files or producing fixed-format output.

25.7 Common CSV Patterns

25.7.1 Bank Statement Processing (European)

European bank exports use semicolons and comma decimals:

```
Datum;Omschrijving;Bedrag;Saldo
30-04-2026;Salaris;2.500,00;5.234,56
29-04-2026;Albert Heijn;-45,67;2.734,56
28-04-2026;Hypotheek;-1.200,00;2.780,23
```

```
%utlx 1.0
input csv {delimiter: ";"}
output json
---
map($input, (row) -> {
  date: parseDate(row.Datum, "dd-MM-yyyy"),
  description: row.Omschrijving,
  amount: parseEUNumber(row.Bedrag),
  balance: parseEUNumber(row.Saldo),
  type: if (parseEUNumber(row.Bedrag) >= 0) "credit" else "debit"
})
```

25.7.2 Log File Processing (Pipe-Delimited)

```
%utlx 1.0
input csv {delimiter: "|"}
output json
---
$input
|> filter((entry) -> entry.Level == "ERROR" || entry.Level == "CRITICAL")
|> sortBy((entry) -> entry.Timestamp)
|> map((entry) -> {
  timestamp: entry.Timestamp,
  level: entry.Level,
  message: entry.Message,
  source: entry.Source
})
```

25.7.3 CSV Aggregation

Group rows and compute totals:

```
%utlx 1.0
input csv
output json
---
let byCategory = groupBy($input, (row) -> row.Category)

map(keys(byCategory), (cat) -> {
  let rows = byCategory[cat]
  category: cat,
  count: count(rows),
  totalRevenue: sum(map(rows, (r) -> r.Revenue)),
  avgPrice: sum(map(rows, (r) -> r.Price)) / count(rows)
})
```


25.7.4 XML to CSV (Invoice Lines)

A common integration: receive an XML invoice, produce a CSV for the accounting system:

```
%utlx 1.0
input xml
output csv {delimiter: ";", regionalFormat: "european", decimals: 2}
---
map($input.Invoice.Lines.Line, (line) -> {
  invoiceNr: $input.Invoice.@number,
  date: $input.Invoice.Date,
  product: line.Description,
  quantity: line.Quantity,
  unitPrice: line.UnitPrice,
  lineTotal: line.Quantity * line.UnitPrice,
  vatRate: line.VATRate,
  vatAmount: line.Quantity * line.UnitPrice * line.VATRate / 100
})
```

25.7.5 Multi-Format Pipeline: CSV + JSON Enrichment

Read CSV orders, enrich with customer data from JSON:

```
%utlx 1.0
input: orders csv, customers json
output csv {regionalFormat: "usa", decimals: 2}
---
map($orders, (order) -> {
  let customer = find($customers, (c) -> c.id == order.CustomerId)

  orderId: order.OrderId,
  customerName: customer?.name ?? "Unknown",
  country: customer?.country ?? "XX",
  amount: order.Amount,
  domestic: if (customer?.country == "US") "YES" else "NO"
})
```

25.8 CSV-Specific Functions

25.8.1 parseCsv

Parse a CSV string embedded within a transformation:

```
let data = parseCsv(csvString)           // default: comma, headers
let data = parseCsv(csvString, {headers: false}) // no headers
let data = parseCsv(csvString, {delimiter: ";"}) // semicolons
```

This is for the case where CSV is a *value* inside your data — a CSV string in a JSON field, or CSV embedded in an XML element. For normal CSV file processing, use `input csv` in the header.

25.8.2 Spreadsheet Functions

UTL-X includes convenience functions for working with CSV data:

```
csvRows(csv)           // all rows as array of objects
csvColumns(csv)        // header names as array
csvColumn(csv, "Price") // all values in the Price column
```

```

csvRow(csv, 0)           // first row as object
csvCell(csv, 2, "Name")  // cell at row 2, column "Name"

csvFilter(csv, "Status", "eq", "ACTIVE") // filter rows
csvSort(csv, "Price", true)              // sort ascending
csvTranspose(csv)                        // swap rows/columns

csvAddColumn(csv, "NewCol", "default")    // add column
csvRemoveColumns(csv, ["Temp", "Debug"])  // drop columns
csvSelectColumns(csv, ["Name", "Price"])  // keep only these columns

csvSummarize(csv)           // {count, sum, avg, min, max} per numeric column

```

These functions operate on CSV strings and are useful for quick exploration and prototyping.

25.9 Edge Cases

25.9.1 Empty CSV

An empty file (or headers-only) produces an empty array:

```

// Input: "Name,Age\n" (headers only, no data rows)
count($input) // 0

```

25.9.2 Single Row

A CSV with one data row produces a single-element array, not a plain object:

```

// Input: "Name,Age\nAlice,30\n"
$input           // [{Name: "Alice", Age: 30}] ← array of one
$input[0].Name   // "Alice"

```

25.9.3 Whitespace Handling

Unquoted field values are trimmed — leading and trailing spaces are removed. Quoted fields preserve all whitespace:

```

Name,Value
Alice , 30
" Bob ", 40

```

Row 1: Name is "Alice" (trimmed), Value is 30. Row 2: Name is " Bob " (quoted, spaces preserved), Value is 40.

26 YAML Transformations

YAML is the configuration language of the cloud-native era. Kubernetes manifests, Docker Compose files, GitHub Actions workflows, Helm charts, Ansible playbooks, OpenAPI specs — they're all YAML. If you work in DevOps, platform engineering, or API management, you will transform YAML.

YAML is a superset of JSON — every valid JSON document is also valid YAML. This means YAML maps to UDM the same way JSON does for most structures: objects, arrays, strings, numbers, booleans, and null. But YAML adds features that JSON lacks: anchors and aliases, multi-document streams, block vs flow styles, and comments. UTL-X handles all of these.

26.1 YAML and UDM

Like JSON, YAML maps naturally to UDM:

YAML	UDM	Notes
Mapping (key: value)	Object	Key order preserved
Sequence (- item)	Array	Direct mapping
String	Scalar (String)	Quoted or unquoted
Integer	Scalar (Number)	Parsed as Long
Float	Scalar (Number)	Parsed as Double
Boolean (true/false)	Scalar (Boolean)	Also yes/no, on/off
Null (null/)	Null	Also empty value
Timestamp	DateTime	Auto-parsed by default

26.1.1 Anchors and Aliases

YAML anchors (&name) and aliases (*name) are resolved transparently during parsing. By the time your transformation sees the data, aliases have been replaced with the actual values:

```
defaults: &defaults
  timeout: 30
  retries: 3

production:
  <<: *defaults
  timeout: 60
```

In your transformation, `$input.production` is a plain object with `timeout: 60` and `retries: 3`. The anchor mechanism is invisible — you work with the resolved data.

26.1.2 Multi-Document Streams

YAML supports multiple documents in one stream, separated by `---`:

```
---
name: Alice
role: admin
---
name: Bob
role: user
```

UTL-X parses multi-document YAML as an array of documents. Use the `yamlSplitDocuments` function or the multi-document parser option to access individual documents:

```
let docs = yamlSplitDocuments($input)
docs[0].name    // "Alice"
docs[1].role    // "user"
```

26.2 Reading YAML

YAML input works identically to JSON — dot notation, bracket notation, safe navigation:

```
$input.server.port           // 8080
$input.database.credentials.password // nested access
$input.services[0].name      // first service name
$input.features?.experimental?.enabled ?? false // safe with default
```

26.2.1 Timestamps

YAML has native timestamp support. Values like `2026-04-30` and `2026-04-30T14:30:00Z` are automatically parsed as date/time values — unlike JSON where dates are always strings:

```
created: 2026-04-30
updated: 2026-04-30T14:30:00Z
```

```
$input.created    // already a date – no parseDate() needed
$input.updated    // already a datetime
```

This auto-parsing can occasionally be surprising — a value like `2024-01-01` that you intended as a string becomes a date. If this causes issues, the parser can be configured to disable timestamp parsing.

26.2.2 Boolean Variants

YAML is more liberal with booleans than JSON. All of these are parsed as boolean `true`: `true`, `True`, `TRUE`, `yes`, `Yes`, `YES`, `on`, `On`, `ON`. And for `false`: `false`, `False`, `FALSE`, `no`, `No`, `NO`, `off`, `Off`, `OFF`.

This matters when processing configuration files that use `yes/no` or `on/off` conventions — they arrive as booleans in your transformation, not strings.

26.3 Writing YAML

26.3.1 Block Style (Default)

UTL-X outputs YAML in block style by default — human-readable, indented with 2 spaces:

```
%utlx 1.0
input json
output yaml
---
{
  server: $input.hostname,
  port: $input.port,
  database: {
    host: $input.db.host,
    name: $input.db.name,
    credentials: {
      username: $input.db.user,
      password: $input.db.pass
    }
  }
}
```

```
}
}
```

Output:

```
server: prod-01
port: 8080
database:
  host: db.internal
  name: myapp
  credentials:
    username: admin
    password: secret123
```

26.3.2 Number Formatting

Whole numbers are output without a decimal point — 42, not 42.0. This matches YAML conventions and prevents downstream tools from interpreting integers as floats.

26.3.3 XML Attributes in YAML

When converting XML to YAML, attributes use the @ prefix — the same convention as JSON output:

```
'@id': ORD-001
Customer: Alice
Total: 299.99
```

With `writeAttributes: true`, leaf elements with attributes preserve both:

```
Total:
  '@currency': EUR
  '#text': 299.99
```

See Chapter 23 for the full attribute handling design.

26.4 YAML Output Options

```
output yaml           // block style (default)
output yaml {writeAttributes: true} // preserve XML attributes in output
```

The `writeAttributes` option is the only YAML-specific output option currently wired from the header. The serializer uses sensible defaults: 2-space indent, 80-character line width, Unix line breaks, block style, Unicode allowed.

26.4.1 The --- Document Start Marker

YAML's --- document start marker is **not emitted** by default. The serializer has an `explicitStart` option internally (default `false`), but it is not currently exposed as a header option. This means UTL-X YAML output never starts with --- unless you're using multi-document output via `yamlMergeDocuments()`.

If a downstream system requires the --- marker, the workaround is to prepend it as a string operation after serialization. In most cases, a missing --- is not a problem — YAML parsers treat it as optional for single-document files. It is only required when separating multiple documents in one stream.

26.5 YAML-Specific Functions

26.5.1 parseYaml

Parse a YAML string embedded within a transformation:

```
let config = parseYaml(yamlString)
config.server.port    // 8080
```

Use this when YAML is a *value* inside your data — a YAML string in a JSON field, or YAML embedded in an API response. For normal YAML file processing, use `input yaml` in the header.

26.5.2 Multi-Document Functions

```
// Split a multi-document YAML stream into individual documents
let docs = yamlSplitDocuments(yamlString)

// Get a specific document by index
let second = yamlGetDocument(yamlString, 1)

// Merge multiple documents back into a stream
let stream = yamlMergeDocuments(docs)
```

26.5.3 Query and Modify Functions

```
// Query a path in a YAML structure
yamlPath($input, "database.host")

// Set a value at a path (returns new structure)
yamlSet($input, "database.port", 5433)

// Delete a path (returns new structure)
yamlDelete($input, "database.credentials.password")

// Check if a path exists
yamlExists($input, "features.experimental")

// Get all keys or values
yamlKeys($input.database)    // ["host", "name", "port", ...]
yamlValues($input.database)  // ["db.internal", "myapp", 5432, ...]
```

26.5.4 Deep Merge

Merge two YAML structures, with the second overriding the first — useful for configuration overlays:

```
let base = parseYaml(baseConfig)
let overlay = parseYaml(envConfig)
let merged = yamlDeepMerge(base, overlay)
```

This is the UTL-X equivalent of Helm's values merging or Kustomize's strategic merge — but as a function you control in your transformation.

26.6 Dynamic Keys

This is one of the most important YAML topics — and it applies equally to JSON. Many real-world YAML files use **data as keys**: server names, environment names, locale codes, feature flags. The keys are not fixed by a schema — they come from the data itself.

26.6.1 The Problem

Consider a typical server configuration:

```
servers:
  production:
    host: prod-db.example.com
    port: 5432
  staging:
    host: staging-db.example.com
    port: 5432
  development:
    host: localhost
    port: 5433
```

The keys `production`, `staging`, `development` are not fixed — a new environment could appear tomorrow. You cannot write `$input.servers.production` if you don't know the environment names in advance. And you cannot use `map()` because `servers` is an object (with named keys), not an array.

This pattern appears everywhere:

- **Kubernetes:** annotations and labels are key-value maps with arbitrary keys
- **i18n files:** locale codes as keys (`en`, `nl`, `de`, `fr`)
- **Feature flags:** feature names as keys with boolean or config values
- **Data contracts:** model names as keys with schema definitions
- **Terraform:** resource names as keys

26.6.2 Reading Dynamic Keys

UTL-X provides a complete toolkit for working with objects whose keys are unknown at design time:

```
// Get all key names as an array
keys($input.servers)           // ["production", "staging", "development"]

// Get all values as an array (keys discarded)
values($input.servers)         // [{host: "prod-db...", ...}, ...]

// Check if a key exists
hasKey($input.servers, "production") // true

// Access by dynamic key (bracket notation)
let env = "staging"
$input.servers[env]            // {host: "staging-db...", port: 5432}

// Wildcard – all values as array (equivalent to values($input.servers) above)
$input.servers.*               // [{host: "prod-db...", ...}, ...]

// Get key-value pairs for iteration
entries($input.servers)
// [{"production", {host: "prod-db..."}}, {"staging", {host: "staging-db..."}}, ...]
```

26.6.3 Iterating Over Dynamic Keys

The `entries()` function is the key to working with dynamic keys — it converts an object into an array of `[key, value]` pairs that you can `map`, `filter`, and `reduce`:

```
// List all environments with their hosts
entries($input.servers) |> map((entry) -> {
```

```

    environment: entry[0],
    host: entry[1].host,
    port: entry[1].port
  })
  // [{environment: "production", host: "prod-db...", port: 5432}, ...]

```

Transform keys and values:

```

// Transform keys (e.g., uppercase environment names)
mapKeys($input.servers, (key) -> toUpperCase(key))
// {PRODUCTION: {...}, STAGING: {...}, DEVELOPMENT: {...}}

// Transform values (e.g., add a computed field)
mapValues($input.servers, (config) -> {
  ...config,
  connectionString: concat("postgres://", config.host, ":", toString(config.port))
})

// Transform both keys and values
mapEntries($input.servers, (key, value) -> {
  key: toUpperCase(key),
  value: { ...value, env: key }
})

```

Filter entries:

```

// Keep only non-production environments
filterEntries($input.servers, (key, value) -> key != "production")

// Check if any server uses a non-standard port
someEntry($input.servers, (key, value) -> value.port != 5432)

// Check if all servers have a host defined
everyEntry($input.servers, (key, value) -> value.host != null)

```

26.6.4 Generating Dynamic Keys (Output)

When you need to **produce** YAML or JSON with dynamic keys, use `fromEntries()` — it builds an object from an array of `[key, value]` pairs:

```

%utlx 1.0
input json
output yaml
---
{
  servers: fromEntries(
    map($input.serverList, (server) -> [
      server.environment,      // dynamic key: "production", "staging", etc.
      {
        host: server.host,
        port: server.port
      }
    ])
  )
}

```


Input:

```
{ "serverList": [
  { "environment": "production", "host": "prod-db.example.com", "port": 5432 },
  { "environment": "staging", "host": "staging-db.example.com", "port": 5432 }
]}
```

Output:

```
servers:
  production:
    host: prod-db.example.com
    port: 5432
  staging:
    host: staging-db.example.com
    port: 5432
```

The array of servers became an object keyed by environment name. This is the inverse of `entries()` — `entries()` converts object-with-dynamic-keys to array, `fromEntries()` converts array back to object-with-dynamic-keys.

26.6.5 Computed Property Names

For simple cases where you need one or two dynamic keys, use computed property names with bracket syntax:

```
{
  [ $input.environmentName ]: {
    host: $input.host,
    port: $input.port
  }
}
```

The brackets around `$input.environmentName` evaluate the expression and use the result as the property name. This is the same syntax as JavaScript's computed property names.

26.6.6 Dynamic Keys Summary

Task	Function	Direction
Get all keys	<code>keys(obj)</code>	Read
Get all values	<code>values(obj)</code>	Read
Get key-value pairs	<code>entries(obj)</code>	Read
Check key exists	<code>hasKey(obj, key)</code>	Read
Access by variable	<code>obj[variable]</code>	Read
Access all	<code>obj.*</code>	Read
Transform keys	<code>mapKeys(obj, fn)</code>	Read/Write
Transform values	<code>mapValues(obj, fn)</code>	Read/Write
Transform both	<code>mapEntries(obj, fn)</code>	Read/Write
Filter entries	<code>filterEntries(obj, fn)</code>	Read/Write
Build from pairs	<code>fromEntries(array)</code>	Write

One dynamic key	{[expr]: value}	Write
-----------------	-----------------	-------

26.7 Common YAML Patterns

26.7.1 Kubernetes Manifest Transformation

Update a Deployment's image tag and replica count:

```
%utlx 1.0
input yaml
output yaml
---
{
  ...$input,
  spec: {
    ...$input.spec,
    replicas: 3,
    template: {
      ...$input.spec.template,
      spec: {
        ...$input.spec.template.spec,
        containers: map($input.spec.template.spec.containers, (c) -> {
          ...c,
          image: if (c.name == "app") "myapp:v2.1.0" else c.image
        })
      }
    }
  }
}
```

The spread operator (...) copies all existing fields and lets you override specific ones — essential for YAML transformations where you want to change one field without listing all the others.

26.7.2 Docker Compose Environment Injection

Add environment variables to a service:

```
%utlx 1.0
input yaml
output yaml
---
{
  ...$input,
  services: {
    ...$input.services,
    app: {
      ...$input.services.app,
      environment: {
        ...$input.services.app.environment,
        DATABASE_URL: "postgres://prod-db:5432/myapp",
        LOG_LEVEL: "warn"
      }
    }
  }
}
```

26.7.3 Configuration File Migration (JSON to YAML)

Convert a JSON config to YAML — often needed when migrating tools:

```
%utlx 1.0
input json
output yaml
---
$input
```

A pass-through transformation is all you need. JSON is a subset of YAML, so every JSON structure converts to valid YAML. The YAML output is more readable — no quotes around keys, indentation instead of braces.

26.7.4 OpenAPI Spec Processing

Extract all endpoints from an OpenAPI 3.0 spec:

```
%utlx 1.0
input yaml
output json
---
flatten(map(keys($input.paths), (path) ->
  map(keys($input.paths[path]), (method) -> {
    let op = $input.paths[path][method]
    path: path,
    method: toUpperCase(method),
    operationId: op.operationId,
    summary: op.summary ?? "",
    tags: op.tags ?? []
  })
))
```

26.7.5 Helm Values Override

Generate environment-specific Helm values from a base config:

```
%utlx 1.0
input: base yaml, env yaml
output yaml
---
yamlDeepMerge($base, $env)
```

The base `values.yaml` provides defaults. The environment overlay (`production.yaml`) overrides specific values. `yamlDeepMerge` handles nested structures — it merges recursively, not just at the top level.

26.8 YAML vs JSON: When to Use Which

Use YAML when	Use JSON when
Human readability matters (configs, docs)	Machine consumption (APIs, data exchange)
Comments are needed	Strict parsing is required
Deep nesting (YAML indentation is cleaner)	Interop with JavaScript/web tooling
Multi-document streams	Single-document payloads
Kubernetes, Helm, Ansible, CI/CD	REST APIs, message queues, databases

UTL-X treats both as first-class formats. Converting between them is trivial — the UDM is the same. The only loss going YAML to JSON: comments and anchors disappear (JSON has neither). Going JSON to YAML: lossless — YAML is a superset.

27 OData Transformations

UTL-X supports OData v4 JSON only. The older Atom/XML format (OData v1-v3) and the verbose JSON format (OData v3) are not supported. OData v4 is the current OASIS standard and the format used by all modern Microsoft services (Dynamics 365, Dataverse, Graph API), SAP Gateway, and other OData v4 providers.

OData (Open Data Protocol) is the REST-based data access standard behind Microsoft’s ecosystem: Dynamics 365, SharePoint, Power Platform, Dataverse, and Azure APIs. If you integrate with Microsoft products — or with SAP systems that expose OData services — you will encounter OData JSON.

OData has evolved through several versions, each with different payload formats:

Version	Payload format	Status	UTL-X support
OData v1-v3	XML (Atom/AtomPub)	Legacy — largely deprecated	Not supported
OData v3	JSON (verbose format)	Legacy	Not supported
OData v4	JSON with @odata.* annotations	Current OASIS standard	Fully supported

OData v4 JSON looks like regular JSON but carries metadata annotations: `@odata.context`, `@odata.type`, `@odata.id`. These tell the client about the entity’s type, its URL, and where to find the schema. UTL-X understands these annotations natively and can strip, preserve, or generate them as needed.

If you encounter OData v1-v3 XML (Atom) payloads from a legacy service, parse them as standard XML (input `xml`) and navigate the Atom structure manually. But in practice, this is rare — upgrade the service endpoint to v4 if at all possible.

27.1 OData v4 JSON and UDM

When UTL-X parses OData JSON, it separates the metadata annotations from the business data:

```
{
  "@odata.context": "https://crm.example.com/$metadata#Accounts",
  "@odata.type": "#Microsoft.Dynamics.CRM.account",
  "accountid": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Acme Corp",
  "revenue": 5000000,
  "primarycontactid@odata.type": "#Microsoft.Dynamics.CRM.contact"
}
```

After parsing:

- Business properties (`accountid`, `name`, `revenue`) become regular UDM Object properties — accessed with dot notation
- `@odata.*` annotations are moved to UDM attributes — accessed with `@` prefix
- Per-property annotations (`primarycontactid@odata.type`) are stored as attributes on the target property

```
$input.name           // "Acme Corp"
$input.accountid      // "550e8400-..."
$input.revenue        // 5000000
$input["@odata.context"] // "https://crm.example.com/$metadata#Accounts"
```

This separation is the key design: your transformation works with clean business data, while the OData metadata is preserved and available when needed.

27.1.1 Collection Responses

OData collection responses wrap the entity array in a `value` property:

```
{
  "@odata.context": "https://crm.example.com/$metadata#Accounts",
  "@odata.count": 42,
  "value": [
    {"accountid": "...", "name": "Acme Corp"},
    {"accountid": "...", "name": "Globex Inc"}
  ]
}
```

UTL-X's OData parser intelligently unwraps this: when the only data property is `value` and no collection-level OData attributes are needed, it unwraps to a bare array. Otherwise it preserves the wrapper. In either case, you access the entities naturally:

```
// If unwrapped to array:
$input[0].name           // "Acme Corp"

// If wrapper preserved:
$input.value[0].name     // "Acme Corp"
count($input.value)      // 42
```

27.2 Reading OData

27.2.1 Dynamics 365 API Responses

A typical D365 sales order response:

```
%utlx 1.0
input odata
output json
---
map($input, (order) -> {
  orderId: order.salesorderid,
  orderNumber: order.ordernumber,
  customer: order.customerid_account?.name,
  total: order.totalamount,
  currency: order.transactioncurrencyid?.isocurrencycode,
  status: order.statuscode,
  lines: map(order.order_details ?? [], (line) -> {
    product: line.productid?.name ?? line.productdescription,
    quantity: line.quantity,
    unitPrice: line.priceperunit,
    lineTotal: line.extendedamount
  })
})
```

Note the safe navigation (`?.`) — OData navigation properties (like `customerid_account`) may be null if the related entity wasn't expanded in the API query.

27.2.2 SharePoint List Items

```
%utlx 1.0
input odata
output json
---
map($input, (item) -> {
  id: item.Id,
  title: item.Title,
  created: item.Created,
  author: item.Author?.Title ?? "Unknown",
  status: item.Status,
  attachments: item.AttachmentFiles ?? []
})
```

27.2.3 SAP OData Services

SAP Gateway exposes business objects via OData. SAP uses some conventions of its own (SAP-specific metadata), but the core OData JSON format is standard:

```
%utlx 1.0
input odata
output json
---
map($input, (bp) -> {
  partnerId: bp.BusinessPartner,
  name: bp.BusinessPartnerFullName,
  category: bp.BusinessPartnerCategory,
  country: bp.to_BusinessPartnerAddress[0]?.Country ?? "XX",
  email: bp.to_BusinessPartnerAddress[0]?.to_EmailAddress[0]?.EmailAddress
})
```

27.3 Writing OData

27.3.1 Output Options

```
output odata                                // minimal metadata (default)
output odata {metadata: "full"}              // all annotations
output odata {metadata: "none"}              // no annotations (plain JSON)
output odata {context: "https://api.example.com/$metadata#Customers"}
output odata {wrapCollection: true}          // wrap array in {value: [...]}
```

Option	Type	Default	What it does
metadata	String	"minimal"	"minimal": context only. "full": all annotations. "none": plain JSON
context	String	null	The @odata.context URL to include
wrapCollection	Boolean	true	Wrap arrays in {"value": [...]}

27.3.2 Metadata Levels

Minimal (default) — only the @odata.context URL:

```
{
  "@odata.context": "https://api.example.com/$metadata#Customers",
```

```
"value": [{"name": "Acme Corp"}, {"name": "Globex Inc"}]
}
```

Full — all available annotations:

```
{
  "@odata.context": "https://api.example.com/$metadata#Customers",
  "@odata.count": 2,
  "value": [
    {
      "@odata.type": "#Example.Customer",
      "@odata.id": "Customers('C-42')",
      "name": "Acme Corp"
    }
  ]
}
```

None — plain JSON, no OData annotations. Use this when the target system doesn't understand OData conventions.

27.3.3 Generating OData for Dynamics 365 Import

Transform external data into D365-compatible OData JSON:

```
%utlx 1.0
input json
output odata {metadata: "minimal", context: "https://org.crm.dynamics.com/api/data/v9.2/$metadata#accounts"}
---
map($input.customers, (c) -> {
  name: c.companyName,
  revenue: c.annualRevenue,
  telephone1: c.phone,
  emailAddress1: c.email,
  address1_city: c.city,
  address1_country: c.country
})
```

27.4 OData Schema (EDMX/CSDL)

OData services publish their schema as EDMX (Entity Data Model XML) — also known as CSDL (Common Schema Definition Language). UTL-X can read and write EDMX using the `osch` format:

```
%utlx 1.0
input osch
output yaml
---
$input
```

This reads an EDMX metadata document and outputs it as human-readable YAML with USDL directives — useful for understanding complex OData service models.

27.4.1 What EDMX Contains

An EDMX document defines:

- **Entity types** — the structure of business objects (Customer, Order, Product)

- **Complex types** — reusable structured types (Address, Money)
- **Enum types** — allowed values (OrderStatus, PaymentMethod)
- **Navigation properties** — relationships between entities (Customer has Orders)
- **Entity container** — the service root with entity sets and singletons
- **Referential constraints** — foreign key relationships

UTL-X parses all of these into USDL directives, making the OData model available for schema-to-schema conversion.

27.4.2 Cross-Format Schema Conversion

Convert an OData schema to other schema formats:

```
// EDMX → JSON Schema
%utlx 1.0
input osch
output jsch
---
$input

// EDMX → XSD
%utlx 1.0
input osch
output xsd
---
$input

// JSON Schema → EDMX
%utlx 1.0
input jsch
output osch
---
$input
```

The USDL classification system (Chapter 12) handles the translation between formats. OData-specific directives like %entityType, %navigation, and %cardinality are Format-native — they map to EDMX natively and become documentation comments in other formats.

27.5 Common OData Patterns

27.5.1 Stripping Metadata

Convert OData JSON to clean JSON by removing all annotations:

```
%utlx 1.0
input odata
output json
---
$input
```

The OData parser moves annotations to UDM attributes. The JSON serializer doesn't write attributes by default. Result: clean JSON without @odata.* noise.

27.5.2 OData to CSV for Reporting

Export D365 data as CSV for Excel or Power BI:

```
%utlx 1.0
input odata
output csv {delimiter: ";", regionalFormat: "european"}
---
map($input, (account) -> {
  id: account.accountid,
  name: account.name,
  revenue: account.revenue,
  country: account.address1_country,
  industry: account.industrycode,
  created: account.createdon
})
```

27.5.3 OData to XML for Legacy Systems

Transform a D365 response into XML for a legacy ERP:

```
%utlx 1.0
input odata
output xml {root: "Customers"}
---
map($input, (c) -> {
  Customer: {
    Id: c.accountid,
    Name: c.name,
    Country: c.address1_country,
    Revenue: c.revenue
  }
})
```

27.5.4 Pagination Handling

OData uses `@odata.nextLink` for pagination. While UTL-X doesn't handle HTTP pagination itself (that's the orchestration layer's job), you can detect and flag it:

```
%utlx 1.0
input odata
output json
---
{
  data: map($input.value ?? $input, (item) -> {
    ...item
  }),
  hasMore: $input["@odata.nextLink"] != null,
  nextLink: $input["@odata.nextLink"]
}
```

27.6 OData vs Standard JSON: When to Use Which

Use input odata	Use input json
Dynamics 365, SharePoint, Dataverse APIs	Standard REST APIs
Response has <code>@odata.*</code> annotations	No OData annotations
You want annotations stripped automatically	You want all properties as-is
SAP Gateway OData services	Generic JSON APIs

You need <code>@odata.context</code> in output	No metadata needed in output
--	------------------------------

If your JSON has `@odata.*` properties and you use input `json`, they'll appear as regular properties named `@odata.context` (accessible via bracket notation). With input `odata`, they're separated into attributes — cleaner to work with.

28 Schema Formats

UTL-X treats schemas as data. An XSD, a JSON Schema, an Avro schema, a Protobuf definition — they are all inputs that can be parsed, transformed, and serialized to a different format. This is what makes schema-to-schema conversion possible: read a schema in one format, the USDL classification system (Chapter 12) normalizes it, and output it in another format.

Six schema formats are supported, each with a full parser and serializer:

Format	Header keyword	Domain	Used by
XSD	xsd	XML	Enterprise XML: UBL, HL7, ISO 20022, SWIFT
JSON Schema	jsch	JSON / REST	REST APIs, OpenAPI, MongoDB, Kafka
Avro Schema	avro	Data pipelines	Kafka, Spark, Hadoop, data lakes
Protobuf	proto	Microservices	gRPC, Google APIs, mobile apps
OData Schema	osch	Enterprise REST	Dynamics 365, SAP Gateway, SharePoint
Table Schema	tsch	Tabular data	CSV metadata, Frictionless Data, CKAN

Every format can appear on either side of a transformation — `input xsd / output jsch` converts XSD to JSON Schema, `input avro / output proto` converts Avro to Protobuf. Any combination works.

28.1 The USDL Bridge

All schema conversions go through USDL (Chapter 12) as the intermediate representation:

```
XSD → USDL → JSON Schema
Avro → USDL → Protobuf
EDMX → USDL → XSD
```

USDL’s classification determines what survives the conversion:

- **Core:** types, names, descriptions — every format has these, always converted
- **Shared:** constraints, validation — converted when the target format supports them, otherwise preserved as comments
- **Format-native:** features unique to one format (XSD facets, Protobuf field numbers, Avro logical types) — converted for the matching format, documented for others

This means a schema conversion is never lossy in a destructive way — information that can’t be expressed in the target format is preserved as metadata or comments, not silently dropped.

28.2 XSD (XML Schema Definition)

The most complex and feature-rich schema format. XSD defines the structure of XML documents with types, constraints, inheritance, and namespace management.

```
%utlx 1.0
input xsd
output yaml
---
$input
```

This reads an XSD and outputs it as readable YAML with USDL directives — often the fastest way to understand a complex enterprise schema.

28.2.1 XSD Features Supported

- Complex types and simple types
- Restrictions (minLength, maxLength, pattern, enumeration, min/maxInclusive)
- Extensions and inheritance
- Element references (ref)
- xs:include and xs:import (same and cross-namespace composition)
- Global and local elements and types
- All five design patterns: Russian Doll, Venetian Blind, Salami Slice, Garden of Eden, Swiss Army Knife (see Chapter 29)
- Annotations and documentation

28.2.2 XSD Output Options

```
output xsd                                // default
output xsd {pattern: "venetian-blind"}    // generate Venetian Blind pattern
output xsd {version: "1.1"}               // XSD 1.1 output
output xsd {addDocumentation: true}       // include xs:documentation from USDL
descriptions
output xsd {elementFormDefault: "qualified"} // element form
```

28.2.3 Stdlib Functions

```
let schema = parseXSDSchema(xsdString)    // XSD XML string → USDL
let xsd = renderXSDSchema(usdlSchema)     // USDL → XSD XML string
let xsd = renderXSDSchema(usdlSchema, true) // pretty-printed
```

28.3 JSON Schema

The schema format for REST APIs and modern web services. JSON Schema validates JSON data structure, types, and constraints.

```
%utlx 1.0
input jsch
output yaml
---
$input
```

28.3.1 Supported Drafts

UTL-X reads draft-07 and 2020-12. Output is always 2020-12 (the current standard). When reading draft-07, keywords like `definitions` are automatically converted to the 2020-12 equivalent `$defs`.

28.3.2 JSON Schema Features Supported

- Object types with properties, required, additionalProperties
- Array types with items, minItems, maxItems, uniqueItems
- String constraints: minLength, maxLength, pattern, format, enum
- Number constraints: minimum, maximum, exclusiveMinimum, exclusiveMaximum
- Composition: \$ref, \$defs, allOf, anyOf, oneOf, not
- const, default, examples, description
- Draft 2020-12 additions: \$dynamicRef, \$dynamicAnchor, unevaluatedProperties

28.3.3 Stdlib Functions

```
let schema = parseJSONSchema(jsonSchemaString) // JSON Schema string → USDL
let jsch = renderJSONSchema(usdlSchema)        // USDL → JSON Schema string
let jsch = renderJSONSchema(usdlSchema, true)   // pretty-printed
```

28.4 Avro Schema

Apache Avro is the schema format for data pipelines — Kafka, Spark, Hadoop, and data lake architectures. Avro schemas are JSON documents that define record structures with strict typing and schema evolution support.

```
%utlx 1.0
input avro
output yaml
---
$input
```

28.4.1 Avro Features Supported

- Primitive types: null, boolean, int, long, float, double, bytes, string
- Complex types: records, enums, arrays, maps, unions, fixed
- Logical types: date, time-millis, time-micros, timestamp-millis, timestamp-micros, decimal, uuid
- Namespace and name resolution
- Field defaults and ordering
- Aliases (field and type renaming for schema evolution)
- Documentation strings

28.4.2 Schema Evolution

Avro's schema evolution support — adding fields with defaults, renaming via aliases — is preserved through USDL. When converting Avro to JSON Schema or XSD, aliases become documentation annotations.

28.4.3 Stdlib Functions

```
let schema = parseAvroSchema(avroSchemaString) // Avro JSON string → USDL
let avro = renderAvroSchema(usdlSchema)        // USDL → Avro JSON string
let avro = renderAvroSchema(usdlSchema, true)   // pretty-printed
```

28.5 Protobuf (Protocol Buffers)

Protocol Buffers define the message format for gRPC services and Google APIs. UTL-X supports Proto3 (the current version).

```
%utlx 1.0
input proto
output yaml
---
$input
```

28.5.1 Protobuf Features Supported

- Messages with typed fields
- Enums (with zero-value requirement)
- Repeated fields (arrays)

- Map fields
- Oneof groups
- Nested messages
- Package and import declarations
- Field numbers (preserved as `%field_number` metadata for round-trip fidelity)
- Reserved fields and ranges

28.5.2 Field Number Preservation

Protobuf field numbers are critical for wire compatibility. UTL-X preserves them through USDL's Format-native directives:

```
// Input: message Order { string id = 1; double total = 3; }
// USDL: %fields: { id: {%type: "string", %fieldNumber: 1}, total: {%type: "number",
%fieldNumber: 3} }
// Output: message Order { string id = 1; double total = 3; }
```

When converting Protobuf to JSON Schema, field numbers become documentation annotations. When converting back, they're restored.

28.5.3 Stdlib Functions

```
let schema = parseProtobufSchema(protoString)    // .proto string → USDL
let proto = renderProtobufSchema(usdlSchema)     // USDL → .proto string
```

28.6 OData Schema (EDMX/CSDL)

OData services publish their model as EDMX metadata documents. UTL-X reads and writes these using the `osch` format. See Chapter 27 for details on OData data transformations and EDMX structure.

```
%utlx 1.0
input osch
output jsch
---
$input
```

This converts an OData entity model to JSON Schema — useful for generating API documentation or validation rules from a Dynamics 365 metadata endpoint.

28.7 Table Schema (TSCH)

The Frictionless Data Table Schema describes the structure of tabular data — column names, types, constraints, and foreign keys. It's the metadata companion for CSV files.

```
%utlx 1.0
input tsch
output yaml
---
$input
```

28.7.1 Table Schema Features Supported

- Field definitions with name, type, and description
- Type system: string, number, integer, boolean, date, time, datetime, year, yearmonth, object, array, geopoint, geojson
- Constraints: required, unique, minLength, maxLength, minimum, maximum, pattern, enum

- Primary key and foreign key declarations

Table Schema is particularly useful for documenting CSV data sources and generating validation rules for CSV imports.

28.8 Schema-to-Schema Conversion Examples

28.8.1 XSD to JSON Schema

The most common enterprise conversion — XML-first systems moving to REST APIs:

```
%utlx 1.0
input xsd
output jsch
---
$input
```

XSD complex types become JSON Schema object definitions. XSD restrictions become JSON Schema constraints. `xs:include/xs:import` chains are resolved and become `$ref/$defs`.

28.8.2 Avro to Protobuf

Data pipeline to microservice gateway:

```
%utlx 1.0
input avro
output proto
---
$input
```

Avro records become Protobuf messages. Avro unions become Protobuf oneof groups. Field numbers are auto-assigned (sequential) since Avro doesn't have them.

28.8.3 JSON Schema to XSD

REST API contract to enterprise XML validation:

```
%utlx 1.0
input jsch
output xsd {pattern: "venetian-blind"}
---
$input
```

JSON Schema `$ref` definitions become named XSD complex types. JSON Schema `required` becomes `minOccurs="1"`. The `pattern` option controls the XSD design pattern.

28.8.4 Any Schema to Human-Readable YAML

The quickest way to understand any schema:

```
%utlx 1.0
input xsd           // or jsch, avro, proto, osch, tsch
output yaml
---
$input
```

USDL in YAML is readable without tooling — field names, types, constraints, and descriptions all visible in one place.

28.9 Schema Functions Summary

Four schema formats expose stdlib functions for use inside transformation bodies:

Format	Parse function	Render function
XSD	<code>parseXSDSchema(str)</code>	<code>renderXSDSchema(usdl, pretty?)</code>
JSON Schema	<code>parseJSONSchema(str)</code>	<code>renderJSONSchema(usdl, pretty?)</code>
Avro	<code>parseAvroSchema(str)</code>	<code>renderAvroSchema(usdl, pretty?)</code>
Protobuf	<code>parseProtobufSchema(str)</code>	<code>renderProtobufSchema(usdl)</code>

These are for programmatic schema manipulation within a transformation — for example, reading an embedded schema string from a message field and extracting type information. For normal schema-to-schema conversion, use `input/output` format declarations in the header.

29 XSD Design Patterns Deep Dive

XML Schema (XSD) is the most complex schema format in use today. The same data structure can be expressed in fundamentally different ways — and the pattern choice affects reusability, tooling compatibility, and how well the schema converts to other formats. This chapter covers the design patterns you'll encounter in enterprise XML and how UTL-X handles each one.

29.1 Why Patterns Matter

Consider a simple order with a customer and address. In XSD, there are at least five valid ways to express this. Each produces a valid schema that validates the same XML — but they differ in how types are defined, where elements are declared, and how reusable the components are.

When you convert an XSD to JSON Schema, Avro, or Protobuf, the pattern determines the output structure. A schema with named reusable types converts cleanly to JSON Schema `$defs`. A schema with deeply nested anonymous types produces a monolithic JSON Schema with no reusable definitions. Understanding the pattern tells you what to expect.

29.2 The Five Core Patterns

All five patterns below are shown with the same domain — an Order containing a Customer containing an Address — so you can focus on **how each pattern organizes the same structure** (global vs. local elements, named vs. anonymous types) rather than on changing example data. The final pattern (Swiss Army Knife) adds a few extra fields to show a realistic mixed schema.

29.2.1 1. Russian Doll

Everything nested inside one global element. Types are defined inline (anonymous), deeply nested — like a matryoshka doll: open one, find another inside.

```
<xs:element name="Order">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="Customer">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="Name" type="xs:string"/>
            <xs:element name="Address">
              <xs:complexType>
                <xs:sequence>
                  <xs:element name="Street" type="xs:string"/>
                  <xs:element name="City" type="xs:string"/>
                </xs:sequence>
              </xs:complexType>
            </xs:element>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Pros: self-contained, one file, clear hierarchy, easy to read top-to-bottom.

Cons: no reuse — types cannot be referenced from other elements or schemas. The Address type is buried inside Customer inside Order. If another element needs the same Address structure, it must be duplicated.

Used by: simple schemas, auto-generated XSDs, quick prototypes.

Converting to JSON Schema: produces nested inline definitions — no `$defs` section. The JSON Schema is as deeply nested as the XSD.

29.2.2 2. Salami Slice

All elements are global (top-level). Types may be inline, but elements are independently defined and assembled via `ref`:

```
<xs:element name="Street" type="xs:string"/>
<xs:element name="City" type="xs:string"/>

<xs:element name="Address">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="Street"/>
      <xs:element ref="City"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>

<xs:element name="Customer">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="Name" type="xs:string"/>
      <xs:element ref="Address"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Pros: elements reusable via `ref`, substitution groups possible, flat structure.

Cons: all elements share one global namespace — collision risk. Every element is a potential document root.

Used by: some industry standards, schemas designed for element substitution.

Converting to JSON Schema: global elements become `$defs` entries. References become `$ref` pointers.

29.2.3 3. Venetian Blind

All types are global (named), elements reference them by type. Types are reusable, elements are local:

```
<xs:complexType name="AddressType">
  <xs:sequence>
    <xs:element name="Street" type="xs:string"/>
    <xs:element name="City" type="xs:string"/>
  </xs:sequence>
</xs:complexType>

<xs:complexType name="CustomerType">
  <xs:sequence>
    <xs:element name="Name" type="xs:string"/>
    <xs:element name="Address" type="AddressType"/>
  </xs:sequence>
</xs:complexType>

<xs:element name="Order">
  <xs:complexType>
    <xs:sequence>
```

```

    <xs:element name="Customer" type="CustomerType"/>
  </xs:sequence>
</xs:complexType>
</xs:element>

```

Pros: maximum type reuse, clean separation of types from elements, extensible via type derivation.

Cons: more files to manage, requires type naming discipline.

Used by: most enterprise standards — UBL, HL7 CDA, ISO 20022, SWIFT, XBRL. This is the dominant pattern in professional XML.

Converting to JSON Schema: named types map directly to \$defs with \$ref. This is the cleanest conversion because the type structure already matches JSON Schema's reference model.

29.2.4 4. Garden of Eden

Both types AND elements are global. The combination of Salami Slice and Venetian Blind — maximum reuse and substitutability:

```

<xs:complexType name="AddressType">
  <xs:sequence>
    <xs:element ref="Street"/>
    <xs:element ref="City"/>
  </xs:sequence>
</xs:complexType>

<xs:element name="Street" type="xs:string"/>
<xs:element name="City" type="xs:string"/>
<xs:element name="Address" type="AddressType"/>

```

Pros: ultimate reuse — both types and elements are independently referenceable. Supports the most advanced XSD features (substitution groups, type derivation, element polymorphism).

Cons: the most complex pattern to read and maintain. Requires careful namespace management.

Used by: W3C specifications, meta-standards, large-scale schema architectures.

29.2.5 5. Swiss Army Knife (Hybrid)

A pragmatic mix of global and local types and elements. Use whatever works for each part — named types for reusable structures, inline types for one-off elements:

```

<!-- Named type for reuse -->
<xs:complexType name="AddressType">
  <xs:sequence>
    <xs:element name="Street" type="xs:string"/>
    <xs:element name="City" type="xs:string"/>
  </xs:sequence>
</xs:complexType>

<!-- Inline type (one-off) -->
<xs:element name="Order">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="OrderId" type="xs:string"/>
      <xs:element name="ShippingAddress" type="AddressType"/>
      <xs:element name="Notes">

```

```

        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:maxLength value="500"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
</xs:element>

```

Pros: practical, flexible, reflects how most people actually write schemas.

Cons: inconsistent — harder to tool and convert because the pattern varies within the same file.

Used by: most hand-written schemas, internal enterprise schemas, schemas that evolved organically.

29.3 Advanced Patterns

29.3.1 Chameleon Include

A schema file with no `targetNamespace` that adopts the namespace of whatever schema includes it. Like a chameleon changing color:

```

<!-- common-types.xsd (no targetNamespace) -->
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:complexType name="AddressType">...</xs:complexType>
</xs:schema>

<!-- order.xsd (has targetNamespace) -->
<xs:schema targetNamespace="urn:example:orders" ...>
  <xs:include schemaLocation="common-types.xsd"/>
  <!-- AddressType now belongs to urn:example:orders -->
</xs:schema>

```

This allows the same reusable types file to be included by multiple schemas with different namespaces. UTL-X resolves chameleon includes correctly — the types are assigned the including schema's namespace.

29.3.2 Abstract Types

Uses abstract complex types and substitution groups to achieve polymorphism — the XSD equivalent of interfaces and inheritance:

```

<xs:complexType name="PaymentType" abstract="true"/>

<xs:complexType name="CreditCardPayment">
  <xs:complexContent>
    <xs:extension base="PaymentType">
      <xs:sequence>
        <xs:element name="CardNumber" type="xs:string"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

<xs:complexType name="BankTransferPayment">
  <xs:complexContent>
    <xs:extension base="PaymentType">

```

```

<xs:sequence>
  <xs:element name="IBAN" type="xs:string"/>
</xs:sequence>
</xs:extension>
</xs:complexContent>
</xs:complexType>

```

When converting to JSON Schema, abstract types become **oneOf** compositions. When converting to Protobuf, they become **oneof** groups. UTL-X preserves the inheritance structure through USDL.

29.4 The Pattern Matrix

Every XSD pattern is determined by three structural decisions: whether elements are global or local, whether types are global (named) or local (anonymous), and how composition is handled (include, import, abstract). The full matrix:

Pattern	Elements	Types	Composition	Reusability
1. Russian Doll	Local (nested)	Local (anonymous)	None	None — self-contained
2. Salami Slice	Global	Local (anonymous)	ref to global elements	Element reuse
3. Venetian Blind	Local	Global (named)	type references	Type reuse
4. Garden of Eden	Global	Global (named)	ref + type references	Both reusable
5. Swiss Army Knife	Mixed	Mixed	Mixed	Varies per section
6. Chameleon	Local	Global (named)	Namespace-less include	Cross-namespace reuse
7. Abstract	Global	Global (abstract)	Substitution groups	Polymorphic reuse
8. Adapter	Local	Global (wrapping)	import + wrapper types	Cross-schema bridging

The first four are the classical patterns described by the XML Schema community. Patterns 5-8 are pragmatic variations found in real-world enterprise schemas. The three parameters that determine the pattern:

1. **Element scope** — are elements declared at the schema top level (global) or nested inside other types (local)?
2. **Type scope** — are complex types named at the schema top level (global/named) or defined inline where they're used (local/anonymous)?
3. **Composition mechanism** — how are components connected? Element **ref**, type name reference, **xs:include**, **xs:import**, substitution groups, or a mix?

29.5 Pattern Detection

UTL-X can analyze an XSD and identify which pattern it uses. This is useful when:

- Receiving an unfamiliar schema from a partner or vendor
- Choosing the best conversion strategy to another format
- Deciding how to refactor a schema for better reusability

The analysis looks at the ratio of global vs local elements and types:

Pattern	Global elements	Global types
Russian Doll	1 (root only)	0 (all inline)
Salami Slice	Many	Few (inline)
Venetian Blind	1 (root only)	Many
Garden of Eden	Many	Many
Swiss Army Knife	Mixed	Mixed

29.5.1 Accessing Pattern Metadata with ^

When UTL-X parses an XSD, the parser attaches metadata to the UDM tree — including the detected pattern. Access it with the ^ prefix (see Chapter 8):

```
%utlx 1.0
input xsd
output json
---
{
  pattern: $input.^xsdPattern,           // "venetian-blind", "russian-doll", etc.
  version: $input.^xsdVersion,           // "1.0" or "1.1"
  globalElements: $input.^xsdGlobalElements, // "3"
  globalTypes: $input.^xsdGlobalTypes,    // "12"
  inlineTypes: $input.^xsdInlineTypes    // "0"
}
```

Available XSD metadata keys:

Metadata key	Value
^schemaType	UDM node type: "xsd-schema", "xsd-element", "xsd-complexType", etc.
^xsdPattern	Detected design pattern: "russian-doll", "venetian-blind", "salami-slice", "garden-of-eden", "swiss-army-knife"
^xsdVersion	XSD version: "1.0" or "1.1"
^xsdGlobalElements	Count of global element declarations (as string)
^xsdGlobalTypes	Count of global type declarations (as string)
^xsdInlineTypes	Count of anonymous/inline type declarations (as string)
^xsdElementDeclaration	Per-element: "global" or "local"
^xsdTypeDeclaration	Per-type: "global" or "local"

This metadata is transient — it exists during transformation but is never included in the output. It's the parser telling you about the schema's structure so you can make decisions in your transformation.

29.6 Pattern Conversion

UTL-X can convert between patterns. The most common conversion: Russian Doll to Venetian Blind — extracting inline anonymous types into named reusable types.

```
%utlx 1.0
input xsd
output xsd {pattern: "venetian-blind"}
---
$input
```

This reads a Russian Doll XSD and outputs a Venetian Blind XSD — same structure, same validation rules, but with named types that can be reused and referenced. The reverse (Venetian Blind to Russian Doll) inlines all named types back into their usage points.

29.7 Patterns in Cross-Format Conversion

The XSD pattern directly affects the quality of cross-format conversion:

XSD Pattern	JSON Schema result	Quality
Russian Doll	Deeply nested inline, no <code>\$defs</code>	Works but not reusable
Venetian Blind	Clean <code>\$defs</code> with <code>\$ref</code>	Excellent — idiomatic JSON Schema
Salami Slice	<code>\$defs</code> from global elements	Good
Garden of Eden	Rich <code>\$defs</code> with both types and elements	Good but verbose
Swiss Army Knife	Mixed inline and <code>\$ref</code>	Varies by section

If you're converting enterprise XSD to JSON Schema, Avro, or Protobuf, Venetian Blind produces the best results. If the source is Russian Doll, consider converting to Venetian Blind first (output `xsd {pattern: "venetian-blind"}`), then converting to the target format.

29.8 Real-World XSD Architectures

29.8.1 UBL (Universal Business Language)

UBL uses Venetian Blind with a deep type hierarchy:

```
UBL-Invoice-2.1.xsd
├─ imports: UBL-CommonAggregateComponents-2.1.xsd
│   └─ imports: UBL-CommonBasicComponents-2.1.xsd
│       └─ imports: UBL-UnqualifiedDataTypes-2.1.xsd
│           └─ imports: CCTS_CCT_SchemaModule-2.1.xsd
```

Five levels of imports, hundreds of named types. UTL-X resolves the entire chain and makes all types available in a single type graph.

29.8.2 ISO 20022 (Financial Messaging)

ISO 20022 (SWIFT MX) uses Venetian Blind with strict naming conventions — every type name ends with a version number (`CustomerCreditTransferInitiationV11`). Hundreds of message types share common component types. UTL-X handles the full ISO 20022 library.

29.8.3 HL7 CDA (Clinical Document Architecture)

Healthcare's CDA uses a mix of Venetian Blind and abstract types for clinical document extensibility. Abstract types (`ActRelationship`, `Participation`) are specialized via restriction — the XSD equivalent of Java interfaces. UTL-X preserves the inheritance structure during conversion.

29.9 XSD Patterns in Integration Tools

Different integration tools generate and consume XSD in different patterns. Knowing which pattern a tool uses helps you choose the right conversion strategy when migrating or integrating:

Tool	XSD pattern	Notes
TIBCO BusinessWorks	Russian Doll	BW's schema editor generates deeply nested inline types. One global root element with all types anonymous. Simple to read but no reuse — types are locked inside their parent element.

		When migrating from BW, convert to Venetian Blind first for cleaner downstream conversion.
IBM WTX / Mercator	Venetian Blind	Named types with explicit type trees. Strong separation between structure and content.
SAP PI/PO	Venetian Blind + imports	IDoc XSDs use named types across multiple imported schema files. Heavy use of <code>xs:import</code> for cross-namespace composition.
MuleSoft DataWeave	Venetian Blind	Anypoint Platform generates Venetian Blind from RAML/OAS type definitions.
Microsoft BizTalk	Russian Doll (default)	BizTalk's schema editor defaults to Russian Doll with a flat editor view. Can be configured for other patterns but rarely is.
Oracle SOA Suite	Garden of Eden	Uses both global elements and global types for maximum substitutability in BPEL processes.
WSO2	Venetian Blind	WSDL-first approach with named types.

30 Cross-Format Patterns

The core promise of UTL-X is format agnosticism: write transformation logic once, apply it regardless of whether the data arrives as XML, JSON, CSV, or YAML. This chapter covers the practical patterns for converting between formats — the common conversions, the edge cases, and the strategies for multi-format integration flows.

30.1 The Power of Format Agnosticism

In a traditional integration stack, you write different code for each format pair: an XSLT for XML-to-XML, a jq script for JSON-to-JSON, a Python script for CSV-to-JSON, and custom code for everything else. Six formats yield 30 possible pairs — each with its own tool and syntax.

In UTL-X, one language handles all of them. The UDM (Chapter 10) normalizes every format into the same tree structure. Your transformation works on the UDM, and the serializer produces whatever output format you declare:

```
%utlx 1.0
input xml          // could be json, csv, yaml – same body works
output json        // could be xml, csv, yaml – same body works
---
{
  orderId: $input.Order.OrderId,
  customer: $input.Order.Customer,
  total: toNumber($input.Order.Total)
}
```

Change `input xml` to `input json` and the same transformation works — because `$input.Order.OrderId` navigates the UDM tree, not the XML DOM. The format is a declaration, not a dependency.

30.2 XML to JSON

The most common enterprise conversion. Every modernization project eventually needs it.

30.2.1 Pass-Through (Identity)

The simplest conversion — no transformation logic at all:

```
%utlx 1.0
input xml
output json
---
$input
```

Or from the command line:

```
cat order.xml | utlx --from xml --to json
```

UTL-X parses the XML into UDM and serializes it as JSON. Element names become property names, text content becomes values, repeated elements become arrays.

30.2.2 What Happens to XML Features in JSON

XML has features that JSON lacks. Here's how each one translates:

XML feature	JSON representation	Notes
Child elements	Object properties	Direct mapping

Text content	String/number value	Auto-unwrapped from <code>_text</code>
Attributes (non-leaf)	@attr properties	Always preserved
Attributes (leaf+text)	Dropped (default)	<code>writeAttributes: true</code> to preserve
Repeated elements	Arrays	Auto-detected
Namespaces	Prefixed names	<code>cbc:ID</code> accessed via bracket notation
CDATA	String value	CDATA markers stripped
Comments	Dropped	Not representable in JSON
Processing instructions	Dropped	Not representable in JSON

To preserve leaf element attributes (like `currency="EUR"` on `<Total>299.99</Total>`), use output `json {writeAttributes: true}`. Chapter 23 covers the attribute handling design in detail.

30.2.3 Reshaping During Conversion

Usually you don't want a 1:1 XML-to-JSON conversion — you want to reshape:

```
%utlx 1.0
input xml
output json
---
map($input.Orders.Order, (order) -> {
  id: order.@id,
  customer: order.Customer,
  total: toNumber(order.Total),
  lines: map(order.Lines.Line, (line) -> {
    product: line.Product,
    quantity: toNumber(line.Qty),
    price: toNumber(line.Price)
  })
})
```

The XML hierarchy is flattened, renamed, and retyped in one step.

30.3 JSON to XML

The reverse — common when feeding data into enterprise systems that expect XML.

30.3.1 Pass-Through

```
%utlx 1.0
input json
output xml {root: "Order"}
---
$input
```

The `root` option specifies the XML root element name. Without it, UTL-X uses the first property name as the root.

30.3.2 Creating Attributes

JSON has no concept of attributes. To create XML attributes, use the `@` prefix in property names:

```
%utlx 1.0
input json
output xml
```

```

---
{
  Order: {
    "@id": $input.orderId,
    "@status": $input.status,
    Customer: $input.customerName,
    Total: $input.total
  }
}

```

Output:

```

<Order id="ORD-001" status="CONFIRMED">
  <Customer>Alice</Customer>
  <Total>299.99</Total>
</Order>

```

Properties starting with @ become XML attributes on the parent element. Properties without @ become child elements.

30.3.3 Adding Namespaces

For namespace-qualified XML (UBL, SOAP, FHIR):

```

%utlx 1.0
input json
output xml
---
{
  "Invoice": {
    "@xmlns": "urn:oasis:names:specification:ubl:schema:xsd:Invoice-2",
    "@xmlns:cbc": "urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2",
    "cbc:ID": $input.invoiceId,
    "cbc:IssueDate": $input.date,
    "cbc:DocumentCurrencyCode": $input.currency
  }
}

```

Namespace declarations are attributes — use @xmlns and @xmlns:prefix.

30.4 CSV to JSON

Turn flat tabular data into structured JSON:

```

%utlx 1.0
input csv {delimiter: ";"}
output json
---
map($input, (row) -> {
  id: row.OrderId,
  customer: row.Customer,
  amount: toNumber(row.Amount),
  date: parseDate(row.OrderDate, "dd-MM-yyyy")
})

```

CSV rows become JSON objects. The header names become property names. Type conversion is explicit — CSV values are strings by default (except auto-detected numbers and booleans).

30.5 JSON to CSV

Flatten structured JSON into tabular rows:

```
%utlx 1.0
input json
output csv
---
map($input.orders, (order) -> {
  orderId: order.id,
  customer: order.customerName,
  total: order.total,
  status: order.status
})
```

The property names of the first object become CSV headers. The order of properties determines column order.

30.5.1 Flattening Nested JSON

CSV is flat — nested objects must be denormalized:

```
%utlx 1.0
input json
output csv
---
flatten(map($input.orders, (order) ->
  map(order.lines, (line) -> {
    orderId: order.id,
    customer: order.customer,
    product: line.product,
    qty: line.qty,
    lineTotal: line.qty * line.price
  })
))
```

Parent fields repeat for each child row. For complex hierarchies, the `unnest()` function (Chapter 21) automates this.

30.6 XML to CSV

The most complex cross-format conversion — hierarchical to flat. This combines XML parsing with denormalization:

```
%utlx 1.0
input xml
output csv {delimiter: ";", regionalFormat: "european"}
---
flatten(map($input.Orders.Order, (order) ->
  map(order.Lines.Line, (line) -> {
    orderId: order.@id,
    customer: order.Customer,
    product: line.Product,
    quantity: toNumber(line.Qty),
    unitPrice: toNumber(line.Price),
```

```

    lineTotal: toNumber(line.Qty) * toNumber(line.Price)
  })
})

```

XML attributes (@id), text content, and type conversion all happen in one step. The output uses European CSV conventions (semicolons, comma decimals).

30.7 YAML to JSON and JSON to YAML

The simplest conversions — YAML is a superset of JSON, so the structures are equivalent:

```

%utlx 1.0
input yaml
output json
---
$input

```

```

%utlx 1.0
input json
output yaml
---
$input

```

Pass-through is all you need. The only loss: YAML comments and anchors disappear in JSON (JSON has neither). Going JSON to YAML is lossless.

30.8 XML to YAML and YAML to XML

XML to YAML follows the same rules as XML to JSON — the same attribute handling, namespace conventions, and CDATA treatment apply. The only difference is the output serializer:

```

%utlx 1.0
input xml
output yaml
---
$input

```

To preserve leaf element attributes, use `output yaml {writeAttributes: true}` — the same option as JSON. Chapter 23 covers the attribute handling design in detail.

YAML to XML is straightforward — use @ prefixed properties for attributes and namespace declarations:

```

%utlx 1.0
input yaml
output xml
---
$input

```

30.9 Multi-Format Integration Flows

Real-world integrations often involve more than two formats. The canonical pattern: normalize all sources to a common structure, then convert to the target.

30.9.1 Order Processing: XML + CSV + JSON to UBL XML

```
%utlx 1.0
input: orderData xml, pricing csv {delimiter: ";"}, customers json
output xml
---
let order = $orderData.Order
let price = find($pricing, (p) -> p.SKU == order.ProductCode)
let customer = find($customers, (c) -> c.id == order.CustomerId)

{
  Invoice: {
    "@xmlns": "urn:oasis:names:specification:ubl:schema:xsd:Invoice-2",
    "cbc:ID": order.@id,
    "cbc:IssueDate": formatDate(now(), "yyyy-MM-dd"),
    "cac:AccountingCustomerParty": {
      "cac:Party": {
        "cbc:Name": customer?.name ?? "Unknown"
      }
    },
    "cac:InvoiceLine": {
      "cbc:ID": "1",
      "cbc:InvoicedQuantity": order.Quantity,
      "cbc:LineExtensionAmount": toNumber(order.Quantity) * (price?.UnitPrice ?? 0),
      "cac:Item": {
        "cbc:Name": price?.Description ?? order.ProductCode
      }
    }
  }
}
```

Three formats in, one format out. Each source contributes different data — the order structure from XML, pricing from CSV, customer details from JSON. The transformation merges them into a UBL invoice.

30.9.2 Data Lake Export: OData to CSV + JSON + YAML

One source, multiple outputs — same transformation logic, different output declarations:

```
// report.utlx – used three times with different output formats
%utlx 1.0
input odata
output csv // change to json or yaml for other targets
---
map($input, (account) -> {
  id: account.accountid,
  name: account.name,
  revenue: account.revenue,
  country: account.address1_country
})
```

```
cat accounts.json | utlx report.utlx --to csv > report.csv
cat accounts.json | utlx report.utlx --to json > report.json
cat accounts.json | utlx report.utlx --to yaml > report.yaml
```

Same transformation, three formats. The `--to` flag overrides the output declaration.

30.10 The Canonical Data Model Pattern

For organizations with many integrations, define a canonical (internal) data model and transform to/from it:

```
Source A (XML) → Canonical JSON → Target X (CSV)
Source B (CSV) → Canonical JSON → Target Y (XML)
Source C (YAML) → Canonical JSON → Target Z (OData)
```

Each source has one transformation: source → canonical. Each target has one transformation: canonical → target. Adding a new source means writing one new transformation, not N (one per target). Adding a new target means writing one new transformation, not M (one per source).

With N sources and M targets:

- **Without canonical model:** N x M transformations
- **With canonical model:** N + M transformations

For 5 sources and 5 targets: 25 transformations vs 10. For 10 and 10: 100 vs 20. The canonical model scales.

UTL-X's format agnosticism makes this practical — the canonical format can be JSON, and every source/target conversion is a standard `.utlx` file regardless of the source or target format.

30.11 Format Conversion Quick Reference

From → To	Complexity	Key considerations
XML → JSON	Low	Attributes, namespaces, repeated elements
JSON → XML	Low	Root element name, @ for attributes
CSV → JSON	Low	Type conversion (strings to numbers/dates)
JSON → CSV	Medium	Flatten nested structures first
XML → CSV	Medium	Denormalize hierarchy + type conversion
CSV → XML	Low	Add structure and element names
YAML → JSON	Trivial	Pass-through — lossless
JSON → YAML	Trivial	Pass-through — lossless
XML → YAML	Low	Same as XML → JSON, different serializer
OData → JSON	Low	Strips @odata.* annotations automatically
JSON → OData	Low	Add @odata.context and metadata
Any → CSV	Medium	Must flatten to tabular structure

Part IV

Real-World Applications

Enterprise integration, cloud deployment, and production patterns

31 Enterprise Integration

This chapter shows UTL-X in the real world — where it sits in integration architecture, how it handles the major industry use cases, and how it connects to message brokers and monitoring systems.

31.1 Where UTL-X Fits

In a traditional enterprise integration architecture, data transformation is one component in a larger flow:

Source System → Message Broker → Transformation → Target System
 (SAP) (Service Bus) (UTL-X) (Peppol AP)

UTL-X handles the transformation step — the part where data changes format, structure, and content. It does not handle routing, orchestration, or transport. Those are the message broker's job.

This is a deliberate architectural choice. Full iPaaS platforms (MuleSoft, Boomi, Azure Logic Apps, TIBCO BusinessWorks) bundle routing, transformation, and transport into one monolithic product. UTL-X does one thing well: transformation. You pair it with whatever broker, orchestrator, or transport your organization already uses.

31.1.1 UTL-X vs Full iPaaS

Aspect	Full iPaaS	UTL-X + Broker
Routing	Built-in	Message broker (Service Bus, Pub/Sub, Kafka)
Transformation	Built-in (proprietary DSL)	UTL-X (open source, format-agnostic)
Transport	Built-in connectors	Dapr, native SDKs, HTTP
Deployment	Vendor cloud or on-prem appliance	Container (any cloud, any Kubernetes)
Lock-in	High (proprietary everything)	Low (open source transformation, standard broker)
Cost	\$50K-500K/year	Container compute cost only
Transformation language	Vendor-specific (DataWeave, XSLT, visual mapper)	UTL-X (portable, testable, version-controlled)

The key advantage: your transformation logic is in `.utlx` files that you own, version-control, test, and deploy independently. Moving from Azure to GCP means changing the broker configuration, not rewriting transformations.

31.2 Use Case: European E-Invoicing (Peppol/UBL)

The European e-invoicing mandate requires businesses to send invoices in UBL 2.1 XML via the Peppol network. If your ERP is Dynamics 365, SAP, or any system that produces JSON or proprietary XML, you need a transformation.

31.2.1 The Flow

Dynamics 365 → Service Bus → UTLXe → Peppol Access Point
 (OData JSON) (queue) (→ UBL XML) (AS4 transport)

31.2.2 The Transformation

```
%utlx 1.0
input odata
output xml
---
let inv = $input

{
  "Invoice": {
    "@xmlns": "urn:oasis:names:specification:ubl:schema:xsd:Invoice-2",
    "@xmlns:cac":
      "urn:oasis:names:specification:ubl:schema:xsd:CommonAggregateComponents-2",
    "@xmlns:cbc": "urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2",

    "cbc:CustomizationID":
      "urn:cen.eu:en16931:2017#compliant#urn:fdc:peppol.eu:2017:poacc:billing:3.0",
    "cbc:ProfileID": "urn:fdc:peppol.eu:2017:poacc:billing:01:1.0",
    "cbc:ID": inv.invoicenumber,
    "cbc:IssueDate": formatDate(parseDate(inv.createdon, "yyyy-MM-dd'T'HH:mm:ss'Z'"),
      "yyyy-MM-dd"),
    "cbc:InvoiceTypeCode": "380",
    "cbc:DocumentCurrencyCode": inv.transactioncurrencyid?.isocurrencycode ?? "EUR",

    "cac:AccountingSupplierParty": {
      "cac:Party": {
        "cac:PartyName": { "cbc:Name": "Your Company B.V." },
        "cac:PostalAddress": {
          "cbc:StreetName": "Keizersgracht 42",
          "cbc:CityName": "Amsterdam",
          "cbc:PostalZone": "1015 CR",
          "cac:Country": { "cbc:IdentificationCode": "NL" }
        },
        "cac:PartyTaxScheme": {
          "cbc:CompanyID": "NL123456789B01",
          "cac:TaxScheme": { "cbc:ID": "VAT" }
        }
      }
    },

    "cac:AccountingCustomerParty": {
      "cac:Party": {
        "cac:PartyName": { "cbc:Name": inv.customerid_account?.name ?? "Unknown" },
        "cac:PostalAddress": {
          "cbc:CityName": inv.billto_city ?? "",
          "cac:Country": { "cbc:IdentificationCode": inv.billto_country ?? "NL" }
        }
      }
    },

    "cac:LegalMonetaryTotal": {
      "cbc:LineExtensionAmount": { "@currencyID": "EUR", "_text":
toString(inv.totallineitemamount ?? 0) },
      "cbc:TaxExclusiveAmount": { "@currencyID": "EUR", "_text":
toString(inv.totalamountlessfreight ?? 0) },
      "cbc:TaxInclusiveAmount": { "@currencyID": "EUR", "_text":
toString(inv.totalamount ?? 0) },
      "cbc:PayableAmount": { "@currencyID": "EUR", "_text": toString(inv.totalamount ??
```

```

0) }
  },

  "cac:InvoiceLine": map(inv.invoice_details ?? [], (line) -> {
    "cbc:ID": toString(line.sequencenumber ?? 1),
    "cbc:InvoicedQuantity": { "@unitCode": "EA", "_text": toString(line.quantity ??
1) },
    "cbc:LineExtensionAmount": { "@currencyID": "EUR", "_text":
toString(line.extendedamount ?? 0) },
    "cac:Item": { "cbc:Name": line.productdescription ?? "Product" },
    "cac:Price": { "cbc:PriceAmount": { "@currencyID": "EUR", "_text":
toString(line.priceperunit ?? 0) } }
  })
}
}

```

This is a complete, production-ready Dynamics 365 to Peppol BIS 3.0 transformation. The UBL namespace prefixes use bracket notation ([`"cbc:ID"`] in reading, `"cbc:ID"`: in writing). Amounts use the `_text` + `@currencyID` pattern for elements with both text content and attributes.

31.3 Use Case: Healthcare (HL7 FHIR)

Healthcare data exchange is moving from HL7 v2 (pipe-delimited) and proprietary formats to FHIR R4 (JSON and XML). UTL-X handles FHIR's unique conventions natively.

31.3.1 FHIR's Value Attribute Convention

FHIR XML uses `value` attributes instead of text content — a deliberate design for extensibility:

```

<!-- FHIR style -->
<birthDate value="1955-03-12"/>
<active value="true"/>
<name>
  <family value="Simpson"/>
  <given value="Homer"/>
</name>

```

UTL-X accesses these with the standard `@` prefix:

```

$input.Patient.birthDate.@value    // "1955-03-12"
$input.Patient.active.@value       // "true"
$input.Patient.name[0].family.@value // "Simpson"

```

No special FHIR mode — the `@value` accessor works like any XML attribute.

31.3.2 Proprietary EHR to FHIR R4

```

%utlx 1.0
input json
output json
---
{
  resourceType: "Patient",
  id: $input.patientId,
  meta: { profile: ["http://hl7.org/fhir/StructureDefinition/Patient"] },
  name: [{

```

```

    family: $input.lastName,
    given: [$input.firstName],
    use: "official"
  }],
  birthDate: formatDate(parseDate($input.dob, "dd/MM/yyyy"), "yyyy-MM-dd"),
  gender: if ($input.sex == "M") "male" else if ($input.sex == "F") "female" else
"unknown",
  address: [{
    line: [$input.street],
    city: $input.city,
    postalCode: $input.zipCode,
    country: $input.country
  }],
  telecom: filter([
    if ($input.phone != null) { system: "phone", value: $input.phone, use: "home" } else
null,
    if ($input.email != null) { system: "email", value: $input.email } else null
  ], (t) -> t != null)
}

```

This produces a valid FHIR R4 Patient resource from a flat JSON record. The `filter` at the end removes null telecom entries — FHIR arrays must not contain nulls.

31.4 Use Case: Financial Services (ISO 20022)

ISO 20022 (SWIFT MX) is replacing the legacy SWIFT MT format for international payments. The standard uses complex XSD schemas with hundreds of types.

31.4.1 Payment Initiation (pain.001)

```

%utlx 1.0
input json
output xml
---
{
  "Document": {
    "@xmlns": "urn:iso:std:iso:20022:tech:xsd:pain.001.001.11",
    "CstmrCdtTrfInitn": {
      "GrpHdr": {
        "MsgId": $input.messageId,
        "CreDtTm": formatDate(now(), "yyyy-MM-dd'T'HH:mm:ss"),
        "NbOfTxS": toString(count($input.payments)),
        "CtrlSum": toString(sum(map($input.payments, (p) -> p.amount))),
        "InitgPty": { "Nm": $input.initiator.name }
      },
      "PmtInf": map($input.payments, (pmt) -> {
        "PmtInfId": pmt.id,
        "PmtMtd": "TRF",
        "NbOfTxS": "1",
        "PmtTpInf": { "SvcLvl": { "Cd": "SEPA" } },
        "ReqdExctnDt": { "Dt": pmt.executionDate },
        "Dbtr": { "Nm": $input.debtor.name },
        "DbtrAcct": { "Id": { "IBAN": $input.debtor.iban } },
        "DbtrAgt": { "FinInstnId": { "BICFI": $input.debtor.bic } },
        "CdtTrfTxInf": {
          "PmtId": { "EndToEndId": pmt.endToEndId },
          "Amt": { "InstAmt": { "@Ccy": pmt.currency, "_text": toString(pmt.amount) } },
          "CdtrAgt": { "FinInstnId": { "BICFI": pmt.creditorBic } },

```

```

    "Cdtr": { "Nm": pmt.creditorName },
    "CdtrAcct": { "Id": { "IBAN": pmt.creditorIban } },
    "RmtInf": { "Ustrd": pmt.reference }
  }
}
}
}
}
}

```

ISO 20022 messages have deep nesting with abbreviated element names (CstmrCdtTrfInitn, GrpHdr, NbOfTxS). UTL-X handles the structure directly — no intermediary mapping tool needed.

31.5 Use Case: Retail and E-Commerce

31.5.1 Order Synchronization

Omnichannel retail needs orders from web shops, POS systems, and marketplaces normalized into one format:

```

%utlx 1.0
input json
output json
---
{
  orderId: $input.order_id ?? $input.orderId ?? $input.OrderID,
  source: $input._source ?? "unknown",
  customer: {
    name: $input.customer_name ?? $input.customerName ?? concat($input.first_name ?? "", "
", $input.last_name ?? ""),
    email: $input.customer_email ?? $input.email
  },
  lines: map($input.items ?? $input.line_items ?? $input.orderLines ?? [], (item) -> {
    sku: item.sku ?? item.product_id ?? item.SKU,
    name: item.name ?? item.product_name ?? item.description,
    quantity: toNumber(item.quantity ?? item.qty ?? 1),
    unitPrice: toNumber(item.price ?? item.unit_price ?? item.unitPrice ?? 0)
  }),
  total: toNumber($input.total ?? $input.order_total ?? $input.grandTotal ?? 0),
  currency: $input.currency ?? $input.currency_code ?? "EUR"
}

```

The ?? chains handle the field name variations across different source systems. Shopify uses `line_items`, WooCommerce uses `items`, SAP uses `orderLines` — the canonical transformation normalizes all of them.

31.6 Workflow and Low-Code Integration

Workflow platforms — Azure Logic Apps, Power Automate, n8n, Make (formerly Integromat), Zapier — are increasingly popular for business automation. They excel at orchestration (trigger on event, call API, send email, update database) but have weak transformation capabilities. Their built-in mapping is typically limited to simple field assignment: drag source field A to target field B.

This breaks down when you need:

- Format conversion (XML order → JSON API payload)
- Conditional logic (different VAT rates by country)
- Array operations (map order lines, filter active items, aggregate totals)
- Nested restructuring (flat CSV → hierarchical JSON)
- Cross-field computation (line total = quantity x price)

These are exactly the problems UTL-X solves. The integration pattern: the workflow handles orchestration, UTL-X handles transformation.

31.6.1 Pattern: HTTP Action in a Workflow

Most workflow platforms support calling an HTTP endpoint as a step. UTLXe in HTTP mode is that endpoint:

```
Workflow Step 1: Trigger (new order in Shopify)
Workflow Step 2: Get order data (Shopify API)
Workflow Step 3: HTTP POST to UTLXe → transform to UBL invoice XML
Workflow Step 4: Send invoice to Peppol Access Point
```

Step 3 is one HTTP call — the workflow sends the Shopify JSON to UTLXe, receives UBL XML back. The transformation logic lives in a `.utlx` file, version-controlled and testable — not embedded in the workflow’s visual designer where it’s invisible to code review, untestable, and lost when the workflow is deleted.

31.6.2 Azure Logic Apps / Power Automate

```
Logic App Trigger (Service Bus message)
  → HTTP Action: POST to UTLXe container
  → Send result to target API
```

UTLXe runs as a Container App in the same Azure subscription. The Logic App calls it via HTTP — no connector needed, no marketplace dependency. The transformation is external to the workflow: change the `.utlx` file without touching the Logic App.

31.6.3 n8n / Make / Zapier

These platforms support “Webhook” or “HTTP Request” actions:

- **n8n:** HTTP Request node → UTLXe endpoint
- **Make:** HTTP module → UTLXe endpoint
- **Zapier:** Webhooks by Zapier → UTLXe endpoint

The pattern is identical across platforms: send data, receive transformed data. The workflow doesn’t know or care that UTL-X is involved — it’s just an HTTP call that returns the right format.

31.6.4 Why Not Transform Inside the Workflow?

Workflow platforms offer basic transformation features — “expression builder”, “data mapper”, “code step.” Why not use those?

Aspect	Workflow built-in	UTL-X external
Complexity	Simple field mapping only	Any complexity: conditionals, loops, aggregation
Format support	JSON only (usually)	XML, JSON, CSV, YAML, OData
Testability	Manual — click through the workflow	Automated — conformance suite, CI/CD
Version control	Workflow export (opaque JSON/YAML)	<code>.utlx</code> files in Git
Reuse	Copy-paste between workflows	Shared <code>.utlx</code> library, imported by reference
Debugging	Workflow run history (limited)	IDE with live preview, sample data
Performance	Workflow engine overhead per step	86K msg/s for complex transforms

Lock-in	Platform-specific expressions	Portable across any platform with HTTP
---------	-------------------------------	--

The sweet spot: use the workflow for what it does well (triggering, routing, API calls, human tasks) and UTL-X for what it does well (data transformation). They complement each other.

31.7 RPA (Robotic Process Automation)

RPA platforms — UiPath, Automation Anywhere, Blue Prism, Power Automate Desktop — automate tasks that involve user interfaces: clicking buttons, reading screens, filling forms, scraping web pages. The data they extract is often unstructured or semi-structured: screen-scraped text, PDF tables, email bodies, Excel exports.

This data almost always needs transformation before it can enter a business system. An invoice number scraped from a PDF needs to become a field in a JSON API call. A table copied from a web page needs to become a CSV import. An email body with order details needs to become a structured XML message.

31.7.1 The Pattern: RPA Extracts, UTL-X Transforms

```
RPA Robot → extracts data (screen scrape, PDF parse, email parse)
    → raw data (CSV, JSON, unstructured text)
    → HTTP POST to UTLXe
    → structured output (JSON for API, XML for ERP, CSV for database)
    → RPA Robot continues (API call, form fill, database insert)
```

The RPA robot handles the UI interaction and data extraction. UTL-X handles the structural transformation. The robot doesn't need complex data manipulation logic — it calls UTLXe and gets the right format back.

31.7.2 UiPath Integration

UiPath has an HTTP Request activity that calls external APIs:

```
UiPath Sequence:
1. Read PDF Invoice (Document Understanding)
2. Extract fields → JSON object
3. HTTP Request: POST to UTLXe (JSON → UBL XML)
4. Write file: save UBL invoice
5. Upload to Peppol Access Point
```

Step 3 is the transformation — the robot sends extracted invoice data as JSON, receives a compliant UBL XML invoice back. The mapping logic (field names, VAT calculation, country codes) lives in a `.utlx` file, not in UiPath's expression builder.

31.7.3 Why RPA Needs External Transformation

RPA platforms have basic data manipulation — assign variables, simple string operations, basic JSON parsing. But they struggle with:

- **Format conversion:** RPA extracts data as JSON or DataTable. Producing XML with namespaces, attributes, and nested structures is beyond most RPA expression builders.
- **Complex mapping:** conditional fields, cross-reference lookups, array aggregation — these require a transformation language, not a visual variable assignment.
- **Reusability:** RPA processes are monolithic. Extracting the transformation into a `.utlx` file means the same mapping can be used by multiple robots, workflows, and integrations.
- **Compliance:** regulated industries need auditable, testable transformations. A `.utlx` file in Git with conformance tests is auditable. Logic embedded in a UiPath sequence is not.

31.7.4 Common RPA + UTL-X Scenarios

- **Invoice processing:** robot scrapes invoice PDF → UTL-X transforms to UBL XML → submit to e-invoicing network
- **Employee onboarding:** robot reads HR email → UTL-X transforms to SAP IDoc → create employee in SAP
- **Order entry:** robot scrapes web portal → UTL-X normalizes to canonical order format → push to ERP
- **Report generation:** robot exports database query → UTL-X transforms to CSV with regional formatting → email to finance
- **Legacy migration:** robot reads legacy screen → UTL-X transforms to new system's API format → POST to modern API

In each case, the robot handles what requires a UI (clicking, typing, navigating). UTL-X handles what requires data intelligence (restructuring, converting, enriching).

31.8 Message Broker Integration

UTLXe connects to message brokers via Dapr sidecars or native transports.

A note on Dapr: Dapr is a CNCF open-source project — not an Azure product, despite its Microsoft origins. The same Dapr sidecar runs on Azure, AWS, GCP, and any Kubernetes cluster. You change which broker Dapr talks to by swapping a YAML component configuration file — not by changing any UTLXe code or configuration. UTLXe's transformation is completely broker-agnostic: it receives a message from Dapr and returns a result, regardless of whether Dapr pulled it from Azure Service Bus, AWS SQS, GCP Pub/Sub, Apache Kafka, RabbitMQ, or IBM MQ.

31.8.1 Azure Service Bus

Service Bus Queue → Dapr Sidecar → UTLXe Container → Dapr → Target Queue

UTLXe subscribes to a queue via Dapr's pub/sub binding. Each message triggers a transformation. The result is published to the target queue. Dead letters go to the DLQ for retry or manual inspection.

31.8.2 GCP Pub/Sub

Pub/Sub Topic → Push Subscription → UTLXe (HTTP mode) → Target API

UTLXe's HTTP transport receives Pub/Sub push messages directly — no sidecar needed. Each request contains the message payload; the response is the transformed result.

31.8.3 Apache Kafka

Kafka Topic → Dapr Sidecar → UTLXe → Dapr → Kafka Topic

Same Dapr pattern as Service Bus. UTLXe doesn't know or care whether the broker is Kafka, Service Bus, or RabbitMQ — Dapr abstracts the transport.

31.8.4 AWS SQS and EventBridge

AWS offers several messaging services. The most relevant for UTL-X integration:

SQS (Simple Queue Service) — point-to-point message queue:

SQS Queue → Lambda / ECS Task → UTLXe → SQS Queue / API

Two deployment patterns:

- **Lambda:** an AWS Lambda function receives the SQS message and calls UTLXe via HTTP (UTLXe runs as an ECS service) or via the Python/Node.js wrapper (UTLXe subprocess inside Lambda — cold start is 3s due to JVM)
- **ECS/Fargate:** UTLXe container polls SQS directly via Dapr's AWS SQS binding or the AWS SDK embedded in a Java wrapper

SNS (Simple Notification Service) — pub/sub fan-out:

```
SNS Topic → SQS Subscription → UTLXe → target
                → SQS Subscription → UTLXe → different target (different transform)
```

SNS fans out to multiple SQS queues, each with its own UTLXe container running a different transformation. One event, multiple output formats — the fan-out pattern from Chapter 20.

EventBridge — event-driven routing with content-based filtering:

```
EventBridge Rule → ECS Task (UTLXe) → Target (API Gateway, SQS, Lambda)
```

EventBridge rules can filter events by content before invoking UTLXe — only orders over \$1,000, only events from a specific source, only messages matching a pattern. This moves routing logic out of UTL-X and into the AWS infrastructure.

Amazon MQ — managed ActiveMQ or RabbitMQ:

```
Amazon MQ → JMS consumer → UTLXe (in-process Java SDK) → Amazon MQ
```

Same JMS pattern as described above. Amazon MQ is a managed broker — UTLXe connects via JMS client, no MQ infrastructure to manage.

Kinesis Data Streams — high-throughput event streaming (AWS's Kafka equivalent):

```
Kinesis Stream → Lambda / ECS → UTLXe → Kinesis / S3 / API
```

For high-volume data pipelines — IoT events, clickstreams, log processing. UTLXe transforms the events before they land in S3, Redshift, or a downstream API.

31.8.5 JMS (Java Message Service)

JMS is the enterprise Java messaging standard — used by ActiveMQ, IBM MQ, TIBCO EMS, Oracle AQ, and JBoss/WildFly messaging. Many organizations have years of JMS infrastructure in place.

Since UTLXe is a JVM application, JMS integration uses the Java SDK (Chapter 34) directly — no Dapr, no HTTP, no subprocess. The transformation engine runs in-process inside your JMS consumer:

```
// JMS consumer with embedded UTL-X transformation
MessageConsumer consumer = session.createConsumer(inputQueue);
MessageProducer producer = session.createProducer(outputQueue);

UtlxEngine engine = new UtlxEngine();
engine.initialize(bundlePath);

consumer.setMessageListener(message -> {
    String payload = ((TextMessage) message).getText();
    String result = engine.execute("order-to-invoice", payload);
});
```

```
producer.send(session.createTextMessage(result));
});
```

This is the most efficient JMS pattern — zero network overhead, zero serialization overhead. The message goes from JMS → UTL-X → JMS without leaving the JVM.

For organizations that prefer decoupled deployment, the Java stdio wrapper (Chapter 34) works too — your JMS consumer spawns UTLXe as a subprocess and communicates via protobuf. This allows upgrading UTLXe independently from the JMS application.

```
JMS Queue → Java Consumer → UTLXe (subprocess or in-process) → JMS Queue
      (your app)           (transformation)                (target)
```

JMS integration is particularly relevant for:

- **TIBCO EMS migration:** replace BW mapper activities with UTL-X transformations while keeping the EMS broker
- **IBM MQ environments:** transform MQ messages (often XML/CSV) to JSON for modern APIs
- **ActiveMQ/Artemis:** lightweight message transformation without a full ESB
- **Legacy modernization:** keep the JMS infrastructure, replace the transformation layer

31.8.6 IBM MQ with UTL-X

IBM MQ (formerly MQ Series, then WebSphere MQ) is the dominant message broker in banking, insurance, government, and large manufacturing. Organizations have decades of MQ infrastructure — thousands of queues, millions of messages per day, deeply embedded in business processes. Replacing MQ is rarely an option; modernizing the transformation layer on top of it is.

The typical IBM MQ landscape has transformation challenges at every turn:

- **Queue-to-queue transformations:** messages arrive in one format on queue A, must be delivered in another format on queue B. Traditionally done by IBM Integration Bus (IIB/ACE) or custom Java EJBs.
- **Mainframe integration:** MQ bridges to CICS/IMS on the mainframe. Messages are often fixed-length COBOL copybook format or XML. The modern API expects JSON.
- **Multi-system routing:** one MQ message triggers updates in SAP (IDoc), Dynamics 365 (OData JSON), and a data warehouse (CSV). Three different output formats from one input.
- **Regulatory reporting:** financial messages (ISO 20022, SWIFT) flow through MQ and need transformation for regulatory submissions.

UTL-X replaces the transformation component while MQ stays untouched:

```
IBM MQ Queue → Java Consumer → 
      (source)      (MQ client)      ↗ SAP (IDoc XML)
                                   ↳ D365 (OData JSON)
                                   ↳ Data Warehouse (CSV)
                                   ← UTL-X transforms
```

31.8.6.1 Deployment Options

Option 1: Embedded in MQ client application (direct API)

```
MQueueManager qmgr = new MQueueManager("QM1");
MQueue inputQueue = qmgr.accessQueue("ORDER.IN", MQOO_INPUT_SHARED);
MQueue outputQueue = qmgr.accessQueue("INVOICE.OUT", MQOO_OUTPUT);

UtlxEngine engine = new UtlxEngine();
```

```

engine.initialize(bundlePath);

while (true) {
    MQMessage msg = new MQMessage();
    inputQueue.get(msg);
    String payload = msg.readStringOfByteLength(msg.getMessageLength());

    String result = engine.execute("order-to-invoice", payload);

    MQMessage outMsg = new MQMessage();
    outMsg.writeString(result);
    outputQueue.put(outMsg);
}

```

Runs alongside MQ client libraries. No additional infrastructure. Suitable for dedicated transformation services.

Option 2: Container with MQ JMS connector

IBM MQ → JMS (MQ JMS client) → UTLXe Container → JMS → IBM MQ

UTLXe runs as a container. The JMS connector (IBM's `com.ibm.mq.jakarta.client`) provides the MQ connection. Deployable on Kubernetes, OpenShift (common in MQ shops), or any container platform.

Option 3: IBM MQ + Dapr

Dapr has an IBM MQ binding component. UTLXe receives messages via Dapr without any MQ-specific code:

IBM MQ → Dapr (MQ binding) → UTLXe → Dapr → IBM MQ

This is the most decoupled option — UTLXe doesn't know it's talking to MQ. Useful when migrating away from MQ in the future (swap Dapr binding, UTLXe unchanged).

31.8.6.2 Replacing IBM Integration Bus (IIB/ACE)

Many MQ environments use IBM Integration Bus (formerly Message Broker, now App Connect Enterprise) for transformation. IIB/ACE is powerful but expensive (\$50K-200K/year licensing), complex to operate, and uses a proprietary ESQL language for mapping.

UTL-X can replace the transformation component of IIB/ACE:

Aspect	IBM IIB/ACE	UTL-X + MQ
Licensing	\$50K-200K/year	Open source (AGPL)
Transformation language	ESQL (proprietary)	UTL-X (open, testable)
Deployment	IIB runtime (dedicated server)	Container (any cloud/on-prem)
Format support	XML, JSON, CSV, DFDL	XML, JSON, CSV, YAML, OData + 6 schema formats
Testing	IIB Toolkit (manual)	Conformance suite (automated CI/CD)
Throughput	1,000-10,000 msg/s	20,000-86,000 msg/s (COMPILED)
MQ connection	Native (same product family)	JMS client or Dapr binding

Skill availability	Declining (ESQL expertise rare)	Growing (functional syntax, familiar to JS/Kotlin devs)
--------------------	---------------------------------	---

The migration path: keep MQ, replace IIB/ACE with UTLXe containers, translate ESQL message flows to .utlx files. The MQ queues, topics, and routing stay identical — only the transformation layer changes.

31.8.7 RabbitMQ

RabbitMQ → Dapr Sidecar → UTLXe → Dapr → RabbitMQ

Same Dapr pattern. Alternatively, use the Java SDK with the RabbitMQ Java client for direct, in-process consumption — same approach as JMS.

31.9 Error Handling in Production

31.9.1 Validation Errors

Pre-validation (Chapter 19) catches malformed input before the transformation runs:

Message arrives → Pre-validate → FAIL → Dead Letter Queue
→ PASS → Transform → Post-validate → Output

Dead-lettered messages include the validation error details — which field, which constraint, what value. Operations can inspect and fix without guessing.

31.9.2 Transformation Errors

Use try/catch in transformations for fields that might fail:

```
{
  orderId: $input.orderId,
  orderDate: try { parseDate($input.date, "yyyy-MM-dd") } catch { null },
  amount: try { toNumber($input.total) } catch { 0 }
}
```

A failed date parse or number conversion doesn't crash the entire transformation — it produces a fallback value.

31.9.3 Monitoring

UTLXe exposes Prometheus metrics:

- `utlx_messages_processed_total` — total messages processed (counter)
- `utlx_messages_failed_total` — failed transformations (counter)
- `utlx_transformation_duration_seconds` — processing time (histogram)
- `utlx_active_workers` — currently busy workers (gauge)

The health endpoint (`/health`) returns readiness and liveness status for Kubernetes probes.

31.9.4 Alerting Rules

Typical Prometheus alert rules for UTLXe:

- **Error rate:** alert when `utlx_messages_failed_total / utlx_messages_processed_total > 0.01` (more than 1% failures)
- **Latency:** alert when `utlx_transformation_duration_seconds{quantile="0.99"} > 0.1` (p99 over 100ms)
- **Queue depth:** alert when dead letter queue depth exceeds threshold (broker-specific metric)

- **Health:** alert when `/health` returns non-200 for more than 30 seconds

32 Engine Lifecycle: Design-Time, Init-Time, Runtime

UTLXe engine feature. This chapter covers the lifecycle of transformations in the UTLXe production engine — the three phases, execution strategies, and worker threading model. The CLI (`utlx`) collapses all three phases into a single invocation: parse, execute, serialize, done.

Every UTL-X transformation goes through three distinct phases. Understanding them is key to performance tuning, debugging, and choosing the right execution strategy.

32.1 Phase 1: Design-Time

Where: developer's laptop, VS Code + `utlxd` daemon.

When: while writing the transformation.

What happens:

1. Developer writes a `.utlx` file
2. The daemon (`utlxd`) parses the source in real-time (grammar $\rightarrow$ AST)
3. The IDE shows syntax errors, autocompletion, and type hints
4. Live preview: the transformation runs on sample data with every keystroke
5. Optionally: schema-to-schema mapping with the visual mapper

Design-time is human time — minutes to hours per transformation. The optimization here is a good IDE, schema-driven development, and reusable patterns. AI-assisted generation (Chapter 45) can reduce this significantly.

Artifacts produced:

- `.utlx` file — the transformation source
- `transform.yaml` — engine configuration (strategy, validation policy, schemas)
- Schema files — input/output validation contracts (optional)

32.2 Phase 2: Init-Time (Load and Compile)

Where: UTLXe engine, at startup or on hot-reload.

When: when a transformation is loaded — at startup from a bundle, or dynamically via the API.

What happens:

1. Parse `.utlx` source $\rightarrow$ AST (same parser as design-time)
2. Resolve schemas (if referenced in `transform.yaml`)
3. Select execution strategy:

Strategy	What init does	Init cost
TEMPLATE	Parse to AST — ready to interpret	Instant (milliseconds)
COPY	Build skeleton DOM from output schema, pre-allocate object pool	Fast (tens of milliseconds)
COMPILED	Compile AST $\rightarrow$ JVM bytecode via ASM library	Paid once at init-time (seconds), before the engine reports ready; then cached. Not a first-message cost.
COPY + COMPILED	Build skeleton DOM <i>and</i> compile fill logic to JVM bytecode	Paid once at init-time (seconds), before ready; then cached. Not a first-message cost.
AUTO	Schema present $\rightarrow$ COPY, no schema $\rightarrow$ TEMPLATE	Depends on schema

4. Register transformation in the registry (`ConcurrentHashMap`)
5. Transformation is now ready to accept messages

Init-time is the **most important phase for performance thinking**. A slow init is paid once. A slow runtime is paid per message — thousands or millions of times. Invest init-time to save runtime.

Compilation is **eager**: COMPILED transformations are turned into bytecode while the bundle loads (or at upload time), not on the first message. The engine reports ready only once every transformation is compiled, so the first message already hits compiled code — there is no “slow first message”. The COMPILED init cost is a one-time **startup** cost (and, in a scaled deployment, a per-replica startup cost), never a runtime latency spike.

32.2.1 Compilation Cache

The COMPILED strategy generates JVM bytecode from the AST using the ASM bytecode library — the same technology Java itself uses. The generated bytecode is cached by a SHA-256 hash of the `.utlx` source. If the source hasn’t changed, the cached bytecode is reused — no recompilation.

This means (all at load/init-time, before the engine reports ready — never on a message):

- First load of a new transformation: seconds (compilation)
- Subsequent loads of the same transformation: instant (cache hit)
- Changing one character in the source: recompiles (new hash)

32.2.2 Object Pool (COPY Strategy)

The COPY strategy pre-builds a skeleton UDM tree from the output schema at init-time. At runtime, each message gets a clone of the skeleton with values filled in — faster than building the output tree from scratch.

The skeleton is pooled (`ConcurrentLinkedQueue`, 32 slots). Workers take a skeleton from the pool, fill it, serialize the output, and return the skeleton. This avoids garbage collection pressure from creating and discarding thousands of UDM trees per second.

32.3 Phase 3: Runtime (Execute)

Where: UTLXe engine, per incoming message.

When: every time a message arrives — HTTP request, Dapr input binding, gRPC call, stdin.

What happens:

1. Parse input data (XML/JSON/CSV/YAML → UDM)
2. Pre-validate against input schema (if configured)
3. Execute the transformation:

Strategy	How it executes
TEMPLATE	Walk the AST, evaluate each expression (tree-walking interpreter)
COPY	Clone skeleton, fill values from input UDM (fast path, no interpretation)
COMPILED	Call generated JVM bytecode (JIT-optimized by HotSpot)

4. Post-validate against output schema (if configured)
5. Serialize output (UDM → target format)
6. Return result

Runtime cost: 0.01ms (simple pass-through) to 50ms (complex transformation with large messages). The strategy choice determines the constant factor; the message size determines the linear factor.

32.4 How Strategies Compare

Strategy	Init cost	Runtime per message	Best for
TEMPLATE	Instant	Slower (interpret AST)	Development, simple transforms, low volume
COPY	Fast (build skeleton)	Fast (clone + fill)	Schema-driven, high throughput, predictable structures
COMPILED	Seconds at startup (one-time, before ready)	Fastest (JVM bytecode)	Maximum throughput, complex logic, hot paths
COPY + COMPILED	Seconds at startup (one-time, before ready)	Fastest possible	Production: skeleton for structure, bytecode for logic
AUTO	Depends	Depends	Production default — lets the engine choose

32.4.1 The COPY + COMPILED Hybrid

The fastest execution path combines both strategies:

- **COPY** builds the output skeleton at init-time — the structure is pre-allocated
- **COMPILED** generates bytecode for the fill logic — expressions are JVM-optimized
- **Runtime:** clone the skeleton (fast) + run compiled fill code (fast) = fastest possible path

This is not a separate **strategy**: value — it is what **COPY** does automatically: selecting **COPY** (or letting **AUTO** choose it) also compiles the fill logic to bytecode where possible. This combined path is what carries transformations that process 86,000+ messages per second.

32.5 The Three Phases Visualized

DESIGN-TIME (human)

Developer → VS Code → utlxd → .utlx file + schemas
Cost: minutes to hours, paid once per transformation

| .utlx file + transform.yaml + schemas



INIT-TIME (engine startup)

UTLXe loads .utlx → parse → compile/prepare → register
Cost: milliseconds to seconds, paid once per engine start

TEMPLATE: AST ready (instant)

COPY: build skeleton + object pool (fast)

COMPILED: AST → bytecode via ASM (one-time at startup, before ready; cached after)

| registered transformation, ready for messages



RUNTIME (per message)

Message → parse → validate → transform → serialize → output
Cost: 0.01-50ms per message, paid millions of times

8-128 workers process messages concurrently

Back-pressure: queue full → caller waits

32.6 Worker Threading Model

UTLXe uses a fixed-size worker thread pool for concurrent message processing:

```
Message Queue → [Worker 1] → transform → output
               → [Worker 2] → transform → output
               → [Worker 3] → transform → output
               → ...
               → [Worker N] → transform → output
```

Each worker is independent — no shared mutable state between workers. A transformation’s execution is completely isolated: each worker has its own environment, its own UDM tree, its own output buffer.

32.6.1 Back-Pressure

The worker pool uses an `ArrayBlockingQueue` with `CallerRunsPolicy`:

- Messages enter the queue
- Workers take messages from the queue and process them
- When the queue is full (all workers busy, queue at capacity): the **calling thread** executes the work itself
- This prevents unbounded queue growth (no out-of-memory risk)
- Natural flow control: the producer (message broker) slows down when workers are saturated

32.6.2 Tuning Workers

The `--workers` flag controls the pool size:

```
utlxe --bundle transforms/ --workers 32
```

Rules of thumb:

- **Start with 8 workers per vCPU.** A 4-vCPU container: 32 workers.
- **Increase for I/O-heavy transformations** — if workers wait on downstream HTTP calls, more workers keep the CPU busy while others wait.
- **Decrease for memory-heavy transformations** — large XML documents (10MB+) consume significant heap per worker. Fewer workers = less concurrent memory pressure.
- **Monitor:** watch `utlxe_active_workers` (Prometheus gauge) and queue depth. If workers are always at capacity, add more. If they’re mostly idle, reduce to save memory.

32.6.3 Thread Safety

UTL-X transformations are inherently thread-safe because they are **purely functional**:

- No mutable variables (all bindings are `let`, not `var`)
- No side effects (no file I/O, no database writes, no network calls from within the transformation)
- No shared state (each execution gets its own environment)
- `$input` is read-only (the UDM tree is never modified)

This means any transformation can run safely on any number of concurrent workers without locks, synchronization, or race conditions. The engine guarantees isolation; the language guarantees purity.

33 Cloud Deployment

UTLX engine feature. Cloud deployment uses the UTLX engine, packaged as a Docker container. The CLI (`utlx`) is for development; UTLX is for production.

UTLX is designed for containerized deployment on any cloud platform. The Docker image runs on Azure Container Apps, GCP Cloud Run, AWS ECS/Fargate, or any Kubernetes cluster. This chapter covers deployment on each platform, health monitoring, and operational patterns.

33.1 The Docker Image

UTLX is published as a container image:

```
docker pull ghcr.io/utlx-lang/utlx:latest
```

The image contains:

- UTLX engine (JVM-based, GraalVM runtime)
- All format parsers and serializers (XML, JSON, CSV, YAML, OData + 6 schema formats)
- Full stdlib (652 functions)
- Health endpoint and Prometheus metrics

33.1.1 Transport Modes

UTLX supports multiple transport modes for receiving messages:

Mode	Flag	Use case
HTTP	<code>--mode http</code>	REST API, Cloud Run, webhooks, Pub/Sub push
gRPC	<code>--mode grpc</code>	High-throughput service-to-service
Stdio (protobuf)	<code>--mode stdio-proto</code>	Dapr sidecar, Container Apps
Stdio (JSON)	<code>--mode stdio-json</code>	Testing, simple integrations

33.1.2 Basic Docker Run

```
docker run -p 8080:8080 \
  -v ./transforms:/transforms \
  ghcr.io/utlx-lang/utlx:latest \
  --bundle /transforms \
  --mode http \
  --workers 8
```

This starts UTLX in HTTP mode, loads all transformations from the `/transforms` directory, and processes messages with 8 worker threads.

33.2 Azure Deployment

UTLX is available on the Azure Marketplace as a Managed Application, deployable directly from the Azure Portal.

33.2.1 Azure Container Apps

The recommended Azure deployment uses Container Apps with a Dapr sidecar:

```
Azure Service Bus → Dapr Sidecar → UTLX Container → Dapr → Target
(queue)           (input binding) (transform)   (output binding)
```

The Bicep template provisions:

- Container Apps Environment with Dapr enabled
- UTLXe container with configurable workers
- Dapr components for Service Bus or Event Hub bindings
- KEDA auto-scaling based on queue depth or HTTP concurrency
- Managed identity (no credentials in the container)

33.2.2 Marketplace Plans

Plan	Price	Workers	Use case
Starter	\$35/month	Up to 4	Development, testing, low-volume integration
Professional	\$105/month	Up to 32	Production workloads, multi-pipeline

Pricing covers the UTLXe license. Azure compute costs (Container Apps, Service Bus) are separate and billed by Azure directly.

33.2.3 Deployment via Azure Portal

1. Search “UTL-X” in Azure Marketplace
2. Select the plan (Starter or Professional)
3. Configure: resource group, container settings, worker count
4. Deploy — the Bicep template creates all resources

33.2.4 Deployment via CLI

```
az deployment group create \
  --resource-group myRG \
  --template-file deploy/azure/main.bicep \
  --parameters workers=8 plan=starter
```

33.3 GCP Deployment

33.3.1 Cloud Run

The simplest GCP deployment — serverless, scales to zero, pay per request:

```
gcloud run deploy utlxe \
  --image ghcr.io/utlx-lang/utlxe:latest \
  --port 8080 \
  --args="--mode,http,--bundle,/transforms,--workers,4" \
  --memory 512Mi \
  --cpu 1 \
  --min-instances 0 \
  --max-instances 10
```

Cloud Run handles auto-scaling, TLS, and load balancing. UTLXe’s HTTP mode receives requests directly — no sidecar needed.

33.3.2 Pub/Sub Integration

GCP Pub/Sub can push messages directly to Cloud Run via HTTP:

```
Pub/Sub Topic → Push Subscription → Cloud Run (UTLXe) → Target API
```

Configure a push subscription with the Cloud Run URL as the endpoint. Each Pub/Sub message becomes an HTTP POST to UTLXe. The response is the transformed result.

33.3.3 Terraform Module

A Terraform module is included for infrastructure-as-code deployment:

```
module "utlxe" {
  source = "../deploy/gcp"

  project_id = "my-project"
  region     = "europe-west1"
  workers    = 8
  plan       = "starter"
}
```

This provisions Cloud Run, IAM roles, and optional Pub/Sub integration.

33.4 AWS Deployment

33.4.1 ECS/Fargate

AWS deployment uses ECS with Fargate (serverless containers):

```
SQS Queue → EventBridge → ECS Task (UTLXe) → Target
```

The CloudFormation template provisions:

- ECS cluster with Fargate launch type
- Task definition with UTLXe container
- Application Load Balancer (for HTTP mode)
- SQS queue and EventBridge rule (for event-driven mode)
- Auto-scaling based on queue depth or CPU utilization

33.4.2 Pricing Note

AWS adds a 20% container overhead compared to Azure and GCP for equivalent compute. Factor this into cost planning: a \$35 Starter workload on Azure costs approximately \$44 on AWS.

33.5 Kubernetes (Any Cloud)

For organizations running their own Kubernetes clusters:

33.5.1 Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: utlxe
spec:
  replicas: 2
  selector:
    matchLabels:
      app: utlxe
  template:
    metadata:
      labels:
        app: utlxe
    spec:
      containers:
        - name: utlxe
          image: ghcr.io/utlx-lang/utlxe:latest
          args: ["--mode", "http", "--bundle", "/transforms", "--workers", "8"]
```

```
ports:
- containerPort: 8080
  name: http
- containerPort: 8081
  name: metrics
livenessProbe:
  httpGet:
    path: /health/live
    port: 8080
  initialDelaySeconds: 5
readinessProbe:
  httpGet:
    path: /health/ready
    port: 8080
  initialDelaySeconds: 10
resources:
  requests:
    memory: "256Mi"
    cpu: "500m"
  limits:
    memory: "512Mi"
    cpu: "1000m"
volumeMounts:
- name: transforms
  mountPath: /transforms
volumes:
- name: transforms
  configMap:
    name: utlxe-transforms
```

33.5.2 Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: utlxe-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: utlxe
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

33.5.3 Dapr on Kubernetes

Add Dapr annotations for message broker integration:

```
metadata:
  annotations:
    dapr.io/enabled: "true"
    dapr.io/app-id: "utlxe"
    dapr.io/app-port: "8080"
```

Dapr provides the pub/sub binding — UTLXe receives messages via Dapr, transforms them, and publishes results via Dapr. The broker (Kafka, RabbitMQ, Service Bus) is configured in Dapr components, not in UTLXe.

33.6 Health and Monitoring

33.6.1 Health Endpoints

UTLXe exposes two health endpoints on the main port:

- `/health/live` — liveness probe: is the process alive?
- `/health/ready` — readiness probe: is the engine initialized and ready to accept messages?

Use these for Kubernetes probes, Cloud Run health checks, and load balancer health monitoring.

33.6.2 Prometheus Metrics

UTLXe exposes Prometheus metrics on port 8081:

```
# Total messages processed
utlxe_messages_processed_total{transformation="order-to-invoice"} 42531

# Failed transformations
utlxe_messages_failed_total{transformation="order-to-invoice"} 12

# Processing duration (histogram)
utlxe_transformation_duration_seconds_bucket{le="0.01"} 38000
utlxe_transformation_duration_seconds_bucket{le="0.05"} 42000
utlxe_transformation_duration_seconds_bucket{le="0.1"} 42500

# Active workers (gauge)
utlxe_active_workers 6
```

33.6.3 Grafana Dashboard

A typical UTLXe Grafana dashboard shows:

- **Throughput:** messages per second (rate of `utlxe_messages_processed_total`)
- **Error rate:** failures as percentage of total
- **Latency:** p50, p95, p99 from the duration histogram
- **Workers:** active vs total, queue depth
- **Per-transformation:** breakdown by transformation name (label)

33.6.4 Integration with Cloud Monitoring

- **Azure Monitor:** Container Apps metrics are collected automatically. Add Prometheus scraping via Azure Monitor managed Prometheus for UTLXe-specific metrics.
- **GCP Cloud Monitoring:** Cloud Run metrics are built-in. Use the Prometheus sidecar pattern or OpenTelemetry for custom metrics.
- **AWS CloudWatch:** ECS task metrics are built-in. Use the CloudWatch agent with Prometheus scraping for UTLXe metrics.

33.7 Hot Reload

UTLXe supports loading transformations at runtime without restart:

33.7.1 Bundle Loading (Startup)

```
utlxe --bundle /transforms
```

All `.utlx` files and `transform.yaml` configurations in the directory are loaded at startup. This is the standard deployment pattern.

33.7.2 Dynamic Loading (Runtime)

The HTTP API accepts new transformations while the engine is running:

```
curl -X POST http://localhost:8080/api/load \
  -H "Content-Type: application/json" \
  -d '{"id": "new-transform", "source": "...", "config": {...}}'
```

The new transformation is parsed, compiled (if COMPILED strategy), and registered — all without affecting in-flight messages on other transformations.

33.7.3 Stateless Design

UTLXe is stateless — no data is stored in the container. Transformations are loaded from the bundle (mounted volume or ConfigMap) or via the API. This means:

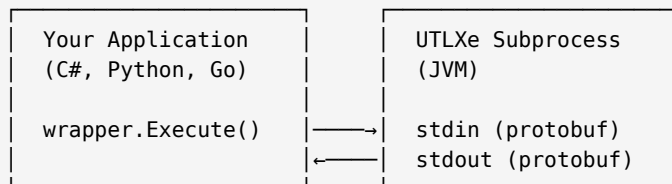
- **Scale horizontally:** add more replicas, each loads the same bundle
- **Rolling updates:** replace containers one at a time, no coordination needed
- **Recovery:** a crashed container restarts and reloads the bundle automatically
- **No sticky sessions:** any replica can handle any message

34 SDKs and Wrapper Libraries

UTLX_e is a JVM process written in Kotlin. But not every application is JVM — your service might be C#, Python, Go, Rust, or Node.js. This chapter covers how non-JVM applications embed UTL-X transformation capabilities.

34.1 The Wrapper Pattern

The wrapper pattern runs UTLX_e as a subprocess of your application, communicating via protobuf-encoded messages over stdin/stdout:



The wrapper library handles process lifecycle (start, health-check, restart), message framing (varint-delimited protobuf), and correlation IDs (concurrent Execute calls multiplexed on one process).

34.1.1 Why Stdio Instead of HTTP?

Aspect	Stdio (protobuf)	HTTP
Latency	Sub-millisecond	1ms (TCP overhead)
Port management	None — no port conflicts	Requires available port
Sandboxed environments	Works (no network)	May be blocked by firewall
Process lifecycle	Natural binding — parent kills child	Requires separate process management
Concurrency	Multiplexed via correlation IDs	Native HTTP concurrency
Complexity	Requires SDK	Any HTTP client works

Stdio is faster and simpler for embedded use. HTTP is simpler for polyglot/shared-service use. Both are available — choose based on your architecture.

34.2 Available SDKs

34.2.1 C# / .NET Wrapper

Status: Built and tested. Targets .NET 9 and .NET 10.

```

// Install: dotnet add package UtlxClient

var client = new UtlxeClient();

// Load a transformation
await client.LoadAsync("order-to-invoice", utlxSource);

// Execute a transformation
var result = await client.ExecuteAsync("order-to-invoice", jsonPayload);
Console.WriteLine(result);

// Batch processing
var results = await client.BatchAsync("order-to-invoice", payloads);
  
```

```
// Pipeline (multi-step)
var output = await client.PipelineAsync(new[] {
    ("normalize", null),
    ("enrich", enrichmentData),
    ("generate-invoice", null)
}, inputPayload);

// Cleanup
await client.UnloadAsync("order-to-invoice");
client.Dispose();
```

Features:

- `UtlxeClient` — high-level async API: Load, Execute, Batch, Pipeline, Unload
- `UtlxeProcess` — JVM subprocess lifecycle management (start, health, restart)
- `VarintCodec` — varint-delimited protobuf framing
- Correlation ID multiplexing — concurrent Execute calls on one subprocess
- Automatic process recovery — restarts UTLXe if it crashes

Use cases:

- Azure Functions: HTTP trigger → UTL-X transform → response
- ASP.NET Web API: transformation middleware
- Console applications: batch file conversion
- Dynamics 365 plugins: transform OData payloads

34.2.2 Go Wrapper

Status: Built and tested. 12 integration tests.

```
import "github.com/utlx-lang/utlxclient"

// Create client (starts UTLXe subprocess)
client, err := utlxclient.New()
defer client.Close()

// Load transformation
err = client.Load("order-to-invoice", utlxSource)

// Execute
result, err := client.Execute("order-to-invoice", jsonPayload)
fmt.Println(result)

// Batch
results, err := client.Batch("order-to-invoice", payloads)

// Health check
status, err := client.Health()
```

Features:

- Full client API: New, Close, Load, Execute, Batch, Pipeline, Unload, Health
- Protobuf over stdio communication
- Thread-safe — safe for concurrent goroutines

Use cases:

- Go microservices requiring data transformation
- Open-M controller integration

- CLI tools written in Go

34.2.3 Java / Kotlin

Java has two integration options — each with different trade-offs:

34.2.3.1 Option 1: Direct API (In-Process)

UTLXe IS Java/Kotlin — import the library and call it directly:

```
val engine = UtlxEngine()
engine.initialize(bundlePath)
val result = engine.execute("order-to-invoice", inputPayload)
```

Advantage: zero latency, zero IPC overhead, direct method calls.

Disadvantage: compile-time binding. Your application depends on UTLXe's JAR and all its transitive dependencies (SnakeYAML, Jackson, ASM, Apache XML Security). Upgrading UTLXe means recompiling and redeploying your application. Dependency conflicts — your app uses Jackson 2.15, UTLXe needs 2.17 — can break the build. Your app and UTLXe share the same JVM heap, the same classpath, the same lifecycle.

34.2.3.2 Option 2: Java Stdio Wrapper (Decoupled)

The same subprocess pattern as C# and Go — but from Java:

```
var client = new UtlxeClient();

// Same interface as C# and Go
client.load("order-to-invoice", utlxSource);
var result = client.execute("order-to-invoice", jsonPayload);
var results = client.batch("order-to-invoice", payloads);
client.unload("order-to-invoice");
client.close();
```

Advantage: no compile-time binding. UTLXe runs as a separate process — upgrade it by swapping the JAR without recompiling your app. No dependency conflicts. No shared heap. Hot-swappable: stop the subprocess, start a new version, resume processing. And critically: the Java SDK uses the **same interface and protocol** as C#, Go, and all future SDKs.

Disadvantage: 0.5ms IPC overhead per call (protobuf serialization + stdio). Negligible for most use cases, but measurable at 100K+ calls per second.

34.2.3.3 Which to Choose?

Criterion	Direct API	Java Wrapper
Latency	Zero	Sub-millisecond
Dependency coupling	Tight (shared classpath)	None (separate process)
UTLXe upgrades	Recompile + redeploy app	Swap JAR, restart subprocess
Dependency conflicts	Possible (shared JVM)	Impossible (separate JVM)
Hot-swap	No (same process)	Yes (restart subprocess)
Memory isolation	Shared heap	Separate heaps
Best for	Embedded, maximum throughput	Microservices, production ops

For production microservices, the wrapper is usually the better choice — the sub-millisecond overhead is negligible, and the operational benefits (decoupled upgrades, no dependency conflicts, hot-swap) are

significant. The direct API is for specialized cases where you need absolute maximum throughput and control the entire dependency tree.

34.3 The Uniform SDK Interface

All SDKs — C#, Go, Java wrapper, and future Python and Node.js — implement the same interface contract:

```
Load(id, source)           → ok / error
Execute(id, payload)       → result / error
Batch(id, payloads[])     → results[]
Pipeline(steps[])         → result
Unload(id)                → ok
Health()                  → status
```

This uniformity is deliberate:

- **Same mental model:** a C# team and a Java team use the same API. Documentation covers all SDKs at once.
- **Portable knowledge:** learning the Go wrapper means you already know the C# wrapper. The function names, parameters, and behavior are identical.
- **Consistent behavior:** every SDK communicates via the same protobuf protocol. The error codes, timeout handling, and correlation ID multiplexing are identical across languages.
- **Testable:** integration tests can be written once (as protocol-level expectations) and verified across all SDKs.

The direct Java API is the exception — it bypasses the protocol layer for zero-overhead access. But it exposes a different API surface (engine internals rather than the SDK interface). For teams that want consistency across their polyglot stack, the Java wrapper is preferred even when the host is JVM.

34.3.1 Python Wrapper (Planned)

Not yet built — expected to follow the same stdio-protobuf pattern.

Would enable:

- Jupyter notebook transformations (explore data, transform inline)
- Django/Flask middleware
- Data science pipelines (pandas DataFrame → UTL-X → JSON/CSV)
- AWS Lambda (Python runtime + UTLXe subprocess)

34.3.2 Node.js / TypeScript Wrapper (Planned)

Not yet built — would use `child_process.spawn` + protobuf over stdio.

Would enable:

- Express/Fastify middleware
- Serverless functions (AWS Lambda, Vercel)
- VS Code extension backend

34.4 Alternative: HTTP API (No SDK Needed)

UTLXe in HTTP mode exposes a REST API that any language can call — no SDK required:

```
# Load a transformation
curl -X POST http://localhost:8085/api/load \
  -H "Content-Type: application/json" \
  -d '{"id": "order-transform", "source": "..."}'
```

```
# Execute a transformation
curl -X POST http://localhost:8085/api/execute/order-transform \
  -H "Content-Type: application/json" \
  -d '{"orderId": "ORD-001", "customer": "Acme"}'

# One-shot transform (load + execute in one call)
curl -X POST http://localhost:8085/api/transform \
  -H "Content-Type: application/json" \
  -d '{"utlxSource": "...", "payload": "..."}'
```

Any language with an HTTP client works — PHP, Ruby, Rust, shell scripts. The trade-off: 1ms HTTP latency per call vs sub-millisecond stdio. For most use cases, this latency is negligible.

34.5 Alternative: Dapr Sidecar (Cloud-Native)

Dapr provides language-agnostic integration without any direct connection to UTLXe:

```
Your App → publishes to Service Bus → Dapr → UTLXe → Dapr → result topic
```

Your application publishes messages to a broker (Service Bus, Kafka, RabbitMQ). Dapr routes them to UTLXe. UTLXe transforms and publishes the result via Dapr. Your application never talks to UTLXe directly.

Best for: Azure Container Apps, Kubernetes with Dapr, event-driven architectures where the broker is the integration backbone.

34.6 SDK Comparison

Integration	Language	Protocol	Latency	Status
C# wrapper	.NET	Stdio protobuf	Sub-ms	Built
Go wrapper	Go	Stdio protobuf	Sub-ms	Built
Java wrapper	JVM	Stdio protobuf	Sub-ms	Built
Java direct	JVM	In-process API	Zero	Native
HTTP API	Any	HTTP REST	1ms	Built
Dapr sidecar	Any	HTTP/gRPC	2ms	Built
Python wrapper	Python	Stdio protobuf	Sub-ms	Planned
Node.js wrapper	JS/TS	Stdio protobuf	Sub-ms	Planned

34.6.1 Choosing the Right Integration

- **Your app is JVM and you want decoupled upgrades:** use the Java wrapper — same interface as C#/Go, hot-swappable, no dependency conflicts
- **Your app is JVM and you need absolute maximum throughput:** use the direct API — zero overhead, but tight coupling
- **Your app is C# or Go:** use the wrapper SDK — sub-millisecond, typed API
- **Your app is any other language:** use the HTTP API — universal, no SDK needed
- **Your architecture is event-driven:** use Dapr — decoupled, broker-agnostic
- **You want consistency across a polyglot stack:** use wrappers everywhere (same interface in every language)
- **You need maximum simplicity:** HTTP API — curl is your SDK

35 Migration Guides

Moving from an existing transformation tool to UTL-X? This chapter provides side-by-side translations for the most common migration paths: XSLT, DataWeave, jq, TIBCO BusinessWorks, and custom code.

35.1 Migrating from XSLT

XSLT and UTL-X solve the same problem — transforming XML — but with fundamentally different approaches. XSLT uses template-based pattern matching (push model). UTL-X uses functional expressions (pull model). The mapping is surprisingly direct:

XSLT	UTL-X
<code><xsl:value-of select="Order/Customer"/></code>	<code>\$input.Order.Customer</code>
<code><xsl:for-each select="Items/Item"></code>	<code>map(\$input.Items.Item, (item) -> ...)</code>
<code><xsl:if test="Total > 100"></code>	<code>if (\$input.Total > 100) ... else ...</code>
<code><xsl:choose>/<xsl:when>/<xsl:otherwise></code>	<code>match or chained if/else</code>
<code><xsl:attribute name="id"></code>	<code>"@id": \$input.orderId</code>
<code><xsl:variable name="x" select="..."/></code>	<code>let x = ...</code>
<code><xsl:template name="CalcTax"></code>	<code>function CalcTax(amount, rate) { ... }</code>
<code><xsl:apply-templates/></code>	<code>Not needed — expressions compose directly</code>
<code><xsl:sort select="@price"/></code>	<code>sortBy(items, (i) -> i.@price)</code>
<code><xsl:copy-of select="."/></code>	<code>...\$input (spread)</code>
<code>position()</code>	<code>Index parameter in map</code>
<code>count(Items/Item)</code>	<code>count(\$input.Items.Item)</code>
<code>sum(Items/Item/@price)</code>	<code>sum + map + toNumber</code>
<code>concat(FirstName, ' ', LastName)</code>	<code>concat with three arguments</code>

35.1.1 Side-by-Side Example

XSLT:

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/Orders">
    <Invoices>
      <xsl:for-each select="Order">
        <Invoice>
          <xsl:attribute name="id"><xsl:value-of select="@orderId"/></xsl:attribute>
          <Customer><xsl:value-of select="CustomerName"/></Customer>
          <Total><xsl:value-of select="sum(Item/Price * Item/Qty)"/></Total>
          <xsl:if test="sum(Item/Price * Item/Qty) > 1000">
            <Priority>HIGH</Priority>
          </xsl:if>
        </Invoice>
      </xsl:for-each>
    </Invoices>
  </xsl:template>
</xsl:stylesheet>
```

UTL-X:

```
%utlx 1.0
input xml
```

```

output xml
---
{
  Invoices: map($input.Orders.Order, (order) -> {
    let total = sum(map(order.Item, (i) -> toNumber(i.Price) * toNumber(i.Qty)))
    Invoice: {
      "@id": order.@orderId,
      Customer: order.CustomerName,
      Total: total,
      ...if (total > 1000) { Priority: "HIGH" } else {}
    }
  })
}

```

16 lines of XSLT become 14 lines of UTL-X. More importantly: the UTL-X version reads top-to-bottom as “map orders to invoices, calculate total, add priority if high.” The XSLT version requires understanding template matching, the `xsl:` namespace, and attribute value templates.

35.1.2 What XSLT Does That UTL-X Doesn’t

- **Template matching with priorities:** XSLT’s push model (`apply-templates` + pattern matching) has no direct UTL-X equivalent. UTL-X uses explicit navigation and `match` expressions instead.
- **XPath axes:** `preceding-sibling`, `ancestor`, `following` — XSLT can navigate the tree in any direction. UTL-X navigates downward from `$input` and uses `filter()` for sibling access.
- **XSLT 2.0+ grouping:** `xsl:for-each-group` — use `groupBy()` in UTL-X.
- **Multiple output documents:** `xsl:result-document` — use pipeline fan-out in UTLXe.

35.2 Migrating from MuleSoft DataWeave

DataWeave and UTL-X are the most similar transformation languages — both functional, both format-agnostic, both use `@` for XML attributes. Migration is often nearly mechanical:

DataWeave	UTL-X
<code>payload.orderId</code>	<code>\\$input.orderId</code>
<code>payload.order.@id</code>	<code>\\$input.order.@id</code>
<code>payload map (item) -> ...</code>	<code>map(\\$input, (item) -> ...)</code>
<code>payload filter (item) -> ...</code>	<code>filter(\\$input, (item) -> ...)</code>
<code>payload reduce (acc, item) -> ...</code>	<code>reduce(\\$input, init, (acc, item) -> ...)</code>
<code>sizeOf(payload)</code>	<code>count(\\$input)</code>
<code>payload groupBy \\$.category</code>	<code>groupBy(\\$input, (x) -> x.category)</code>
<code>payload orderBy \\$.name</code>	<code>sortBy(\\$input, (x) -> x.name)</code>
<code>payload distinctBy \\$.id</code>	<code>unique(map(\\$input, (x) -> x.id))</code>
<code>payload[0]</code>	<code>\\$input[0]</code>
<code>upper(name)</code>	<code>toUpperCase(name)</code>
<code>now()</code>	<code>now()</code>
<code>{(vars.key): value}</code>	<code>{[key]: value}</code>
<code>writeAttributes: true</code>	<code>{writeAttributes: true}</code>

35.2.1 Key Differences

- **Variable prefix:** DataWeave uses `payload`, `vars`, `attributes`. UTL-X uses `$input` (or `$name` for multi-input).

- **Function call syntax:** DataWeave: `payload map (x) -> x.name`. UTL-X: `map($input, (x) -> x.name)` — the collection is a parameter, not a method receiver.
- **User-defined functions:** DataWeave: `fun calcTax(amount, rate) = ....` UTL-X: `function CalcTax(amount, rate) { ... }` — PascalCase required.
- **Output directive:** DataWeave: `output application/json`. UTL-X: `output json`.
- **Conditional output:** DataWeave: `if (condition) field: value`. UTL-X: `...if (condition) { field: value } else {}` with spread.

35.2.2 MuleSoft Migration Strategy

1. Export DataWeave scripts from Anypoint Studio
2. Translate syntax (usually mechanical — see table above)
3. Replace `payload` with `$input`, adjust function call order
4. Run against sample data in UTL-X IDE — compare output
5. Add to conformance suite as regression tests

35.3 Migrating from jq

jq users will feel at home — UTL-X's expression mode was designed to mirror jq syntax:

jq	UTL-X
<code>.name</code>	<code>.name</code> or <code>\\$input.name</code>
<code>.[] .name</code>	<code>map(., (x) -> x.name)</code>
<code>select(.active)</code>	<code>filter(., (x) -> x.active)</code>
<code>sort_by(.name)</code>	<code>sortBy(., (x) -> x.name)</code>
<code>group_by(.category)</code>	<code>groupBy(., (x) -> x.category)</code>
<code>length</code>	<code>count(.)</code>
<code>keys</code>	<code>keys(.)</code>
<code>to_entries</code>	<code>entries(.)</code>
<code>from_entries</code>	<code>fromEntries(.)</code>
<code>map(select(...))</code>	<code>filter(., ...)</code>
<code>. @base64</code>	<code>base64Encode(.)</code>
<code>. @json</code>	<code>renderJson(.)</code>
<code>\{a, b\}</code>	<code>{a: .a, b: .b}</code>
<code>if ... then ... else ... end</code>	<code>if (...) ... else ...</code>

35.3.1 What jq Does That UTL-X Doesn't

- **Recursive descent:** `..` (jq) walks the entire tree. UTL-X has no recursive wildcard — use explicit paths or `.*` for one level.
- **String interpolation:** `"Hello \(.name)"` — UTL-X uses `concat("Hello ", .name)`.
- **/// `alternative operator:` jq's `try-alternative`. UTL-X uses `try { ... } catch { fallback }`.**

35.3.2 What UTL-X Does That jq Doesn't

- **XML, CSV, YAML, OData input/output** — jq is JSON-only
- **User-defined functions with PascalCase** — reusable, named logic
- **Schema validation** — UTLXe validates input/output against schemas
- **Pipeline chaining** — multi-step transformations with UDM hand-off
- **652 stdlib functions** — jq has 50 built-in functions

35.4 Migrating from TIBCO BusinessWorks

TIBCO BW uses a visual mapper for XML-to-XML transformations. The mapper generates XSD-driven mappings that correspond to UTL-X expressions:

TIBCO BW concept	UTL-X equivalent
Mapper activity	.utlx transformation file
Input schema (XSD)	input xml (or input xml {schema: "..."} — planned)
Output schema (XSD)	output xml
Line mapping (drag-and-drop)	Property assignment: target: \\${input.source}
For-Each group	map(array, (item) -> ...)
Choice / If-Then	if (...) ... else ... or match
XSLT sub-activity	Inline in the transformation body
Global variables	let bindings
BW process (multi-step)	UTLXe pipeline (Chapter 20)
JMS binding	Dapr input binding or HTTP API
Process variables	Not needed — functional, no mutable state

35.4.1 XSD Pattern Consideration

TIBCO BW generates Russian Doll XSD schemas (Chapter 29) — all types inline, one global root element. When migrating:

1. Export the input and output XSDs from BW
2. Convert to Venetian Blind if needed: `utlx --from xsd --to xsd {pattern: "venetian-blind"}`
3. Use the XSD to understand the structure, then write the UTL-X transformation
4. The UTL-X transformation doesn't need the XSD at runtime (CLI is lenient) — but UTLXe can validate against it

35.4.2 Performance Comparison

BW runs on a JVM application server with resource allocation per process. UTLXe runs as a container with worker threads:

- **BW:** typically 10-50 messages/second per process engine (depends on complexity)
- **UTLXe TEMPLATE:** 1,000-5,000 messages/second per container
- **UTLXe COMPILED:** 10,000-86,000 messages/second per container

The difference: BW's mapper interpreter is slower than UTL-X's tree-walking interpreter, and much slower than the COMPILED bytecode strategy. Plus UTLXe's worker model processes messages concurrently — BW processes require separate engine instances.

35.5 Migrating from Custom Code

Many organizations have transformations written in Java, C#, or Python — hand-coded XML/JSON processing.

35.5.1 When to Migrate

Migrate to UTL-X when:

- The transformation logic is primarily **field mapping** (`A.field → B.field`)
- Multiple developers need to maintain the transformation
- You need format flexibility (today XML, tomorrow JSON)
- You want testable, version-controlled transformation artifacts
- The custom code has grown unmaintainable (500+ lines of DOM manipulation)

Keep custom code when:

- The transformation involves **complex business logic** (not just mapping)
- You need database access, HTTP calls, or file I/O during transformation
- The transformation is performance-critical AND tightly coupled to the host application
- The code is well-tested and stable — “if it ain’t broke, don’t fix it”

35.5.2 Hybrid Approach

The best migration strategy is often hybrid: UTL-X for the mapping, custom code for the business logic:

Custom Code	UTL-X
1. Receive message	
2. Validate business rules	
3. Call external API for enrichment	
	4. Transform structure (field mapping)
	5. Format conversion (XML → JSON)
6. Send to target system	

Steps 4-5 are the mapping — move these to `.utlx` files. Steps 1-3 and 6 stay in custom code. Connect them via the SDK wrapper (Chapter 34) or HTTP API.

This gives you the best of both worlds: custom code for orchestration and business logic, UTL-X for the mapping that changes most frequently and benefits most from format agnosticism.

36 Performance and Optimization

Primarily UTLXe. Performance tuning applies to the UTLXe production engine. The CLI processes one message at a time — its performance is dominated by startup time (under 10ms with native binary), not throughput.

UTLXe achieves 86,000+ messages per second on a single container with 8 workers. This chapter explains where that number comes from, how to tune for your workload, and what to monitor.

36.1 Execution Strategies

The strategy choice is the single biggest performance lever. It determines how the transformation executes at runtime:

Strategy	How it works	Throughput	When to use
TEMPLATE	Walk AST, interpret each expression	1,000-5,000 msg/s	Development, simple transforms, low volume
COPY	Clone pre-built skeleton, fill values	5,000-20,000 msg/s	Schema-driven, predictable output structure
COMPILED	Execute JVM bytecode generated from AST	20,000-86,000 msg/s	Maximum throughput, complex logic
COPY+COMPILED	Clone skeleton + compiled fill logic	50,000-86,000+ msg/s	Ultimate throughput
AUTO	Schema present → COPY, else → TEMPLATE	Varies	Production default — engine chooses

The numbers above are for a single container with 8 workers on 1 vCPU, processing typical JSON-to-JSON transformations (1KB messages). Larger messages, more complex transformations, and XML parsing reduce throughput proportionally.

36.1.1 Why COMPILED Is Fast

The COMPILED strategy compiles UTL-X expressions to JVM bytecode using the ASM library — the same bytecode generation technology that Java itself uses. The generated bytecode:

- Runs at native JVM speed (no interpretation overhead)
- Benefits from HotSpot JIT compilation (further optimized at runtime)
- Eliminates the AST tree-walking overhead (10x faster than TEMPLATE for complex expressions)
- Uses typed operations (no boxing/unboxing for arithmetic)

The compilation happens once at init-time — while the bundle loads, before the engine reports ready — and is cached by SHA-256 hash of the source. The first message therefore hits already-compiled bytecode; there is no first-message latency penalty. Subsequent starts with the same transformation skip compilation entirely (cache hit).

36.2 Memory Model

36.2.1 UDM Expansion Factor

When UTL-X parses a message, it creates a UDM tree in memory. This tree is larger than the original message because:

- Every string becomes a Java String object (24+ bytes overhead per string)
- Every number becomes a boxed Double or Long (24 bytes)

- Every object becomes a HashMap with entry overhead
- Every array becomes an ArrayList with element pointers

The expansion factor depends on the format:

Format	Typical expansion	Example
JSON	5-10x	1KB JSON → 5-10KB heap
XML	10-20x	1KB XML → 10-20KB heap (attributes, namespaces add overhead)
CSV	3-5x	1KB CSV → 3-5KB heap (simpler structure)
YAML	5-10x	Similar to JSON

This means a 100KB XML document consumes 1-2MB of JVM heap during transformation. With 32 concurrent workers, that's 32-64MB just for input messages — plus the output UDM, plus intermediate values.

36.2.2 Heap Sizing

Rule of thumb: set JVM heap to 75% of container memory.

```
# Container with 512MB memory
docker run -e JAVA_OPTS="-Xmx384m -Xms384m" ...

# Container with 2GB memory
docker run -e JAVA_OPTS="-Xmx1536m -Xms1536m" ...
```

Set `-Xms` equal to `-Xmx` to avoid heap resizing during operation. The remaining 25% is for JVM metaspace, thread stacks, and native memory.

36.2.3 Sizing Guide

Workload	Message size	Workers	Recommended memory
Small messages	Under 10KB	8-16	256-512MB
Medium messages	10-100KB	8-16	512MB-1GB
Large messages	100KB-1MB	4-8	1-2GB
Very large messages	1MB+	2-4	2-4GB

For large messages, reduce worker count — fewer concurrent messages means less simultaneous heap usage. For small messages, increase workers to maximize throughput.

36.3 GraalVM Native Image

36.3.1 CLI: Native Binary

The CLI (`utlx`) is distributed as a GraalVM native binary:

- **Startup:** under 10ms (vs 250ms for JVM JAR)
- **Memory:** 40MB resident (vs 150MB for JVM)
- **Distribution:** single binary, no JVM required

This makes the CLI practical for shell scripts, CI/CD pipelines, and interactive use — where startup time dominates.

36.3.2 Engine: JVM Only

UTLX runs on the JVM, not as a native image. The JVM is actually **faster** for long-running engine workloads because:

- HotSpot JIT compiles hot paths to native code at runtime
- The COMPILED strategy generates JVM bytecode that benefits from JIT
- G1GC handles large heaps efficiently for sustained throughput
- Reflection (used by SnakeYAML, Jackson) works without configuration

Native image would save startup time (250ms → 10ms), but UTLXe starts once and runs for days — startup time is irrelevant. Runtime throughput matters, and the JVM wins there.

36.4 Benchmarking

36.4.1 Quick Benchmark

Measure throughput with the HTTP API:

```
# Start UTLXe with a test transformation
utlxe --bundle test-transforms/ --mode http --port 8080 --workers 8

# Benchmark with hey (HTTP load generator)
hey -n 10000 -c 32 -m POST \
  -H "Content-Type: application/json" \
  -d '{"orderId": "ORD-001", "total": 299.99}' \
  http://localhost:8080/api/execute/test-transform
```

This gives you requests/second, latency percentiles (p50, p95, p99), and error rate.

36.4.2 What to Measure

Metric	Target	Action if exceeded
p50 latency	Under 5ms	Check transformation complexity
p99 latency	Under 50ms	Check for GC pauses, large messages
Throughput	1,000+ msg/s per worker	Switch to COMPILED strategy
Error rate	Under 0.1%	Check validation, input format issues
Heap usage	Under 75% of max	Increase memory or reduce workers
GC pause time	Under 10ms	Tune G1GC, reduce heap pressure

36.4.3 Conformance Suite as Benchmark

The conformance suite (500+ tests) can be run in throughput mode to establish a baseline:

```
python3 conformance-suite/run_tests.py --benchmark
```

This runs all tests repeatedly and reports average execution time per test — useful for detecting performance regressions after code changes.

36.5 Production Tuning

36.5.1 Worker Count

Start with 8 workers per vCPU:

```
# 1 vCPU container
utlxe --workers 8

# 4 vCPU container
utlxe --workers 32
```

Adjust based on monitoring:

- **Workers always at capacity, queue building up:** add more workers or more instances
- **Workers mostly idle:** reduce to save memory
- **High GC pause times:** reduce workers (less concurrent heap pressure)

36.5.2 Horizontal vs Vertical Scaling

Approach	When	How
More workers (vertical)	Workers are saturated, memory is available	Increase <code>--workers</code>
More instances (horizontal)	Memory is saturated, or need fault tolerance	Add container replicas
Bigger container	Large messages need more heap	Increase memory limit
COMPILED strategy	TEMPLATE throughput is insufficient	Set strategy in <code>transform.yaml</code>

Horizontal scaling (more instances) is generally preferred — it adds fault tolerance and distributes load across nodes. Vertical scaling (more workers in one instance) is simpler but has a ceiling (one JVM, one heap).

36.5.3 GC Tuning

For containerized UTLXe, G1GC with these settings works well:

```
JAVA_OPTS="-Xmx1536m -Xms1536m \
-XX:+UseG1GC \
-XX:MaxGCPauseMillis=10 \
-XX:G1HeapRegionSize=4m \
-XX:+UseStringDeduplication"
```

- `MaxGCPauseMillis=10` — target 10ms GC pauses (acceptable for transformation latency)
- `G1HeapRegionSize=4m` — good for mixed small and medium objects (UDM trees)
- `UseStringDeduplication` — reduces heap for messages with repeated string values (field names, enum values)

36.5.4 Back-Pressure

UTLXe uses `ArrayBlockingQueue` with `CallerRunsPolicy`:

- When the work queue is full (all workers busy): the calling thread executes the transformation itself
- This naturally slows the producer when the engine is saturated
- No risk of out-of-memory from unbounded queue growth
- No messages are dropped — they're executed by the caller thread instead

Monitor queue depth via Prometheus (`utlxe_queue_depth`). Sustained high queue depth means you need more capacity.

36.6 Performance Anti-Patterns

36.6.1 Avoid: `filter()` Inside `map()`

```
// SLOW – O(N × M): scans all lines for every order
map($input.orders, (order) ->
  filter($input.lines, (l) -> l.orderId == order.id)
)

// FAST – O(N + M): build index once, look up per order
```

```
let linesByOrder = groupBy($input.lines, (l) -> l.orderId)
map($input.orders, (order) ->
  linesByOrder[order.id] ?? []
)
```

`nestBy()` handles the indexing automatically.

36.6.2 Avoid: Repeated Computation

```
// SLOW – computes the same filter twice
{
  activeCount: count(filter($input.users, (u) -> u.active)),
  activeNames: map(filter($input.users, (u) -> u.active), (u) -> u.name)
}

// FAST – compute once, reuse
let activeUsers = filter($input.users, (u) -> u.active)
{
  activeCount: count(activeUsers),
  activeNames: map(activeUsers, (u) -> u.name)
}
```

Bind intermediate results to `let` variables. The cost of the binding is zero; the cost of recomputation is proportional to the data size.

36.6.3 Avoid: Large String Concatenation in Loops

```
// SLOW – creates N intermediate strings
reduce($input.items, "", (acc, item) ->
  concat(acc, item.name, ", ")
)

// FAST – use join()
join(map($input.items, (item) -> item.name), ", ")
```

`join()` builds the result in one pass. `concat()` in a `reduce()` creates a new string for every iteration.

37 Message Parsing, Memory, and Size Boundaries

Every UTL-X transformation loads the entire message into memory as a UDM tree. This chapter explains why, what it costs, and how to plan for it. Understanding the memory model is essential for production sizing — the difference between a container that handles 86K messages per second and one that crashes with out-of-memory.

37.1 How UTL-X Parses Messages

37.1.1 DOM-Style Parsing

UTL-X uses a DOM-style (Document Object Model) approach: the entire input is parsed into a complete in-memory tree before transformation begins.

Raw bytes → Parser → UDM Tree (complete, in memory) → Transformation → Serializer → Output

Why DOM-style? Because transformation requires **random access**. Consider:

```
map($input.Orders.Order, (order) -> {
  orderId: order.@id,
  customer: $input.CustomerInfo.Name,    // ← reference to a DIFFERENT subtree
  total: sum(map(order.Lines.Line, (l) -> toNumber(l.Price) * toNumber(l.Qty)))
})
```

This transformation accesses `$input.CustomerInfo.Name` while iterating over `$input.Orders.Order` — two different branches of the tree. The `sum()` needs all line items. These operations require the full document in memory simultaneously.

37.1.2 Why Not Streaming?

Two alternative parsing approaches exist but are not suitable for UTL-X:

SAX (event-based): reads sequentially, fires events (`startElement`, `text`, `endElement`). Never loads the full document. Very low memory — can process gigabyte files. But no random access: when you're at line 500, line 10 is already gone. You cannot do `$input.Orders.Order[3].Price` because items 0-2 were discarded.

StAX (cursor-based): developer pulls the next element on demand. Lower memory than DOM, more control than SAX. Still sequential — no random access.

Approach	Memory	Random access	Use case
DOM (UTL-X)	Full document	Yes	Complex transformation, any field access
SAX	Constant (events)	No	Streaming filters, very large files
StAX	Low (cursor)	No	Simple XML streaming
JSON streaming	Low (token)	No	Large JSON arrays, ETL pipelines

UTL-X uses DOM because transformation **requires** random access. This is a fundamental architectural decision, not a limitation — it's the only approach that supports the full UTL-X language (aggregation, cross-references, sorting, grouping).

37.2 Memory Expansion Factor

37.2.1 Why 100KB of XML Becomes 5MB in Memory

Raw bytes on disk are compact. In-memory objects have overhead:


```

XML on disk (68 bytes):
  <Customer><Name>Alice Johnson</Name><Age>30</Age></Customer>

UDM in memory (~500+ bytes):
  UDM.Object("Customer")
    properties: HashMap (48 bytes + array)
      "Name" → UDM.Object
        properties: HashMap (48 bytes + array)
          "_text" → UDM.Scalar
            String("Alice Johnson"): 14 chars × 2 bytes + 24 bytes header
            Scalar header: 16 bytes
            HashMap.Entry: 32 bytes
          "Age" → UDM.Object (similar)
        attributes: HashMap (48 bytes – empty but allocated)
      name: String("Customer"): 40 bytes

```

68 bytes of XML become 500 bytes in UDM. That's a 7-10x expansion for simple structures. Complex XML with attributes, namespaces, and deep nesting can be worse.

37.2.2 Expansion by Format

Format	Typical expansion	Why
XML	10-50x	Verbose: every element becomes Object + HashMap + attributes HashMap
JSON	5-20x	Object overhead, string copies, HashMap entries
CSV	3-10x	Compact on disk, each row becomes Object with header-keyed properties
YAML	5-20x	Same as JSON (parses to same UDM structure)
OData JSON	5-20x	Same as JSON, plus metadata attribute objects

XML is the worst case because every element — even a leaf like `<Name>Alice</Name>` — becomes a UDM Object with a properties HashMap, an attributes HashMap (even if empty), and a name String. JSON is more efficient because it has no attributes, no namespaces, and no element names to store.

37.2.3 Worked Example: A 50 KB UBL Invoice

To make the expansion factor concrete, consider a typical Peppol UBL invoice — 50 KB of XML with 20 line items:

```

On disk (50 KB XML):
  <?xml version="1.0" encoding="UTF-8"?>
  <Invoice xmlns="urn:oasis:names:...">          20 bytes namespace URI
    <cbc:ID>INV-2026-001</cbc:ID>                  stored as: name + text + namespace
    <cbc:IssueDate>2026-05-04</cbc:IssueDate>
    ...
    <cac:InvoiceLine> × 20 lines                    each line: ~10 elements
      <cbc:ID>1</cbc:ID>                            each element → UDM.Object + HashMap
      <cbc:InvoicedQuantity>2</cbc:InvoicedQuantity>
      <cbc:LineExtensionAmount currencyID="EUR">50.00</cbc:LineExtensionAmount>
      ...                                            ← attributes stored separately
    </cac:InvoiceLine>
  </Invoice>

In memory (UDM tree – estimated 1.5 MB):

```

```

Root UDM.Object          48 bytes (object header + references)
  properties HashMap      48 bytes + 16 entries × 32 bytes
    "Invoice" → UDM.Object 48 bytes
      attributes: {"xmlns": "urn:..."} 48 bytes HashMap + 80 bytes entry
      properties HashMap 48 bytes + entries
        "cbc:ID" → UDM.Object 48 + 48 + 80 bytes (text unwrapped at
access)
      "cbc:IssueDate" → UDM.Object similar
      "cac:InvoiceLine" → UDM.Array 48 bytes + array of 20 Objects
        [0] → UDM.Object 48 + HashMap + 10 child properties
          "cbc:LineExtensionAmount" 48 + attributes HashMap (currencyID)
            attributes: {"currencyID": "EUR"} 48 + 32 + 40 bytes
            _text: "50.00" 48 + 40 bytes
        [1] → UDM.Object ... × 20 items

Total: ~1.5 MB (30x expansion from 50 KB)

```

The overhead comes from: HashMap per element (96 bytes minimum even when empty), String headers (40+ bytes per string), Object headers (16 bytes each), and attribute maps allocated even when elements have no attributes.

For this invoice, one UTLXe worker processes it in 5ms. Eight workers handle 1,600 invoices/second. Memory per worker: 3 MB (input UDM + output UDM + intermediate values). Total container need: well under 512 MB.

37.2.4 Real-World Memory Examples

Message size	Format	UDM memory	Workers	Container memory
1 KB	JSON	10 KB	8	256 MB
10 KB	XML	200 KB	8	256 MB
100 KB	XML	5 MB	8	512 MB
500 KB	XML	25 MB	8	1 GB
1 MB	XML	50 MB	8	2 GB
5 MB	XML	250 MB	4	4 GB
10 MB	XML	500 MB	2	8 GB
50 MB	XML	2.5 GB	1	8 GB (near limit)

Container memory accounts for: all workers processing simultaneously, JVM base overhead, GC overhead. In practice, not all workers hit peak simultaneously — the table shows worst case.

37.3 Message Size Boundaries

37.3.1 Sweet Spot: Under 1 MB

This is where UTL-X excels. Typical integration messages — API responses, events, orders, invoices — fall in this range:

- 8+ concurrent workers, no memory pressure
- 86K+ msg/s at 1 KB, 5K msg/s at 100 KB
- Standard 2-4 GB container is more than sufficient
- Sub-millisecond to single-digit millisecond latency

37.3.2 Feasible: 1 MB to 10 MB

Works but needs memory planning:

- Reduce workers to 2-4 to control peak memory
- Increase container memory to 4-8 GB
- Typical: large UBL invoices with 1,000+ lines, FHIR bundles, complex IDoc messages
- Latency: 50-500ms per message (acceptable for batch-like workloads)

37.3.3 Not Recommended: 10 MB to 50 MB

Technically possible with sufficient memory but problematic:

- Only 1-2 workers (most memory consumed by one document)
- GC pressure: large object allocation triggers garbage collection pauses
- Latency: seconds per message
- Better approach: split the document before transformation

37.3.4 Not Feasible: Over 50 MB

UDM expansion would require 4+ GB for a single message — no room for workers, JVM overhead, or concurrent messages. This is where streaming parsers (SAX, StAX) or big data tools (Spark, Dataflow) are appropriate.

UTL-X is a **transformation tool**, not a **big data tool**. Different scales need different tools.

37.3.5 “Why Not Just Use 1 GB Messages?”

A common question: “My server has 64 GB of RAM — can’t I just process a 1 GB XML file?”

The answer is no, and it is not just about memory — it is about **three walls** you hit simultaneously:

Wall 1: Memory expansion. A 1 GB XML file expands to 30-50 GB in UDM. No single JVM can hold this. Even with 64 GB physical RAM, the JVM’s garbage collector cannot efficiently manage a 50 GB heap — GC pauses become seconds to minutes.

Wall 2: Parse time. Parsing is $O(n)$ — linear in message size. A 1 KB message parses in microseconds. A 1 GB message takes minutes just to build the UDM tree, before any transformation logic runs.

Wall 3: GC pressure. The JVM’s garbage collector must track every object in the heap. A 50 KB message creates 1,000 objects. A 50 MB message creates 1,000,000 objects. A 1 GB message creates 20,000,000 objects. GC pause duration grows with object count — not linearly, but with increasing overhead as the heap grows.

37.3.6 CPU Time: Linear, Not Exponential

Good news: transformation time scales **linearly** with message size, not exponentially. Processing a message twice as large takes roughly twice as long — not four times:

Message size	Parse time	Transform time	Total	Scaling
1 KB	< 1ms	< 1ms	1ms	baseline
10 KB	1ms	1ms	2ms	2x size → 2x time
100 KB	5ms	5ms	10ms	10x size → 10x time
1 MB	30ms	20ms	50ms	linear
10 MB	300ms	200ms	500ms	linear
50 MB	2s	1s	3s	linear
100 MB	5s	3s	8s	linear — but GC pauses add unpredictability

The transformation itself (evaluating the UTL-X body) is $O(n)$ — it visits each element once. Functions like `map`, `filter`, `groupBy` are linear. Even `sortBy` is $O(n \log n)$. There are no $O(n^2)$ operations in normal transformations.

However: at large sizes, the bottleneck shifts from CPU to memory management. GC pauses become unpredictable — a 50 MB message might process in 3 seconds or 8 seconds depending on when GC triggers. This is why the “Not Recommended” boundary exists: not because CPU time is exponential, but because GC behavior becomes non-deterministic.

37.3.7 The Real Boundaries

Message size	UDM mem-ory	Objects cre-ated	Feasibility	Workers (4GB)	Approach
Under 10 KB	< 200 KB	500	Excellent	8-32	Standard UTL-X
10-100 KB	0.2-5 MB	500-50K	Excellent	8-16	Standard
100 KB-1 MB	5-50 MB	50K-500K	Good	4-8	Monitor mem-ory
1-10 MB	50-500 MB	500K-5M	Feasible	2-4	More memory, fewer workers
10-50 MB	0.5-2.5 GB	5M-25M	Marginal	1	Split first if pos-sible
50-100 MB	2.5-5 GB	25M-50M	Risky	1	Dedicated 8GB+ con-tainer
Over 100 MB	5+ GB	50M+	Not feasible	–	Streaming or big data tools

For the vast majority of integration workloads — API messages, events, orders, invoices, FHIR bundles, IDoc segments — messages are under 1 MB. UTL-X handles these effortlessly. The 50+ MB range is for bulk data exports, full database dumps, or log files — these belong in streaming pipelines (Kafka Streams, Spark, Dataflow), not in a transformation tool.

37.3.8 When to Split Instead of Grow

If your messages are large because they contain arrays of independent records, split before transforming:

```
// Instead of transforming a 50 MB file with 10,000 orders:
// Split into individual orders (each ~5 KB), transform each separately.
// UTLXe's pipeline chaining handles this natively.
```

A 50 MB file with 10,000 orders → 10,000 × 5 KB messages → each processes in 2ms → total: 20 seconds with 8 workers. Same data, no memory problem, predictable latency per message.

37.4 The Poison Message Problem

A single oversized message can crash the entire JVM — taking down all workers and all in-flight transformations. This is the **poison message** scenario:

```
Normal operation:
Worker 1: processing 50 KB invoice      ✓ (3 MB UDM)
Worker 2: processing 80 KB order        ✓ (4 MB UDM)
Worker 3: processing 20 KB event        ✓ (1 MB UDM)
Free heap: 1.5 GB
```

```
Poison message arrives (2 GB XML file):
Worker 4: starts parsing 2 GB XML...
```

- UDM expansion: 2 GB × 30 = 60 GB needed
- JVM heap limit: 2 GB
- java.lang.OutOfMemoryError
- JVM CRASHES
- Workers 1, 2, 3 also die (shared JVM)
- All in-flight messages lost

This is not theoretical. In enterprise integration, a partner accidentally sends a database dump instead of a single order. A batch system sends an entire day's data as one message. A test system sends production data without pagination.

37.4.1 Defense: Input Size Limits

UTLX should reject messages above a configurable size limit **before** parsing:

```
# TransformConfig
limits:
  maxInputSize: 10485760    # 10 MB – reject anything larger
  maxInputSizeAction: "reject" # or "dead-letter"
```

The check happens on raw bytes — before UDM expansion, before memory allocation. A 2 GB message is rejected in microseconds, not after consuming all heap trying to parse it.

Without this check, a single malformed or oversized message can take down a production engine that normally handles thousands of messages per second.

37.4.2 Defense: Worker Isolation (Future)

A more robust approach: run each worker in a memory-limited sandbox. If one worker exceeds its allocation, only that worker dies — not the entire JVM. This requires either:

- Separate JVM processes per worker (overhead but safe)
- JVM memory regions per thread (not supported by standard JVMs)
- Container-level isolation (one transformation per container — Kubernetes pod pattern)

For now, input size limits are the practical defense.

37.5 Swap: Never Use It

Operating system swap (paging memory to disk) is catastrophic for UTL-X workloads. Never enable swap on a UTLX host.

Why swap destroys performance:

The JVM's garbage collector must scan the entire heap to find live objects. When part of the heap is on disk (swapped out), every GC scan triggers disk I/O — turning a 10ms GC pause into a 10-second pause. The GC doesn't know which pages are swapped; it touches them all.

```
Without swap (heap fits in RAM):
  GC pause: 10-50ms
  Transformation: 5ms
  Total: 15-55ms per message

With swap (heap partially on disk):
  GC pause: 5-30 SECONDS (disk I/O on every scan)
  Transformation: 5ms (if data is in RAM) or 500ms+ (if swapped)
  Total: 5-30 SECONDS per message – 1000x slower
```

Swap makes the problem worse, not better. The system appears to have enough memory (no `OutOfMemoryError`) but performance degrades by orders of magnitude. This is harder to diagnose than a clean crash — the engine appears to be “running” but latency is catastrophic.

37.5.1 Container Configuration

For Kubernetes / Azure Container Apps:

```
# Disable swap in container spec
resources:
  limits:
    memory: "4Gi"      # hard limit – container killed if exceeded
  requests:
    memory: "4Gi"      # guaranteed – no overcommit

# JVM configuration (in Dockerfile or env)
JAVA_OPTS: "-Xmx3g -XX:+UseContainerSupport -XX:+AlwaysPreTouch"
```

`-XX:+AlwaysPreTouch` forces the JVM to allocate all heap pages at startup — this ensures the OS commits real RAM, not virtual memory. If the container doesn’t have enough physical memory, it fails immediately at startup instead of degrading slowly under load.

`-XX:+UseContainerSupport` tells the JVM to respect container memory limits (cgroup) rather than looking at host memory. Without this, a JVM in a 4 GB container on a 64 GB host might try to use 16 GB of heap — and get killed by the OOM killer.

37.6 Memory Sizing Formula

For production container sizing:

```
Container memory = JVM base (200 MB)
                  + (max_message_size × expansion_factor × workers × 2)
                  + 25% GC overhead
```

The $\times 2$ accounts for input UDM and output UDM both in memory during transformation.

Example: 100 KB XML, 8 workers:

```
= 200 MB + (100 KB × 30 × 8 × 2) + 25%
= 200 MB + 48 MB + 62 MB
= 310 MB → round up to 512 MB
```

Example: 1 MB XML, 8 workers:

```
= 200 MB + (1 MB × 30 × 8 × 2) + 25%
= 200 MB + 480 MB + 170 MB
= 850 MB → use 1 GB or 2 GB
```

37.6.1 The 75% Rule

UTL-X deployment templates set JVM heap to 75% of container memory:

- 1 GB container → 768 MB heap (`-Xmx768m`)
- 2 GB container → 1536 MB heap (`-Xmx1536m`)
- 4 GB container → 3072 MB heap (`-Xmx3072m`)

The remaining 25% is for JVM metaspace, code cache, thread stacks, native memory (HTTP transport buffers), and OS overhead. Do NOT set `-Xmx` to 100% of container memory — the container will be OOM-killed by the orchestrator.

37.7 The Split-Before-Transform Pattern

For messages over 10 MB, split before transforming:

```
50 MB XML (10,000 orders)
→ Pre-processor splits into 10,000 × 5 KB messages
→ UTL-X processes at 86K msg/s = done in 0.1 seconds
→ Post-processor reassembles if needed
```

This is **faster** than loading 50 MB at once — and uses far less memory. The pre-processor can be a simple Python script, a streaming XML parser (`xmlstarlet`, `xml_split`), or a shell script with `csplit`.

The pattern applies to any large batch file: a CSV with 100,000 rows, an XML with thousands of records, a JSON array with thousands of elements. Split by record, transform each record independently, collect results.

37.8 UTL-X vs Streaming and Big Data Tools

Aspect	UTL-X (DOM)	Apache Spark	SAX/StAX
Message size	Under 10 MB	Unlimited	Unlimited
Random access	Yes	Via RDD/DataFrame	No
Complex transforms	Yes (full language)	Yes (Scala/Python code)	Very limited
Latency	1-50ms	Seconds to minutes	Milliseconds
Format support	11 formats	JSON, CSV, Parquet	XML only
Use case	Integration	Big data analytics	Simple streaming
Deployment	Container	Cluster	Library

These tools serve different scales:

- **Under 10 MB, complex transforms:** UTL-X
- **Over 10 MB, simple transforms:** split + UTL-X, or streaming parser
- **Over 1 GB, batch analytics:** Apache Spark, Google Dataflow, AWS Glue

38 Logging, Observability, and Compliance

Data transformations sit at the most sensitive point in an integration pipeline — they see every field of every message. A payment transformation sees card numbers. A healthcare transformation sees patient records. A financial transformation sees account balances. This makes logging decisions critical: log too much and you violate PCI DSS, GDPR, or HIPAA. Log too little and you can't debug production issues.

This chapter covers UTL-X's approach: log metadata (what happened), never data (what was in the message).

38.1 What UTL-X Logs

38.1.1 Default Behavior (INFO Level)

UTLXe logs operational events, never message content:

- **Startup:** engine version, loaded transformations, worker count, transport mode
- **Transformation lifecycle:** load, unload, reload events with transformation ID
- **Request processing:** transformation ID, duration, success/failure — but NOT the message
- **Errors:** parse errors, validation failures, runtime exceptions — with field paths, not field values
- **Health:** worker pool status, memory usage, queue depth

What UTL-X does NOT log by default:

- Input message content
- Output message content
- Intermediate UDM state
- Field values during transformation

This is safe for PCI DSS, GDPR, HIPAA, and SOX environments without any configuration.

38.1.2 Log Levels

Level	What it logs	When to use
ERROR	Transformation failures, parse errors, unrecoverable issues	Always on
WARN	Validation warnings (WARN policy), deprecated functions	Always on
INFO	Startup, shutdown, load/unload, health, request metadata	Production (default)
DEBUG	Parser trace, interpreter steps, UDM tree structure	Development only
TRACE	Every function call, every property access	Debugging only

Never use DEBUG or TRACE in production. These levels dump message content — UDM trees, field values, intermediate results — into logs. In a payment or healthcare pipeline, this exposes sensitive data. The performance impact is also severe: TRACE can reduce throughput by 100x.

38.1.3 Configuring Logging

UTLXe: uses SLF4J + Logback. Configure via `logback.xml` in the container, or override with environment variables:

```
# Set log level via environment
UTLXE_LOG_LEVEL=INFO      # default
UTLXE_LOG_LEVEL=DEBUG     # development only
```

CLI: console output with `--verbose` flag for additional detail.

Cloud platforms: UTLXe logs to stdout. Container platforms capture stdout automatically:

- Azure Container Apps → Log Analytics
- GCP Cloud Run → Cloud Logging
- AWS ECS → CloudWatch Logs
- Kubernetes → node-level log collection (Fluentd, Fluent Bit, Vector)

38.2 When NOT to Log: Compliance Requirements

38.2.1 PCI DSS (Payment Card Industry)

PCI DSS protects cardholder data. If your transformation processes payments:

Never log:

- Primary Account Number (PAN) — the 16-digit card number
- CVV/CVC — the 3-4 digit security code (must never be stored, period)
- Full magnetic stripe data
- PIN or PIN block

May log (with care):

- Truncated PAN: first 6 + last 4 digits (411111****1111)
- Transaction reference ID
- Timestamp, amount, currency (metadata)

UTL-X implication: DEBUG logging exposes card data in the UDM tree. Never enable DEBUG in payment flows. Error messages must reference field names, not values:

```
Good: "Validation failed: field 'cardNumber' does not match pattern"
Bad:  "Validation failed: value '4111-1111-1111-1111' does not match pattern"
```

38.2.2 GDPR (General Data Protection Regulation)

GDPR protects personal data of EU residents. If your transformation processes customer data:

Personal data includes: name, email, address, phone, IP address, BSN (Dutch), SSN (US), health data, financial data, location data, biometric data.

Key obligations:

- **Data minimization:** log only what's necessary for the purpose
- **Right to erasure:** if you log personal data, you must be able to delete it on request
- **Purpose limitation:** logs collected for debugging can't be used for marketing

UTL-X implication: INFO level logs no message content — GDPR-safe by default. If you enable DEBUG for troubleshooting, disable it immediately after and purge the debug logs. Configure log retention to match your GDPR data retention policy.

38.2.3 HIPAA (Health Insurance Portability and Accountability Act)

HIPAA protects health information. If your transformation processes HL7, FHIR, or clinical data:

Protected Health Information (PHI): patient name, date of birth, SSN, diagnosis codes (ICD-10), medication lists, lab results, insurance IDs.

Key obligations:

- **Minimum necessary:** access only what's needed for the transformation
- **Audit trail:** log WHO accessed WHAT, WHEN — but not the PHI itself
- **Encryption:** PHI must be encrypted at rest and in transit

UTL-X implication: healthcare transformations process PHI — the transformation itself is fine (it needs to see the data to map it). But logs must never contain PHI. Log the transformation ID, timestamp, and success/failure. Don't log patient names, diagnosis codes, or lab values.

38.2.4 NEN 7510 (Dutch Healthcare Security)

Dutch implementation of ISO 27001 for healthcare, required for all Dutch healthcare organizations:

- Strict access control and data classification
- Audit logging requirements
- Data must stay within the organization's tenant

UTL-X in hospital environments: deploy UTLXe in the hospital's own Azure tenant. All processing, logging, and data stay within the tenant boundary. No external cloud logging services.

38.2.5 SOX (Sarbanes-Oxley — Financial Reporting)

SOX requires audit trails for data that affects financial reports:

- Log: WHO transformed WHAT, WHEN, WHICH transformation version
- Don't log: actual financial amounts, account numbers, balances
- Transformation version must be traceable — .utlx files in Git with commit hashes

38.3 Designing for Compliance

38.3.1 Principle: Log Metadata, Not Data

Log this (metadata)	Don't log this (data)
Transformation ID	Input message content
Timestamp	Output message content
Duration (milliseconds)	Field values
Success / failure	PII (name, email, BSN, SSN)
Error message (field path)	Error data (field value)
Worker ID	Session tokens or API keys
Source system identifier	Credentials or passwords
Schema validation result (pass/fail)	The data that failed validation
Message size (bytes)	Message content
Input/output format	Actual payload

38.3.2 Audit Logging Pattern

For regulated industries, add an audit step to the pipeline:

Input → Transform → Validate → Audit Log → Output

The audit log records:

- Transaction ID (for correlation across systems)
- Transformation name and version (Git commit hash)
- Timestamp (ISO 8601, UTC)
- Input format and size (not content)
- Output format and size (not content)
- Validation result (pass/fail, which rules)
- Error details (field paths, not values)

This provides a complete audit trail — who, what, when, which version — without exposing a single sensitive value.

38.3.3 Masking in Error Messages

Error messages are the most common source of accidental data exposure. UTL-X error messages reference field paths, not values:

```
Good: "Type error at $.payment.cardNumber: expected string, got number"
Bad:  "Type error: 4111111111111111 is not a string"

Good: "Validation failed: field 'patient.bsn' does not match pattern"
Bad:  "Validation failed: '123456789' does not match pattern NL[0-9]{9}"
```

When writing custom error handling with `try/catch`, follow the same principle:

```
try {
  parseDate($input.birthDate, "yyyy-MM-dd")
} catch {
  // Good: log what failed, not the value
  null    // or: error("Invalid date format in field 'birthDate'")
}
```

38.4 Prometheus Metrics: Safe by Design

UTLX's Prometheus metrics (port 8081) contain no message data — only operational counters and histograms:

```
utlxe_messages_processed_total{transformation="order-to-invoice"} 42531
utlxe_messages_failed_total{transformation="order-to-invoice"} 12
utlxe_transformation_duration_seconds_bucket{le="0.01"} 38000
utlxe_active_workers 6
utlxe_transformations_loaded 5
```

These metrics are safe for every compliance regime — PCI DSS, GDPR, HIPAA, NEN 7510, SOX. They tell you how the system is performing without revealing what data it's processing.

38.5 The Security Library Connection

UTL-X's security library (Chapter 17) provides functions for masking, hashing, and encrypting sensitive data **within** transformations. If you need to produce an audit-safe version of the output:

```
{
  orderId: $input.orderId,
  customerName: mask($input.customerName, 3),    // "Ali***"
  email: hashSHA256($input.email),               // irreversible hash
  cardNumber: maskPAN($input.cardNumber),        // "411111*****1111"
  total: $input.total                            // non-sensitive, keep as-is
}
```

This produces a record that's safe to log, store in a data warehouse, or send to analytics — the sensitive fields are masked or hashed, the business metadata is preserved. See Chapter 17 for the full security function reference.

39 Quality Assurance and Testing

A transformation that produces wrong output is worse than one that crashes — because wrong output travels downstream silently. The order with a missing VAT field gets invoiced without tax. The patient record with a swapped first/last name enters the hospital system. The payment with a wrong decimal position moves 100x the intended amount.

UTL-X's testing strategy is designed to catch these errors before they reach production, with five layers from unit tests to black-box conformance testing.

39.1 The Five Testing Layers

UTL-X uses layered testing — each layer catches different classes of bugs:

```

Layer 5: Conformance Suite (Python, 500+ tests)
├─ Layer 4: Integration Tests (Kotlin)
│   └─ Layer 3: Module Tests (Kotlin)
│       └─ Layer 2: Format Tests (Kotlin)
│           └─ Layer 1: Unit Tests (Kotlin)

```

Each outer layer depends on the inner layers passing. If a unit test fails, everything above it is suspect.

39.2 Layer 1: Unit Tests

Test individual functions and classes in isolation:

- UDM type construction, access, and mutation
- Scalar operations (asString, asNumber, asBoolean, type coercion)
- Parser edge cases (special characters, Unicode, empty input, deeply nested structures)
- Each stdlib function (652 functions, each with at least one test)

```
./gradlew :modules:core:test
```

These run in milliseconds and catch logic errors in individual components. If `toNumber("1.234,56")` returns the wrong value, a unit test catches it instantly.

39.3 Layer 2: Format Tests

Test each format parser and serializer independently. Every format module has its own test suite:

- **JSON:** parser (BOM, precision, special characters), serializer (pretty/compact, writeAttributes)
- **XML:** parser (namespaces, attributes, CDATA, encoding), serializer (element ordering, escaping)
- **CSV:** parser (delimiters, quoting, headers, regional numbers), serializer (regional formats, BOM output)
- **YAML:** parser (anchors, aliases, multi-document, timestamps), serializer (block/flow style)
- **OData:** parser (annotation extraction, collection unwrapping), serializer (metadata levels)
- **Schema formats:** XSD (pattern detection), JSON Schema (draft versions), Avro, Protobuf, EDMX, Table Schema — each with parse/serialize/round-trip tests

```

./gradlew :formats:json:test    # one format
./gradlew :formats:test        # all formats

```

These catch format-specific regressions. When the XML `_text` serialization leak was fixed, format tests caught the behavior change immediately.

39.4 Layer 3: Module Tests

Test cross-cutting concerns within a module:

- **Core:** interpreter tests (expression evaluation, function calls, let bindings, closures), type system tests, parser tests (grammar edge cases)
- **CLI:** command parsing, expression mode (-e), identity mode (format flip), multi-input handling
- **Engine:** strategy tests (TEMPLATE, COPY, COMPILED produce identical results), worker pool tests, back-pressure tests
- **Daemon:** LSP protocol tests, live preview tests

```
./gradlew :modules:cli:test
./gradlew :modules:engine:test
```

These catch interaction bugs — where two components work individually but fail together.

39.5 Layer 4: Integration Tests

Test end-to-end flows through multiple modules:

- Input parsing → transformation → output serialization (the full pipeline)
- Multi-format tests: XML in → JSON out, CSV in → YAML out, OData in → XML out
- Schema validation integration: load schema → validate → transform → validate
- Multi-input tests: two inputs with different formats → merged output
- UTLXe transport tests: HTTP endpoint, gRPC, stdio-proto — same transformation, same result

```
./gradlew test    # all modules
```

These catch end-to-end regressions — the parser produces valid UDM, the interpreter evaluates correctly, but the serializer mishandles a specific UDM shape.

39.6 Layer 5: Conformance Suite

The most important testing layer. The conformance suite is an external test runner (Python) that tests the CLI as a **black box** — exactly like a user would.

39.6.1 Why Python, Not Kotlin?

The conformance suite deliberately runs outside the JVM:

- Tests the **binary** (native or JAR) — the actual artifact users install
- No internal access — tests what the user sees, not internal state
- Language-independent — could test a future WASM or Rust implementation of UTL-X
- Easy to extend — add a YAML file, no compilation needed

39.6.2 Test Format

Each test is a YAML file with input, transformation, and expected output:

```
name: "xml_to_json_passthrough"
input:
  format: xml
  data: "<Order><Customer>Alice</Customer></Order>"
transformation: |
  %utlx 1.0
  input xml
  output json
  ---
  $input
expected:
```

```
format: json
data: '{"Order":{"Customer":"Alice"}}'
```

500+ tests cover:

- **Formats:** every format pair (XML→JSON, JSON→CSV, YAML→XML, etc.)
- **Stdlib:** function behavior (string operations, math, date/time, arrays, objects)
- **Examples:** real-world transformations (orders, invoices, IDoc, FHIR)
- **Multi-input:** named inputs from different formats
- **Error handling:** parse errors, validation failures, edge cases
- **Schema:** XSD, JSON Schema, Avro, Protobuf parse/serialize/round-trip
- **Regional:** CSV with European, French, Swiss number formatting

39.6.3 Running the Suite

```
cd conformance-suite
python3 utlx/runners/cli-runner/simple-runner.py
```

Output shows pass/fail per test with details on failures:

```
Running 470 tests...
[PASS] xml_to_json_passthrough
[PASS] csv_orders_grouping_semicolon
[FAIL] json_to_csv_regional_french
  Expected: "1 234,56"
  Got:      "1234.56"
...
467/470 passed, 3 failed
```

39.7 Auto-Capture: Recording Tests from Usage

UTL-X can automatically record test cases from real CLI usage:

```
utlx --capture transform order-transform.utlx order.xml
```

This runs the transformation normally AND saves a conformance test YAML file with the input, transformation, and actual output. The test is saved to `conformance-suite/utlx/tests/auto-captured/`.

Warning: auto-captured tests cement current behavior. They validate what IS, not what SHOULD BE. If the output has a bug, the test captures the bug as “expected” — and the test passes, hiding the bug. Always review auto-captured tests manually before trusting them. This was learned the hard way: auto-captured tests had `_text` in expected output, which cemented the bug as “correct” behavior.

Auto-capture is valuable for:

- Building regression tests from production data (sanitized)
- Documenting existing behavior before refactoring
- Onboarding: capture a senior developer’s transformations as test cases

39.8 UTLX Engine Tests

The engine has its own conformance suite testing production-specific behavior:

- **Transport parity:** same test against HTTP, gRPC, and stdio-proto — must produce identical results

- **Strategy parity:** same test against TEMPLATE, COPY, and COMPILED — must produce identical results
- **Throughput tests:** verify 86K+ messages/second under sustained load
- **Batch tests:** 100+ instances per schema to catch edge cases in real data
- **Validation tests:** pre/post validation with all 7 schema validators

Transport parity and strategy parity are the most important engine tests. If COMPILED produces a different result than TEMPLATE for the same transformation, it's a bytecode generation bug — and these are the hardest bugs to find without automated testing.

39.9 Test-Driven Transformation Development

For production transformations, write the test first:

1. **Define the contract:** write the expected output for a representative input
2. **Write the transformation:** make the test pass
3. **Add edge cases:** null fields, empty arrays, missing properties, wrong types
4. **Add error cases:** invalid input, schema violations, boundary values
5. **Run the conformance suite:** verify no regressions in other transformations

```
# Step 1: Write this test file FIRST
name: "dynamics365_order_to_ubl_invoice"
input:
  format: json
  data: '{"ordernumber": "ORD-001", "totalamount": 299.99, ...}'
transformation: |
  %utlx 1.0
  input odata
  output xml
  ---
  ...
expected:
  format: xml
  data: '<Invoice xmlns="urn:oasis:..."><cbc:ID>ORD-001</cbc:ID>...</Invoice>'
```

Then write the .utlx file to make it pass. This inverts the common “write code, then test” pattern — the test IS the specification.

39.10 Continuous Integration

UTL-X uses GitHub Actions for automated testing on every push and PR:

39.10.1 CLI Pipeline (cli-ci.yml)

1. Build JAR (Kotlin/Gradle)
2. Run Kotlin tests (layers 1-4)
3. Build native binary (GraalVM)
4. Run conformance suite against native binary (layer 5)
5. Performance benchmark (detect regressions)

39.10.2 Release Pipeline (release.yml)

1. All CI steps above
2. Build native binaries for 3 platforms (macOS, Linux, Windows)
3. Build Docker image
4. Run conformance suite against Docker image
5. Publish artifacts (GitHub Releases, Homebrew, Chocolatey, Docker registry)

39.10.3 Pull Request Checks

Every PR must pass:

- All Kotlin tests (layers 1-4)
- Full conformance suite (layer 5)
- No new test failures (regressions blocked)

This ensures that no code reaches the main branch without passing 500+ tests across all formats, functions, and patterns.

Part V

Future Outlook

Semantic validation, API contracts, and the road ahead

40 Semantic Validation: Beyond Syntax

Chapter 19 covered syntactical validation — checking that data matches a schema’s structure, types, and constraints. This chapter covers the next frontier: **semantic** validation — checking that data is not just structurally correct but **meaningfully** correct.

40.1 The Gap Between Syntax and Meaning

Syntactical validation answers: “Is this valid XML? Does it match the XSD? Are required fields present?”

Semantic validation answers: “Is the invoice total correct? Does the VAT ID match the country? Is the delivery date after the order date?”

Consider a Peppol invoice that passes all syntactical checks:

```
{
  "invoiceId": "INV-001",
  "country": "NL",
  "vatId": "DE123456789",
  "issueDate": "2026-04-30",
  "dueDate": "2026-03-15",
  "lines": [
    {"product": "Widget", "qty": 10, "unitPrice": 30.00, "lineTotal": 300.00},
    {"product": "Gadget", "qty": 5, "unitPrice": 49.99, "lineTotal": 249.95}
  ],
  "total": 999.99
}
```

This is valid JSON. It could pass a JSON Schema. But it has three semantic errors:

- `vatId` starts with “DE” but `country` is “NL” — cross-field inconsistency
- `dueDate` is **before** `issueDate` — temporal violation
- `total` is 999.99 but line totals sum to 549.95 — arithmetic mismatch

No structural schema can catch these. They require **business rule** validation.

40.2 What Schemas Cannot Express

Syntactical (schema)	Semantic (business rules)
Field is required	If country is NL, VAT ID must start with “NL”
Field is a number	Invoice total = sum of line totals
Field matches pattern	End date must be after start date
Array has min 1 item	Quantity x unit price = line total
Value is in enum list	Order status transition: DRAFT → CONFIRMED → SHIPPED (not backwards)
String length 1-100	IBAN checksum is valid (modulo 97)
Type is date	BSN (Dutch citizen ID) passes the 11-check

The left column is what JSON Schema, XSD, and Avro can express. The right column requires a validation language that understands **relationships** between fields.

40.3 Semantic Validation Languages in Industry

40.3.1 Schematron (XML)

The ISO standard (ISO/IEC 19757-3) for rule-based XML validation. Uses XPath assertions:

```
<sch:rule context="cac:InvoiceLine">
  <sch:assert test="cbc:LineExtensionAmount =
    cbc:InvoicedQuantity * cac:Price/cbc:PriceAmount">
    Line total must equal quantity times unit price
  </sch:assert>
</sch:rule>
```

Peppol BIS 3.0 ships with 65+ Schematron rules that every invoice must pass. UBL, HL7 CDA, DITA, and government XML standards all use Schematron extensively.

Strength: mature, ISO standardized, XPath-based. **Limitation:** XML-only — cannot validate JSON, CSV, or YAML output.

40.3.2 FEEL (Friendly Enough Expression Language)

Part of DMN (Decision Model and Notation), an OMG standard. A readable expression language for business rules:

```
invoice.total = sum(invoice.lines.amount)
if order.country = "NL" then vat.rate = 21 else vat.rate = 0
date(order.delivery) > date(order.placed) + duration("P2D")
```

Used by Camunda, jBPM, Kogito, and Drools for decision tables and business rules.

Strength: readable, standardized, rich temporal and numeric expressions. **Limitation:** not schema-aware, designed for decision tables rather than data validation.

40.3.3 Other Approaches

- **OPA/Rego (Open Policy Agent):** policy-as-code, used in Kubernetes admission control and API authorization. A distinct approach because rules live as **external policy** evaluated across many data sources, rather than being attached to a single document's schema.
- **CUE:** configuration language with built-in validation constraints — validates and generates. Distinct in that the **same** constraints both check data and produce derived configuration, unifying validation with generation.
- **CEL (Common Expression Language):** Google's lightweight expression language for security policies and IAM conditions. Distinct as a tiny, sandboxed, **embeddable** evaluator — ideal when semantic checks must run inside an API or edge runtime with minimal overhead.
- **Ajv custom keywords:** extend JSON Schema with JavaScript validators — JSON-only. Distinct because the semantic rule is expressed **inside JSON Schema itself** rather than in a separate language — at the cost of being tied to JavaScript and JSON.

40.4 How Semantic Validation Could Work in UTL-X

UTL-X already has a rich expression language with 652 functions, conditionals, cross-field access, and arithmetic. Adding semantic validation is an **extension** of existing capabilities, not a new language.

40.4.1 The Proposed Approach: Native Assertions

Add a `validate` block to the `.utlx` file — assertions that run after the transformation, before output serialization:

```
%utlx 1.0
input json
output json
---
```

```

{
  invoiceId: $input.id,
  country: $input.country,
  vatId: $input.vatId,
  issueDate: $input.issueDate,
  dueDate: $input.dueDate,
  lines: map($input.lines, (line) -> {
    product: line.product,
    qty: line.qty,
    unitPrice: line.unitPrice,
    lineTotal: line.qty * line.unitPrice
  }),
  total: sum(map($input.lines, (l) -> l.qty * l.unitPrice))
}

validate {
  assert $output.total == sum(map($output.lines, (l) -> l.lineTotal)),
    "Invoice total must equal sum of line totals"

  assert parseDate($output.dueDate, "yyyy-MM-dd") > parseDate($output.issueDate, "yyyy-MM-dd"),
    "Due date must be after issue date"

  assert if ($output.country == "NL") startsWith($output.vatId, "NL") else true,
    "VAT ID must match country code"
}

```

The `validate` block uses standard UTL-X expressions — no new syntax to learn. Each `assert` takes a boolean expression and an error message. If any assertion fails, the transformation result is rejected (STRICT policy) or a warning is logged (WARN policy).

40.4.2 Why This Works

The `validate` block has access to both `$input` (the original message) and `$output` (the transformation result). This enables:

- **Cross-field validation:** compare any field with any other field
- **Computed validation:** sum line totals, check averages, verify checksums
- **Conditional validation:** different rules for different countries, message types, or customer segments
- **Temporal validation:** date comparisons using UTL-X's date functions
- **Pattern validation:** regex matching, string operations, format checks

All using the same 652 `stdlib` functions available in the transformation body.

40.4.3 Integration with the Validation Orchestrator

Semantic validation slots into UTLXe's sandwich pattern — and it can run at **two** points: on the input before transforming, and on the output afterward:

1. PRE-VALIDATION	Validate input against input schema (syntactical)
2. INPUT SEMANTICS	(optional) Assert business rules on <code>\$input</code> before transforming
3. TRANSFORMATION	Execute the <code>.utlx</code> body
4. POST-VALIDATION	Validate output against output schema (syntactical)
5. OUTPUT SEMANTICS	Run <code>validate { ... }</code> assertions on the result (semantic)

Validating the input semantically catches bad source data early — before a transformation is wasted on it (e.g. a Peppol invoice whose `vatId` country prefix doesn't match `country`). Validating the output

catches calculation and business-logic errors introduced by the mapping itself. Most pipelines use output semantics; input semantics is worth adding when the source is untrusted or expensive to process.

If semantic validation fails, the behavior depends on the validation policy:

- **STRICT:** transformation fails, output rejected, error returned
- **WARN:** warning logged with assertion message, output produced anyway

40.5 Semantic Validation in Practice Today

The `validate` block is not yet implemented. But you can achieve semantic validation today using `if/else` in the transformation body:

```
let total = sum(map($input.lines, (l) -> l.qty * l.unitPrice))
let calculatedTotal = sum(map($input.lines, (l) -> l.lineTotal))

if (total != calculatedTotal) {
  error: "Validation failed: total mismatch",
  expected: total,
  actual: calculatedTotal
} else {
  // normal output
  invoiceId: $input.id,
  total: total,
  ...
}
```

This works but mixes validation logic with transformation logic. The `validate` block will separate these concerns — the body produces output, the `validate` block checks it.

40.6 Common Semantic Validation Rules

40.6.1 Financial

- Invoice total = sum of line totals
- Line total = quantity x unit price
- Tax amount = taxable amount x tax rate
- Balance = previous balance + credits - debits
- IBAN checksum (modulo 97)
- Currency code is valid ISO 4217

40.6.2 Healthcare

- Patient age matches date of birth
- Medication dosage within approved range
- Diagnosis code (ICD-10) is valid for the patient's age/gender
- Referral date before appointment date
- BSN (Dutch) passes 11-check

40.6.3 E-Invoicing (Peppol)

- Buyer and seller are not the same entity
- Tax category matches the tax rate
- Document currency matches line amounts currency
- Allowance/charge amounts add up correctly
- Payment means code is valid for the country

40.6.4 Logistics

- Delivery date after order date

- Total weight = sum of line item weights
- Shipping address country matches delivery terms (Incoterms)
- Package dimensions within carrier limits

40.7 The Schematron Bridge

For organizations that already have Schematron rules (Peppol compliance, government XML), a future UTL-X feature could evaluate Schematron assertions against XML output:

```
%utlx 1.0
input json
output xml {schema: "invoice.xsd", schematron: "peppol-bis3.sch"}
---
// transformation body
```

The Schematron file contains the 65+ Peppol business rules. UTL-X would evaluate them against the XML output after post-validation. This provides Peppol compliance without reimplementing the rules in UTL-X — use the official Schematron files directly.

40.8 Status

- **Syntactical validation (7 schema formats):** implemented (Chapter 19)
- **Native assertion syntax (`validate block`):** planned — uses existing UTL-X expressions
- **Schematron support:** planned — for Peppol/UBL compliance
- **FEEL integration:** under consideration — for DMN/decision table workflows
- **Workaround today:** use `if/else` in the transformation body for semantic checks

41 API Contracts and Data Contracts

In enterprise integration, the most expensive bugs come from mismatched expectations: the producer sends data in one shape, the consumer expects another. API contracts and data contracts formalize these expectations — they are the agreements that prevent “it works on my machine” from becoming “it broke in production.”

This chapter covers the two dimensions of contracts, the major standards, and how UTL-X fits as the transformation layer between contract boundaries.

41.1 Two Dimensions of Contracts

41.1.1 Hierarchical Contracts (API-Centric)

Defined from the API perspective: “Here is my API, here are its operations.”

```
API Specification
├── Endpoints / Operations
│   ├── Request schema
│   └── Response schema
├── Authentication
├── Error definitions
└── Documentation
```

Examples: OpenAPI, RAML, SOAP/WSDL, AsyncAPI, gRPC `.proto`

Hierarchical contracts describe **how to access data** — endpoints, methods, authentication, protocols.

41.1.2 Relational Contracts (Data-Centric)

Defined from the data perspective: “Here is my data, here are its quality guarantees.”

```
Data Contract
├── Schema (structure of the data)
├── Quality rules (semantic validation)
├── SLA (freshness, availability, completeness)
├── Ownership (team, domain, contact)
└── Lineage (where data comes from)
```

Examples: Data Contract (datacontract.com), ODCS (PayPal), Data Mesh contracts

Relational contracts describe **what the data is and its guarantees** — independent of how it’s delivered.

41.1.3 The Key Difference

Aspect	Hierarchical (API)	Relational (Data)
Perspective	How to access data	What the data is
Transport	Tied to protocol (HTTP, gRPC, SOAP)	Transport-agnostic
Focus	Operations and endpoints	Quality and governance
Schema	Embedded in API spec	Standalone, reusable
SLA	Uptime and latency	Freshness and completeness
Ownership	API team	Data domain team
Example	GET /orders returns OrderResponse	Orders dataset has 99.9% completeness

Both are needed: hierarchical for API integration (calling services), relational for data pipelines (data quality, governance).

41.2 Hierarchical API Contract Standards

41.2.1 OpenAPI (REST APIs)

The dominant REST API specification standard (formerly Swagger). Versions 2.0, 3.0, and 3.1.

OpenAPI defines endpoints, operations, parameters, request/response bodies, security schemes, and servers. The data schema language is JSON Schema, embedded in the OpenAPI spec under **components/schemas**.

UTL-X can process OpenAPI specs as YAML input — extract endpoints, schemas, and parameters for transformation or documentation:

```
%utlx 1.0
input yaml
output json
---
// Extract all endpoints from an OpenAPI 3.0 spec
flatten(map(keys($input.paths), (path) ->
  map(keys($input.paths[path]), (method) -> {
    let op = $input.paths[path][method]
    path: path,
    method: toUpperCase(method),
    operationId: op.operationId,
    summary: op.summary ?? "",
    tags: op.tags ?? []
  })
))
```

41.2.2 SOAP / WSDL (XML Web Services)

The original API contract. WSDL defines operations, messages, port types, and bindings in XML. The schema language is XSD.

Still dominant in banking, insurance, government, and healthcare. UTL-X handles WSDL's XSD types directly — parse the types section, transform messages against the schema. See Chapter 22 for SOAP handling and Chapter 29 for XSD patterns.

41.2.3 AsyncAPI (Event-Driven APIs)

OpenAPI for event-driven architectures. Defines channels, messages, servers, and protocol bindings for Kafka, AMQP, MQTT, WebSocket, and Service Bus.

AsyncAPI supports multiple schema languages: JSON Schema, Avro, and Protobuf. UTL-X handles all three — making it suitable for transforming messages across event-driven systems regardless of the schema format.

41.2.4 gRPC / Protocol Buffers

Google's RPC framework. The contract is the **.proto** file that defines services, methods, and message types. UTL-X can read and write Protobuf schemas (Chapter 28), and the Go and Java wrappers (Chapter 34) communicate with UTLX via protobuf over stdio.

41.3 Abstract vs Concrete Contracts

Every API contract has two layers:

41.3.1 Abstract Contracts (Schema-Only)

Define the **shape** of data without API-specific details. Reusable across multiple APIs and protocols:

- JSON Schema: `{"type": "object", "properties": {"name": {"type": "string"}}}`
- XSD: `<xs:complexType name="CustomerType">...</xs:complexType>`
- Avro: `{"type": "record", "name": "Customer", "fields": [...]}`
- Protobuf: `message Customer { string name = 1; }`
- RAML DataType Fragment: standalone type definition

Abstract contracts answer: “**What does the data look like?**”

41.3.2 Concrete Contracts (Full API Specification)

Tie abstract schemas to specific endpoints, protocols, and operations:

- OpenAPI: `GET /customers` returns `CustomerResponse` (references JSON Schema)
- WSDL: `GetCustomer` operation returns `CustomerType` (references XSD)
- AsyncAPI: `orders-topic` carries `OrderCreated` message (references Avro)
- gRPC: `service CustomerService { rpc GetCustomer(...) returns (...); }`

Concrete contracts answer: “**How do I call this API and what do I get back?**”

41.3.3 The Relationship

Concrete contracts **reference** abstract contracts:

OpenAPI spec	→ <code>\$ref: "#/components/schemas/Customer"</code>	→ JSON Schema (abstract)
WSDL	→ <code><xsd:import .../></code>	→ XSD (abstract)
AsyncAPI	→ <code>\$ref: "../schemas/Order.avsc"</code>	→ Avro schema (abstract)

This separation enables schema reuse across multiple APIs, independent evolution (schema changes without API changes), and cross-protocol compatibility (same schema for HTTP and Kafka).

UTL-X works at the **abstract contract layer** — it reads and writes the schemas (JSON Schema, XSD, Avro, Protobuf, EDMX, Table Schema) that API contracts reference.

41.4 Relational Data Contract Standards

41.4.1 Data Contract (datacontract.com)

The open standard for data contracts in Data Mesh architectures:

```
dataContractSpecification: 0.9.3
id: orders-contract
info:
  title: Orders Dataset
  owner: order-team
schema:
  type: object
  properties:
    orderId: {type: string, required: true}
    total: {type: number, minimum: 0}
  quality:
    - type: completeness
      field: orderId
      threshold: 99.9%
  sla:
    freshness: PT1H # max 1 hour old (ISO 8601 duration)
    availability: 99.9%
```

Key difference from OpenAPI: no endpoints, no HTTP methods. It describes the **data**, not the API. Quality rules, SLA, and ownership are first-class concerns — not afterthoughts.

41.4.2 ODCS (Open Data Contract Standard — PayPal)

PayPal’s open-source data contract standard, more detailed than datacontract.com on quality and governance:

- **Quality:** accuracy, completeness, timeliness, uniqueness — with specific rules and thresholds
- **Security:** data classification (PII, sensitive, public), access control
- **Lineage:** source systems, transformations applied
- **Governance:** domain, steward, approvers, review cadence

41.4.3 Comparison

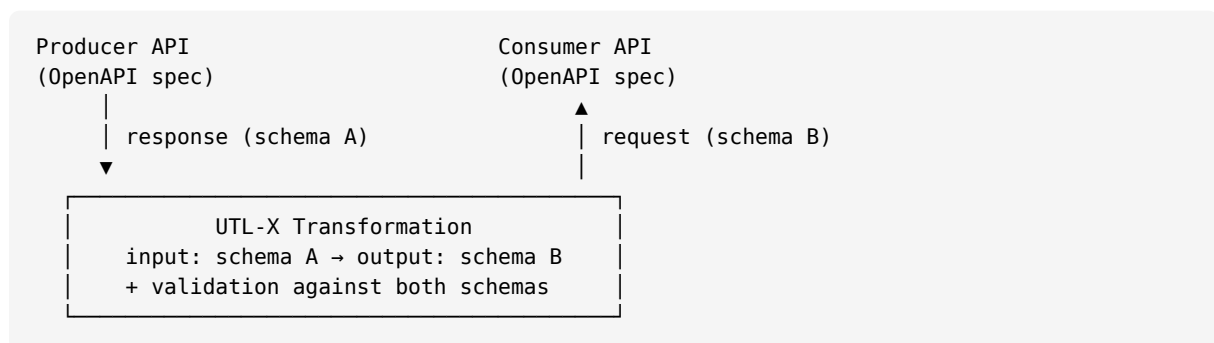
Aspect	datacontract.com	ODCS (PayPal)
Schema	JSON Schema-like	Custom + JSON Schema
Quality rules	Basic (threshold)	Detailed (accuracy, timeliness)
SLA	Freshness, availability	+ Latency, retention
Security	Minimal	Classification, PII tagging
Lineage	Minimal	Source systems, transforms
Governance	Owner, team	+ Domain, steward, approvers
Adoption	Broader community	PayPal + enterprises

Update — the two standards have merged. Since this comparison was first drawn, the ecosystem has converged on a single standard. With ODCS 3.1, the Data Contract Specification (datacontract.com) was deprecated in favour of the **Open Data Contract Standard** (ODCS), now an open-source project governed under the Linux Foundation’s Bitol initiative (bitol-io.github.io/open-data-contract-standard). New data-contract work should target ODCS; the two columns above are best read as ODCS’s heritage rather than two competing standards.

41.5 How UTL-X Fits in the Contract Landscape

41.5.1 UTL-X as the Transformation Layer

UTL-X sits **between** contract boundaries — where data from one contract must be transformed to match another:



The input schema comes from the producer’s contract. The output schema comes from the consumer’s contract. UTL-X transforms between them — and can validate against both (Chapter 19).

41.5.2 Schema Format Coverage

UTL-X supports all major abstract contract schema formats:

API Contract	Schema Format	UTL-X support
OpenAPI	JSON Schema	Read + write + validate
WSDL/SOAP	XSD	Read + write + validate
AsyncAPI	JSON Schema / Avro / Protobuf	All three supported
gRPC	Protobuf	Read + write
OData	EDMX/CSDL	Read + write + validate
Frictionless Data	Table Schema	Read + write + validate
datacontract.com	JSON Schema-like	Read via YAML input
ODCS	JSON Schema	Read via YAML/JSON input

41.5.3 Contract-Driven Transformation Development

The ideal workflow:

1. **Receive** both contracts: producer's API spec + consumer's API spec
2. **Extract** schemas from both contracts (JSON Schema, XSD, etc.)
3. **Write** the UTL-X transformation that maps schema A to schema B
4. **Validate** input against schema A (pre-validation) and output against schema B (post-validation)
5. **Test** with sample data generated from the schemas
6. **Deploy** with confidence: the transformation is contract-verified

UTL-X makes this practical because it reads the same schema formats that API contracts reference. The schemas in the OpenAPI spec or WSDL are the same schemas UTLXe validates against.

41.5.4 Data Contract Processing

UTL-X can process data contracts as YAML input — extracting schemas, quality rules, and metadata:

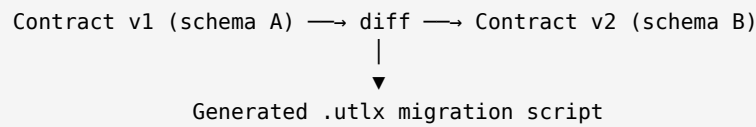
```
%utlx 1.0
input yaml
output json
---
// Extract the schema from a datacontract.com specification
{
  contractId: $input.id,
  owner: $input.info.owner,
  fields: entries($input.schema.properties) |> map((entry) -> {
    name: entry[0],
    type: entry[1].type,
    required: entry[1].required ?? false,
    description: entry[1].description ?? ""
  }),
  qualityRules: $input.quality ?? [],
  sla: $input.sla ?? {}
}
```

This transforms a data contract into a structured JSON report — useful for governance dashboards, contract registries, or generating validation rules.

41.6 The Contract-Driven Future

41.6.1 Contract Differencing

Given two versions of a contract (v1 and v2), UTL-X could detect schema differences and generate the transformation that bridges them:



This is schema evolution automation — detect what changed (renamed field, new required field, type change) and produce the mapping that handles it.

41.6.2 Contract as Test Oracle

Data contracts define what valid data looks like — quality thresholds, completeness rules, SLA targets. These can be converted into conformance tests:

- **Completeness:** generate test data with missing fields → verify transformation handles them
- **Type constraints:** generate boundary values → verify no overflow or truncation
- **Quality thresholds:** run against production sample → verify output meets contract SLA

The contract IS the specification. UTL-X transformations are the implementation. Conformance tests verify the implementation matches the specification.

42 Formats Not Yet Covered

UTL-X supports 5 data formats (JSON, XML, CSV, YAML, OData) and 6 schema formats (XSD, JSON Schema, Avro, Protobuf, EDMX, Table Schema). That's 11 formats — more than any competing tool. But the integration world has more formats. This chapter surveys the ones UTL-X does not yet support, explains why, and describes the path to adding them.

42.1 Medium Priority: EDI / EDIFACT

EDI (Electronic Data Interchange) is the backbone of logistics, retail, automotive, and government procurement. Two standards dominate:

- **UN/EDIFACT:** international standard, dominant in Europe. Used for: purchase orders (ORDERS), invoices (INVOIC), despatch advice (DESADV), customs declarations.
- **ANSI X12:** North American standard. Used for: purchase orders (850), invoices (810), advance ship notices (856).

EDI is segment-based — not XML, not JSON. A typical EDIFACT message:

```
UNH+1+ORDERS:D:96A:UN'
BGM+220+P0-001+9'
DTM+137:20260501:102'
NAD+BY+++Acme Corp'
LIN+1++WIDGET:SA'
QTY+21:100'
UNT+7+1'
```

Every segment has a tag (UNH, BGM, DTM), elements separated by +, and sub-elements separated by :. The segment terminator is ' '.

Why not yet: complex parsing (thousands of message types, version-dependent segment definitions), element-level validation rules, composite data elements. Adding EDI properly requires a segment dictionary — not just a parser.

Effort: large (parser + segment definitions + validation rules).

Current workaround: convert EDI to XML using an external tool (Bots, Smooks, or a commercial EDI converter), then process with UTL-X. This works but adds a preprocessing step.

Promising integration path: reverseXSL (Apache 2.0, Java) is an open-source “anything-to-XML” parser that handles EDIFACT, X12, IATA, SWIFT, and fixed-length records using regex-driven DEF (definition) files. It converts structured text formats to XML via a 4-step process: identify, cut, extract, validate. Since it's Java-based and Apache 2.0 licensed, it could be integrated directly into UTL-X as a pre-parser — input `edi` would invoke reverseXSL with the appropriate DEF file, producing XML that UTL-X transforms further. The DEF file approach is similar in spirit to UTL-X's declarative philosophy: describe the format, let the engine parse it.

Would enable: direct EDI-to-JSON transformation — eliminating the XML intermediate step. Huge value for logistics and retail integration.

42.2 Medium Priority: HL7 v2

HL7 version 2 is the pipe-delimited healthcare messaging standard. Still the most widely deployed health data exchange format in the world — hospitals, labs, pharmacies, and insurance companies send billions of HL7 v2 messages daily.

```
MSH|^~\&|SendApp|SendFac|RecvApp|RecvFac|20260501||ADT^A01|MSG001|P|2.5
PID|||12345^^^MRN||Simpson^Homer^J||19550312|M
```

```
NK1|1|Simpson^Marge|SP0
PV1|1|I|ICU^Bed3
```

Segments (MSH, PID, NK1, PV1) are separated by newlines. Fields are separated by `|`. Components by `^`. Sub-components by `&`. Repetitions by `~`. Escape character `\`.

Why not yet: specialized parser needed (4 levels of delimiters), segment definitions vary by message type and version (2.3, 2.4, 2.5, 2.7).

Effort: medium (well-documented segment structure, many reference implementations).

Current workaround: convert HL7 v2 to JSON using HAPI (Java) or Mirth Connect, then transform with UTL-X to FHIR R4. Alternatively, reverseXSL (see EDI section above) can parse HL7 v2 pipe-delimited messages to XML using DEF files — the same tool that handles EDI and IATA formats.

Would enable: direct HL7 v2 to FHIR transformation — the “healthcare holy grail” for modernization projects.

42.3 Medium Priority: Parquet / Arrow

Apache Parquet is the columnar storage format for big data — used by Spark, Databricks, Snowflake, BigQuery, and pandas. Apache Arrow is the in-memory columnar format.

Why not yet: binary format requiring native libraries (not pure JVM). Complex internal structure: row groups, column chunks, page headers, compression codecs (Snappy, Zstd, LZ4).

Effort: large.

Current workaround: export Parquet to CSV or JSON via Spark/pandas, then process with UTL-X.

Would enable: direct data lake integration — read Parquet files, transform, write back without leaving UTL-X.

42.4 Low Priority: TOML

Tom’s Obvious Minimal Language — the configuration format of the Rust ecosystem (`Cargo.toml`), Python (`pyproject.toml`), and Hugo.

Why not yet: lower priority — YAML covers most configuration use cases. TOML is stricter than YAML (no ambiguous types, explicit table syntax) but less widely used in integration.

Effort: small (TOML is simpler than YAML, well-defined spec).

Would enable: `Cargo.toml` to `package.json` conversion, `pyproject.toml` processing, configuration format migration.

42.5 Low Priority: JSON-LD

JSON for Linked Data — W3C standard for semantic web data in JSON. Used by Schema.org, Google Knowledge Graph, and ActivityPub (Mastodon/Fediverse).

Key difference from JSON: `@context`, `@id`, `@type`, `@graph` annotations that link data to ontologies via URIs.

Why not yet: specialized use case. The JSON is parseable today (input `json`), but `@context` resolution (following URIs to load vocabulary definitions) is complex.

Effort: medium (parsing is JSON, but context resolution requires HTTP fetching and RDF reasoning).

Current workaround: parse as JSON, access `@context/@type` with bracket notation.

42.6 Low Priority: MessagePack / CBOR

Binary JSON formats — smaller and faster than text JSON, same data model.

- **MessagePack:** used by Redis, FluentBit, some game servers
- **CBOR (RFC 8949):** used by IoT devices, WebAuthn, COSE (FIDO2)

Why not yet: lower priority — same data model as JSON, just binary encoding.

Effort: small (same UDM mapping as JSON, just a different parser/serializer).

Would enable: IoT message transformation without JSON round-trip, Redis data processing.

42.7 Low Priority: GraphQL SDL

GraphQL Schema Definition Language — defines types, queries, mutations, and subscriptions.

Why not yet: GraphQL responses are JSON (already supported). The SDL is a schema language similar to Protobuf — types, fields, enums, unions.

Effort: medium (SDL parser, response flattening for nested queries).

What could be added: SDL to JSON Schema conversion, query result normalization.

42.8 Low Priority: ORC (Apache Optimized Row Columnar)

Apache ORC — columnar storage optimized for Hive workloads. Similar to Parquet but with built-in ACID transaction support and predicate pushdown.

Why not yet: binary format, Hadoop-specific, overlaps with Parquet use cases.

Effort: large.

Current workaround: export via Hive/Spark to CSV/JSON.

42.9 Low Priority: RAML

RESTful API Modeling Language by MuleSoft. YAML-based API description with its own type system.

Two versions with important differences:

- **RAML 0.8:** basic `!include` (untyped raw text insertion), `schema:` keyword for external JSON Schema references
- **RAML 1.0:** typed fragments (`DataType`, `Trait`, `ResourceType`, `Library`), native type system (unions, inheritance, facets), `type: !include user.json` for JSON Schema inclusion

RAML 1.0 `DataType` fragments are the key migration artifact — standalone type definitions used across MuleSoft projects. Converting these to JSON Schema or USDL is the primary value for UTL-X.

Why not yet: declining relative to OpenAPI 3.x, MuleSoft-centric ecosystem.

Effort: medium (YAML-based parsing, but the type system with facets and inheritance is complex).

Current workaround: convert RAML to OpenAPI 3.0 via `oas-raml-converter`, then process with UTL-X.

42.10 Potential: JSON Sample Generation (Mock Data)

Not a format but a capability: given a schema, generate a conforming sample instance.

Use cases:

- **API mocking:** generate realistic sample responses for testing
- **Documentation:** show “what does a valid message look like?”
- **Test data:** seed test suites with conforming instances
- **Onboarding:** new developers see concrete examples from abstract schemas

How it could work:

```
%utlx 1.0
input jsch           // or xsd, avro, proto
output json {mode: "sample"} // generate a sample instance
---
$input
```

The serializer would use `%default` and `%example` values where defined, generate realistic fake data for types (string to "example", integer to 42), respect constraints (`minLength`, `pattern`, `enum` picks first value), and build nested structures following the schema hierarchy.

Not yet implemented but a natural extension of the schema-as-data model. Related tools: json-schema-faker, Prism (Stoplight), WireMock.

42.11 Declining: Apache Thrift / FlatBuffers

Apache Thrift: Facebook's RPC framework (predecessor to gRPC). Declining usage — gRPC/Protobuf won the mindshare battle. Low priority unless specific customer demand.

FlatBuffers: Google's zero-copy serialization format. Used in game engines and performance-critical mobile apps. Very specialized, small user base for integration.

42.12 Format Priority Matrix

Format	Demand	Effort	Priority	Workaround
EDI/EDIFACT	High	Large	Medium	EDI to XML externally
HL7 v2	High	Medium	Medium	HL7 to JSON via HAPI/Mirth
Parquet	High	Large	Medium	Export via Spark/pandas
JSON Sample Gen	Medium	Small	Medium	Manual transformation
TOML	Medium	Small	Low	YAML covers most cases
FHIR native mode	Medium	Large	Low	@value accessor works today
JSON-LD	Low	Medium	Low	Parse as JSON, manual @context
MessagePack/CBOR	Low	Small	Low	JSON round-trip
GraphQL SDL	Medium	Medium	Low	JSON responses work
ORC	Medium	Large	Low	Export via Hive/Spark
RAML	Medium	Medium	Low	RAML to OAS converter
Thrift	Low	Medium	None	Protobuf preferred
FlatBuffers	Low	Large	None	Too specialized

The priority reflects both market demand and strategic value. EDI and HL7 v2 would open large markets (logistics and healthcare) currently served by expensive proprietary tools. TOML and MessagePack are small effort with moderate value. The rest have viable workarounds that reduce urgency.

43 Competitive Landscape

UTL-X occupies a unique position: an open-source, format-agnostic transformation language with a production engine. This chapter maps the competitive landscape — CLI tools, enterprise platforms, hyperscaler services, ERP vendors, and open-source alternatives — and shows where UTL-X fits.

43.1 Command-Line Transformation Tools

43.1.1 jq

The JSON Swiss Knife. Pipe-friendly, ubiquitous on Linux, fast. Every developer who processes JSON on the command line knows jq.

Strengths: lightweight (single binary), installed everywhere, great for shell scripts, concise syntax for JSON queries.

Limitations: JSON only. Cannot process XML, CSV, YAML, or OData. No schema validation, no production engine, no user-defined functions.

UTL-X: all formats with the same syntax. The `-e` expression mode mirrors jq deliberately (Chapter 5). jq users feel at home — and gain XML, CSV, YAML, and OData for free.

43.1.2 yq

Mike Farah’s multi-format processor: YAML, JSON, XML, CSV, TOML, properties files. The closest CLI competitor to UTL-X for quick format conversion.

Strengths: multi-format, Go binary, jq-like syntax, actively maintained.

Limitations: simple field access and modification only. No complex transformations (no `map`, `filter`, `reduce`), no user-defined functions, no schema validation, no production engine, no pipeline chaining.

UTL-X: 652 stdlib functions, schema validation, compiled strategy (86K msg/s), production engine, IDE integration. yq is a Swiss Army knife; UTL-X is a workshop.

43.1.3 JSONata

Declarative JSON query and transformation language. Elegant expression syntax, runs in browsers and Node.js.

Strengths: clean syntax, lightweight, used in Node-RED and some API gateways.

Limitations: JSON only, no XML/CSV/YAML, no schema validation, no production engine, limited function library.

UTL-X: all formats, production engine, schema validation, 652 functions vs JSONata’s 30.

43.1.4 XSLT

The W3C standard for XML transformation. 25+ years old, supported by every XML tool.

Strengths: standardized, powerful template matching (push model), vast ecosystem, XPath.

Limitations: XML only, verbose syntax (XML for transforming XML), steep learning curve, no JSON/CSV/YAML output without extensions.

UTL-X: all formats, modern functional syntax, Chapter 35 provides a migration guide. XSLT is still dominant in government, banking, and healthcare legacy — but new projects increasingly choose alternatives.

43.2 Enterprise Integration Platforms (iPaaS)

43.2.1 MuleSoft / DataWeave

The closest competitor in language design. DataWeave is format-agnostic and functional — the same paradigm as UTL-X.

Strengths: mature, enterprise-grade, Salesforce ecosystem, Anypoint Platform with 400+ connectors, strong developer community.

DataWeave similarities: functional expressions, @ for XML attributes, map/filter/reduce, writeAttributes option (same name, same default — deliberate compatibility).

Limitations: proprietary (closed source), expensive (\$1,250+/month for Anypoint Platform), vendor lock-in via Salesforce, transformations locked inside the platform.

UTL-X: open source (AGPL), \$35/month on Azure Marketplace, .utlx files version-controlled in Git, no vendor lock-in. Chapter 35 covers DataWeave migration.

43.2.2 TIBCO BusinessWorks

Visual mapper with XSLT generation. Enterprise customer base in telecommunications, financial services, and manufacturing.

Strengths: mature visual IDE, drag-and-drop field mapping, enterprise support contracts.

Limitations: XML-centric (JSON requires conversion steps), expensive licensing (\$50K+/year), heavy runtime, Russian Doll XSD patterns (Chapter 29).

UTL-X: native JSON/CSV/YAML, lightweight container vs full application server, 10-100x cheaper. Chapter 35 covers BW migration, Chapter 29 covers XSD pattern conversion.

43.2.3 Boomi

Cloud-native iPaaS with visual data mapping.

Strengths: easy-to-use UI, cloud-native, good connector library, citizen developer friendly.

Limitations: mapping is visual-only (no scripting language), limited transformation logic, pricing per connection, transformations not version-controllable.

UTL-X: scriptable transformations in .utlx files, testable (conformance suite), version-controlled (Git).

43.2.4 Informatica / IICS

Enterprise data integration, ETL/ELT, data quality, and MDM.

Strengths: data quality, master data management, governance, batch processing at scale.

Limitations: heavy, expensive, batch-oriented (not real-time per-message), XML-centric mapping.

UTL-X: real-time (86K+ msg/s), lightweight container, event-driven. Different market segment — Informatica is for data warehousing, UTL-X is for integration messaging.

43.3 Hyperscaler Native Tools

Each cloud provider offers some transformation capability — but none have a format-agnostic transformation language:

Capability	Azure	AWS	GCP	UTL-X
JSON transformation	Liquid (limited)	Lambda (code)	Jsonnet (limited)	Native
XML transformation	XSLT (Data Mapper)	Lambda (code)	Apigee (basic)	Native
CSV transformation	No	No	No	Native

YAML transformation	No	No	No	Native
OData transformation	No	No	No	Native
Schema validation	XSD only	No	No	7 formats
Declarative language	XSLT/Liquid	No	Jsonnet	UTL-X
Cost per month	\$300+ (Integration Account)	Lambda pricing	\$300+ (Data Fusion)	\$35

43.3.1 Azure

Azure's transformation options: Data Mapper (XSLT-based, JSON requires internal conversion), Liquid templates (JSON only), Azure Functions (custom code), Integration Account (\$300+/month for XSLT maps). UTL-X on Azure: Container App at \$35/month with all formats, available on the Marketplace.

43.3.2 AWS

AWS has no native transformation language. Step Functions orchestrate but don't transform. Lambda requires custom code per transformation. EventBridge Input Transformer does basic JSON path extraction (very limited). Glue is batch ETL (Spark). UTL-X on AWS: ECS/Fargate at \$44/month.

43.3.3 GCP

Application Integration has Jsonnet templates (JSON only). Cloud Data Fusion is batch ETL. Dataflow (Apache Beam) is stream processing, not data mapping. Apigee has basic JSON/XML mediation policies. UTL-X on GCP: Cloud Run at \$44/month with native Pub/Sub push.

43.4 ERP Vendors

Every ERP exposes data via APIs (REST, SOAP, OData). None have a format-agnostic transformation language. UTL-X bridges the gap:

ERP API response → UTL-X → target format (e-invoice, FHIR, EDI, canonical JSON)

ERP	API format	UTL-X integration
SAP	OData, IDoc (XML), BAPI	OData/XML → UTL-X → any format
Dynamics 365	OData JSON	OData → UTL-X → UBL, FHIR, CSV
Oracle	REST (JSON), SOAP (XML)	JSON/XML → UTL-X → any format
Odoo	JSON-RPC, REST (JSON)	JSON → UTL-X → any format
SAP CPI	Visual Mapping + Groovy	Alternative: UTL-X for SAP-to-non-SAP

43.5 Open-Source Alternatives

43.5.1 Apache Camel

Integration framework (Java) with 300+ connectors and Enterprise Integration Patterns. Not a transformation language — transformation happens via Java code, XSLT, or embedded JSONata.

UTL-X advantage: declarative transformation language vs imperative Java code. Camel does routing and connectivity; UTL-X does transformation. They can work together — Camel routes messages, UTL-X transforms them (via the Java SDK, Chapter 34).

43.5.2 Smooks

Java-based transformation framework with support for XML, JSON, CSV, EDI, and Java objects.

Strengths: EDI support (EDIFACT, X12) — one of few open-source tools that handles EDI natively.

UTL-X advantage: simpler syntax, cloud-native (container), active development. When UTL-X adds EDI support (Chapter 42), Smooks' main differentiator disappears.

43.5.3 Ballerina

Integration-oriented programming language by WSO2 with built-in HTTP, gRPC, GraphQL, and data mapping.

Strengths: language-level integration support, strong type system, sequence diagrams from code.

UTL-X advantage: focused on transformation (not general-purpose programming), simpler learning curve, no WSO2 ecosystem dependency.

43.6 Positioning Matrix

Tool	Type	Formats	Open source	Production engine	Price
UTL-X	Language + Engine	All 11	Yes (AGPL)	Yes (86K msg/s)	\$35/mo
jq	CLI	JSON only	Yes	No	Free
yq	CLI	Multi (basic)	Yes	No	Free
JSONata	Language	JSON only	Yes	No	Free
XSLT	Language	XML only	Standard	No	Free
DataWeave	Language + Platform	Multi	No	Yes	\$1,250+/mo
TIBCO BW	Platform	XML + JSON	No	Yes	Enterprise
Boomi	Platform	Visual	No	Yes	Enterprise
Azure DM	Feature	XML + JSON	No	Azure only	\$300+/mo
AWS	Custom code	Any (code)	N/A	Lambda	Usage-based
GCP	Feature	JSON	No	GCP only	\$300+/mo
Camel	Framework	Multi (code)	Yes	No (framework)	Free
Smooks	Framework	Multi + EDI	Yes	No (library)	Free

43.7 UTL-X's Unique Position

No other tool combines all of these:

1. **Format-agnostic:** 5 data formats + 6 schema formats in one language
2. **Open source:** AGPL license, no vendor lock-in, `.utlx` files you own
3. **Production engine:** 86K+ msg/s compiled strategy, not just a CLI
4. **Schema validation:** 7 validators, pre/post validation orchestrator
5. **Cloud-native:** Docker container, Azure/GCP/AWS Marketplace
6. **AI-friendly:** natural language round-trip, MCP server (Chapter 45)
7. **Affordable:** \$35/month vs \$300-1,250+/month for alternatives

The closest competitor is DataWeave — same paradigm, same `@` convention, same `writeAttributes` option. But DataWeave is proprietary, expensive, and locked inside MuleSoft. UTL-X is open, portable, and 35x cheaper.

44 Open-M: The Integration Controller

UTL-X transforms data. But in a real integration landscape, transformation is one step in a larger flow: receive a message, route it to the right transformation, validate the output, deliver it to the target, handle errors, retry on failure, log for compliance. That orchestration layer is what Open-M provides.

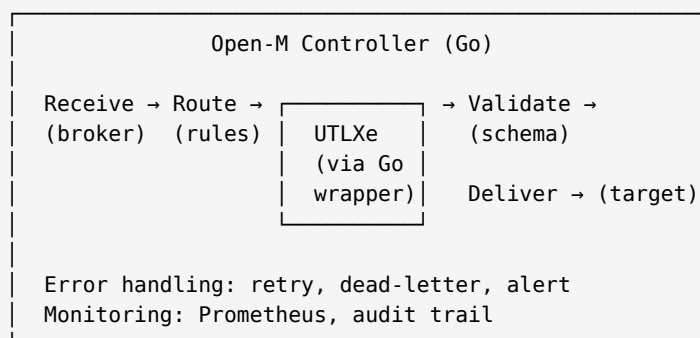
Open-M is in active development. This chapter previews the architecture and concepts. UTL-X is fully functional without Open-M — you can use it standalone with Docker, Dapr, or any orchestration tool. Open-M is an optional layer on top.

44.1 What Is Open-M?

Open-M is an open-source integration controller written in Go. It manages the lifecycle of messages as they flow between systems — receiving, transforming, validating, routing, and delivering.

Think of it as:

- **Open-M** = the orchestra conductor (decides what happens, when, and where)
- **UTL-X** = the musicians (perform the transformation work)



44.2 Why Open-M + UTL-X?

Separation of concerns:

Open-M handles	UTL-X handles
WHERE data goes (routing)	WHAT happens to data (transformation)
WHEN to process (scheduling, triggers)	HOW to transform (expressions, functions)
Error handling (retry, dead-letter)	Data validation (schema, assertions)
Monitoring and audit trail	Format conversion and enrichment
Connector management	650+ stdlib functions

Each does one thing well. UTL-X can be used WITHOUT Open-M (standalone CLI, engine, Docker). Open-M can use other transformation engines — but UTL-X is the default.

44.3 MPPM: Message Processing Pipeline Model

44.3.1 What Is MPPM?

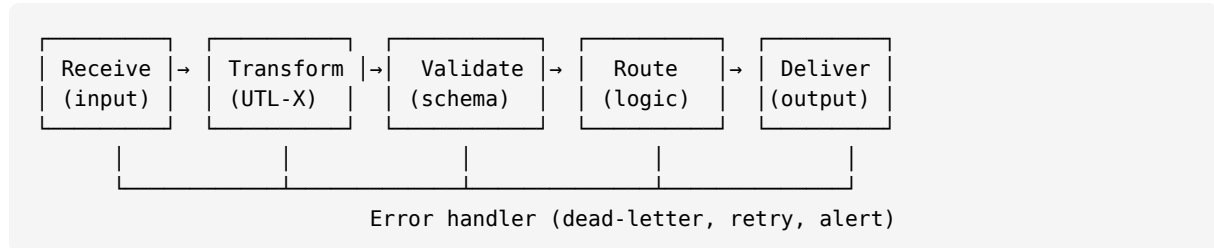
MPPM (Message Processing Pipeline Model) is Open-M's core concept: every integration is a **message pipeline** — an ordered sequence of processing stages that a message flows through.

Like BPMN standardizes business processes, MPPM standardizes message flows. It defines:

- How messages flow through processing stages

- What happens at each stage (transform, validate, route, enrich)
- How errors are handled (retry, dead-letter, compensate)
- How stages compose into flows

44.3.2 Pipeline Stages



Each stage is a **plugin** — swap implementations without changing the pipeline definition. UTL-X is the default Transform stage plugin. The Validate stage uses UTLXe’s schema validators. The Receive and Deliver stages connect to message brokers, APIs, file systems, or databases.

44.3.3 The Message Envelope

Every message in Open-M travels in a standard envelope:

```

{
  "messageId": "msg-550e8400",
  "correlationId": "flow-xyz-001",
  "timestamp": "2026-04-30T10:30:00Z",
  "source": "dynamics365",
  "contentType": "application/json",
  "headers": {
    "transformId": "order-to-invoice",
    "priority": "normal",
    "retryCount": 0
  },
  "payload": "{\"orderId\": \"ORD-001\", ...}"
}
  
```

The envelope carries **metadata** about the message — ID, correlation, source, content type, processing headers. The payload carries the actual **data**. UTL-X transforms the payload. Open-M manages the envelope.

This separation means UTL-X never needs to know about routing, retries, or correlation — it receives a payload, transforms it, returns the result. Open-M handles everything else.

44.3.4 How MPPM Maps to UTL-X

MPPM concept	UTL-X equivalent
Pipeline	Pipeline chaining (Chapter 20)
Transform stage	.utlx transformation file
Validate stage	Schema validation (Chapter 19)
Message payload	\$input
Message headers	Accessible via transport headers
Error handling	try/catch in .utlx, dead-letter in pipeline
Stage configuration	transform.yaml / output options
Multiple pipelines	Multiple UTLXe bundles

A single UTL-X transformation IS an MPPM Transform stage. An UTLXe pipeline IS an MPPM pipeline restricted to transformation. Open-M adds routing, error handling, monitoring, and non-transformation stages around it.

44.4 Open-M Architecture

44.4.1 Go-Based Controller

Open-M is written in Go — lightweight, fast startup, single binary, native Kubernetes integration. It communicates with UTLXe via the Go wrapper (Chapter 34) using protobuf over stdio.

Why Go for the controller and Kotlin/JVM for the engine?

- **Controller (Go):** needs fast startup, low memory, native Kubernetes client, lightweight HTTP server. Go excels here.
- **Engine (Kotlin/JVM):** needs 650+ stdlib functions, schema validation, bytecode compilation, SnakeYAML/Jackson parsers. The JVM ecosystem provides these.

The Go wrapper bridges them — the controller spawns UTLXe as a subprocess and communicates via the uniform SDK interface (Chapter 34).

44.4.2 Flow Definitions

Integration flows are defined in YAML:

```
name: order-to-invoice
version: "1.0"
stages:
  - name: receive
    type: azure-service-bus
    config:
      queue: orders-inbound

  - name: transform
    type: utlx
    config:
      transformation: order-to-ubl.utlx
      strategy: COMPILED

  - name: validate
    type: schema
    config:
      schema: ubl-invoice-2.1.xsd
      policy: STRICT

  - name: deliver
    type: http
    config:
      url: https://peppol-ap.example.com/submit
      method: POST

errorHandler:
  type: dead-letter
  config:
    queue: orders-dlq
    maxRetries: 3
    retryDelay: PT30S
```

This YAML defines a complete integration flow: receive from Service Bus, transform with UTL-X, validate against UBL XSD, deliver to Peppol Access Point, dead-letter on failure with 3 retries. The `.utlx` file is referenced by name — change the transformation without touching the flow definition.

44.4.3 Plugin Architecture

Every stage type is a plugin:

- **Receive plugins:** Azure Service Bus, Kafka, GCP Pub/Sub, AWS SQS, HTTP webhook, file watcher, MQTT
- **Transform plugins:** UTL-X (default), pass-through, custom (via HTTP call)
- **Validate plugins:** UTLXe schema validator, custom (via HTTP call)
- **Route plugins:** content-based routing, header-based routing, round-robin
- **Deliver plugins:** HTTP, Service Bus, Kafka, file, SMTP, database insert

Adding a new connector means implementing the plugin interface — not modifying the core controller.

44.5 What This Means for UTL-X Users

44.5.1 If You're Using UTL-X Standalone

Nothing changes. UTL-X works perfectly on its own — CLI for development, UTLXe for production, Docker for deployment. Open-M is optional.

44.5.2 If You Need Orchestration

When you outgrow “just transformation” and need routing, retry logic, dead-letter handling, flow monitoring, and audit trails — Open-M provides it. Your `.utlx` files work unchanged in Open-M. Your UTLXe deployment works unchanged. You don't rewrite anything — you wrap your transformations in MPPM flow definitions.

44.5.3 The Growth Path

Level 1: utlx CLI	(development, scripting)
Level 2: UTLXe container	(production, high throughput)
Level 3: UTLXe + Dapr	(broker integration, cloud-native)
Level 4: Open-M + UTLXe	(full orchestration, enterprise-grade)

Each level adds capabilities without replacing the previous one. Your `.utlx` files, your conformance tests, your schemas — all carry forward unchanged.

45 AI and UTL-X

45.1 The Natural Language Round-Trip

Most programming languages have a large gap between what you *say* and what you *write*. A business analyst says “nest the order lines under each order by order ID” — and the developer writes 20 lines of imperative Java with nested loops, hash maps, and null checks. The analyst cannot read the code back and verify it matches their intent.

UTL-X was designed to minimize this gap:

```
Business analyst says:
"Nest the order lines under each order by order ID,
 look up the customer name from the customer table,
 and calculate the total per order."

Developer writes:
let orders = nestBy($input.orders, $input.lines,
  (o) -> o.orderId, (l) -> l.orderId, "lines")

map(orders, (order) -> {
  let customer = lookupBy(order.customerId, $input.customers, (c) -> c.id)
  orderId: order.orderId,
  customerName: customer?.name ?? "Unknown",
  total: sum(map(order.lines, (l) -> l.qty * l.price))
})

Anyone reads back:
"Nest lines into orders by orderId.
 Look up customer by customerId.
 Map orders to orderId, customerName, and total."
```

The source code and the specification are almost the same sentence. This is not an accident — it’s the result of deliberate naming decisions: `nestBy` reads as “nest by”, `lookupBy` reads as “look up by”, `filter` reads as “filter where”, `sortBy` reads as “sort by”. The function names ARE the natural language.

This has a profound implication for AI.

45.2 Why UTL-X Is Uniquely Suited for AI

The distance between natural language and code determines how well an AI can generate that code — and how easily a human can verify the result. UTL-X has the shortest distance of any transformation language:

Language	“Nest lines under orders by key”	Gap
Java	20+ lines: HashMap, for-loops, null checks, generics	Huge
XSLT	15+ lines: xsl:for-each-group, xsl:key, templates	Large
DataWeave	3-5 lines: groupBy + mapObject	Medium
jq	Not possible (JSON only, no multi-array join)	N/A
UTL-X	<code>nestBy(orders, lines, ..., "lines")</code>	Minimal

A smaller gap means:

- **LLMs generate more accurate code** — less translation needed
- **Fewer iteration rounds** — the first attempt is closer to correct
- **Humans can verify the output** — a business analyst can read `nestBy(..., "lines")` and confirm “yes, that’s what I meant”

- **Errors are obvious** — wrong function name or wrong key is visible to non-programmers

45.2.1 The Constrained Domain Advantage

General-purpose languages have millions of APIs, design patterns, and architectural choices. An LLM must choose between Java Streams, for-loops, Apache Commons, Guava, and a dozen other approaches for the same task. The search space is enormous.

UTL-X has:

- 652 stdlib functions (not 50,000 APIs)
- One paradigm: functional, expression-based (not OOP + functional + imperative)
- One input: `$input` (not dependency injection, configuration objects, database connections)
- One output: the body expression (not return statements, print, side effects)
- Declarative intent: “what to produce” not “how to produce it”

This makes UTL-X one of the most LLM-friendly languages in existence. The model has fewer choices to make, and more of them are correct.

45.3 What Works Today

General-purpose LLMs (Claude, GPT-4, Gemini) can already generate UTL-X code when given context about the language:

Human: "Convert this XML order to JSON. Include the order ID, customer name, and a lines array with product and quantity."

LLM generates:

```
%utlx 1.0
input xml
output json
---
{
  orderId: $input.Order.@id,
  customer: $input.Order.Customer,
  lines: map($input.Order.Lines.Line, (line) -> {
    product: line.Product,
    quantity: toNumber(line.Quantity)
  })
}
```

This works because UTL-X syntax resembles JSON and JavaScript — familiar territory for any LLM trained on code. Quality is 60-80% correct on first attempt for simple transformations. The main failure modes: hallucinated function names, invented syntax, and missed edge cases (null handling, array-vs-single-element).

45.3.1 The MCP Server

UTL-X includes an MCP (Model Context Protocol) server that provides LLMs with the language reference, function signatures, and examples as structured context. This bridges the gap without fine-tuning — the LLM receives exactly the information it needs to generate correct UTL-X.

The MCP server turns any general-purpose LLM into a UTL-X-aware assistant. Combined with the IDE’s live preview, the workflow becomes:

1. Describe what you want in natural language
2. LLM generates a `.utlx` transformation via MCP
3. IDE shows the output instantly (live preview)
4. Adjust the description or edit the code directly
5. Iterate until the output matches expectations

45.4 The Vision: AI-Native Transformation Development

45.4.1 Token Efficiency

Today, each LLM conversation requires injecting UTL-X context — language syntax, relevant functions, format conventions. This costs 10,000-15,000 tokens before the actual question. A fine-tuned model or well-configured RAG system would reduce this to under 1,000 tokens — a 90%+ reduction in cost per transformation.

Scenario	Without optimization	With RAG or fine-tuning
Context per request	15,000 tokens	500-1,000 tokens
Output quality	60-80% correct	85-95% correct
Iterations to correct	2-4 rounds	0-1 rounds
Cost per transformation	\$0.12 - \$0.60	\$0.01 - \$0.05

45.4.2 Three Approaches

RAG (Retrieval-Augmented Generation): Index all UTL-X documentation, examples, and stdlib functions. When a user asks a question, retrieve the relevant context and inject it into the prompt. No fine-tuning needed — works with any LLM. Best as an immediate, low-cost starting point.

LoRA Fine-Tuning: Train a lightweight adapter on top of an open model (Llama, Mistral) using UTL-X examples. The model learns the language natively — no context injection needed. Cost: \$500-2,000 in compute. Requires 1,000-5,000 hand-crafted example pairs.

Frontier Model Fine-Tuning: Fine-tune GPT-4o or Claude using the provider's API. Highest quality but higher cost and vendor dependency. Best for a premium product tier.

The recommended path: start with RAG (immediate value), collect user interactions as training data, fine-tune when enough examples exist.

45.4.3 Available Training Data

UTL-X already has a rich corpus for AI training:

- 652 stdlib function definitions with signatures, descriptions, and examples
- 453+ conformance test cases (input + transformation + expected output)
- 100+ real-world examples across JSON, XML, CSV, YAML, OData
- Complete language grammar and parser specification
- USDL schema language specification
- This book: 200+ pages of explanations, patterns, and worked examples

The conformance suite is particularly valuable — it provides *verifiable* quality metrics. A fine-tuned model's output can be tested against the suite: not “does it look right?” but “do 453 tests pass?”

45.5 Capabilities of an AI-Assisted UTL-X Workflow

What becomes possible when the natural language round-trip is fast and reliable:

45.5.1 Generation from Spec

"I have a Dynamics 365 sales order in OData JSON format.
Convert it to a Peppol BIS 3.0 UBL invoice in XML.
Include all line items, VAT calculation at 21%,
and the customer's billing address."

The LLM generates a complete `.utlx` transformation. The developer reviews it in the IDE with live preview against sample data. Minutes instead of hours.

45.5.2 Migration from Other Tools

"Here's my XSLT stylesheet that converts orders to invoices.
Rewrite it in UTL-X."

The LLM reads the XSLT, understands the intent, and produces equivalent UTL-X. This is more reliable than generic code translation because UTL-X's function names map closely to XSLT operations: `xsl:for-each` becomes `map()`, `xsl:if` becomes `if/else`, `xsl:sort` becomes `sortBy()`.

45.5.3 Explanation and Documentation

"Explain what this transformation does in plain English."

Because UTL-X code already reads close to English, the LLM's explanation is nearly a paraphrase of the code itself. This is the natural language round-trip completing its circle — code that was generated from English is explained back as English.

45.5.4 Test Generation

"Generate edge-case test data for this transformation.
Include: empty arrays, null fields, special characters,
maximum field lengths, and boundary values."

The LLM reads the transformation, infers what inputs it expects, and generates test cases that exercise edge conditions. The conformance suite format (YAML with input + expected output) makes this directly usable.

45.6 AI at Design-Time Only

A critical design principle: **AI assists at design-time, never at runtime.**

The `.utlx` file is the transformation. It is deterministic, inspectable, testable, and version-controlled. There is no AI inference in the production engine's hot path. When UTLXe processes 86,000 messages per second, every message is transformed by the same deterministic code — no LLM call, no API latency, no stochastic variation.

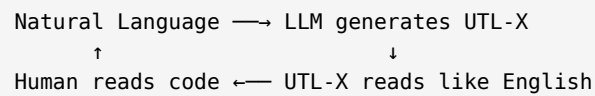
This separation matters for:

- **Compliance:** regulated industries require deterministic, auditable transformations. "An AI decided this field mapping" is not acceptable in banking or healthcare.
- **Performance:** LLM calls take 500ms-5s. Transformation takes 0.01ms. AI in the hot path would reduce throughput by 50,000x.
- **Reliability:** LLMs are stochastic — the same prompt can produce different outputs. Production transformations must produce identical output for identical input, every time.
- **Debugging:** when a transformation produces wrong output, you read the `.utlx` file and find the bug. There is no hidden AI layer to investigate.

AI generates the code. Humans review and test it. The engine executes it deterministically. Each layer does what it's best at.

45.7 The Virtuous Circle

UTL-X's natural language affinity creates a virtuous circle:



The better the function names map to English, the better the LLM generates code, the easier it is to verify, the more trust users place in the AI workflow, the more transformations get built, the more training data exists, the better the LLM becomes.

This circle only works when the programming language is designed — from the ground up — to minimize the distance between human intent and machine instruction. That is what UTL-X is.

46 Why Kotlin and GraalVM

“Why didn’t you write it in Go?” “Why not Rust?” “Why not TypeScript?” These are the most common questions about UTL-X’s technology choices. This chapter explains the reasoning — and why the answers matter for users, not just developers.

46.1 Why Kotlin

Kotlin was chosen for UTL-X because its language features map directly to the problems a transformation engine must solve:

46.1.1 Sealed Classes for the UDM Type Hierarchy

UTL-X’s Universal Data Model has a fixed set of types: `Scalar`, `Object`, `Array`, `DateTime`, `Date`, `Time`, `Binary`, `Lambda`. Kotlin’s sealed classes enforce this at compile time:

```
sealed class UDM {
    class Scalar(val value: Any?) : UDM()
    class Object(val properties: Map<String, UDM>) : UDM()
    class Array(val elements: List<UDM>) : UDM()
    class DateTime(val instant: Instant) : UDM()
    // ...
}
```

The `when` expression with sealed classes guarantees exhaustive matching — the compiler refuses to compile if you forget a type. When a new UDM type is added, every `when` block that handles UDM types shows a compile error until updated. This prevents an entire class of runtime bugs.

46.1.2 Null Safety

Kotlin’s type system distinguishes nullable (`String?`) from non-nullable (`String`) at compile time. This maps directly to UTL-X’s safe navigation:

```
$input.Order?.Customer?.Name ?? "Unknown"
```

The interpreter’s Kotlin code handles this naturally — `?.` and `?:` are native Kotlin operators with the same semantics as UTL-X. The language implementation mirrors the language being implemented.

46.1.3 Extension Functions

Add methods to existing types without inheritance or wrapper classes:

```
fun UDM.asString(): String = when (this) {
    is Scalar -> value?.toString() ?: ""
    else -> throw RuntimeException("Expected string")
}
```

This keeps the UDM classes clean (pure data) while providing rich access methods. 650+ stdlib functions are registered without bloating the core types.

46.1.4 Java Ecosystem

100% Java interoperability means access to:

- Jackson (JSON parsing, used internally alongside the custom parser)
- SnakeYAML (YAML parsing)
- ASM (JVM bytecode generation for the COMPILED strategy)
- Apache XML Security (XML canonicalization, C14N)
- Every cryptographic algorithm in the JDK

The JVM ecosystem has 25+ years of battle-tested libraries. UTL-X stands on their shoulders.

46.1.5 Functional Style

Kotlin’s `map`, `filter`, `reduce`, `flatMap` — and its lambda syntax — are the same paradigm as UTL-X. Writing the interpreter for a functional transformation language in a functional-capable language is natural. The interpreter’s Kotlin code reads similarly to the UTL-X code it interprets.

46.2 Why Not Other Languages?

Language	Why not
Java	Too verbose for parser/AST and interpreter code. Weaker null safety. Same JVM as Kotlin, but less expressive — Kotlin’s data classes, sealed hierarchies, and null safety mean far less boilerplate.
Scala	Complex type system (implicits, macros). Slow compilation. Smaller community. SBT build tool friction.
Go	No sealed/sum types for a clean AST. Not expression-based. Verbose error handling for a tree-walking interpreter. Great for the controller (Open-M), wrong for the engine.
Rust	Too low-level for a transformation engine. Ownership/borrowing model adds complexity to tree manipulation. No garbage collector — UDM trees would need manual lifecycle management.
Python	Too slow for production (86K msg/s not achievable). GIL limits concurrency. Dynamic typing makes the interpreter error-prone. Great for the conformance suite runner, wrong for the engine.
TypeScript	Single-threaded (Node.js event loop). Float-only numbers ($0.1 + 0.2 \neq 0.3$). No native compilation path. Different XML parser in every environment.
C#	Would limit to .NET ecosystem. Less Unix tooling. No GraalVM equivalent for native binary. Good for the wrapper SDK, wrong for the engine.

Notice: several “wrong for the engine” languages are used elsewhere in the UTL-X ecosystem — Go for Open-M, Python for the conformance suite, C# for the wrapper SDK. The right language for the right job.

46.3 Why GraalVM

46.3.1 Native Image for the CLI

GraalVM compiles Kotlin/JVM bytecode to a native machine binary — no JVM at runtime:

```
Without GraalVM (JVM JAR):
$ time java -jar utlx.jar --version
real 0.245s ← too slow for shell scripts

With GraalVM (native binary):
$ time ./utlx --version
real 0.008s ← instant, like any native tool
```

This is why `brew install utlx` gives you a native binary, not a JAR. For CLI usage — shell scripts, CI/CD pipelines, interactive development — startup time is everything. A 250ms penalty per invocation makes for `for f in *.xml; do utlx ...; done` painfully slow. With native: 100 files/second. With JVM: 4 files/second.

Native binary characteristics:

- Startup: 7ms (comparable to `jq`)

- Memory: 40MB resident (vs 150MB for JVM)
- Distribution: single file, no JVM installation required
- Size: 84MB (includes all format parsers, 650+ stdlib functions, and the GraalVM runtime)

46.3.2 GraalVM Challenges and How They Were Solved

GraalVM native-image performs ahead-of-time compilation — it strips everything not statically reachable. This creates challenges for a dynamic language runtime like UTL-X:

Resource bundles: The Apache XML Security library loads resource bundles dynamically via `ResourceBundle.getBundle()`. GraalVM’s static analysis doesn’t detect this. The fix: explicitly list resource bundles in `resource-config.json`.

Reflection: UTL-X’s 650+ stdlib functions were originally loaded via Java reflection (`Class.forName` → `getMethod` → `invoke`). This worked on the JVM but failed silently in native image — GraalVM couldn’t predict which classes would be accessed reflectively.

The solution: **eliminate reflection entirely**. Instead of reflective dispatch, the CLI registers all stdlib functions as direct function references at startup — using a lazy registration pattern:

- A lookup map is built once (on first stdlib call, not at startup — zero startup cost)
- When a function is called for the first time, the map provides a direct reference (HashMap lookup, no reflection)
- Subsequent calls go through the registered reference directly

This approach is faster than reflection on JVM too, and works identically on native image. The `Interpreter.setStdlibLookup(map, eager)` API supports both lazy (CLI) and eager (engine) modes — the engine can pre-register all functions at startup to avoid first-call latency in production.

The 10-point native binary validation checklist (embedded in the GitHub Actions build workflow) runs automatically on all three platforms (Linux, macOS, Windows) before release — preventing broken binaries from being published.

46.3.3 JVM for the Engine

UTLXe runs on the standard JVM, not as a native image. This is deliberate — the JVM is actually **faster** for long-running engine workloads:

- **HotSpot JIT:** compiles hot paths to native code at runtime, optimizing based on actual execution patterns
- **COMPILED strategy:** generates JVM bytecode via ASM — this bytecode benefits from JIT, a native image would not
- **G1GC:** handles large heaps efficiently for sustained throughput with thousands of concurrent messages
- **Eager stdlib loading:** the engine calls `setStdlibLookup(map, eager = true)` at startup, pre-registering all 650+ functions — no first-message latency penalty

Native image would save startup time (250ms → 10ms), but UTLXe starts once and runs for days. Startup time is irrelevant; sustained throughput matters. The JVM wins.

46.3.4 GraalVM for Cloud (Future)

GraalVM native for UTLXe would enable true scale-to-zero on serverless platforms:

- JVM cold start: 5-15 seconds (too slow for Cloud Run scale-from-zero)
- Native cold start: under 1 second (viable for serverless)
- Memory: 40MB vs 150MB (direct cost reduction)

This is deferred due to remaining native-image challenges in third-party dependencies (dynamic class loading and serialization-library metadata) — UTL-X’s own stdlib dispatch is already reflection-free (see §46.3.2). When GraalVM’s reachability metadata improves, UTLXe native becomes practical.

46.4 Why NOT JavaScript/TypeScript

The most frequently suggested alternative. Here's why it would fail:

46.4.1 Performance Ceiling

Node.js is single-threaded (event loop). UTLX achieves 86K msg/s with 8 concurrent workers processing simultaneously. Node.js Worker Threads exist but share no memory — serialization overhead between threads eliminates the concurrency benefit.

46.4.2 Number Precision

JavaScript has only number (IEEE 754 double-precision float). Financial calculations break:

```
0.1 + 0.2 === 0.3 // false (JavaScript)
```

UTL-X handles currencies, tax calculations, invoice totals. Kotlin/JVM has `Long` (64-bit integer), `Double`, and `BigDecimal` — proper numeric types for financial data.

46.4.3 XML Inconsistency

The browser DOM API is different from Node.js XML parsers (`xml2js`, `fast-xml-parser`, `libxmljs`). No single XML parser works across all JS environments. Namespace handling is inconsistent across implementations. UTL-X's XML parser is 450 lines of Kotlin — the JS equivalent would be 3x larger and platform-dependent.

46.5 Why NOT WebAssembly (WASM)

46.5.1 The Promise

“Compile Kotlin to WASM — run UTL-X in browsers, edge functions, everywhere.” Kotlin/WASM exists (experimental).

46.5.2 Why It Doesn't Work Today

- **No system clock:** WASM sandboxes I/O. `now()`, `today()`, `parseDate()` — 68 date functions depend on platform time. Would need WASI (not yet standardized across runtimes).
- **Limited reflection:** Kotlin/WASM doesn't support the Kotlin reflection API, which some dependencies (serialization, introspection) still rely on. UTL-X's own `stdlib` dispatch is already reflection-free (see §46.3.2), but the surrounding libraries are not.
- **String handling:** WASM uses linear memory with manual allocation. UTL-X processes thousands of strings per transformation. `WasmGC` (garbage collection) helps but is not universal.
- **Binary size:** 20MB WASM module — too large for edge deployment.
- **650+ stdlib functions:** each needs WASM-compatible implementation. Crypto, XML parsing, file I/O — none available in WASM sandbox.

WASM is 2-3 years away from being viable for UTL-X. When Kotlin/WASM matures and `WasmGC` is universal — revisit.

46.6 One Implementation, Zero Deviation

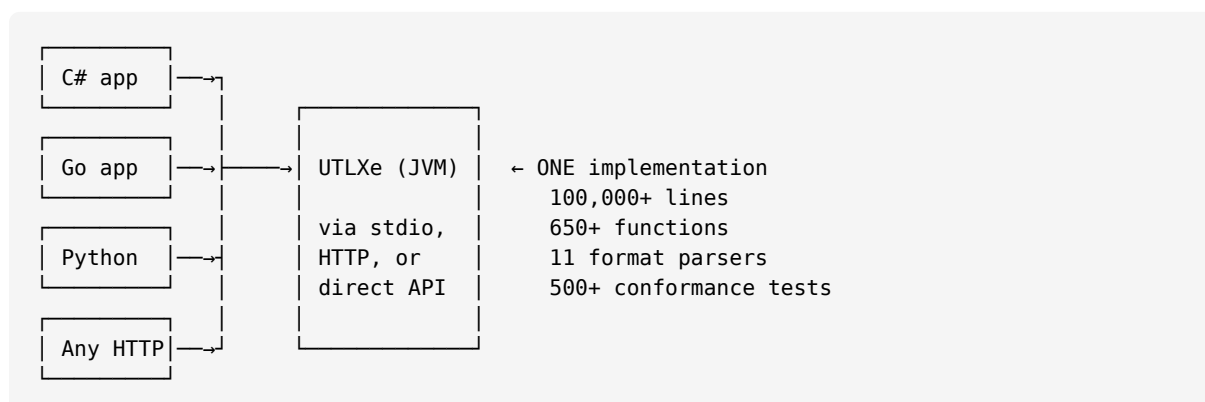
The most important architectural decision: UTL-X has ONE implementation (Kotlin/JVM), accessed through multiple interfaces.

If UTL-X were implemented in Kotlin + JavaScript + Python + Go:

- 4 parsers handling edge cases differently
- 4 interpreters with subtle behavioral differences
- 4 x 650+ = 2,600+ `stdlib` implementations with guaranteed inconsistency
- 4 x 11 format parsers where XML namespace handling WILL differ

Real-world precedent: XSLT has multiple implementations (Saxon, Xalan, libxslt, MSXML). They produce DIFFERENT output for the same stylesheet. This has plagued XML developers for 25 years.

UTL-X's design: one implementation, multiple access paths:



The wrapper is 200-300 lines of thin client code. The engine (100,000+ lines of transformation logic) exists ONCE. All languages get identical behavior — verified by 500+ conformance tests running against the same binary.

One implementation = one behavior = one conformance suite = zero deviation.

Part VI

Case Studies

Real-world use cases, step-by-step walkthroughs, and recipes

47 Case Studies and Recipes

This chapter presents real-world integration scenarios and reusable transformation recipes. Each case study shows the architecture, the transformation code, and the results. The recipes are standalone patterns you can copy and adapt.

47.1 Case Study 1: European E-Invoicing (Peppol / UBL)

Scenario: A Dutch wholesale distributor (200 employees, 850 B2B customers) must comply with EU Directive 2014/55 — all B2G invoices in UBL 2.1 XML via the Peppol network.

Source: Dynamics 365 Business Central (OData JSON). **Target:** UBL 2.1 XML, delivered via Peppol BIS 3.0 Access Point.

Architecture:

Use Case 1: European E-Invoicing — Peppol BIS / UBL Compliance

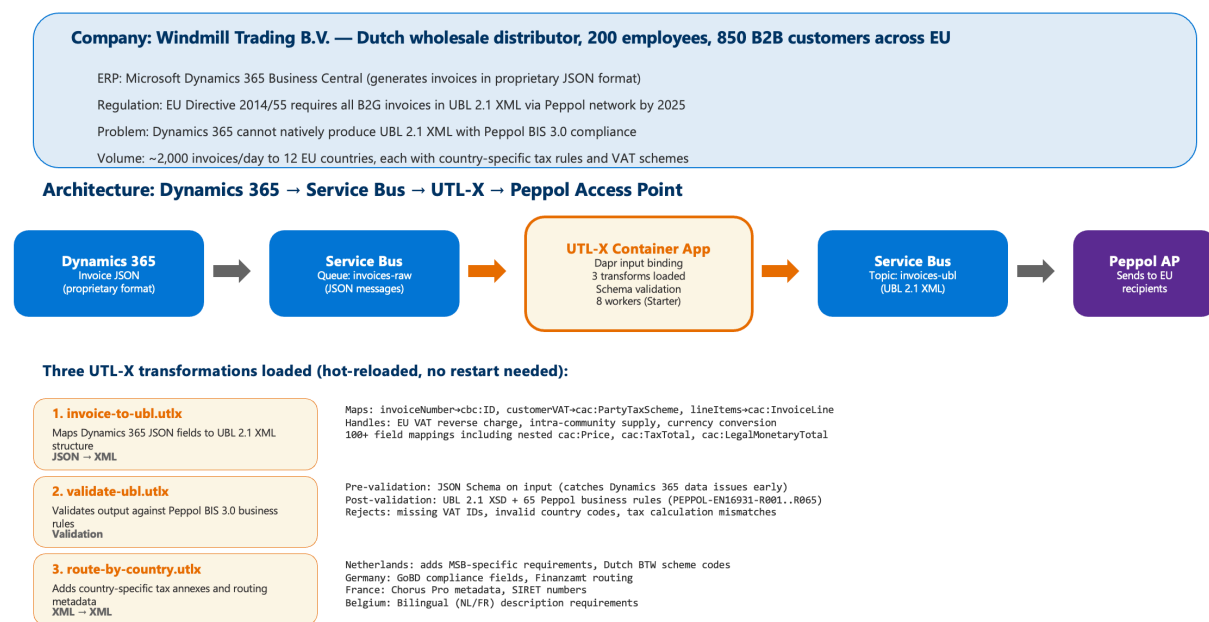


Figure 13: Use Case 1 — Dynamics 365 → Service Bus → UTL-X Container App → Peppol Access Point. Three hot-reloadable transformations handle invoice mapping, validation, and country-specific routing.

Use Case 1: The UTL-X Transformation — invoice-to-ubl.utlx

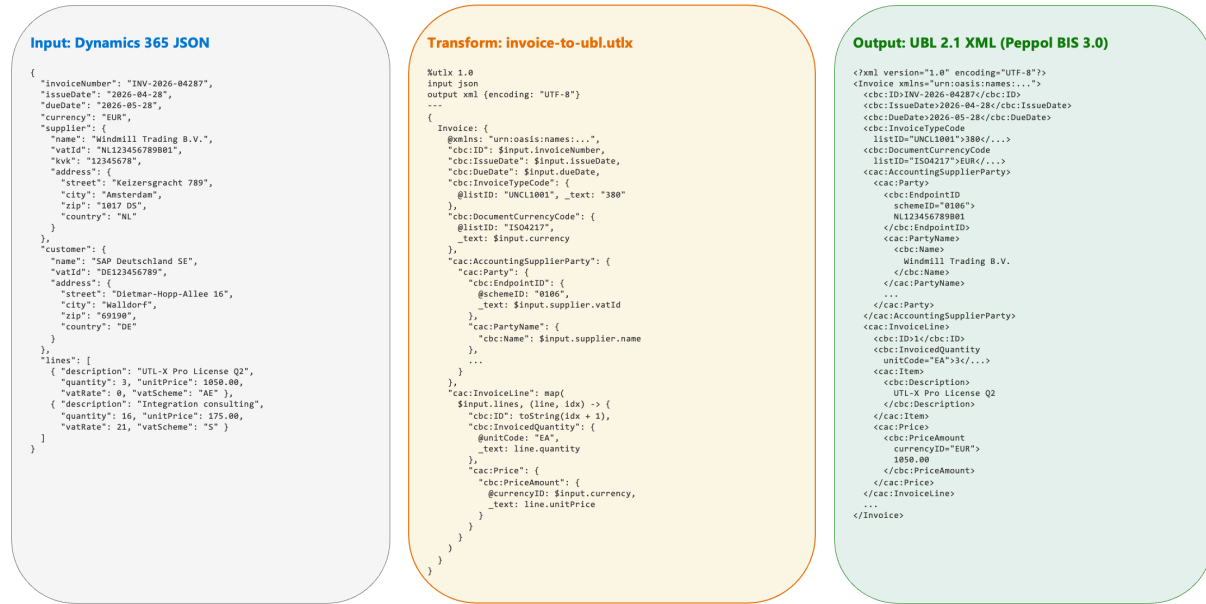


Figure 14: Use Case 1 — The UTL-X transformation in detail: Dynamics 365 JSON input (left), invoice-to-ubl.utlx transformation (center), UBL 2.1 XML output (right).

Transformation highlights:

```
%utlx 1.0
input odata
output xml
...
let inv = $input

function VATCategory(countryCode) {
  if (countryCode == "NL") "S"
  else if (contains(["BE", "DE", "FR", "IT", "ES"], countryCode)) "S"
  else "0"
}

function FormatAmount(amount, currency) {
  {"@currencyID": currency, "_text": toString(round(amount * 100) / 100)}
}

{
  "Invoice": {
    "@xmlns": "urn:oasis:names:specification:ubl:schema:xsd:Invoice-2",
    "@xmlns:cbc": "urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2",
    "@xmlns:cac": "urn:oasis:names:specification:ubl:schema:xsd:CommonAggregateComponents-2",

    "cbc:CustomizationID":
    "urn:cen.eu:en16931:2017#compliant#urn:fdc:peppol.eu:2017:poacc:billing:3.0",
    "cbc:ID": inv.invoiceNumber,
    "cbc:IssueDate": formatDate(now(), "yyyy-MM-dd"),
    "cbc:InvoiceTypeCode": "380",
    "cbc:DocumentCurrencyCode": inv.transactioncurrencyid?.isocurrencycode ?? "EUR",
  }
}
```

```

    "cac:InvoiceLine": map(inv.invoice_details ?? [], (line) -> {
      "cbc:ID": toString(line.sequencenumber ?? 1),
      "cbc:InvoicedQuantity": {"@unitCode": "EA", "_text": toString(line.quantity ?? 1)},
      "cbc:LineExtensionAmount": FormatAmount(line.extendedamount ?? 0, "EUR"),
      "cac:Item": {"cbc:Name": line.productdescription ?? "Product"},
      "cac:Price": {"cbc:PriceAmount": FormatAmount(line.priceperunit ?? 0, "EUR")}
    })
  }
}

```

Results:

Use Case 1: Business Impact — Why UTL-X?

Before UTL-X

Custom C# Azure Function per country (12 functions, ~8,000 lines)
 3 developers maintaining country-specific XSLT maps
 6-week lead time for new country onboarding
 No schema validation — Peppol rejections discovered days later
 €45,000/year in developer time for maintenance alone
 Each Dynamics 365 update breaks 2-3 XSLT maps

After UTL-X

3 UTL-X transformations (total ~400 lines) handle all 12 countries
 Hot re load: update transforms via API — no redeployment
 New country: add routing rules to route-by-country.utlx (1 hour)
 Pre+post validation catches issues at transformation time
 €35/month Starter plan + €50/month Azure compute = €85/month total
 Dynamics 365 JSON changes: update 1 mapping file, test, reload

Production Numbers (6 months in production)

2,147

invoices/day
avg volume

12

EU countries
NL, DE, FR, BE, AT, IT, ES,
PT, IE, DK, SE, PL

14ms

avg transform
latency

99.97%

Peppol acceptance
rate (was 94.2%)

€85/mo

total cost
(UTL-X + Azure)

€44,000

annual savings
vs custom code

Figure 15: Use Case 1 — Business impact: before vs after UTL-X, with production numbers after 6 months.

- 2,147 invoices/day, 14ms average latency
- 99.97% Peppol Access Point acceptance rate
- Cost: €85/month (UTL-X + Azure compute)
- Savings: €44,000/year vs previous solution (12 custom Azure Functions in C#)

47.2 Case Study 2: Healthcare — FHIR Integration

Scenario: A Dutch academic hospital (1,200 beds) must exchange patient data via FHIR R4 for the MedMij/VIPP national health information exchange program.

Sources: Lab system (HL7 v2 → JSON via Mirth), Radiology (DICOM SR → JSON), EHR (proprietary JSON), Pharmacy (HL7 v2.5 → JSON). **Target:** FHIR R4 Bundles via the NUTS network.

Architecture:

Use Case 2: Healthcare HL7 FHIR — Hospital System Integration

Organization: Amsterdam UMC — Academic hospital, 1,200 beds, 12,000 staff, 40+ clinical systems

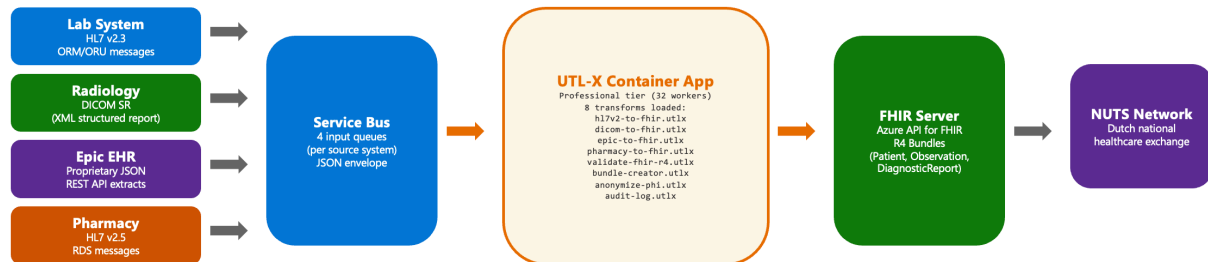
Challenge: MedMij/VIPP program mandates HL7 FHIR R4 for patient data exchange between Dutch healthcare providers

Reality: Lab system sends HL7 v2 (pipe-delimited), Radiology sends DICOM SR (XML), EHR uses proprietary JSON, Pharmacy uses HL7 v2.5

Requirement: All systems must exchange data via FHIR R4 Bundles through the national NUTS network

Constraint: Patient data cannot leave the hospital's Azure tenant — strict GDPR + NEN 7510 compliance

Architecture: Multiple source systems → Service Bus → UTL-X → FHIR R4 Bundle → NUTS Network



Key: UTL-X runs inside the hospital's Azure tenant. Patient data never leaves the tenant boundary. No external API calls. GDPR + NEN 7510 compliant by architecture — not by policy.

Figure 16: Use Case 2 — Four source systems (Lab, Radiology, EHR, Pharmacy) → Service Bus → UTL-X Container App → FHIR Server → NUTS Network. All processing stays inside the hospital's Azure tenant for GDPR + NEN 7510 compliance.

Use Case 2: Transformation Detail — Lab HL7 v2 → FHIR R4 Observation

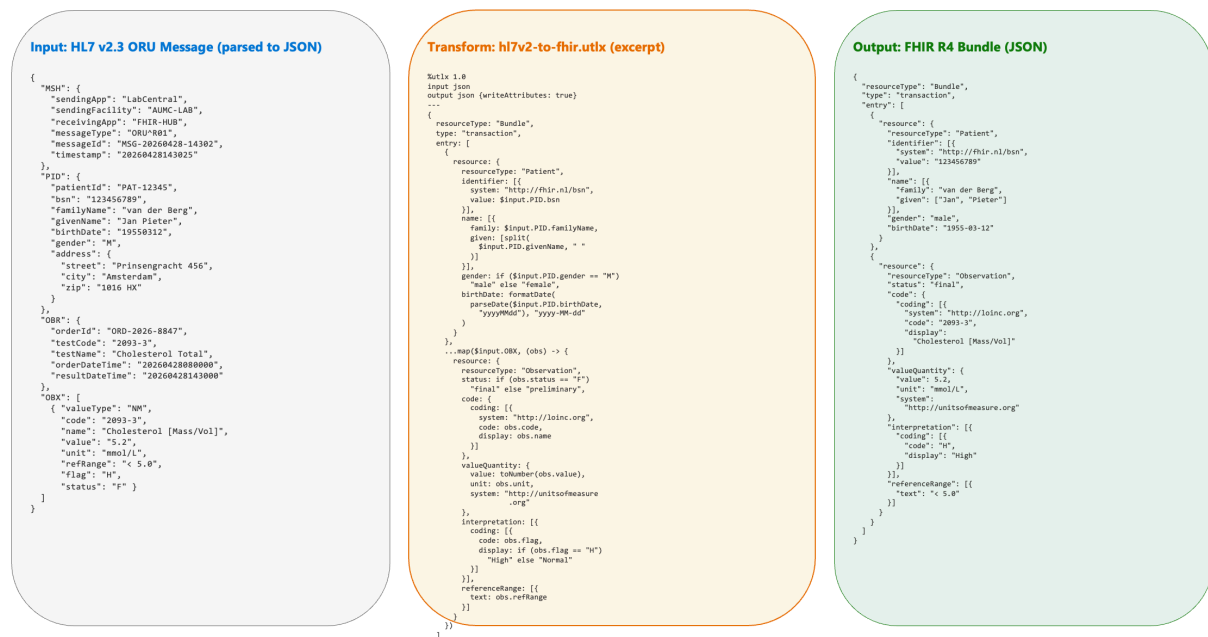


Figure 17: Use Case 2 — Transformation detail: HL7 v2.3 OBX message parsed to JSON (left), hl7v2-to-fhir.utlx transformation excerpt (center), FHIR R4 Observation Bundle (right).

Key transformation — Lab result to FHIR Observation:

```

%utlx 1.0
input json
  
```

```
output json
---
{
  resourceType: "Observation",
  id: $input.labResultId,
  status: "final",
  category: [{
    coding: [{
      system: "http://terminology.hl7.org/CodeSystem/observation-category",
      code: "laboratory"
    }]
  }],
  code: {
    coding: [{
      system: "http://loinc.org",
      code: $input.loincCode,
      display: $input.testName
    }]
  },
  subject: {
    reference: concat("Patient/", $input.patientId)
  },
  effectiveDateTime: formatDate(
    parseDate($input.collectionDate, "yyyyMMddHHmm"), "yyyy-MM-dd'T'HH:mm:ss'Z'"
  ),
  valueQuantity: {
    value: toNumber($input.resultValue),
    unit: $input.unit,
    system: "http://unitsofmeasure.org",
    code: $input.ucumCode
  },
  referenceRange: [{
    low: {value: toNumber($input.refLow), unit: $input.unit},
    high: {value: toNumber($input.refHigh), unit: $input.unit}
  }]
}
```

Results:

Use Case 2: Business Impact — Why UTL-X?

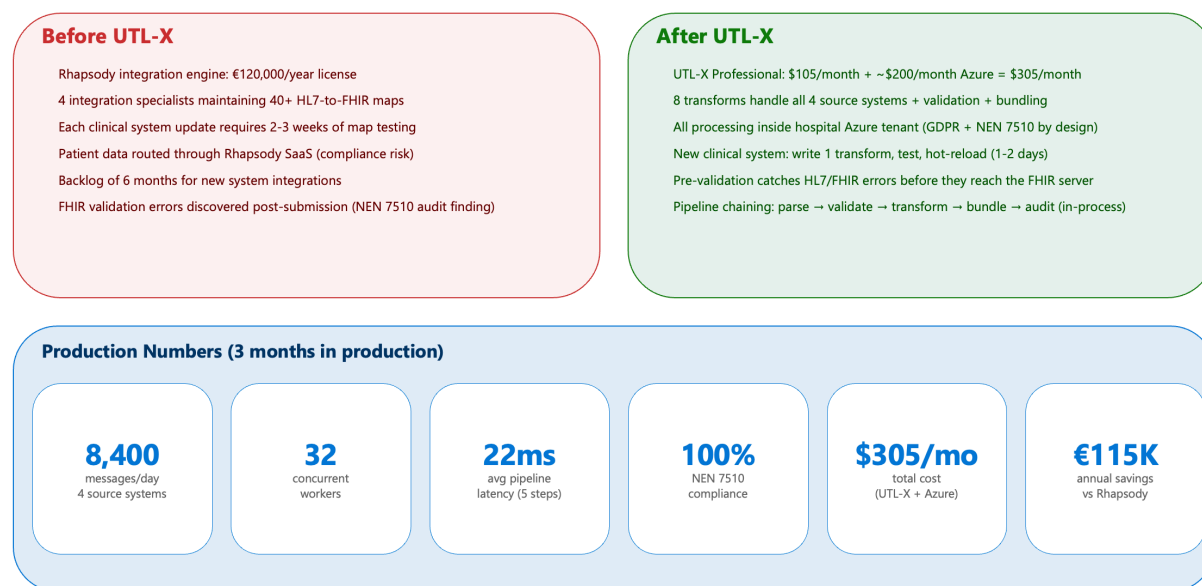


Figure 18: Use Case 2 — Business impact: before vs after UTL-X, with production numbers after 3 months.

- 8,400 messages/day across 4 source systems
- 22ms average pipeline latency (5-step pipeline)
- 100% NEN 7510 compliant by architecture (all data stays in hospital's Azure tenant)
- Cost: \$305/month (UTL-X Professional + Azure compute)
- Savings: €115,000/year vs Rhapsody integration engine license

47.3 Case Study 3: Retail — Multi-Channel Order Normalization

Scenario: An online retailer with 4 sales channels needs a unified order format for their ERP, warehouse, and accounting systems.

Sources: Shopify (JSON), WooCommerce (JSON), Amazon (XML), manual entry (CSV). **Target:** canonical order JSON → fan-out to ERP (JSON), warehouse (XML), accounting (CSV).

Shopify normalizer:

```
%utlx 1.0
input json
output json
---
{
  orderId: concat("SHOP-", toString($input.id)),
  source: "shopify",
  orderDate: $input.created_at,
  customer: {
    name: concat($input.customer.first_name, " ", $input.customer.last_name),
    email: $input.customer.email
  },
  lines: map($input.line_items, (item) -> {
    sku: item.sku,
    name: item.title,
    quantity: item.quantity,
```

```

    unitPrice: toNumber(item.price)
  )),
  total: toNumber($input.total_price),
  currency: $input.currency
}

```

Amazon normalizer (XML input):

```

%utlx 1.0
input xml
output json
---
{
  orderId: concat("AMZ-", $input.Order.AmazonOrderId),
  source: "amazon",
  orderDate: $input.Order.PurchaseDate,
  customer: {
    name: $input.Order.ShippingAddress.Name,
    email: null
  },
  lines: map($input.Order.OrderItem, (item) -> {
    sku: item.SellerSKU,
    name: item.Title,
    quantity: toNumber(item.QuantityOrdered),
    unitPrice: toNumber(item.ItemPrice.Amount)
  }),
  total: toNumber($input.Order.OrderTotal.Amount),
  currency: $input.Order.OrderTotal.CurrencyCode
}

```

Four normalizers (one per channel) produce the same canonical format. Three output formatters transform the canonical format to ERP, warehouse, and accounting. Total: 7 .utlx files.

47.4 Recipes: Common Transformation Patterns

47.4.1 Recipe 1: REST API Response Flattening

Flatten a nested API response to CSV-ready rows:

```

%utlx 1.0
input json
output csv
---
flatten(map($input.data, (user) ->
  map(user.orders, (order) -> {
    userId: user.id,
    userName: user.name,
    orderId: order.id,
    amount: order.total,
    date: order.createdAt
  })
))

```

47.4.2 Recipe 2: Date Format Normalization

Normalize dates from mixed formats to ISO 8601:

```
function NormalizeDate(dateStr) {
  if (dateStr == null) null
  else if (contains(dateStr, "/")) {
    if (length(split(dateStr, "/")[2]) == 4)
      formatDate(parseDate(dateStr, "MM/dd/yyyy"), "yyyy-MM-dd")
    else
      formatDate(parseDate(dateStr, "dd/MM/yy"), "yyyy-MM-dd")
  }
  else if (contains(dateStr, "-") && length(dateStr) == 10)
    dateStr
  else if (contains(dateStr, "T"))
    substring(dateStr, 0, 10)
  else dateStr
}

map($input, (row) -> {
  ...row,
  orderDate: NormalizeDate(row.orderDate),
  shipDate: NormalizeDate(row.shipDate)
})
```

47.4.3 Recipe 3: Currency Conversion with Lookup

```
%utlx 1.0
input: orders json, rates json
output json
---
map($orders, (order) -> {
  let rate = find($rates, (r) -> r.code == order.currency)
  ...order,
  originalAmount: order.amount,
  originalCurrency: order.currency,
  amountEUR: round(order.amount * (rate?.rate ?? 1) * 100) / 100,
  currency: "EUR"
})
```

47.4.4 Recipe 4: Address Normalization

Different systems format addresses differently. Normalize to a common structure:

```
function NormalizeAddress(addr) {
  {
    line1: addr.street ?? addr.streetAddress ?? addr.address1 ?? addr.Address?.Street ??
    "",
    line2: addr.street2 ?? addr.address2 ?? addr.Address?.Street2 ?? "",
    city: addr.city ?? addr.City ?? addr.town ?? "",
    postalCode: addr.zip ?? addr.zipCode ?? addr.postalCode ?? addr.PostalCode ?? "",
    country: toUpperCase(addr.country ?? addr.countryCode ?? addr.Country ?? "")
  }
}

{
  ...$input,
  shippingAddress: NormalizeAddress($input.shipping ?? $input.shippingAddress ?? {}),
  billingAddress: NormalizeAddress($input.billing ?? $input.billingAddress ?? {})
}
```

47.4.5 Recipe 5: Error Response Standardization

Different APIs return errors in different formats. Normalize to one structure:

```
function StandardError(input) {
  {
    error: true,
    code: input.error?.code ?? input.errorCode ?? input.status ?? "UNKNOWN",
    message: input.error?.message ?? input.message ?? input.error_description ??
input.detail ?? "Unknown error",
    details: input.error?.details ?? input.errors ?? input.validationErrors ?? []
  }
}

StandardError($input)
```

47.4.6 Recipe 6: CSV Regional Format Conversion

Convert US CSV (comma decimal) to European CSV (comma delimiter conflicts — use semicolons):

```
%utlx 1.0
input csv
output csv {delimiter: ";", regionalFormat: "european", decimals: 2}
---
$input
```

One line of transformation body. The format options handle the rest — comma decimals, dot thousands, semicolon delimiters.

47.4.7 Recipe 7: XML Namespace Stripping

Remove all namespace prefixes from XML for downstream systems that don't handle them:

```
%utlx 1.0
input xml
output json
---
function StripPrefix(name) {
  if (contains(name, ":")) substring(name, indexOf(name, ":") + 1) else name
}

// Access by original prefixed names, output with clean names
{
  invoiceId: $input["cbc:ID"],
  issueDate: $input["cbc:IssueDate"],
  customer: $input["cac:AccountingCustomerParty"]["cac:Party"]["cbc:Name"],
  total: toNumber($input["cac:LegalMonetaryTotal"]["cbc:PayableAmount"])
}
```

The transformation accesses elements by their prefixed names (bracket notation) but outputs clean, unprefixed property names.

47.4.8 Recipe 8: Conditional Output Format

Same transformation, different output based on a field value:

```
%utlx 1.0
input json
```

```
output json
---
if ($input.format == "summary") {
  id: $input.orderId,
  total: $input.total,
  items: count($input.lines)
} else {
  id: $input.orderId,
  total: $input.total,
  lines: map($input.lines, (l) -> {
    product: l.product,
    qty: l.qty,
    price: l.price,
    lineTotal: l.qty * l.price
  }),
  tax: $input.total * 0.21,
  grandTotal: $input.total * 1.21
}
```

The `format` field in the input controls whether the output is a summary or full detail.

Part VII

Reference

Grammar, appendices, and general reference

48 Grammar Reference

This chapter provides the formal grammar, operator precedence, reserved words, and syntax rules for UTL-X 1.0. Use it as a desk reference when writing transformations.

48.1 File Structure

Every `.utlx` file has two sections separated by `---`:

```
%utlx 1.0           ← version directive
input <format>       ← input declaration
output <format>      ← output declaration
---                 ← separator
<expression>        ← transformation body (single expression)
```

The header is parsed as declarations. The body is parsed as a single expression that produces the output.

48.2 Header Syntax

48.2.1 Version Directive

```
%utlx 1.0
```

Required. Must be the first non-empty line. Currently always `1.0`.

48.2.2 Input Declaration

```
input <format>                // single unnamed input
input <name> <format>         // single named input
input <name> <format> {key: value, ...} // with options
input: <name1> <format1>, <name2> <format2> // multiple inputs (colon required)
input: <name1> <format1>,      // multi-line (comma-separated)
      <name2> <format2> {key: value}
```

Name comes before format. The colon after `input:` signals multiple comma-separated inputs. Without a name, the default name is `input` (accessed as `$input`). Named inputs are accessed as `$name`.

48.2.3 Output Declaration

```
output <format>                // basic output
output <format> {key: value, ...} // with options
output <format> %<dialect> <version> // with dialect annotation
```

The dialect annotation (`%name version`) is a generic mechanism. Currently no dialects have functional behavior — they are parsed and stored but do not change processing. Future dialects may include schema inference or compatibility modes.

48.2.4 Format Keywords

Data formats: `json`, `xml`, `csv`, `yaml`, `odata`, `auto`

Schema formats: `xsd`, `jsch`, `avro`, `proto`, `osch`, `tsch`

48.2.5 Format Options

Options use `{key: value}` syntax after the format keyword:

```

{delimiter: ";"}           // string value
{headers: true}            // boolean value
{decimals: 2}              // number value
{regionalFormat: "european"} // string value
{writeAttributes: true}    // boolean value
{encoding: "UTF-8"}        // string value
{bom: true}                // boolean value

```

48.3 Expression Grammar (Precedence)

From lowest to highest precedence:

Level	Name	Operators	Associativity
1	Pipe	>	Left
2	Conditional	if ... else ...	Right
3	Ternary	? :	Right
4	Nullish coalescing	??	Left
5	Logical OR	\ \	Left
6	Logical AND	&&	Left
7	Equality	== !=	Left
8	Comparison	< <= > >=	Left
9	Addition	+ -	Left
10	Multiplication	* / %	Left
11	Exponentiation	**	Right
12	Unary	! - (prefix)	Right
13	Postfix	. ?. [] ()	Left

48.3.1 Grammar Rules (EBNF)

```

expression  → pipe
pipe        → conditional ("|>" conditional)*
conditional → nullish ("if" nullish "else" conditional)?
nullish     → logical_or ("??" logical_or)*
logical_or  → logical_and ("||" logical_and)*
logical_and → equality ("&&" equality)*
equality    → comparison ("==" | "!=") comparison)*
comparison  → term ("<" | "<=" | ">" | ">=") term)*
term        → factor ("+" | "-") factor)*
factor      → exponent (("*" | "/" | "%") exponent)*
exponent    → unary ("**" unary)*
unary       → ("!" | "-") unary | postfix
postfix     → primary ( "." IDENTIFIER | "?." IDENTIFIER
                    | "[" expression "]"
                    | "(" arguments ")"
                    | "." "@" IDENTIFIER
                    | "." "**")*
primary     → NUMBER | STRING | BOOLEAN | NULL
            | IDENTIFIER
            | "$" IDENTIFIER
            | "(" expression ")"
            | "{" object_properties "}"

```



```
| "[" array_elements "]"
| lambda
| let_binding
| function_def
| match_expression
| try_catch
| if_expression
```

48.4 Literals

48.4.1 Numbers

```
42           // integer
3.14         // decimal
1e10        // scientific notation
-7          // negative (unary minus)
```

48.4.2 Strings

```
"hello world" // double-quoted
'hello world' // single-quoted
"line1\nline2" // escape sequences: \n \t \\ \" \'
```

48.4.3 Booleans and Null

```
true        // boolean true
false       // boolean false
null        // null value
```

48.5 Object Construction

Properties are separated by commas or newlines. Commas are required on the same line; newlines are sufficient across lines.

```
{
  key: value           // identifier key (newline separates from next)
  "key-with-dashes": value // string key (for special characters)
  [dynamicKey]: value  // computed key (expression in brackets)
  ...otherObject       // spread operator
  ...if (cond) {k: v} else {} // conditional spread
}
```

48.6 Array Construction

```
[1, 2, 3]           // literal array
[item1, item2]       // array of expressions
```

48.7 Let Bindings

`let` binds a value to a name. The binding is available from the point of declaration to the end of the enclosing scope.

48.7.1 Top-Level Let (Body Scope)

At the top of the transformation body, `let` creates a binding visible to everything that follows:

```
let tax = 0.21
let threshold = 1000

map($input.orders, (order) -> {
  total: order.amount,
  vat: order.amount * tax,           // tax visible here
  priority: order.amount > threshold // threshold visible here
})
```

Multiple `let` bindings are sequential — each one can reference the previous:

```
let items = $input.order.items
let total = sum(map(items, (i) -> i.price * i.qty))
let average = total / count(items)

{itemCount: count(items), total: total, average: average}
```

48.7.2 Let Inside Object Construction

`let` inside an object creates a binding scoped to that object:

```
{
  let customer = find($input.customers, (c) -> c.id == $input.order.customerId)

  orderId: $input.order.id,
  customerName: customer?.name ?? "Unknown", // customer visible here
  country: customer?.country                 // and here
}
```

The `customer` binding is only available within this object — not outside it.

48.7.3 Let Inside Lambda

`let` inside a lambda is scoped to that lambda invocation:

```
map($input.orders, (order) -> {
  let lineTotal = sum(map(order.lines, (l) -> l.qty * l.price))
  let discount = if (lineTotal > 500) 0.1 else 0

  orderId: order.id,
  subtotal: lineTotal,
  discount: discount,
  total: lineTotal * (1 - discount)
})
```

Each iteration of `map` gets its own `lineTotal` and `discount` — they don't leak between iterations.

48.7.4 Let Inside Function Definitions

`let` inside a function body is scoped to that function call:

```
function CalculateShipping(weight, country) {
  let baseRate = if (country == "NL") 5.95 else 12.50
  let weightSurcharge = if (weight > 20) (weight - 20) * 0.50 else 0
  baseRate + weightSurcharge
}
```

48.7.5 Scoping Summary

Where	Visible to	Example
Top of body	Everything below in the body	<code>let tax = 0.21</code>
Inside object <code>{}</code>	Properties in that object	<code>{let x = ... ↵ a: x}</code>
Inside lambda	That lambda invocation only	<code>(item) -> {let x = ... ↵ ...}</code>
Inside function body	That function call only	<code>function F(n) {let x = ... ↵ ...}</code>
Inside if/else	That branch only	<code>if (...) {let x = ... ↵ x} else ...</code>

48.7.6 Separators: Commas and Newlines

UTL-X uses newline-sensitive parsing for separators. The rule is simple:

- **Same line:** commas are required between properties, let bindings, or array elements
- **Different lines:** newlines are sufficient — commas are optional

Context	Same line	Different lines
Object properties	<code>{a: 1, b: 2}</code>	<code>{ ↵ a: 1 ↵ b: 2 ↵ }</code>
Let bindings in object	<code>{let x = 1, y: x}</code>	<code>{ ↵ let x = 1 ↵ y: x ↵ }</code>
Let before array return	N/A	<code>let x = 1 ↵ [x, x + 1]</code>
Array elements	<code>[1, 2, 3]</code>	Commas required (arrays stay comma-separated)
Function arguments	<code>map(arr, fn)</code>	Commas required

The `[` bracket on a new line is treated as an **array literal**, not an index operator. This resolves the ambiguity between `let x = 20[arr]` (index) and `let x = 20` followed by `[arr]` (array):

```
let x = arr[0]    // same line: index access
let x = arr
[0, 1, 2]        // new line: array literal (not index)
```

This parsing model follows Kotlin and Go — newlines act as implicit separators where syntactically unambiguous.

```
// Multi-line object – no commas needed
{
  let customer = find($input.customers, (c) -> c.id == $input.customerId)
  orderId: $input.id
  customerName: customer?.name
}

// Lambda body – no commas needed
map($input.items, (item) -> {
  let total = item.qty * item.price
  name: item.name
  total: total
})

// Single-line – commas required
{orderId: $input.id, total: $input.amount}
```

Commas are always accepted — existing code with commas continues to work unchanged.

48.8 Function Definitions

```
function Name(param1, param2) {
    expression
}

function Name(param: Type): ReturnType {
    expression
}

def Name(param1, param2) = expression    // shorthand alias
```

Function names **MUST** start with an uppercase letter (PascalCase). See Chapter 15.

48.9 Lambda Expressions

UTL-X supports two arrow syntaxes for lambdas: `->` (thin arrow) and `=>` (fat arrow). Both are equivalent — use whichever you prefer.

48.9.1 Thin Arrow (`->`)

```
(x) -> x * 2                // single parameter
(x, y) -> x + y             // multiple parameters
(item) -> { name: item.name, age: item.age } // object body
() -> "constant"           // no parameters
```

Parentheses around parameters are always required with `->`.

48.9.2 Fat Arrow (`=>`)

```
(x) => x * 2                // with parentheses – same as ->
x => x * 2                  // WITHOUT parentheses – single param only
(x, y) => x + y             // multiple params – parentheses required
item => item.name           // short form for single parameter
```

The fat arrow `=>` allows omitting parentheses for single-parameter lambdas. This mirrors JavaScript's arrow function syntax: `x => x * 2` instead of `(x) -> x * 2`. For multiple parameters, parentheses are required with both styles.

48.9.3 Which to Use?

Both are identical in behavior. Convention:

- `->` is the standard style used throughout this book and the conformance suite
- `=>` is accepted for developers coming from JavaScript/TypeScript who prefer it
- `x => x.name` (no parens) is the shortest form for simple single-parameter lambdas

48.9.4 In Match Expressions

The `=>` arrow is also used in match case arms:

```
match ($input.status) {
    "ACTIVE" => "green",
    "PENDING" => "yellow",
    _ => "gray"
}
```

48.10 Conditional Expressions

```
if (condition) thenExpr else elseExpr

if (x > 0) "positive"
else if (x < 0) "negative"
else "zero"
```

if/else is an expression — it returns a value. The `else` branch is required.

48.11 Match Expression

```
match (value) {
  pattern1 -> result1,
  pattern2 -> result2,
  _ -> defaultResult
}
```

48.12 Try/Catch

```
try { riskyExpression } catch { fallbackValue }
```

If the expression in `try` throws an error, `catch` returns the fallback value.

48.13 Safe Navigation

```
$input.order?.customer?.name      // null if any step is null
$input.items?[0]?.price            // safe index + safe property
```

`?.` short-circuits to null when the left side is null. Combine with `??` for defaults:

```
$input.name ?? "Unknown"
$input.order?.customer?.name ?? "No customer"
```

48.14 Property Access

```
$input.name                // dot notation
$input["content-type"]     // bracket notation (string key)
$input[variable]           // bracket notation (dynamic key)
$input.*                   // wildcard (all children as array)
$input.@id                 // XML attribute
$input.*                   // all attributes as array
$input.^metadata           // metadata access
```

48.15 Pipe Operator

```
$input
  |> filter((x) -> x.active)
  |> map((x) -> x.name)
  |> sortBy((x) -> x)
```

The pipe operator passes the left-hand result as the first argument to the right-hand function.

48.16 Spread Operator

```
{...object}           // copy all properties
{...obj1, ...obj2}     // merge (later wins)
{...obj, key: override} // copy + override
[...array1, ...array2] // concatenate arrays
```

48.17 Reserved Words

48.17.1 Language Keywords

Keyword	Purpose
if / else	Conditional expressions
let	Variable binding
function	User-defined function declaration
def	Alias for function
match	Pattern matching
try / catch	Error handling
in	Used in <code>let...in</code> expressions
import / as	Module import (reserved, not yet implemented)
template	Template definition
apply	Template application
true / false	Boolean literals
null	Null literal
input / output	Header declarations
auto	Auto-detect format

48.17.2 Format Keywords

json, xml, csv, yaml, odata, xsd, jsch, avro, proto, osch, tsch

These are reserved in the header but can be used as identifiers in the body (though not recommended for clarity).

48.17.3 NOT Reserved

map, filter, reduce — these are token types for parser optimization but usable as identifiers. The 652 stdlib function names (lowercase) are not reserved — they're resolved by the interpreter at runtime. User-defined functions (PascalCase) never collide with stdlib (camelCase).

48.18 Operators Reference

Operator	Name	Example
+	Addition / string concatenation	1 + 2, "a" + "b"
-	Subtraction / unary negation	5 - 3, -x
*	Multiplication	3 * 4
/	Division	10 / 3
%	Modulo	10 % 3 → 1
**	Exponentiation	2 ** 10 → 1024

<code>==</code>	Equality	<code>x == 42</code>
<code>!=</code>	Inequality	<code>x != null</code>
<code>< <= > >=</code>	Comparison	<code>x > 0</code>
<code>&&</code>	Logical AND	<code>a && b</code>
<code>\ \ </code>	Logical OR	<code>a \ \ b</code>
<code>!</code>	Logical NOT	<code>!active</code>
<code>??</code>	Nullish coalescing	<code>x ?? "default"</code>
<code>?.</code>	Safe navigation	<code>obj?.prop</code>
<code>\ ></code>	Pipe	<code>arr \ > filter(...)</code>
<code>...</code>	Spread	<code>{...obj}</code>
<code>.</code>	Property access	<code>obj.name</code>
<code>@</code>	Attribute access	<code>elem.@id</code>
<code>^</code>	Metadata access	<code>elem.^source</code>
<code>*</code> (postfix)	Wildcard	<code>obj.*</code>
<code>-></code>	Lambda arrow (standard)	<code>(x) -> x * 2</code>
<code>=></code>	Lambda arrow (fat arrow)	<code>(x) => x * 2</code> or <code>x => x * 2</code>
<code>..</code>	Recursive descent (reserved)	Not yet implemented

48.19 Identifier Rules

- **Start character:** letter (Unicode) or underscore
- **Continue characters:** letter, digit, underscore
- **Case-sensitive:** name and Name are different identifiers
- **Hyphenated names:** supported in header (input names) via bracket notation in body
- **\$ prefix:** used for input variables (`$input`, `$orders`), not part of the identifier itself
- **PascalCase requirement:** user-defined functions must start with uppercase (`function MyFunc`)
- **camelCase convention:** stdlib functions use camelCase (`map`, `filter`, `parseDate`)

48.20 USDL Grammar (Universal Schema Definition Language)

USDL is a convention within UTL-X — not a separate language (see Chapter 12). USDL directives use the `%` prefix and represent schema information in a format-agnostic way. They are automatically added when parsing schema formats (Tier 2 input) and automatically detected by schema serializers (Tier 2 output).

48.20.1 USDL Document Structure

```
usdl_document → "{" usdl_directives "}"

usdl_directives → ("%namespace" ":" STRING)?
                  ("%version" ":" STRING)?
                  ("%types" ":" "{" type_definitions "}")?
                  ("%entityContainer" ":" "{" entity_container "}")?
```

48.20.2 Type Definitions

```
type_definitions → (TYPE_NAME ":" type_definition)*

type_definition → "{"
```

```

("%kind" ":" ("object" | "enum" | "union"))?
("%description" ":" STRING)?
("%fields" ":" "{" field_definitions "}")?
("%values" ":" "[" STRING ("," STRING)* "]" )?    // enums only
("%key" ":" "[" STRING ("," STRING)* "]" )?       // entity key
"}"

```

48.20.3 Field Definitions

```

field_definitions → (FIELD_NAME ":" field_definition)*

field_definition → "{"
  "%type" ":" TYPE_STRING
  ("%description" ":" STRING)?
  ("%required" ":" BOOLEAN)?
  ("%nullable" ":" BOOLEAN)?
  ("%array" ":" BOOLEAN)?
  ("%default" ":" VALUE)?
  ("%example" ":" VALUE)?
  // Shared: Common constraints
  ("%minLength" ":" NUMBER)?
  ("%maxLength" ":" NUMBER)?
  ("%pattern" ":" STRING)?
  ("%minimum" ":" NUMBER)?
  ("%maximum" ":" NUMBER)?
  ("%enum" ":" "[" VALUE ("," VALUE)* "]" )?
  ("%format" ":" STRING)?
  // Format-native: Format-specific
  ("%fieldNumber" ":" NUMBER)?    // Protobuf
  ("%logicalType" ":" STRING)?    // Avro
  ("%precision" ":" NUMBER)?      // Avro decimal
  ("%scale" ":" NUMBER)?          // Avro decimal
  ("%primaryKey" ":" BOOLEAN)?    // Database
  ("%foreignKey" ":" STRING)?     // Database
  ("%navigation" ":" BOOLEAN)?    // OData
  ("%target" ":" STRING)?         // OData
  ("%cardinality" ":" STRING)?    // OData
"}"

```

48.20.4 USDL Directive Classification

Level	Scope	Directives	Applies to
Core	Portable	%namespace, %version, %types, %kind, %name, %type, %description, %fields, %values	All formats
Shared	Widespread	%required, %nullable, %array, %default, %minLength, %maxLength, %pattern, %minimum, %maximum, %enum, %format, %example	Most formats
Format-native	Exclusive	%fieldNumber (Proto), %logicalType (Avro), %primaryKey (DB), %navigation (OData), %choice (XSD), etc.	One format
Reserved	Future	Future USDL versions	Planned

48.20.5 USDL Type Strings

The %type directive accepts these type names:

USDL type	Maps to
string	String in all formats
integer	int/Int32/long — whole numbers
number	double/float/decimal — fractional numbers
boolean	true/false
date	Date without time
time	Time without date
datetime	Date and time (ISO 8601)
binary	Byte array / Base64
object	Nested type (reference to another type name)
any	Untyped / dynamic

For type references, use the type name directly: `%type: "CustomerType"` references the type defined in `%types`.

48.20.6 Example: Complete USDL Document

```
{
  "%namespace": "com.example.orders",
  "%version": "1.0.0",
  "%types": {
    "Order": {
      "%kind": "object",
      "%description": "A customer order",
      "%fields": {
        "orderId": {
          "%type": "string",
          "%required": true,
          "%pattern": "^ORD-[0-9]{5}$",
          "%description": "Unique order identifier"
        },
        "customer": {
          "%type": "Customer",
          "%required": true
        },
        "total": {
          "%type": "number",
          "%minimum": 0,
          "%description": "Order total in EUR"
        },
        "lines": {
          "%type": "OrderLine",
          "%array": true,
          "%description": "Order line items"
        }
      }
    },
    "Customer": {
      "%kind": "object",
      "%fields": {
        "name": { "%type": "string", "%required": true, "%maxLength": 100 },
        "email": { "%type": "string", "%format": "email" }
      }
    }
  }
}
```

```
    },
    "OrderLine": {
      "%kind": "object",
      "%fields": {
        "product": { "%type": "string", "%required": true },
        "quantity": { "%type": "integer", "%minimum": 1 },
        "unitPrice": { "%type": "number", "%minimum": 0 }
      }
    },
    "OrderStatus": {
      "%kind": "enum",
      "%values": ["DRAFT", "CONFIRMED", "SHIPPED", "DELIVERED", "CANCELLED"]
    }
  }
}
```

This USDL document can be converted to JSON Schema, XSD, Avro, Protobuf, or EDMX — the USDL classification system determines what survives in each target format (Chapter 12).

Note (F08): USDL enrichment is implemented. All 6 Tier 2 parsers produce enriched UDM with both raw structure and USDL % properties. Tier 2 → Tier 2 schema conversions (e.g., input `xsd` / output `jsch`) work via USDL passthrough. The stdlib functions (`parseXSDSchema`, `parseJSONSchema`, etc.) also work independently.

49 Appendices

49.1 Appendix A: Format Conversion Matrix

Every cell shows whether the conversion is direct (pass-through works), needs mapping (transformation logic required), or is not applicable.

From / To	JSON	XML	CSV	YAML	OData	Any Schema
JSON	–	Direct	Flatten	Direct	Add @odata	Parse schema
XML	Direct	–	Flatten	Direct	Strip+add	Parse schema
CSV	Direct	Add structure	–	Direct	Add @odata	N/A
YAML	Direct	Direct	Flatten	–	Add @odata	Parse schema
OData	Strip @odata	Strip+map	Strip+flatten	Strip @odata	–	EDMX parse

Direct: `$input` pass-through produces valid output (structures are compatible).

Flatten: nested data must be denormalized for CSV (one row per child, parent fields repeated). Use `unnest()` (Chapter 21) or manual `flatten(map(...))`.

Add structure: CSV is flat — converting to XML requires adding element names and nesting.

Strip: OData annotations (`\@odata.*`) are automatically separated during parsing, producing clean data.

Parse schema: schema formats (XSD, JSON Schema, etc.) are parsed as data — the schema structure becomes the transformation input.

49.2 Appendix B: Common Error Messages

49.2.1 Parser Errors

Error	Fix
Expected expression	Check for missing value after <code>let x =</code> or empty transformation body
Expected property name	Object key is missing or uses reserved word — quote it: <code>"input": value</code>
Expected <code>'</code> after arguments	Mismatched parentheses in function call — count opening and closing
Unclosed string literal	Missing closing <code>"</code> or <code>'</code> — check for unescaped quotes inside strings
User-defined functions must start with uppercase	Rename <code>myFunc</code> to <code>MyFunc</code> — PascalCase required
Expected <code>'output'</code> declaration	Multiple <code>input</code> lines not supported — use <code>input:</code> with commas

49.2.2 Runtime Errors

Error	Fix
-------	-----

Undefined variable: name	Variable not declared with <code>let</code> , or misspelled. Check: is the input named?
Cannot index non-array with number	Trying <code>obj[0]</code> on an object — use <code>.propertyName</code> instead
Cannot index non-object with string	Trying <code>arr["key"]</code> on an array — use <code>arr[0]</code> (number index)
Array index out of bounds	Index exceeds array size — check with <code>count()</code> first or use <code>?.</code>
Division by zero	Denominator is zero — guard with <code>if (divisor != 0) ... else 0</code>
Type error: expected string got number	Use <code>toString(value)</code> to convert, or check input data types

49.2.3 Validation Errors (UTLXe)

Error	Fix
Validation error at \$.field: required field missing	The input or output is missing a required field — check mapping
Does not match pattern	String value doesn't match the regex in the schema — check format
Value is less than minimum	Number below the schema's minimum — check calculation
Type mismatch: expected string got number	Schema expects a string but transformation produces a number — add <code>toString()</code>

49.3 Appendix C: UTL-X vs Competitors (Feature Comparison)

49.3.1 UTL-X vs XSLT

Feature	XSLT	UTL-X
Input formats	XML only	XML, JSON, CSV, YAML, OData
Output formats	XML (text output for others)	All 11 formats natively
Paradigm	Template matching (push)	Functional expressions (pull)
Syntax	XML (verbose)	Concise, JSON-like
Functions	XPath 2.0/3.0 functions	650+ stdlib functions (superset of XPath 2.0, partial XPath 3.0)
User functions	Named templates	<code>function</code> with PascalCase
Variables	<code>xsl:variable</code> (immutable)	<code>let</code> (immutable)
Schema validation	XSD only	7 schema formats
Production engine	No (processor required)	UTLXe (86K msg/s)
IDE	XML editors	VS Code + live preview
Learning curve	Steep	Moderate (familiar syntax)

49.3.2 UTL-X vs DataWeave

Feature	DataWeave	UTL-X
Format agnostic	Yes	Yes
Attribute prefix	@	@
<code>writeAttributes</code>	Same name, same default	Same name, same default

Function style	Functional	Functional
Open source	No (proprietary)	Yes (AGPL)
Platform	MuleSoft Anypoint only	Any (Docker, CLI, cloud)
Price	\$1,250+/month	\$35/month
Schema formats	JSON Schema, XSD	7 formats (+ USDL bridge)
Production engine	Mule Runtime	UTLXe (86K msg/s)
Version control	Embedded in Mule app	.utlx files in Git
Conformance suite	No	500+ tests

49.3.3 UTL-X vs jq

Feature	jq	UTL-X
Formats	JSON only	11 formats
Dot notation	.name	.name (identical)
Raw output	-r	-r (identical)
Pipe	\	\ > (explicit)
Filter	select()	filter()
Map	[.[] \ .name]	map(., (x) -> x.name)
User functions	def f: ...;	function F(...) { ... }
Schema validation	No	7 formats
Production engine	No	UTLXe
Stdlib functions	50	650+
XML support	No	Full (namespaces, attributes, C14N)

49.4 Appendix D: Industry Standards Quick Reference

Standard	Domain	UTL-X relevance
UBL 2.1 / Peppol BIS 3.0	E-invoicing (EU)	XML output with UBL namespaces (Ch30)
HL7 FHIR R4	Healthcare	@value accessor for FHIR XML (Ch21, Ch30)
ISO 20022 (SWIFT MX)	Financial messaging	pain.001, camt.053 XML (Ch30)
EDIFACT	Logistics/retail EDI	Planned — reverseXSL integration (Ch41)
ANSI X12	North American EDI	Planned — reverseXSL integration (Ch41)
OpenAPI 3.x	REST API contracts	YAML/JSON input, schema extraction (Ch40)
AsyncAPI	Event-driven APIs	Kafka/AMQP message schemas (Ch40)
XBRL	Financial reporting	QName functions for taxonomy processing (Ch21)
SOAP/WSDL	Web services	Envelope parsing, WS-Addressing (Ch21)
OData v4	Enterprise REST	Native format with annotation handling (Ch26)
NEN 7510	Dutch healthcare security	Tenant isolation architecture (Ch37)
PCI DSS	Payment card security	Log metadata, not data (Ch37)
GDPR	EU data protection	Data minimization in logging (Ch37)

49.5 Appendix E: Glossary

Term	Definition
AST	Abstract Syntax Tree — the parsed representation of a <code>.utlx</code> file
BOM	Byte Order Mark (U+FEFF) — encoding indicator stripped on input, optional on CSV output
C14N	XML Canonicalization — W3C standard for deterministic XML serialization
CDATA	Character Data section in XML — content not parsed as markup
Dapr	Distributed Application Runtime — CNCF sidecar for broker-agnostic messaging
EDMX	Entity Data Model XML — OData metadata schema format (OSCH in UTL-X)
FHIR	Fast Healthcare Interoperability Resources — HL7's modern health data standard
GraalVM	JVM with native image compilation — used for UTL-X CLI binary
IDoc	Intermediate Document — SAP's data exchange format
iPaaS	Integration Platform as a Service — cloud-based integration tools
JMS	Java Message Service — enterprise Java messaging standard
KEDA	Kubernetes Event-Driven Autoscaling — scales containers based on queue depth
MCP	Model Context Protocol — provides LLMs with UTL-X language context
MPPM	Message Processing Pipeline Model — Open-M's integration flow standard
OData	Open Data Protocol — REST-based data access (Microsoft, SAP)
Peppol	Pan-European Public Procurement Online — e-invoicing network
QName	Qualified Name — namespace prefix + local name in XML
RAML	RESTful API Modeling Language — MuleSoft's API description format
UBL	Universal Business Language — OASIS standard for business documents
UDM	Universal Data Model — UTL-X's format-agnostic data representation
USDL	Universal Schema Definition Language — UTL-X's schema dialect
UTLXe	UTL-X Engine — the production runtime for high-throughput transformation
XBRL	eXtensible Business Reporting Language — financial reporting standard

49.6 Appendix F: Version History

Version	Date	Highlights
1.0.0	April 2026	Initial release. CLI, daemon, engine. 5 data formats, 6 schema formats, 652 stdlib functions.
1.0.1	April 2026	CLI enhancements: <code>-e</code> expression mode, identity flip, <code>--from/--to</code> flags.
1.0.2	April 2026	Bug fixes: XML text unwrapping in UDM, <code>_text</code> leaking into JSON/YAML output. New: <code>writeAttributes</code> option, <code>#text</code> key for text content, <code>@</code> prefix consistency for JSON and YAML.
1.1.0	May 2026	Major feature release. Newline separators (commas optional in multi-line objects and match cases). Data restructuring functions: <code>nestBy</code> , <code>lookupBy</code> , <code>chunkBy</code> , <code>unnest</code> . USDL enrichment for all 6 Tier 2 parsers. <code>groupBy</code> returns <code>Object</code> + <code>mapGroups</code> . <code>#text</code> alias for <code>_text</code> . <code>xs:decimal</code> type mapping with JSON Schema <code>"format": "decimal"</code> . GraalVM native binary fixes — 650+ stdlib functions work in native image. Native binary startup 7ms.

1.2.0	May 2026	Non-UTF-8 encoding support (B20) across core, formats, and CLI — read and write data in legacy encodings, not just UTF-8.
1.2.1	May 2026	Security library merged into the standard library (F11). JSON Patch — RFC 6902 (F15). Systematic Gradle test coverage for every stdlib function (F19). IDE XSLT Migration Assistant (IF02).
1.2.2	June 2026	New <code>convert</code> command for quick, script-free format conversion (F20). Stdlib functions now work inside <code>map/filter/reduce</code> (B25). CLI error-handling fixes: no more double-printed errors, stack-trace dumps, or hangs on missing input (B26).
1.3.0	July 2026	Breaking — package namespace migration. Every package moved from <code>org.apache.utlx.*</code> to <code>com.glomidco.utlx.*</code> and the Maven group changed to <code>com.glomidco.utlx</code> . Library consumers: find/replace imports and update the dependency groupId; genuine Apache dependencies are unaffected. No functional or API changes — names only; the CLI and native binaries are drop-in.

49.7 Appendix G: License

UTL-X is dual-licensed:

49.7.1 Open Source: AGPL-3.0

The UTL-X parser, interpreter, engine, and standard library are licensed under the GNU Affero General Public License v3.0. This means:

- **Free to use:** run UTL-X for any purpose
- **Free to modify:** change the source code
- **Free to distribute:** share your modified version
- **Copyleft:** if you distribute a modified version or offer it as a network service, you must share your modifications under AGPL-3.0

Your transformations are yours. The AGPL covers the UTL-X **software** — the parser, interpreter, and engine. It does NOT cover the `.utlx` transformation files you write or the data you process. Your transformations are your intellectual property.

49.7.2 Commercial Licensing

For organizations that cannot use AGPL (proprietary embedding, SaaS without source disclosure):

- Azure Marketplace: Starter (\$35/month) and Professional (\$105/month) plans include a commercial license
- Direct licensing: contact Glomidco B.V. for custom terms

49.7.3 Third-Party Components

UTL-X includes components under compatible open-source licenses:

- SnakeYAML: Apache 2.0
- Jackson: Apache 2.0
- ASM: BSD 3-Clause
- Apache XML Security: Apache 2.0
- SLF4J + Logback: MIT / EPL-1.0
- reverseXSL (vendored): Apache 2.0

Part VIII

Standard Library Encyclopedia

Complete reference for all 652 functions — organized by category with examples

50 Standard Library Reference

This appendix lists the UTL-X standard library functions alphabetically. Each entry shows:

- **Signature:** `functionName(required, required, optional?)` — parameters with ? are optional
- **Category tag:** Str, Arr, Obj, Num, Date, Fmt, XML, JSON, CSV, YAML, Sec, Bin, Geo, URL, Sys, Type
- **Example:** practical, runnable usage
- **Anti-pattern:** what NOT to do (where applicable)

Function Index

<code>abs(number) → number</code>	353	<code>c14n11WithComments(xml) → string</code>	366	<code>csvTranspose(csv, options?, header?)</code>	376
<code>acos(number) → number</code>	353	<code>c14nFingerprint(xml) → string</code>	366	<code>currentDir() → string</code>	377
<code>addBOM(data) → binary</code>	353	<code>c14nPhysical(xml, options?) → string</code>	366	<code>floor(number) → integer</code>	377
<code>addNamespaceDeclarations(xml, name-</code>	353	<code>c14nSubset(xml, xpath) → string</code>	366	<code>round(number) → integer</code>	377
<code>addQueryParam(url, name, value) →</code>	353	<code>c14nWithComments(xml) → string</code>	366	<code>chunk(array, size) → array of arrays</code>	377
<code>addTax(amount, rate) → number</code>	354	<code>calculateDiscount(price, rate) → num-</code>	367	<code>coalesce(value1, value2, ...) → value</code>	377
<code>age(birthdate, referenceDate?) → num-</code>	354	<code>calculateTax(amount, rate) → number</code>	367	<code>compact(array) → array</code>	378
<code>analyzeString(string, pattern) → object</code>	354	<code>camelCase(string) → string</code>	367	<code>concat(string1, string2, ...) → string</code>	378
<code>and(values...) → boolean</code>	355	<code>camelize(string) → string</code>	367	<code>contains(haystack, needle) → boolean</code>	378
<code>asin(number) → number</code>	355	<code>canCoerce(value, targetType) →</code>	368	<code>count(array) → number</code>	379
<code>assert(condition, message?) → null</code>	355	<code>canonicalizeWithAlgorithm(xml, algo-</code>	368	<code>countBy(array, predicate) → number</code>	379
<code>assertEqual(actual, expected) → null</code>	355	<code>canonical.JSONHash(json, algorithm?)</code>	368	<code>createQname(localName, name-</code>	379
<code>atan(number) → number</code>	356	<code>canonical.JSONSize(json) → number</code>	368	<code>csvFilter(csv, column, operator, value)</code>	380
<code>atan2(y, x) → number</code>	356	<code>capitalize(string) → string</code>	368	<code>day(date) → number</code>	380
<code>attribute(element, name) → string</code>	356	<code>ceil(number) → integer</code>	369	<code>dayOfMonth(date) → number</code>	380
<code>attributes(element) → object</code>	356	<code>charAt(string, index) → string</code>	369	<code>dayOfWeek(date) → number</code>	381
<code>availableProcessors() → number</code>	357	<code>charCodeAt(string, index) → number</code>	369	<code>dayOfWeekName(date) → string</code>	381
Date Format Patterns	357	<code>childCount(element) → number</code>	369	<code>dayOfYear(date) → number</code>	381
<code>addDays(date, count) → date</code>	357	<code>childNames(element) → array</code>	369	<code>daysBetween(date1, date2) → number</code>	381
<code>addHours(datetime, count) → datetime</code>	358	<code>chunkBy(array, predicate) → array of</code>	369	<code>daysInMonth(date) → number</code>	381
<code>addMinutes(datetime, count) → date-</code>	358	<code>clearLogs() → null</code>	370	<code>daysInYear(date) → number</code>	382
<code>addMonths(date, count) → date</code>	358	<code>coerce(value, targetType, default?) →</code>	370	<code>debug(value, message?) → value</code>	382
<code>addQuarters(date, count) → date</code>	359	<code>coerceAll(array, targetType, default?)</code>	370	<code>debugPrint(value, label?, indent?) →</code>	382
<code>addSeconds(datetime, count) → date-</code>	359	<code>compactCSV(csv, options?) → string</code>	370	<code>debugPrintCompact(value, label?) →</code>	382
<code>addWeeks(date, count) → date</code>	359	<code>compactJSON(json, options?, indent?)</code>	371	<code>decodeJWS(token) → object</code>	383
<code>addYears(date, count) → date</code>	359	<code>compactXML(xml, options?, indent?) →</code>	371	<code>decodeJWT(token) → object</code>	383
<code>all(array, predicate) → boolean</code>	359	<code>compareDates(date1, date2) → number</code>	371	<code>decompress(data, algorithm?) → binary</code>	383
<code>any(array, predicate) → boolean</code>	360	<code>compoundInterest(principal, rate, peri-</code>	371	<code>decryptAES(data, key, iv) → string</code>	383
<code>avg(array) → number</code>	360	<code>compress(data, algorithm?) → binary</code>	372	<code>decryptAES256(data, key, iv) → string</code>	383
<code>avgBy(array, keyFn) → number</code>	360	<code>constantCase(string) → string</code>	372	<code>deepClone(object) → object</code>	384
<code>base64Encode(data) → string</code>	361	<code>containsValue(object, value) → boolean</code>	372	<code>deepMerge(obj1, obj2) → object</code>	384
<code>base64Decode(string) → string</code>	361	<code>convertTimezone(datetime, fromTz</code>	372	<code>deepMergeAll(objects) → object</code>	384
<code>bearing(lat1, lon1, lat2, lon2) → number</code>	361	<code>convertXMLEncoding(xml, targetEn-</code>	373	<code>deflate(data) → binary</code>	384
<code>binaryConcat(binary1, binary2, ...) →</code>	362	<code>cos(radians) → number</code>	373	<code>destinationPoint(lat, lon, bearing, dis-</code>	385
<code>binaryEquals(binary1, binary2) →</code>	362	<code>cosh(number) → number</code>	373	<code>detectBOM(binary) → string</code>	385
<code>binaryLength(binary) → number</code>	362	<code>countEntries(object, predicate?) →</code>	373	<code>detectXMLEncoding(xmlString) →</code>	385
<code>binarySlice(binary, start, end) → binary</code>	362	<code>createCDATA(content) → string</code>	373	<code>diffDays(date1, date2) → number</code>	385
<code>binaryToString(binary, encoding) →</code>	363	<code>createSOAPEnvelope(body, header?) →</code>	373	<code>diffHours(datetime1, datetime2) →</code>	386
<code>bitwiseAnd(binary1, binary2) → binary</code>	363	<code>crossJoin(array1, array2) → array</code>	374	<code>diffMinutes(datetime1, datetime2) →</code>	386
<code>bitwiseNot(binary) → binary</code>	363	<code>csvAddColumn(csv, name, default-</code>	374	<code>diffMonths(date1, date2) → number</code>	386
<code>bitwiseOr(binary1, binary2) → binary</code>	363	<code>csvCell(csv, row, column) → string</code>	374	<code>diffSeconds(datetime1, datetime2) →</code>	386
<code>bitwiseXor(binary1, binary2) → binary</code>	363	<code>csvColumn(csv, name) → array</code>	374	<code>diffWeeks(date1, date2) → number</code>	387
<code>boundingBox(coordinates) → object</code>	364	<code>csvColumns(csv) → array</code>	375	<code>diffYears(date1, date2) → number</code>	387
<code>buildQueryString(params) → string</code>	364	<code>csvRemoveColumns(csv, columns) →</code>	375	<code>distance(lat1, lon1, lat2, lon2) → num-</code>	387
<code>buildURL(components) → string</code>	364	<code>csvRow(csv, index, options?) → object</code>	375	<code>distinct(array) → array</code>	387
<code>c14n(xml) → string</code>	365	<code>csvRows(csv) → number</code>	375	<code>distinctBy(array, keyFn) → array</code>	388
<code>c14nHash(xml, algorithm?) → string</code>	365	<code>csvSelectColumns(csv, columns) →</code>	376	<code>divideBy(object, n) → array</code>	388
<code>c14nEquals(xml1, xml2) → boolean</code>	365	<code>csvSort(csv, column, ascending?) →</code>	376	<code>dotCase(string) → string</code>	388
<code>c14n11(xml) → string</code>	365	<code>csvSummarize(csv, options?) → object</code>	376	<code>drop(array, n) → array</code>	389

take(array, n) → array	389	last(array) → element or null	404	hasNumeric(string) → boolean	417
e() → number	389	tail(array) → array	404	hexDecode(hex) → binary	418
elementPath(element) → string	389	flatMap(array, fn) → array	404	hexEncode(data) → string	418
encryptAES(data, key, iv) → string	390	flatten(array) → array	405	hmac(data, key, algorithm) → string	418
encryptAES256(data, key, iv) → string	390	flattenDeep(array) → array	405	hmacSHA256(data, key) → string	418
endOfDay(date) → datetime	390	formatDate(date, pattern) → string	405	hmacBase64(data, key, algorithm) →	419
endOfMonth(date) → datetime	390	generateIV() → string	406	hmacMD5(data, key) → string	419
endOfQuarter(date) → datetime	391	generateKey(bits?) → string	406	hmacSHA1(data, key) → string	419
endOfWeek(date) → datetime	391	generateUuid() → string	406	hmacSHA384(data, key) → string	419
endOfYear(date) → datetime	391	generateUuidV4() → string	406	hmacSHA512(data, key) → string	419
endsWith(string, suffix) → boolean	391	generateUuidV7() → string	406	homeDir() → string	420
startsWith(string, prefix) → boolean	391	generateUuidV7Batch(count) → array	407	hours(datetime) → number	420
endTimer(label) → number	392	get(object, path) → any	407	ifThenElse(condition, thenValue, else-	420
enforceNamespacePrefixes(xml, pre-	392	getBaseURL(url) → string	407	implies(a, b) → boolean	420
entries(object) → array	392	getBOMBytes(encoding) → binary	407	includes(array, value) → boolean	421
fromEntries(pairs) → object	392	getCurrencyDecimals(currency) →	408	indexOf(array, value) → number	421
env(name) → string	393	getFragment(url) → string	408	inflate(data) → binary	421
envOrDefault(name, default) → string	393	getHost(url) → string	408	info(message) → null	421
envAll() → object	393	getJWSAlgorithm(token) → string	408	insertAfter(array, index, element) → ar-	422
environment() → string	393	getJWSHeader(token) → object	409	insertBefore(array, index, element) →	422
equals(a, b) → boolean	394	getJWSInfo(token) → object	409	invert(object) → object	422
error(message) → never	394	getJWSKeyId(token) → string	409	iqr(numbers) → number	422
escapeXML(text) → string	394	getJWSPayload(token) → object	409	isAfter(date1, date2) → boolean	423
every(array, predicate) → boolean	394	getJWSSigningInput(token) → string	409	isAlpha(string) → boolean	423
everyEntry(object, predicate) →	395	getJWSTokenType(token) → string	409	isAlphanumeric(string) → boolean	423
excC14n(xml) → string	395	getJWTAudience(token) → string	410	isAscii(string) → boolean	423
excC14nWithComments(xml) → string	395	getJWTClaim(token, claim) → any	410	isBefore(date1, date2) → boolean	424
exp(x) → number	395	getJWTClaims(token) → object	410	isBetween(date, start, end) → boolean	424
extractBetween(string, start, end) →	396	getJWTIssuer(token) → string	410	isCanonicalJSON(json) → boolean	424
extractCDATA(text) → string	396	getJWTSubject(token) → string	410	isCDATA(text) → boolean	424
extractTimestampFromUuidV7(uuid)	396	getLogs() → array	411	isDebugMode() → boolean	425
filter(array, predicate) → array	396	getNamespaces(element) → object	411	isEmptyElement(element) → boolean	425
filterEntries(object, predicate) → object	397	getPath(url) → string	411	isGzipped(data) → boolean	425
find(array, predicate) → element or null	397	getPort(url) → number	411	isHexadecimal(string) → boolean	425
findIndex(array, predicate) → number	398	getProtocol(url) → string	412	isJarFile(data) → boolean	425
findAllMatches(string, pattern) → array	398	getQuery(url) → string	412	isJWSFormat(token) → boolean	425
findLastIndex(array, predicate) → num-	398	getQueryParams(url) → object	412	isJWTExpired(token) → boolean	426
first(array) → element or null	399	getTimezone(datetime) → string	412	isLeapYearFunc(year) → boolean	426
formatCurrency(amount, currency?, lo-	399	getTimezoneName() → string	412	isLocalDateTime(value) → boolean	426
formatDateInTimezone(datetime, lo-	399	getTimezoneOffsetHours(datetime,	413	isLowerCase(string) → boolean	426
formatEmptyElements(xml, style?) →	399	getTimezoneOffsetSeconds(datetime) →	413	isNotEmpty(value) → boolean	426
formatNumber(number, pattern?) →	400	getType(value) → string	413	isNumeric(string) → boolean	427
formatPlural(count, word) → string	400	getUuidVersion(uuid) → number	413	isPlural(word) → boolean	427
fromBase64(encoded) → binary	400	goldenRatio() → number	414	isPointInCircle(lat, lon, centerLat, cen-	427
fromBytes(byteArray) → binary	400	groupBy(array, keyFn) → object	414	isPointInPolygon(lat, lon, polygon) →	427
fromCamelCase(string) → string	401	gunzip(data) → binary	414	isPrintable(string) → boolean	428
fromCharCode(code) → string	401	gzip(data) → binary	414	isSameDay(date1, date2) → boolean	428
fromConstantCase(string) → string	401	hasAlpha(string) → boolean	415	isSingular(word) → boolean	428
fromDotCase(string) → string	401	hasAttribute(element, name) → boolean	415	isToday(date) → boolean	428
fromHex(hexString) → binary	402	hasBOM(data) → boolean	415	isUpperCase(string) → boolean	428
fromKebabCase(string) → string	402	hasContent(element) → boolean	415	isUuidV7(uuid) → boolean	429
fromPascalCase(string) → string	402	hasEnv(name) → boolean	415	isValidAmount(amount) → boolean	429
fromPathCase(string) → string	402	hasKey(object, key) → boolean	416	isValidCoordinates(lat, lon) → boolean	429
fromSnakeCase(string) → string	402	containsKey(object, key) → boolean	416	isValidCurrency(code) → boolean	429
fromTitleCase(string) → string	403	hash(data, algorithm?) → string	416	isValidTimezone(timezone) → boolean	430
fromUTC(datetime, timezone) → date-	403	md5(data) → string	417	isValidURL(url) → boolean	430
fullOuterJoin(left, right, leftKey	403	sha256(data) → string	417	isValidUuid(uuid) → boolean	430
futureValue(presentValue, rate, periods)	403	sha512(data) → string	417	isWhitespace(string) → boolean	430
head(array) → element or null	404	hasNamespace(element, uri) → boolean	417	isZipArchive(data) → boolean	430

<code>lastIndexOf(haystack, needle) → num-</code>	431	<code>measure(fn) → object</code>	445	<code>prepareForSignature(xml) → string</code>	457
<code>isArray(value) → boolean</code>	431	<code>median(array) → number</code>	445	<code>presentValue(futureAmount, rate, peri-</code>	458
<code>isBoolean(value) → boolean</code>	431	<code>mode(array) → number</code>	445	<code>prettyPrint(string, indent?) → string</code>	458
<code>isNotNull(value) → boolean</code>	431	<code>stdDev(array) → number</code>	445	<code>prettyPrintCSV(csv, options?) → string</code>	458
<code>isNumber(value) → boolean</code>	432	<code>variance(array) → number</code>	445	<code>prettyPrintFormat(value, format) →</code>	458
<code>isObject(value) → boolean</code>	432	<code>percentile(array, p) → number</code>	445	<code>prettyPrintJSON(json, indent?) →</code>	458
<code>isString(value) → boolean</code>	432	<code>memoryInfo() → object</code>	446	<code>prettyPrintXML(xml, options?) →</code>	458
<code>isDate(value) → boolean</code>	432	<code>merge(obj1, obj2, ...) → object</code>	446	<code>prettyPrintYAML(yaml, options?) →</code>	458
<code>isDateTime(value) → boolean</code>	432	<code>midpoint(lat1, lon1, lat2, lon2) → object</code>	446	<code>quarter(date) → number</code>	459
<code>isTime(value) → boolean</code>	432	<code>minutes(datetime) → number</code>	446	<code>quartiles(array) → object</code>	459
<code>isBlank(value) → boolean</code>	433	<code>month(date) → number</code>	446	<code>random() → number</code>	459
<code>isEmpty(value) → boolean</code>	433	<code>monthName(date) → string</code>	447	<code>readByte(binary, offset) → number</code>	459
<code>isDefined(value) → boolean</code>	433	<code>nand(a, b) → boolean</code>	447	<code>readDouble(binary, offset) → number</code>	459
<code>isLeapYear(year) → boolean</code>	433	<code>nestBy(parents, children, parentKeyFn</code>	447	<code>readFloat(binary, offset) → number</code>	459
<code>isWeekday(date) → boolean</code>	433	<code>nodeType(node) → string</code>	448	<code>readInt16(binary, offset) → number</code>	460
<code>isWeekend(date) → boolean</code>	434	<code>none(array, predicate) → boolean</code>	448	<code>readInt32(binary, offset) → number</code>	460
<code>javaVersion() → string</code>	434	<code>nor(a, b) → boolean</code>	448	<code>readInt64(binary, offset) → number</code>	460
<code>jcs(json) → string</code>	434	<code>normalizeBOM(data, targetEncoding</code>	448	<code>readJarEntry(jar, entryName) → binary</code>	460
<code>canonicalizeJSON(json) → string</code>	434	<code>normalizeXMLEncoding(xml, targetEn-</code>	449	<code>readJarManifest(jar) → object</code>	460
<code>jsonEquals(json1, json2) → boolean</code>	434	<code>not(value) → boolean</code>	449	<code>readZipEntry(zip, entryName) → binary</code>	460
<code>join(array, separator) → string</code>	435	<code>now() → datetime</code>	449	<code>reduce(array, initial, accumulator) →</code>	461
<code>joinToString(array, separator?) → string</code>	435	<code>currentDate() → date</code>	449	<code>reduceEntries(object, initial, accumula-</code>	462
<code>joinWith(left, right, leftKeyFn</code>	435	<code>currentTime() → time</code>	449	<code>regexGroups(string, pattern) → array</code>	462
<code>kebabCase(string) → string</code>	436	<code>normalizeSpace(string) → string</code>	449	<code>regexNamedGroups(string, pattern) →</code>	462
<code>keys(object) → array</code>	436	<code>numberOrDefault(value, default) →</code>	450	<code>remove(string, search) → string</code>	462
<code>values(object) → array</code>	436	<code>omit(object, keys) → object</code>	450	<code>removeBOM(data) → data</code>	462
<code>leftJoin(left, right, leftKeyFn</code>	436	<code>pick(object, keys) → object</code>	450	<code>removeQueryParam(url, param) →</code>	462
<code>length(value) → number</code>	437	<code>osArch() → string</code>	450	<code>removeTax(totalWithTax, rate) → num-</code>	463
<code>listJarEntries(jarData) → array</code>	437	<code>osVersion() → string</code>	451	<code>render(value, format, pretty?) → string</code>	463
<code>listZipEntries(zipData) → array</code>	437	<code>pad(string, length, char?) → string</code>	451	<code>renderJson(value, pretty?) → string</code>	463
<code>ln(number) → number</code>	437	<code>padLeft(string, length, char?) → string</code>	451	<code>renderCsv(value) → string</code>	463
<code>localName(element) → string</code>	438	<code>padRight(string, length, char?) → string</code>	451	<code>renderXml(value) → string</code>	463
<code>namespaceUri(element) → string</code>	438	<code>parent(element) → element</code>	451	<code>renderYaml(value) → string</code>	464
<code>namespacePrefix(element) → string</code>	438	<code>parse(string, format?) → value</code>	452	<code>repeat(string, n) → string</code>	464
<code>qualifiedName(element) → string</code>	438	<code>parseJson(string) → value</code>	452	<code>replace(string, search, replacement) →</code>	464
<code>log(number, base?) → number</code>	438	<code>parseXml(string) → value</code>	452	<code>replaceRegex(string, regex, replace-</code>	464
<code>log10(number) → number</code>	439	<code>parseYaml(string) → value</code>	452	<code>replaceWithFunction(string, pattern</code>	464
<code>log2(number) → number</code>	439	<code>parseCsv(string, options?) → array</code>	452	<code>resolveQName(qname, namespaces) →</code>	464
<code>logCount(value) → value</code>	439	<code>parseBoolean(value) → boolean</code>	453	<code>reverse(array) → array</code>	465
<code>logPretty(value) → value</code>	439	<code>parseCurrency(string) → number</code>	453	<code>reverseString(string) → string</code>	465
<code>logSize(value) → value</code>	439	<code>parseDate(string, pattern) → date</code>	453	<code>rightJoin(left, right, leftKey, rightKey)</code>	465
<code>logType(value) → value</code>	440	<code>parseDateTimeWithTimezone(string,</code>	453	<code>roundToCents(number) → number</code>	465
<code>lookupBy(searchValue, referenceArray</code>	440	<code>parseDouble(string) → number</code>	454	<code>roundToDecimalPlaces(number, places)</code>	465
<code>lower(string) → string</code>	440	<code>parseEUNumber(string) → number</code>	454	<code>runtimeInfo() → object</code>	466
<code>lowerCase(string) → string</code>	440	<code>parseUSNumber(string) → number</code>	454	<code>scan(array, initial, fn) → array</code>	466
<code>upperCase(string) → string</code>	441	<code>parseFloat(string) → number</code>	454	<code>seconds(datetime) → number</code>	466
<code>map(array, fn) → array</code>	441	<code>parseInt(string, radix?) → number</code>	454	<code>setConsoleLogging(enabled) → null</code>	466
<code>mapEntries(object, fn) → object</code>	441	<code>parseNumber(string) → number</code>	455	<code>setLogLevel(level) → null</code>	466
<code>mapKeys(object, fn) → object</code>	442	<code>parseQueryString(string) → object</code>	455	<code>setPath(url, path) → string</code>	466
<code>mapValues(object, fn) → object</code>	442	<code>parseURL(url) → object</code>	455	<code>sha224(data) → string</code>	467
<code>mapGroups(array, keySelector, trans-</code>	442	<code>partition(array, predicate) → matching</code>	455	<code>sha256(data) → string</code>	467
<code>mapTree(data, transformer) → object </code>	442	<code>pascalCase(string) → string</code>	456	<code>sha512(data) → string</code>	467
<code>mask(string, visibleChars) → string</code>	443	<code>pathCase(string) → string</code>	456	<code>sha1(data) → string</code>	467
<code>matches(string, regex) → boolean</code>	443	<code>percentageChange(oldValue, newValue)</code>	456	<code>sha384(data) → string</code>	467
<code>matchesWhole(string, pattern) →</code>	444	<code>pi() → number</code>	456	<code>sha3_256(data) → string</code>	467
<code>max(array) → number</code>	444	<code>platform() → string</code>	457	<code>sha3_512(data) → string</code>	467
<code>min(array) → number</code>	444	<code>pluralize(word) → string</code>	457	<code>shiftLeft(binary, positions) → binary</code>	468
<code>maxBy(array, fn) → element</code>	444	<code>pluralizeWithCount(word, count) →</code>	457	<code>shiftRight(binary, positions) → binary</code>	468
<code>minBy(array, fn) → element</code>	444	<code>pow(base, exponent) → number</code>	457	<code>shouldUseCDATA(content, threshold?)</code>	468

<code>simpleInterest(principal, rate, time) →</code>	468	<code>toUTC(datetime) → datetime</code>	479	<code>xor(a, b) → boolean</code>	489
<code>sin(x) → number</code>	468	<code>trace(value) → value</code>	479	<code>yamlEntries(object) → array</code>	489
<code>singularize(word) → string</code>	468	<code>translate(string, from, to) → string</code>	479	<code>yamlExists(yaml, path) → boolean</code>	489
<code>sinh(x) → number</code>	469	<code>treeDepth(object) → number</code>	479	<code>yamlFilterByKeyPattern(object, pat-</code>	490
<code>slice(value, start, end?) → array or</code>	469	<code>treeFilter(object, predicate) → object</code>	479	<code>yamlFindByField(yaml, fieldName) →</code>	490
<code>slugify(string) → string</code>	469	<code>treeFind(object, path) → value</code>	479	<code>yamlFindObjectsWithField(yaml, field-</code>	490
<code>smartCoerce(value) → value</code>	469	<code>treeFlatten(object) → array</code>	480	<code>yamlFromEntries(entries) → object</code>	490
<code>snakeCase(string) → string</code>	469	<code>treeMap(object, fn) → object</code>	480	<code>yamlGetDocument(yaml, index) →</code>	490
<code>some(array, predicate) → boolean</code>	469	<code>treePaths(object) → array</code>	480	<code>yamlHasRequiredFields(object, fields)</code>	490
<code>someEntry(object, predicate) → boolean</code>	470	<code>trim(string) → string</code>	480	<code>yamlKeys(object) → array</code>	491
<code>sort(array) → array</code>	470	<code>leftTrim(string) → string</code>	480	<code>yamlMerge(obj1, obj2) → object</code>	491
<code>sortBy(array, keyFn) → array</code>	470	<code>rightTrim(string) → string</code>	480	<code>yamlMergeAll(objects) → object</code>	491
<code>split(string, separator) → array</code>	470	<code>transpose(array2D) → array2D</code>	480	<code>yamlOmitKeys(object, keys) → object</code>	491
<code>splitWithMatches(string, pattern) → ar-</code>	471	<code>truncate(string, maxLength, suffix?) →</code>	481	<code>yamlSelectKeys(object, keys) → object</code>	491
<code>sqrt(x) → number</code>	471	<code>tryCoerce(value, type) → value</code>	481	<code>yamlSort(object, comparator?) → object</code>	491
<code>startOfDay(datetime) → datetime</code>	471	<code>typeof(value) → string</code>	481	<code>yamlSplitDocuments(yaml) → array</code>	492
<code>startOfMonth(datetime) → datetime</code>	471	<code>typeof(value) → string</code>	481	<code>yamlMergeDocuments(docs) → string</code>	492
<code>startOfQuarter(datetime) → datetime</code>	471	<code>udmToJSON(value, pretty?) → string</code>	481	<code>yamlPath(yaml, path) → value</code>	492
<code>startOfWeek(datetime) → datetime</code>	471	<code>udmToXML(value, rootName?, op-</code>	482	<code>yamlSet(yaml, path, value) → value</code>	492
<code>startOfYear(datetime) → datetime</code>	472	<code>udmToYAML(value, options?) → string</code>	482	<code>yamlDelete(yaml, path) → value</code>	492
<code>startTimer() → null</code>	472	<code>uncamelize(string) → string</code>	482	<code>yamlValidate(yaml, rules) → object</code>	492
<code>stringOrDefault(value, default) → string</code>	472	<code>unescapeXML(string) → string</code>	482	<code>yamlValidateKeyPattern(object, pat-</code>	493
<code>stripBOM(string) → string</code>	472	<code>union(arr1, arr2) → array</code>	482	<code>yamlValues(object) → array</code>	493
<code>substring(string, start, end?) → string</code>	472	<code>intersect(arr1, arr2) → array</code>	482	<code>zip(arr1, arr2) → array</code>	493
<code>substringBefore(string, delimiter) →</code>	472	<code>difference(arr1, arr2) → array</code>	483	<code>zipWith(arr1, arr2, fn) → array</code>	493
<code>substringAfter(string, delimiter) →</code>	473	<code>symmetricDifference(arr1, arr2) → array</code>	483	<code>zipWithIndex(array) → array</code>	493
<code>substringAfterLast(string, delimiter) →</code>	473	<code>unique(array) → array</code>	483	<code>zipAll(arrays, pad?) → array</code>	494
<code>substringBeforeLast(string, delimiter)</code>	473	<code>unnest(array, childKey) → array</code>	483	<code>zipArchive(entries) → binary</code>	494
<code>sum(array) → number</code>	473	<code>unwrapCDATA(string) → string</code>	484		
<code>sumBy(array, fn) → number</code>	473	<code>unzip(pairs) → [array, array]</code>	484		
<code>systemPropertiesAll() → object</code>	474	<code>unzipArchive(zip, options?) → object</code>	484		
<code>systemProperty(name) → string</code>	474	<code>unzipN(tuples) → array of arrays</code>	484		
<code>systemPropertyOrDefault(name, de-</code>	474	<code>updateXMLEncoding(xml, encoding) →</code>	485		
<code>tan(x) → number</code>	474	<code>updateXMLEncoding(xml, encoding) →</code>	485		
<code>tanh(x) → number</code>	474	<code>upper(string) → string</code>	485		
<code>tempDir() → string</code>	475	<code>uptime() → number</code>	485		
<code>textContent(element) → string</code>	475	<code>urlDecodeComponent(string) → string</code>	485		
<code>textContent(element) → string</code>	475	<code>urlEncode(string) → string</code>	485		
<code>timerCheck(name) → number</code>	475	<code>urlDecode(string) → string</code>	485		
<code>timerClear() → null</code>	475	<code>urlEncodeComponent(string) → string</code>	486		
<code>timerList() → array</code>	475	<code>username() → string</code>	486		
<code>timerReset(name) → null</code>	475	<code>validateDate(string, pattern?) →</code>	486		
<code>timerStart(name) → null</code>	475	<code>validateDigest(xml, expectedDigest) →</code>	486		
<code>timerStats(name) → object</code>	476	<code>validateEncoding(encoding) → boolean</code>	486		
<code>timerStop(name) → number</code>	476	<code>values(object) → array</code>	486		
<code>timestamp() → string</code>	476	<code>version() → string</code>	487		
<code>titleCase(string) → string</code>	476	<code>warn(value) → value</code>	487		
<code>toArray(value) → array</code>	476	<code>weekOfYear(date) → number</code>	487		
<code>toBase64(binary) → string</code>	476	<code>windowed(array, size) → array of arrays</code>	487		
<code>toBinary(string, encoding?) → binary</code>	476	<code>wordCase(string) → string</code>	487		
<code>toBoolean(value) → boolean</code>	477	<code>wrapIfNeeded(content, threshold?, tag?)</code>	487		
<code>toBytes(binary) → array</code>	477	<code>writeByte(value) → binary</code>	488		
<code>toDate(string) → date</code>	477	<code>writeDouble(value) → binary</code>	488		
<code>toDegrees(radians) → number</code>	477	<code>writeFloat(value) → binary</code>	488		
<code>toHex(binary) → string</code>	477	<code>writeInt16(value) → binary</code>	488		
<code>toNumber(value) → number</code>	477	<code>writeInt32(value) → binary</code>	488		
<code>toObject(value) → object</code>	478	<code>writeInt64(value) → binary</code>	488		
<code>toRadians(degrees) → number</code>	478	<code>xmlEscape(string) → string</code>	489		
<code>toString(value) → string</code>	478	<code>xmlUnescape(string) → string</code>	489		
<code>toTitleCase(string) → string</code>	478	<code>xnor(a, b) → boolean</code>	489		

50.1 A

50.1.1 `abs(number) → number` (Num)

Returns the absolute value of a number.

- `number` (required): the number to take the absolute value of

```
echo '{"balance": -1500.50}' | utlx -e 'abs(.balance)'
# 1500.5
```

```
abs(-42)      // 42
abs(42)       // 42 (positive unchanged)
abs(0)        // 0
```

50.1.2 `acos(number) → number` (Num)

Arc cosine (inverse cosine). Returns angle in radians.

- `number` (required): value between -1 and 1

```
acos(1)       // 0.0
acos(0)       // 1.5707963267948966 (π/2)
acos(-1)      // 3.141592653589793 (π)
```

50.1.3 `addBOM(data) → binary` (Bin)

Prepend a Byte Order Mark (BOM) to binary data. Detects encoding from the data.

- `data` (required): binary data to prepend BOM to

```
// Programmatic BOM insertion for binary file construction
let fileBytes = toBinary($input.content, "UTF-8")
{
  withBOM: addBOM(fileBytes)
}
```

Note: for normal output, prefer the header option `output csv {bom: true}` (Chapter 25) which handles BOM automatically. Use `addBOM()` only when constructing raw binary content programmatically.

50.1.4 `addNamespaceDeclarations(xml, namespaces) → xml` (XML)

Add namespace declarations to an XML element.

- `xml` (required): XML UDM value
- `namespaces` (required): object mapping prefix to URI — `{"cbc": "urn:...", "cac": "urn:..."}"`

```
// See Chapter 22 for XML namespace handling.
{
  result: addNamespaceDeclarations($input, {
    "cbc": "urn:oasis:names:specification:ubl:schema:xsd:CommonBasicComponents-2",
    "cac": "urn:oasis:names:specification:ubl:schema:xsd:CommonAggregateComponents-2"
  })
}
```

50.1.5 `addQueryParam(url, name, value) → string` (URL)

Add a query parameter to a URL string.

- **url** (required): the base URL
- **name** (required): parameter name
- **value** (required): parameter value

```
echo '{"baseUrl": "https://api.example.com/search", "term": "hello world"}' \
  | utlx -e 'addQueryParam($input.baseUrl, "q", $input.term)'
# "https://api.example.com/search?q=hello+world"
```

```
{
  url: addQueryParam(addQueryParam($input.baseUrl, "page", "1"), "limit", "50")
}
```

50.1.6 addTax(amount, rate) → number (Fin)

Calculate total amount including tax.

- **amount** (required): net amount before tax
- **rate** (required): tax rate as decimal (e.g., 0.21 for 21%)

```
echo '{"price": 100, "vatRate": 0.21}' | utlx -e 'addTax($input.price, $input.vatRate)'
# 121.0
```

```
{
  netPrice: $input.price,
  totalWithVAT: addTax($input.price, 0.21)
}
```

50.1.7 age(birthdate, referenceDate?) → number (Date)

Calculate age in whole years from a birthdate. Uses today if no reference date provided.

- **birthdate** (required): date of birth
- **referenceDate** (optional): date to calculate age at (defaults to today)

```
echo '{"dob": "1990-03-15"}' | utlx -e 'age(parseDate($input.dob, "yyyy-MM-dd"))'
# 36
```

```
{
  age: age(parseDate($input.dateOfBirth, "yyyy-MM-dd")),
  isMinor: age(parseDate($input.dateOfBirth, "yyyy-MM-dd")) < 18
}
```

50.1.8 analyzeString(string, pattern) → object (Str)

Analyze a string against a regex pattern, returning match status and captured groups.

- **string** (required): the string to analyze
- **pattern** (required): regex pattern with capture groups

Returns: {match: boolean, groups: [string...], start: number, end: number}

```
echo '{"email": "user@example.com"}' \
| utlx -e 'analyzeString($input.email, "(.+)\@(.+)\.\.+(.+)" )'
# {"match": true, "groups": ["user", "example", "com"], "start": 0, "end": 16}
```

```
// Input: {"date": "2026-05-01"}
let result = analyzeString($input.date, "^(\d{4})-(\d{2})-(\d{2})$")
// result = {match: true, groups: ["2026", "05", "01"], start: 0, end: 10}
{
  valid: result.match,           // true
  year: if (result.match) result.groups[0] else null, // "2026"
  month: if (result.match) result.groups[1] else null // "05"
}
// Output: {"valid": true, "year": "2026", "month": "05"}
```

50.1.9 and(values...) → boolean (Type)

Logical AND — returns true only if all arguments are truthy.

- **values** (variadic): boolean values to combine

```
echo '{"active": true, "verified": true, "paid": false}' \
| utlx -e 'and($input.active, $input.verified, $input.paid)'
# false (all must be true, but paid is false)
```

```
// Input: {"active": true, "verified": true, "paid": true}
{
  canShip: and($input.active, $input.verified, $input.paid), // true
  canRefund: and($input.paid, $input.delivered)              // false if delivered is
null
}
```

50.1.10 asin(number) → number (Num)

Arc sine (inverse sine). Returns angle in radians.

- **number** (required): value between -1 and 1

```
asin(0)      // 0.0
asin(1)      // 1.5707963267948966 ( $\pi/2$ )
asin(-1)     // -1.5707963267948966 ( $-\pi/2$ )
```

50.1.11 assert(condition, message?) → null (Sys)

Assert that a condition is true. Throws an error with the message if condition is false.

- **condition** (required): boolean expression to verify
- **message** (optional): error message if assertion fails

```
assert(count($input.items) > 0, "Input must have at least one item")
assert($input.amount >= 0, "Amount cannot be negative")
```

50.1.12 assertEquals(actual, expected) → null (Sys)

Assert two values are equal. Throws an error showing both values if they differ.

- **actual** (required): the value to test

- **expected** (required): the expected value

```
assertEqual(count($input.items), 3)
assertEqual($input.status, "ACTIVE")
```

50.1.13 atan(number) → number (Num)

Arc tangent (inverse tangent). Returns angle in radians.

- **number** (required): any numeric value

```
atan(0)          // 0.0
atan(1)          // 0.7853981633974483 (π/4)
atan(-1)         // -0.7853981633974483 (-π/4)
```

50.1.14 atan2(y, x) → number (Num)

Two-argument arc tangent. Converts Cartesian coordinates (x, y) to polar angle in radians.

- **y** (required): y-coordinate
- **x** (required): x-coordinate

```
atan2(1, 1)      // 0.7853981633974483 (π/4 – 45°)
atan2(0, -1)     // 3.141592653589793 (π – 180°)
atan2(-1, 0)     // -1.5707963267948966 (-π/2 – -90°)
```

50.1.15 attribute(element, name) → string (XML)

Get a specific attribute value from an XML element by name. Useful when the attribute name is dynamic (stored in a variable). For static attribute access, prefer the @ operator: `$input.@id`.

- **element** (required): XML UDM element
- **name** (required): attribute name (string)

```
// Input: <Order id="ORD-123" status="active">...</Order>
// See Chapter 22 for XML attribute access (@ operator).

// Preferred: use @ for known attribute names
{
  orderId: $input.@id,           // "ORD-123"
  status: $input.@status        // "active"
}

// Use attribute() when the name is dynamic (e.g. from input data)
{
  value: attribute($input, $input.lookupField) // attribute name from data
}
```

50.1.16 attributes(element) → object (XML)

Get all attributes from an XML element as a key-value object. Useful for iterating or spreading all attributes. For accessing individual attributes, prefer `$input.@name`.

- **element** (required): XML UDM element

```
// Input: <Product id="P-1" sku="ABC123" category="electronics"/>
// See Chapter 22 for XML attribute access.
```



```
{
  allAttrs: attributes($input),          // {"id": "P-1", "sku": "ABC123", "category":
  "electronics"}
  attrCount: count(keys(attributes($input))) // 3
}
```

50.1.17 availableProcessors() → number (Sys)

Get the number of available CPU cores/processors on the current system.

```
{
  cpuCores: availableProcessors()
}
// Output: {"cpuCores": 8}
```

50.1.18 Date Format Patterns (Date — Reference)

Before the date functions: UTL-X uses Java's `DateTimeFormatter` pattern tokens. Case matters — **MM** is months, **mm** is minutes:

Token	Meaning	Example
yyyy	4-digit year	2026
yy	2-digit year	26
MM	Month (01-12)	05
MMM	Month abbreviation	May
MMMM	Month full name	May
dd	Day of month (01-31)	01
HH	Hour 24h (00-23)	14
hh	Hour 12h (01-12)	02
mm	Minute (00-59)	30
ss	Second (00-59)	45
EEEE	Day name	Thursday
EEE	Day abbreviation	Thu
a	AM/PM	PM
XXX	Timezone offset	+02:00
'...'	Literal text	'T' outputs T

Why uppercase **MM but lowercase **mm**?** To distinguish months from minutes. `yyyy-MM-dd` is year-month-day. `HH:mm:ss` is hour-minute-second. Mixing them up (`yyyy-mm-dd`) parses minutes where months should be — a common bug.

50.1.19 addDays(date, count) → date (Date)

Add (or subtract) a number of days to a date.

- **date** (required): the starting date or datetime
- **count** (required): number of days to add. Negative to subtract.

```
echo '{"orderDate": "2026-05-01", "deliveryDays": 14}' \
| utlx -e 'formatDate(addDays(parseDate(.orderDate, "yyyy-MM-dd"), .deliveryDays),
```

```
"yyyy-MM-dd" '
# "2026-05-15"
```

```
%utlx 1.0
input json
output json
---
let order = parseDate($input.orderDate, "yyyy-MM-dd")
{
  orderDate: $input.orderDate,
  deliveryDate: formatDate(addDays(order, $input.deliveryDays), "yyyy-MM-dd"),
  paymentDue: formatDate(addDays(order, 30), "yyyy-MM-dd")
}
```

```
addDays(parseDate("2026-05-01", "yyyy-MM-dd"), 14)    // 2026-05-15
addDays(parseDate("2026-05-01", "yyyy-MM-dd"), -7)    // 2026-04-24 (subtract)
addDays(parseDate("2026-02-27", "yyyy-MM-dd"), 2)     // 2026-03-01 (crosses month
boundary)
```

50.1.20 addHours(datetime, count) → datetime (Date)

Add (or subtract) hours to a datetime.

- **datetime** (required): the starting datetime
- **count** (required): number of hours to add. Negative to subtract.

```
addHours(now(), 3)                // 3 hours from now
addHours(now(), -24)              // 24 hours ago (same as yesterday, same time)
```

50.1.21 addMinutes(datetime, count) → datetime (Date)

Add (or subtract) minutes to a datetime.

- **datetime** (required): the starting datetime
- **count** (required): number of minutes to add. Negative to subtract.

```
addMinutes(now(), 90)             // 1.5 hours from now
addMinutes(now(), -30)            // 30 minutes ago
```

50.1.22 addMonths(date, count) → date (Date)

Add (or subtract) months to a date. If the resulting day exceeds the target month's length, it is clamped to the last day of that month.

- **date** (required): a date or datetime value — NOT a string. If your date is a string (JSON, XML, CSV input), convert it first with `parseDate()`. YAML auto-parses dates, so YAML input values are already dates.
- **count** (required): number of months to add. Negative to subtract.

```
// From JSON/XML/CSV input – date is a string, parseDate needed:
let orderDate = parseDate($input.orderDate, "yyyy-MM-dd")
addMonths(orderDate, 3)           // 2026-08-01

// From YAML input – date is already a date value, no parseDate needed:
```

```

addMonths($input.orderDate, 3)           // works directly

// With string literals in examples, parseDate is always needed:
addMonths(parseDate("2026-05-01", "yyyy-MM-dd"), -2) // 2026-03-01

// Edge case: end-of-month clamping
addMonths(parseDate("2026-01-31", "yyyy-MM-dd"), 1)   // 2026-02-28 (not Feb 31)
addMonths(parseDate("2026-03-31", "yyyy-MM-dd"), 1)   // 2026-04-30 (not Apr 31)

```

50.1.23 addQuarters(date, count) → date (Date)

Add (or subtract) quarters (3-month periods) to a date.

- **date** (required): the starting date or datetime
- **count** (required): number of quarters to add. Negative to subtract.

```

addQuarters(parseDate("2026-01-15", "yyyy-MM-dd"), 1) // 2026-04-15
addQuarters(parseDate("2026-01-15", "yyyy-MM-dd"), 4) // 2027-01-15 (1 year)

```

50.1.24 addSeconds(datetime, count) → datetime (Date)

Add (or subtract) seconds to a datetime.

- **datetime** (required): the starting datetime
- **count** (required): number of seconds to add. Negative to subtract.

```

addSeconds(now(), 3600)           // 1 hour from now (3600 seconds)
addSeconds(now(), -1)            // 1 second ago

```

50.1.25 addWeeks(date, count) → date (Date)

Add (or subtract) weeks to a date. Equivalent to `addDays(date, count * 7)`.

- **date** (required): the starting date or datetime
- **count** (required): number of weeks to add. Negative to subtract.

```

addWeeks(parseDate("2026-05-01", "yyyy-MM-dd"), 2) // 2026-05-15
addWeeks(parseDate("2026-05-01", "yyyy-MM-dd"), -1) // 2026-04-24

```

50.1.26 addYears(date, count) → date (Date)

Add (or subtract) years to a date. Handles leap year edge cases.

- **date** (required): the starting date or datetime
- **count** (required): number of years to add. Negative to subtract.

```

addYears(parseDate("2026-05-01", "yyyy-MM-dd"), 1) // 2027-05-01
addYears(parseDate("2026-05-01", "yyyy-MM-dd"), -10) // 2016-05-01

// Edge case: leap year
addYears(parseDate("2024-02-29", "yyyy-MM-dd"), 1) // 2025-02-28 (2025 is not a leap year)

```

50.1.27 all(array, predicate) → boolean (Arr)

Returns true if ALL elements satisfy the predicate. Returns true for empty arrays (vacuously true).

- **array** (required): the array to test
- **predicate** (required): `lambda (element) -> boolean`

```
echo '{"items": [{"price": 10}, {"price": 25}, {"price": 5}]}' \
| utlx -e 'all($input.items, (item) -> item.price > 0)'
# true
```

```
{
  allPositive: all($input.items, (item) -> item.price > 0),
  allExpensive: all($input.items, (item) -> item.price > 20)
}
// Output: {"allPositive": true, "allExpensive": false}
```

50.1.28 **any(array, predicate) → boolean** (Arr)

Returns true if at least ONE element satisfies the predicate. Returns false for empty arrays.

- **array** (required): the array to test
- **predicate** (required): `lambda (element) -> boolean`

```
echo '{"orders": [{"status": "PENDING"}, {"status": "SHIPPED"}, {"status": "PENDING"}]}' \
| utlx -e 'any($input.orders, (o) -> o.status == "SHIPPED")'
# true
```

```
{
  hasShipped: any($input.orders, (o) -> o.status == "SHIPPED"),
  hasCancelled: any($input.orders, (o) -> o.status == "CANCELLED")
}
// Output: {"hasShipped": true, "hasCancelled": false}
```

50.1.29 **avg(array) → number** (Num)

Average of numeric values in an array. Returns 0 for empty arrays.

- **array** (required): array of numbers

```
echo '{"scores": [85, 92, 78, 95, 88]}' | utlx -e 'avg($input.scores)'
# 87.6
```

```
{
  averageScore: avg($input.scores),
  averageEmpty: avg([])           // 0 (safe on empty arrays)
}
```

Anti-pattern: `sum(arr) / count(arr)` — crashes on empty arrays (division by zero). Use `avg()` which handles this safely.

50.1.30 **avgBy(array, keyFn) → number** (Num)

Average of values extracted from an array of objects using a key function.

- **array** (required): array of objects
- **keyFn** (required): `lambda (element) -> number`

```
echo '{"products": [{ "name": "A", "price": 10}, {"name": "B", "price": 30}, {"name": "C",
"price": 20}]} ' \
| utlx -e 'avgBy($input.products, (p) -> p.price)'
# 20
```

```
{
  avgPrice: avgBy($input.products, (p) -> p.price),
  avgWeight: avgBy($input.products, (p) -> p.weight)
}
```

50.2 B

50.2.1 base64Encode(data) → string (Sec)

Encode data to a Base64 string for safe transport (e.g., in URLs or headers).

- **data** (required): string to encode

```
{
  encoded: base64Encode("Hello, World!")
}
// Output: {"encoded": "SGVsbG8sIFdvcmxkIQ=="}
```

50.2.2 base64Decode(string) → string (Sec)

Decode a Base64-encoded string back to its original value.

- **string** (required): Base64-encoded string to decode

```
{
  decoded: base64Decode("SGVsbG8sIFdvcmxkIQ==")
}
// Output: {"decoded": "Hello, World!"}

// Real-world: decode a Base64-encoded JWT payload
let payload = split($input.token, ".")[1]
let decoded = parseJson(base64Decode(payload))
{
  subject: decoded.sub
}
// Output: {"subject": "user@example.com"}
```

50.2.3 bearing(lat1, lon1, lat2, lon2) → number (Geo)

Calculate the initial bearing (forward azimuth) from point1 to point2 in degrees (0-360).

- **lat1** (required): latitude of origin point
- **lon1** (required): longitude of origin point
- **lat2** (required): latitude of destination point
- **lon2** (required): longitude of destination point

```
echo '{"from": {"lat": 52.37, "lon": 4.90}, "to": {"lat": 48.86, "lon": 2.35}}' \
| utlx -e 'bearing($input.from.lat, $input.from.lon, $input.to.lat, $input.to.lon)'
# 210.5 (degrees – roughly southwest)
```

```
{
  heading: bearing($input.origin.lat, $input.origin.lon, $input.dest.lat, $input.dest.lon)
}
```

50.2.4 **binaryConcat(binary1, binary2, ...) → binary** (Bin)

Concatenate multiple binary values into a single binary.

- **binary1** (required): first binary segment
- **binary2** (required): second binary segment
- ... (optional): additional binary segments

```
let header = toBinary("HDR:", "UTF-8")
let payload = toBinary($input.data, "UTF-8")
{
  frame: binaryConcat(header, payload)
}
```

50.2.5 **binaryEquals(binary1, binary2) → boolean** (Bin)

Compare two binary values for byte-level equality.

- **binary1** (required): first binary
- **binary2** (required): second binary

```
{
  match: binaryEquals(toBinary("hello", "UTF-8"), toBinary("hello", "UTF-8")),
  differ: binaryEquals(toBinary("hello", "UTF-8"), toBinary("world", "UTF-8"))
}
// Output: {"match": true, "differ": false}
```

50.2.6 **binaryLength(binary) → number** (Bin)

Get the length of a binary value in bytes.

- **binary** (required): binary data

```
{
  sizeBytes: binaryLength(toBinary($input.content, "UTF-8"))
}
```

50.2.7 **binarySlice(binary, start, end) → binary** (Bin)

Extract a subsequence of bytes from binary data.

- **binary** (required): source binary
- **start** (required): start byte offset (0-based)
- **end** (required): end byte offset (exclusive)

```
let data = toBinary("Hello World", "UTF-8")
{
  first5: binaryToString(binarySlice(data, 0, 5), "UTF-8"),
  rest: binaryToString(binarySlice(data, 6, 11), "UTF-8")
}
// Output: {"first5": "Hello", "rest": "World"}
```

50.2.8 binaryToString(binary, encoding) → string (Bin)

Decode binary data to a string using the specified character encoding.

- **binary** (required): binary data to decode
- **encoding** (required): character encoding — "UTF-8", "ISO-8859-1", "US-ASCII", etc.

```
{
  text: binaryToString(base64Decode($input.payload), "UTF-8")
}
```

50.2.9 bitwiseAnd(binary1, binary2) → binary (Bin)

Perform bitwise AND on two binary values.

- **binary1** (required): first operand
- **binary2** (required): second operand (same length)

```
let mask = fromHex("FF00FF00")
let data = fromHex("AABBCCDD")
{
  masked: toHex(bitwiseAnd(data, mask))
}
// Output: {"masked": "aa00cc00"}
```

50.2.10 bitwiseNot(binary) → binary (Bin)

Perform bitwise NOT (inversion) on a binary value.

- **binary** (required): binary data to invert

```
{
  inverted: toHex(bitwiseNot(fromHex("FF00")))
}
// Output: {"inverted": "00ff"}
```

50.2.11 bitwiseOr(binary1, binary2) → binary (Bin)

Perform bitwise OR on two binary values.

- **binary1** (required): first operand
- **binary2** (required): second operand (same length)

```
{
  combined: toHex(bitwiseOr(fromHex("AA00"), fromHex("0055")))
}
// Output: {"combined": "aa55"}
```

50.2.12 bitwiseXor(binary1, binary2) → binary (Bin)

Perform bitwise XOR on two binary values.

- **binary1** (required): first operand
- **binary2** (required): second operand (same length)

```
{
  xored: toHex(bitwiseXor(fromHex("AAFF"), fromHex("55FF")))
}
```

```
}
// Output: {"xored": "ff00"}
```

50.2.13 boundingBox(coordinates) → object (Geo)

Calculate the bounding box (min/max latitude and longitude) for an array of coordinates.

- **coordinates** (required): array of {lat, lon} objects

```
echo '{"points": [{"lat": 52.37, "lon": 4.90}, {"lat": 48.86, "lon": 2.35}, {"lat": 51.51,
"lon": -0.13}]}' \
| utlx -e 'boundingBox($input.points)'
# {"minLat": 48.86, "maxLat": 52.37, "minLon": -0.13, "maxLon": 4.90}
```

```
{
  bounds: boundingBox($input.locations)
}
```

50.2.14 buildQueryString(params) → string (URL)

Build a URL query string from an object of key-value pairs.

- **params** (required): object with parameter names and values

```
echo '{"page": 2, "limit": 50, "sort": "name"}' \
| utlx -e 'buildQueryString($input)'
# "page=2&limit=50&sort=name"
```

```
{
  queryString: buildQueryString({q: $input.search, page: "1", format: "json"})
}
```

50.2.15 buildURL(components) → string (URL)

Build a complete URL from its component parts.

- **components** (required): object with protocol, host, port?, path?, query?, fragment?

```
echo '{"host": "api.example.com", "path": "/v2/users"}' \
| utlx -e 'buildURL({protocol: "https", host: $input.host, path: $input.path, query:
{active: "true"}})'
# "https://api.example.com/v2/users?active=true"
```

```
{
  endpoint: buildURL({
    protocol: "https",
    host: $input.apiHost,
    path: concat("/api/v1/", $input.resource),
    query: {format: "json"}
  })
}
```


50.3 C

50.3.1 `c14n(xml) → string` (XML)

Canonicalize an XML document using W3C C14N (sorted attributes, normalized whitespace). See Chapter 22 for full details.

- `xml` (required): XML UDM value to canonicalize

```
echo '<Order id="1" status="new"/>' | utlx -f xml -e 'c14n($input)'
# canonical XML string (sorted attributes, normalized whitespace)
```

```
{
  canonical: c14n($input),
  fingerprint: c14nFingerprint($input)
}
```

Also: `c14nWithComments(xml)`, `excC14n(xml)` (exclusive, for SOAP), `c14n11(xml)` (version 1.1), `c14nFingerprint(xml)` (short hash for logging).

50.3.2 `c14nHash(xml, algorithm?) → string` (XML)

Compute a hash digest of the canonical form of an XML document.

- `xml` (required): XML UDM value to canonicalize and hash
- `algorithm` (optional, default "SHA-256"): hash algorithm (e.g., "SHA-512")

```
echo '<Invoice id="1"><Total>100</Total></Invoice>' \
  | utlx -f xml -e 'c14nHash($input)'
# "a1b2c3d4e5..." (SHA-256 hex digest of canonical form)
```

```
{
  digest256: c14nHash($input),
  digest512: c14nHash($input, "SHA-512")
}
```

50.3.3 `c14nEquals(xml1, xml2) → boolean` (XML)

Compare two XML documents semantically (ignoring formatting differences) by comparing their canonical forms. See Chapter 22.

- `xml1` (required): first XML UDM value
- `xml2` (required): second XML UDM value

```
{
  match: c14nEquals($input.expected, $input.actual),
  status: if (c14nEquals($input.expected, $input.actual)) "PASS" else "FAIL"
}
```

50.3.4 `c14n11(xml) → string` (XML)

Canonicalize XML using Canonical XML 1.1 (without comments). Handles `xml:id` and other 1.1-specific features.

- `xml` (required): XML UDM value

```
{
  canonical: c14n11($input)
}
```

50.3.5 c14n11WithComments(xml) → string (XML)

Canonicalize XML using Canonical XML 1.1 with comments preserved.

- `xml` (required): XML UDM value

```
{
  canonical: c14n11WithComments($input)
}
```

50.3.6 c14nFingerprint(xml) → string (XML)

Create a short hash fingerprint of the canonical form of XML. Useful for deduplication and logging.

- `xml` (required): XML UDM value

```
{
  fingerprint: c14nFingerprint($input),
  isDuplicate: c14nFingerprint($input) == $input.lastSeen
}
```

50.3.7 c14nPhysical(xml, options?) → string (XML)

Canonicalize XML using Physical Canonical XML (preserves physical structure more faithfully).

- `xml` (required): XML UDM value
- `options` (optional): canonicalization options

```
{
  physical: c14nPhysical($input)
}
```

50.3.8 c14nSubset(xml, xpath) → string (XML)

Canonicalize a subset of an XML document selected by XPath expression.

- `xml` (required): XML UDM value
- `xpath` (required): XPath expression selecting the subset to canonicalize

```
// See Chapter 22 for XML canonicalization.
{
  bodyCanonical: c14nSubset($input, "//soap:Body")
}
```

50.3.9 c14nWithComments(xml) → string (XML)

Canonicalize XML using Canonical XML 1.0 with comments preserved.

- `xml` (required): XML UDM value

```
{
  canonical: c14nWithComments($input)
}
```

50.3.10 calculateDiscount(price, rate) → number (Fin)

Calculate the price after applying a percentage discount.

- **price** (required): original price
- **rate** (required): discount rate as decimal (e.g., 0.10 for 10%)

```
echo '{"price": 200, "discount": 0.15}' | utlx -e 'calculateDiscount($input.price,
$input.discount)'
# 170.0
```

```
{
  originalPrice: $input.price,
  discountedPrice: calculateDiscount($input.price, 0.10)
}
```

50.3.11 calculateTax(amount, rate) → number (Fin)

Calculate the tax amount for a given amount and rate (returns only the tax portion, not the total).

- **amount** (required): taxable amount
- **rate** (required): tax rate as decimal (e.g., 0.21 for 21%)

```
echo '{"subtotal": 500, "vatRate": 0.21}' | utlx -e 'calculateTax($input.subtotal,
$input.vatRate)'
# 105.0
```

```
{
  subtotal: $input.amount,
  tax: calculateTax($input.amount, 0.21),
  total: addTax($input.amount, 0.21)
}
```

50.3.12 camelCase(string) → string (Str)

Convert a string to camelCase.

- **string** (required): the string to convert

```
camelCase("hello world")      // "helloWorld"
camelCase("some-kebab-case")  // "someKebabCase"
camelCase("SCREAMING_SNAKE")  // "screamingSnake"
```

50.3.13 camelize(string) → string (Str)

Alias for camelCase(). Convert a string to camelCase. Prefer camelCase().

- **string** (required): the string to convert

```
camelize("my_field_name")     // "myFieldName"
```

50.3.14 canCoerce(value, targetType) → boolean (Type)

Check if a value can be coerced to a target type without error.

- **value** (required): the value to test
- **targetType** (required): target type as string — "number", "boolean", "date", etc.

```
canCoerce("123", "number")    // true
canCoerce("abc", "number")    // false
canCoerce("true", "boolean")  // true
```

50.3.15 canonicalizeWithAlgorithm(xml, algorithm) → string (XML)

Canonicalize XML using a specified W3C canonicalization algorithm URI.

- **xml** (required): XML UDM value
- **algorithm** (required): algorithm URI (e.g., "http://www.w3.org/2001/10/xml-exc-c14n#")

```
// See Chapter 22 for XML canonicalization.
{
  canonical: canonicalizeWithAlgorithm($input, "http://www.w3.org/2001/10/xml-exc-c14n#")
}
```

50.3.16 canonicalJSONHash(json, algorithm?) → string (JSON)

Canonicalize JSON per RFC 8785 and compute a cryptographic hash of the result.

- **json** (required): JSON string or UDM value
- **algorithm** (optional, default "SHA-256"): hash algorithm

```
// See Chapter 24 for JSON processing.
{
  digest: canonicalJSONHash(renderJson($input)),
  digest512: canonicalJSONHash(renderJson($input), "SHA-512")
}
```

50.3.17 canonicalJSONSize(json) → number (JSON)

Get the size in bytes (UTF-8) of the canonical JSON form.

- **json** (required): JSON string or UDM value

```
// See Chapter 24 for JSON processing.
{
  sizeBytes: canonicalJSONSize(renderJson($input))
}
```

50.3.18 capitalize(string) → string (Str)

Capitalize the first letter of a string. Only affects the first character.

- **string** (required): the string to capitalize

```
capitalize("hello world")    // "Hello world"
capitalize("HELLO")          // "HELLO" (already uppercase — no change)
capitalize("")                // ""
```

50.3.19 `ceil(number) → integer (Num)`

Round up to the nearest integer (ceiling).

- `number` (required): the number to round up

```
ceil(3.2)      // 4
ceil(3.9)      // 4
ceil(-3.2)     // -3 (towards zero for negatives)
ceil(4.0)      // 4 (already integer)
```

50.3.20 `charAt(string, index) → string (Str)`

Get the character at a specific index (0-based).

- `string` (required): the source string
- `index` (required): position (0-based)

```
charAt("Hello", 0)  // "H"
charAt("Hello", 4)  // "o"
```

50.3.21 `charCodeAt(string, index) → number (Str)`

Get the Unicode code point of the character at a specific index.

- `string` (required): the source string
- `index` (required): position (0-based)

```
charCodeAt("A", 0)  // 65
charCodeAt("a", 0)  // 97
charCodeAt("€", 0)  // 8364
```

50.3.22 `childCount(element) → number (XML)`

Count the number of child elements in an XML element.

- `element` (required): XML UDM element

```
// Input: <Order><Item/><Item/><Item/></Order>
// See Chapter 22 for XML processing.
{
  itemCount: childCount($input)  // 3
}
```

50.3.23 `childNames(element) → array (XML)`

Get the names of all child elements.

- `element` (required): XML UDM element

```
// Input: <Order><Id>1</Id><Date>2026-05-01</Date><Total>100</Total></Order>
// See Chapter 22 for XML processing.
{
  fields: childNames($input)  // ["Id", "Date", "Total"]
}
```

50.3.24 `chunkBy(array, predicate) → array of arrays (Arr)`

Split a flat sequence into chunks. A new chunk starts whenever the predicate returns true.

- **array** (required): the array to split
- **predicate** (required): `lambda (element) -> boolean` — `true` starts a new chunk

```
echo '{"items": [1, 2, 10, 11, 12, 20, 21]}' \
| utlx -e 'chunkBy($input.items, (x) -> x >= 10 and x % 10 == 0)'
# [[1, 2], [10, 11, 12], [20, 21]]
```

```
// See Chapter 20 for data restructuring patterns.
{
  groups: chunkBy($input.records, (r) -> r.isHeader)
}
```

50.3.25 `clearLogs()` → `null` (Sys)

Clear all accumulated log entries from the log buffer.

```
clearLogs()
// all previous log entries are discarded
```

50.3.26 `coerce(value, targetType, default?)` → `value` (Type)

Coerce a value to a target type, returning a default if coercion fails.

- **value** (required): value to coerce
- **targetType** (required): target type — `"number"`, `"boolean"`, `"string"`, `"date"`
- **default** (optional): fallback value on failure

```
coerce("123", "number", 0)      // 123
coerce("abc", "number", 0)      // 0 (fallback)
coerce("true", "boolean", false) // true
```

50.3.27 `coerceAll(array, targetType, default?)` → `array` (Type)

Coerce all values in an array to a target type.

- **array** (required): array of values to coerce
- **targetType** (required): target type — `"number"`, `"boolean"`, `"string"`
- **default** (optional): fallback value for failed coercions

```
coerceAll(["1", "2", "three", "4"], "number", 0)
// [1, 2, 0, 4]
```

50.3.28 `compactCSV(csv, options?)` → `string` (CSV)

Compact a CSV string by removing extra whitespace.

- **csv** (required): CSV string to compact
- **options** (optional): formatting options

```
// See Chapter 25 for CSV processing.
{
  compacted: compactCSV($input.csvData)
}
```

50.3.29 compactJSON(json, options?, indent?) → string (JSON)

Compact a JSON string by removing all unnecessary whitespace.

- `json` (required): JSON string to compact
- `options` (optional): formatting options
- `indent` (optional): indentation level

```
echo '{"data": {"name": "Alice", "age": 30}}' | utlx -e 'compactJSON(renderJson($input))'
# {"name":"Alice","age":30}
```

```
// See Chapter 24 for JSON processing.
{
  minified: compactJSON(renderJson($input))
}
```

50.3.30 compactXML(xml, options?, indent?) → string (XML)

Compact an XML string by removing unnecessary whitespace between elements.

- `xml` (required): XML string to compact
- `options` (optional): formatting options
- `indent` (optional): indentation level

```
// See Chapter 22 for XML processing.
{
  minified: compactXML(renderXml($input))
}
```

50.3.31 compareDates(date1, date2) → number (Date)

Compare two dates. Returns negative if date1 is before date2, zero if equal, positive if after.

- `date1` (required): first date
- `date2` (required): second date

```
let d1 = parseDate("2026-01-01", "yyyy-MM-dd")
let d2 = parseDate("2026-06-01", "yyyy-MM-dd")
{
  before: compareDates(d1, d2),    // negative (d1 is before d2)
  after:  compareDates(d2, d1),    // positive (d2 is after d1)
  equal:  compareDates(d1, d1)     // 0 (equal)
}
```

50.3.32 compoundInterest(principal, rate, periods) → number (Fin)

Calculate compound interest (total amount after compounding).

- `principal` (required): initial amount
- `rate` (required): interest rate per period as decimal
- `periods` (required): number of compounding periods

```
echo '{"principal": 1000, "rate": 0.05, "years": 10}' \
| utlx -e 'compoundInterest($input.principal, $input.rate, $input.years)'
# 1628.89 (approximately)
```

```
{
  futureValue: compoundInterest($input.principal, $input.annualRate, $input.years)
}
```

50.3.33 compress(data, algorithm?) → binary (Bin)

Compress binary data using the specified algorithm.

- **data** (required): binary data to compress
- **algorithm** (optional, default "gzip"): compression algorithm — "gzip", "deflate"

```
{
  compressed: base64Encode(compress(toBinary($input.payload, "UTF-8"), "gzip"))
}
```

50.3.34 constantCase(string) → string (Str)

Convert a string to CONSTANT_CASE (uppercase with underscores).

- **string** (required): the string to convert

```
constantCase("hello world")      // "HELLO_WORLD"
constantCase("camelCase")        // "CAMEL_CASE"
constantCase("some-kebab-case")  // "SOME_KEBAB_CASE"
```

50.3.35 containsValue(object, value) → boolean (Obj)

Check if an object contains a specific value (searches all values).

- **object** (required): object to search
- **value** (required): value to find

```
let config = {host: "localhost", port: 5432, db: "mydb"}
{
  hasLocalhost: containsValue(config, "localhost"), // true
  hasRedis: containsValue(config, "redis")           // false
}
```

50.3.36 convertTimezone(datetime, fromTz, toTz) → datetime (Date)

Convert a datetime from one timezone to another.

- **datetime** (required): the datetime value to convert
- **fromTz** (required): source timezone (e.g., "America/New_York")
- **toTz** (required): target timezone (e.g., "Europe/Amsterdam")

```
echo '{"timestamp": "2026-05-01T09:00:00", "fromTz": "America/New_York", "toTz": "Europe/Amsterdam"}' \
| utlx -e 'formatDate(convertTimezone(parseDate($input.timestamp, "yyyy-MM-dd'\''T'\''HH:mm:ss"), $input.fromTz, $input.toTz), "HH:mm")'
# "15:00"
```

```
{
  localTime: convertTimezone(now(), "UTC", "Europe/Amsterdam")
}
```


50.3.37 convertXMLEncoding(xml, targetEncoding) → binary (XML)

Convert an XML document to a different character encoding. See Chapter 22.

- `xml` (required): XML string or UDM value
- `targetEncoding` (required): target encoding — "UTF-8", "ISO-8859-1", "UTF-16", etc.

```
// See Chapter 22 for XML encoding handling.
{
  latin1Xml: convertXMLEncoding($input, "ISO-8859-1")
}
```

50.3.38 cos(radians) → number (Num)

Cosine of an angle in radians.

- `radians` (required): angle in radians

```
cos(0)          // 1.0
cos(pi())       // -1.0
cos(pi() / 2)   // ~0.0 (very small number due to floating point)
```

50.3.39 cosh(number) → number (Num)

Hyperbolic cosine.

- `number` (required): input value

```
cosh(0)        // 1.0
cosh(1)        // 1.5430806348152437
```

50.3.40 countEntries(object, predicate?) → number (Obj)

Count entries in an object, optionally filtered by a predicate.

- `object` (required): the object to count entries of
- `predicate` (optional): `lambda (key, value) -> boolean`

```
let obj = {a: 1, b: null, c: 3, d: null}
{
  total: countEntries(obj),                // 4
  nonNull: countEntries(obj, (k, v) -> v != null) // 2
}
```

50.3.41 createCDATA(content) → string (XML)

Create a CDATA section wrapping the given content.

- `content` (required): text content to wrap

```
// See Chapter 22 for XML processing.
{
  wrapped: createCDATA("<script>alert('hi')</script>")
}
// Output: {"wrapped": "<![CDATA[<script>alert('hi')</script>]]>"}
```

50.3.42 createSOAPEnvelope(body, header?) → xml (XML)

Create a SOAP envelope with proper namespace prefixes.

- **body** (required): XML content for the SOAP Body
- **header** (optional): XML content for the SOAP Header

```
// See Chapter 22 for XML/SOAP processing.
{
  soapMessage: createSOAPEnvelope($input.requestBody, $input.authHeader)
}
```

50.3.43 crossJoin(array1, array2) → array (Arr)

Cartesian product of two arrays — every combination of elements from both.

- **array1** (required): first array
- **array2** (required): second array

```
echo '{"sizes": ["S", "M", "L"], "colors": ["red", "blue"]}' \
| utlx -e 'crossJoin($input.sizes, $input.colors)'
# [["S","red"],["S","blue"],["M","red"],["M","blue"],["L","red"],["L","blue"]]
```

```
{
  variants: map(crossJoin($input.sizes, $input.colors), (pair) -> {
    size: pair[0], color: pair[1]
  })
}
```

50.3.44 csvAddColumn(csv, name, defaultValue) → string (CSV)

Add a new column with a default value to all rows in a CSV string.

- **csv** (required): CSV string
- **name** (required): new column name
- **defaultValue** (required): default value for all rows

```
let csv = "Name,Age\nAlice,30\nBob,25"
{
  result: csvAddColumn(csv, "Status", "active")
}
// Output: {result: "Name,Age,Status\nAlice,30,active\nBob,25,active"}
```

50.3.45 csvCell(csv, row, column) → string (CSV)

Get a specific cell value by row index and column name.

- **csv** (required): CSV string
- **row** (required): row index (0-based, excluding header)
- **column** (required): column name

```
let csv = "Name,Age\nAlice,30\nBob,25"
{
  name: csvCell(csv, 0, "Name"),    // "Alice"
  age: csvCell(csv, 1, "Age")       // "25"
}
```

50.3.46 csvColumn(csv, name) → array (CSV)

Get all values from a specific column as an array.

- `csv` (required): CSV string
- `name` (required): column name

```
let csv = "Name,Age\nAlice,30\nBob,25\nCharlie,35"
{
  names: csvColumn(csv, "Name")    // ["Alice", "Bob", "Charlie"]
}
```

50.3.47 `csvColumns(csv) → array (CSV)`

Get all column names (headers) from CSV data.

- `csv` (required): CSV string

```
let csv = "Name,Age,Email\nAlice,30,alice@example.com"
{
  headers: csvColumns(csv)    // ["Name", "Age", "Email"]
}
```

50.3.48 `csvRemoveColumns(csv, columns) → string (CSV)`

Remove specified columns from a CSV string.

- `csv` (required): CSV string
- `columns` (required): array of column names to remove

```
let csv = "Name,Age,Email,Phone\nAlice,30,a@b.com,555-1234"
{
  stripped: csvRemoveColumns(csv, ["Phone", "Email"])
}
// Output: {stripped: "Name,Age\nAlice,30"}
```

50.3.49 `csvRow(csv, index, options?) → object (CSV)`

Get a specific row by index as an object (keyed by column names).

- `csv` (required): CSV string
- `index` (required): row index (0-based, excluding header)
- `options` (optional): parsing options

```
let csv = "Name,Age\nAlice,30\nBob,25"
{
  firstRow: csvRow(csv, 0)    // {Name: "Alice", Age: "30"}
}
```

50.3.50 `csvRows(csv) → number (CSV)`

Get the number of data rows in a CSV string (excluding header).

- `csv` (required): CSV string

```
let csv = "Name,Age\nAlice,30\nBob,25\nCharlie,35"
{
  rowCount: csvRows(csv)    // 3
}
```

50.3.51 csvSelectColumns(csv, columns) → string (CSV)

Select/project only specific columns from a CSV string.

- **csv** (required): CSV string
- **columns** (required): array of column names to keep

```
let csv = "Name,Age,Email,Phone\nAlice,30,a@b.com,555-1234"
{
  projected: csvSelectColumns(csv, ["Name", "Email"])
}
// Output: {projected: "Name,Email\nAlice,a@b.com"}
```

50.3.52 csvSort(csv, column, ascending?) → string (CSV)

Sort CSV rows by a specified column.

- **csv** (required): CSV string
- **column** (required): column name to sort by
- **ascending** (optional, default `true`): sort direction

```
let csv = "Name,Age\nCharlie,35\nAlice,30\nBob,25"
{
  sorted: csvSort(csv, "Name", true)
}
// Output: {sorted: "Name,Age\nAlice,30\nBob,25\nCharlie,35"}
```

50.3.53 csvSummarize(csv, options?) → object (CSV)

Calculate summary statistics for numeric CSV columns (count, sum, avg, min, max).

- **csv** (required): CSV string
- **options** (optional): summarization options

```
let csv = "Product,Price,Qty\nA,10,5\nB,20,3\nC,15,8"
{
  stats: csvSummarize(csv)
}
// Output: {stats: {Price: {count: 3, sum: 45, avg: 15, min: 10, max: 20},
//   Qty: {count: 3, sum: 16, ...}}}
```

50.3.54 csvTranspose(csv, options?, header?, separator?) → string (CSV)

Transpose CSV — swap rows and columns.

- **csv** (required): CSV string
- **options** (optional): formatting options
- **header** (optional): include header row
- **separator** (optional): field separator

```
let csv = "Name,Age\nAlice,30\nBob,25"
{
  transposed: csvTranspose(csv)
}
// Output: {transposed: "Name,Alice,Bob\nAge,30,25"}
```

50.3.55 `currentDir()` → `string` (`Sys`)

Get the current working directory path.

```
{
  workingDir: currentDir()
}
// Output: {"workingDir": "/home/user/project"}
```

50.3.56 `floor(number)` → `integer` (`Num`)

Round down to the nearest integer (floor).

- `number` (required): the number to round down

```
floor(3.8)      // 3
floor(3.1)      // 3
floor(-3.8)     // -4 (away from zero for negatives)
floor(4.0)      // 4 (already integer)
```

50.3.57 `round(number)` → `integer` (`Num`)

Round to the nearest integer (half-up rounding).

- `number` (required): the number to round

```
round(3.5)      // 4
round(3.4)      // 3
round(-3.5)     // -3

// Round to 2 decimal places (common pattern for currency):
round(6.2895 * 100) / 100 // 6.29
```

Anti-pattern: `round(price)` for currency — loses cents entirely. Use `roundToCents(price)` or `roundToDecimalPlaces(price, 2)` instead.

50.3.58 `chunk(array, size)` → `array of arrays` (`Arr`)

Split an array into sub-arrays of the given size. Last chunk may be smaller.

- `array` (required): the array to split
- `size` (required): maximum elements per chunk

```
echo '{"items": ["A", "B", "C", "D", "E", "F", "G"]}' \
| utlx -e 'chunk($input.items, 3)'
# [["A", "B", "C"], ["D", "E", "F"], ["G"]]
```

```
// Batch processing – send items in groups of 100
{
  batches: map(chunk($input.records, 100), (batch) -> {
    batchSize: count(batch),
    items: batch
  })
}
```

50.3.59 `coalesce(value1, value2, ...)` → `value` (`Type`)

Returns the first non-null argument. Accepts any number of arguments.

- `value1, value2, ...` (variadic): values to check in order

```
echo '{"nickname": null, "displayName": null, "fullName": "Alice Johnson"}' \
| utlx -e 'coalesce($input.nickname, $input.displayName, $input.fullName, "Anonymous")'
# "Alice Johnson"
```

```
{
  name: coalesce($input.nickname, $input.displayName, $input.fullName, "Anonymous")
}
```

Note: for two values, the `??` operator is cleaner: `$input.name ?? "Unknown"` (see Chapter 9).

50.3.60 `compact(array) → array (Arr)`

Remove null, empty string, and false values from an array.

- `array` (required): the array to compact

```
echo '{"tags": ["urgent", null, "", "review", null, "important"]}' \
| utlx -e 'compact($input.tags)'
# ["urgent", "review", "important"]
```

```
{
  contacts: compact([
    $input.name,
    $input.email,
    $input.phone
  ])
}
// null values are removed from the resulting array
```

50.3.61 `concat(string1, string2, ...) → string (Str)`

Concatenate any number of strings.

- `string1, string2, ...` (variadic): strings to concatenate

```
echo '{"title": "Dr.", "firstName": "Alice", "lastName": "Johnson"}' \
| utlx -e 'concat($input.title, " ", $input.firstName, " ", $input.lastName)'
# "Dr. Alice Johnson"
```

```
{
  fullName: concat($input.title, " ", $input.firstName, " ", $input.lastName),
  reference: concat("Order-", toString($input.orderId))
}
```

Anti-pattern: building long strings with `reduce + concat`. Use `join(array, separator)` instead.

50.3.62 `contains(haystack, needle) → boolean (Str/Arr)`

Check if a string contains a substring, or an array contains a value.

- `haystack` (required): the string or array to search in
- `needle` (required): the value to search for

```
echo '{"roles": ["admin", "editor", "viewer"]}' \
| utlx -e 'contains($input.roles, "admin")'
# true
```

```
{
  hasWorld: contains("Hello World", "World"),    // true (string variant)
  isAdmin: contains($input.roles, "admin"),       // true (array variant)
  activeOrders: filter($input.orders, (o) -> contains(["ACTIVE", "PENDING"], o.status))
}
```

50.3.63 count(array) → number (Arr)

Count all elements in an array.

- array (required): the array to count

```
echo '{"orders": [{"status": "ACTIVE"}, {"status": "CLOSED"}, {"status": "ACTIVE"}]}' \
| utlx -e 'count($input.orders)'
# 3
```

```
{
  totalOrders: count($input.orders),
  totalItems: count($input.items)
}
```

50.3.64 countBy(array, predicate) → number (Arr)

Count elements in an array that match a predicate.

- array (required): the array to count
- predicate (required): lambda (element) -> boolean

```
echo '{"orders": [{"status": "ACTIVE"}, {"status": "CLOSED"}, {"status": "ACTIVE"}]}' \
| utlx -e 'countBy($input.orders, (o) -> o.status == "ACTIVE")'
# 2
```

```
let active = filter($input.orders, (o) -> o.status == "ACTIVE")
{
  activeCount: count(active),
  closedCount: countBy($input.orders, (o) -> o.status == "CLOSED"),
  activeNames: map(active, (o) -> o.name)
}
```

50.3.65 createQname(localName, namespaceUri, prefix?) → object (XML)

Create a structured QName from its parts. See Chapter 22.

- localName (required): the local element name without prefix
- namespaceUri (required): the full namespace URI
- prefix (optional): the namespace prefix

```
// See Chapter 22 for XML namespaces.
{
```

```

    qname: createQname("Invoice",
        "urn:oasis:names:specification:ubl:schema:xsd:Invoice-2", "cbc")
  }
  // Output: {qname: {localName: "Invoice",
  //   namespaceUri: "urn:oasis:names:specification:ubl:schema:xsd:Invoice-2",
  //   prefix: "cbc", qualifiedName: "cbc:Invoice"}}

  // Without prefix:
  {
    qname: createQname("Order", "urn:example:orders")
  }
  // Output: {qname: {localName: "Order",
  //   namespaceUri: "urn:example:orders",
  //   prefix: "", qualifiedName: "Order"}}

```

50.3.66 csvFilter(csv, column, operator, value) → string (CSV)

Filter CSV rows by a column condition. Returns a new CSV string. See Chapter 25.

- **csv** (required): CSV string to filter
- **column** (required): column name to test
- **operator** (required): comparison — "eq", "ne", "contains", "startswith", "endswith", "gt", "lt", "gte", "lte"
- **value** (required): value to compare against

```

// See Chapter 25 for CSV processing.
let csv = "Name,Status,Amount\nAlice,ACTIVE,100\nBob,CLOSED,200\nCharlie,ACTIVE,50"
{
  active: csvFilter(csv, "Status", "eq", "ACTIVE"),
  // "Name,Status,Amount\nAlice,ACTIVE,100\nCharlie,ACTIVE,50"
  highValue: csvFilter(csv, "Amount", "gt", "75")
  // "Name,Status,Amount\nAlice,ACTIVE,100\nBob,CLOSED,200"
}

```

Also: csvSort(csv, column, ascending?), csvColumns(csv), csvRows(csv), csvRow(csv, index), csvCell(csv, row, column), csvColumn(csv, name), csvTranspose(csv), csvAddColumn(csv, name, default), csvRemoveColumns(csv, names), csvSelectColumns(csv, names), csvSummarize(csv).

50.4 D

50.4.1 day(date) → number (Date)

Extract the day-of-month component (1-31) from a date.

- **date** (required): a date or datetime value

```

{
  dayOfMonth: day(parseDate("2026-05-15", "yyyy-MM-dd")), // 15
  today: day(now()) // current day of month
}

```

50.4.2 dayOfMonth(date) → number (Date)

Alias for `day()`. Extract the day-of-month component (1-31) from a date.

- **date** (required): a date or datetime value


```
{
  dom: dayOfMonth(parseDate("2026-05-15", "yyyy-MM-dd"))    // 15
}
```

50.4.3 dayOfWeek(date) → number (Date)

Return the day of the week as a number (1=Monday, 7=Sunday).

- **date** (required): a date or datetime value

```
let d = parseDate("2026-05-01", "yyyy-MM-dd")
{
  weekday: dayOfWeek(d)          // 5 (Thursday – 1=Monday, 7=Sunday)
}
```

50.4.4 dayOfWeekName(date) → string (Date)

Return the day of the week as a name (e.g., “Thursday”).

- **date** (required): a date or datetime value

```
let d = parseDate("2026-05-01", "yyyy-MM-dd")
{
  weekdayName: dayOfWeekName(d)    // "Thursday"
}
```

50.4.5 dayOfYear(date) → number (Date)

Return the day number within the year (1-365 or 1-366 for leap years).

- **date** (required): a date or datetime value

```
let d = parseDate("2026-05-01", "yyyy-MM-dd")
{
  ordinalDay: dayOfYear(d)          // 121 (121st day of 2026)
}
```

Also: `daysInMonth(year, month) → daysInMonth(2026, 2)` returns 28. `daysInYear(year) → daysInYear(2024)` returns 366 (leap year). `isLeapYear(year)`.

50.4.6 daysBetween(date1, date2) → number (Date)

Alias for `diffDays()`. Calculate the difference in days between two dates.

- **date1** (required): start date
- **date2** (required): end date

```
let start = parseDate("2026-05-01", "yyyy-MM-dd")
let end = parseDate("2026-05-15", "yyyy-MM-dd")
{
  days: daysBetween(start, end)    // 14
}
```

50.4.7 daysInMonth(date) → number (Date)

Get the number of days in the month of the given date.

- **date** (required): a date or datetime value

```
{
  feb2026: daysInMonth(parseDate("2026-02-01", "yyyy-MM-dd")), // 28
  feb2024: daysInMonth(parseDate("2024-02-01", "yyyy-MM-dd")), // 29 (leap year)
  jan: daysInMonth(parseDate("2026-01-15", "yyyy-MM-dd")) // 31
}
```

50.4.8 daysInYear(date) → number (Date)

Get the number of days in the year of the given date (365 or 366 for leap years).

- **date** (required): a date or datetime value

```
{
  y2026: daysInYear(parseDate("2026-01-01", "yyyy-MM-dd")), // 365
  y2024: daysInYear(parseDate("2024-01-01", "yyyy-MM-dd")) // 366 (leap year)
}
```

50.4.9 debug(value, message?) → value (Sys)

Log a value at DEBUG level and pass it through (does not consume the value).

- **value** (required): value to log and return
- **message** (optional): label for the log entry

```
{
  result: debug($input.amount, "processing amount")
}
// logs: [DEBUG] processing amount: 150.00
// output: {"result": 150.00}
```

50.4.10 debugPrint(value, label?, indent?) → string (Sys)

Create a human-readable debug representation of a UDM value (multi-line, indented).

- **value** (required): value to represent
- **label** (optional): label prefix
- **indent** (optional): indentation level

```
{
  dump: debugPrint($input, "request")
}
// Output: {"dump": "[request] {name: \"Alice\", age: 30, ...}"}
```

50.4.11 debugPrintCompact(value, label?) → string (Sys)

Create a compact single-line debug representation of a UDM value.

- **value** (required): value to represent
- **label** (optional): label prefix

```
{
  dump: debugPrintCompact($input, "payload")
}
// Output: {"dump": "[payload] {name:Alice,age:30}"}
```

50.4.12 decodeJWS(token) → object (Sec)

Decode a JWS (JSON Web Signature) token WITHOUT verifying the signature. Returns header and payload.

- **token** (required): JWS token string

```
{
  decoded: decodeJWS($input.token),
  algorithm: decodeJWS($input.token).header.alg
}
```

50.4.13 decodeJWT(token) → object (Sec)

Decode a JWT token WITHOUT verification. Returns header, payload (claims), and signature.

- **token** (required): JWT token string

```
echo '{"token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc2VyMSJ9.sig"}' \
| utlx -e 'decodeJWT($input.token).payload.sub'
# "user1"
```

```
let jwt = decodeJWT($input.authToken)
{
  subject: jwt.payload.sub,
  issuer: jwt.payload.iss,
  expired: jwt.payload.exp < timestamp()
}
```

50.4.14 decompress(data, algorithm?) → binary (Bin)

Decompress binary data using the specified algorithm.

- **data** (required): compressed binary data
- **algorithm** (optional, default "gzip"): decompression algorithm — "gzip", "deflate"

```
{
  content: binaryToString(decompress(base64Decode($input.compressedPayload), "gzip"),
    "UTF-8")
}
```

50.4.15 decryptAES(data, key, iv) → string (Sec)

Decrypt data using AES-128-CBC.

- **data** (required): Base64-encoded encrypted data
- **key** (required): 16-byte encryption key (Base64 or hex)
- **iv** (required): 16-byte initialization vector (Base64 or hex)

```
{
  plaintext: decryptAES($input.encrypted, $input.key, $input.iv)
}
```

50.4.16 decryptAES256(data, key, iv) → string (Sec)

Decrypt data using AES-256-CBC (requires 32-byte key).

- **data** (required): Base64-encoded encrypted data

- **key** (required): 32-byte encryption key (Base64 or hex)
- **iv** (required): 16-byte initialization vector (Base64 or hex)

```
{
  plaintext: decryptAES256($input.ciphertext, $input.key256, $input.iv)
}
```

50.4.17 deepClone(object) → object (Obj)

Create a deep (recursive) copy of an object. Modifications to the clone do not affect the original.

- **object** (required): the object to clone

```
let original = {nested: {value: 42}}
let copy = deepClone(original)
// copy is a fully independent copy – no shared references
{
  cloned: copy    // {nested: {value: 42}}
}
```

50.4.18 deepMerge(obj1, obj2) → object (Obj)

Recursively merge two objects. At each level, properties from **obj2** override **obj1**. Nested objects are merged recursively (not replaced).

- **obj1** (required): base object
- **obj2** (required): override object

```
let base = {server: {host: "localhost", port: 5432, ssl: false}}
let prod = {server: {host: "prod-db.example.com", ssl: true}}
{
  config: deepMerge(base, prod)
}
// Output: {config: {server: {host: "prod-db.example.com", port: 5432, ssl: true}}}
// port survived from base – deep merge, not replace
```

Contrast with spread: `{...base, ...prod}` would REPLACE the entire **server** object, losing **port**. **deepMerge** preserves nested properties.

50.4.19 deepMergeAll(objects) → object (Obj)

Deep merge multiple objects in order (later objects override earlier ones at each nesting level).

- **objects** (required): array of objects to merge

```
let configs = [
  {server: {host: "localhost", port: 5432}},
  {server: {host: "staging-db.internal"}},
  {server: {ssl: true}}
]
{
  merged: deepMergeAll(configs)
}
// Output: {merged: {server: {host: "staging-db.internal", port: 5432, ssl: true}}}
```

50.4.20 deflate(data) → binary (Bin)

Compress data using raw Deflate algorithm (no gzip header/trailer).

- **data** (required): binary data to compress

```
{
  deflated: base64Encode(deflate(toBinary($input.payload, "UTF-8")))
}
```

50.4.21 **destinationPoint(lat, lon, bearing, distance) → object** (Geo)

Calculate a destination point given a starting point, bearing, and distance.

- **lat** (required): starting latitude
- **lon** (required): starting longitude
- **bearing** (required): bearing in degrees (0-360)
- **distance** (required): distance in kilometers

```
{
  destination: destinationPoint(52.37, 4.90, 180, 100)
}
// Output: {"lat": 51.47, "lon": 4.90} (approximately 100km due south)
```

50.4.22 **detectBOM(binary) → string** (XML)

Detect the Byte Order Mark type from binary data. Returns the encoding name or null if no BOM found.

- **binary** (required): binary data to inspect

```
{
  bomType: detectBOM($input.rawData)
}
// Output: {"bomType": "UTF-8"} or {"bomType": null}
```

50.4.23 **detectXMLEncoding(xmlString) → string** (XML)

Detect the encoding declared in an XML document's declaration.

- **xmlString** (required): XML string or UDM value

```
// Input: <?xml version="1.0" encoding="ISO-8859-1"?><Order>...</Order>
// See Chapter 22 for XML processing.
{
  encoding: detectXMLEncoding($input)
}
// Output: {"encoding": "ISO-8859-1"}

// Input: <Order>...</Order> (no encoding declaration – defaults to UTF-8)
{
  encoding: detectXMLEncoding($input)
}
// Output: {"encoding": "UTF-8"}
```

Also: `convertXMLEncoding(xml, targetEncoding)` re-encodes the document.

50.4.24 **diffDays(date1, date2) → number** (Date)

Difference between two dates in days. Variants: `diffMonths`, `diffYears`, `diffHours`, `diffMinutes`, `diffSeconds`, `diffWeeks`.

- **date1** (required): start date

- **date2** (required): end date

```
echo '{"start": "2026-05-01", "end": "2026-06-15"}' \
| utlx -e 'diffDays(parseDate($input.start, "yyyy-MM-dd"), parseDate($input.end, "yyyy-MM-dd"))'
# 45
```

```
let start = parseDate($input.startDate, "yyyy-MM-dd")
let end = parseDate($input.endDate, "yyyy-MM-dd")
{
  daysBetween: diffDays(start, end),
  weeksBetween: diffWeeks(start, end),
  overdue: if (diffDays(end, now()) > 0) "Yes" else "No"
}
```

50.4.25 diffHours(datetime1, datetime2) → number (Date)

Difference in hours between two datetimes.

- **datetime1** (required): start datetime
- **datetime2** (required): end datetime

```
let start = parseDate("2026-05-01T08:00:00", "yyyy-MM-dd'T'HH:mm:ss")
let end = parseDate("2026-05-01T17:30:00", "yyyy-MM-dd'T'HH:mm:ss")
{
  hours: diffHours(start, end)    // 9
}
```

50.4.26 diffMinutes(datetime1, datetime2) → number (Date)

Difference in minutes between two datetimes.

- **datetime1** (required): start datetime
- **datetime2** (required): end datetime

```
let start = parseDate("2026-05-01T08:00:00", "yyyy-MM-dd'T'HH:mm:ss")
let end = parseDate("2026-05-01T08:45:00", "yyyy-MM-dd'T'HH:mm:ss")
{
  minutes: diffMinutes(start, end)    // 45
}
```

50.4.27 diffMonths(date1, date2) → number (Date)

Difference in months between two dates (approximate whole months).

- **date1** (required): start date
- **date2** (required): end date

```
let start = parseDate("2026-01-15", "yyyy-MM-dd")
let end = parseDate("2026-05-15", "yyyy-MM-dd")
{
  months: diffMonths(start, end)    // 4
}
```

50.4.28 diffSeconds(datetime1, datetime2) → number (Date)

Difference in seconds between two datetimes.

- **datetime1** (required): start datetime
- **datetime2** (required): end datetime

```
let start = parseDate("2026-05-01T08:00:00", "yyyy-MM-dd'T'HH:mm:ss")
let end = parseDate("2026-05-01T08:01:30", "yyyy-MM-dd'T'HH:mm:ss")
{
  seconds: diffSeconds(start, end)    // 90
}
```

50.4.29 diffWeeks(date1, date2) → number (Date)

Difference in weeks between two dates.

- **date1** (required): start date
- **date2** (required): end date

```
let start = parseDate("2026-05-01", "yyyy-MM-dd")
let end = parseDate("2026-05-22", "yyyy-MM-dd")
{
  weeks: diffWeeks(start, end)    // 3
}
```

50.4.30 diffYears(date1, date2) → number (Date)

Difference in years between two dates.

- **date1** (required): start date
- **date2** (required): end date

```
let start = parseDate("2020-01-01", "yyyy-MM-dd")
let end = parseDate("2026-05-01", "yyyy-MM-dd")
{
  years: diffYears(start, end)    // 6
}
```

50.4.31 distance(lat1, lon1, lat2, lon2) → number (Geo)

Calculate the distance in kilometers between two geographic coordinates using the Haversine formula.

- **lat1** (required): latitude of point 1
- **lon1** (required): longitude of point 1
- **lat2** (required): latitude of point 2
- **lon2** (required): longitude of point 2

```
echo '{"from": {"lat": 52.37, "lon": 4.90}, "to": {"lat": 48.86, "lon": 2.35}}' \
| utlx -e 'distance($input.from.lat, $input.from.lon, $input.to.lat, $input.to.lon)'
# 430.5 (approximately – Amsterdam to Paris in km)
```

```
{
  distanceKm: distance($input.origin.lat, $input.origin.lon, $input.dest.lat,
    $input.dest.lon)
}
```

50.4.32 distinct(array) → array (Arr)

Remove duplicate values from an array using value equality.

- **array** (required): the array to deduplicate

```
echo '{"tags": ["a", "b", "a", "c", "b"]}' | utlx -e 'distinct($input.tags)'
# ["a", "b", "c"]
```

```
{
  uniqueCustomers: distinct(map($input.orders, (o) -> o.customerId)),
  uniqueStatuses: distinct(map($input.orders, (o) -> o.status))
}
```

50.4.33 **distinctBy(array, keyFn) → array** (Arr)

Remove duplicate values from an array using a key extractor to determine uniqueness. Keeps the first element for each key.

- **array** (required): the array to deduplicate
- **keyFn** (required): lambda (element) -> key

```
echo '{"orders": [{"id": 1, "cust": "C-42"}, {"id": 2, "cust": "C-42"}, {"id": 3, "cust": "C-41"}]}' \
| utlx -e 'distinctBy($input.orders, (o) -> o.cust)'
# [{"id": 1, "cust": "C-42"}, {"id": 3, "cust": "C-41"}]
```

```
{
  onePerCustomer: distinctBy($input.orders, (o) -> o.customerId)
}
// keeps first order per customer, removes duplicates
```

50.4.34 **divideBy(object, n) → array** (Obj)

Divide an object into sub-objects each containing at most N key-value pairs.

- **object** (required): object to split
- **n** (required): max entries per sub-object

```
echo '{"a": 1, "b": 2, "c": 3, "d": 4, "e": 5}' | utlx -e 'divideBy($input, 2)'
# [{"a": 1, "b": 2}, {"c": 3, "d": 4}, {"e": 5}]
```

```
{
  batches: divideBy($input.config, 3)
}
```

50.4.35 **dotCase(string) → string** (Str)

Convert a string to dot.case (lowercase words separated by dots).

- **string** (required): the string to convert

```
dotCase("hello world")      // "hello.world"
dotCase("camelCase")        // "camel.case"
dotCase("SCREAMING_SNAKE")  // "screaming.snake"
```


50.4.36 drop(array, n) → array (Arr)

Remove the first N elements from an array, returning the rest.

- **array** (required): the source array
- **n** (required): number of elements to drop

```
echo '{"items": ["A", "B", "C", "D", "E"]}' | utlx -e 'drop($input.items, 2)'
# ["C", "D", "E"]
```

```
// Skip CSV header row (when headers: false)
let dataRows = drop($input, 1)
{
  rows: map(dataRows, (row) -> { name: row[0], value: row[1] })
}
```

50.4.37 take(array, n) → array (Arr)

Keep only the first N elements from an array, discarding the rest.

- **array** (required): the source array
- **n** (required): number of elements to keep

```
echo '{"items": ["A", "B", "C", "D", "E"]}' | utlx -e 'take($input.items, 3)'
# ["A", "B", "C"]
```

```
{
  top10: take(sortBy($input.products, (p) -> -p.sales), 10),
  preview: take($input.items, 5)
}
```

50.5 E**50.5.1 e() → number (Num)**

Returns Euler's number (approximately 2.71828182845904523536).

```
e()                                // 2.718281828459045
{
  euler: e(),
  expGrowth: e() ** $input.rate
}
```

50.5.2 elementPath(element) → string (XML)

Get the XPath-like path of an XML element within its document tree. See Chapter 22.

- **element** (required): XML UDM element

```
{
  path: elementPath($input.Invoice.Lines.Line),
  debug: elementPath($input.root.child)
}
```

50.5.3 encryptAES(data, key, iv) → string (Sec)

Encrypts data using AES-128-CBC. Returns Base64-encoded ciphertext. See Chapter 38.

- **data** (required): plaintext string to encrypt
- **key** (required): 16-byte encryption key (hex or Base64)
- **iv** (required): initialization vector

```
let key = generateKey(128)
let iv = generateIV()
{
  encrypted: encryptAES($input.secret, key, iv),
  key: key,
  iv: iv
}
```

50.5.4 encryptAES256(data, key, iv) → string (Sec)

Encrypts data using AES-256-CBC. Requires a 32-byte key. See Chapter 38.

- **data** (required): plaintext string to encrypt
- **key** (required): 32-byte encryption key
- **iv** (required): initialization vector

```
let key = generateKey(256)
let iv = generateIV()
{
  encrypted: encryptAES256($input.payload, key, iv),
  key: key,
  iv: iv
}
```

50.5.5 endOfDay(date) → datetime (Date)

Get the end of day (23:59:59.999) for a given date.

- **date** (required): date or datetime

```
echo '{"date": "2026-05-01"}' | utlx -e 'endOfDay(parseDate($input.date, "yyyy-MM-dd"))'
# "2026-05-01T23:59:59.999"
```

```
{
  dayEnd: endOfDay(now()),
  deadline: endOfDay(parseDate($input.dueDate, "yyyy-MM-dd"))
}
```

50.5.6 endOfMonth(date) → datetime (Date)

Get the last moment of the last day of the month.

- **date** (required): date or datetime

```
{
  monthEnd: endOfMonth(now()),
  invoiceDue: endOfMonth(parseDate($input.invoiceDate, "yyyy-MM-dd"))
}
```

50.5.7 endOfQuarter(date) → datetime (Date)

Get the last moment of the last day of the quarter.

- **date** (required): date or datetime

```
{
  quarterEnd: endOfQuarter(now()),
  reportDeadline: endOfQuarter(parseDate($input.date, "yyyy-MM-dd"))
}
```

50.5.8 endOfWeek(date) → datetime (Date)

Get the end of the week (Sunday 23:59:59.999).

- **date** (required): date or datetime

```
{
  weekEnd: endOfWeek(now())
}
```

50.5.9 endOfYear(date) → datetime (Date)

Get the last moment of December 31 for the given date's year.

- **date** (required): date or datetime

```
{
  yearEnd: endOfYear(now()),
  fiscalEnd: endOfYear(parseDate($input.startDate, "yyyy-MM-dd"))
}
```

50.5.10 endsWith(string, suffix) → boolean (Str)

Check if a string ends with a given substring.

- **string** (required): the string to test
- **suffix** (required): the substring to check for

```
echo '{"filename": "invoice-2026.xml"}' | utlx -e 'endsWith($input.filename, ".xml")'
# true
```

```
{
  isXml: endsWith($input.filename, ".xml"),
  isJson: endsWith($input.filename, ".json"),
  utlxFiles: filter($input.files, (f) -> endsWith(f.name, ".utlx"))
}
```

50.5.11 startsWith(string, prefix) → boolean (Str)

Check if a string starts with a given substring.

- **string** (required): the string to test
- **prefix** (required): the substring to check for

```
echo '{"orderId": "ORD-001"}' | utlx -e 'startsWith($input.orderId, "ORD-")'
# true
```

```
if (!startsWith($input.id, "ORD-")) error("Invalid order ID format")
{
  validFormat: startsWith($input.orderId, "ORD-"),
  orderId: $input.orderId
}
```

50.5.12 `endTimeer(label) → number` (Sys)

Logs elapsed time since `startTimeer()` was called with the same label. Returns elapsed milliseconds.

- `label` (required): timer label (must match a previous `startTimeer` call)

```
startTimeer("transform")
let result = map($input.items, (i) -> { name: upperCase(i.name) })
endTimeer("transform")
result
```

50.5.13 `enforceNamespacePrefixes(xml, prefixMap) → string` (XML)

Enforces specific namespace prefixes on an XML string, renaming prefixes to match the given mapping. See Chapter 22.

- `xml` (required): XML string to process
- `prefixMap` (optional): object mapping namespace URIs to desired prefixes

```
{
  normalized: enforceNamespacePrefixes($input.xml)
}
```

50.5.14 `entries(object) → array` (Obj)

Decompose an object into an array of `[key, value]` pairs. Essential for dynamic key processing.

- `object` (required): the object to decompose

```
echo '{"servers": {"prod": {"host": "prod-db"}, "staging": {"host": "stg-db"}}}' \
| utlx -e 'entries($input.servers)'
# [{"prod", {"host": "prod-db"}}, {"staging", {"host": "stg-db"}}]
```

```
{
  environments: entries($input.servers) |> map((entry) -> {
    environment: entry[0],
    host: entry[1].host
  })
}
```

50.5.15 `fromEntries(pairs) → object` (Obj)

Build an object from an array of `[key, value]` pairs. The inverse of `entries()`.

- `pairs` (required): array of `[key, value]` arrays

```
echo '{"items": [{"id": "A", "name": "Widget"}, {"id": "B", "name": "Gadget"}]}' \
| utlx -e 'fromEntries(map($input.items, (i) -> [i.id, i.name]))'
# {"A": "Widget", "B": "Gadget"}
```

```
{
  lookup: fromEntries(map($input.items, (i) -> [i.id, i.name]))
}
```

50.5.16 `env(name) → string (Sys)`

Read an environment variable from the host system. Returns null if not set.

- `name` (required): environment variable name

```
{
  home: env("HOME"),                // "/Users/alice"
  dbHost: env("DB_HOST") ?? "localhost" // fallback if not set
}
```

Also: `hasEnv(name) → boolean`, `envAll() → object` with all environment variables.

Security note: `env()` is unrestricted in the CLI and IDE. In the UTLXe engine, environment variable access can be disabled or restricted via the security policy configuration (Chapter 38) to prevent transformations from reading host secrets in multi-tenant deployments.

50.5.17 `envOrDefault(name, default) → string (Sys)`

Read an environment variable, returning a default value if not set.

- `name` (required): environment variable name
- `default` (required): fallback value if the variable is not set

```
envOrDefault("LOG_LEVEL", "INFO")      // "INFO" if LOG_LEVEL not set
{
  logLevel: envOrDefault("LOG_LEVEL", "INFO"),
  dbUrl: envOrDefault("DATABASE_URL", "postgres://localhost:5432/mydb")
}
```

50.5.18 `envAll() → object (Sys)`

Get all environment variables as a key-value object.

```
let allEnv = envAll()
{
  home: allEnv.HOME,
  path: allEnv.PATH,
  count: count(entries(allEnv))
}
```

Security note: can be restricted in UTLXe via security policy (Chapter 38) to prevent exfiltration of host secrets in multi-tenant deployments.

50.5.19 `environment() → string (Sys)`

Get the current environment name (e.g., “development”, “production”).

```
{
  env: environment(),
  isProduction: environment() == "production"
}
```

50.5.20 equals(a, b) → boolean (Type)

Deep equality comparison of two values. Works for all types including objects and arrays.

- a (required): first value
- b (required): second value

```
echo '{"a": [1,2,3], "b": [1,2,3]}' | utlx -e 'equals($input.a, $input.b)'
# true
```

```
{
  same: equals($input.expected, $input.actual),
  match: equals($input.config, $input.defaults)
}
```

50.5.21 error(message) → never (Sys)

Throw a runtime error with a message. Stops the transformation.

- message (required): error description string

```
// Validate input before processing:
if ($input.total < 0) error("Total cannot be negative")
if ($input.currency == null) error("Currency is required")

// Use in a validation pipeline:
let amount = toNumber($input.amount)
if (amount > 1000000) error(concat("Amount exceeds limit: ", toString(amount)))

// Combine with try/catch in the caller:
try {
  if ($input.type == "UNKNOWN") error("Unknown order type")
  // ... process order
} catch {
  {error: true, message: "Processing failed"}
}
```

50.5.22 escapeXML(text) → string (XML)

Escapes special XML characters (<, >, &, ", ') in text without using CDATA. See Chapter 22.

- text (required): string to escape

```
echo '{"text": "price < 100 & qty > 0"}' | utlx -e 'escapeXML($input.text)'
# "price &lt; 100 &amp; qty &gt; 0"
```

```
{
  safeText: escapeXML($input.userInput)
}
```

50.5.23 every(array, predicate) → boolean (Arr)

Check if ALL elements in an array satisfy a predicate. Returns true for empty arrays.

- array (required): the array to test
- predicate (required): lambda (element) -> boolean

```
echo '{"scores": [85, 92, 78, 95]}' | utlx -e 'every($input.scores, (s) -> s >= 70)'
# true
```

```
{
  allPassing: every($input.scores, (s) -> s >= 70),
  allActive: every($input.users, (u) -> u.active),
  allPositive: every($input.amounts, (a) -> a > 0)
}
```

50.5.24 everyEntry(object, predicate) → boolean (Obj)

Check if ALL entries in an object satisfy a predicate. The predicate receives key and value.

- **object** (required): the object to test
- **predicate** (required): lambda (key, value) -> boolean

```
echo '{"a": 1, "b": 2, "c": 3}' | utlx -e 'everyEntry($input, (k, v) -> v > 0)'
# true
```

```
{
  allPositive: everyEntry($input.metrics, (k, v) -> v > 0),
  allNonNull: everyEntry($input, (k, v) -> v != null)
}
```

50.5.25 excC14n(xml) → string (XML)

Canonicalizes XML using Exclusive Canonical XML (without comments). Used for XML digital signatures. See Chapter 22.

- **xml** (required): XML string to canonicalize

```
{
  canonical: excC14n($input.xml)
}
```

50.5.26 excC14nWithComments(xml) → string (XML)

Canonicalizes XML using Exclusive Canonical XML (with comments preserved). See Chapter 22.

- **xml** (required): XML string to canonicalize

```
{
  canonical: excC14nWithComments($input.xml)
}
```

50.5.27 exp(x) → number (Num)

Returns e raised to the power of x (e^x).

- **x** (required): the exponent

```
echo '{"rate": 0.05}' | utlx -e 'exp($input.rate)'
# 1.0512710963760241
```

```
{
  growth: exp($input.rate * $input.years),
  eSquared: exp(2)
}
```

50.5.28 `extractBetween(string, start, end) → string` (Str)

Extract the substring between two delimiter strings.

- `string` (required): the source string
- `start` (required): the start delimiter
- `end` (required): the end delimiter

```
echo '{"msg": "Hello [world] today"}' | utlx -e 'extractBetween($input.msg, "[", "]")'
# "world"
```

```
{
  tag: extractBetween($input.xml, "<name>", "</name>"),
  token: extractBetween($input.header, "Bearer ", " ")
}
```

50.5.29 `extractCDATA(text) → string` (XML)

Extracts the content from a CDATA section, stripping the `<![CDATA[` and `]]>` wrappers. See Chapter 22.

- `text` (required): string containing CDATA section

```
echo '{"xml": "<![CDATA[Hello & World]]>"}' | utlx -e 'extractCDATA($input.xml)'
# "Hello & World"
```

```
{
  content: extractCDATA($input.rawField)
}
```

50.5.30 `extractTimestampFromUuidV7(uuid) → datetime` (Sec)

Extract the embedded timestamp from a UUID v7 value. See Chapter 38.

- `uuid` (required): UUID v7 string

```
echo '{"id": "018f6c30-a2b0-7000-8000-000000000001"}' | utlx -e
'extractTimestampFromUuidV7($input.id)'
# "2024-05-21T..."
```

```
{
  createdAt: extractTimestampFromUuidV7($input.messageId),
  age: diffHours(now(), extractTimestampFromUuidV7($input.messageId))
}
```

50.6 F

50.6.1 `filter(array, predicate) → array` (Arr)

Keep elements that match a predicate. Always returns an array (even if 0 or 1 match).

- **array** (required): the array to filter
- **predicate** (required): `lambda (element) -> boolean`

```
echo '{"name": "Alice", "active": true}, {"name": "Bob", "active": false}]' \
| utlx -e 'filter(., (u) -> u.active)'
# [{"name": "Alice", "active": true}]
```

```
{
  activeProducts: filter($input.products, (p) -> p.active),
  premiumActive: filter($input.products, (p) -> p.price > 100 && p.active),
  overBudget: filter($input.products, (p) -> p.price > 1000)
}
```

Anti-pattern: `$input.products[price > 10]` — bracket predicate syntax does NOT work in UTL-X. Always use `filter()`. See Chapter 8.

Anti-pattern: `filter()` when you want ONE result — use `find()` instead (returns the element, not an array).

50.6.2 `filterEntries(object, predicate) → object (Obj)`

Filter object properties by key and/or value. Returns a new object with only matching entries.

- **object** (required): the object to filter
- **predicate** (required): `lambda (key, value) -> boolean`

```
echo '{"name": "Alice", "email": "alice@example.com", "password": "secret", "temp": null}' \
| utlx -e 'filterEntries(., (key, value) -> value != null)'
# {"name": "Alice", "email": "alice@example.com", "password": "secret"}
```

```
{
  nonNull: filterEntries($input, (key, value) -> value != null),
  safe: filterEntries($input, (key, value) -> key != "password" && key != "temp")
}
```

Also: `someEntry(obj, pred) → true` if any entry matches, `everyEntry(obj, pred) → true` if all match, `countEntries(obj, pred) → count` of matching entries.

50.6.3 `find(array, predicate) → element or null (Arr)`

Returns the FIRST element matching a predicate, or `null` if no match.

- **array** (required): the array to search
- **predicate** (required): `lambda (element) -> boolean`

```
echo '{"users": [{"id": 1, "email": "alice@example.com"}, {"id": 2, "email": "bob@example.com"}]}' \
| utlx -e 'find($input.users, (u) -> u.email == "bob@example.com")'
# {"id": 2, "email": "bob@example.com"}
```

```
{
  bob: find($input.users, (u) -> u.email == "bob@example.com"),
}
```

```
unknown: find($input.users, (u) -> u.email == "unknown@example.com")
}
```

Anti-pattern: `filter($input.users, ...)[0]` — use `find()`. It's cleaner and returns null instead of an index-out-of-bounds error on empty results.

50.6.4 `findIndex(array, predicate) → number (Arr)`

Returns the zero-based index of the FIRST matching element, or -1 if not found.

- `array` (required): the array to search
- `predicate` (required): `lambda (element) -> boolean`

```
echo '{"users": [{"id": 1}, {"id": 2}, {"id": 3}]}' \
| utlx -e 'findIndex($input.users, (u) -> u.id == 2)'
# 1
```

```
{
  position: findIndex($input.users, (u) -> u.id == 2),
  missing: findIndex($input.users, (u) -> u.id == 99)
}
```

Also: `findLastIndex(array, predicate)` — searches from the end.

50.6.5 `findAllMatches(string, pattern) → array (Str)`

Finds all matches of a regex pattern and returns them with positions.

- `string` (required): the string to search
- `pattern` (required): regex pattern

```
echo '{"text": "Order ORD-001 and ORD-002 shipped"}' | utlx -e
'findAllMatches($input.text, "ORD-\\d+")'
# [{"match": "ORD-001", "start": 6}, {"match": "ORD-002", "start": 18}]
```

```
{
  orderIds: findAllMatches($input.text, "ORD-\\d+") |> map((m) -> m.match)
}
```

50.6.6 `findLastIndex(array, predicate) → number (Arr)`

Find the index of the LAST element matching a predicate, or -1 if not found.

- `array` (required): the array to search
- `predicate` (required): `lambda (element) -> boolean`

```
echo '{"items": ["A", "B", "A", "C"]}' | utlx -e 'findLastIndex($input.items, (x) -> x ==
"A")'
# 2
```

```
{
  lastActive: findLastIndex($input.users, (u) -> u.active)
}
```

50.6.7 first(array) → element or null (Arr)

Returns the first element of an array, or `null` if the array is empty.

- `array` (required): the source array

```
first(["Apple", "Banana", "Cherry"]) // "Apple"
first([42])                          // 42
first([])                            // null

// Use case: get the cheapest product
{
  cheapest: first(sortBy($input.products, (p) -> p.price))
}
```

50.6.8 formatCurrency(amount, currency?, locale?) → string (Fin)

Formats a number as currency with locale-specific formatting.

- `amount` (required): numeric amount
- `currency` (optional): ISO 4217 currency code (e.g., “USD”, “EUR”)
- `locale` (optional): locale for formatting (e.g., “en-US”, “de-DE”)

```
echo '{"amount": 1234.5}' | utlx -e 'formatCurrency($input.amount, "USD", "en-US")'
# "$1,234.50"
```

```
{
  price: formatCurrency($input.total, "EUR", "de-DE"),
  usd: formatCurrency($input.amount, "USD", "en-US")
}
```

50.6.9 formatDateTimeInTimezone(datetime, timezone) → string (Date)

Format a datetime value displayed in a specific timezone.

- `datetime` (required): datetime value
- `timezone` (required): timezone ID (e.g., “America/New_York”)

```
echo '{"ts": "2026-05-01T14:30:00Z"}' | utlx -e
'formatDateTimeInTimezone(parseDate($input.ts, "yyyy-MM-dd\\'T\\'HH:mm:ss\\'Z\\'"),
"America/New_York")'
# "2026-05-01T10:30:00-04:00"
```

```
{
  localTime: formatDateTimeInTimezone(now(), "Europe/Amsterdam"),
  userTime: formatDateTimeInTimezone(now(), $input.userTimezone)
}
```

50.6.10 formatEmptyElements(xml, style?) → string (XML)

Formats empty XML elements according to the specified style (self-closing `<br/>` or expanded `<br></br>`). See Chapter 22.

- `xml` (required): XML string
- `style` (optional): “self-closing” or “expanded”

```
{
  selfClosing: formatEmptyElements($input.xml, "self-closing"),
  expanded: formatEmptyElements($input.xml, "expanded")
}
```

50.6.11 `formatNumber(number, pattern?) → string (Num)`

Format a number using a pattern string.

- `number` (required): number to format
- `pattern` (optional): format pattern (e.g., `"#,##0.00"`)

```
echo '{"amount": 1234567.891}' | utlx -e 'formatNumber($input.amount, "#,##0.00")'
# "1,234,567.89"
```

```
{
  formatted: formatNumber($input.price, "#,##0.00"),
  integer: formatNumber($input.qty, "#,##0"),
  percent: formatNumber($input.rate * 100, "0.0")
}
```

50.6.12 `formatPlural(count, word) → string (Str)`

Creates a formatted string with count and correctly pluralized word.

- `count` (required): numeric count
- `word` (required): singular form of the word

```
echo '{"n": 3}' | utlx -e 'formatPlural($input.n, "item")'
# "3 items"
```

```
{
  summary: formatPlural(count($input.errors), "error"),
  files: formatPlural($input.fileCount, "file")
}
```

50.6.13 `fromBase64(encoded) → binary (Sec)`

Create binary data from a Base64-encoded string. See Chapter 38.

- `encoded` (required): Base64-encoded string

```
echo '{"data": "SGVsbG8gV29ybGQ="}' | utlx -e 'toString(fromBase64($input.data))'
# "Hello World"
```

```
{
  decoded: toString(fromBase64($input.encodedPayload))
}
```

50.6.14 `fromBytes(byteArray) → binary (Bin)`

Create binary data from a byte array (array of integers 0-255).

- `byteArray` (required): array of byte values

```
{
  binary: fromBytes([72, 101, 108, 108, 111])
}
```

50.6.15 fromCamelCase(string) → string (Str)

Convert from camelCase to separate words.

- string (required): camelCase string

```
echo '{"name": "myVariableName"}' | utlx -e 'fromCamelCase($input.name)'
# "my variable name"
```

```
{
  words: fromCamelCase($input.fieldName)
}
```

50.6.16 fromCharCode(code) → string (Str)

Create a string from a Unicode character code (code point).

- code (required): integer code point

```
fromCharCode(65)           // "A"
fromCharCode(8364)         // "€"
{
  char: fromCharCode($input.code)
}
```

50.6.17 fromConstantCase(string) → string (Str)

Convert from CONSTANT_CASE to separate words.

- string (required): CONSTANT_CASE string

```
echo '{"name": "MAX_RETRY_COUNT"}' | utlx -e 'fromConstantCase($input.name)'
# "max retry count"
```

```
{
  readable: fromConstantCase($input.envVar)
}
```

50.6.18 fromDotCase(string) → string (Str)

Convert from dot.case to separate words.

- string (required): dot.case string

```
fromDotCase("config.max.retries") // "config max retries"
{
  words: fromDotCase($input.propertyPath)
}
```

50.6.19 fromHex(hexString) → binary (Str)

Create binary data from a hexadecimal string.

- `hexString` (required): hex-encoded string

```
{
  data: fromHex($input.hexPayload),
  text: toString(fromHex("48656c6c6f"))
}
```

50.6.20 fromKebabCase(string) → string (Str)

Convert from kebab-case to separate words.

- `string` (required): kebab-case string

```
echo '{"slug": "my-page-title"}' | utlx -e 'fromKebabCase($input.slug)'
# "my page title"
```

```
{
  title: titleCase(fromKebabCase($input.slug))
}
```

50.6.21 fromPascalCase(string) → string (Str)

Convert from PascalCase to separate words.

- `string` (required): PascalCase string

```
fromPascalCase("MyClassName")          // "my class name"
{
  label: fromPascalCase($input.className)
}
```

50.6.22 fromPathCase(string) → string (Str)

Convert from path/case to separate words.

- `string` (required): path/case string

```
fromPathCase("my/path/name")           // "my path name"
{
  label: fromPathCase($input.route)
}
```

50.6.23 fromSnakeCase(string) → string (Str)

Convert from snake_case to separate words.

- `string` (required): snake_case string

```
echo '{"col": "first_name"}' | utlx -e 'fromSnakeCase($input.col)'
# "first name"
```

```
{
  label: titleCase(fromSnakeCase($input.columnName))
}
```

50.6.24 fromTitleCase(string) → string (Str)

Convert from Title Case to separate lowercase words.

- **string** (required): Title Case string

```
fromTitleCase("My Title Case")          // "my title case"
{
  normalized: fromTitleCase($input.heading)
}
```

50.6.25 fromUTC(datetime, timezone) → datetime (Date)

Convert a UTC datetime to a local datetime in the specified timezone.

- **datetime** (required): UTC datetime
- **timezone** (required): target timezone ID (e.g., “America/New_York”)

```
echo '{"utc": "2026-05-01T14:00:00Z"}' | utlx -e 'fromUTC(parseDate($input.utc, "yyyy-MM-dd'\''T'\''HH:mm:ss'\''Z'\''"), "Europe/Amsterdam")'
# "2026-05-01T16:00:00+02:00"
```

```
{
  localTime: fromUTC(now(), "Asia/Tokyo")
}
```

50.6.26 fullOuterJoin(left, right, leftKey, rightKey) → array (Arr)

Full outer join – returns all items from both arrays, with null for non-matching sides.

- **left** (required): left array
- **right** (required): right array
- **leftKey** (required): lambda to extract join key from left elements
- **rightKey** (required): lambda to extract join key from right elements

```
let employees = [{id: 1, name: "Alice"}, {id: 2, name: "Bob"}]
let salaries = [{empId: 1, amount: 5000}, {empId: 3, amount: 6000}]
{
  joined: fullOuterJoin(employees, salaries, (e) -> e.id, (s) -> s.empId)
}
```

50.6.27 futureValue(presentValue, rate, periods) → number (Fin)

Calculates the future value of a present amount given compound interest.

- **presentValue** (required): current amount
- **rate** (required): interest rate per period (e.g., 0.05 for 5%)
- **periods** (required): number of compounding periods

```
echo '{"pv": 1000, "rate": 0.05, "years": 10}' | utlx -e 'futureValue($input.pv,
$input.rate, $input.years)'
# 1628.89
```

```
{
  futureAmount: futureValue($input.principal, $input.annualRate, $input.years)
}
```

50.6.28 head(array) → element or null (Arr)

Alias for first(). Returns the first element of an array, or null if empty.

- array (required): the source array

```
head(["Apple", "Banana", "Cherry"]) // "Apple"
head([])                             // null
```

50.6.29 last(array) → element or null (Arr)

Returns the last element of an array, or null if the array is empty.

- array (required): the source array

```
last(["Apple", "Banana", "Cherry"]) // "Cherry"
last([42])                          // 42
last([])                            // null

// Use case: get the most recent event
{
  latest: last(sortBy($input.events, (e) -> e.timestamp))
}
```

50.6.30 tail(array) → array (Arr)

Returns everything EXCEPT the first element. Returns an empty array if the input has 0 or 1 elements.

- array (required): the source array

```
tail(["Apple", "Banana", "Cherry"]) // ["Banana", "Cherry"]
tail(["Apple"])                     // []
tail([])                            // []

// Use case: skip the header row in a headerless CSV
let dataRows = tail($input)          // all rows except the first
{ rows: dataRows }
```

50.6.31 flatMap(array, fn) → array (Arr)

Map each element to an array, then flatten one level. Equivalent to flatten(map(...)). Use when each element produces multiple results.

- array (required): the array to process
- fn (required): lambda (element) -> array


```
echo '{"orders": [{"lines": [1,2]}, {"lines": [3]}]}' \
| utlx -e 'flatMap(.orders, (o) -> o.lines)'
# [1, 2, 3]
```

```
{
  allLines: flatMap($input.orders, (o) -> o.lines)
}
```

50.6.32 flatten(array) → array (Arr)

Remove one level of array nesting. Each element that is an array is unwrapped; non-array elements are kept as-is.

- **array** (required): the nested array to flatten

```
{
  flat: flatten([[1, 2], [3, 4], [5]]),
  // [1, 2, 3, 4, 5]
  mixed: flatten([[1, 2], 3, [4, 5]]),
  // [1, 2, 3, 4, 5]
  oneLevel: flatten([[[1, 2]], [[3]]])
  // [[1, 2], [3]] (only ONE level removed)
}
```

50.6.33 flattenDeep(array) → array (Arr)

Remove ALL levels of array nesting, recursively. Produces a completely flat array.

- **array** (required): the deeply nested array to flatten

```
{
  deep: flattenDeep([[1, [2, [3, [4]]]]]),
  // [1, 2, 3, 4]
  nested: flattenDeep([[[["deep"]]]]),
  // ["deep"]
  flat: flattenDeep([1, 2, 3])
  // [1, 2, 3] (already flat – no change)
}
```

50.6.34 formatDate(date, pattern) → string (Date)

Format a date or datetime as a string using a pattern. See the Date Format Patterns section later in this chapter.

- **date** (required): date or datetime value
- **pattern** (required): format pattern string

Pattern tokens: yyyy (year), MM (month 01-12), dd (day 01-31), HH (hour 00-23), mm (minute 00-59), ss (second 00-59), EEEE (day name), MMMM (month name), EEE (short day), MMM (short month).

```
echo '{"timestamp": "2026-05-01T14:30:00Z"}' \
| utlx -e 'formatDate(parseDate($input.timestamp, "yyyy-MM-dd'\''T'\''HH:mm:ss'\''Z'\''"), "dd/MM/yyyy")'
# "01/05/2026"
```

```
let dt = parseDate($input.timestamp, "yyyy-MM-dd'T'HH:mm:ss'Z'")
{
  isoDate: formatDate(dt, "yyyy-MM-dd"),
  european: formatDate(dt, "dd/MM/yyyy"),
  withTime: formatDate(dt, "dd-MM-yyyy HH:mm"),
  human: formatDate(dt, "EEEE, MMMM d, yyyy"),
  invoiceDate: formatDate(now(), "yyyy-MM-dd")
}
```

50.7 G

50.7.1 generateIV() → string (Sec)

Generates a random initialization vector (IV) for use with AES encryption. See Chapter 38.

```
let iv = generateIV()
{
  iv: iv,
  encrypted: encryptAES($input.data, $input.key, iv)
}
```

50.7.2 generateKey(bits?) → string (Sec)

Generates a random encryption key of the specified bit length. See Chapter 38.

- bits (optional): key length in bits (128 or 256, default 128)

```
let key128 = generateKey(128)
let key256 = generateKey(256)
{
  aes128key: key128,
  aes256key: key256
}
```

50.7.3 generateUuid() → string (Sys)

Generate a random UUID (v4). Alias for generateUuidV4().

```
generateUuid()          // "550e8400-e29b-41d4-a716-446655440000" (v4 random)
```

Also: isValidUuid(string), getUuidVersion(string).

50.7.4 generateUuidV4() → string (Sys)

Generate a random UUID version 4.

```
generateUuidV4()        // "550e8400-e29b-41d4-a716-446655440000" (random)
```

50.7.5 generateUuidV7() → string (Sys)

Generate a time-ordered UUID version 7 (sortable by creation time).

```
generateUuidV7()        // "018f6c30-a2b0-7000-8000-000000000001" (time-ordered)

// Use case: generate correlation IDs for messages
{
```

```

    messageId: generateUuidV7(),
    timestamp: now(),
    payload: $input
  }

```

Also: `isUuidV7(string)`.

50.7.6 `generateUuidV7Batch(count) → array` (Sec)

Generate a batch of UUID v7s with monotonic guarantee (each is greater than the previous). See Chapter 38.

- `count` (required): number of UUIDs to generate

```

echo '{"n": 3}' | utlx -e 'generateUuidV7Batch($input.n)'
# ["018f6c30-a2b0-7000-...", "018f6c30-a2b0-7001-...", "018f6c30-a2b0-7002-..."]

```

```

{
  batchIds: generateUuidV7Batch(10)
}

```

50.7.7 `get(object, path) → any` (Obj)

Gets a value from an object or array by key or index.

- `object` (required): object or array
- `path` (required): key name (string) or index (number)

```

echo '{"items": ["a", "b", "c"]}' | utlx -e 'get($input.items, 1)'
# "b"

```

```

{
  second: get($input.items, 1),
  name: get($input, "name")
}

```

50.7.8 `getBaseURL(url) → string` (URL)

Get the base URL (protocol + host + port) from a URL string.

- `url` (required): URL string

```

echo '{"url": "https://api.example.com:8080/v1/users?page=1"}' | utlx -e
'getBaseURL($input.url)'
# "https://api.example.com:8080"

```

```

{
  base: getBaseURL($input.endpoint)
}

```

50.7.9 `getBOMBytes(encoding) → binary` (XML)

Get the BOM (Byte Order Mark) bytes for a given encoding. See Chapter 22.

- `encoding` (required): encoding name (e.g., “UTF-8”, “UTF-16LE”)

```
{
  bom: getBOMBytes("UTF-8")
}
```

50.7.10 `getCurrencyDecimals(currency) → number` (Fin)

Gets the number of decimal places for a currency code (ISO 4217).

- `currency` (required): ISO 4217 currency code

```
echo '{"cur": "JPY"}' | utlx -e 'getCurrencyDecimals($input.cur)'
# 0
```

```
{
  usdDecimals: getCurrencyDecimals("USD"),    // 2
  jpyDecimals: getCurrencyDecimals("JPY")     // 0
}
```

50.7.11 `getFragment(url) → string` (URL)

Get the fragment (hash) portion from a URL string.

- `url` (required): URL string

```
echo '{"url": "https://example.com/page#section2"}' | utlx -e 'getFragment($input.url)'
# "section2"
```

```
{
  fragment: getFragment($input.url)
}
```

50.7.12 `getHost(url) → string` (URL)

Get the host from a URL string.

- `url` (required): URL string

```
echo '{"url": "https://api.example.com/v1/users"}' | utlx -e 'getHost($input.url)'
# "api.example.com"
```

```
{
  host: getHost($input.endpoint)
}
```

50.7.13 `getJWSAlgorithm(token) → string` (Sec)

Gets the algorithm (alg) from a JWS token header. See Chapter 38.

- `token` (required): JWS/JWT token string

```
{
  algorithm: getJWSAlgorithm($input.token)    // "RS256"
}
```

50.7.14 getJWSHeader(token) → object (Sec)

Extracts the complete header from a JWS token as an object. See Chapter 38.

- token (required): JWS/JWT token string

```
{
  header: getJWSHeader($input.token)
  // {"alg": "RS256", "typ": "JWT", "kid": "key-1"}
}
```

50.7.15 getJWSInfo(token) → object (Sec)

Gets information about the JWS token structure (header, payload size, etc.). See Chapter 38.

- token (required): JWS/JWT token string

```
{
  info: getJWSInfo($input.token)
}
```

50.7.16 getJWSKeyId(token) → string (Sec)

Gets the Key ID (kid) from a JWS token header. See Chapter 38.

- token (required): JWS/JWT token string

```
{
  keyId: getJWSKeyId($input.token)
}
```

50.7.17 getJWSPayload(token) → object (Sec)

Extracts the payload from a JWS token WITHOUT signature verification. See Chapter 38.

- token (required): JWS/JWT token string

```
{
  payload: getJWSPayload($input.token)
}
```

50.7.18 getJWSSigningInput(token) → string (Sec)

Extracts the signing input (header.payload) from a JWS token for manual verification. See Chapter 38.

- token (required): JWS/JWT token string

```
{
  signingInput: getJWSSigningInput($input.token)
}
```

50.7.19 getJWSTokenType(token) → string (Sec)

Gets the token type (typ) from a JWS token header. See Chapter 38.

- token (required): JWS/JWT token string

```
{
  type: getJWSTokenType($input.token)    // "JWT"
}
```

50.7.20 getJWTAudience(token) → string (Sec)

Gets the audience (aud) claim from a JWT token. See Chapter 38.

- token (required): JWT token string

```
{
  audience: getJWTAudience($input.token)
}
```

50.7.21 getJWTClaim(token, claim) → any (Sec)

Gets a specific claim from a JWT token payload. See Chapter 38.

- token (required): JWT token string
- claim (required): claim name (e.g., “sub”, “email”, custom claims)

```
{
  email: getJWTClaim($input.token, "email"),
  role: getJWTClaim($input.token, "role")
}
```

50.7.22 getJWTClaims(token) → object (Sec)

Extracts all claims from a JWT payload WITHOUT verification. See Chapter 38.

- token (required): JWT token string

```
let claims = getJWTClaims($input.token)
{
  subject: claims.sub,
  issuer: claims.iss,
  expiry: claims.exp
}
```

50.7.23 getJWTIssuer(token) → string (Sec)

Gets the issuer (iss) claim from a JWT token. See Chapter 38.

- token (required): JWT token string

```
{
  issuer: getJWTIssuer($input.token)
}
```

50.7.24 getJWTSubject(token) → string (Sec)

Gets the subject (sub) claim from a JWT token. See Chapter 38.

- token (required): JWT token string

```
{
  subject: getJWTSubject($input.token)
}
```

50.7.25 getLogs() → array (Sys)

Retrieves all log entries that have been recorded during the current transformation.

```
info("Processing started")
let result = map($input.items, (i) -> i.name)
info("Processing complete")
{
  result: result,
  logs: getLogs()
}
```

50.7.26 getNamespaces(element) → object (XML)

Get all namespace declarations from an XML element as a prefix-to-URI map. See Chapter 22.

- `element` (required): XML UDM element

```
let ns = getNamespaces($input.Invoice)
{
  namespaces: ns,
  isSoap: hasKey(ns, "soap"),
  hasCommonBasic: hasKey(ns, "cbc")
}
```

50.7.27 getPath(url) → string (URL)

Get the path component from a URL string.

- `url` (required): URL string

```
echo '{"url": "https://api.example.com/v1/users/123"}' | utlx -e 'getPath($input.url)'
# "/v1/users/123"
```

```
{
  path: getPath($input.endpoint)
}
```

50.7.28 getPort(url) → number (URL)

Get the port number from a URL string.

- `url` (required): URL string

```
echo '{"url": "https://api.example.com:8443/v1"}' | utlx -e 'getPort($input.url)'
# 8443
```

```
{
  port: getPort($input.url)
}
```

50.7.29 `getProtocol(url) → string` (URL)

Get the protocol/scheme from a URL string.

- `url` (required): URL string

```
echo '{"url": "https://example.com"}' | utlx -e 'getProtocol($input.url)'  
# "https"
```

```
{  
  protocol: getProtocol($input.url),  
  isSecure: getProtocol($input.url) == "https"  
}
```

50.7.30 `getQuery(url) → string` (URL)

Get the query string from a URL (without the leading ?).

- `url` (required): URL string

```
echo '{"url": "https://example.com/search?q=hello&page=2"}' | utlx -e  
'getQuery($input.url)'  
# "q=hello&page=2"
```

```
{  
  queryString: getQuery($input.url)  
}
```

50.7.31 `getQueryParams(url) → object` (URL)

Get query parameters from a URL as a key-value object.

- `url` (required): URL string

```
echo '{"url": "https://example.com/search?q=hello&page=2"}' | utlx -e  
'getQueryParams($input.url)'  
# {"q": "hello", "page": "2"}
```

```
{  
  params: getQueryParams($input.url),  
  searchTerm: getQueryParams($input.url).q  
}
```

50.7.32 `getTimezone(datetime) → string` (Date)

Get the timezone offset string from a datetime value.

- `datetime` (required): datetime with timezone

```
{  
  offset: getTimezone(now())           // "+02:00"  
}
```

50.7.33 `getTimezoneName() → string` (Date)

Get the timezone name/ID for the current system.


```
{
  tz: getTimezoneName()           // "Europe/Amsterdam"
}
```

50.7.34 `getTimezoneOffsetHours(datetime, timezone?) → number` (Date)

Get the timezone offset in hours for a given datetime or timezone.

- `datetime` (required): datetime value
- `timezone` (optional): timezone ID

```
{
  offsetHours: getTimezoneOffsetHours(now(), "America/New_York") // -4 or -5
}
```

50.7.35 `getTimezoneOffsetSeconds(datetime) → number` (Date)

Get the timezone offset in seconds for a datetime value.

- `datetime` (required): datetime with timezone

```
{
  offsetSecs: getTimezoneOffsetSeconds(now()) // 7200 for +02:00
}
```

50.7.36 `getType(value) → string` (Type)

Returns the type of a value as a string (“string”, “number”, “boolean”, “array”, “object”, “null”, “date”, “datetime”).

- `value` (required): the value to inspect

```
echo '{"name": "Alice", "age": 30}' | utlx -e 'getType($input.name)'
# "string"
```

```
{
  nameType: getType($input.name),           // "string"
  ageType:  getType($input.age),            // "number"
  itemsType: getType($input.items)         // "array"
}
```

50.7.37 `getUuidVersion(uuid) → number` (Sec)

Get the version number from a UUID string. See Chapter 38.

- `uuid` (required): UUID string

```
echo '{"id": "550e8400-e29b-41d4-a716-446655440000"}' | utlx -e
'getUuidVersion($input.id)'
# 4
```

```
{
  version: getUuidVersion($input.correlationId)
}
```

50.7.38 goldenRatio() → number (Num)

Returns the golden ratio (approximately 1.61803398874989484820).

```
goldenRatio() // 1.618033988749895
{
  ratio: goldenRatio(),
  scaled: $input.width * goldenRatio()
}
```

50.7.39 groupBy(array, keyFn) → object (Arr)

Group array elements by a computed key. Returns an object where keys are the group values and values are arrays of matching elements.

- **array** (required): the array to group
- **keyFn** (required): `lambda (element) -> groupKey`

```
echo '{"employees": [{ "name": "Alice", "dept": "Eng"}, { "name": "Bob", "dept": "Sales"}]}' \
| utlx -e 'groupBy($input.employees, (e) -> e.dept)'
# {"Eng": [{ "name": "Alice", "dept": "Eng"}], "Sales": [{ "name": "Bob", "dept": "Sales"}]}
```

```
let groups = groupBy($input.orders, (o) -> o.status)
{
  byStatus: groups,
  summary: entries(groups) |> map((entry) -> {
    status: entry[0],
    count: count(entry[1]),
    total: sum(map(entry[1], (o) -> o.amount))
  })
}
```

50.7.40 gunzip(data) → binary (Bin)

Decompress gzip-compressed binary data.

- **data** (required): gzip-compressed binary

```
{
  decompressed: gunzip($input.compressedPayload)
}
```

50.7.41 gzip(data) → binary (Bin)

Compress binary data using gzip.

- **data** (required): binary data to compress

```
{
  compressed: gzip(toBinary($input.largeText)),
  encoded: base64Encode(gzip(toBinary($input.payload)))
}
```

50.8 H

50.8.1 `hasAlpha(string) → boolean (Str)`

Returns true if the string contains at least one alphabetic character.

- `string` (required): string to check

```
echo '{"code": "ABC123"}' | utlx -e 'hasAlpha($input.code)'
# true
```

```
{
  hasLetters: hasAlpha($input.value),
  numericOnly: !hasAlpha($input.code)
}
```

50.8.2 `hasAttribute(element, name) → boolean (XML)`

Check if an XML element has a specific attribute. See Chapter 22.

- `element` (required): XML UDM element
- `name` (required): attribute name to check

```
{
  hasId: hasAttribute($input.element, "id"),
  hasClass: hasAttribute($input.element, "class")
}
```

50.8.3 `hasBOM(data) → boolean (XML)`

Check if binary data starts with a BOM (Byte Order Mark). See Chapter 22.

- `data` (required): binary data to check

```
{
  hasBom: hasBOM($input.fileData),
  clean: if (hasBOM($input.fileData)) removeBOM($input.fileData) else $input.fileData
}
```

50.8.4 `hasContent(element) → boolean (XML)`

Check if an XML element has any content (child elements or text). See Chapter 22.

- `element` (required): XML UDM element

```
{
  hasBody: hasContent($input.element),
  emptyNodes: filter($input.nodes, (n) -> !hasContent(n))
}
```

50.8.5 `hasEnv(name) → boolean (Sys)`

Check if an environment variable exists (is set, even if empty).

- `name` (required): environment variable name

```
{
  hasDatabase: hasEnv("DATABASE_URL"),
  hasSecret: hasEnv("API_SECRET")
}
```

Security note: can be restricted in UTLXe via security policy (Chapter 38) to prevent probing for host secrets.

50.8.6 `hasKey(object, key) → boolean (Obj)`

Check if an object has a property with the given key name.

- **object** (required): the object to check
- **key** (required): property name as string

```
echo '{"name": "Alice", "email": "alice@example.com"}' | utlx -e 'hasKey($input, "email")'
# true
```

```
if (hasKey($input, "shippingAddress")) {
  address: $input.shippingAddress
} else {
  address: $input.billingAddress
}
```

Also: `containsValue(object, value)` — check if any property has a specific value.

50.8.7 `containsKey(object, key) → boolean (Obj)`

Alias for `hasKey()`. Check if an object has a property with the given key name.

- **object** (required): the object to check
- **key** (required): property name as string

```
containsKey($input, "email")           // true
containsKey($input, "phone")           // false
{
  hasEmail: containsKey($input, "email")
}
```

50.8.8 `hash(data, algorithm?) → string (Sec)`

Compute a cryptographic hash with an explicit algorithm. Returns hex-encoded digest string. See Chapter 38.

- **data** (required): string to hash
- **algorithm** (optional, default "SHA-256"): algorithm name ("MD5", "SHA-1", "SHA-256", "SHA-384", "SHA-512", "SHA3-256", "SHA3-512")

```
hash("hello", "SHA3-256")              // "3338be694f50c5f338..."
hash("hello", "SHA-256")               // "2cf24dba5fb0a30e..."
{
  digest: hash($input.payload, "SHA-256")
}
```

Also: `sha1(data)`, `sha224(data)`, `sha384(data)`, `sha3_256(data)`, `sha3_512(data)`.

50.8.9 md5(data) → string (Sec)

Compute an MD5 hash. Returns hex-encoded digest string. See Chapter 38.

- **data** (required): string to hash

```
md5("hello")                // "5d41402abc4b2a76b9719d911017c592"
{
  checksum: md5($input.content)
}
```

Anti-pattern: `md5()` for security — MD5 is cryptographically broken. Use `sha256()` minimum. MD5 is acceptable only for non-security checksums (file deduplication, cache keys).

50.8.10 sha256(data) → string (Sec)

Compute a SHA-256 hash. Returns hex-encoded digest string. See Chapter 38.

- **data** (required): string to hash

```
sha256("hello")             // "2cf24dba5fb0a30e..."

// Use case: content-addressed caching
let contentHash = sha256(renderJson($input))
{...$input, hash: contentHash}
```

50.8.11 sha512(data) → string (Sec)

Compute a SHA-512 hash. Returns hex-encoded digest string. See Chapter 38.

- **data** (required): string to hash

```
sha512("hello")             // "9b71d224bd62f378..."
{
  digest: sha512($input.document)
}
```

50.8.12 hasNamespace(element, uri) → boolean (XML)

Check if an XML element has a specific namespace URI declared. See Chapter 22.

- **element** (required): XML UDM element
- **uri** (required): namespace URI to check

```
{
  hasSoap: hasNamespace($input.root, "http://schemas.xmlsoap.org/soap/envelope/"),
  hasUBL: hasNamespace($input.Invoice,
    "urn:oasis:names:specification:ubl:schema:xsd:Invoice-2")
}
```

50.8.13 hasNumeric(string) → boolean (Str)

Returns true if the string contains at least one numeric digit.

- **string** (required): string to check

```
echo '{"code": "ABC123"}' | utlx -e 'hasNumeric($input.code)'
# true
```

```
{
  hasDigits: hasNumeric($input.password),
  alphaOnly: !hasNumeric($input.name)
}
```

50.8.14 `hexDecode(hex) → binary (Bin)`

Decode a hex-encoded string to binary data.

- `hex` (required): hex string to decode

```
{
  data: hexDecode($input.hexPayload)
}
```

50.8.15 `hexEncode(data) → string (Bin)`

Encode binary data as a hexadecimal string.

- `data` (required): binary data to encode

```
{
  hex: hexEncode(toBinary($input.text))
}
```

50.8.16 `hmac(data, key, algorithm) → string (Sec)`

Compute an HMAC (Hash-based Message Authentication Code) with an explicit algorithm. Returns hex-encoded string. See Chapter 38.

- `data` (required): the message to authenticate
- `key` (required): the secret key
- `algorithm` (required): hash algorithm (e.g., "SHA-256", "SHA-512")

```
hmac("message", "key", "SHA-512") // "... " (HMAC-SHA512 hex)
{
  signature: hmac($input.body, env("SECRET"), "SHA-256")
}
```

Also: `hmacSHA512(data, key)`, `hmacSHA1(data, key)`, `hmacMD5(data, key)`, `hmacBase64(data, key, algorithm)` (returns Base64 instead of hex).

50.8.17 `hmacSHA256(data, key) → string (Sec)`

Compute an HMAC-SHA256 for verifying message integrity and authenticity. Returns hex-encoded string. See Chapter 38.

- `data` (required): the message to authenticate
- `key` (required): the secret key

```
hmacSHA256("message-to-verify", "my-secret-key") // "4a8f3d..."

// Use case: verify webhook signature
let expectedSig = hmacSHA256($input.body, env("WEBHOOK_SECRET"))
if (expectedSig != $input.headers.signature) error("Invalid signature")
{ verified: true, payload: $input.body }
```

50.8.18 hmacBase64(data, key, algorithm) → string (Sec)

Computes HMAC and returns the result as Base64-encoded string. See Chapter 38.

- **data** (required): the message to authenticate
- **key** (required): the secret key
- **algorithm** (required): hash algorithm (e.g., “SHA-256”)

```
{  
  signature: hmacBase64($input.payload, env("SECRET"), "SHA-256")  
}
```

50.8.19 hmacMD5(data, key) → string (Sec)

Computes HMAC-MD5 hash. Returns hex-encoded string. See Chapter 38.

- **data** (required): the message
- **key** (required): the secret key

```
{  
  hash: hmacMD5($input.message, $input.key)  
}
```

50.8.20 hmacSHA1(data, key) → string (Sec)

Computes HMAC-SHA1 hash. Returns hex-encoded string. See Chapter 38.

- **data** (required): the message
- **key** (required): the secret key

```
{  
  hash: hmacSHA1($input.message, $input.key)  
}
```

50.8.21 hmacSHA384(data, key) → string (Sec)

Computes HMAC-SHA384 hash. Returns hex-encoded string. See Chapter 38.

- **data** (required): the message
- **key** (required): the secret key

```
{  
  hash: hmacSHA384($input.message, $input.key)  
}
```

50.8.22 hmacSHA512(data, key) → string (Sec)

Computes HMAC-SHA512 hash. Returns hex-encoded string. See Chapter 38.

- **data** (required): the message
- **key** (required): the secret key

```
{  
  hash: hmacSHA512($input.message, $input.key)  
}
```

50.8.23 homeDir() → string (Sys)

Get the current user's home directory path.

```
homeDir()                // "/Users/alice" or "/home/alice"
{
  home: homeDir()
}
```

50.8.24 hours(datetime) → number (Date)

Extract the hours component (0-23) from a datetime value.

- **datetime** (required): datetime value

```
echo '{"ts": "2026-05-01T14:30:00Z"}' | utlx -e 'hours(parseDate($input.ts, "yyyy-MM-
dd\\''T\\''HH:mm:ss\\''Z\\''))'
# 14
```

```
{
  hour: hours(now()),
  isBusinessHours: hours(now()) >= 9 && hours(now()) < 17
}
```

50.9 I**50.9.1 ifThenElse(condition, thenValue, elseValue) → any (Type)**

Inline if-then-else conditional expression. Returns **thenValue** if condition is true, otherwise **elseValue**.

- **condition** (required): boolean condition
- **thenValue** (required): value if true
- **elseValue** (required): value if false

```
echo '{"age": 20}' | utlx -e 'ifThenElse($input.age >= 18, "adult", "minor")'
# "adult"
```

```
{
  status: ifThenElse($input.active, "ACTIVE", "INACTIVE"),
  label: ifThenElse($input.count > 0, concat(toString($input.count), " items"), "empty")
}
```

50.9.2 implies(a, b) → boolean (Type)

Logical implication (material conditional). Returns **false** only when **a** is true and **b** is false.

- **a** (required): antecedent boolean
- **b** (required): consequent boolean

```
implies(true, true)      // true
implies(true, false)     // false
implies(false, true)     // true
implies(false, false)    // true

{
```



```
valid: implies($input.isPremium, $input.hasSubscription)
}
```

50.9.3 includes(array, value) → boolean (Arr)

Check if an array contains a specific value (strict equality).

- **array** (required): array to search
- **value** (required): value to find

```
echo '{"tags": ["urgent", "billing", "support"]}' | utlx -e 'includes($input.tags,
"urgent")'
# true
```

```
{
  isUrgent: includes($input.tags, "urgent"),
  isVIP: includes($input.roles, "VIP")
}
```

50.9.4 indexOf(array, value) → number (Arr)

Find the 0-based position of the FIRST occurrence of a value in an array. Returns `-1` if not found.

- **array** (required): array to search in
- **value** (required): value to find

```
echo '{"items": ["Apple", "Banana", "Cherry"]}' \
| utlx -e 'indexOf($input.items, "Banana")'
# 1 (0-based: Apple=0, Banana=1, Cherry=2)
```

```
{
  pos: indexOf($input.items, "Banana"), // 1 (0-based index)
  missing: indexOf($input.items, "Grape") // -1 (not found)
}
```

50.9.5 inflate(data) → binary (Bin)

Decompress Deflate-compressed binary data.

- **data** (required): deflate-compressed binary

```
{
  decompressed: inflate($input.compressedData)
}
```

50.9.6 info(message) → null (Sys)

Log a message at INFO level. Returns null (passthrough for pipeline usage).

- **message** (required): message to log

```
info("Starting transformation")
let result = map($input.items, (i) -> i.name)
info(concat("Processed ", toString(count(result)), " items"))
```

```
{
  items: result
}
// Logs: "Starting transformation", "Processed 2 items"
// Output: {"items": ["A", "B"]}
```

50.9.7 insertAfter(array, index, element) → array (Arr)

Insert an element after the specified index in an array.

- **array** (required): source array
- **index** (required): position after which to insert
- **element** (required): element to insert

```
echo '{"items": ["a", "b", "d"]}' | utlx -e 'insertAfter($input.items, 1, "c")'
# ["a", "b", "c", "d"]
```

```
{
  updated: insertAfter($input.steps, 2, {name: "validation", order: 3})
}
```

50.9.8 insertBefore(array, index, element) → array (Arr)

Insert an element before the specified index in an array.

- **array** (required): source array
- **index** (required): position before which to insert
- **element** (required): element to insert

```
echo '{"items": ["a", "c", "d"]}' | utlx -e 'insertBefore($input.items, 1, "b")'
# ["a", "b", "c", "d"]
```

```
{
  updated: insertBefore($input.items, 0, "first")
}
```

50.9.9 invert(object) → object (Obj)

Invert an object by swapping keys and values. Values become keys and keys become values.

- **object** (required): object to invert

```
echo '{"US": "United States", "GB": "United Kingdom"}' | utlx -e 'invert($input)'
# {"United States": "US", "United Kingdom": "GB"}
```

```
{
  countryByName: invert($input.codeToName)
}
```

50.9.10 iqr(numbers) → number (Num)

Calculate the interquartile range ($IQR = Q3 - Q1$) of a numeric array.

- **numbers** (required): array of numbers

```
echo '{"data": [1, 3, 5, 7, 9, 11, 13]}' | utlx -e 'iqr($input.data)'
# 8
```

```
{
  spread: iqr($input.values),
  outlierThreshold: iqr($input.values) * 1.5
}
```

50.9.11 isAfter(date1, date2) → boolean (Date)

Check if the first date is after the second date.

- **date1** (required): date to compare
- **date2** (required): reference date

```
{
  isOverdue: isAfter(now(), parseDate($input.dueDate, "yyyy-MM-dd")),
  isExpired: isAfter(now(), $input.expiresAt)
}
```

50.9.12 isAlpha(string) → boolean (Str)

Returns true if the string contains only alphabetic characters (A-Z, a-z, Unicode letters).

- **string** (required): string to test

```
echo '{"name": "Alice"}' | utlx -e 'isAlpha($input.name)'
# true
```

```
{
  validName: isAlpha($input.firstName),
  hasSpecialChars: !isAlpha($input.input)
}
```

50.9.13 isAlphanumeric(string) → boolean (Str)

Returns true if the string contains only alphanumeric characters (A-Z, a-z, 0-9, Unicode letters).

- **string** (required): string to test

```
isAlphanumeric("Hello123")           // true
isAlphanumeric("Hello 123")           // false (contains space)
{
  validCode: isAlphanumeric($input.productCode)
}
```

50.9.14 isAscii(string) → boolean (Str)

Returns true if the string contains only ASCII characters (code points 0-127).

- **string** (required): string to test

```
isAscii("Hello!")                     // true
isAscii("café\u0301")                 // false (contains accent)
```

```
{
  asciiSafe: isAscii($input.text)
}
```

50.9.15 isBefore(date1, date2) → boolean (Date)

Check if the first date is before the second date.

- **date1** (required): date to compare
- **date2** (required): reference date

```
{
  notYetDue: isBefore(now(), parseDate($input.dueDate, "yyyy-MM-dd")),
  isPast: isBefore($input.eventDate, now())
}
```

50.9.16 isBetween(date, start, end) → boolean (Date)

Check if a date is between two other dates (inclusive).

- **date** (required): date to test
- **start** (required): range start date
- **end** (required): range end date

```
{
  inRange: isBetween(now(), $input.startDate, $input.endDate),
  inQ1: isBetween($input.date, parseDate("2026-01-01", "yyyy-MM-dd"),
    parseDate("2026-03-31", "yyyy-MM-dd"))
}
```

50.9.17 isCanonicalJSON(json) → boolean (JSON)

Validates that a JSON string is in canonical form per RFC 8785 (JSON Canonicalization Scheme). See Chapter 24.

- **json** (required): JSON string to validate

```
echo '{"json": "{\\"a\\":1,\\"b\\":2}\\"}' | utlx -e 'isCanonicalJSON($input.json)'
# true
```

```
{
  isCanonical: isCanonicalJSON(renderJson($input))
}
```

50.9.18 isCDATA(text) → boolean (XML)

Checks if a string is a CDATA section (wrapped in <![CDATA[...]]>). See Chapter 22.

- **text** (required): string to check

```
{
  isCdata: isCDATA($input.xmlField),
  content: if (isCDATA($input.field)) extractCDATA($input.field) else $input.field
}
```

50.9.19 isDebugMode() → **boolean** (Sys)

Check if the current transformation is running in debug mode.

```
{
  debug: isDebugMode(),
  extra: if (isDebugMode()) { logs: getLogs() } else null
}
```

50.9.20 isEmptyElement(element) → **boolean** (XML)

Check if an XML element is empty (no children, no text content). See Chapter 22.

- **element** (required): XML UDM element

```
{
  isEmpty: isEmptyElement($input.node),
  nonEmpty: filter($input.elements, (e) -> !isEmptyElement(e))
}
```

50.9.21 isGzipped(data) → **boolean** (Bin)

Check if binary data is gzip-compressed (by checking magic bytes).

- **data** (required): binary data to check

```
{
  compressed: isGzipped($input.payload),
  content: if (isGzipped($input.data)) gunzip($input.data) else $input.data
}
```

50.9.22 isHexadecimal(string) → **boolean** (Str)

Returns true if the string contains only valid hexadecimal characters (0-9, A-F, a-f).

- **string** (required): string to test

```
isHexadecimal("1a2b3c")           // true
isHexadecimal("xyz")               // false
{
  validHex: isHexadecimal($input.colorCode)
}
```

50.9.23 isJarFile(data) → **boolean** (Bin)

Check if binary data is a JAR file.

- **data** (required): binary data to check

```
{
  isJar: isJarFile($input.fileData)
}
```

50.9.24 isJWSFormat(token) → **boolean** (Sec)

Checks if a string is in valid JWS format (three Base64URL segments separated by dots). See Chapter 38.

- **token** (required): string to validate

```
{
  validFormat: isJWSFormat($input.token),
  canDecode: isJWSFormat($input.authHeader)
}
```

50.9.25 isJWTExpired(token) → boolean (Sec)

Checks if a JWT is expired based on its exp claim. See Chapter 38.

- token (required): JWT token string

```
{
  expired: isJWTExpired($input.token),
  needsRefresh: isJWTExpired($input.accessToken)
}
```

50.9.26 isLeapYearFunc(year) → boolean (Date)

Check if a year is a leap year.

- year (required): year number

```
isLeapYearFunc(2024)           // true
isLeapYearFunc(2026)           // false
{
  leap: isLeapYearFunc(year(now()))
}
```

50.9.27 isLocalDateTime(value) → boolean (Date)

Check if a value is a local datetime (datetime without timezone information).

- value (required): value to test

```
{
  isLocal: isLocalDateTime($input.timestamp)
}
```

50.9.28 isLowerCase(string) → boolean (Str)

Returns true if all alphabetic characters in the string are lowercase. Non-alphabetic characters are ignored.

- string (required): string to test

```
isLowerCase("hello")           // true
isLowerCase("Hello")           // false
isLowerCase("hello123")        // true (digits ignored)
{
  isLower: isLowerCase($input.code)
}
```

50.9.29 isEmpty(value) → boolean (Type)

Checks if a value is not empty (inverse of isEmpty). Returns true for non-null, non-empty values.

- value (required): value to test

```
isEmpty("hello")           // true
isEmpty("")                // false
isEmpty(null)              // false
isEmpty([1, 2])            // true
{
  hasName: isEmpty($input.name),
  hasItems: isEmpty($input.items)
}
```

50.9.30 isNumeric(string) → boolean (Str)

Returns true if the string contains only numeric digits (0-9).

- **string** (required): string to test

```
echo '{"zip": "12345"}' | utlx -e 'isNumeric($input.zip)'
# true
```

```
{
  validZip: isNumeric($input.zipCode),
  numericId: isNumeric($input.id)
}
```

50.9.31 isPlural(word) → boolean (Str)

Checks if a word is in plural form.

- **word** (required): word to check

```
isPlural("cats")           // true
isPlural("cat")            // false
isPlural("children")       // true
```

50.9.32 isPointInCircle(lat, lon, centerLat, centerLon, radiusKm) → boolean (Geo)

Checks if a point is within a circular radius from a center point.

- **lat** (required): point latitude
- **lon** (required): point longitude
- **centerLat** (required): circle center latitude
- **centerLon** (required): circle center longitude
- **radiusKm** (required): radius in kilometers

```
{
  inRange: isPointInCircle($input.lat, $input.lon, 52.3676, 4.9041, 10),
  nearStore: isPointInCircle($input.userLat, $input.userLon, $input.storeLat,
    $input.storeLon, 5)
}
```

50.9.33 isPointInPolygon(lat, lon, polygon) → boolean (Geo)

Checks if a point is inside a polygon using the ray casting algorithm.

- **lat** (required): point latitude
- **lon** (required): point longitude
- **polygon** (required): array of [lat, lon] coordinate pairs

```
let zone = [[52.0, 4.0], [52.5, 4.0], [52.5, 5.0], [52.0, 5.0]]
{
  inZone: isPointInPolygon($input.lat, $input.lon, zone)
}
```

50.9.34 isPrintable(string) → boolean (Str)

Returns true if the string contains only printable characters (letters, digits, punctuation, spaces).

- **string** (required): string to test

```
isPrintable("Hello, World!")           // true
isPrintable("Hello\x00World")          // false (contains null byte)
{
  safe: isPrintable($input.userInput)
}
```

50.9.35 isSameDay(date1, date2) → boolean (Date)

Check if two dates fall on the same calendar day (ignoring time).

- **date1** (required): first date
- **date2** (required): second date

```
{
  sameDay: isSameDay($input.created, $input.modified),
  createdToday: isSameDay($input.created, now())
}
```

50.9.36 isSingular(word) → boolean (Str)

Checks if a word is in singular form.

- **word** (required): word to check

```
isSingular("cat")           // true
isSingular("cats")          // false
isSingular("child")         // true
```

50.9.37 isToday(date) → boolean (Date)

Check if a date is today.

- **date** (required): date to check

```
{
  isNew: isToday($input.createdAt),
  todayOrders: filter($input.orders, (o) -> isToday(o.date))
}
```

50.9.38 isUpperCase(string) → boolean (Str)

Returns true if all alphabetic characters in the string are uppercase. Non-alphabetic characters are ignored.

- **string** (required): string to test


```
isUpperCase("HELLO")           // true
isUpperCase("Hello")           // false
isUpperCase("ABC123")           // true (digits ignored)
{
  isUpper: isUpperCase($input.countryCode)
}
```

50.9.39 isUuidV7(uuid) → boolean (Sec)

Check if a UUID string is specifically version 7 (time-ordered). See Chapter 38.

- `uuid` (required): UUID string to check

```
{
  isV7: isUuidV7($input.messageId),
  canExtractTime: isUuidV7($input.id)
}
```

50.9.40 isValidAmount(amount) → boolean (Fin)

Validates if an amount is within acceptable range (finite, not NaN).

- `amount` (required): numeric value to validate

```
{
  valid: isValidAmount($input.total),
  canProcess: isValidAmount($input.payment) && $input.payment > 0
}
```

50.9.41 isValidCoordinates(lat, lon) → boolean (Geo)

Checks if coordinates are valid (latitude −90 to 90, longitude −180 to 180).

- `lat` (required): latitude value
- `lon` (required): longitude value

```
{
  valid: isValidCoordinates($input.lat, $input.lon)
}
```

50.9.42 isValidCurrency(code) → boolean (Fin)

Validates if a currency code is valid according to ISO 4217.

- `code` (required): currency code string (e.g., “USD”, “EUR”)

```
echo '{"currency": "USD"}' | utlx -e 'isValidCurrency($input.currency)'
# true
```

```
{
  valid: isValidCurrency($input.currencyCode),
  error: if (!isValidCurrency($input.currency)) "Invalid currency" else null
}
```

50.9.43 isValidTimezone(timezone) → boolean (Date)

Check if a timezone ID string is valid.

- `timezone` (required): timezone ID to validate

```
isValidTimezone("America/New_York")    // true
isValidTimezone("Invalid/Zone")        // false
{
  valid: isValidTimezone($input.userTimezone)
}
```

50.9.44 isValidURL(url) → boolean (URL)

Validate if a string is a well-formed URL.

- `url` (required): string to validate

```
echo '{"url": "https://example.com/path"}' | utlx -e 'isValidURL($input.url)'
# true
```

```
{
  valid: isValidURL($input.website),
  links: filter($input.urls, (u) -> isValidURL(u))
}
```

50.9.45 isValidUuid(uuid) → boolean (Sec)

Validate if a string is a properly formatted UUID (any version). See Chapter 38.

- `uuid` (required): string to validate

```
echo '{"id": "550e8400-e29b-41d4-a716-446655440000"}' | utlx -e 'isValidUuid($input.id)'
# true
```

```
{
  valid: isValidUuid($input.correlationId),
  error: if (!isValidUuid($input.id)) "Invalid UUID format" else null
}
```

50.9.46 isWhitespace(string) → boolean (Str)

Returns true if the string contains only whitespace characters (spaces, tabs, newlines).

- `string` (required): string to test

```
isWhitespace(" ")          // true
isWhitespace(" \t\n ")     // true
isWhitespace(" a ")        // false
{
  blank: isWhitespace($input.field)
}
```

50.9.47 isZipArchive(data) → boolean (Bin)

Check if binary data is a zip archive (by checking magic bytes).

- **data** (required): binary data to check

```
{
  isZip: isZipArchive($input.fileData),
  entries: if (isZipArchive($input.data)) listZipEntries($input.data) else []
}
```

50.9.48 **lastIndexOf(haystack, needle) → number** (Str/Arr)

Find the position of the LAST occurrence of a value. Returns `-1` if not found.

- **haystack** (required): string or array to search in
- **needle** (required): value to find

```
lastIndexOf("abcabc", "bc")           // 4 (last occurrence, not first at 1)
lastIndexOf(["A", "B", "A", "C"], "A") // 2 (last A)
lastIndexOf("hello", "xyz")           // -1
```

Also: `findIndex(array, predicate)` — find by condition instead of value. `findLastIndex(array, predicate)` — last match by condition.

50.9.49 **isArray(value) → boolean** (Type)

Returns true if the value is an array.

- **value** (required): the value to test

```
isArray([1, 2, 3])           // true
isArray("hello")             // false
isArray($input.items)        // true if items is an array

// Use case: handle XML single-vs-array ambiguity
let items = if (isArray($input.Item)) $input.Item else [$input.Item]
// Ensures items is always an array, even when XML has a single <Item>
```

50.9.50 **isBoolean(value) → boolean** (Type)

Returns true if the value is a boolean (`true` or `false`).

- **value** (required): the value to test

```
isBoolean(true)              // true
isBoolean(false)             // true
isBoolean("true")            // false (string, not boolean)
isBoolean(1)                 // false (number, not boolean)
```

50.9.51 **isNull(value) → boolean** (Type)

Returns true if the value is null.

- **value** (required): the value to test

```
isNull(null)                 // true
isNull("")                   // false (empty string is not null)
isNull(0)                    // false (zero is not null)
isNull($input.optionalField) // true if field is missing or null
```

50.9.52 isNumber(value) → boolean (Type)

Returns true if the value is a number (integer or decimal).

- **value** (required): the value to test

```
isNumber(42)           // true
isNumber(3.14)         // true
isNumber("42")         // false (string, not number)
isNumber(true)         // false
```

50.9.53 isObject(value) → boolean (Type)

Returns true if the value is an object (key-value map).

- **value** (required): the value to test

```
isObject({name: "Alice"}) // true
isObject([1, 2, 3])       // false (array, not object)
isObject("hello")         // false
isObject($input)          // true (root is typically an object)
```

50.9.54 isString(value) → boolean (Type)

Returns true if the value is a string.

- **value** (required): the value to test

```
isString("hello")       // true
isString(42)            // false
isString(null)          // false
```

50.9.55 isDate(value) → boolean (Type)

Returns true if the value is a date (not a string representation of a date).

- **value** (required): the value to test

```
isDate(parseDate("2026-05-01", "yyyy-MM-dd")) // true
isDate("2026-05-01")                          // false (string, not date)
```

50.9.56 isDateTime(value) → boolean (Type)

Returns true if the value is a datetime.

- **value** (required): the value to test

```
isDateTime(now())       // true
isDateTime("2026-05-01T14:30:00Z") // false (string, not datetime)
```

50.9.57 isTime(value) → boolean (Type)

Returns true if the value is a time value.

- **value** (required): the value to test

```
isTime(parseTime("14:30:00", "HH:mm:ss")) // true
isTime("14:30:00")                       // false (string, not time)
```

50.9.58 isBlank(value) → boolean (Type)

Returns true if the value is null, empty string, or whitespace-only.

- **value** (required): the value to test

```
isBlank(null)           // true
isBlank("")             // true
isBlank(" ")            // true (whitespace-only)
isBlank("hello")        // false
```

50.9.59 isEmpty(value) → boolean (Type)

Returns true if the value is null, empty string, or empty array.

- **value** (required): the value to test

```
isEmpty(null)           // true
isEmpty("")             // true
isEmpty([])             // true (empty array)
isEmpty("hello")        // false
isEmpty([1])            // false
```

50.9.60 isDefined(value) → boolean (Type)

Returns true if the value is not null. Empty strings and zero are considered defined.

- **value** (required): the value to test

```
isDefined(null)         // false
isDefined("")           // true (empty string IS defined)
isDefined(0)            // true (zero IS defined)
isDefined($input.name)  // true if field exists and is not null
```

50.9.61 isLeapYear(year) → boolean (Date)

Returns true if the given year is a leap year.

- **year** (required): year number

```
isLeapYear(2024)        // true (divisible by 4, not by 100, or by 400)
isLeapYear(2026)        // false
```

50.9.62 isWeekday(date) → boolean (Date)

Returns true if the date falls on a weekday (Monday through Friday).

- **date** (required): date or datetime

```
isWeekday(parseDate("2026-05-01", "yyyy-MM-dd")) // true (Thursday)

// Use case: calculate business days
let startDate = parseDate($input.start, "yyyy-MM-dd")
let workDays = filter(
  map(range(0, 30), (i) -> addDays(startDate, i)),
  (d) -> isWeekday(d)
)
{ businessDays: count(workDays) }
```

50.9.63 isWeekend(date) → boolean (Date)

Returns true if the date falls on a weekend (Saturday or Sunday).

- **date** (required): date or datetime

```
isWeekend(parseDate("2026-05-03", "yyyy-MM-dd")) // true (Sunday)
```

50.10 J**50.10.1 javaVersion() → string (Sys)**

Get the Java/JVM version string of the runtime.

```
{
  runtime: javaVersion()           // "21.0.2"
}
```

50.10.2 jcs(json) → string (JSON)

JSON Canonicalization Scheme (RFC 8785). Produces deterministic JSON — identical output regardless of key order or whitespace. See Chapter 24.

- **json** (required): JSON UDM value to canonicalize

```
echo '{"z": 3, "a": 1, "m": 2}' | utlx -e 'jcs($input)'
# {"a":1,"m":2,"z":3} (keys sorted, no whitespace)
```

Also: `canonicalJSONHash(json, algorithm?)` (hash the canonical form), `canonicalJSONSize(json)` (byte size), `isCanonicalJSON(string)`.

50.10.3 canonicalizeJSON(json) → string (JSON)

Alias for `jcs()`. Produces deterministic JSON using RFC 8785.

- **json** (required): JSON UDM value to canonicalize

```
echo '{"z": 3, "a": 1, "m": 2}' | utlx -e 'canonicalizeJSON($input)'
# {"a":1,"m":2,"z":3} (keys sorted, no whitespace)
```

50.10.4 jsonEquals(json1, json2) → boolean (JSON)

Compare two JSON values semantically, ignoring key order and whitespace. See Chapter 24.

- **json1** (required): first value to compare
- **json2** (required): second value to compare

```
{
  same: jsonEquals({b: 2, a: 1}, {a: 1, b: 2}), // true (same content, different key
  order)
  diff: jsonEquals({a: 1}, {a: 2})             // false
}
```

```
// Use case: detect changes between two API responses. See Chapter 24.
if (!jsonEquals($input.previous, $input.current)) {
  changed: true,
```

```
hash: canonicalJSONHash($input.current)
}
```

50.10.5 join(array, separator) → string (Str)

Join array elements into a single string with a separator. This is the **string** join — for data restructuring (nesting children under parents), see `nestBy()` in Chapter 20.

- **array** (required): array of values to join (non-strings are converted automatically)
- **separator** (required): string to insert between elements

```
echo '{"tags": ["urgent", "billing", "review"]}' | utlx -e 'join($input.tags, ", ")'
# urgent, billing, review
```

```
{
  label: join($input.tags, " | "),
  csvLine: join([$input.name, toString($input.age), $input.city], ";"),
  path: join(["usr", "local", "bin"], "/")
}
```

Anti-pattern: `reduce(arr, "", (acc, x) -> concat(acc, x, ", "))` — creates N intermediate strings. `join()` builds the result in one pass.

50.10.6 joinToString(array, separator?) → string (Str)

Join array elements into a string with optional separator (defaults to ", "). Alias-style alternative to `join()`.

- **array** (required): array of values to join
- **separator** (optional): delimiter string, defaults to ", "

```
echo '["a", "b", "c"]' | utlx -e 'joinToString($input, " - ")'
# a - b - c
```

50.10.7 joinWith(left, right, leftKeyFn, rightKeyFn, combinerFn) → array (Arr)

Inner join two arrays by key with a custom combiner function. Unlike `join()` which returns `{l, r}` pairs, `joinWith` lets you shape the output.

- **left** (required): left array
- **right** (required): right array
- **leftKeyFn** (required): lambda extracting key from left items
- **rightKeyFn** (required): lambda extracting key from right items
- **combinerFn** (required): lambda (`leftItem, rightItem`) -> `result`

```
let customers = [{id: 1, name: "Alice"}, {id: 2, name: "Bob"}]
let orders = [{customerId: 1, product: "Widget"}]

{
  joined: joinWith(customers, orders,
    (c) -> c.id, (o) -> o.customerId,
    (c, o) -> {name: c.name, product: o.product}
  )
}
```

```
// [{name: "Alice", product: "Widget"}]
}
```

50.11 K

50.11.1 kebabCase(string) → string (Str)

Convert a string to kebab-case (lowercase words separated by hyphens).

- **string** (required): the string to convert

```
echo "orderLineItem" | utlx -e 'kebabCase($input)'
# order-line-item
```

```
{
  fromCamel: kebabCase("OrderLineItem"),    // "order-line-item"
  fromSnake: kebabCase("some_snake_case")    // "some-snake-case"
}
```

50.11.2 keys(object) → array (Obj)

Get all property names (keys) from an object. Key order is preserved (insertion order).

- **object** (required): the object to inspect

```
echo '{"name": "Alice", "age": 30, "city": "Amsterdam"}' | utlx -e 'keys($input)'
# ["name", "age", "city"]
```

```
// Iterate over dynamic keys
{
  servers: map(keys($input.servers), (env) -> {
    environment: env,
    host: $input.servers[env].host
  })
}
```

50.11.3 values(object) → array (Obj)

Get all property values from an object. Order matches keys().

- **object** (required): the object to inspect

```
echo '{"name": "Alice", "age": 30, "city": "Amsterdam"}' | utlx -e 'values($input)'
# ["Alice", 30, "Amsterdam"]
```

50.12 L

50.12.1 leftJoin(left, right, leftKeyFn, rightKeyFn) → array (Arr)

Left join – returns all items from the left array, with matching items from the right (or null for non-matches).

- **left** (required): left array (all items preserved)
- **right** (required): right array
- **leftKeyFn** (required): lambda extracting key from left items

- `rightKeyFn` (required): lambda extracting key from right items

```
let customers = [{id: 1, name: "Alice"}, {id: 2, name: "Bob"}]
let orders = [{customerId: 1, product: "Widget"}]

{
  result: leftJoin(customers, orders, (c) -> c.id, (o) -> o.customerId)
  // [{l: {id: 1, name: "Alice"}, r: {customerId: 1, product: "Widget"}},
  //   {l: {id: 2, name: "Bob"}, r: null}]
}
```

50.12.2 `length(value) → number` (Str/Arr)

Length of a string (character count) or array (element count). Alias for `count()` on arrays.

- `value` (required): string or array

```
{
  str: length("hello"),           // 5
  unicode: length("日本語"),     // 3 (Unicode characters, not bytes)
  arr: length([1, 2, 3]),        // 3
  emptyStr: length(""),         // 0
  emptyArr: length([])          // 0
}
```

50.12.3 `listJarEntries(jarData) → array` (Bin)

List all entries (file paths) in a JAR file.

- `jarData` (required): binary JAR data

```
{
  entries: listJarEntries($input.jar)
  // ["META-INF/MANIFEST.MF", "com/example/Main.class", ...]
}
```

50.12.4 `listZipEntries(zipData) → array` (Bin)

List all entries (file paths) in a ZIP archive.

- `zipData` (required): binary ZIP data

```
{
  files: listZipEntries($input.archive)
  // ["readme.txt", "data/orders.csv", "data/customers.csv"]
}
```

50.12.5 `ln(number) → number` (Num)

Natural logarithm (base e).

- `number` (required): positive number

```
ln(1)           // 0
ln(e())         // 1
ln(10)          // ~2.302585
```

50.12.6 `localName(element) → string (XML)`

Extract the local name (without prefix) from an XML qualified name. See Chapter 22.

- `element` (required): XML UDM element

```
{
  local: localName($input)           // "InvoiceTypeCode"
}
```

50.12.7 `namespaceUri(element) → string (XML)`

Extract the namespace URI from an XML element. See Chapter 22.

- `element` (required): XML UDM element

```
{
  ns: namespaceUri($input)           // "urn:oasis:names:specification:ubl:..."
}
```

```
// Use case: filter XML elements by namespace (XBRL taxonomy)
let usgaapFacts = filter($input.*, (elem) ->
  namespaceUri(elem) == "http://fasb.org/us-gaap/2024"
)
{
  facts: usgaapFacts
}
```

50.12.8 `namespacePrefix(element) → string (XML)`

Extract the namespace prefix from an XML element. See Chapter 22.

- `element` (required): XML UDM element

```
{
  prefix: namespacePrefix($input)    // "cbc"
}
```

50.12.9 `qualifiedName(element) → string (XML)`

Get the full qualified name (prefix:localName) of an XML element. See Chapter 22.

- `element` (required): XML UDM element

```
{
  qname: qualifiedName($input)        // "cbc:InvoiceTypeCode"
}
```

Also: `resolveQname(string, context)`, `matchesQname(element, pattern)`, `hasNamespace(element)`.

50.12.10 `log(number, base?) → number (Num)`

Logarithm with optional base. Without base, defaults to natural logarithm (base e).

- `number` (required): positive number
- `base` (optional): logarithm base (default: e)

```
log(100, 10)           // 2
log(8, 2)              // 3
log(e())               // 1 (natural log)
```

50.12.11 log10(number) → number (Num)

Base-10 logarithm.

- **number** (required): positive number

```
log10(1000)           // 3
log10(1)              // 0
```

50.12.12 log2(number) → number (Num)

Base-2 logarithm.

- **number** (required): positive number

```
log2(8)               // 3
log2(1024)            // 10
```

50.12.13 logCount(value) → value (Sys)

Log the count/length of a value to stderr and pass the value through unchanged. Useful for debugging pipelines.

- **value** (required): array, string, or object to count

```
{
  items: logCount($input.orders) // logs "count: 42" to stderr, passes array
  through
}
```

50.12.14 logPretty(value) → value (Sys)

Log a pretty-printed representation of a value to stderr and pass the value through unchanged.

- **value** (required): any value to inspect

```
{
  result: logPretty($input.payload) // logs formatted JSON to stderr, passes value
  through
}
```

50.12.15 logSize(value) → value (Sys)

Log the byte size of a value to stderr and pass the value through unchanged.

- **value** (required): any value to measure

```
{
  output: logSize($input.document) // logs "size: 4096 bytes" to stderr, passes
  value through
}
```

50.12.16 logType(value) → value (Sys)

Log the type of a value to stderr and pass the value through unchanged.

- **value** (required): any value to inspect

```
{
  data: logType($input.field)           // logs "type: string" to stderr, passes value
  through
}
```

50.12.17 lookupBy(searchValue, referenceArray, keyFn) → element | null (Arr)

Find one matching record from a reference array by key. Returns the first match or null. The go-to function for 1:1 enrichment – adding data from a lookup table to each record. See Chapter 20.

- **searchValue** (required): the value to search for (e.g., a customer ID)
- **referenceArray** (required): the array to search in (e.g., all customers)
- **keyFn** (required): lambda extracting the comparison key from each reference record

```
// Enrich order lines with product names from a product catalog
let products = $input.products           // [{sku: "W-01", name: "Widget"}, ...]

{
  orders: map($input.orders, (order) -> {
    ...order,
    productName: lookupBy(order.sku, products, (p) -> p.sku).name
  })
}
```

```
echo '{"id": "C-42", "customers": [{"id": "C-42", "name": "Acme"}]}' | utlx -e
'lookupBy($input.id, $input.customers, (c) -> c.id)'
# {"id": "C-42", "name": "Acme"}
```

Choosing the right function: `lookupBy` for 1:1 enrichment, `groupBy` for O(1) keyed map, `nestBy` for 1:N parent-child nesting.

50.12.18 lower(string) → string (Str)

Alias for `lowerCase()`. Convert a string to all lowercase.

- **string** (required): the string to convert

```
{
  result: lower("Hello World")           // "hello world"
}
```

50.12.19 lowerCase(string) → string (Str)

Convert a string to all lowercase.

- **string** (required): the string to convert

```
{
  result: lowerCase("Hello World")       // "hello world"
}
```

Also: `camelCase`, `snakeCase`, `kebabCase`, `pascalCase`, `titleCase`, `dotCase`, `pathCase`, `constantCase`, `slugify`.

50.12.20 `upperCase(string) → string (Str)`

Convert a string to all uppercase.

- `string` (required): the string to convert

```
{
  result: upperCase("Hello World"),          // "HELLO WORLD"

  // Use case: normalize API field names
  normalized: mapKeys($input, (key) -> camelCase(key))
  // {"first_name": "Alice"} -> {"firstName": "Alice"}
}
```

50.13 M

50.13.1 `map(array, fn) → array (Arr)`

Transform every element of an array. The most-used function in UTL-X. Returns a new array of the same length.

- `array` (required): the array to transform
- `fn` (required): `lambda (element) -> newValue` or `(element, index) -> newValue`

```
echo '[1, 2, 3]' | utlx -e 'map(., (x) -> x * 2)'
# [2, 4, 6]
```

```
{
  lines: map($input.items, (item) -> {
    product: item.name,
    priceWithTax: item.price * 1.21
  }),

  // With index (second parameter):
  numbered: map($input.items, (item, index) -> {
    lineNumber: index + 1,
    product: item.name
  })
}
```

Anti-pattern: using `map()` to filter — `map(arr, (x) -> if (x.active) x else null)` produces nulls. Use `filter()` to remove, then `map()` to transform.

50.13.2 `mapEntries(object, fn) → object (Obj)`

Transform both keys and values of an object.

- `object` (required): the object to transform
- `fn` (required): `lambda (key, value) -> {key: newKey, value: newValue}`

```
echo '{"first_name": "Alice", "last_name": "Johnson", "age": 30}' | utlx -e
'mapEntries($input, (key, value) -> {key: upperCase(key), value: if (isString(value))
upperCase(value) else value})'
# {"FIRST_NAME": "ALICE", "LAST_NAME": "JOHNSON", "AGE": 30}
```

50.13.3 mapKeys(object, fn) → object (Obj)

Transform the keys of an object, keeping values unchanged.

- **object** (required): the object to transform
- **fn** (required): `lambda (key) -> newKey`

```
echo '{"first_name": "Alice", "last_name": "Johnson", "age": 30}' | utlx -e
'mapKeys($input, (key) -> camelCase(key))'
# {"firstName": "Alice", "lastName": "Johnson", "age": 30}
```

50.13.4 mapValues(object, fn) → object (Obj)

Transform the values of an object, keeping keys unchanged.

- **object** (required): the object to transform
- **fn** (required): `lambda (value) -> newValue`

```
echo '{"first_name": "Alice", "last_name": "Johnson", "age": 30}' | utlx -e
'mapValues($input, (value) -> toString(value))'
# {"first_name": "Alice", "last_name": "Johnson", "age": "30"}
```

50.13.5 mapGroups(array, keySelector, transform) → array (Arr)

Group array elements by key and transform each group. Returns an array of transformed results – ideal for reporting and aggregation. See Chapter 20.

- **array** (required): input array to group
- **keySelector** (required): `lambda (item) -> key` or string property name
- **transform** (required): `lambda` receiving `{key, value}` group object

```
// Input: flat array of orders with region field
let sales = $input.orders
// [{region: "EU", amount: 100}, {region: "US", amount: 200}, ...]

{
  summary: mapGroups(sales, "region", (group) -> {
    region: group.key,
    count: count(group.value),
    total: sum(map(group.value, (o) -> o.amount))
  })
  // Output: [{region: "EU", count: 3, total: 450},
  //           {region: "US", count: 2, total: 380}]
}
```

```
echo ' [{"dept": "Eng", "name": "Alice"}, {"dept": "Eng", "name": "Bob"},
{"dept": "Sales", "name": "Carol"} ]' | utlx -e 'mapGroups($input, "dept", (g) -> {dept:
g.key, headcount: count(g.value)})'
# [{"dept": "Eng", "headcount": 2}, {"dept": "Sales", "headcount": 1}]
```

Difference from `groupBy`: `groupBy` returns an Object (keyed map for O(1) lookup); `mapGroups` returns an Array (for iteration/reporting).

50.13.6 mapTree(data, transformer) → object | array (Obj)

Recursively transform all values in a nested structure (objects and arrays) by applying a transformer function. Walks the entire tree depth-first.

- **data** (required): nested object or array to traverse
- **transformer** (required): `lambda (value, path) -> newValue`

```
// Input: {a: "hello", b: {c: "world"}}
// Uppercase all string values, no matter how deeply nested
{
  result: mapTree($input, (v, p) ->
    if (typeof(v) == "string") upperCase(v) else v
  )
  // {a: "HELLO", b: {c: "WORLD"}}
}
```

50.13.7 `mask(string, visibleChars) → string` (Sec)

Mask a string, keeping only the first N characters visible. For PII protection in logs and reports. See Chapter 38.

- **string** (required): the value to mask
- **visibleChars** (required): number of characters to leave visible

```
{
  name: mask("Alice Johnson", 3),           // "Ali***"
  card: mask("4111111111111111", 4),       // "4111*****"
  email: mask("alice@example.com", 5),      // "alice*****"
  vatId: mask("NL123456789B01", 2)         // "NL*****"
}
```

```
// Use case: audit-safe output
{
  customerName: mask($input.name, 3),
  email: sha256($input.email),              // irreversible hash
  orderId: $input.orderId                   // non-sensitive, keep as-is
}
```

50.13.8 `matches(string, regex) → boolean` (Str)

Test if a string matches a regular expression (full match).

- **string** (required): the string to test
- **regex** (required): regular expression pattern

```
{
  order: matches("ORD-001", "^ORD-[0-9]+$"), // true
  wrong: matches("INV-001", "^ORD-[0-9]+$"), // false
  email: matches("alice@example.com",
    "^[^@]+@[^@]+\\.([^@]+$)"                // true
  )
}
```

```
// Use case: validate field formats
if (!matches($input.vatId, "^[A-Z]{2}[0-9]{9}B[0-9]{2}$"))
  error("Invalid Dutch VAT ID format")
```

Also: `matchesQname(element, pattern)` for XML QName matching (Chapter 22).

50.13.9 matchesWhole(string, pattern) → boolean (Str)

Test if a string matches a pattern completely (entire string must match, not just a substring). Equivalent to anchoring with `^...$`.

- **string** (required): the string to test
- **pattern** (required): regular expression pattern

```
{
  full: matchesWhole("abc123", "[a-z]+[0-9]+"),      // true (whole string matches)
  partial: matchesWhole("abc123xyz", "[a-z]+[0-9]+") // false (trailing "xyz" prevents
full match)
}
```

50.13.10 max(array) → number (Num/Arr)

Find the maximum value in a numeric array.

- **array** (required): array of numbers

```
{
  highest: max([3, 1, 4, 1, 5, 9])      // 9
}
```

50.13.11 min(array) → number (Num/Arr)

Find the minimum value in a numeric array.

- **array** (required): array of numbers

```
{
  lowest: min([3, 1, 4, 1, 5, 9])      // 1
}
```

50.13.12 maxBy(array, fn) → element (Num/Arr)

Find the element with the maximum value of a key extractor. Returns the entire element, not just the value.

- **array** (required): array to search
- **fn** (required): `lambda (element) -> comparable`

```
echo '{"products": [{"name": "Widget", "price": 25}, {"name": "Gadget", "price": 150},
{"name": "Gizmo", "price": 10}]}' | utlx -e 'maxBy($input.products, (p) -> p.price)'
# {"name": "Gadget", "price": 150} (the ENTIRE object, not just 150)
```

50.13.13 minBy(array, fn) → element (Num/Arr)

Find the element with the minimum value of a key extractor. Returns the entire element, not just the value.

- **array** (required): array to search
- **fn** (required): `lambda (element) -> comparable`

```
echo '{"products": [{"name": "Widget", "price": 25}, {"name": "Gadget", "price": 150},
{"name": "Gizmo", "price": 10}]}' | utlx -e 'minBy($input.products, (p) -> p.price)'
# {"name": "Gizmo", "price": 10}
```


50.13.14 measure(fn) → object (Sys)

Measure execution time of an expression. Returns an object with the result and elapsed time.

- `fn` (required): `lambda () -> value to measure`

```
let m = measure(() -> map($input.items, (x) -> x * 2))
// m = {result: [...], elapsed: 12.5, unit: "ms"}

{
  data: m.result,
  timing: m.elapsed
}
```

50.13.15 median(array) → number (Num)

Compute the median (middle value) of a numeric array.

- `array` (required): array of numbers

```
echo '{"scores": [72, 85, 90, 95, 88, 76, 92]}' | utlx -e 'median($input.scores)'
# 88
```

50.13.16 mode(array) → number (Num)

Find the most frequently occurring value in a numeric array.

- `array` (required): array of numbers

```
{
  result: mode([1, 2, 2, 3, 3, 3])      // 3 (most frequent)
}
```

50.13.17 stdDev(array) → number (Num)

Compute the standard deviation of a numeric array.

- `array` (required): array of numbers

```
echo '{"scores": [72, 85, 90, 95, 88, 76, 92]}' | utlx -e 'stdDev($input.scores)'
# ~8.5
```

50.13.18 variance(array) → number (Num)

Compute the variance of a numeric array.

- `array` (required): array of numbers

```
echo '{"scores": [72, 85, 90, 95, 88, 76, 92]}' | utlx -e 'variance($input.scores)'
# ~72.2
```

50.13.19 percentile(array, p) → number (Num)

Compute the p-th percentile of a numeric array.

- `array` (required): array of numbers
- `p` (required): percentile value 0-100

```
echo '{"scores": [72, 85, 90, 95, 88, 76, 92]}' | utlx -e 'percentile($input.scores, 90)'
# ~94.2 (90th percentile)
```

Also: `iqr(array)` — interquartile range, `quartiles(array)` — [Q1, Q2, Q3].

50.13.20 `memoryInfo()` → **object** (Sys)

Get JVM memory information in bytes. Returns an object with `maxMemory`, `totalMemory`, `freeMemory`, and `usedMemory`.

```
{
  mem: memoryInfo()
  // {maxMemory: 4294967296, totalMemory: 2147483648, freeMemory: 1073741824, usedMemory:
  1073741824}
}
```

50.13.21 `merge(obj1, obj2, ...)` → **object** (Obj)

Shallow merge of objects. Later arguments override earlier ones. Same as `spread` but as a function.

- `obj1, obj2, ...` (variadic): objects to merge

```
{
  two: merge({a: 1, b: 2}, {b: 3, c: 4}),    // {a: 1, b: 3, c: 4}
  three: merge({a: 1}, {b: 2}, {c: 3})      // {a: 1, b: 2, c: 3}
}
```

Note: for deep (recursive) merge, use `deepMerge(obj1, obj2)`.

50.13.22 `midpoint(lat1, lon1, lat2, lon2)` → **object** (Geo)

Calculate the geographic midpoint between two coordinates using the Haversine formula.

- `lat1` (required): latitude of first point
- `lon1` (required): longitude of first point
- `lat2` (required): latitude of second point
- `lon2` (required): longitude of second point

```
{
  center: midpoint(37.7749, -122.4194, 34.0522, -118.2437)
  // {lat: 35.9135, lon: -120.3315}
}
```

50.13.23 `minutes(datetime)` → **number** (Date)

Extract the minutes component (0-59) from a datetime or time value.

- `datetime` (required): datetime or time value

```
{
  mins: minutes(parseDate("2026-05-01T14:35:00Z")) // 35
}
```

50.13.24 `month(date)` → **number** (Date)

Extract the month component (1-12) from a date or datetime value.

- `date` (required): date or datetime value

```
{
  m: month(parseDate("2026-05-01"))      // 5
}
```

50.13.25 monthName(date) → string (Date)

Get the full month name from a date or datetime value.

- **date** (required): date or datetime value

```
{
  current: monthName(parseDate("2026-05-01")), // "May"
  xmas: monthName(parseDate("2026-12-25"))      // "December"
}
```

50.14 N

50.14.1 nand(a, b) → boolean (Type)

Logical NAND (NOT AND). Returns **true** unless both arguments are **true**.

- **a** (required): first boolean
- **b** (required): second boolean

```
{
  bothTrue: nand(true, true),           // false
  mixed: nand(true, false),             // true
  bothFalse: nand(false, false)        // true
}
```

50.14.2 nestBy(parents, children, parentKeyFn, childKeyFn, childProperty) → array (Arr)

Nest children under parents by matching keys. Creates a 1:N parent-child hierarchy – the most common flat-to-hierarchical integration pattern. Performance is $O(N+M)$ using a hash index. See Chapter 20.

- **parents** (required): array of parent records
- **children** (required): array of child records
- **parentKeyFn** (required): lambda extracting the join key from each parent
- **childKeyFn** (required): lambda extracting the join key from each child
- **childProperty** (required): string name for the new property on each parent

```
// Input: flat orders and lines from two separate sources
let orders = $input.orders
// [{orderId: "0-1", customer: "Alice"}, {orderId: "0-2", customer: "Bob"}]
let lines = $input.lines
// [{orderId: "0-1", sku: "W-01"}, {orderId: "0-1", sku: "G-02"},
//  {orderId: "0-2", sku: "W-01"}]

{
  nested: nestBy(orders, lines, (o) -> o.orderId, (l) -> l.orderId, "lines")
  // Output:
  // [{orderId: "0-1", customer: "Alice",
  //   lines: [{orderId: "0-1", sku: "W-01"}, {orderId: "0-1", sku: "G-02"}]},
  //  {orderId: "0-2", customer: "Bob",
  //   lines: [{orderId: "0-2", sku: "W-01"}]}]
}
```

```
echo '{"depts":[{"id":"D1","name":"Eng"}],"emps":[{"dept":"D1","name":"Alice"},
{"dept":"D1","name":"Bob"}]}' | utlx -e 'nestBy($input.depts, $input.emps, (d) -> d.id,
(e) -> e.dept, "members")'
# [{"id":"D1","name":"Eng","members":[{"dept":"D1","name":"Alice"},
{"dept":"D1","name":"Bob"}]}]
```

Inverse: `unnest(array, "children")` flattens the hierarchy back to a flat array.

50.14.3 `nodeType(node) → string` (XML)

Get the node type of an XML UDM node (element, attribute, text, etc.). See Chapter 22.

- `node` (required): XML UDM node

```
{
  type: nodeType($input)           // "element"
}
```

50.14.4 `none(array, predicate) → boolean` (Arr)

Check if no elements in the array match the predicate (all return `false`). Opposite of `some()`.

- `array` (required): array to test
- `predicate` (required): `lambda (element) -> boolean`

```
{
  allSmall: none([1, 2, 3], (x) -> x > 10),    // true (no element > 10)
  hasLarge: none([1, 2, 3], (x) -> x > 2)      // false (3 > 2)
}
```

50.14.5 `nor(a, b) → boolean` (Type)

Logical NOR (NOT OR). Returns `true` only when both arguments are `false`.

- `a` (required): first boolean
- `b` (required): second boolean

```
{
  bothFalse: nor(false, false),           // true
  oneTrue: nor(true, false),              // false
  otherTrue: nor(false, true)             // false
}
```

50.14.6 `normalizeBOM(data, targetEncoding, addBom) → binary` (XML)

Convert binary data to a target encoding with BOM (Byte Order Mark) handling. See Chapter 22.

- `data` (required): binary data to convert
- `targetEncoding` (required): target encoding (e.g., "UTF-8", "UTF-16")
- `addBom` (required): boolean – whether to add BOM to the output

```
{
  output: normalizeBOM($input.xmlBytes, "UTF-8", false)
}
```

50.14.7 normalizeXMLEncoding(xml, targetEncoding) → string (XML)

Auto-detect the encoding of an XML string and convert it to a target encoding. Also updates the XML declaration. See Chapter 22.

- `xml` (required): XML string to re-encode
- `targetEncoding` (required): target encoding (e.g., "UTF-8")

```
{
  normalized: normalizeXMLEncoding($input.xmlPayload, "UTF-8")
}
```

50.14.8 not(value) → boolean (Type)

Logical NOT. Negates a boolean value.

- `value` (required): boolean value

```
{
  negTrue: not(true),           // false
  negFalse: not(false),        // true
  hasName: not(isEmpty($input.name)) // true if name is non-empty
}
```

50.14.9 now() → datetime (Date)

Return the current UTC datetime.

```
// Use case: add timestamp to output
{
  ...$input,
  timestamp: now(),           // 2026-05-01T14:30:00Z (UTC datetime)
  processedAt: formatDate(now(), "yyyy-MM-dd'T'HH:mm:ss'Z'")
}
```

50.14.10 currentDate() → date (Date)

Return the current date (no time component).

```
{
  today: currentDate(),       // 2026-05-01 (date only, no time)
  formatted: formatDate(currentDate(), "yyyy-MM-dd") // "2026-05-01"
}
```

50.14.11 currentTime() → time (Date)

Return the current time (no date component).

```
{
  time: currentTime()         // 14:30:00 (time only, no date)
}
```

50.14.12 normalizeSpace(string) → string (Str)

Collapse all whitespace sequences (spaces, tabs, newlines) to single spaces. Trim leading/trailing.

- `string` (required): the string to normalize

```
{
  spaces: normalizeSpace("  hello  world  "),      // "hello world"
  mixed: normalizeSpace("line1\n  line2\t\tline3"), // "line1 line2 line3"
  empty: normalizeSpace("")                          // ""
}
```

50.14.13 `numberOrDefault(value, default) → number` (Type)

Safely convert a value to a number, returning the default if conversion fails or the value is null.

- `value` (required): value to convert
- `default` (required): fallback number if conversion fails

```
echo '{"qty": "abc", "price": "19.99"}' | utlx -e '{qty: numberOrDefault($input.qty, 0),
price: numberOrDefault($input.price, 0)}'
# {"qty": 0, "price": 19.99}
```

```
{
  valid: numberOrDefault("42", 0),          // 42
  missing: numberOrDefault(null, -1),       // -1
  invalid: numberOrDefault("not-a-number", 0) // 0
}
```

50.15 O

50.15.1 `omit(object, keys) → object` (Obj)

Return a new object WITHOUT the listed properties.

- `object` (required): the source object
- `keys` (required): array of property names to exclude

```
echo '{"name": "Alice", "email": "alice@example.com", "password": "secret", "role":
"admin"}' | utlx -e 'omit($input, ["password"])'
# {"name": "Alice", "email": "alice@example.com", "role": "admin"}
```

```
let safe = omit($input, ["password", "apiKey", "token", "secret"])
{
  sanitized: safe
}
```

50.15.2 `pick(object, keys) → object` (Obj)

Return a new object WITH ONLY the listed properties.

- `object` (required): the source object
- `keys` (required): array of property names to keep

```
echo '{"name": "Alice", "email": "alice@example.com", "password": "secret", "role":
"admin"}' | utlx -e 'pick($input, ["name", "email"])'
# {"name": "Alice", "email": "alice@example.com"}
```

50.15.3 `osArch() → string` (Sys)

Get the operating system architecture (e.g. "amd64", "aarch64").

```
{
  arch: osArch()
}
```

50.15.4 osVersion() → string (Sys)

Get the operating system version string.

```
{
  osVersion: osVersion()
}
```

50.16 P

50.16.1 pad(string, length, char?) → string (Str)

Pad a string on the left to the given length. Default pad character is space.

- **string** (required): the string to pad
- **length** (required): target length
- **char** (optional): pad character (default " ")

```
echo '{"id": "42"}' | utlx -e 'pad($input.id, 5, "0")'
# "00042"
```

```
{
  invoiceNo: pad(toString($input.seq), 8, "0")
}
```

50.16.2 padLeft(string, length, char?) → string (Str)

Explicit left padding — alias for pad().

- **string** (required): the string to pad
- **length** (required): target length
- **char** (optional): pad character (default " ")

```
padLeft("42", 5, "0")           // "00042"
```

50.16.3 padRight(string, length, char?) → string (Str)

Pad a string on the right to the given length.

- **string** (required): the string to pad
- **length** (required): target length
- **char** (optional): pad character (default " ")

```
padRight("hello", 10, ".")      // "hello....."
```

50.16.4 parent(element) → element (XML)

Get the parent element of an XML node (if metadata is available). See Chapter 22.

- **element** (required): an XML element

```
let p = parent($input.Order.LineItem)
p.OrderId // access sibling of parent
```

50.16.5 `parse(string, format?) → value (Fmt)`

Parse a string into a navigable UDM value, auto-detecting format or using the specified format.

- `string` (required): the string to parse
- `format` (optional, default auto-detect): "json", "xml", "yaml", "csv"

```
// Parse XML from a CDATA section:
let innerXml = parse($input.Payload, "xml")
innerXml.Order.Customer // "Acme Corp"

// Auto-detect format:
let parsed = parse($input.rawData) // auto-detects JSON, XML, or YAML
```

For normal file processing, use `input json/input xml` in the header — these functions are for the embedded-format-as-value case.

50.16.6 `parseJson(string) → value (Fmt)`

Parse a JSON string into a navigable UDM value. See Chapter 24.

- `string` (required): the JSON string to parse

```
let config = parseJson($input.configJson)
config.database.host // "localhost"
```

50.16.7 `parseXml(string) → value (Fmt)`

Parse an XML string into a navigable UDM value. See Chapter 22.

- `string` (required): the XML string to parse

```
let doc = parseXml($input.xmlPayload)
doc.Order.Customer // "Acme Corp"
```

50.16.8 `parseYaml(string) → value (Fmt)`

Parse a YAML string into a navigable UDM value. See Chapter 26.

- `string` (required): the YAML string to parse

```
let config = parseYaml($input.yamlConfig)
config.database.host // "localhost"
```

50.16.9 `parseCsv(string, options?) → array (Fmt)`

Parse a CSV string into an array of rows (objects if headers, arrays if not). See Chapter 25.

- `string` (required): the CSV string to parse
- `options` (optional): {headers: false}, {delimiter: ";"}

```
let data = parseCsv($input.csvData, {delimiter: ";", headers: true})
data[0].Name // first row, Name column
```


Also: `render(value, format, pretty?)`, `renderJson(value, pretty?)`, `renderXml(value)`, `renderYaml(value)`, `renderCsv(value)`.

50.16.10 `parseBoolean(value) → boolean` (Type)

Parse a string or number into a boolean. Accepts "true", "false", "yes", "no", "1", "0", etc.

- `value` (required): string or number to parse

```
echo '{"active": "yes"}' | utlx -e 'parseBoolean($input.active)'
# true
```

```
{
  enabled: parseBoolean($input.flags.enabled)
}
```

50.16.11 `parseCurrency(string) → number` (Fin)

Parse a currency-formatted string (with symbol and thousands separators) into a number.

- `string` (required): currency string like "\$1,234.56" or "EUR 1.000,00"

```
echo '{"price": "$1,234.56"}' | utlx -e 'parseCurrency($input.price)'
# 1234.56
```

```
{
  amount: parseCurrency($input.total)
}
```

50.16.12 `parseDate(string, pattern) → date` (Date)

Parse a date or datetime string using a format pattern. See `formatDate` for pattern tokens.

- `string` (required): the date string to parse
- `pattern` (required): format pattern

```
let d = parseDate($input.date, "dd/MM/yyyy")
{
  isoDate: formatDate(d, "yyyy-MM-dd"),
  display: formatDate(d, "MMMM d, yyyy")
}
```

```
echo '{"ts": "2026-05-01T14:30:00Z"}' | utlx -e 'formatDate(parseDate($input.ts, "yyyy-MM-dd'\''T'\'''\''HH:mm:ss'\''Z'\'''), "dd MMM yyyy")'
# 01 May 2026
```

Anti-pattern: assuming date format — 01/02/2026 is January 2nd (US MM/dd/yyyy) or February 1st (EU dd/MM/yyyy). Always specify the pattern explicitly.

Also: `parseDateTimeWithTimezone(string, pattern, timezone)`.

50.16.13 `parseDateTimeWithTimezone(string, pattern, timezone) → datetime` (Date)

Parse a datetime string using a format pattern and explicit timezone.

- `string` (required): the datetime string to parse

- **pattern** (required): format pattern
- **timezone** (required): timezone ID (e.g. "Europe/Amsterdam")

```
let dt = parseDateTimeWithTimezone($input.ts, "yyyy-MM-dd HH:mm:ss", "Europe/Amsterdam")
{
  utc: toUTC(dt)
}
```

50.16.14 **parseDouble(string) → number** (Num)

Parse a string into a double-precision floating-point number. Alias for **parseNumber**.

- **string** (required): numeric string to parse

```
parseDouble("3.14159")           // 3.14159
```

50.16.15 **parseEUNumber(string) → number** (Num)

Parse a European-formatted number string (dot=thousands, comma=decimal) to a standard number. See Chapter 25.

- **string** (required): the formatted number string

```
parseEUNumber("1.234,56")        // 1234.56 (European: dot=thousands,
comma=decimal)

// Use case: CSV from European source
map($input, (row) -> {
  product: row.Product,
  price: parseEUNumber(row.Price),    // "29,99" → 29.99
  weight: parseEUNumber(row.Weight)  // "1.500,00" → 1500.0
})
```

Also: **renderEUNumber(number)** for the reverse direction, **parseFrenchNumber(string)**, **parseSwissNumber(string)**.

50.16.16 **parseUSNumber(string) → number** (Num)

Parse a US-formatted number string (comma=thousands, dot=decimal) to a standard number.

- **string** (required): the formatted number string

```
parseUSNumber("1,234.56")        // 1234.56 (US: comma=thousands, dot=decimal)
```

Also: **renderUSNumber(number)** for the reverse direction.

50.16.17 **parseFloat(string) → number** (Num)

Parse a string into a floating-point number. Alias for **parseNumber**.

- **string** (required): numeric string to parse

```
parseFloat("2.718")              // 2.718
```

50.16.18 **parseInt(string, radix?) → number** (Num)

Parse a string into an integer, optionally with a radix (base).

- **string** (required): numeric string to parse

- **radix** (optional): numeric base (default 10)

```
echo '{"hex": "FF"}' | utlx -e 'parseInt($input.hex, 16)'
# 255
```

```
parseInt("42")           // 42
parseInt("1010", 2)      // 10
```

50.16.19 **parseNumber(string) → number** (Num)

Primary function for converting string values to numbers. Handles integers and decimals.

- **string** (required): numeric string to parse

```
parseNumber("123.45")    // 123.45
parseNumber($input.Order.Quantity) // XML values are strings – convert for
arithmetic
```

50.16.20 **parseQueryString(string) → object** (URL)

Parse a query string into an object of key-value pairs.

- **string** (required): query string (with or without leading ?)

```
echo '{"qs": "name=Alice&age=30&city=Amsterdam"}' | utlx -e 'parseQueryString($input.qs)'
# {"name": "Alice", "age": "30", "city": "Amsterdam"}
```

```
let params = parseQueryString($input.queryString)
{
  userName: params.name
}
```

50.16.21 **parseURL(url) → object** (URL)

Parse a URL string into its components (protocol, host, port, path, query, fragment).

- **url** (required): the URL string to parse

```
echo '{"url": "https://example.com:8080/api/v1?key=abc#section"}' | utlx -e
'parseURL($input.url)'
# {"protocol": "https", "host": "example.com", "port": 8080,
# "path": "/api/v1", "query": "key=abc", "fragment": "section"}
```

```
let parts = parseURL($input.endpoint)
{
  host: parts.host,
  isSecure: parts.protocol == "https"
}
```

50.16.22 **partition(array, predicate) → [matching, nonMatching]** (Arr)

Split an array into two: elements that match the predicate, and elements that don't. Returns a 2-element array of arrays.

- **array** (required): the array to partition

- `predicate` (required): `lambda (element) -> boolean`

```
echo '{"orders": [{"id": 1, "amount": 500}, {"id": 2, "amount": 1500}, {"id": 3, "amount": 200}]}' | utlx -e 'partition($input.orders, (o) -> o.amount > 1000)'
# [{"id": 2, "amount": 1500}], [{"id": 1, "amount": 500}, {"id": 3, "amount": 200}]
```

```
let validated = partition($input.records, (r) -> r.email != null && r.name != null)
{
  valid: validated[0],
  invalid: validated[1],
  validCount: count(validated[0]),
  invalidCount: count(validated[1])
}
```

50.16.23 `pascalCase(string) → string (Str)`

Convert a string to PascalCase (UpperCamelCase).

- `string` (required): the string to convert

```
pascalCase("hello world")           // "HelloWorld"
pascalCase("some-kebab-case")       // "SomeKebabCase"
```

50.16.24 `pathCase(string) → string (Str)`

Convert a string to path/case (words separated by `/`).

- `string` (required): the string to convert

```
pathCase("hello world")             // "hello/world"
pathCase("someVariable")            // "some/variable"
```

50.16.25 `percentageChange(oldValue, newValue) → number (Num)`

Calculate the percentage change between two values.

- `oldValue` (required): the original value
- `newValue` (required): the new value

```
echo '{"before": 100, "after": 125}' | utlx -e 'percentageChange($input.before, $input.after)'
# 25.0
```

```
{
  growth: percentageChange($input.lastYear, $input.thisYear)
}
```

50.16.26 `pi() → number (Num)`

Returns the mathematical constant pi (approximately 3.14159265358979323846).

```
let circumference = 2 * pi() * $input.radius
{
```

```
    circumference: circumference
}
```

50.16.27 platform() → string (Sys)

Get the platform/operating system name (e.g. "Linux", "Mac OS X", "Windows 10").

```
{
  platform: platform()
}
```

50.16.28 pluralize(word) → string (Str)

Convert a singular noun to its plural form.

- word (required): singular noun

```
pluralize("child")           // "children"
pluralize("item")           // "items"
pluralize("category")       // "categories"
```

50.16.29 pluralizeWithCount(word, count) → string (Str)

Return a formatted string with count and correctly pluralized word.

- word (required): singular noun
- count (required): the number

```
echo '{"n": 5}' | utlx -e 'pluralizeWithCount("item", $input.n)'
# "5 items"
```

```
{
  summary: pluralizeWithCount("error", count($input.errors))
}
```

50.16.30 pow(base, exponent) → number (Num)

Raise a number to a power.

- base (required): the base number
- exponent (required): the exponent

```
pow(2, 10)           // 1024
pow(10, 3)           // 1000
```

50.16.31 prepareForSignature(xml) → string (XML)

Prepare XML for digital signature (XMLDSig) by canonicalizing it. See Chapter 22.

- xml (required): XML string to prepare

```
let canonical = prepareForSignature($input.xmlPayload)
{
  signatureInput: canonical
}
```

50.16.32 presentValue(futureAmount, rate, periods) → number (Fin)

Calculate the present value of a future amount given a discount rate and number of periods.

- **futureAmount** (required): the future amount
- **rate** (required): discount rate per period (e.g. 0.05 for 5%)
- **periods** (required): number of periods

```
presentValue(10000, 0.05, 3)           // ~8638.38 (PV of 10000 in 3 years at 5%)
```

50.16.33 prettyPrint(string, indent?) → string (Fmt)

Auto-detect format and pretty-print a string (JSON, XML, or YAML).

- **string** (required): the string to format
- **indent** (optional): indentation size

```
prettyPrint($input.compactJson)       // auto-detects JSON and formats
```

50.16.34 prettyPrintCSV(csv, options?) → string (Fmt)

Format a CSV string with aligned columns for readability.

- **csv** (required): the CSV string
- **options** (optional): formatting options

```
prettyPrintCSV($input.csvData)       // columns aligned with padding
```

50.16.35 prettyPrintFormat(value, format) → string (Fmt)

Pretty-print a UDM object in the specified format.

- **value** (required): UDM value to format
- **format** (required): "json", "xml", or "yaml"

```
prettyPrintFormat($input, "json")    // indented JSON output
```

50.16.36 prettyPrintJSON(json, indent?) → string (Fmt)

Pretty-print a JSON string with optional indentation.

- **json** (required): JSON string to format
- **indent** (optional): indentation size (default 2)

```
prettyPrintJSON($input.compactJson)  // formatted with 2-space indent
```

50.16.37 prettyPrintXML(xml, options?) → string (Fmt)

Pretty-print an XML string with optional formatting options.

- **xml** (required): XML string to format
- **options** (optional): formatting options

```
prettyPrintXML($input.xmlPayload)    // indented XML
```

50.16.38 prettyPrintYAML(yaml, options?) → string (Fmt)

Pretty-print a YAML string with optional formatting options.

- **yaml** (required): YAML string to format
- **options** (optional): formatting options

```
prettyPrintYAML($input.yamlData)           // formatted YAML
```

50.17 Q

50.17.1 quarter(date) → number (Date)

Get the quarter (1-4) from a date.

- **date** (required): a date or datetime value

```
quarter(parseDate("2026-05-01", "yyyy-MM-dd")) // 2
```

50.17.2 quartiles(array) → object (Num)

Calculate quartiles (Q1, Q2/median, Q3) from a numeric array.

- **array** (required): array of numbers

```
quartiles([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
// {q1: 3, q2: 5.5, q3: 8}
```

50.18 R

50.18.1 random() → number (Num)

Generate a random number between 0.0 (inclusive) and 1.0 (exclusive).

```
{
  sampleId: floor(random() * 1000)
}
```

50.18.2 readByte(binary, offset) → number (Bin)

Read a single byte from binary data at the given offset.

- **binary** (required): binary data
- **offset** (required): byte offset (0-based)

```
readByte($input.data, 0)           // first byte value (0-255)
```

50.18.3 readDouble(binary, offset) → number (Bin)

Read a 64-bit double from binary data at the given offset (big endian).

- **binary** (required): binary data
- **offset** (required): byte offset

```
readDouble($input.data, 0)         // 64-bit floating point value
```

50.18.4 readFloat(binary, offset) → number (Bin)

Read a 32-bit float from binary data at the given offset (big endian).

- **binary** (required): binary data

- **offset** (required): byte offset

```
readFloat($input.data, 4)           // 32-bit floating point value
```

50.18.5 readInt16(binary, offset) → number (Bin)

Read a 16-bit integer from binary data at the given offset (big endian).

- **binary** (required): binary data
- **offset** (required): byte offset

```
readInt16($input.data, 0)           // 16-bit signed integer
```

50.18.6 readInt32(binary, offset) → number (Bin)

Read a 32-bit integer from binary data at the given offset (big endian).

- **binary** (required): binary data
- **offset** (required): byte offset

```
readInt32($input.data, 0)           // 32-bit signed integer
```

50.18.7 readInt64(binary, offset) → number (Bin)

Read a 64-bit integer from binary data at the given offset (big endian).

- **binary** (required): binary data
- **offset** (required): byte offset

```
readInt64($input.data, 0)           // 64-bit signed integer
```

50.18.8 readJarEntry(jar, entryName) → binary (Bin)

Read a single entry from a JAR file by name.

- **jar** (required): JAR binary data
- **entryName** (required): path inside the JAR (e.g. "META-INF/MANIFEST.MF")

```
let manifest = readJarEntry($input.jarData, "META-INF/MANIFEST.MF")
{
  content: binaryToString(manifest, "UTF-8")
}
```

50.18.9 readJarManifest(jar) → object (Bin)

Read and parse the JAR manifest into an object of key-value pairs.

- **jar** (required): JAR binary data

```
let mf = readJarManifest($input.jarData)
{
  version: mf."Implementation-Version"
}
```

50.18.10 readZipEntry(zip, entryName) → binary (Bin)

Read a single entry from a zip archive by name.

- `zip` (required): zip binary data
- `entryName` (required): path inside the archive

```
let content = readZipEntry($input.archive, "data.json")
{
  parsed: parseJson(binaryToString(content, "UTF-8"))
}
```

50.18.11 `reduce(array, initial, accumulator) → value (Arr)`

Accumulate a single result by processing each element sequentially. The most powerful array function — and the most error-prone.

- `array` (required): the array to reduce
- `initial` (required): the starting value for the accumulator
- `accumulator` (required): `lambda (accumulated, element) -> newAccumulated`

```
// Sum numbers:
reduce([10, 20, 30], 0, (sum, x) -> sum + x)
// Step by step: 0→10→30→60. Output: 60

// Build a comma-separated string:
reduce(["Alice", "Bob", "Charlie"], "", (acc, name) ->
  if (acc == "") name else concat(acc, ", ", name)
)
// "Alice, Bob, Charlie"

// Build a lookup object from an array:
reduce($input, {}, (acc, item) -> {
  ...acc,
  [item.id]: item.name
})
// {"A": "Widget", "B": "Gadget"}

// Count occurrences:
reduce(["a", "b", "a", "c", "a"], {}, (acc, x) -> {
  ...acc,
  [x]: (acc[x] ?? 0) + 1
})
// {"a": 3, "b": 1, "c": 1}
```

Anti-pattern: using `reduce` for operations that have dedicated functions:

```
// BAD – use sum() instead:
reduce(arr, 0, (acc, x) -> acc + x)

// BAD – use max() instead:
reduce(arr, 0, (acc, x) -> if (x > acc) x else acc)

// BAD – use join() instead:
reduce(arr, "", (acc, x) -> concat(acc, x, ","))

// GOOD use of reduce – complex accumulation with no dedicated function:
reduce($input.transactions, {balance: 0, count: 0}, (acc, tx) -> {
  balance: acc.balance + tx.amount,
```

```
    count: acc.count + 1
  })
```

50.18.12 `reduceEntries(object, initial, accumulator) → value (Obj)`

Reduce all entries in an object to a single value.

- `object` (required): the source object
- `initial` (required): starting accumulator value
- `accumulator` (required): `lambda (acc, key, value) -> newAcc`

```
reduceEntries({a: 1, b: 2, c: 3}, 0, (acc, k, v) -> acc + v) // 6
```

50.18.13 `regexGroups(string, pattern) → array (Str)`

Extract all capture groups from the first match of a regex pattern.

- `string` (required): the string to search
- `pattern` (required): regex with capture groups

```
regexGroups("2026-05-01", "(\\d{4})-(\\d{2})-(\\d{2})")
// ["2026", "05", "01"]
```

50.18.14 `regexNamedGroups(string, pattern) → object (Str)`

Extract named capture groups from the first match of a regex pattern.

- `string` (required): the string to search
- `pattern` (required): regex with named groups (`?<name>...`)

```
regexNamedGroups("2026-05-01", "(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})")
// {"year": "2026", "month": "05", "day": "01"}
```

50.18.15 `remove(string, search) → string (Str)`

Remove all occurrences of a substring from a string.

- `string` (required): the source string
- `search` (required): substring to remove

```
remove("Hello World", " ") // "HelloWorld"
remove("$1,234.56", ",",) // "$1234.56"
```

50.18.16 `removeBOM(data) → data (XML)`

Remove the Byte Order Mark (BOM) if present at the start of the data. See Chapter 22.

- `data` (required): binary or string data

```
let clean = removeBOM($input.fileContent)
```

50.18.17 `removeQueryParam(url, param) → string (URL)`

Remove a query parameter from a URL string.

- `url` (required): the URL string
- `param` (required): parameter name to remove

```
removeQueryParam("https://example.com?page=1&size=10", "page")
// "https://example.com?size=10"
```

50.18.18 `removeTax(totalWithTax, rate) → number` (Fin)

Calculate the original amount from a total that includes tax.

- `totalWithTax` (required): the total including tax
- `rate` (required): tax rate (e.g. 0.21 for 21%)

```
removeTax(121, 0.21) // 100.0 (remove 21% VAT from 121)
```

50.18.19 `render(value, format, pretty?) → string` (Fmt)

Serialize a UDM value to a string in the specified format.

- `value` (required): UDM value to serialize
- `format` (required): "json", "xml", "yaml", "csv"
- `pretty` (optional, default false): pretty-print with indentation

```
render({name: "Alice"}, "json", true) // pretty-printed JSON
render({Order: {Id: "1"}}, "xml")    // "<Order><Id>1</Id></Order>"

// Use case: embed XML inside a JSON field (CDATA pattern)
{
  messageId: generateUuid(),
  payload: render($input, "xml") // XML as a string value
}
```

50.18.20 `renderJson(value, pretty?) → string` (Fmt)

Serialize a UDM value to a JSON string. See Chapter 24.

- `value` (required): UDM value to serialize
- `pretty` (optional, default false): pretty-print with indentation

```
renderJson({name: "Alice", age: 30}) // '{"name":"Alice","age":30}'
renderJson({name: "Alice", age: 30}, true) // pretty-printed with newlines
```

50.18.21 `renderCsv(value) → string` (CSV)

Render a UDM array as a CSV string. See Chapter 25.

- `value` (required): array of objects (each object becomes a row)

```
renderCsv([ {name: "Alice", age: 30}, {name: "Bob", age: 25} ])
// "name,age\nAlice,30\nBob,25"
```

50.18.22 `renderXml(value) → string` (XML)

Render a UDM object as an XML string. See Chapter 22.

- `value` (required): UDM value to serialize

```
renderXml({Order: {Id: "1", Item: "Widget"}})
// "<Order><Id>1</Id><Item>Widget</Item></Order>"
```

50.18.23 renderYaml(value) → string (YAML)

Render a UDM object as a YAML string. See Chapter 26.

- **value** (required): UDM value to serialize

```
renderYaml({database: {host: "localhost", port: 5432}})
// "database:\n host: localhost\n port: 5432"
```

50.18.24 repeat(string, n) → string (Str)

Repeat a string n times.

- **string** (required): the string to repeat
- **n** (required): number of repetitions

```
repeat("ab", 3)           // "ababab"
repeat("-", 40)          // "-----"
```

50.18.25 replace(string, search, replacement) → string (Str)

Replace all literal occurrences of a substring.

- **string** (required): the string to modify
- **search** (required): literal substring to find
- **replacement** (required): what to replace with

```
replace("Hello World", "World", "UTL-X") // "Hello UTL-X"
replace("2026-05-01", "-", "/")          // "2026/05/01" (replaces ALL occurrences)
```

50.18.26 replaceRegex(string, regex, replacement) → string (Str)

Replace all occurrences matching a regular expression.

- **string** (required): the string to modify
- **regex** (required): regular expression pattern
- **replacement** (required): what to replace with

```
replaceRegex("Order #123 on 2026-05-01", "[0-9]+", "X") // "Order #X on X-X-X"
replaceRegex("  extra  spaces  ", "\\s+", " ")           // " extra spaces "
```

50.18.27 replaceWithFunction(string, pattern, fn) → string (Str)

Replace all matches of a pattern using a function to compute the replacement.

- **string** (required): the source string
- **pattern** (required): regex pattern
- **fn** (required): lambda (match) -> replacement

```
replaceWithFunction("hello world", "[a-z]+", (m) -> upper(m))
// "HELLO WORLD"
```

50.18.28 resolveQName(qname, namespaces) → object (XML)

Resolve a QName string to a full qualified name with namespace URI. See Chapter 22.

- **qname** (required): qualified name (e.g. "ns:Element")
- **namespaces** (required): namespace map object

```
resolveQName("soap:Envelope", {"soap": "http://schemas.xmlsoap.org/soap/envelope/"})
// {"localName": "Envelope",
//  "namespaceUri": "http://schemas.xmlsoap.org/soap/envelope/",
//  "prefix": "soap"}
```

50.18.29 reverse(array) → array (Arr)

Reverse the order of elements in an array.

- array (required): the array to reverse

```
reverse([1, 2, 3, 4, 5])           // [5, 4, 3, 2, 1]

// Use case: most recent first
reverse(sortBy($input.events, (e) -> e.timestamp))
```

50.18.30 reverseString(string) → string (Str)

Reverse the characters in a string.

- string (required): the string to reverse

```
reverseString("hello")           // "olleh"
```

50.18.31 rightJoin(left, right, leftKey, rightKey) → array (Arr)

Right join — returns all items from the right array, with matching items from the left. Unmatched left fields are null.

- left (required): left array
- right (required): right array
- leftKey (required): lambda to extract join key from left elements
- rightKey (required): lambda to extract join key from right elements

```
let orders = [{id: 1, product: "A"}, {id: 2, product: "B"}]
let shipments = [{orderId: 2, date: "2026-05-01"}, {orderId: 3, date: "2026-05-02"}]
rightJoin(orders, shipments, (o) -> o.id, (s) -> s.orderId)
// includes shipment for orderId 3 even though no matching order
```

50.18.32 roundToCents(number) → number (Num)

Round a number to 2 decimal places (financial rounding for currency).

- number (required): the value to round

```
roundToCents(29.999)             // 30.0
roundToCents(10.004)             // 10.0

// Use case: invoice line total with correct rounding
let lineTotal = roundToCents(qty * unitPrice)
```

50.18.33 roundToDecimalPlaces(number, places) → number (Num)

Round a number to a specified number of decimal places.

- number (required): the value to round
- places (required): number of decimal places

```
roundToDecimalPlaces(3.14159, 3)    // 3.142
roundToDecimalPlaces(100.0, 0)      // 100
```

50.18.34 runtimeInfo() → object (Sys)

Get comprehensive runtime information (JVM version, memory, OS, etc.).

```
{
  runtime: runtimeInfo()
}
```

50.19 S

50.19.1 scan(array, initial, fn) → array (Arr)

Like `reduce` but returns all intermediate results as an array.

- `array` (required): the array to scan
- `initial` (required): starting accumulator value
- `fn` (required): lambda (`acc, element`) -> `newAcc`

```
scan([1, 2, 3, 4], 0, (acc, x) -> acc + x)
// [1, 3, 6, 10] (running totals)
```

50.19.2 seconds(datetime) → number (Date)

Extract the seconds component (0-59) from a datetime.

- `datetime` (required): a datetime value

```
seconds(now())    // e.g. 45
```

50.19.3 setConsoleLogging(enabled) → null (Sys)

Enable or disable console logging output.

- `enabled` (required): `true` to enable, `false` to disable

```
let _ = setConsoleLogging(true)
```

50.19.4 setLogLevel(level) → null (Sys)

Set the minimum log level for output ("`TRACE`", "`DEBUG`", "`INFO`", "`WARN`", "`ERROR`").

- `level` (required): log level string

```
let _ = setLogLevel("DEBUG")
```

50.19.5 setPath(url, path) → string (URL)

Set or replace the path component of a URL.

- `url` (required): the URL string
- `path` (required): new path to set

```
setPath("https://example.com/old/path", "/api/v2/resource")  
// "https://example.com/api/v2/resource"
```

50.19.6 sha224(data) → string (Sec)

Compute a SHA-224 hash. Returns 56-char hex string. See Chapter 38.

- **data** (required): string to hash

```
sha224("sensitive data")           // 56-char hex string
```

50.19.7 sha256(data) → string (Sec)

Compute a SHA-256 hash. Returns 64-char hex string. See `hash` for generic algorithm selection. See Chapter 38.

- **data** (required): string to hash

```
sha256("sensitive data")           // 64-char hex string
```

50.19.8 sha512(data) → string (Sec)

Compute a SHA-512 hash. Returns 128-char hex string. See Chapter 38.

- **data** (required): string to hash

```
sha512("sensitive data")           // 128-char hex string
```

50.19.9 sha1(data) → string (Sec)

Compute a SHA-1 hash. Returns 40-char hex string. Avoid for security purposes. See Chapter 38.

- **data** (required): string to hash

```
sha1("sensitive data")             // 40-char hex string (avoid for security)
```

50.19.10 sha384(data) → string (Sec)

Compute a SHA-384 hash. Returns 96-char hex string. See Chapter 38.

- **data** (required): string to hash

```
sha384("sensitive data")           // 96-char hex string
```

50.19.11 sha3_256(data) → string (Sec)

Compute a SHA3-256 hash (if available). Returns 64-char hex string. See Chapter 38.

- **data** (required): string to hash

```
sha3_256("sensitive data")         // 64-char hex string
```

50.19.12 sha3_512(data) → string (Sec)

Compute a SHA3-512 hash (if available). Returns 128-char hex string. See Chapter 38.

- **data** (required): string to hash

```
sha3_512("sensitive data")           // 128-char hex string
```

50.19.13 **shiftLeft(binary, positions) → binary** (Bin)

Shift bits left by the specified number of positions.

- **binary** (required): binary data
- **positions** (required): number of positions to shift

```
shiftLeft(toBinary("A", "UTF-8"), 2)   // bits shifted left by 2
```

50.19.14 **shiftRight(binary, positions) → binary** (Bin)

Shift bits right by the specified number of positions.

- **binary** (required): binary data
- **positions** (required): number of positions to shift

```
shiftRight(toBinary("A", "UTF-8"), 2)  // bits shifted right by 2
```

50.19.15 **shouldUseCDATA(content, threshold?) → boolean** (XML)

Determine if content should be wrapped in a CDATA section (based on special character count). See Chapter 22.

- **content** (required): the text content to evaluate
- **threshold** (optional): number of special chars that triggers CDATA

```
shouldUseCDATA("<script>alert('hi')</script>") // true
shouldUseCDATA("plain text")                  // false
```

50.19.16 **simpleInterest(principal, rate, time) → number** (Fin)

Calculate simple interest: principal **rate** time.

- **principal** (required): the principal amount
- **rate** (required): annual interest rate (e.g. 0.05 for 5%)
- **time** (required): time in years

```
simpleInterest(1000, 0.05, 3)           // 150.0 (interest on 1000 at 5% for 3 years)
```

50.19.17 **sin(x) → number** (Num)

Compute the sine of an angle in radians.

- **x** (required): angle in radians

```
sin(pi() / 2)                          // 1.0
```

50.19.18 **singularize(word) → string** (Str)

Convert a plural noun to its singular form.

- **word** (required): plural noun


```
singularize("children")      // "child"
singularize("items")        // "item"
singularize("categories")    // "category"
```

50.19.19 sinh(x) → number (Num)

Compute the hyperbolic sine.

- `x` (required): a number

```
sinh(1)                      // ~1.1752
```

50.19.20 slice(value, start, end?) → array or string (Arr/Str)

Extract a portion of an array or string by index. Zero-based.

- `value` (required): array or string
- `start` (required): starting index (inclusive)
- `end` (optional): ending index (exclusive). If omitted, goes to end.

```
slice([10, 20, 30, 40, 50], 1, 4)    // [20, 30, 40] (index 1,2,3)
slice([10, 20, 30, 40, 50], 2)       // [30, 40, 50] (from index 2 to end)
slice("Hello World", 6, 11)          // "World"
slice("Hello World", 6)               // "World"
```

50.19.21 slugify(string) → string (Str)

Convert a string to a URL-safe slug (lowercase, hyphens, no special chars).

- `string` (required): the string to slugify

```
slugify("Hello World! 2026")        // "hello-world-2026"
slugify("Cafe Resume")               // "cafe-resume"
```

50.19.22 smartCoerce(value) → value (Type)

Smart coercion — infers the target type from context and coerces the value.

- `value` (required): value to coerce

```
smartCoerce("42")                // 42 (number)
smartCoerce("true")               // true (boolean)
smartCoerce("2026-05-01")         // date value
```

50.19.23 snakeCase(string) → string (Str)

Convert a string to `snake_case`.

- `string` (required): the string to convert

```
snakeCase("helloWorld")           // "hello_world"
snakeCase("Hello World")           // "hello_world"
```

50.19.24 some(array, predicate) → boolean (Arr)

Check if any element in the array matches the predicate.

- `array` (required): the array to test

- `predicate` (required): `lambda (element) -> boolean`

```
some([1, 2, 3, 4], (x) -> x > 3)      // true
some([1, 2, 3], (x) -> x > 5)      // false
```

50.19.25 `someEntry(object, predicate) → boolean (Obj)`

Check if any entry in the object matches the predicate.

- `object` (required): the object to test
- `predicate` (required): `lambda (key, value) -> boolean`

```
someEntry({a: 1, b: 5, c: 3}, (k, v) -> v > 4) // true
```

50.19.26 `sort(array) → array (Arr)`

Sort an array using natural ordering (numbers ascending, strings alphabetical).

- `array` (required): the array to sort

```
echo '[3, 1, 4, 1, 5]' | utlx -e 'sort()'
# [1, 1, 3, 4, 5]
```

```
sort([3, 1, 4, 1, 5, 9])           // [1, 1, 3, 4, 5, 9]
sort(["banana", "apple", "cherry"]) // ["apple", "banana", "cherry"]
```

50.19.27 `sortBy(array, keyFn) → array (Arr)`

Sort an array using a key extractor function.

- `array` (required): the array to sort
- `keyFn` (required): `lambda (element) -> sortKey`

```
echo '{"products": [{ "name": "Widget", "price": 25}, {"name": "Gadget", "price": 150}, {"name": "Gizmo", "price": 10}]}' | utlx -e 'sortBy($input.products, (p) -> p.price)'
# [{"name": "Gizmo", "price": 10}, {"name": "Widget", "price": 25}, {"name": "Gadget", "price": 150}]
```

```
{
  cheapestFirst: sortBy($input.products, (p) -> p.price),
  expensiveFirst: sortBy($input.products, (p) -> -p.price),
  alphabetical: sortBy($input.products, (p) -> p.name)
}
```

50.19.28 `split(string, separator) → array (Str)`

Split a string into an array of substrings.

- `string` (required): the string to split
- `separator` (required): delimiter string

```
split("a,b,c", ",")           // ["a", "b", "c"]
split("Hello World", " ")      // ["Hello", "World"]
split("user@example.com", "@") // ["user", "example.com"]
```

```
// Use case: extract domain from email
let parts = split($input.email, "@")
parts[1] // "example.com"

// Use case: parse a path
split("/usr/local/bin", "/") // ["", "usr", "local", "bin"]
```

50.19.29 `splitWithMatches(string, pattern) → array (Str)`

Split a string by a pattern, keeping the matched parts as separate elements in the result.

- `string` (required): the string to split
- `pattern` (required): regex pattern to split on

```
splitWithMatches("hello123world456", "\\d+")
// ["hello", "123", "world", "456"]
```

50.19.30 `sqrt(x) → number (Num)`

Compute the square root.

- `x` (required): a non-negative number

```
sqrt(16) // 4.0
sqrt(2) // ~1.4142
```

50.19.31 `startOfDay(datetime) → datetime (Date)`

Get the start of the day (midnight 00:00:00) for the given date.

- `datetime` (required): a date or datetime value

```
startOfDay(now()) // today at 00:00:00
```

50.19.32 `startOfMonth(datetime) → datetime (Date)`

Get the first day of the month for the given date.

- `datetime` (required): a date or datetime value

```
startOfMonth(parseDate("2026-05-15", "yyyy-MM-dd"))
// 2026-05-01T00:00:00
```

50.19.33 `startOfQuarter(datetime) → datetime (Date)`

Get the first day of the quarter for the given date.

- `datetime` (required): a date or datetime value

```
startOfQuarter(parseDate("2026-05-15", "yyyy-MM-dd"))
// 2026-04-01T00:00:00
```

50.19.34 `startOfWeek(datetime) → datetime (Date)`

Get the start of the week (Monday) for the given date.

- `datetime` (required): a date or datetime value

```
startOfWeek(now()) // Monday of current week
```

50.19.35 startOfYear(datetime) → datetime (Date)

Get the first day of the year for the given date.

- **datetime** (required): a date or datetime value

```
startOfYear(parseDate("2026-05-15", "yyyy-MM-dd"))  
// 2026-01-01T00:00:00
```

50.19.36 startTimer() → null (Sys)

Start a timer to measure execution time. Use with `endTimer()` to log elapsed time.

```
let _ = startTimer()  
// ... expensive operations ...  
let _ = endTimer()
```

50.19.37 stringOrDefault(value, default) → string (Type)

Safely convert a value to string, returning the default if null or undefined.

- **value** (required): value to convert
- **default** (required): fallback string

```
stringOrDefault($input.name, "Unknown") // "Unknown" if name is null
```

50.19.38 stripBOM(string) → string (XML)

Remove the BOM character (U+FEFF) from the beginning of a string. See Chapter 22.

- **string** (required): string that may start with BOM

```
let clean = stripBOM($input.fileContent)
```

50.19.39 substring(string, start, end?) → string (Str)

Extract part of a string by index (zero-based).

- **string** (required): the source string
- **start** (required): starting index (zero-based, inclusive)
- **end** (optional): ending index (exclusive). If omitted, goes to end.

```
substring("Hello World", 6) // "World"  
substring("Hello World", 0, 5) // "Hello"
```

50.19.40 substringBefore(string, delimiter) → string (Str)

Return the part of a string before the first occurrence of a delimiter. Returns the full string if delimiter is not found.

- **string** (required): the source string
- **delimiter** (required): the delimiter to search for

```
substringBefore("user@example.com", "@") // "user"
substringBefore("no-delimiter", "@")     // "no-delimiter" (not found – returns all)
```

Also: `substringBeforeLast("a.b.c.d", ".")` returns `"a.b.c"`.

50.19.41 `substringAfter(string, delimiter) → string (Str)`

Return the part of a string after the first occurrence of a delimiter. Returns empty string if delimiter is not found.

- `string` (required): the source string
- `delimiter` (required): the delimiter to search for

```
substringAfter("user@example.com", "@") // "example.com"
substringAfter("no-delimiter", "@")     // "" (not found – returns empty)
```

Also: `substringAfterLast("a.b.c.d", ".")` returns `"d"`.

50.19.42 `substringAfterLast(string, delimiter) → string (Str)`

Return the part of a string after the LAST occurrence of a delimiter.

- `string` (required): the source string
- `delimiter` (required): the delimiter to search for

```
substringAfterLast("a.b.c.d", ".")      // "d"
substringAfterLast("/usr/local/bin", "/") // "bin"
```

50.19.43 `substringBeforeLast(string, delimiter) → string (Str)`

Return the part of a string before the LAST occurrence of a delimiter.

- `string` (required): the source string
- `delimiter` (required): the delimiter to search for

```
substringBeforeLast("a.b.c.d", ".")      // "a.b.c"
substringBeforeLast("file.tar.gz", ".")   // "file.tar"
```

50.19.44 `sum(array) → number (Num)`

Sum all numeric values in an array. Returns 0 for empty arrays.

- `array` (required): array of numbers

```
sum([10, 20, 30])           // 60
sum([])                     // 0 (empty array)
```

Anti-pattern: `reduce($input.items, 0, (acc, i) -> acc + i.price)` — use `sum(map(...))` or `sumBy()`.

50.19.45 `sumBy(array, fn) → number (Num)`

Sum values extracted from an array of objects using a key function.

- `array` (required): array of objects
- `fn` (required): `lambda (element) -> number`

```
echo '{"items": [{"qty": 2, "price": 25}, {"qty": 5, "price": 10}, {"qty": 1, "price": 100}]}' | utlx -e 'sumBy($input.items, (i) -> i.qty * i.price)'
# 200
```

```
{
  orderTotal: sumBy($input.items, (i) -> i.qty * i.price)
}
```

50.19.46 `systemPropertiesAll()` → `object` (Sys)

Get all Java system properties as an object.

```
{
  allProps: systemPropertiesAll()
}
```

Security note: can be restricted in UTLXe via security policy (Chapter 38) to prevent exfiltration of host configuration in multi-tenant deployments.

50.19.47 `systemProperty(name)` → `string` (Sys)

Get a Java system property by name. Returns null if not set.

- `name` (required): property name (e.g. "java.version")

```
{
  javaHome: systemProperty("java.home")
}
```

Security note: can be restricted in UTLXe via security policy (Chapter 38).

50.19.48 `systemPropertyOrDefault(name, default)` → `string` (Sys)

Get a system property with a fallback default value.

- `name` (required): property name
- `default` (required): fallback value if property is not set

```
{
  encoding: systemPropertyOrDefault("file.encoding", "UTF-8")
}
```

Security note: can be restricted in UTLXe via security policy (Chapter 38).

50.20 T

50.20.1 `tan(x)` → `number` (Num)

Compute the tangent of an angle in radians.

- `x` (required): angle in radians

```
tan(pi() / 4) // ~1.0
```

50.20.2 `tanh(x)` → `number` (Num)

Compute the hyperbolic tangent.

- `x` (required): a number

```
tanh(1) // ~0.7616
```

50.20.3 `tempDir()` → `string` (Sys)

Get the system temporary directory path.

```
{
  tmpDir: tempDir()
}
```

Security note: can be restricted in UTLXe via security policy (Chapter 38) to prevent revealing host file system paths.

50.20.4 `textContent(element)` → `string` (XML)

Get concatenated text content from an XML element (all text nodes). See Chapter 22.

- `element` (required): an XML element

```
// Input: <p>Hello <b>world</b>!</p>
textContent($input.p) // "Hello world!"
```

50.20.5 `timerCheck(name)` → `number` (Sys)

Get elapsed time of a named timer without stopping it.

- `name` (required): timer name

```
let elapsed = timerCheck("process") // milliseconds since start
```

50.20.6 `timerClear()` → `null` (Sys)

Clear all timers and measurements.

```
let _ = timerClear()
```

50.20.7 `timerList()` → `array` (Sys)

List all active timer names.

```
let active = timerList() // ["process", "transform"]
```

50.20.8 `timerReset(name)` → `null` (Sys)

Reset a named timer to zero without stopping it.

- `name` (required): timer name

```
let _ = timerReset("process")
```

50.20.9 `timerStart(name)` → `null` (Sys)

Start a named timer.

- `name` (required): timer name

```
let _ = timerStart("transform")
```

50.20.10 timerStats(name) → object (Sys)

Get statistics for a named timer (min, max, avg, count).

- **name** (required): timer name

```
let stats = timerStats("transform")  
// {min: 12, max: 45, avg: 28, count: 100}
```

50.20.11 timerStop(name) → number (Sys)

Stop a named timer and return elapsed time in milliseconds.

- **name** (required): timer name

```
let elapsed = timerStop("transform")    // e.g. 42
```

50.20.12 timestamp() → string (Date)

Get the current timestamp as an ISO 8601 string.

```
{  
  processedAt: timestamp()  
}
```

50.20.13 titleCase(string) → string (Str)

Convert a string to Title Case (capitalize first letter of each word).

- **string** (required): the string to convert

```
titleCase("hello world")    // "Hello World"  
titleCase("the quick brown fox") // "The Quick Brown Fox"
```

50.20.14 toArray(value) → array (Type)

Convert a value to an array. Wraps non-array values in a single-element array; arrays pass through.

- **value** (required): value to convert

```
toArray("hello")            // ["hello"]  
toArray([1, 2, 3])         // [1, 2, 3] (unchanged)  
toArray(null)              // []
```

50.20.15 toBase64(binary) → string (Bin)

Convert binary data to a Base64-encoded string.

- **binary** (required): binary data to encode

```
toBase64(toBinary("Hello", "UTF-8"))    // "SGVsbG8="
```

50.20.16 toBinary(string, encoding?) → binary (Bin)

Create binary data from a string with the specified encoding.

- **string** (required): the string to convert
- **encoding** (optional): character encoding (default "UTF-8")

```
let data = toBinary("Hello World", "UTF-8")
{
  length: binaryLength(data)
}
```

50.20.17 toBoolean(value) → boolean (Type)

Convert a value to boolean.

- **value** (required): string, number, or boolean to convert

```
toBoolean("true")           // true
toBoolean("false")          // false
toBoolean(1)                 // true
toBoolean(0)                 // false
toBoolean("yes")             // true
toBoolean("no")              // false
```

50.20.18 toBytes(binary) → array (Bin)

Convert binary data to an array of byte values (0-255).

- **binary** (required): binary data

```
toBytes(toBinary("AB", "UTF-8")) // [65, 66]
```

50.20.19 toDate(string) → date (Date)

Convert a string to a date. Alias for **parseDate** with single argument (ISO 8601 auto-detection).

- **string** (required): date string in ISO format

```
toDate("2026-05-01")          // date value
```

50.20.20 toDegrees(radians) → number (Num)

Convert radians to degrees.

- **radians** (required): angle in radians

```
toDegrees(pi())               // 180.0
toDegrees(pi() / 2)           // 90.0
```

50.20.21 toHex(binary) → string (Bin)

Convert binary data to a hexadecimal string.

- **binary** (required): binary data

```
toHex(toBinary("AB", "UTF-8")) // "4142"
```

50.20.22 toNumber(value) → number (Type)

Convert a value to a number. Throws an error if the value cannot be parsed.

- **value** (required): string, boolean, or number to convert

```
toNumber("42")           // 42
toNumber("3.14")         // 3.14
toNumber(true)           // 1
toNumber(false)          // 0
toNumber("not-a-number") // ERROR – runtime error

// Use case: XML values are always strings – convert for arithmetic
let quantity = toNumber($input.Order.Quantity)
let price = toNumber($input.Order.Price)
{lineTotal: quantity * price}
```

Anti-pattern: `toNumber()` on unvalidated user input without error handling:

```
// BAD – crashes on invalid input:
toNumber($input.userProvidedValue)

// GOOD – safe with fallback:
try { toNumber($input.userProvidedValue) } catch { 0 }
```

50.20.23 `toObject(value) → object` (Type)

Try to convert a value to an object. Arrays of pairs become objects; objects pass through.

- **value** (required): value to convert

```
toObject([["name", "Alice"], ["age", 30]]) // {"name": "Alice", "age": 30}
```

50.20.24 `toRadians(degrees) → number` (Num)

Convert degrees to radians.

- **degrees** (required): angle in degrees

```
toRadians(180) // ~3.14159 (pi)
toRadians(90)  // ~1.5708
```

50.20.25 `toString(value) → string` (Type)

Convert any value to its string representation.

- **value** (required): any value to convert

```
toString(42)           // "42"
toString(3.14)         // "3.14"
toString(true)         // "true"
toString(null)         // "null"
toString([1, 2])       // "[1, 2]"
```

Also: `toDate(value)`, `toArray(value)` (wraps non-array in array), `toObject(value)`, `toBinary(string)`.

50.20.26 `toTitleCase(string) → string` (Str)

Convert a string to title case (capitalize first letter of each word). Alias for `titleCase`.

- **string** (required): the string to convert

```
toTitleCase("hello world")           // "Hello World"
```

50.20.27 toUTC(datetime) → datetime (Date)

Convert a local datetime to UTC.

- **datetime** (required): datetime with timezone information

```
{
  utcTime: toUTC($input.localTimestamp)
}
```

50.20.28 trace(value) → value (Sys)

Log a value with TRACE level and return it (passthrough for debugging in pipelines).

- **value** (required): value to log and pass through

```
let result = trace($input.data)       // logs and passes through
```

50.20.29 translate(string, from, to) → string (Str)

Translate characters in a string: each character in **from** is replaced by the corresponding character in **to**.

- **string** (required): the source string
- **from** (required): characters to replace
- **to** (required): replacement characters (same length)

```
translate("hello", "elo", "ELO")      // "hELLO"
translate("2026-05-01", "-", "/")     // "2026/05/01"
```

50.20.30 treeDepth(object) → number (Obj)

Get the maximum nesting depth of a tree structure.

- **object** (required): nested object/array structure

```
treeDepth({a: {b: {c: 1}}})           // 3
```

50.20.31 treeFilter(object, predicate) → object (Obj)

Filter tree nodes by a predicate, removing branches that don't match.

- **object** (required): tree structure
- **predicate** (required): lambda (node) -> boolean

```
treeFilter($input.menu, (node) -> node.visible == true)
```

50.20.32 treeFind(object, path) → value (Obj)

Find a node by path in a tree structure.

- **object** (required): tree structure
- **path** (required): path to search for

```
treeFind($input.org, "engineering.backend")
```

50.20.33 treeFlatten(object) → array (Obj)

Flatten a tree to an array of leaf nodes.

- **object** (required): tree structure

```
treeFlatten({a: {b: 1, c: 2}, d: 3})    // [1, 2, 3]
```

50.20.34 treeMap(object, fn) → object (Obj)

Recursively transform all values in a nested structure.

- **object** (required): tree structure
- **fn** (required): lambda (value) -> newValue

```
treeMap($input, (v) -> if (isString(v)) trim(v) else v)
```

50.20.35 treePaths(object) → array (Obj)

Get all paths in a tree structure as an array of path strings.

- **object** (required): tree structure

```
treePaths({a: {b: 1}, c: 2})           // ["a.b", "c"]
```

50.20.36 trim(string) → string (Str)

Remove whitespace from both ends of a string.

- **string** (required): the string to trim

```
trim("  hello  ")                      // "hello"

// Use case: clean up CSV values
map($input, (row) -> mapValues(row, (v) -> if (isString(v)) trim(v) else v))
```

50.20.37 leftTrim(string) → string (Str)

Remove whitespace from the LEFT (start) of a string only.

- **string** (required): the string to trim

```
leftTrim("  hello  ")                  // "hello  "
```

50.20.38 rightTrim(string) → string (Str)

Remove whitespace from the RIGHT (end) of a string only.

- **string** (required): the string to trim

```
rightTrim("  hello  ")                 // "  hello"
```

Also: `normalizeSpace(string)` — trims AND collapses internal whitespace to single spaces.

50.20.39 transpose(array2D) → array2D (Arr)

Transpose a 2D array — rows become columns, columns become rows.

- **array2D** (required): array of arrays (all same length)

```
transpose([[1, 2, 3], [4, 5, 6]])           // [[1, 4], [2, 5], [3, 6]]
```

```
{
  pivoted: transpose($input.table)
}
```

50.20.40 truncate(string, maxLength, suffix?) → string (Str)

Truncate a string to a maximum length, appending a suffix (default "...").

- **string** (required): the string to truncate
- **maxLength** (required): maximum total length (including suffix)
- **suffix** (optional): suffix to append (default "...")

```
truncate("Hello World, this is a long string", 15)
// "Hello World,..."

truncate("Hello World", 20)           // "Hello World" (no truncation needed)
```

50.20.41 tryCoerce(value, type) → value (Type)

Try to coerce a value to the target type, returning null on failure (instead of throwing).

- **value** (required): value to coerce
- **type** (required): target type string (e.g. "number", "boolean")

```
tryCoerce("42", "number")             // 42
tryCoerce("not-a-number", "number")   // null (no error)
```

50.20.42 typeOf(value) → string (Type)

Return the type of a value as a string ("string", "number", "boolean", "array", "object", "null").

- **value** (required): any value

```
typeOf("hello")                       // "string"
typeOf(42)                            // "number"
typeOf([1, 2])                        // "array"
typeOf({a: 1})                        // "object"
```

50.20.43 typeof(value) → string (Type)

Alias for typeOf — JavaScript/TypeScript naming compatibility.

- **value** (required): any value

```
typeof(null)                         // "null"
typeof(true)                         // "boolean"
```

50.21 U

50.21.1 udmToJSON(value, pretty?) → string (Fmt)

Pretty-print a UDM object as JSON.

- **value** (required): UDM value to format

- **pretty** (optional): pretty-print with indentation

```
udmToJSON($input, true)           // formatted JSON string
```

50.21.2 **udmToXML**(value, rootName?, options?) → string (Fmt)

Pretty-print a UDM object as XML.

- **value** (required): UDM value to format
- **rootName** (optional): root element name
- **options** (optional): formatting options

```
udmToXML($input, "Order")        // XML string with <Order> root
```

50.21.3 **udmToYAML**(value, options?) → string (Fmt)

Pretty-print a UDM object as YAML.

- **value** (required): UDM value to format
- **options** (optional): formatting options

```
udmToYAML($input)                // YAML string
```

50.21.4 **uncamelize**(string) → string (Str)

Convert from camelCase to separate words. Legacy naming — prefer `fromCamelCase`.

- **string** (required): camelCase string

```
uncamelize("helloWorld")         // "hello world"
uncamelize("firstName")          // "first name"
```

50.21.5 **unescapeXML**(string) → string (XML)

Unescape XML entities (<, >, &, ", ') back to their characters. See Chapter 22.

- **string** (required): string with XML entities

```
unescapeXML("price &lt; 100 &amp; qty &gt; 0")
// "price < 100 & qty > 0"
```

50.21.6 **union**(arr1, arr2) → array (Arr)

Combine two arrays, removing duplicates. Returns all unique values from both.

- **arr1** (required): first array
- **arr2** (required): second array

```
union([1, 2, 3], [3, 4, 5])      // [1, 2, 3, 4, 5]
union(["A", "B"], ["B", "C", "D"]) // ["A", "B", "C", "D"]
```

50.21.7 **intersect**(arr1, arr2) → array (Arr)

Return values present in BOTH arrays.

- **arr1** (required): first array
- **arr2** (required): second array

```
intersect([1, 2, 3], [2, 3, 4])           // [2, 3]
intersect(["A", "B", "C"], ["X", "Y"])    // [] (no common elements)
```

50.21.8 difference(arr1, arr2) → array (Arr)

Return values in `arr1` that are NOT in `arr2`. Order matters — `difference(a, b)` is not the same as `difference(b, a)`.

- `arr1` (required): the source array
- `arr2` (required): the array to subtract

```
difference([1, 2, 3], [2, 3, 4])          // [1]
difference([2, 3, 4], [1, 2, 3])          // [4]
```

```
let previous = map($input.previousOrders, (o) -> o.id)
let current = map($input.currentOrders, (o) -> o.id)
{
  newOrders: difference(current, previous),
  removedOrders: difference(previous, current)
}
```

50.21.9 symmetricDifference(arr1, arr2) → array (Arr)

Return values that are in EITHER array but NOT in both. The “exclusive or” of two arrays.

- `arr1` (required): first array
- `arr2` (required): second array

```
symmetricDifference([1, 2, 3], [2, 3, 4]) // [1, 4]
symmetricDifference([1, 2], [1, 2])       // [] (identical)
```

50.21.10 unique(array) → array (Arr)

Remove duplicate values. Alias for `distinct()`. Preserves first occurrence.

- `array` (required): the array to deduplicate

```
unique([1, 2, 2, 3, 3, 3])                // [1, 2, 3]
unique(["apple", "banana", "apple"])      // ["apple", "banana"]

// Use case: collect unique customer IDs
{
  customers: unique(map($input.orders, (o) -> o.customerId))
}
```

50.21.11 unnest(array, childKey) → array (Arr)

Flatten nested children alongside parent fields. The reverse of `nestBy` — converts hierarchical data to flat rows. See Chapter 20.

- `array` (required): array of parent objects containing nested children
- `childKey` (required): the key holding the children array

```
echo '{"orders": [{"customer": "Alice", "items": [{"sku": "A1", "qty": 2}, {"sku": "B2", "qty": 1}]}, {"customer": "Bob", "items": [{"sku": "C3", "qty": 5}]}]}' | utlx -e
```

```
'unnest($input.orders, "items")'
# [{"customer": "Alice", "sku": "A1", "qty": 2}, {"customer": "Alice", "sku": "B2", "qty": 1},
  {"customer": "Bob", "sku": "C3", "qty": 5}]
```

```
// Input: [{customer: "Alice",
//          items: [{sku: "A1", qty: 2}, {sku: "B2", qty: 1}]}]
// Output: [{customer: "Alice", sku: "A1", qty: 2},
//          {customer: "Alice", sku: "B2", qty: 1}]
// Typical use: denormalize order/line-items for flat CSV export
let flat = unnest($input.orders, "items")
map(flat, (row) -> {
  customer: row.customer,
  sku: row.sku,
  quantity: row.qty
})
```

50.21.12 unwrapCDATA(string) → string (XML)

Unwrap CDATA section if present, otherwise return the original string. See Chapter 22.

- **string** (required): string that may be wrapped in `<![CDATA[...]]>`

```
unwrapCDATA("<![CDATA[Hello <World>]]>") // "Hello <World>"
unwrapCDATA("plain text")                // "plain text"
```

50.21.13 unzip(pairs) → [array, array] (Bin)

Unzip an array of pairs into two separate arrays (inverse of `zip`).

- **pairs** (required): array of 2-element arrays

```
unzip([[1, "a"], [2, "b"], [3, "c"]]) // [[1, 2, 3], ["a", "b", "c"]]
```

50.21.14 unzipArchive(zip, options?) → object (Bin)

Extract all files from a zip archive into an object keyed by entry name.

- **zip** (required): zip binary data
- **options** (optional): extraction options

```
let files = unzipArchive($input.archive)
{
  readme: binaryToString(files."README.md", "UTF-8")
}
```

50.21.15 unzipN(tuples) → array of arrays (Bin)

Unzip an array of N-tuples into N separate arrays (generalized `unzip`).

- **tuples** (required): array of N-element arrays

```
unzipN([[1, "a", true], [2, "b", false]])
// [[1, 2], ["a", "b"], [true, false]]
```


50.21.16 updateXMLEncoding(xml, encoding) → string (XML)

Update the encoding declaration in an XML processing instruction. See Chapter 22.

- `xml` (required): XML string
- `encoding` (required): new encoding name (e.g. "UTF-16")

```
updateXMLEncoding($input.xmlData, "UTF-8")
```

50.21.17 upper(string) → string (Str)

Convert a string to uppercase. Legacy naming — prefer `upperCase`.

- `string` (required): the string to convert

```
upper("hello")           // "HELLO"
```

50.21.18 uptime() → number (Sys)

Get JVM uptime in milliseconds.

```
{
  uptimeMs: uptime()
}
```

50.21.19 urlDecodeComponent(string) → string (URL)

URL-decode a component string (RFC 3986) — decodes `%20` as spaces.

- `string` (required): the encoded string

```
urlDecodeComponent("hello%20world")    // "hello world"
```

50.21.20 urlEncode(string) → string (URL)

URL-encode a string (percent-encoding per RFC 3986).

- `string` (required): the string to encode

```
urlEncode("hello world")           // "hello%20world"
urlEncode("price=10&currency=EUR") // "price%3D10%26currency%3DEUR"

// Use case: build a query string
let qs = join(map(entries($input.params), (e) ->
  concat(urlEncode(e[0]), "=", urlEncode(toString(e[1])))
), "&")
// Or use the dedicated function:
buildQueryString($input.params)
```

Also: `urlEncodeComponent(string)`, `buildURL(base, path, params)`, `parseURL(url)`, `getHost(url)`, `getPath(url)`, `getQuery(url)`, `getQueryParams(url)`.

50.21.21 urlDecode(string) → string (URL)

Decode a URL-encoded string (percent-encoding per RFC 3986).

- `string` (required): the string to decode

```
urlDecode("hello%20world")           // "hello world"
```

Also: `urlDecodeComponent(string)`.

50.21.22 `urlEncodeComponent(string) → string` (URL)

URL-encode a component string (RFC 3986) — encodes spaces as `%20` for URI paths.

- `string` (required): the string to encode

```
urlEncodeComponent("hello world/path") // "hello%20world%2Fpath"
```

50.21.23 `username() → string` (Sys)

Get the current system username.

```
{
  user: username()
}
```

50.22 V-W

50.22.1 `validateDate(string, pattern?) → boolean` (Date)

Validate whether a string is a valid date (ISO 8601 or custom pattern).

- `string` (required): the date string to validate
- `pattern` (optional): format pattern to validate against

```
validateDate("2026-05-01")           // true
validateDate("2026-13-01")           // false (month 13 invalid)
validateDate("01/05/2026", "dd/MM/yyyy") // true
```

50.22.2 `validateDigest(xml, expectedDigest) → boolean` (Sec)

Validate that an XML digest matches the expected value. See Chapter 38.

- `xml` (required): XML string to validate
- `expectedDigest` (required): expected hash value

```
validateDigest($input.signedXml, $input.expectedHash)
// true if digest matches
```

50.22.3 `validateEncoding(encoding) → boolean` (XML)

Check if an encoding name is valid and supported. See Chapter 22.

- `encoding` (required): encoding name string

```
validateEncoding("UTF-8")             // true
validateEncoding("INVALID-ENC")       // false
```

50.22.4 `values(object) → array` (Obj)

See keys above. Returns all property values as an array.

```
values({name: "Alice", age: 30})           // ["Alice", 30]
```

50.22.5 version() → string (Sys)

Get the UTL-X engine version.

```
{
  engineVersion: version()
}
```

50.22.6 warn(value) → value (Sys)

Log a value with WARN level and return it (passthrough).

- **value** (required): value to log and pass through

```
let result = warn("Deprecated field used")
```

50.22.7 weekOfYear(date) → number (Date)

Get the ISO week number (1-53) for a date.

- **date** (required): a date or datetime value

```
weekOfYear(parseDate("2026-05-01", "yyyy-MM-dd")) // 18
```

50.22.8 windowed(array, size) → array of arrays (Arr)

Create a sliding window over an array. Returns overlapping sub-arrays of the given size.

- **array** (required): the source array
- **size** (required): window size

```
windowed([1, 2, 3, 4, 5], 3)           // [[1, 2, 3], [2, 3, 4], [3, 4, 5]]
windowed([1, 2, 3, 4, 5], 2)           // [[1, 2], [2, 3], [3, 4], [4, 5]]
```

```
let prices = $input.dailyPrices
{
  movingAvg3day: map(windowed(prices, 3), (w) -> avg(w)),
  consecutiveDups: filter(windowed($input.events, 2), (pair) -> pair[0].type ==
pair[1].type)
}
```

50.22.9 wordCase(string) → string (Str)

Convert a string to word case (capitalize first letter, rest lowercase).

- **string** (required): the string to convert

```
wordCase("HELLO WORLD")               // "Hello world"
wordCase("hello")                      // "Hello"
```

50.22.10 wrapIfNeeded(content, threshold?, tag?) → string (XML)

Automatically wrap content in CDATA if it contains enough special characters to benefit. See Chapter 22.

- **content** (required): the text content
- **threshold** (optional): special char count threshold
- **tag** (optional): wrapper format

```
wrapIfNeeded("<script>alert('hi')</script>")  
// "<![CDATA[<script>alert('hi')</script>]]>"  
  
wrapIfNeeded("plain text")           // "plain text" (no wrapping needed)
```

50.22.11 writeByte(value) → binary (Bin)

Write a single byte value to binary.

- **value** (required): byte value (0-255)

```
writeByte(65)           // binary containing byte 0x41
```

50.22.12 writeDouble(value) → binary (Bin)

Write a 64-bit double to binary (big endian).

- **value** (required): double value

```
writeDouble(3.14159)     // 8 bytes of binary data
```

50.22.13 writeFloat(value) → binary (Bin)

Write a 32-bit float to binary (big endian).

- **value** (required): float value

```
writeFloat(3.14)         // 4 bytes of binary data
```

50.22.14 writeInt16(value) → binary (Bin)

Write a 16-bit integer to binary (big endian).

- **value** (required): integer value

```
writeInt16(256)          // 2 bytes: [0x01, 0x00]
```

50.22.15 writeInt32(value) → binary (Bin)

Write a 32-bit integer to binary (big endian).

- **value** (required): integer value

```
writeInt32(65536)        // 4 bytes of binary data
```

50.22.16 writeInt64(value) → binary (Bin)

Write a 64-bit integer to binary (big endian).

- **value** (required): integer value

```
writeInt64(1000000000)    // 8 bytes of binary data
```

50.23 X-Z

50.23.1 `xmlEscape(string) → string` (XML)

Escape XML special characters (<, >, &, ", '). See Chapter 22.

- `string` (required): the string to escape

```
xmlEscape("price < 100 & tax > 0")           // "price &lt; 100 &amp; tax &gt; 0"
```

```
{
  Comment: xmlEscape($input.userComment)
}
```

50.23.2 `xmlUnescape(string) → string` (XML)

Unescape XML entity references back to their original characters. See Chapter 22.

- `string` (required): the string to unescape

```
xmlUnescape("price &lt; 100 &amp; tax &gt; 0") // "price < 100 & tax > 0"
```

50.23.3 `xnor(a, b) → boolean` (Type)

Logical XNOR (exclusive NOR / equivalence) — returns true when both operands are the same.

- `a` (required): first boolean
- `b` (required): second boolean

```
xnor(true, true)           // true
xnor(false, false)        // true
xnor(true, false)         // false
```

50.23.4 `xor(a, b) → boolean` (Type)

Logical XOR (exclusive OR) — returns true when exactly one operand is true.

- `a` (required): first boolean
- `b` (required): second boolean

```
xor(true, false)          // true
xor(true, true)           // false
xor(false, false)         // false
```

50.23.5 `yamlEntries(object) → array` (YAML)

Get entries (key-value pairs) from a YAML object as an array of [`key`, `value`] pairs. See Chapter 26.

- `object` (required): YAML object

```
yamlEntries({host: "localhost", port: 5432})
// [{"host", "localhost"}, {"port", 5432}]
```

50.23.6 `yamlExists(yaml, path) → boolean` (YAML)

Check if a path exists in a YAML structure. See Chapter 26.

- `yaml` (required): YAML value

- **path** (required): dot-separated path

```
yamlExists($input, "database.host")      // true or false
```

50.23.7 **yamlFilterByKeyPattern(object, pattern) → object (YAML)**

Filter a YAML object, keeping only keys that match a pattern. See Chapter 26.

- **object** (required): YAML object
- **pattern** (required): regex pattern for keys

```
yamlFilterByKeyPattern($input, "^db_.*") // keys starting with "db_"
```

50.23.8 **yamlFindByField(yaml, fieldName) → array (YAML)**

Find all values in a YAML structure by field name (recursive search). See Chapter 26.

- **yaml** (required): YAML value
- **fieldName** (required): field name to search for

```
yamlFindByField($input, "version")      // all "version" values in the tree
```

50.23.9 **yamlFindObjectsWithField(yaml, fieldName) → array (YAML)**

Find all objects containing a specific field (recursive search). See Chapter 26.

- **yaml** (required): YAML value
- **fieldName** (required): field name to search for

```
yamlFindObjectsWithField($input, "image")  
// all objects that have an "image" field
```

50.23.10 **yamlFromEntries(entries) → object (YAML)**

Create a YAML object from an array of [key, value] pairs. See Chapter 26.

- **entries** (required): array of [key, value] pairs

```
yamlFromEntries([["host", "localhost"], ["port", 5432]])  
// {host: "localhost", port: 5432}
```

50.23.11 **yamlGetDocument(yaml, index) → value (YAML)**

Get a specific document from a multi-document YAML by index. See Chapter 26.

- **yaml** (required): multi-document YAML string
- **index** (required): zero-based document index

```
yamlGetDocument(multiDocYaml, 0)      // first document
```

50.23.12 **yamlHasRequiredFields(object, fields) → boolean (YAML)**

Check if a YAML object has all the required fields. See Chapter 26.

- **object** (required): YAML object to check
- **fields** (required): array of required field names

```
yamlHasRequiredFields($input, ["apiVersion", "kind", "metadata"])  
// true if all three fields exist
```

50.23.13 **yamlKeys(object) → array** (YAML)

Get all keys from a YAML object. See Chapter 26.

- **object** (required): YAML object

```
yamlKeys($input) // ["apiVersion", "kind", "metadata", "spec"]
```

50.23.14 **yamlMerge(obj1, obj2) → object** (YAML)

Deep merge two YAML objects. Values from **obj2** override **obj1** for matching keys. See Chapter 26.

- **obj1** (required): base object
- **obj2** (required): override object

```
yamlMerge({a: 1, b: {c: 2}}, {b: {d: 3}})  
// {a: 1, b: {c: 2, d: 3}}
```

50.23.15 **yamlMergeAll(objects) → object** (YAML)

Merge multiple YAML objects in order (last wins for conflicts). See Chapter 26.

- **objects** (required): array of objects to merge

```
yamlMergeAll([{"a": 1}, {"b": 2}, {"a": 3}]) // {a: 3, b: 2}
```

50.23.16 **yamlOmitKeys(object, keys) → object** (YAML)

Remove specific keys from a YAML object. See Chapter 26.

- **object** (required): YAML object
- **keys** (required): array of keys to remove

```
yamlOmitKeys($input, ["password", "secret"])
```

50.23.17 **yamlSelectKeys(object, keys) → object** (YAML)

Keep only specific keys from a YAML object. See Chapter 26.

- **object** (required): YAML object
- **keys** (required): array of keys to keep

```
yamlSelectKeys($input, ["name", "version"])
```

50.23.18 **yamlSort(object, comparator?) → object** (YAML)

Sort YAML object keys alphabetically. See Chapter 26.

- **object** (required): YAML object to sort
- **comparator** (optional): custom comparator function

```
yamlSort({z: 1, a: 2, m: 3}) // {a: 2, m: 3, z: 1}
```

50.23.19 yamlSplitDocuments(yaml) → array (YAML)

Split a multi-document YAML string (separated by ---) into an array of documents. See Chapter 26.

- **yaml** (required): YAML string containing multiple documents

```
let docs = yamlSplitDocuments(multiDocString)
docs[0]           // first document
docs[1]           // second document
```

50.23.20 yamlMergeDocuments(docs) → string (YAML)

Merge an array of YAML documents back into a single multi-document string joined with --- separators. See Chapter 26.

- **docs** (required): array of documents

```
yamlMergeDocuments(docs)           // joined with --- separators
```

50.23.21 yamlPath(yaml, path) → value (YAML)

Access a nested value in a YAML structure using a dot-separated path. See Chapter 26.

- **yaml** (required): YAML UDM value
- **path** (required): dot-separated path string

```
yamlPath($input, "database.host")   // "localhost"
```

50.23.22 yamlSet(yaml, path, value) → value (YAML)

Return a new YAML structure with the value at the given path replaced. See Chapter 26.

- **yaml** (required): YAML UDM value
- **path** (required): dot-separated path string
- **value** (required): value to set at path

```
yamlSet($input, "database.port", 5433) // returns new structure with port changed
```

50.23.23 yamlDelete(yaml, path) → value (YAML)

Return a new YAML structure with the value at the given path removed. See Chapter 26.

- **yaml** (required): YAML UDM value
- **path** (required): dot-separated path string

```
yamlDelete($input, "database.password") // returns structure without password
```

Also: `yamlDeepMerge(obj1, obj2)`, `yamlKeys(obj)`, `yamlValues(obj)`, `yamlSort(obj)`, `yamlValidate(yaml, rules)`, `yamlFilterByKeyPattern(obj, pattern)`.

50.23.24 yamlValidate(yaml, rules) → object (YAML)

Validate YAML against a set of rules. Returns validation result with errors. See Chapter 26.

- **yaml** (required): YAML value to validate
- **rules** (required): validation rules object


```
yamlValidate($input, {required: ["apiVersion", "kind"]})
// {valid: true, errors: []}
```

50.23.25 `yamlValidateKeyPattern(object, pattern) → boolean` (YAML)

Validate that all keys in a YAML object match a given pattern. See Chapter 26.

- `object` (required): YAML object to validate
- `pattern` (required): regex pattern that keys must match

```
yamlValidateKeyPattern($input, "^[a-z][a-zA-Z0-9]*$")
// true if all keys are camelCase
```

50.23.26 `yamlValues(object) → array` (YAML)

Get all values from a YAML object. See Chapter 26.

- `object` (required): YAML object

```
yamlValues({host: "localhost", port: 5432})
// ["localhost", 5432]
```

50.23.27 `zip(arr1, arr2) → array` (Arr)

Combine two arrays element-by-element into pairs. Truncated to the shorter array's length.

- `arr1` (required): first array
- `arr2` (required): second array

```
zip([1, 2, 3], ["a", "b", "c"]) // [[1, "a"], [2, "b"], [3, "c"]]
```

```
let headers = ["Name", "Age", "City"]
let row = ["Alice", "30", "Amsterdam"]
{
  record: fromEntries(zip(headers, row))
}
```

Also: `zipAll(arrays)` — zips any number of arrays, `unzip(pairs)` — reverse of `zip`.

50.23.28 `zipWith(arr1, arr2, fn) → array` (Arr)

Combine two arrays element-by-element using a merge function.

- `arr1` (required): first array
- `arr2` (required): second array
- `fn` (required): lambda (`elem1, elem2`) -> combined

```
zipWith([1, 2, 3], [10, 20, 30], (a, b) -> a + b) // [11, 22, 33]
```

50.23.29 `zipWithIndex(array) → array` (Arr)

Pair each element with its zero-based index.

- `array` (required): the array to index

```
zipWithIndex(["Apple", "Banana", "Cherry"]) // [["Apple", 0], ["Banana", 1], ["Cherry", 2]]
```

```
map(zipWithIndex($input.items), (pair) -> {
  lineNumber: pair[1] + 1,
  ...pair[0]
})
```

50.23.30 zipAll(arrays, pad?) → array (Bin)

Zip multiple arrays together, padding shorter arrays with null (or a specified value).

- **arrays** (required): array of arrays to zip
- **pad** (optional): padding value for shorter arrays

```
zipAll([[1, 2, 3], ["a", "b"]]) // [[1, "a"], [2, "b"], [3, null]]
```

50.23.31 zipArchive(entries) → binary (Bin)

Create a zip archive from an object of entries (name -> binary content).

- **entries** (required): object mapping file names to binary content

```
let archive = zipArchive({
  "data.json": toBinary(renderJson($input), "UTF-8"),
  "meta.txt": toBinary("generated", "UTF-8")
})
{
  archiveSize: binaryLength(archive)
}
```

50.24 Functions Not Listed Individually

The following function groups follow the same patterns as their listed counterparts. Use the MCP server (`get_function_info`) or the IDE's function browser for signatures and examples.

- **Geospatial (12)**: distance, bearing, midpoint, destinationPoint, boundingBox, inCircle, inPolygon, isValidCoordinates, geoBearing, geoDistance, geoMidpoint, geoBounds
- **Binary (12)**: binaryConcat, binaryEquals, binaryLength, binarySlice, binaryToString, readByte, readInt16, readInt32, readInt64, writeByte, writeInt16, writeInt32
- **URL (16)**: buildURL, parseURL, getHost, getPath, getPort, getProtocol, getQuery, getFragment, getQueryParams, buildQueryString, addQueryParam, removeQueryParam, getBaseURL, getURLPath, parseQueryString, getExtension
- **JWT/JWS (18)**: createJWT, verifyJWT, decodeJWT, getJWTClaim, getJWTClaims, getJWTIssuer, getJWTSubject, getJWTAudience, isJWTExpired, validateJWTStructure, decodeJWS, getJWSHeader, getJWSPayload, getJWSAlgorithm, getJWSKeyId, isJWSFormat, getJWSInfo, getJWSSigningInput
- **Encryption (6)**: encryptAES, decryptAES, encryptAES256, decryptAES256, generateKey, generateIV
- **Compression (6)**: compress, decompress, gzip, gunzip, deflate, inflate
- **Timer/Debug (14)**: timerStart, timerStop, timerStats, timerCheck, timerReset, timerClear, timerList, debug, debugPrint, trace, warn, info, log, measure
- **Tree (7)**: treeMap, treeFilter, treeFind, treeFlatten, treePaths, treeDepth, mapTree
- **Case conversion (12)**: camelCase, snakeCase, kebabCase, pascalCase, titleCase, dotCase, pathCase, constantCase, wordCase, slugify, fromCamelCase, fromSnakeCase
- **Math (12)**: pow, sqrt, exp, ln, log10, log2, sin, cos, tan, asin, acos, atan

- **Bitwise (6):** bitwiseAnd, bitwiseOr, bitwiseXor, bitwiseNot, shiftLeft, shiftRight
- **Financial (6):** simpleInterest, compoundInterest, presentValue, futureValue, calculateTax, calculateDiscount

Total: **692 functions** across 16 categories.

INDEX

1 Introduction	17
1.1 The Data Transformation Problem	17
1.2 What Is UTL-X?	17
1.3 The Swiss Army Knife Analogy	17
1.4 UTL-X vs The Alternatives	18
1.5 Architecture Overview	18
1.6 The UTL-X Ecosystem	18
1.7 A First Example	19
1.8 What's Ahead	20
2 Licensing: Open Source and Commercial	21
2.1 UTL-X Is Open Source (AGPL-3.0)	21
2.2 What You Can Do (Without Paying Anything)	21
2.3 The Key AGPL Condition	21
2.4 Your Transformations Are Yours	21
2.5 Usage Scenarios	22
2.6 The Subprocess Exception	22
2.7 The Commercial License	22
2.7.1 Why It Exists	22
2.7.2 Cloud Marketplace (Simplest Path)	22
2.7.3 Direct Commercial License	23
2.8 Dual Licensing: How It Works	23
2.9 Frequently Asked Questions	23
3 Transformation: The Cornerstone of Integration	24
3.1 The Three Pillars of Integration	24
3.2 Why Transformation Is the Hardest Pillar	24
3.3 Transformation in the Enterprise Service Bus	25
3.4 The JSON-to-XML-to-XSLT Anti-Pattern	25
3.5 The UTL-X Approach: No Intermediate Conversion	25
3.6 Why UTLXe Needs Messaging Capabilities	26
3.7 The Canonical Data Model	26
3.8 What This Means for You	26
4 Getting Started	28
4.1 Installation	28
4.1.1 macOS (Homebrew) — Recommended	28
4.1.2 Windows (Chocolatey)	28
4.1.3 Linux / macOS / Windows — Direct Download	28
4.1.4 Building from Source	29
4.1.5 Native Binary vs JVM JAR	29
4.2 Your First Transformation	29
4.3 The .utlx File Structure	30
4.3.1 Input Format Options	30
4.3.2 Output Format Options	31
4.3.3 Multi-Input	31
4.3.4 Multi-Input Syntax Rules	31
4.4 Trying It Out: Five Quick Examples	32
4.5 Your Own Functions	32
4.6 What's Next	33

5 UTL-X on the Command Line	34
5.1 Three Ways to Use the CLI	34
5.2 Expression Mode: The jq Experience	34
5.2.1 Raw Output (-r)	34
5.2.2 Expressions with Any Format	34
5.2.3 Complex Expressions	35
5.3 Identity Mode: The Flip	35
5.4 Shell Scripting Patterns	36
5.4.1 Batch File Conversion	36
5.4.2 Pipeline Integration	36
5.4.3 Using UTL-X in CI/CD	36
5.4.4 Combining with jq	37
5.5 The UTL-X jq Cheat Sheet	37
5.6 When to Use Which Mode	37
5.7 Performance Notes	37
6 The Three Executables: utlx, utlxd, and utlxe	39
6.1 One Language, Three Runtimes	39
6.2 utlx — The CLI	39
6.3 utlxd — The Daemon	40
6.4 utlxe — The Engine	40
6.5 Comparison at a Glance	42
6.6 The Wrapper Pattern	42
6.7 When to Use Which	42
7 The UTL-X IDE	44
7.1 Installing the VS Code Extension	44
7.2 Two Modes: IDE Runtime and IDE Design Time	44
7.2.1 IDE Runtime	44
7.2.2 IDE Design Time	45
7.2.2.1 What happens at runtime	45
7.2.2.2 What happens at design-time	45
7.2.2.3 Why this distinction matters	46
7.2.2.4 Tier 1 vs Tier 2: which mode applies	47
7.2.2.5 Summary: IDE Design Time vs IDE Runtime	48
7.2.2.6 What IDE Design Time provides	48
7.2.3 When to Use Which Mode	49
7.3 The Three-Panel Layout	49
7.4 The IDE vs UTLXe	50
7.5 The UDM Tree Browser	50
7.6 The Function Library	50
7.7 Real-Time Diagnostics	52
7.8 Autocompletion	52
7.9 The Header Editor	52
7.10 Keyboard Shortcuts	53
7.11 Tips for Productive IDE Usage	53
8 Language Fundamentals	54
8.1 Scalar Types	54
8.2 Objects	54
8.2.1 Object Literals	54

8.2.2	Property Access	55
8.2.3	Nested Access	55
8.2.4	XML Attribute Access	55
8.2.5	Wildcard Access	55
8.2.6	Bracket Notation: Indexing Only	56
8.2.7	Metadata Access (^)	56
8.2.8	Computed Property Names	57
8.2.9	The Spread Operator	57
8.2.9.1	Basic Usage	57
8.2.9.2	The “Keep Everything, Change Some” Pattern	57
8.2.9.3	Order Matters	58
8.2.9.4	Spread Is Shallow	58
8.2.9.5	Excluding Fields	58
8.3	Arrays	59
8.3.1	Array Literals	59
8.3.2	Index Access	59
8.3.3	Array Operations	59
8.4	The \$input Variable	59
8.4.1	Safe Navigation	60
8.4.2	Recursive Descent	60
8.5	Variable Binding (let)	60
8.5.1	Top-Level: No Separator Needed	61
8.5.2	Inside Object Literals: Commas Required	61
8.5.3	Inside Lambdas Returning Arrays: Semicolons Required	61
8.5.4	Summary of Let Separators	62
8.6	Conditional Expressions	62
8.6.1	if / else	62
8.6.2	Ternary Operator	62
8.6.3	Nullish Coalescing (??)	62
8.6.4	Combining Conditionals	62
8.7	Comments	63
8.8	Type Annotations (Optional)	63
8.9	Putting It Together	63
9	Operators	65
9.1	Arithmetic Operators	65
9.2	Comparison Operators	65
9.3	Logical Operators	66
9.3.1	Truthy and Falsy Values	66
9.4	Null Handling Operators	66
9.4.1	Nullish Coalescing (??)	66
9.4.2	Safe Navigation (?.)	67
9.5	Access and Navigation Operators	67
9.6	Data Construction Operators	68
9.6.1	Spread (...)	68
9.6.2	Computed Property Names ([expr]:)	68
9.7	Flow Operators	68
9.7.1	Pipe (>)	68
9.7.2	Lambda Arrows (-> and =>)	68
9.7.3	Binding (=)	69
9.8	String Concatenation	69

9.9	Operator Precedence	69
9.10	Reserved Operator	70
10	The Universal Data Model (UDM)	71
10.1	Why UDM Exists	71
10.2	UDM Types	71
10.3	How Formats Map to UDM	72
10.3.1	JSON → UDM	72
10.3.2	XML → UDM	72
10.3.3	CSV → UDM	73
10.3.4	YAML → UDM	73
10.3.5	OData → UDM	73
10.4	UDM Navigation	73
10.4.1	Dot Notation	73
10.4.2	Safe Navigation	74
10.4.3	Recursive Descent	74
10.5	UDM and Type Coercion	74
10.5.1	Automatic Coercion	74
10.5.2	Explicit Coercion	74
10.5.3	XML Text Node Unwrapping	74
10.6	UDM is Format-Agnostic, Transformations are Format-Agnostic	75
10.7	UDM and Flat/Relational Data	75
10.7.1	The Flat Data Challenge	75
10.7.2	How to Handle It in UTL-X	75
10.7.3	Hierarchical vs Relational: A Design Trade-Off	76
10.8	UDM and Schema Formats (Tier 2)	76
10.8.1	Combined Transformations: Tier 1 + Tier 2	77
10.9	When UDM Matters	78
11	Schema-to-Schema Mapping	79
11.1	Why Schema Mapping Matters	79
11.2	How Traditional Tools Do It	79
11.2.1	The XSLT Approach	79
11.2.2	The JSON Problem	80
11.3	The UTL-X Approach	81
11.4	Tier 1: Instance Data Mapping	82
11.4.1	Every Data Format Has an Accompanying Schema Format	82
11.4.2	Common Instance Mapping Scenarios	82
11.5	Tier 2: Metadata Mapping (Schema-to-Schema)	83
11.5.1	Why Metadata Mapping Is Powerful	83
11.5.2	The Full Schema-to-Schema Matrix	83
11.5.3	Practical Use Cases for Schema Mapping	83
11.6	Nested Schemas: Include and Import	84
11.6.1	XSD: Include vs Import	84
11.6.2	Other Format Mechanisms	84
11.6.3	How UTL-X Handles Nested Schemas	85
11.7	JSON Schema to JSON Schema Mapping	85
11.8	Schema-Driven Validation in UTL-X	85
11.8.1	Pre-Validation	85
11.8.2	Post-Validation	85

11.8.3	The Validation Orchestrator	86
11.9	The Bridge: USDL	86
12	USDL: Universal Schema Definition Language	87
12.1	What USDL Is	87
12.2	How USDL Works — Automatically	87
12.2.1	1. Enrichment (Input — Tier 2 Schema Formats)	87
12.2.2	2. Detection (Output — Tier 2 Schema Serializers)	87
12.2.3	3. Carrier (YAML/JSON as Human-Readable Schema)	88
12.3	The USDL Directive Classification	88
12.4	Core Directives (Universal)	88
12.4.1	Example: Customer schema using only Core directives	89
12.5	Shared Directives (Most Formats) (80%+ Format Support)	89
12.5.1	Example: Customer with Core + Shared constraints	90
12.6	Format-Native Directives	90
12.6.1	Protobuf-Specific (proto)	90
12.6.2	Avro-Specific (avro)	91
12.6.3	XSD-Specific (xsd)	92
12.6.4	Database/SQL-Specific (future — no DDL output format yet)	92
12.6.5	OData-Specific (osch)	93
12.6.6	JSON Schema-Specific (jsch)	94
12.6.7	Table Schema-Specific (tsch — Frictionless Data)	94
12.7	How Directive Levels Interact During Conversion	94
12.8	Converting Between Formats via USDL	95
12.8.1	XSD to JSON Schema	95
12.8.2	Avro to Protobuf	95
12.8.3	Any Schema to Human-Readable YAML	95
12.9	USDL for Validation	95
12.10	USDL as Documentation	95
12.11	USDL vs Standalone Schema Languages	96
13	Format Support Overview	97
13.1	Tier 1: Data Formats	97
13.1.1	JSON	97
13.1.2	XML	97
13.1.3	CSV	97
13.1.4	YAML	97
13.1.5	OData	98
13.2	Tier 2: Schema Formats	98
13.2.1	XSD (XML Schema Definition)	98
13.2.2	JSCH (JSON Schema)	98
13.2.3	Avro	98
13.2.4	Protobuf (Protocol Buffers)	98
13.2.5	OSCH (OData Schema / EDMX)	98
13.2.6	TSCH (Table Schema)	98
13.3	Format Detection	98
13.4	Format Conversion Matrix	99
13.5	Identity Mode Defaults	99
13.6	Format Options Summary	99
13.7	BOM (Byte Order Mark)	100
13.8	What's Next	100

14 Expressions and Operators	102
14.1 Arithmetic Operators	102
14.2 Comparison Operators	102
14.3 Logical Operators	102
14.3.1 Truthy and Falsy Values	103
14.4 String Concatenation	103
14.5 Object Construction	103
14.5.1 Conditional Properties	104
14.6 Array Construction	104
14.7 The Pipe Operator (<code> ></code>)	104
14.8 Nullish Coalescing (<code>??</code>)	105
14.9 Safe Navigation (<code>?.</code>)	105
14.10 Operator Precedence	105
14.11 Practical Expression Patterns	106
14.11.1 Price Calculation	106
14.11.2 Conditional Formatting	106
14.11.3 Data Normalization	106
14.12 Complete Operator Reference	107
14.12.1 Arithmetic	107
14.12.2 Comparison	107
14.12.3 Logical	107
14.12.4 Null Handling	107
14.12.5 Access and Navigation	107
14.12.6 Data Construction	108
14.12.7 Function and Flow	108
14.12.8 Reserved (Not Yet Implemented)	108
15 Functions and Lambdas	109
15.1 Lambda Expressions	109
15.1.1 Basic Syntax	109
15.1.2 Multiple Parameters	109
15.1.3 No Parameters	109
15.1.4 Lambdas in Action	109
15.2 User-Defined Functions	110
15.2.1 The PascalCase Rule	110
15.2.2 Function Definitions Are Expressions	111
15.2.3 Recursive Functions	111
15.3 Higher-Order Functions	111
15.3.1 <code>map</code> — Transform Each Element	112
15.3.2 <code>filter</code> — Select Matching Elements	113
15.3.2.1 Basic Examples	113
15.3.2.2 Common Anti-Pattern: Array Predicate Syntax	113
15.3.2.3 Common Anti-Pattern: Filtering Then Counting	113
15.3.2.4 Common Anti-Pattern: Filter + Map When You Need Both	114
15.3.2.5 Multi-Condition Filters	114
15.3.2.6 Negation: Exclude Instead of Include	114
15.3.2.7 Filter with Safe Navigation	114
15.3.2.8 Filter Returns an Array (Always)	115
15.3.2.9 Lambda Return Types: Objects vs Bare Expressions	115
15.3.3 <code>reduce</code> — Accumulate a Result	116
15.3.3.1 MapReduce in UTL-X	116

15.3.4	find — First Match	117
15.3.5	sortBy — Order by Computed Value	117
15.3.6	groupBy — Group by Key	117
15.4	Function Composition	117
15.4.1	Pipe Chains	117
15.4.2	Nested Functions	118
15.4.3	Combining map and filter	118
15.5	Closures	118
15.6	Practical Patterns	119
15.6.1	Flatten and Restructure	119
15.6.2	Aggregate with Multiple Results	119
15.6.3	Lookup Table Pattern	119
15.6.4	Conditional Transformation	120
15.7	The 652 Standard Library Functions	120
16	Standard Library Overview	121
16.1	String Functions (83 functions)	121
16.1.1	Most-Used String Functions	121
16.1.2	Case Conversion	121
16.1.3	Padding and Formatting	121
16.2	Array Functions (67 functions)	121
16.2.1	Transform and Filter	122
16.2.2	Search and Access	122
16.2.3	Aggregate	122
16.2.4	Reshape	122
16.3	Math Functions (37 functions)	122
16.4	Date/Time Functions (68 functions)	123
16.4.1	Current Time	123
16.4.2	Parse and Format	123
16.4.3	Arithmetic	123
16.4.4	Extract Components	123
16.5	Type Functions (27 functions)	123
16.5.1	Type Checking	123
16.5.2	Type Conversion	123
16.6	Encoding Functions (30 functions)	123
16.7	Security Functions (16 functions)	124
16.8	XML Functions (60 functions)	124
16.8.1	QName Functions (9)	124
16.8.2	Canonicalization Functions (11)	124
16.9	Other Categories	125
16.9.1	JSON Functions (6)	125
16.9.2	CSV Functions (12)	125
16.9.3	YAML Functions (22)	125
16.9.4	Financial Functions (16)	125
16.9.5	Geospatial Functions (8)	125
16.9.6	Binary Functions (47)	125
16.9.7	Utility Functions (27)	125
16.9.8	Object Functions (1)	126
16.10	Finding the Right Function	126
16.10.1	In the CLI	126
16.10.2	In the IDE	126

16.10.3 In This Book	126
16.11 A Note on Naming	126
17 Security Library and Message Security	127
17.1 The Security Library (stdlib/crypto)	127
17.1.1 Available Functions	127
17.1.2 Hashing	127
17.1.3 HMAC (Hash-based Message Authentication)	127
17.2 Transport Security: mTLS	128
17.3 Message-Level Encryption	128
17.3.1 When It's Required	128
17.3.2 The Encrypt-Transform-Re-encrypt Pattern	128
17.3.3 Why This Is a Maintenance Burden	128
17.3.4 Recommendation	129
17.4 Digital Signatures and Integrity	129
17.4.1 When You Need Signatures	129
17.4.2 Impact on Transformation	129
17.4.3 What UTL-X Can Do Today	129
17.4.4 What UTL-X Already Has	129
17.4.5 What UTL-X Cannot Do Yet (Future)	129
17.5 Large Files and Sensitive Data	130
17.5.1 The Claim Check Pattern	130
17.5.2 Avoid Base64 Embedding	130
17.6 What NOT to Log	130
18 Pattern Matching and Control Flow	131
18.1 Match Expressions	131
18.1.1 Basic Value Matching	131
18.1.2 Matching with Guards	131
18.1.3 Complex Matching	131
18.1.4 Match vs If/Else Chains	131
18.2 Try/Catch — Error Recovery	132
18.2.1 Basic Try/Catch	132
18.2.2 Using the Error Variable	132
18.2.3 Try/Catch in Map	132
18.2.4 When to Use Try/Catch vs Null Coalescing	133
18.3 Practical Control Flow Patterns	133
18.3.1 Message Type Routing	133
18.3.2 API Version Handling	133
18.3.3 Country-Specific Business Rules	134
18.3.4 Data Cleansing with Fallbacks	134
18.3.5 Polymorphic Output (Different Structure Based on Input)	134
18.4 Control Flow Summary	135
19 Schema Validation	136
19.1 Where Validation Lives — and Why	136
19.1.1 The CLI Must Accept Anything	136
19.1.2 UTLXe Operates in Middleware Where Validation Is Vital	136
19.1.3 The Right Default in the Right Executable	137
19.2 Why Validate?	137
19.3 Supported Schema Formats	137

19.4	The Validation Orchestrator	138
19.4.1	Configuring Validation	138
19.4.2	CLI Static Validation	139
19.5	Validation Policies	139
19.6	JSON Schema Validation	139
19.6.1	What JSON Schema Checks	139
19.6.2	Validation Errors	140
19.7	XSD Validation	140
19.8	Syntactical vs Semantic Validation	140
19.8.1	What Syntactical Validation Catches	140
19.8.2	What Syntactical Validation Cannot Catch	140
19.8.3	The Gap	140
19.9	Validation in Practice	141
19.9.1	Development Workflow	141
19.9.2	Production Workflow	141
19.9.3	When to Skip Validation	141
19.10	Validation and the Conformance Suite	141
20	Pipeline Chaining and Multi-Hop Transformations	142
20.1	Single-Hop vs Multi-Hop	142
20.1.1	Single-Hop (Simple Transformation)	142
20.1.2	Multi-Hop (Pipeline)	142
20.2	Why Pipelines?	142
20.2.1	Separation of Concerns	142
20.2.2	Reusability	143
20.2.3	Testability	143
20.2.4	Maintainability	143
20.3	Multi-Input Transformations (Single Step)	143
20.4	Defining Pipelines	143
20.4.1	Pipeline via CLI (Unix Pipes)	143
20.4.2	Pipeline via UTLXe (In-Process)	143
20.5	Pipeline Patterns	144
20.5.1	Enrichment Pipeline	144
20.5.2	Validation Sandwich	144
20.5.3	Fan-Out	144
20.5.4	Fan-In (Aggregation)	144
20.5.5	Conditional Routing	145
20.6	Multi-Hop with Additional Inputs per Stage	145
20.6.1	The Problem	145
20.6.2	Use Cases	145
20.6.3	Status	145
20.7	Performance Characteristics	145
21	Data Restructuring: Flat, Hierarchical, and Back	147
21.1	The Six Restructuring Patterns	147
21.2	Pattern 1: Flat → Hierarchical (join)	147
21.2.1	The Problem	147
21.2.2	The Solution: nestBy()	147
21.2.3	Today's Workaround (without join)	148
21.3	Pattern 2: Record Enrichment (lookupBy)	148
21.3.1	The Problem	148

21.3.2	The Solution: <code>lookupBy()</code>	148
21.3.3	Today's Workaround (without <code>lookupBy</code>)	149
21.4	Pattern 3: Sequential Grouping (<code>chunkBy</code>)	149
21.4.1	The Problem	149
21.4.2	The Solution: <code>chunkBy()</code>	149
21.4.3	Today's Workaround (without <code>chunkBy</code>)	149
21.5	Pattern 4: Denormalization (<code>unnest</code>)	149
21.5.1	The Problem	149
21.5.2	The Solution: <code>unnest()</code>	150
21.5.3	Today's Workaround (without <code>unnest</code>)	150
21.6	Pattern 5: Key-Based Grouping (<code>groupBy</code> — exists)	150
21.7	Pattern 6: Array Flattening (<code>flatten</code> — exists)	150
21.8	Combining Patterns	151
21.8.1	IDoc → Enriched Hierarchical JSON	151
21.8.2	Hierarchical → Flat CSV Report	151
21.9	Choosing the Right Pattern	152
21.10	Naming Convention: Why Some Functions End in “By”	152
22	XML Transformations	154
22.1	Reading XML	154
22.1.1	Element Access	154
22.1.2	Attribute Access	154
22.1.3	Repeated Elements	154
22.1.4	The Single-Element Problem	155
22.1.5	Nested Access	155
22.1.6	Namespace-Prefixed Elements	155
22.2	Writing XML	156
22.2.1	Basic Element Output	156
22.2.2	Creating Attributes	156
22.2.3	Self-Closing Elements	156
22.2.4	Namespace Declarations	157
22.2.5	Encoding	157
22.3	XML Encoding Detection and Conversion	157
22.4	XML Canonicalization (C14N)	157
22.4.1	Hashing and Comparison	158
22.4.2	Round-Trip Verification	158
22.5	QName Functions (Qualified Names)	158
22.5.1	XBRL (Financial Reporting)	159
22.6	XML Attributes in JSON/YAML Output	160
22.7	Common XML Patterns	160
22.7.1	SOAP (Simple Object Access Protocol)	160
22.7.1.1	Extracting the Business Payload	160
22.7.1.2	Accessing SOAP Headers	160
22.7.1.3	SOAP 1.1 vs 1.2	161
22.7.1.4	Handling SOAP Faults	161
22.7.1.5	Constructing SOAP Requests	162
22.7.2	UBL Invoice (Peppol BIS 3.0)	163
22.7.3	HL7 FHIR XML	163
22.7.4	SAP IDoc XML	163
22.8	CDATA Sections	163
22.8.1	How UTL-X Handles CDATA	164

22.8.2	XML Inside CDATA	164
22.9	XML Comments and Processing Instructions	165
22.9.1	Comments	165
22.9.2	Processing Instructions	165
22.10	XML Documentation (xs:annotation)	166
22.11	XML Round-Trip	166
23	XML Attribute Handling: Design Decisions	167
23.1	How UTL-X Handles Attributes Internally	167
23.2	The <code>_text</code> Convention	167
23.3	The <code>writeAttributes</code> Option	167
23.3.1	Default Behavior (<code>writeAttributes: false</code>)	168
23.3.2	With <code>writeAttributes: true</code>	168
23.4	What Gets Kept vs Dropped	168
23.5	Why Drop by Default?	168
23.6	JSON and YAML Consistency	169
23.7	Industry Conventions	169
23.8	FHIR XML: The value Attribute Convention	169
24	JSON Transformations	171
24.1	JSON and UDM: A Natural Fit	171
24.1.1	What You See Is What You Get	171
24.2	Reading JSON	172
24.2.1	Property Access	172
24.2.2	Safe Navigation	172
24.2.3	Number Precision	172
24.3	Writing JSON	172
24.3.1	Object Construction	172
24.3.2	Array Construction	173
24.3.3	Spread Operator	173
24.3.4	Conditional Fields	173
24.3.5	Pretty Printing vs Compact Output	174
24.4	JSON-Specific Functions	174
24.4.1	<code>parseJson</code> and <code>renderJson</code>	174
24.4.2	JSON Canonicalization (RFC 8785)	175
24.4.3	<code>parse</code> and <code>render</code> (Generic)	175
24.5	Common JSON Patterns	175
24.5.1	REST API Response Transformation	175
24.5.2	Flattening Nested API Responses	176
24.5.3	Key Renaming (<code>snake_case</code> to <code>camelCase</code>)	176
24.5.4	Merging Multiple JSON Objects	176
24.5.5	Filtering and Reshaping Arrays	177
24.5.6	Building Lookup Maps	177
24.5.7	Aggregation	177
24.6	JSON to Other Formats	178
24.6.1	JSON to XML	178
24.6.2	JSON to CSV	178
24.6.3	JSON to YAML	179
24.7	JSON Schema	179
24.7.1	Reading JSON Schema	179
24.7.2	Writing JSON Schema	180

24.7.3	Schema Functions	180
24.8	When JSON Isn't Simple	180
24.8.1	Dates Are Strings	180
24.8.2	Numbers Can Be Ambiguous	180
24.8.3	Null vs Missing vs Empty	180
24.8.4	Deeply Nested Optional Structures	181
25	CSV Transformations	182
25.1	CSV and UDM	182
25.2	CSV Options	182
25.2.1	Input Options	182
25.2.2	Output Options	183
25.3	Regional Number Formats	183
25.3.1	Output Formatting	183
25.3.2	Input Parsing	184
25.4	Type Inference	184
25.5	Reading CSV	185
25.5.1	Basic CSV to JSON	185
25.5.2	BOM Handling	185
25.5.3	Quoting and Special Characters	185
25.5.4	CSV Without Headers	186
25.5.5	Tab-Separated Values (TSV)	186
25.5.6	Semicolon-Separated (European)	186
25.6	Writing CSV	186
25.6.1	JSON to CSV	186
25.6.2	Column Ordering	187
25.6.3	Flattening Nested JSON for CSV	187
25.6.4	CSV with BOM for Excel	187
25.6.5	Suppressing Headers	187
25.7	Common CSV Patterns	188
25.7.1	Bank Statement Processing (European)	188
25.7.2	Log File Processing (Pipe-Delimited)	188
25.7.3	CSV Aggregation	188
25.7.4	XML to CSV (Invoice Lines)	189
25.7.5	Multi-Format Pipeline: CSV + JSON Enrichment	189
25.8	CSV-Specific Functions	189
25.8.1	parseCsv	189
25.8.2	Spreadsheet Functions	189
25.9	Edge Cases	190
25.9.1	Empty CSV	190
25.9.2	Single Row	190
25.9.3	Whitespace Handling	190
26	YAML Transformations	191
26.1	YAML and UDM	191
26.1.1	Anchors and Aliases	191
26.1.2	Multi-Document Streams	191
26.2	Reading YAML	192
26.2.1	Timestamps	192
26.2.2	Boolean Variants	192
26.3	Writing YAML	192

26.3.1	Block Style (Default)	192
26.3.2	Number Formatting	193
26.3.3	XML Attributes in YAML	193
26.4	YAML Output Options	193
26.4.1	The --- Document Start Marker	193
26.5	YAML-Specific Functions	194
26.5.1	parseYaml	194
26.5.2	Multi-Document Functions	194
26.5.3	Query and Modify Functions	194
26.5.4	Deep Merge	194
26.6	Dynamic Keys	194
26.6.1	The Problem	195
26.6.2	Reading Dynamic Keys	195
26.6.3	Iterating Over Dynamic Keys	195
26.6.4	Generating Dynamic Keys (Output)	196
26.6.5	Computed Property Names	197
26.6.6	Dynamic Keys Summary	197
26.7	Common YAML Patterns	198
26.7.1	Kubernetes Manifest Transformation	198
26.7.2	Docker Compose Environment Injection	198
26.7.3	Configuration File Migration (JSON to YAML)	199
26.7.4	OpenAPI Spec Processing	199
26.7.5	Helm Values Override	199
26.8	YAML vs JSON: When to Use Which	199
27	OData Transformations	201
27.1	OData v4 JSON and UDM	201
27.1.1	Collection Responses	202
27.2	Reading OData	202
27.2.1	Dynamics 365 API Responses	202
27.2.2	SharePoint List Items	203
27.2.3	SAP OData Services	203
27.3	Writing OData	203
27.3.1	Output Options	203
27.3.2	Metadata Levels	203
27.3.3	Generating OData for Dynamics 365 Import	204
27.4	OData Schema (EDMX/CSDL)	204
27.4.1	What EDMX Contains	204
27.4.2	Cross-Format Schema Conversion	205
27.5	Common OData Patterns	205
27.5.1	Stripping Metadata	205
27.5.2	OData to CSV for Reporting	205
27.5.3	OData to XML for Legacy Systems	206
27.5.4	Pagination Handling	206
27.6	OData vs Standard JSON: When to Use Which	206
28	Schema Formats	208
28.1	The USDL Bridge	208
28.2	XSD (XML Schema Definition)	208
28.2.1	XSD Features Supported	209
28.2.2	XSD Output Options	209

28.2.3	Stdlib Functions	209
28.3	JSON Schema	209
28.3.1	Supported Drafts	209
28.3.2	JSON Schema Features Supported	209
28.3.3	Stdlib Functions	210
28.4	Avro Schema	210
28.4.1	Avro Features Supported	210
28.4.2	Schema Evolution	210
28.4.3	Stdlib Functions	210
28.5	Protobuf (Protocol Buffers)	210
28.5.1	Protobuf Features Supported	210
28.5.2	Field Number Preservation	211
28.5.3	Stdlib Functions	211
28.6	OData Schema (EDMX/CSDL)	211
28.7	Table Schema (TSCH)	211
28.7.1	Table Schema Features Supported	211
28.8	Schema-to-Schema Conversion Examples	212
28.8.1	XSD to JSON Schema	212
28.8.2	Avro to Protobuf	212
28.8.3	JSON Schema to XSD	212
28.8.4	Any Schema to Human-Readable YAML	212
28.9	Schema Functions Summary	213
29	XSD Design Patterns Deep Dive	214
29.1	Why Patterns Matter	214
29.2	The Five Core Patterns	214
29.2.1	1. Russian Doll	214
29.2.2	2. Salami Slice	215
29.2.3	3. Venetian Blind	215
29.2.4	4. Garden of Eden	216
29.2.5	5. Swiss Army Knife (Hybrid)	216
29.3	Advanced Patterns	217
29.3.1	Chameleon Include	217
29.3.2	Abstract Types	217
29.4	The Pattern Matrix	218
29.5	Pattern Detection	218
29.5.1	Accessing Pattern Metadata with $\wedge$	219
29.6	Pattern Conversion	219
29.7	Patterns in Cross-Format Conversion	220
29.8	Real-World XSD Architectures	220
29.8.1	UBL (Universal Business Language)	220
29.8.2	ISO 20022 (Financial Messaging)	220
29.8.3	HL7 CDA (Clinical Document Architecture)	220
29.9	XSD Patterns in Integration Tools	220
30	Cross-Format Patterns	222
30.1	The Power of Format Agnosticism	222
30.2	XML to JSON	222
30.2.1	Pass-Through (Identity)	222
30.2.2	What Happens to XML Features in JSON	222
30.2.3	Reshaping During Conversion	223

30.3	JSON to XML	223
30.3.1	Pass-Through	223
30.3.2	Creating Attributes	223
30.3.3	Adding Namespaces	224
30.4	CSV to JSON	224
30.5	JSON to CSV	225
30.5.1	Flattening Nested JSON	225
30.6	XML to CSV	225
30.7	YAML to JSON and JSON to YAML	226
30.8	XML to YAML and YAML to XML	226
30.9	Multi-Format Integration Flows	226
30.9.1	Order Processing: XML + CSV + JSON to UBL XML	227
30.9.2	Data Lake Export: OData to CSV + JSON + YAML	227
30.10	The Canonical Data Model Pattern	228
30.11	Format Conversion Quick Reference	228
31	Enterprise Integration	230
31.1	Where UTL-X Fits	230
31.1.1	UTL-X vs Full iPaaS	230
31.2	Use Case: European E-Invoicing (Peppol/UBL)	230
31.2.1	The Flow	230
31.2.2	The Transformation	231
31.3	Use Case: Healthcare (HL7 FHIR)	232
31.3.1	FHIR's Value Attribute Convention	232
31.3.2	Proprietary EHR to FHIR R4	232
31.4	Use Case: Financial Services (ISO 20022)	233
31.4.1	Payment Initiation (pain.001)	233
31.5	Use Case: Retail and E-Commerce	234
31.5.1	Order Synchronization	234
31.6	Workflow and Low-Code Integration	234
31.6.1	Pattern: HTTP Action in a Workflow	235
31.6.2	Azure Logic Apps / Power Automate	235
31.6.3	n8n / Make / Zapier	235
31.6.4	Why Not Transform Inside the Workflow?	235
31.7	RPA (Robotic Process Automation)	236
31.7.1	The Pattern: RPA Extracts, UTL-X Transforms	236
31.7.2	UiPath Integration	236
31.7.3	Why RPA Needs External Transformation	236
31.7.4	Common RPA + UTL-X Scenarios	237
31.8	Message Broker Integration	237
31.8.1	Azure Service Bus	237
31.8.2	GCP Pub/Sub	237
31.8.3	Apache Kafka	237
31.8.4	AWS SQS and EventBridge	237
31.8.5	JMS (Java Message Service)	238
31.8.6	IBM MQ with UTL-X	239
31.8.6.1	Deployment Options	239
31.8.6.2	Replacing IBM Integration Bus (IIB/ACE)	240
31.8.7	RabbitMQ	241
31.9	Error Handling in Production	241
31.9.1	Validation Errors	241

31.9.2	Transformation Errors	241
31.9.3	Monitoring	241
31.9.4	Alerting Rules	241
32	Engine Lifecycle: Design-Time, Init-Time, Runtime	243
32.1	Phase 1: Design-Time	243
32.2	Phase 2: Init-Time (Load and Compile)	243
32.2.1	Compilation Cache	244
32.2.2	Object Pool (COPY Strategy)	244
32.3	Phase 3: Runtime (Execute)	244
32.4	How Strategies Compare	245
32.4.1	The COPY + COMPILED Hybrid	245
32.5	The Three Phases Visualized	245
32.6	Worker Threading Model	246
32.6.1	Back-Pressure	246
32.6.2	Tuning Workers	246
32.6.3	Thread Safety	246
33	Cloud Deployment	247
33.1	The Docker Image	247
33.1.1	Transport Modes	247
33.1.2	Basic Docker Run	247
33.2	Azure Deployment	247
33.2.1	Azure Container Apps	247
33.2.2	Marketplace Plans	248
33.2.3	Deployment via Azure Portal	248
33.2.4	Deployment via CLI	248
33.3	GCP Deployment	248
33.3.1	Cloud Run	248
33.3.2	Pub/Sub Integration	248
33.3.3	Terraform Module	249
33.4	AWS Deployment	249
33.4.1	ECS/Fargate	249
33.4.2	Pricing Note	249
33.5	Kubernetes (Any Cloud)	249
33.5.1	Deployment	249
33.5.2	Horizontal Pod Autoscaler	250
33.5.3	Dapr on Kubernetes	250
33.6	Health and Monitoring	251
33.6.1	Health Endpoints	251
33.6.2	Prometheus Metrics	251
33.6.3	Grafana Dashboard	251
33.6.4	Integration with Cloud Monitoring	251
33.7	Hot Reload	252
33.7.1	Bundle Loading (Startup)	252
33.7.2	Dynamic Loading (Runtime)	252
33.7.3	Stateless Design	252
34	SDKs and Wrapper Libraries	253
34.1	The Wrapper Pattern	253
34.1.1	Why Stdio Instead of HTTP?	253

34.2	Available SDKs	253
34.2.1	C# / .NET Wrapper	253
34.2.2	Go Wrapper	254
34.2.3	Java / Kotlin	255
34.2.3.1	Option 1: Direct API (In-Process)	255
34.2.3.2	Option 2: Java Stdio Wrapper (Decoupled)	255
34.2.3.3	Which to Choose?	255
34.3	The Uniform SDK Interface	256
34.3.1	Python Wrapper (Planned)	256
34.3.2	Node.js / TypeScript Wrapper (Planned)	256
34.4	Alternative: HTTP API (No SDK Needed)	256
34.5	Alternative: Dapr Sidecar (Cloud-Native)	257
34.6	SDK Comparison	257
34.6.1	Choosing the Right Integration	257
35	Migration Guides	258
35.1	Migrating from XSLT	258
35.1.1	Side-by-Side Example	258
35.1.2	What XSLT Does That UTL-X Doesn't	259
35.2	Migrating from MuleSoft DataWeave	259
35.2.1	Key Differences	259
35.2.2	MuleSoft Migration Strategy	260
35.3	Migrating from jq	260
35.3.1	What jq Does That UTL-X Doesn't	260
35.3.2	What UTL-X Does That jq Doesn't	260
35.4	Migrating from TIBCO BusinessWorks	261
35.4.1	XSD Pattern Consideration	261
35.4.2	Performance Comparison	261
35.5	Migrating from Custom Code	261
35.5.1	When to Migrate	261
35.5.2	Hybrid Approach	262
36	Performance and Optimization	263
36.1	Execution Strategies	263
36.1.1	Why COMPILED Is Fast	263
36.2	Memory Model	263
36.2.1	UDM Expansion Factor	263
36.2.2	Heap Sizing	264
36.2.3	Sizing Guide	264
36.3	GraalVM Native Image	264
36.3.1	CLI: Native Binary	264
36.3.2	Engine: JVM Only	264
36.4	Benchmarking	265
36.4.1	Quick Benchmark	265
36.4.2	What to Measure	265
36.4.3	Conformance Suite as Benchmark	265
36.5	Production Tuning	265
36.5.1	Worker Count	265
36.5.2	Horizontal vs Vertical Scaling	266
36.5.3	GC Tuning	266
36.5.4	Back-Pressure	266

36.6	Performance Anti-Patterns	266
36.6.1	Avoid: filter() Inside map()	266
36.6.2	Avoid: Repeated Computation	267
36.6.3	Avoid: Large String Concatenation in Loops	267
37	Message Parsing, Memory, and Size Boundaries	268
37.1	How UTL-X Parses Messages	268
37.1.1	DOM-Style Parsing	268
37.1.2	Why Not Streaming?	268
37.2	Memory Expansion Factor	268
37.2.1	Why 100KB of XML Becomes 5MB in Memory	268
37.2.2	Expansion by Format	269
37.2.3	Worked Example: A 50 KB UBL Invoice	269
37.2.4	Real-World Memory Examples	270
37.3	Message Size Boundaries	270
37.3.1	Sweet Spot: Under 1 MB	270
37.3.2	Feasible: 1 MB to 10 MB	270
37.3.3	Not Recommended: 10 MB to 50 MB	271
37.3.4	Not Feasible: Over 50 MB	271
37.3.5	“Why Not Just Use 1 GB Messages?”	271
37.3.6	CPU Time: Linear, Not Exponential	271
37.3.7	The Real Boundaries	272
37.3.8	When to Split Instead of Grow	272
37.4	The Poison Message Problem	272
37.4.1	Defense: Input Size Limits	273
37.4.2	Defense: Worker Isolation (Future)	273
37.5	Swap: Never Use It	273
37.5.1	Container Configuration	274
37.6	Memory Sizing Formula	274
37.6.1	The 75% Rule	274
37.7	The Split-Before-Transform Pattern	275
37.8	UTL-X vs Streaming and Big Data Tools	275
38	Logging, Observability, and Compliance	276
38.1	What UTL-X Logs	276
38.1.1	Default Behavior (INFO Level)	276
38.1.2	Log Levels	276
38.1.3	Configuring Logging	276
38.2	When NOT to Log: Compliance Requirements	277
38.2.1	PCI DSS (Payment Card Industry)	277
38.2.2	GDPR (General Data Protection Regulation)	277
38.2.3	HIPAA (Health Insurance Portability and Accountability Act)	277
38.2.4	NEN 7510 (Dutch Healthcare Security)	278
38.2.5	SOX (Sarbanes-Oxley — Financial Reporting)	278
38.3	Designing for Compliance	278
38.3.1	Principle: Log Metadata, Not Data	278
38.3.2	Audit Logging Pattern	278
38.3.3	Masking in Error Messages	279
38.4	Prometheus Metrics: Safe by Design	279
38.5	The Security Library Connection	279

39	Quality Assurance and Testing	280
39.1	The Five Testing Layers	280
39.2	Layer 1: Unit Tests	280
39.3	Layer 2: Format Tests	280
39.4	Layer 3: Module Tests	280
39.5	Layer 4: Integration Tests	281
39.6	Layer 5: Conformance Suite	281
39.6.1	Why Python, Not Kotlin?	281
39.6.2	Test Format	281
39.6.3	Running the Suite	282
39.7	Auto-Capture: Recording Tests from Usage	282
39.8	UTLXe Engine Tests	282
39.9	Test-Driven Transformation Development	283
39.10	Continuous Integration	283
39.10.1	CLI Pipeline (<code>cli-ci.yml</code>)	283
39.10.2	Release Pipeline (<code>release.yml</code>)	283
39.10.3	Pull Request Checks	284
40	Semantic Validation: Beyond Syntax	286
40.1	The Gap Between Syntax and Meaning	286
40.2	What Schemas Cannot Express	286
40.3	Semantic Validation Languages in Industry	286
40.3.1	Schematron (XML)	286
40.3.2	FEEL (Friendly Enough Expression Language)	287
40.3.3	Other Approaches	287
40.4	How Semantic Validation Could Work in UTL-X	287
40.4.1	The Proposed Approach: Native Assertions	287
40.4.2	Why This Works	288
40.4.3	Integration with the Validation Orchestrator	288
40.5	Semantic Validation in Practice Today	289
40.6	Common Semantic Validation Rules	289
40.6.1	Financial	289
40.6.2	Healthcare	289
40.6.3	E-Invoicing (Peppol)	289
40.6.4	Logistics	289
40.7	The Schematron Bridge	290
40.8	Status	290
41	API Contracts and Data Contracts	291
41.1	Two Dimensions of Contracts	291
41.1.1	Hierarchical Contracts (API-Centric)	291
41.1.2	Relational Contracts (Data-Centric)	291
41.1.3	The Key Difference	291
41.2	Hierarchical API Contract Standards	292
41.2.1	OpenAPI (REST APIs)	292
41.2.2	SOAP / WSDL (XML Web Services)	292
41.2.3	AsyncAPI (Event-Driven APIs)	292
41.2.4	gRPC / Protocol Buffers	292
41.3	Abstract vs Concrete Contracts	292
41.3.1	Abstract Contracts (Schema-Only)	293
41.3.2	Concrete Contracts (Full API Specification)	293

41.3.3	The Relationship	293
41.4	Relational Data Contract Standards	293
41.4.1	Data Contract (datacontract.com)	293
41.4.2	ODCS (Open Data Contract Standard — PayPal)	294
41.4.3	Comparison	294
41.5	How UTL-X Fits in the Contract Landscape	294
41.5.1	UTL-X as the Transformation Layer	294
41.5.2	Schema Format Coverage	294
41.5.3	Contract-Driven Transformation Development	295
41.5.4	Data Contract Processing	295
41.6	The Contract-Driven Future	295
41.6.1	Contract Differencing	295
41.6.2	Contract as Test Oracle	296
42	Formats Not Yet Covered	297
42.1	Medium Priority: EDI / EDIFACT	297
42.2	Medium Priority: HL7 v2	297
42.3	Medium Priority: Parquet / Arrow	298
42.4	Low Priority: TOML	298
42.5	Low Priority: JSON-LD	298
42.6	Low Priority: MessagePack / CBOR	298
42.7	Low Priority: GraphQL SDL	299
42.8	Low Priority: ORC (Apache Optimized Row Columnar)	299
42.9	Low Priority: RAML	299
42.10	Potential: JSON Sample Generation (Mock Data)	299
42.11	Declining: Apache Thrift / FlatBuffers	300
42.12	Format Priority Matrix	300
43	Competitive Landscape	301
43.1	Command-Line Transformation Tools	301
43.1.1	jq	301
43.1.2	yq	301
43.1.3	JSONata	301
43.1.4	XSLT	301
43.2	Enterprise Integration Platforms (iPaaS)	302
43.2.1	MuleSoft / DataWeave	302
43.2.2	TIBCO BusinessWorks	302
43.2.3	Boomi	302
43.2.4	Informatica / IICS	302
43.3	Hyperscaler Native Tools	302
43.3.1	Azure	303
43.3.2	AWS	303
43.3.3	GCP	303
43.4	ERP Vendors	303
43.5	Open-Source Alternatives	303
43.5.1	Apache Camel	303
43.5.2	Smooks	303
43.5.3	Ballerina	304
43.6	Positioning Matrix	304
43.7	UTL-X's Unique Position	304

44	Open-M: The Integration Controller	305
44.1	What Is Open-M?	305
44.2	Why Open-M + UTL-X?	305
44.3	MPPM: Message Processing Pipeline Model	305
44.3.1	What Is MPPM?	305
44.3.2	Pipeline Stages	306
44.3.3	The Message Envelope	306
44.3.4	How MPPM Maps to UTL-X	306
44.4	Open-M Architecture	307
44.4.1	Go-Based Controller	307
44.4.2	Flow Definitions	307
44.4.3	Plugin Architecture	308
44.5	What This Means for UTL-X Users	308
44.5.1	If You're Using UTL-X Standalone	308
44.5.2	If You Need Orchestration	308
44.5.3	The Growth Path	308
45	AI and UTL-X	309
45.1	The Natural Language Round-Trip	309
45.2	Why UTL-X Is Uniquely Suited for AI	309
45.2.1	The Constrained Domain Advantage	310
45.3	What Works Today	310
45.3.1	The MCP Server	310
45.4	The Vision: AI-Native Transformation Development	311
45.4.1	Token Efficiency	311
45.4.2	Three Approaches	311
45.4.3	Available Training Data	311
45.5	Capabilities of an AI-Assisted UTL-X Workflow	311
45.5.1	Generation from Spec	311
45.5.2	Migration from Other Tools	312
45.5.3	Explanation and Documentation	312
45.5.4	Test Generation	312
45.6	AI at Design-Time Only	312
45.7	The Virtuous Circle	312
46	Why Kotlin and GraalVM	314
46.1	Why Kotlin	314
46.1.1	Sealed Classes for the UDM Type Hierarchy	314
46.1.2	Null Safety	314
46.1.3	Extension Functions	314
46.1.4	Java Ecosystem	314
46.1.5	Functional Style	315
46.2	Why Not Other Languages?	315
46.3	Why GraalVM	315
46.3.1	Native Image for the CLI	315
46.3.2	GraalVM Challenges and How They Were Solved	316
46.3.3	JVM for the Engine	316
46.3.4	GraalVM for Cloud (Future)	316
46.4	Why NOT JavaScript/TypeScript	317
46.4.1	Performance Ceiling	317
46.4.2	Number Precision	317

46.4.3	XML Inconsistency	317
46.5	Why NOT WebAssembly (WASM)	317
46.5.1	The Promise	317
46.5.2	Why It Doesn't Work Today	317
46.6	One Implementation, Zero Deviation	317
47	Case Studies and Recipes	320
47.1	Case Study 1: European E-Invoicing (Peppol / UBL)	320
47.2	Case Study 2: Healthcare — FHIR Integration	322
47.3	Case Study 3: Retail — Multi-Channel Order Normalization	325
47.4	Recipes: Common Transformation Patterns	326
47.4.1	Recipe 1: REST API Response Flattening	326
47.4.2	Recipe 2: Date Format Normalization	326
47.4.3	Recipe 3: Currency Conversion with Lookup	327
47.4.4	Recipe 4: Address Normalization	327
47.4.5	Recipe 5: Error Response Standardization	328
47.4.6	Recipe 6: CSV Regional Format Conversion	328
47.4.7	Recipe 7: XML Namespace Stripping	328
47.4.8	Recipe 8: Conditional Output Format	328
48	Grammar Reference	331
48.1	File Structure	331
48.2	Header Syntax	331
48.2.1	Version Directive	331
48.2.2	Input Declaration	331
48.2.3	Output Declaration	331
48.2.4	Format Keywords	331
48.2.5	Format Options	331
48.3	Expression Grammar (Precedence)	332
48.3.1	Grammar Rules (EBNF)	332
48.4	Literals	333
48.4.1	Numbers	333
48.4.2	Strings	333
48.4.3	Booleans and Null	333
48.5	Object Construction	333
48.6	Array Construction	333
48.7	Let Bindings	333
48.7.1	Top-Level Let (Body Scope)	333
48.7.2	Let Inside Object Construction	334
48.7.3	Let Inside Lambda	334
48.7.4	Let Inside Function Definitions	334
48.7.5	Scoping Summary	335
48.7.6	Separators: Commas and Newlines	335
48.8	Function Definitions	336
48.9	Lambda Expressions	336
48.9.1	Thin Arrow (->)	336
48.9.2	Fat Arrow (=>)	336
48.9.3	Which to Use?	336
48.9.4	In Match Expressions	336
48.10	Conditional Expressions	337
48.11	Match Expression	337

48.12	Try/Catch	337
48.13	Safe Navigation	337
48.14	Property Access	337
48.15	Pipe Operator	337
48.16	Spread Operator	338
48.17	Reserved Words	338
48.17.1	Language Keywords	338
48.17.2	Format Keywords	338
48.17.3	NOT Reserved	338
48.18	Operators Reference	338
48.19	Identifier Rules	339
48.20	USDL Grammar (Universal Schema Definition Language)	339
48.20.1	USDL Document Structure	339
48.20.2	Type Definitions	339
48.20.3	Field Definitions	340
48.20.4	USDL Directive Classification	340
48.20.5	USDL Type Strings	340
48.20.6	Example: Complete USDL Document	341
49	Appendices	343
49.1	Appendix A: Format Conversion Matrix	343
49.2	Appendix B: Common Error Messages	343
49.2.1	Parser Errors	343
49.2.2	Runtime Errors	343
49.2.3	Validation Errors (UTLXe)	344
49.3	Appendix C: UTL-X vs Competitors (Feature Comparison)	344
49.3.1	UTL-X vs XSLT	344
49.3.2	UTL-X vs DataWeave	344
49.3.3	UTL-X vs jq	345
49.4	Appendix D: Industry Standards Quick Reference	345
49.5	Appendix E: Glossary	346
49.6	Appendix F: Version History	346
49.7	Appendix G: License	347
49.7.1	Open Source: AGPL-3.0	347
49.7.2	Commercial Licensing	347
49.7.3	Third-Party Components	347
50	Standard Library Reference	349
50	Function Index	349
50.1	A	353
50.1.1	abs(number) → number (Num)	353
50.1.2	acos(number) → number (Num)	353
50.1.3	addBOM(data) → binary (Bin)	353
50.1.4	addNamespaceDeclarations(xml, namespaces) → xml (XML)	353
50.1.5	addQueryParam(url, name, value) → string (URL)	353
50.1.6	addTax(amount, rate) → number (Fin)	354
50.1.7	age(birthdate, referenceDate?) → number (Date)	354
50.1.8	analyzeString(string, pattern) → object (Str)	354
50.1.9	and(values...) → boolean (Type)	355
50.1.10	asin(number) → number (Num)	355
50.1.11	assert(condition, message?) → null (Sys)	355

50.1.12	assertEqual(actual, expected) → null (Sys)	355
50.1.13	atan(number) → number (Num)	356
50.1.14	atan2(y, x) → number (Num)	356
50.1.15	attribute(element, name) → string (XML)	356
50.1.16	attributes(element) → object (XML)	356
50.1.17	availableProcessors() → number (Sys)	357
50.1.18	Date Format Patterns (Date — Reference)	357
50.1.19	addDays(date, count) → date (Date)	357
50.1.20	addHours(datetime, count) → datetime (Date)	358
50.1.21	addMinutes(datetime, count) → datetime (Date)	358
50.1.22	addMonths(date, count) → date (Date)	358
50.1.23	addQuarters(date, count) → date (Date)	359
50.1.24	addSeconds(datetime, count) → datetime (Date)	359
50.1.25	addWeeks(date, count) → date (Date)	359
50.1.26	addYears(date, count) → date (Date)	359
50.1.27	all(array, predicate) → boolean (Arr)	359
50.1.28	any(array, predicate) → boolean (Arr)	360
50.1.29	avg(array) → number (Num)	360
50.1.30	avgBy(array, keyFn) → number (Num)	360
50.2	B	361
50.2.1	base64Encode(data) → string (Sec)	361
50.2.2	base64Decode(string) → string (Sec)	361
50.2.3	bearing(lat1, lon1, lat2, lon2) → number (Geo)	361
50.2.4	binaryConcat(binary1, binary2, ...) → binary (Bin)	362
50.2.5	binaryEquals(binary1, binary2) → boolean (Bin)	362
50.2.6	binaryLength(binary) → number (Bin)	362
50.2.7	binarySlice(binary, start, end) → binary (Bin)	362
50.2.8	binaryToString(binary, encoding) → string (Bin)	363
50.2.9	bitwiseAnd(binary1, binary2) → binary (Bin)	363
50.2.10	bitwiseNot(binary) → binary (Bin)	363
50.2.11	bitwiseOr(binary1, binary2) → binary (Bin)	363
50.2.12	bitwiseXor(binary1, binary2) → binary (Bin)	363
50.2.13	boundingBox(coordinates) → object (Geo)	364
50.2.14	buildQueryString(params) → string (URL)	364
50.2.15	buildURL(components) → string (URL)	364
50.3	C	365
50.3.1	c14n(xml) → string (XML)	365
50.3.2	c14nHash(xml, algorithm?) → string (XML)	365
50.3.3	c14nEquals(xml1, xml2) → boolean (XML)	365
50.3.4	c14n11(xml) → string (XML)	365
50.3.5	c14n11WithComments(xml) → string (XML)	366
50.3.6	c14nFingerprint(xml) → string (XML)	366
50.3.7	c14nPhysical(xml, options?) → string (XML)	366
50.3.8	c14nSubset(xml, xpath) → string (XML)	366
50.3.9	c14nWithComments(xml) → string (XML)	366
50.3.10	calculateDiscount(price, rate) → number (Fin)	367
50.3.11	calculateTax(amount, rate) → number (Fin)	367
50.3.12	camelCase(string) → string (Str)	367
50.3.13	camelize(string) → string (Str)	367
50.3.14	canCoerce(value, targetType) → boolean (Type)	368

50.3.15	<code>canonicalizeWithAlgorithm(xml, algorithm)</code> → string (XML)	368
50.3.16	<code>canonicalJSONHash(json, algorithm?)</code> → string (JSON)	368
50.3.17	<code>canonicalJSONSize(json)</code> → number (JSON)	368
50.3.18	<code>capitalize(string)</code> → string (Str)	368
50.3.19	<code>ceil(number)</code> → integer (Num)	369
50.3.20	<code>charAt(string, index)</code> → string (Str)	369
50.3.21	<code>charCodeAt(string, index)</code> → number (Str)	369
50.3.22	<code>childCount(element)</code> → number (XML)	369
50.3.23	<code>childNames(element)</code> → array (XML)	369
50.3.24	<code>chunkBy(array, predicate)</code> → array of arrays (Arr)	369
50.3.25	<code>clearLogs()</code> → null (Sys)	370
50.3.26	<code>coerce(value, targetType, default?)</code> → value (Type)	370
50.3.27	<code>coerceAll(array, targetType, default?)</code> → array (Type)	370
50.3.28	<code>compactCSV(csv, options?)</code> → string (CSV)	370
50.3.29	<code>compactJSON(json, options?, indent?)</code> → string (JSON)	371
50.3.30	<code>compactXML(xml, options?, indent?)</code> → string (XML)	371
50.3.31	<code>compareDates(date1, date2)</code> → number (Date)	371
50.3.32	<code>compoundInterest(principal, rate, periods)</code> → number (Fin)	371
50.3.33	<code>compress(data, algorithm?)</code> → binary (Bin)	372
50.3.34	<code>constantCase(string)</code> → string (Str)	372
50.3.35	<code>containsValue(object, value)</code> → boolean (Obj)	372
50.3.36	<code>convertTimezone(datetime, fromTz, toTz)</code> → datetime (Date)	372
50.3.37	<code>convertXMLEncoding(xml, targetEncoding)</code> → binary (XML)	373
50.3.38	<code>cos(radians)</code> → number (Num)	373
50.3.39	<code>cosh(number)</code> → number (Num)	373
50.3.40	<code>countEntries(object, predicate?)</code> → number (Obj)	373
50.3.41	<code>createCDATA(content)</code> → string (XML)	373
50.3.42	<code>createSOAPEnvelope(body, header?)</code> → xml (XML)	373
50.3.43	<code>crossJoin(array1, array2)</code> → array (Arr)	374
50.3.44	<code>csvAddColumn(csv, name, defaultValue)</code> → string (CSV)	374
50.3.45	<code>csvCell(csv, row, column)</code> → string (CSV)	374
50.3.46	<code>csvColumn(csv, name)</code> → array (CSV)	374
50.3.47	<code>csvColumns(csv)</code> → array (CSV)	375
50.3.48	<code>csvRemoveColumns(csv, columns)</code> → string (CSV)	375
50.3.49	<code>csvRow(csv, index, options?)</code> → object (CSV)	375
50.3.50	<code>csvRows(csv)</code> → number (CSV)	375
50.3.51	<code>csvSelectColumns(csv, columns)</code> → string (CSV)	376
50.3.52	<code>csvSort(csv, column, ascending?)</code> → string (CSV)	376
50.3.53	<code>csvSummarize(csv, options?)</code> → object (CSV)	376
50.3.54	<code>csvTranspose(csv, options?, header?, separator?)</code> → string (CSV)	376
50.3.55	<code>currentDir()</code> → string (Sys)	377
50.3.56	<code>floor(number)</code> → integer (Num)	377
50.3.57	<code>round(number)</code> → integer (Num)	377
50.3.58	<code>chunk(array, size)</code> → array of arrays (Arr)	377
50.3.59	<code>coalesce(value1, value2, ...)</code> → value (Type)	377
50.3.60	<code>compact(array)</code> → array (Arr)	378
50.3.61	<code>concat(string1, string2, ...)</code> → string (Str)	378
50.3.62	<code>contains(haystack, needle)</code> → boolean (Str/Arr)	378
50.3.63	<code>count(array)</code> → number (Arr)	379
50.3.64	<code>countBy(array, predicate)</code> → number (Arr)	379

50.3.65	createQname(localName, namespaceUri, prefix?) → object (XML)	379
50.3.66	csvFilter(csv, column, operator, value) → string (CSV)	380
50.4	D	380
50.4.1	day(date) → number (Date)	380
50.4.2	dayOfMonth(date) → number (Date)	380
50.4.3	dayOfWeek(date) → number (Date)	381
50.4.4	dayOfWeekName(date) → string (Date)	381
50.4.5	dayOfYear(date) → number (Date)	381
50.4.6	daysBetween(date1, date2) → number (Date)	381
50.4.7	daysInMonth(date) → number (Date)	381
50.4.8	daysInYear(date) → number (Date)	382
50.4.9	debug(value, message?) → value (Sys)	382
50.4.10	debugPrint(value, label?, indent?) → string (Sys)	382
50.4.11	debugPrintCompact(value, label?) → string (Sys)	382
50.4.12	decodeJWS(token) → object (Sec)	383
50.4.13	decodeJWT(token) → object (Sec)	383
50.4.14	decompress(data, algorithm?) → binary (Bin)	383
50.4.15	decryptAES(data, key, iv) → string (Sec)	383
50.4.16	decryptAES256(data, key, iv) → string (Sec)	383
50.4.17	deepClone(object) → object (Obj)	384
50.4.18	deepMerge(obj1, obj2) → object (Obj)	384
50.4.19	deepMergeAll(objects) → object (Obj)	384
50.4.20	deflate(data) → binary (Bin)	384
50.4.21	destinationPoint(lat, lon, bearing, distance) → object (Geo)	385
50.4.22	detectBOM(binary) → string (XML)	385
50.4.23	detectXMLEncoding(xmlString) → string (XML)	385
50.4.24	diffDays(date1, date2) → number (Date)	385
50.4.25	diffHours(datetime1, datetime2) → number (Date)	386
50.4.26	diffMinutes(datetime1, datetime2) → number (Date)	386
50.4.27	diffMonths(date1, date2) → number (Date)	386
50.4.28	diffSeconds(datetime1, datetime2) → number (Date)	386
50.4.29	diffWeeks(date1, date2) → number (Date)	387
50.4.30	diffYears(date1, date2) → number (Date)	387
50.4.31	distance(lat1, lon1, lat2, lon2) → number (Geo)	387
50.4.32	distinct(array) → array (Arr)	387
50.4.33	distinctBy(array, keyFn) → array (Arr)	388
50.4.34	divideBy(object, n) → array (Obj)	388
50.4.35	dotCase(string) → string (Str)	388
50.4.36	drop(array, n) → array (Arr)	389
50.4.37	take(array, n) → array (Arr)	389
50.5	E	389
50.5.1	e() → number (Num)	389
50.5.2	elementPath(element) → string (XML)	389
50.5.3	encryptAES(data, key, iv) → string (Sec)	390
50.5.4	encryptAES256(data, key, iv) → string (Sec)	390
50.5.5	endOfDay(date) → datetime (Date)	390
50.5.6	endOfMonth(date) → datetime (Date)	390
50.5.7	endOfQuarter(date) → datetime (Date)	391
50.5.8	endOfWeek(date) → datetime (Date)	391
50.5.9	endOfYear(date) → datetime (Date)	391

50.5.10	endsWith(string, suffix) → boolean (Str)	391
50.5.11	startsWith(string, prefix) → boolean (Str)	391
50.5.12	endTimer(label) → number (Sys)	392
50.5.13	enforceNamespacePrefixes(xml, prefixMap) → string (XML)	392
50.5.14	entries(object) → array (Obj)	392
50.5.15	fromEntries(pairs) → object (Obj)	392
50.5.16	env(name) → string (Sys)	393
50.5.17	envOrDefault(name, default) → string (Sys)	393
50.5.18	envAll() → object (Sys)	393
50.5.19	environment() → string (Sys)	393
50.5.20	equals(a, b) → boolean (Type)	394
50.5.21	error(message) → never (Sys)	394
50.5.22	escapeXML(text) → string (XML)	394
50.5.23	every(array, predicate) → boolean (Arr)	394
50.5.24	everyEntry(object, predicate) → boolean (Obj)	395
50.5.25	excC14n(xml) → string (XML)	395
50.5.26	excC14nWithComments(xml) → string (XML)	395
50.5.27	exp(x) → number (Num)	395
50.5.28	extractBetween(string, start, end) → string (Str)	396
50.5.29	extractCDATA(text) → string (XML)	396
50.5.30	extractTimestampFromUuidV7(uuid) → datetime (Sec)	396
50.6	F	396
50.6.1	filter(array, predicate) → array (Arr)	396
50.6.2	filterEntries(object, predicate) → object (Obj)	397
50.6.3	find(array, predicate) → element or null (Arr)	397
50.6.4	findIndex(array, predicate) → number (Arr)	398
50.6.5	findAllMatches(string, pattern) → array (Str)	398
50.6.6	findLastIndex(array, predicate) → number (Arr)	398
50.6.7	first(array) → element or null (Arr)	399
50.6.8	formatCurrency(amount, currency?, locale?) → string (Fin)	399
50.6.9	formatDateTimeInTimezone(datetime, timezone) → string (Date)	399
50.6.10	formatEmptyElements(xml, style?) → string (XML)	399
50.6.11	formatNumber(number, pattern?) → string (Num)	400
50.6.12	formatPlural(count, word) → string (Str)	400
50.6.13	fromBase64(encoded) → binary (Sec)	400
50.6.14	fromBytes(byteArray) → binary (Bin)	400
50.6.15	fromCamelCase(string) → string (Str)	401
50.6.16	fromCharCode(code) → string (Str)	401
50.6.17	fromConstantCase(string) → string (Str)	401
50.6.18	fromDotCase(string) → string (Str)	401
50.6.19	fromHex(hexString) → binary (Str)	402
50.6.20	fromKebabCase(string) → string (Str)	402
50.6.21	fromPascalCase(string) → string (Str)	402
50.6.22	fromPathCase(string) → string (Str)	402
50.6.23	fromSnakeCase(string) → string (Str)	402
50.6.24	fromTitleCase(string) → string (Str)	403
50.6.25	fromUTC(datetime, timezone) → datetime (Date)	403
50.6.26	fullOuterJoin(left, right, leftKey, rightKey) → array (Arr)	403
50.6.27	futureValue(presentValue, rate, periods) → number (Fin)	403
50.6.28	head(array) → element or null (Arr)	404

50.6.29	<code>last(array) → element or null (Arr)</code>	404
50.6.30	<code>tail(array) → array (Arr)</code>	404
50.6.31	<code>flatMap(array, fn) → array (Arr)</code>	404
50.6.32	<code>flatten(array) → array (Arr)</code>	405
50.6.33	<code>flattenDeep(array) → array (Arr)</code>	405
50.6.34	<code>formatDate(date, pattern) → string (Date)</code>	405
50.7	G	406
50.7.1	<code>generateIV() → string (Sec)</code>	406
50.7.2	<code>generateKey(bits?) → string (Sec)</code>	406
50.7.3	<code>generateUuid() → string (Sys)</code>	406
50.7.4	<code>generateUuidV4() → string (Sys)</code>	406
50.7.5	<code>generateUuidV7() → string (Sys)</code>	406
50.7.6	<code>generateUuidV7Batch(count) → array (Sec)</code>	407
50.7.7	<code>get(object, path) → any (Obj)</code>	407
50.7.8	<code>getBaseURL(url) → string (URL)</code>	407
50.7.9	<code>getBOMBytes(encoding) → binary (XML)</code>	407
50.7.10	<code>getCurrencyDecimals(currency) → number (Fin)</code>	408
50.7.11	<code>getFragment(url) → string (URL)</code>	408
50.7.12	<code>getHost(url) → string (URL)</code>	408
50.7.13	<code>getJWSAlgorithm(token) → string (Sec)</code>	408
50.7.14	<code>getJWSHeader(token) → object (Sec)</code>	409
50.7.15	<code>getJWSInfo(token) → object (Sec)</code>	409
50.7.16	<code>getJWSKeyId(token) → string (Sec)</code>	409
50.7.17	<code>getJWSPayload(token) → object (Sec)</code>	409
50.7.18	<code>getJWSSigningInput(token) → string (Sec)</code>	409
50.7.19	<code>getJWSTokenType(token) → string (Sec)</code>	409
50.7.20	<code>getJWTAudience(token) → string (Sec)</code>	410
50.7.21	<code>getJWTClaim(token, claim) → any (Sec)</code>	410
50.7.22	<code>getJWTClaims(token) → object (Sec)</code>	410
50.7.23	<code>getJWTIssuer(token) → string (Sec)</code>	410
50.7.24	<code>getJWTSubject(token) → string (Sec)</code>	410
50.7.25	<code>getLogs() → array (Sys)</code>	411
50.7.26	<code>getNamespaces(element) → object (XML)</code>	411
50.7.27	<code>getPath(url) → string (URL)</code>	411
50.7.28	<code>getPort(url) → number (URL)</code>	411
50.7.29	<code>getProtocol(url) → string (URL)</code>	412
50.7.30	<code>getQuery(url) → string (URL)</code>	412
50.7.31	<code>getQueryParams(url) → object (URL)</code>	412
50.7.32	<code>getTimezone(datetime) → string (Date)</code>	412
50.7.33	<code>getTimezoneName() → string (Date)</code>	412
50.7.34	<code>getTimezoneOffsetHours(datetime, timezone?) → number (Date)</code>	413
50.7.35	<code>getTimezoneOffsetSeconds(datetime) → number (Date)</code>	413
50.7.36	<code>getType(value) → string (Type)</code>	413
50.7.37	<code>getUuidVersion(uuid) → number (Sec)</code>	413
50.7.38	<code>goldenRatio() → number (Num)</code>	414
50.7.39	<code>groupBy(array, keyFn) → object (Arr)</code>	414
50.7.40	<code>gunzip(data) → binary (Bin)</code>	414
50.7.41	<code>gzip(data) → binary (Bin)</code>	414
50.8	H	415
50.8.1	<code>hasAlpha(string) → boolean (Str)</code>	415

50.8.2	hasAttribute(element, name) → boolean (XML)	415
50.8.3	hasBOM(data) → boolean (XML)	415
50.8.4	hasContent(element) → boolean (XML)	415
50.8.5	hasEnv(name) → boolean (Sys)	415
50.8.6	hasKey(object, key) → boolean (Obj)	416
50.8.7	containsKey(object, key) → boolean (Obj)	416
50.8.8	hash(data, algorithm?) → string (Sec)	416
50.8.9	md5(data) → string (Sec)	417
50.8.10	sha256(data) → string (Sec)	417
50.8.11	sha512(data) → string (Sec)	417
50.8.12	hasNamespace(element, uri) → boolean (XML)	417
50.8.13	hasNumeric(string) → boolean (Str)	417
50.8.14	hexDecode(hex) → binary (Bin)	418
50.8.15	hexEncode(data) → string (Bin)	418
50.8.16	hmac(data, key, algorithm) → string (Sec)	418
50.8.17	hmacSHA256(data, key) → string (Sec)	418
50.8.18	hmacBase64(data, key, algorithm) → string (Sec)	419
50.8.19	hmacMD5(data, key) → string (Sec)	419
50.8.20	hmacSHA1(data, key) → string (Sec)	419
50.8.21	hmacSHA384(data, key) → string (Sec)	419
50.8.22	hmacSHA512(data, key) → string (Sec)	419
50.8.23	homeDir() → string (Sys)	420
50.8.24	hours(datetime) → number (Date)	420
50.9	I	420
50.9.1	ifThenElse(condition, thenValue, elseValue) → any (Type)	420
50.9.2	implies(a, b) → boolean (Type)	420
50.9.3	includes(array, value) → boolean (Arr)	421
50.9.4	indexOf(array, value) → number (Arr)	421
50.9.5	inflate(data) → binary (Bin)	421
50.9.6	info(message) → null (Sys)	421
50.9.7	insertAfter(array, index, element) → array (Arr)	422
50.9.8	insertBefore(array, index, element) → array (Arr)	422
50.9.9	invert(object) → object (Obj)	422
50.9.10	iqr(numbers) → number (Num)	422
50.9.11	isAfter(date1, date2) → boolean (Date)	423
50.9.12	isAlpha(string) → boolean (Str)	423
50.9.13	isAlphanumeric(string) → boolean (Str)	423
50.9.14	isAscii(string) → boolean (Str)	423
50.9.15	isBefore(date1, date2) → boolean (Date)	424
50.9.16	isBetween(date, start, end) → boolean (Date)	424
50.9.17	isCanonicalJSON(json) → boolean (JSON)	424
50.9.18	isCDATA(text) → boolean (XML)	424
50.9.19	isDebugMode() → boolean (Sys)	425
50.9.20	isEmptyElement(element) → boolean (XML)	425
50.9.21	isGzipped(data) → boolean (Bin)	425
50.9.22	isHexadecimal(string) → boolean (Str)	425
50.9.23	isJarFile(data) → boolean (Bin)	425
50.9.24	isJWSFormat(token) → boolean (Sec)	425
50.9.25	isJWTExpired(token) → boolean (Sec)	426
50.9.26	isLeapYearFunc(year) → boolean (Date)	426

50.9.27	isLocalDateTime(value) → boolean (Date)	426
50.9.28	isLowerCase(string) → boolean (Str)	426
50.9.29	isNotEmpty(value) → boolean (Type)	426
50.9.30	isNumeric(string) → boolean (Str)	427
50.9.31	isPlural(word) → boolean (Str)	427
50.9.32	isPointInCircle(lat, lon, centerLat, centerLon, radiusKm) → boolean (Geo)	427
50.9.33	isPointInPolygon(lat, lon, polygon) → boolean (Geo)	427
50.9.34	isPrintable(string) → boolean (Str)	428
50.9.35	isSameDay(date1, date2) → boolean (Date)	428
50.9.36	isSingular(word) → boolean (Str)	428
50.9.37	isToday(date) → boolean (Date)	428
50.9.38	isUpperCase(string) → boolean (Str)	428
50.9.39	isUuidV7(uuid) → boolean (Sec)	429
50.9.40	isValidAmount(amount) → boolean (Fin)	429
50.9.41	isValidCoordinates(lat, lon) → boolean (Geo)	429
50.9.42	isValidCurrency(code) → boolean (Fin)	429
50.9.43	isValidTimezone(timezone) → boolean (Date)	430
50.9.44	isValidURL(url) → boolean (URL)	430
50.9.45	isValidUuid(uuid) → boolean (Sec)	430
50.9.46	isWhitespace(string) → boolean (Str)	430
50.9.47	isZipArchive(data) → boolean (Bin)	430
50.9.48	lastIndexOf(haystack, needle) → number (Str/Arr)	431
50.9.49	isArray(value) → boolean (Type)	431
50.9.50	isBoolean(value) → boolean (Type)	431
50.9.51	isNull(value) → boolean (Type)	431
50.9.52	isNumber(value) → boolean (Type)	432
50.9.53	isObject(value) → boolean (Type)	432
50.9.54	isString(value) → boolean (Type)	432
50.9.55	isDate(value) → boolean (Type)	432
50.9.56	isDateTime(value) → boolean (Type)	432
50.9.57	isTime(value) → boolean (Type)	432
50.9.58	isBlank(value) → boolean (Type)	433
50.9.59	isEmpty(value) → boolean (Type)	433
50.9.60	isDefined(value) → boolean (Type)	433
50.9.61	isLeapYear(year) → boolean (Date)	433
50.9.62	isWeekday(date) → boolean (Date)	433
50.9.63	isWeekend(date) → boolean (Date)	434
50.10	J	434
50.10.1	javaVersion() → string (Sys)	434
50.10.2	jcs(json) → string (JSON)	434
50.10.3	canonicalizeJSON(json) → string (JSON)	434
50.10.4	jsonEquals(json1, json2) → boolean (JSON)	434
50.10.5	join(array, separator) → string (Str)	435
50.10.6	joinToString(array, separator?) → string (Str)	435
50.10.7	joinWith(left, right, leftKeyFn, rightKeyFn, combinerFn) → array (Arr)	435
50.11	K	436
50.11.1	kebabCase(string) → string (Str)	436
50.11.2	keys(object) → array (Obj)	436
50.11.3	values(object) → array (Obj)	436
50.12	L	436

50.12.1	<code>leftJoin(left, right, leftKeyFn, rightKeyFn) → array (Arr)</code>	436
50.12.2	<code>length(value) → number (Str/Arr)</code>	437
50.12.3	<code>listJarEntries(jarData) → array (Bin)</code>	437
50.12.4	<code>listZipEntries(zipData) → array (Bin)</code>	437
50.12.5	<code>ln(number) → number (Num)</code>	437
50.12.6	<code>localName(element) → string (XML)</code>	438
50.12.7	<code>namespaceUri(element) → string (XML)</code>	438
50.12.8	<code>namespacePrefix(element) → string (XML)</code>	438
50.12.9	<code>qualifiedName(element) → string (XML)</code>	438
50.12.10	<code>log(number, base?) → number (Num)</code>	438
50.12.11	<code>log10(number) → number (Num)</code>	439
50.12.12	<code>log2(number) → number (Num)</code>	439
50.12.13	<code>logCount(value) → value (Sys)</code>	439
50.12.14	<code>logPretty(value) → value (Sys)</code>	439
50.12.15	<code>logSize(value) → value (Sys)</code>	439
50.12.16	<code>logType(value) → value (Sys)</code>	440
50.12.17	<code>lookupBy(searchValue, referenceArray, keyFn) → element null (Arr)</code>	440
50.12.18	<code>lower(string) → string (Str)</code>	440
50.12.19	<code>lowerCase(string) → string (Str)</code>	440
50.12.20	<code>upperCase(string) → string (Str)</code>	441
50.13	M	441
50.13.1	<code>map(array, fn) → array (Arr)</code>	441
50.13.2	<code>mapEntries(object, fn) → object (Obj)</code>	441
50.13.3	<code>mapKeys(object, fn) → object (Obj)</code>	442
50.13.4	<code>mapValues(object, fn) → object (Obj)</code>	442
50.13.5	<code>mapGroups(array, keySelector, transform) → array (Arr)</code>	442
50.13.6	<code>mapTree(data, transformer) → object array (Obj)</code>	442
50.13.7	<code>mask(string, visibleChars) → string (Sec)</code>	443
50.13.8	<code>matches(string, regex) → boolean (Str)</code>	443
50.13.9	<code>matchesWhole(string, pattern) → boolean (Str)</code>	444
50.13.10	<code>max(array) → number (Num/Arr)</code>	444
50.13.11	<code>min(array) → number (Num/Arr)</code>	444
50.13.12	<code>maxBy(array, fn) → element (Num/Arr)</code>	444
50.13.13	<code>minBy(array, fn) → element (Num/Arr)</code>	444
50.13.14	<code>measure(fn) → object (Sys)</code>	445
50.13.15	<code>median(array) → number (Num)</code>	445
50.13.16	<code>mode(array) → number (Num)</code>	445
50.13.17	<code>stdDev(array) → number (Num)</code>	445
50.13.18	<code>variance(array) → number (Num)</code>	445
50.13.19	<code>percentile(array, p) → number (Num)</code>	445
50.13.20	<code>memoryInfo() → object (Sys)</code>	446
50.13.21	<code>merge(obj1, obj2, ...) → object (Obj)</code>	446
50.13.22	<code>midpoint(lat1, lon1, lat2, lon2) → object (Geo)</code>	446
50.13.23	<code>minutes(datetime) → number (Date)</code>	446
50.13.24	<code>month(date) → number (Date)</code>	446
50.13.25	<code>monthName(date) → string (Date)</code>	447
50.14	N	447
50.14.1	<code>nand(a, b) → boolean (Type)</code>	447
50.14.2	<code>nestBy(parents, children, parentKeyFn, childKeyFn, childProperty) → array (Arr)</code>	447
50.14.3	<code>nodeType(node) → string (XML)</code>	448

50.14.4	<code>none(array, predicate)</code>	$\rightarrow$ boolean (Arr)	448
50.14.5	<code>nor(a, b)</code>	$\rightarrow$ boolean (Type)	448
50.14.6	<code>normalizeBOM(data, targetEncoding, addBom)</code>	$\rightarrow$ binary (XML)	448
50.14.7	<code>normalizeXMLEncoding(xml, targetEncoding)</code>	$\rightarrow$ string (XML)	449
50.14.8	<code>not(value)</code>	$\rightarrow$ boolean (Type)	449
50.14.9	<code>now()</code>	$\rightarrow$ datetime (Date)	449
50.14.10	<code>currentDate()</code>	$\rightarrow$ date (Date)	449
50.14.11	<code>currentTime()</code>	$\rightarrow$ time (Date)	449
50.14.12	<code>normalizeSpace(string)</code>	$\rightarrow$ string (Str)	449
50.14.13	<code>numberOrDefault(value, default)</code>	$\rightarrow$ number (Type)	450
50.15	O		450
50.15.1	<code>omit(object, keys)</code>	$\rightarrow$ object (Obj)	450
50.15.2	<code>pick(object, keys)</code>	$\rightarrow$ object (Obj)	450
50.15.3	<code>osArch()</code>	$\rightarrow$ string (Sys)	450
50.15.4	<code>osVersion()</code>	$\rightarrow$ string (Sys)	451
50.16	P		451
50.16.1	<code>pad(string, length, char?)</code>	$\rightarrow$ string (Str)	451
50.16.2	<code>padLeft(string, length, char?)</code>	$\rightarrow$ string (Str)	451
50.16.3	<code>padRight(string, length, char?)</code>	$\rightarrow$ string (Str)	451
50.16.4	<code>parent(element)</code>	$\rightarrow$ element (XML)	451
50.16.5	<code>parse(string, format?)</code>	$\rightarrow$ value (Fmt)	452
50.16.6	<code>parseJson(string)</code>	$\rightarrow$ value (Fmt)	452
50.16.7	<code>parseXml(string)</code>	$\rightarrow$ value (Fmt)	452
50.16.8	<code>parseYaml(string)</code>	$\rightarrow$ value (Fmt)	452
50.16.9	<code>parseCsv(string, options?)</code>	$\rightarrow$ array (Fmt)	452
50.16.10	<code>parseBoolean(value)</code>	$\rightarrow$ boolean (Type)	453
50.16.11	<code>parseCurrency(string)</code>	$\rightarrow$ number (Fin)	453
50.16.12	<code>parseDate(string, pattern)</code>	$\rightarrow$ date (Date)	453
50.16.13	<code>parseDateTimeWithTimezone(string, pattern, timezone)</code>	$\rightarrow$ datetime (Date)	453
50.16.14	<code>parseDouble(string)</code>	$\rightarrow$ number (Num)	454
50.16.15	<code>parseEUNumber(string)</code>	$\rightarrow$ number (Num)	454
50.16.16	<code>parseUSNumber(string)</code>	$\rightarrow$ number (Num)	454
50.16.17	<code>parseFloat(string)</code>	$\rightarrow$ number (Num)	454
50.16.18	<code>parseInt(string, radix?)</code>	$\rightarrow$ number (Num)	454
50.16.19	<code>parseNumber(string)</code>	$\rightarrow$ number (Num)	455
50.16.20	<code>parseQueryString(string)</code>	$\rightarrow$ object (URL)	455
50.16.21	<code>parseURL(url)</code>	$\rightarrow$ object (URL)	455
50.16.22	<code>partition(array, predicate)</code>	$\rightarrow$ [matching, nonMatching] (Arr)	455
50.16.23	<code>pascalCase(string)</code>	$\rightarrow$ string (Str)	456
50.16.24	<code>pathCase(string)</code>	$\rightarrow$ string (Str)	456
50.16.25	<code>percentageChange(oldValue, newValue)</code>	$\rightarrow$ number (Num)	456
50.16.26	<code>pi()</code>	$\rightarrow$ number (Num)	456
50.16.27	<code>platform()</code>	$\rightarrow$ string (Sys)	457
50.16.28	<code>pluralize(word)</code>	$\rightarrow$ string (Str)	457
50.16.29	<code>pluralizeWithCount(word, count)</code>	$\rightarrow$ string (Str)	457
50.16.30	<code>pow(base, exponent)</code>	$\rightarrow$ number (Num)	457
50.16.31	<code>prepareForSignature(xml)</code>	$\rightarrow$ string (XML)	457
50.16.32	<code>presentValue(futureAmount, rate, periods)</code>	$\rightarrow$ number (Fin)	458
50.16.33	<code>prettyPrint(string, indent?)</code>	$\rightarrow$ string (Fmt)	458
50.16.34	<code>prettyPrintCSV(csv, options?)</code>	$\rightarrow$ string (Fmt)	458

50.16.35	<code>prettyPrintFormat(value, format) → string (Fmt)</code>	458
50.16.36	<code>prettyPrintJSON(json, indent?) → string (Fmt)</code>	458
50.16.37	<code>prettyPrintXML(xml, options?) → string (Fmt)</code>	458
50.16.38	<code>prettyPrintYAML(yaml, options?) → string (Fmt)</code>	458
50.17	Q	459
50.17.1	<code>quarter(date) → number (Date)</code>	459
50.17.2	<code>quartiles(array) → object (Num)</code>	459
50.18	R	459
50.18.1	<code>random() → number (Num)</code>	459
50.18.2	<code>readByte(binary, offset) → number (Bin)</code>	459
50.18.3	<code>readDouble(binary, offset) → number (Bin)</code>	459
50.18.4	<code>readFloat(binary, offset) → number (Bin)</code>	459
50.18.5	<code>readInt16(binary, offset) → number (Bin)</code>	460
50.18.6	<code>readInt32(binary, offset) → number (Bin)</code>	460
50.18.7	<code>readInt64(binary, offset) → number (Bin)</code>	460
50.18.8	<code>readJarEntry(jar, entryName) → binary (Bin)</code>	460
50.18.9	<code>readJarManifest(jar) → object (Bin)</code>	460
50.18.10	<code>readZipEntry(zip, entryName) → binary (Bin)</code>	460
50.18.11	<code>reduce(array, initial, accumulator) → value (Arr)</code>	461
50.18.12	<code>reduceEntries(object, initial, accumulator) → value (Obj)</code>	462
50.18.13	<code>regexGroups(string, pattern) → array (Str)</code>	462
50.18.14	<code>regexNamedGroups(string, pattern) → object (Str)</code>	462
50.18.15	<code>remove(string, search) → string (Str)</code>	462
50.18.16	<code>removeBOM(data) → data (XML)</code>	462
50.18.17	<code>removeQueryParam(url, param) → string (URL)</code>	462
50.18.18	<code>removeTax(totalWithTax, rate) → number (Fin)</code>	463
50.18.19	<code>render(value, format, pretty?) → string (Fmt)</code>	463
50.18.20	<code>renderJson(value, pretty?) → string (Fmt)</code>	463
50.18.21	<code>renderCsv(value) → string (CSV)</code>	463
50.18.22	<code>renderXml(value) → string (XML)</code>	463
50.18.23	<code>renderYaml(value) → string (YAML)</code>	464
50.18.24	<code>repeat(string, n) → string (Str)</code>	464
50.18.25	<code>replace(string, search, replacement) → string (Str)</code>	464
50.18.26	<code>replaceRegex(string, regex, replacement) → string (Str)</code>	464
50.18.27	<code>replaceWithFunction(string, pattern, fn) → string (Str)</code>	464
50.18.28	<code>resolveQName(qname, namespaces) → object (XML)</code>	464
50.18.29	<code>reverse(array) → array (Arr)</code>	465
50.18.30	<code>reverseString(string) → string (Str)</code>	465
50.18.31	<code>rightJoin(left, right, leftKey, rightKey) → array (Arr)</code>	465
50.18.32	<code>roundToCents(number) → number (Num)</code>	465
50.18.33	<code>roundToDecimalPlaces(number, places) → number (Num)</code>	465
50.18.34	<code>runtimeInfo() → object (Sys)</code>	466
50.19	S	466
50.19.1	<code>scan(array, initial, fn) → array (Arr)</code>	466
50.19.2	<code>seconds(datetime) → number (Date)</code>	466
50.19.3	<code>setConsoleLogging(enabled) → null (Sys)</code>	466
50.19.4	<code>setLogLevel(level) → null (Sys)</code>	466
50.19.5	<code>setPath(url, path) → string (URL)</code>	466
50.19.6	<code>sha224(data) → string (Sec)</code>	467
50.19.7	<code>sha256(data) → string (Sec)</code>	467

50.19.8	sha512(data) → string (Sec)	467
50.19.9	sha1(data) → string (Sec)	467
50.19.10	sha384(data) → string (Sec)	467
50.19.11	sha3_256(data) → string (Sec)	467
50.19.12	sha3_512(data) → string (Sec)	467
50.19.13	shiftLeft(binary, positions) → binary (Bin)	468
50.19.14	shiftRight(binary, positions) → binary (Bin)	468
50.19.15	shouldUseCDATA(content, threshold?) → boolean (XML)	468
50.19.16	simpleInterest(principal, rate, time) → number (Fin)	468
50.19.17	sin(x) → number (Num)	468
50.19.18	singularize(word) → string (Str)	468
50.19.19	sinh(x) → number (Num)	469
50.19.20	slice(value, start, end?) → array or string (Arr/Str)	469
50.19.21	slugify(string) → string (Str)	469
50.19.22	smartCoerce(value) → value (Type)	469
50.19.23	snakeCase(string) → string (Str)	469
50.19.24	some(array, predicate) → boolean (Arr)	469
50.19.25	someEntry(object, predicate) → boolean (Obj)	470
50.19.26	sort(array) → array (Arr)	470
50.19.27	sortBy(array, keyFn) → array (Arr)	470
50.19.28	split(string, separator) → array (Str)	470
50.19.29	splitWithMatches(string, pattern) → array (Str)	471
50.19.30	sqrt(x) → number (Num)	471
50.19.31	startOfDay(datetime) → datetime (Date)	471
50.19.32	startOfMonth(datetime) → datetime (Date)	471
50.19.33	startOfQuarter(datetime) → datetime (Date)	471
50.19.34	startOfWeek(datetime) → datetime (Date)	471
50.19.35	startOfYear(datetime) → datetime (Date)	472
50.19.36	startTimer() → null (Sys)	472
50.19.37	stringOrDefault(value, default) → string (Type)	472
50.19.38	stripBOM(string) → string (XML)	472
50.19.39	substring(string, start, end?) → string (Str)	472
50.19.40	substringBefore(string, delimiter) → string (Str)	472
50.19.41	substringAfter(string, delimiter) → string (Str)	473
50.19.42	substringAfterLast(string, delimiter) → string (Str)	473
50.19.43	substringBeforeLast(string, delimiter) → string (Str)	473
50.19.44	sum(array) → number (Num)	473
50.19.45	sumBy(array, fn) → number (Num)	473
50.19.46	systemPropertiesAll() → object (Sys)	474
50.19.47	systemProperty(name) → string (Sys)	474
50.19.48	systemPropertyOrDefault(name, default) → string (Sys)	474
50.20	T	474
50.20.1	tan(x) → number (Num)	474
50.20.2	tanh(x) → number (Num)	474
50.20.3	tempDir() → string (Sys)	475
50.20.4	textContent(element) → string (XML)	475
50.20.5	timerCheck(name) → number (Sys)	475
50.20.6	timerClear() → null (Sys)	475
50.20.7	timerList() → array (Sys)	475
50.20.8	timerReset(name) → null (Sys)	475

50.20.9	timerStart(name) → null (Sys)	475
50.20.10	timerStats(name) → object (Sys)	476
50.20.11	timerStop(name) → number (Sys)	476
50.20.12	timestamp() → string (Date)	476
50.20.13	titleCase(string) → string (Str)	476
50.20.14	toArray(value) → array (Type)	476
50.20.15	toBase64(binary) → string (Bin)	476
50.20.16	toBinary(string, encoding?) → binary (Bin)	476
50.20.17	toBoolean(value) → boolean (Type)	477
50.20.18	toBytes(binary) → array (Bin)	477
50.20.19	toDate(string) → date (Date)	477
50.20.20	toDegrees(radians) → number (Num)	477
50.20.21	toHex(binary) → string (Bin)	477
50.20.22	toNumber(value) → number (Type)	477
50.20.23	toObject(value) → object (Type)	478
50.20.24	toRadians(degrees) → number (Num)	478
50.20.25	toString(value) → string (Type)	478
50.20.26	toTitleCase(string) → string (Str)	478
50.20.27	toUTC(datetime) → datetime (Date)	479
50.20.28	trace(value) → value (Sys)	479
50.20.29	translate(string, from, to) → string (Str)	479
50.20.30	treeDepth(object) → number (Obj)	479
50.20.31	treeFilter(object, predicate) → object (Obj)	479
50.20.32	treeFind(object, path) → value (Obj)	479
50.20.33	treeFlatten(object) → array (Obj)	480
50.20.34	treeMap(object, fn) → object (Obj)	480
50.20.35	treePaths(object) → array (Obj)	480
50.20.36	trim(string) → string (Str)	480
50.20.37	leftTrim(string) → string (Str)	480
50.20.38	rightTrim(string) → string (Str)	480
50.20.39	transpose(array2D) → array2D (Arr)	480
50.20.40	truncate(string, maxLength, suffix?) → string (Str)	481
50.20.41	tryCoerce(value, type) → value (Type)	481
50.20.42	typeof(value) → string (Type)	481
50.20.43	typeof(value) → string (Type)	481
50.21	U	481
50.21.1	udmToJSON(value, pretty?) → string (Fmt)	481
50.21.2	udmToXML(value, rootName?, options?) → string (Fmt)	482
50.21.3	udmToYAML(value, options?) → string (Fmt)	482
50.21.4	uncamelize(string) → string (Str)	482
50.21.5	unescapeXML(string) → string (XML)	482
50.21.6	union(arr1, arr2) → array (Arr)	482
50.21.7	intersect(arr1, arr2) → array (Arr)	482
50.21.8	difference(arr1, arr2) → array (Arr)	483
50.21.9	symmetricDifference(arr1, arr2) → array (Arr)	483
50.21.10	unique(array) → array (Arr)	483
50.21.11	unnest(array, childKey) → array (Arr)	483
50.21.12	unwrapCDATA(string) → string (XML)	484
50.21.13	unzip(pairs) → [array, array] (Bin)	484
50.21.14	unzipArchive(zip, options?) → object (Bin)	484

50.21.15	unzipN(tuples) → array of arrays (Bin)	484
50.21.16	updateXMLEncoding(xml, encoding) → string (XML)	485
50.21.17	upper(string) → string (Str)	485
50.21.18	uptime() → number (Sys)	485
50.21.19	urlDecodeComponent(string) → string (URL)	485
50.21.20	urlEncode(string) → string (URL)	485
50.21.21	urlDecode(string) → string (URL)	485
50.21.22	urlEncodeComponent(string) → string (URL)	486
50.21.23	username() → string (Sys)	486
50.22	V-W	486
50.22.1	validateDate(string, pattern?) → boolean (Date)	486
50.22.2	validateDigest(xml, expectedDigest) → boolean (Sec)	486
50.22.3	validateEncoding(encoding) → boolean (XML)	486
50.22.4	values(object) → array (Obj)	486
50.22.5	version() → string (Sys)	487
50.22.6	warn(value) → value (Sys)	487
50.22.7	weekOfYear(date) → number (Date)	487
50.22.8	windowed(array, size) → array of arrays (Arr)	487
50.22.9	wordCase(string) → string (Str)	487
50.22.10	wrapIfNeeded(content, threshold?, tag?) → string (XML)	487
50.22.11	writeByte(value) → binary (Bin)	488
50.22.12	writeDouble(value) → binary (Bin)	488
50.22.13	writeFloat(value) → binary (Bin)	488
50.22.14	writeInt16(value) → binary (Bin)	488
50.22.15	writeInt32(value) → binary (Bin)	488
50.22.16	writeInt64(value) → binary (Bin)	488
50.23	X-Z	489
50.23.1	xmlEscape(string) → string (XML)	489
50.23.2	xmlUnescape(string) → string (XML)	489
50.23.3	xnor(a, b) → boolean (Type)	489
50.23.4	xor(a, b) → boolean (Type)	489
50.23.5	yamlEntries(object) → array (YAML)	489
50.23.6	yamlExists(yaml, path) → boolean (YAML)	489
50.23.7	yamlFilterByKeyPattern(object, pattern) → object (YAML)	490
50.23.8	yamlFindByField(yaml, fieldName) → array (YAML)	490
50.23.9	yamlFindObjectsWithField(yaml, fieldName) → array (YAML)	490
50.23.10	yamlFromEntries(entries) → object (YAML)	490
50.23.11	yamlGetDocument(yaml, index) → value (YAML)	490
50.23.12	yamlHasRequiredFields(object, fields) → boolean (YAML)	490
50.23.13	yamlKeys(object) → array (YAML)	491
50.23.14	yamlMerge(obj1, obj2) → object (YAML)	491
50.23.15	yamlMergeAll(objects) → object (YAML)	491
50.23.16	yamlOmitKeys(object, keys) → object (YAML)	491
50.23.17	yamlSelectKeys(object, keys) → object (YAML)	491
50.23.18	yamlSort(object, comparator?) → object (YAML)	491
50.23.19	yamlSplitDocuments(yaml) → array (YAML)	492
50.23.20	yamlMergeDocuments(docs) → string (YAML)	492
50.23.21	yamlPath(yaml, path) → value (YAML)	492
50.23.22	yamlSet(yaml, path, value) → value (YAML)	492
50.23.23	yamlDelete(yaml, path) → value (YAML)	492

50.23.24	<code>yamlValidate(yaml, rules)</code> → object (YAML)	492
50.23.25	<code>yamlValidateKeyPattern(object, pattern)</code> → boolean (YAML)	493
50.23.26	<code>yamlValues(object)</code> → array (YAML)	493
50.23.27	<code>zip(arr1, arr2)</code> → array (Arr)	493
50.23.28	<code>zipWith(arr1, arr2, fn)</code> → array (Arr)	493
50.23.29	<code>zipWithIndex(array)</code> → array (Arr)	493
50.23.30	<code>zipAll(arrays, pad?)</code> → array (Bin)	494
50.23.31	<code>zipArchive(entries)</code> → binary (Bin)	494
50.24	Functions Not Listed Individually	494

UTL-X: One Language, All Formats

Integration mapping hasn't meaningfully advanced in a decade. XSLT works — for XML. The moment your source or target is JSON, CSV, or YAML, you are on your own. UTL-X eliminates that tax. One transformation language, all formats, open source.

This book covers:

- The Universal Data Model (UDM) — how UTL-X makes all formats interchangeable
- Writing transformations across XML, JSON, CSV, YAML, and OData
- Schema-to-schema conversion via USDL (XSD, JSON Schema, Avro, Protobuf)
- The three executables: utlx (CLI), utlxd (IDE), utlxe (production engine)
- 652 standard library functions with examples
- Production deployment on Azure with Kafka, Prometheus, and hot-reload
- Data restructuring: groupBy, nestBy, lookupBy, chunkBy, unnest
- Case studies: European e-invoicing (Peppol/UBL) and healthcare (HL7 FHIR)

Written for integration developers, data engineers, and architects who are tired of maintaining separate transformation tools for every format. Whether you are migrating from XSLT, evaluating alternatives to MuleSoft or Tibco, or building multi-format ETL pipelines — this book shows you how.

GLOMIDCO B.V.

The Netherlands

Web: www.glomidco.com

GLOMIDCO B.V. is a technology company specialising in enterprise integration, data transformation, and middleware solutions.

